

Django 3 por ejemplo

Tercera edición

Cree una web de Python potente y confiable
aplicaciones desde cero

Antonio Melé

BIRMINGHAM - MUMBAI

Django 3 por ejemplo

Tercera edición

Derechos de autor © 2020 Packt Publishing

Todos los derechos reservados. Ninguna parte de este libro puede reproducirse, almacenarse en un sistema, o transmitido en cualquier forma o por cualquier medio, sin el previo escrito permiso del editor, excepto en el caso de citas breves incrustadas en artículos críticos o reseñas.

Se han hecho todos los esfuerzos posibles en la preparación de este libro para garantizar la precisión de la información presentada. Sin embargo, la información contenida en este libro es vendido sin garantía, ya sea expresa o implícita. Ni el autor ni Packt Publishing o sus distribuidores y distribuidores, serán responsables de cualquier daño causado o presuntamente causado directa o indirectamente por este libro.

Packt Publishing se ha esforzado por proporcionar información de marcas comerciales sobre todos los empresas y productos mencionados en este libro mediante el uso apropiado de capitales. Sin embargo, Packt Publishing no puede garantizar la exactitud de esta información.

Redactor encargado: Pavan Ramchandani

Productor: Ben Renow-Clarke

Editor de adquisiciones - Revisiones por pares: Suresh Jain

Editor de desarrollo de contenido: Joanne Lovell

Editor técnico: Saby D'silva

Editor de proyecto: Kishor Rit

Corrector de pruebas: Safis Editing

Indexador: Pratik Shirodkar

Diseñador de presentaciones: Sandip Tadge

Primera publicación: noviembre de 2015

Segunda edición: mayo de 2018

Tercera edición: marzo de 2020

Referencia de producción: 1310320

Publicado por Packt Publishing Ltd.

Lugar de librea

Calle Librea 35

Birmingham B3 2PB, Reino Unido.

ISBN 978-1-83898-195-2

www.packt.com

*Este libro está dedicado a mis padres, Antonio y Lola,
que siempre me han apoyado en todos mis emprendimientos.*

packt.com

Suscríbase a nuestra biblioteca digital en línea para tener acceso completo a más de 7,000 libros y videos, así como herramientas líderes en la industria para ayudarlo a planificar su desarrollo personal y avanzar en su carrera. Para obtener más información, por favor visite nuestro sitio web.

¿Por qué suscribirse?

- Dedique menos tiempo a aprender y más a codificar con prácticos libros electrónicos y videos de más de 4000 profesionales de la industria
- Aprenda mejor con los planes de habilidades creados especialmente para usted
- Obtenga un libro electrónico o video gratis todos los meses
- Totalmente buscable para un fácil acceso a información vital
- Copie y pegue, imprima y marque contenido

¿Sabía que Packt ofrece versiones de libros electrónicos de todos los libros publicados, con PDF? y archivos ePub disponibles? Puede actualizar a la versión eBook en www.Packt.com y, como cliente de un libro impreso, tiene derecho a un descuento en la copia del libro electrónico. Obtener en contacto con nosotros en customercare@packtpub.com para obtener más detalles.

A www.Packt.com , también puede leer una colección de artículos técnicos gratuitos, regístrese para obtener una variedad de boletines informativos gratuitos y recibir descuentos y ofertas exclusivos en Packt libros y libros electrónicos.

Colaboradores

Sobre el Autor

Antonio Melé es el director de tecnología (CTO) de Nucoro, una empresa con sede en Londres empresa fintech que proporciona una plataforma tecnológica líder para generar riqueza digital soluciones de gestión. Antonio ha estado desarrollando proyectos de Django desde 2006 para clientes de varias industrias. En 2009, fundó Zenx IT, una empresa de desarrollo empresa especializada en la construcción de productos digitales. Ha trabajado como CTO y consultor tecnológico para múltiples start-ups de base tecnológica y ha gestionado equipos de desarrollo construyendo proyectos para grandes negocios digitales. Antonio tiene un M.Sc. en Ciencias de la Computación de la Universidad Pontificia Comillas. Su padre inspiró su pasión por las computadoras y la programación. Puedes saber más sobre Antonio en su sitio web <https://antoniomele.es/>.

Sobre los revisores

Jake Kronika, un ingeniero de software senior con casi 25 años de experiencia, ha estado trabajando con Python desde 2005 y Django desde 2007. Evolucionando junto con el espacio de desarrollo web, su conjunto de habilidades abarca HTML5, CSS3 y JavaScript (ECMAScript 6) en la interfaz, además de Python, Django, Node.js, PHP, Ruby on Rails, y mucho más además del lado del servidor.

Actualmente, arquitecto de software y líder del equipo de desarrollo, Jake colabora con diseñadores calificados, partes interesadas comerciales y desarrolladores de todo el mundo para planificar e implementar aplicaciones web sólidas. En su tiempo libre, también proporciona servicios web de espectro como propietario único de Gridline Design and Development, trabaja en proyectos de mejoras para el hogar y pasa un tiempo valioso con su esposa y dos niños.

Jake es coautor *del libro de cocina de desarrollo web Django 2, tercera edición*, publicado en Octubre de 2018 y el *Libro de recetas de desarrollo web de Django 3, cuarta edición*, publicado en Marzo de 2020. Además, ha actuado como revisor técnico para varios otros Packt títulos, que incluyen:

- *Desarrollo web con el libro de cocina de Django - Segunda edición* (2016)
- *Desarrollo de aplicaciones web receptivas con AJAX y jQuery* (2014)
- *Biblioteca de interfaz de usuario de herramientas jQuery* (2012)
- *jQuery UI 1.8: la biblioteca de interfaz de usuario para jQuery* (2011)
- *Integración de Django JavaScript: AJAX y jQuery* (2011)

"Me gustaría agradecer a mi esposa, Verónica, por todo lo que hace para apoyarme. Sin ella, sería una fracción de la persona que soy y habría logrado solo un pequeño porcentaje de lo que he podido.

Además, me gustaría agradecer a mi gerente, Ravi, por su apoyo y asesoramiento. Su guía ha sido fundamental en varios puntos de avance en mi carrera ".

David Stanek ha desarrollado software profesionalmente durante más de 22 años. Actualmente le gusta construir sistemas distribuidos con Kubernetes y la nube. tecnologías. Python ha sido su idioma preferido durante más de 19 años, pero también disfruta escribiendo en Go y otros idiomas especializados. En los últimos años, ha sido involucrado en varios proyectos relacionados con la nube y disfruta de los desafíos de ejecutar aplicaciones a escala. Para él, no hay nada mejor que escribir un servicio web en Python, implementándolo en la infraestructura orquestada con Terraform y automatizando sus procesos comerciales con Ansible.

David pasa gran parte de su tiempo de trabajo libre trabajando en proyectos de código abierto y afinando sus habilidades técnicas. Le gusta enseñar, leer libros técnicos y escuchar una variedad de podcasts y audiolibros. Cuando no esta trabajando, disfruta pasar tiempo con su hermosa esposa, cuatro maravillosos hijos y una miniatura perro tejonero.

Puede encontrar más información sobre David en su sitio web (<http://dstanek.com/>).

Tabla de contenido

Prefacio	xi
Capítulo 1: Creación de una aplicación de blog	1
Instalación de Django	2
Creando un entorno Python aislado	3
Instalando Django con pip	3
Creando tu primer proyecto	4
Ejecutando el servidor de desarrollo	6
Configuración del proyecto	8
Proyectos y aplicaciones	9
Creando una aplicación	10
Diseñar el esquema de datos del blog	10
Activando la aplicación	13
Crear y aplicar migraciones	13
Creación de un sitio de administración para modelos	15
Creando un superusuario	15
El sitio de administración de Django	dieciséis
Agregar modelos al sitio de administración	17
Personalizar la forma en que se muestran los modelos	19
Trabajar con QuerySets y administradores	21
Creando objetos	21
Actualizar objetos	23
Recuperando objetos	23
Usando el método filter()	23
Usando exclude()	24
Usando order_by()	24
Eliminar objetos	24
Cuando se evalúan los QuerySets	25

[1]

Tabla de contenido

Creando administradores de modelos	25
Lista de construcción y vistas de detalle	26
Crear vistas de lista y detalle	26
Agregar patrones de URL para sus vistas	28
URL canónicas para modelos	29
Creando plantillas para sus vistas	30
Agregar paginación	34
Usar vistas basadas en clases	36
Resumen	38
Capítulo 2: Mejora de su blog con funciones avanzadas	39
Compartir publicaciones por correo electrónico	40
Creando formularios con Django	40
Manejo de formularios en vistas	41
Enviar correos electrónicos con Django	43
Representación de formularios en plantillas	45
Creando un sistema de comentarios	50
Construyendo un modelo	50
Creando formas a partir de modelos	52
Manejo de ModelForms en vistas	53
Agregar comentarios a la plantilla de detalles de la publicación	54
Añadiendo la funcionalidad de etiquetado	58
Recuperar publicaciones por similitud	64

Resumen	66
Capítulo 3: Ampliación de la aplicación de su blog	67
Crear etiquetas y filtros de plantilla personalizados	68
Etiquetas de plantilla personalizadas	68
Filtros de plantilla personalizados	73
Agregar un mapa del sitio a su sitio	76
Crear feeds para las publicaciones de tu blog	80
Agregar búsqueda de texto completo a su blog	82
Instalación de PostgreSQL	83
Búsquedas de búsqueda simples	84
Buscando en múltiples campos	84
Construyendo una vista de búsqueda	85
Derivar y clasificar los resultados	88
Consultas de ponderación	89
Búsqueda con similitud de trigramas	90
Otros motores de búsqueda de texto completo	91
Resumen	91

[ii]

Tabla de contenido

Capítulo 4: Creación de un sitio web social	93
Creación de un proyecto de sitio web social	94
Comenzando su proyecto de sitio web social	94
Usando el marco de autenticación de Django	95
Crear una vista de inicio de sesión	96
Usar vistas de autenticación de Django	101
Vistas de inicio y cierre de sesión	102
Cambiar las vistas de contraseña	108
Restablecer vistas de contraseña	110
Registro de usuarios y perfiles de usuarios	115
Registro de usuario	115
Ampliando el modelo de usuario	119
Usando un modelo de usuario personalizado	125
Usando el marco de mensajes	125
Construyendo un backend de autenticación personalizado	128
Agregar autenticación social a su sitio	130
Ejecutando el servidor de desarrollo a través de HTTPS	132
Autenticación mediante Facebook	134
Autenticación mediante Twitter	140
Autenticación mediante Google	142
Resumen	147
Capítulo 5: Compartir contenido en su sitio web	149
Creación de un sitio web de marcadores de imágenes	150
Construyendo el modelo de imagen	150
Crear relaciones de varios a varios	152
Registro del modelo de imagen en el sitio de administración	153
Publicar contenido de otros sitios web	153
Campos de formulario de limpieza	154
Anulando el método save () de un ModelForm	155
Construyendo un bookmarklet con jQuery	160
Crear una vista de detalle para imágenes	168
Creación de miniaturas de imágenes usando miniaturas fáciles	170
Agregar acciones AJAX con jQuery	172
Cargando jQuery	173
Falsificación de solicitudes entre sitios en solicitudes AJAX	174
Realización de solicitudes AJAX con jQuery	176

Creando decoradores personalizados para tus vistas	179
Agregar paginación AJAX a sus vistas de lista	181
Resumen	186

Tabla de contenido

Capítulo 6: Seguimiento de las acciones del usuario	187
Construyendo un sistema de seguimiento	187
Crear relaciones de muchos a muchos con un modelo intermedio	188
Creación de vistas de lista y detalle para perfiles de usuario	191
Construyendo una vista AJAX para seguir a los usuarios	196
Creación de una aplicación de flujo de actividades genérico	198
Usando el marco de contenttypes	200
Agregar relaciones genéricas a sus modelos	201
Evitar acciones duplicadas en el flujo de actividades	204
Agregar acciones de usuario al flujo de actividades	205
Visualización del flujo de actividades	206
Optimización de QuerySets que involucran objetos relacionados	207
Usando select_related()	207
Usando prefetch_related()	208
Creando plantillas para acciones	208
Usando señales para desnormalizar recuentos	210
Trabajando con señales	211
Clases de configuración de aplicaciones	213
Usar Redis para almacenar vistas de elementos	215
Instalación de Redis	215
Usando Redis con Python	217
Almacenamiento de vistas de elementos en Redis	218
Almacenamiento de una clasificación en Redis	220
Próximos pasos con Redis	223
Resumen	223
Capítulo 7: Creación de una tienda online	225
Creación de un proyecto de tienda online	226
Creación de modelos de catálogo de productos	227
Registro de modelos de catálogo en el sitio de administración	229
Creación de vistas de catálogo	230
Crear plantillas de catálogo	233
Construyendo un carrito de compras	237
Usando sesiones de Django	238
Configuración de la sesión	239
Caducidad de la sesión	240
Almacenar carritos de compras en sesiones	240
Crear vistas de carrito de compras	245
Agregar artículos al carrito	245
Construyendo una plantilla para mostrar el carrito	247
Agregar productos al carrito	249
Actualización de cantidades de productos en el carrito	251

Tabla de contenido

Crear un procesador de contexto para el carrito actual	252
Procesadores de contexto	252
Configurar el carrito en el contexto de la solicitud	253
Registro de pedidos de clientes	255
Creando modelos de pedidos	255
Incluir modelos de pedidos en el sitio de administración	257
Creación de pedidos de clientes	258
Lanzamiento de tareas asincrónicas con Celery	263
Instalación de apio	263
Instalación de RabbitMQ	264
Agregando apio a su proyecto	264
Agregar tareas asincrónicas a su aplicación	265
Monitoreo de apio	267
Resumen	268
Capítulo 8: Gestión de pagos y pedidos	269
Integrando una pasarela de pago	269
Crear una cuenta de caja de arena de Braintree	270
Instalación del módulo Braintree Python	271
Integrando la pasarela de pago	272
Integración de Braintree mediante campos alojados	274
Prueba de pagos	280
Va en directo	283
Exportación de pedidos a archivos CSV	284
Agregar acciones personalizadas al sitio de administración	284
Ampliación del sitio de administración con vistas personalizadas	287
Generar facturas en PDF de forma dinámica	292
Instalación de WeasyPrint	292
Creando una plantilla PDF	292
Representación de archivos PDF	294
Envío de archivos PDF por correo electrónico	297
Resumen	300
Capítulo 9: Ampliación de su tienda	301
Crear un sistema de cupones	301
Construyendo el modelo de cupón	302
Aplicar un cupón al carrito de compras	304
Aplicar cupones a pedidos	312
Añadiendo internacionalización y localización	314
Internacionalización con Django	315
Configuración de internacionalización y localización	315
Comandos de gestión de internacionalización	316
Cómo agregar traducciones a un proyecto de Django	316

Tabla de contenido

Cómo determina Django el idioma actual	316
Preparando su proyecto para la internacionalización	317
Traducir código Python	318
Traducciones estándar	319
Traducciones perezosas	319

Traducciones que incluyen variables	319
Formas plurales en traducciones	319
Traduciendo tu propio código	320
Traducción de plantillas	324
La etiqueta de plantilla {% trans %}	324
La etiqueta de plantilla {% blocktrans %}	324
Traducir las plantillas de la tienda	325
Usando la interfaz de traducción de Rosetta	328
Traducciones difusas	331
Patrones de URL para la internacionalización	332
Agregar un prefijo de idioma a los patrones de URL	332
Traducción de patrones de URL	333
Permitir a los usuarios cambiar de idioma	334
Traducir modelos con django-parler	336
Instalación de django-parler	336
Traducir campos del modelo	337
Integración de traducciones en el sitio de administración	339
Crear migraciones para traducciones de modelos	340
Adaptación de vistas para traducciones	341
Localización de formato	344
Usando django-localflavor para validar campos de formulario	345
Construyendo un motor de recomendaciones	347
Recomendar productos basados en compras anteriores	347
Resumen	356
Capítulo 10: Creación de una plataforma de aprendizaje electrónico	357
Configuración del proyecto de e-learning	358
Construyendo los modelos del curso	359
Registro de modelos en el sitio de administración	361
Usar accesorios para proporcionar datos iniciales para modelos	362
Creando modelos para contenido diverso	365
Usando la herencia del modelo	366
Modelos abstractos	366
Herencia del modelo de múltiples tablas	367
Modelos proxy	367
Creando los modelos de contenido	368
Crear campos de modelo personalizados	370
Agregar orden a los objetos de contenido y módulo	372
Creando un CMS	377
Agregar un sistema de autenticación	377

[vi]

Tabla de contenido

Creando las plantillas de autenticación	378
Crear vistas basadas en clases	381
Usando mixins para vistas basadas en clases	381
Trabajar con grupos y permisos	383
Restringir el acceso a las vistas basadas en clases	385
Gestión de los módulos del curso y sus contenidos	391
Usar conjuntos de formularios para los módulos del curso	391
Agregar contenido a los módulos del curso	396
Gestión de módulos y sus contenidos	402
Reordenación de módulos y su contenido	407
Usando mixins de django-braces	407
Resumen	411
Capítulo 11: Procesamiento y almacenamiento en caché de contenido	413
Visualización de cursos	414
Agregar registro de estudiantes	419
Crear una vista de registro de estudiantes	419
Inscribirse en cursos	422
Acceder a los contenidos del curso	425
Representar diferentes tipos de contenido	429

<u>Usando el marco de caché</u>	<u>432</u>
<u>Backends de caché disponibles</u>	<u>432</u>
<u>Instalación de Memcached</u>	<u>433</u>
<u>Configuración de caché</u>	<u>434</u>
<u>Agregar Memcached a su proyecto</u>	<u>434</u>
<u>Supervisión de Memcached</u>	<u>435</u>
<u>Niveles de caché</u>	<u>436</u>
<u>Usando la API de caché de bajo nivel</u>	<u>436</u>
<u>Almacenamiento en caché basado en datos dinámicos</u>	<u>438</u>
<u>Almacenamiento en caché de fragmentos de plantilla</u>	<u>440</u>
<u>Almacenamiento en caché de vistas</u>	<u>441</u>
<u>Usar la caché por sitio</u>	<u>441</u>
<u>Resumen</u>	<u>442</u>
Capítulo 12: <u>Creación de una API</u>	<u>443</u>
<u>Construyendo una API RESTful</u>	<u>444</u>
<u>Instalación del marco REST de Django</u>	<u>444</u>
<u>Definición de serializadores</u>	<u>445</u>
<u>Comprensión de analizadores y representadores</u>	<u>446</u>
<u>Lista de construcción y vistas de detalle</u>	<u>447</u>
<u>Crear serializadores anidados</u>	<u>450</u>
<u>Crear vistas de API personalizadas</u>	<u>452</u>
<u>Manejo de la autenticación</u>	<u>453</u>

[vii]

Tabla de contenido

<u>Agregar permisos a las vistas</u>	<u>454</u>
<u>Crear conjuntos de vistas y enrutadores</u>	<u>456</u>
<u>Agregar acciones adicionales a conjuntos de vistas</u>	<u>457</u>
<u>Creando permisos personalizados</u>	<u>458</u>
<u>Serializar los contenidos del curso</u>	<u>459</u>
<u>Consumir la API REST</u>	<u>461</u>
<u>Resumen</u>	<u>465</u>
Capítulo 13: <u>Creación de un servidor de chat</u>	<u>467</u>
<u>Creando una aplicación de chat</u>	<u>467</u>
<u>Implementar la vista de la sala de chat</u>	<u>468</u>
<u>Desactivación de la caché por sitio</u>	<u>471</u>
<u>Django en tiempo real con canales</u>	<u>471</u>
<u>Aplicaciones asincrónicas que utilizan ASGI</u>	<u>471</u>
<u>El ciclo de solicitud / respuesta usando canales</u>	<u>472</u>
<u>Instalación de canales</u>	<u>473</u>
<u>Escribiendo un consumidor</u>	<u>476</u>
<u>Enrutamiento</u>	<u>477</u>
<u>Implementar el cliente WebSocket</u>	<u>478</u>
<u>Habilitar una capa de canal</u>	<u>484</u>
<u>Canales y grupos</u>	<u>484</u>
<u>Configurar una capa de canal con Redis</u>	<u>484</u>
<u>Actualización del consumidor para transmitir mensajes</u>	<u>486</u>
<u>Agregar contexto a los mensajes</u>	<u>490</u>
<u>Modificar el consumidor para que sea completamente asincrónico</u>	<u>494</u>
<u>Integrando el chat con vistas existentes</u>	<u>495</u>
<u>Resumen</u>	<u>496</u>
Capítulo 14: <u>Lanzamiento</u>	<u>497</u>
<u>Creando un ambiente de producción</u>	<u>497</u>
<u>Administrar configuraciones para múltiples entornos</u>	<u>498</u>
<u>Usando PostgreSQL</u>	<u>500</u>
<u>Comprobando su proyecto</u>	<u>501</u>
<u>Sirviendo a Django a través de WSGI</u>	<u>501</u>
<u>Instalación de uWSGI</u>	<u>502</u>

Configuración de uWSGI	502
Instalación de NGINX	505
El entorno de producción	506
Configurando NGINX	506
Sirviendo activos estáticos y multimedia	509
Asegurar conexiones con SSL / TLS	511
Crear un certificado SSL / TLS	511

[viii]

Tabla de contenido

Configuración de NGINX para usar SSL / TLS	512
Configurando su proyecto Django para SSL / TLS	514
Redirigir el tráfico HTTP a HTTPS	515
Usar Daphne para canales Django	516
Usar conexiones seguras para WebSockets	517
Incluyendo Daphne en la configuración NGINX	518
Crear un middleware personalizado	520
Crear un middleware de subdominio	522
Sirviendo múltiples subdominios con NGINX	523
Implementar comandos de administración personalizados	524
Resumen	527
Otros libros que puede disfrutar	529
Índice	533

Prefacio

Django es un potente marco web de Python que fomenta el desarrollo rápido y diseño limpio y pragmático, al tiempo que ofrece una curva de aprendizaje relativamente superficial. Esto lo hace atractivo tanto para programadores novatos como expertos.

Este libro lo guiará a lo largo de todo el proceso de desarrollo web profesional. aplicaciones con Django. El libro no solo cubre los aspectos más relevantes de la framework, pero también le enseñará cómo integrar otras tecnologías populares en sus proyectos de Django.

El libro lo guiará a través de la creación de aplicaciones del mundo real, resolviendo problemas comunes e implementación de las mejores prácticas, utilizando un enfoque paso a paso eso es fácil de seguir.

Después de leer este libro, comprenderá bien cómo funciona Django. y cómo crear aplicaciones web prácticas y avanzadas.

para quien es este libro

Este libro está dirigido a desarrolladores con conocimientos de Python que deseen aprender Django de forma pragmática. Quizás eres completamente nuevo en Django, o ya sabes un poco pero quieres sacarle el máximo partido. Este libro te ayudará Dominar las áreas más relevantes del marco mediante la construcción de proyectos prácticos. desde cero. Necesita estar familiarizado con los conceptos de programación para para leer este libro. Se asume algún conocimiento previo de HTML y JavaScript.

[xi]

Que cubre este libro

El Capítulo 1 , Creación de una aplicación de blog , le presentará el marco a través de una aplicación de blog. Creará los modelos básicos de blogs, vistas, plantillas y URL para mostrar publicaciones de blog. Aprenderá a construir QuerySets con el **objeto Django-mapeador relacional (ORM)** , y configurará el sitio de administración de Django.

El Capítulo 2 , Mejora de su blog con funciones avanzadas , le enseñará cómo manejar formularios y ModelForms, envíe correos electrónicos con Django e integre aplicaciones. Implementará un sistema de comentarios para las publicaciones de tu blog y permitirás sus usuarios para compartir publicaciones por correo electrónico. El capítulo también lo guiará a través de proceso de creación de un sistema de etiquetado.

El Capítulo 3 , Ampliación de la aplicación de blog , explora cómo crear una plantilla personalizada etiquetas y filtros. El capítulo también le mostrará cómo usar el marco del mapa del sitio. y cree una fuente RSS para sus publicaciones. Completará la solicitud de tu blog al construir un motor de búsqueda con las capacidades de búsqueda de texto completo de PostgreSQL.

El Capítulo 4 , Creación de un sitio web social , explica cómo crear un sitio web social. Vas a use el marco de autenticación de Django para crear vistas de cuentas de usuario. Vas a Aprenda también a crear un modelo de perfil de usuario personalizado y a crear autenticación social. en su proyecto utilizando las principales redes sociales.

El Capítulo 5 , Compartir contenido en su sitio web , le enseñará cómo transformar su aplicación social en un sitio web de marcadores de imágenes. Definirás muchos a-muchos para los modelos, y creará un bookmarklet AJAX en JavaScript e intégralo en tu proyecto. El capítulo le mostrará cómo Genere miniaturas de imágenes y cree decoradores personalizados para sus vistas.

El capítulo 6 , Seguimiento de las acciones del usuario , le mostrará cómo crear un sistema de seguidores para usuarios. Completará su sitio web de marcadores de imágenes creando una actividad de usuario aplicación de flujo. Aprenderá a optimizar QuerySets y trabajará con señales. Finalmente, integrará Redis en su proyecto para contar las vistas de imágenes.

El Capítulo 7 , Creación de una tienda en línea , explora cómo crear una tienda en línea. Vas a construya modelos de catálogo, y creará un carrito de compras usando sesiones de Django. Construirá un procesador de contexto para el carrito de compras y aprenderá cómo para implementar el envío de notificaciones asincrónicas a los usuarios que utilizan Celery.

El Capítulo 8 , Gestión de pagos y pedidos , explica cómo integrar un pago puerta de entrada a su tienda. También personalizará el sitio de administración para exporta pedidos a archivos CSV y generará facturas en PDF de forma dinámica.

[xii]

Prefacio

El Capítulo 9 , Ampliación de su tienda , le enseñará cómo crear un sistema de cupones para solicitar Descuentos a pedidos. El capítulo también le mostrará cómo agregar internacionalización a su proyecto y cómo traducir modelos. Finalmente, construirás un producto motor de recomendaciones usando Redis.

El Capítulo 10 , Creación de una plataforma de aprendizaje electrónico , lo guiará en la creación de una plataforma de e-learning. Agregará accesorios a su proyecto, usará la herencia del modelo, crear campos de modelo personalizados, usar vistas basadas en clases y administrar grupos y permisos. También creará un sistema de gestión de contenido y manejará conjuntos de formularios.

El Capítulo 11 , Procesamiento y almacenamiento en caché de contenido , le mostrará cómo crear un estudiante sistema de registro y gestionar la inscripción de estudiantes en los cursos. Te rendirás diversos contenidos del curso y aprender a usar el marco de caché.

El Capítulo 12 , Creación de una API , explora la creación de una API RESTful para su proyecto utilizando Marco Django REST.

El Capítulo 13 , Creación de un servidor de chat , explica cómo usar los canales Django para crear

un servidor de chat en tiempo real para estudiantes. Aprenderás a implementar funcionalidades que dependen de la comunicación asincrónica a través de WebSockets.

Capítulo 14, *Que se activarán*, le mostrará cómo configurar un entorno de producción utilizando uWSGI, NGINX y Daphne. Aprenderá a proteger el medio ambiente a través de HTTPS. El capítulo también explica cómo construir un middleware personalizado y crear comandos de gestión personalizados.

Aproveche al máximo este libro

El lector debe:

- Poseer un buen conocimiento práctico de Python
- Siéntete cómodo con HTML y JavaScript
- He repasado las partes 1 a 3 del tutorial en el Django oficial.
documentación en <https://docs.djangoproject.com/en/3.0/intro/tutorial01/>

Descarga los archivos de código de ejemplo

Puede descargar los archivos de código de ejemplo para este libro desde su cuenta en www.packt.com/. Si compró este libro en otro lugar, puede visitar www.packtpub.com/support y regístrese para recibir los archivos directamente por correo electrónico.

[xiii]

Prefacio

Puede descargar los archivos de código siguiendo estos pasos:

1. Inicie sesión o regístrese en <http://www.packt.com>
2. Seleccione la pestaña **Soporte**
3. Haga clic en **Descargas de código**
4. Ingrese el nombre del libro en el cuadro de **búsqueda** y siga las instrucciones en pantalla.
instrucciones

Una vez descargado el archivo, asegúrese de descomprimir o extraer la carpeta utilizando la última versión de:

- WinRAR / 7-Zip para Windows
- Zipeg / iZip / UnRarX para Mac
- 7-Zip / PeaZip para Linux

El paquete de código del libro también está alojado en GitHub en [https://github.com/PacktPublishing / Django-3-by-Example](https://github.com/PacktPublishing/Django-3-by-Example). En caso de que haya una actualización del código, se actualizará en el repositorio de GitHub existente.

También tenemos otros paquetes de códigos de nuestro amplio catálogo de libros y videos disponibles. en <https://github.com/PacktPublishing/>. ¡Échales un vistazo!

Descarga las imágenes en color

También proporcionamos un archivo PDF que tiene imágenes en color de las capturas de pantalla / diagramas utilizado en este libro. Puede descargarlo aquí: https://static.packt-cdn.com/downloads/9781838981952_ColorImages.pdf.

Convenciones utilizadas

Hay una serie de convenciones de texto que se utilizan a lo largo de este libro.

CodeInText : indica palabras de código en el texto, nombres de tablas de bases de datos, nombres de carpetas, nombres de archivo, extensiones de archivo, nombres de ruta, URL ficticias, entrada del usuario y Twitter manejas. Por ejemplo: "Edite el archivo models.py de la aplicación de la tienda ".

Un bloque de código se establece de la siguiente manera:

```
desde django.contrib import admin
desde .models import Post

admin.site.register (Publicar)
```

[xiv]

Prefacio

Cuando deseamos llamar su atención sobre una parte particular de un bloque de código, las líneas o elementos relevantes se establecen en negrita:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'blog.apps.BlogConfig',
]
```

Cualquier entrada o salida de la línea de comandos se escribe de la siguiente manera:

```
python manage.py runserver
```

Negrita : indica un término nuevo, una palabra importante o palabras que ve en la pantalla. Por ejemplo: "Complete el formulario y haga clic en el botón Guardar".

Las advertencias o notas importantes aparecen así.

Los consejos y trucos aparecen así.

Ponerse en contacto

Los comentarios de nuestros lectores es siempre bienvenido.

Comentarios generales : si tiene preguntas sobre cualquier aspecto de este libro, mencione el título del libro en el asunto de su mensaje y envíenos un correo electrónico a [customercare @ packtpub.com](mailto:customercare@packtpub.com) .

Prefacio

Fe de erratas : aunque hemos tomado todas las precauciones para garantizar la precisión de nuestro contenido, los errores ocurren. Si ha encontrado un error en este libro, le agradeceríamos si pudiera informarnos esto. Visite www.packtpub.com/support/errata, seleccione su libro, haga clic en el enlace Formulario de envío de erratas e ingrese los detalles.

Piratería : si encuentra copias ilegales de nuestros trabajos en cualquier forma en el Internet, le agradeceríamos que nos facilitara la dirección de ubicación o Nombre del Sitio Web. Por favor contáctenos en copyright@packt.com con un enlace al material.

Si está interesado en convertirse en autor : Si hay un tema que tiene experiencia y está interesado en escribir o contribuir a un libro, por favor visita autores.packtpub.com .

Reseñas

Por favor deje una reseña. Una vez que haya leído y utilizado este libro, ¿por qué no dejar un revisar en el sitio donde lo compró? Los lectores potenciales pueden ver y use su opinión imparcial para tomar una decisión de compra, nosotros en Packt podemos entender lo que piensa sobre nuestro producto, y nuestro autor puede ver sus comentarios sobre su libro. ¡Gracias!

Para obtener más información sobre Packt, visite packt.com .

1

Creación de una aplicación de blog

Django es un potente framework web Python con un aprendizaje relativamente superficial curva. Puede crear fácilmente aplicaciones web sencillas en poco tiempo. Django es también un marco robusto y escalable que se puede utilizar para crear sitios web a gran escala aplicaciones con requisitos e integraciones complejos. Esto hace que Django atractivo tanto para principiantes como para programadores expertos.

En este libro, aprenderá a crear proyectos completos de Django que estén listos para uso en producción. Si aún no ha instalado Django, descubrirá cómo hacerlo en la primera parte de este capítulo.

Este capítulo cubre cómo crear una aplicación de blog simple usando Django. El propósito del capítulo es ayudarlo a tener una idea general de cómo funciona el marco, una comprensión de cómo los diferentes componentes interactúan entre sí, y las habilidades para crear fácilmente proyectos Django con funcionalidad básica. Tú serás guiado a través de la creación de un proyecto completo, pero entraré en más detalles sobre esto más tarde. Los diferentes componentes del marco se explorarán en detalle a lo largo de este libro.

Este capítulo cubrirá los siguientes temas:

- Instalando Django
- Crear y configurar un proyecto Django
- Creando una aplicación Django
- Diseño de modelos y generación de migraciones de modelos.
- Creación de un sitio de administración para sus modelos.
- Trabajar con QuerySets y administradores

[1]

Creación de una aplicación de blog

- Creación de vistas, plantillas y URL
- Agregar paginación a las vistas de lista
- Usando las vistas basadas en clases de Django

Instalación de Django

Si ya ha instalado Django, puede omitir esta sección y saltar directamente a la sección *Creación de su primer proyecto*. Django viene como un paquete de Python y por lo tanto se puede instalar en cualquier entorno Python. Si aún no ha instalado Django, la siguiente es una guía rápida para instalarlo para el desarrollo local.

Django 3 continúa el camino de proporcionar nuevas funciones mientras mantiene el funcionalidades básicas del marco. La versión 3.0 incluye por primera vez

Compatibilidad con la **interfaz de puerta de enlace de servidor asincrónica (ASGI)**, lo que hace que Django totalmente asincrónico. Django 3.0 también incluye soporte oficial para MariaDB, nuevo restricciones de exclusión en PostgreSQL, mejoras en las expresiones de filtro y enumeraciones para las opciones de campo del modelo, así como otras características nuevas.

Django 3.0 es compatible con Python 3.6, 3.7 y 3.8. En los ejemplos de este libro, usaremos Python 3.8.2. Si está usando Linux o macOS, probablemente tenga Python instalado. Si está usando Windows, puede descargar un instalador de Python en <https://www.python.org/downloads/windows/>.

Si no está seguro de si Python está instalado en su computadora, puede verificar esto escribiendo python en el shell. Si ve algo como lo siguiente, entonces Python está instalado en su computadora:

```
Python 3.8.2 (v3.8.2: 7b3ab5921f, 24 de febrero de 2020, 17:52:18)
```

```
[Clang 6.0 (clang-600.0.57)] en darwin
```

Escriba "ayuda", "derechos de autor", "créditos" o "licencia" para obtener más información.

Si su versión de Python instalada es inferior a 3.6, o si Python no está instalado en su computadora, descargue Python 3.8.2 desde <https://www.python.org/descargas/> e instalarlo.

Dado que utilizará Python 3, no es necesario que instale una base de datos. Esta pitón La versión viene con una base de datos SQLite incorporada. SQLite es una base de datos ligera que puede usar con Django para el desarrollo. Si planea implementar su aplicación en un entorno de producción, debe utilizar una base de datos con todas las funciones, como PostgreSQL, MySQL u Oracle. Puede encontrar más información sobre cómo ejecute su base de datos con Django en <https://docs.djangoproject.com/es/3.0/topics/install/#instalación-base-de-datos>.

[2]

Creando un entorno Python aislado

Desde la versión 3.3, Python viene con la biblioteca venv , que brinda soporte para crear entornos virtuales ligeros. Cada entorno virtual tiene su propio Python binary y puede tener su propio conjunto independiente de paquetes Python instalados en los directorios de su sitio. Usando el módulo venv de Python para crear Python aislado entornos le permite utilizar diferentes versiones de paquetes para diferentes proyectos, que es mucho más práctico que instalar paquetes de Python en todo el sistema. Otro La ventaja de usar venv es que no necesitará ningún privilegio de administración para instalar paquetes de Python.

Cree un entorno aislado con el siguiente comando:

```
python -m venv my_env
```

Esto creará un directorio my_env / , incluido su entorno Python. Alguna Las bibliotecas de Python que instale mientras su entorno virtual está activo entrarán en el directorio my_env / lib / python3.8 / site-packages .

Ejecute el siguiente comando para activar su entorno virtual:

```
fuente my_env / bin / enable
```

El indicador de shell incluirá el nombre del entorno virtual activo adjunto. entre paréntesis, como sigue:

```
(my_env) computadora portátil: ~ zenx $
```

Puede desactivar su entorno en cualquier momento con el comando deactivate . Puede encontrar más información sobre venv en <https://docs.python.org/3/biblioteca/venv.html>.

Instalando Django con pip

El sistema de gestión de paquetes pip es el método preferido para instalar Django. Python 3.8 viene con pip preinstalado, pero puede encontrar la instalación de pip instrucciones en <https://pip.pypa.io/en/stable/installing/>.

Ejecute el siguiente comando en el indicador de shell para instalar Django con pip :

```
pip install "Django == 3.0. *"
```

Django se instalará en el directorio Python site-packages / de su virtual ambiente.

[3]

Creación de una aplicación de blog

Ahora compruebe si Django se ha instalado correctamente. Ejecute Python en un terminal, importe Django y verifique su versión, de la siguiente manera:

```
>>> importar django
>>> django.get_version()
"3.0.4"
```

Si obtiene una salida como 3.0.X , Django se ha instalado correctamente en su máquina.

Django se puede instalar de varias otras formas. Puedes encontrar un guía de instalación completa [en https://docs.djangoproject.com/en/3.0/topics/install/](https://docs.djangoproject.com/en/3.0/topics/install/).

Creando tu primer proyecto

Nuestro primer proyecto de Django será la construcción de un blog completo. Django proporciona un comando que le permite crear una estructura de archivo de proyecto inicial. Ejecutar el siguiente comando de su shell:

```
django-admin startproject mysite
```

Esto creará un proyecto de Django con el nombre mysite .

Evite nombrar proyectos después de módulos integrados de Python o Django en para evitar conflictos.

Echemos un vistazo a la estructura del proyecto generada:

```
mi sitio/
  manage.py
  mi sitio/
    __init__.py
    asgi.py
    wsgi.py
    settings.py
    urls.py
```

Estos archivos son los siguientes:

- `manage.py` : esta es una utilidad de línea de comandos que se utiliza para interactuar con su proyecto. Es un envoltorio delgado alrededor de la herramienta `django-admin.py` . No necesitas editar Este archivo.
- `mysite /` : este es el directorio de su proyecto, que consta de los siguientes archivos:
 - `__init__.py` : un archivo vacío que le dice a Python que trate el `mysite` directorio como un módulo de Python.
 - `asgi.py` : esta es la configuración para ejecutar su proyecto como ASGI, el estándar Python emergente para servidores web asíncronos y aplicaciones.
 - `settings.py` : indica los ajustes y la configuración de su proyecto y contiene la configuración inicial predeterminada.
 - `urls.py` : este es el lugar donde viven sus patrones de URL. Cada URL definido aquí se asigna a una vista.
 - `wsgi.py` : esta es la configuración para ejecutar su proyecto como un **Web Aplicación Server Gateway Interface (WSGI)**.

El archivo `settings.py` generado contiene la configuración del proyecto, incluida una configuración para usar una base de datos SQLite3 y una lista llamada `INSTALLED_APPS` que contiene aplicaciones comunes de Django que se agregan a su proyecto de forma predeterminada. Revisaremos estas aplicaciones más adelante en la sección *Configuración del proyecto* .

Las aplicaciones Django contienen un archivo `models.py` donde se definen los modelos de datos. Cada modelo de datos se asigna a una tabla de base de datos. Para completar la configuración del proyecto, necesidad de crear las tablas asociadas con los modelos de las aplicaciones enumeradas en `INSTALLED_APPS` . Django incluye un sistema de migración que gestiona esto.

Abra el shell y ejecute los siguientes comandos:

```
cd mysite
```

```
python manage.py migrar
```

Notará una salida que termina con las siguientes líneas:

```
Aplicando contenttypes.0001_initial ... OK
Aplicando auth.0001_initial ... OK
Aplicando admin.0001_initial ... OK
Aplicando admin.0002_logentry_remove_auto_add ... OK
Aplicando admin.0003_logentry_add_action_flag_choices ... OK
Aplicando contenttypes.0002_remove_content_type_name ... OK
Aplicando auth.0002_alter_permission_name_max_length ... OK
```

Creación de una aplicación de blog

```
Aplicando auth.0003_alter_user_email_max_length ... OK
Aplicando auth.0004_alter_user_username_opts ... OK
Aplicando auth.0005_alter_user_last_login_null ... OK
Aplicando auth.0006_require_contenttypes_0002 ... OK
Aplicando auth.0007_alter_validators_add_error_messages ... OK
Aplicando auth.0008_alter_user_username_max_length ... OK
Aplicando auth.0009_alter_user_last_name_max_length ... OK
Aplicando auth.0010_alter_group_name_max_length ... OK
Aplicando auth.0011_update_proxy_permissions ... OK
Aplicando sesiones.0001_initial ... OK
```

Las líneas anteriores son las migraciones de bases de datos que aplica Django. Al aplicar migraciones, las tablas para las aplicaciones iniciales se crean en el base de datos. Aprenderá sobre el comando de gestión de migraciones en *Creación y aplicación de la sección de migraciones* de este capítulo.

Ejecutando el servidor de desarrollo

Django viene con un servidor web ligero para ejecutar su código rápidamente, sin necesidad de dedicar tiempo a configurar un servidor de producción. Cuando ejecuta el Servidor de desarrollo Django, sigue buscando cambios en su código. Se recarga automáticamente, liberándose de recargarlo manualmente después de cambios de código. Sin embargo, es posible que no note algunas acciones, como agregar nuevos archivos a su proyecto, por lo que tendrá que reiniciar el servidor manualmente en estos casos.

Inicie el servidor de desarrollo escribiendo el siguiente comando de su proyecto carpeta raíz:

```
python manage.py runserver
```

Debería ver algo como esto:

```
Observando cambios de archivos con StatReloader
Realización de comprobaciones del sistema ...
```

```
La verificación del sistema no identificó problemas (0 silenciados).
```

```
01 de enero de 2020-10: 00: 00
Django versión 3.0, usando la configuración 'mysite.settings'
Iniciando el servidor de desarrollo en http://127.0.0.1:8000/
Salga del servidor con CONTROL-C.
```

[6]

Ahora abra <http://127.0.0.1:8000/> en su navegador. Deberías ver una página indicando que el proyecto se está ejecutando correctamente, como se muestra en la siguiente captura de pantalla:

Figura 1.1: La página predeterminada del servidor de desarrollo Django

La captura de pantalla anterior indica que Django se está ejecutando. Si echas un vistazo a su consola, verá la solicitud GET realizada por su navegador:

```
[01 / Ene / 2020 17:20:30] "GET / HTTP / 1.1" 200 16351
```

El servidor de desarrollo registra cada solicitud HTTP en la consola. Cualquier error que ocurre mientras se ejecuta el servidor de desarrollo también aparecerá en la consola.

Puede ejecutar el servidor de desarrollo Django en un host y puerto personalizados o indicar Django para cargar un archivo de configuración específico, de la siguiente manera:

```
python manage.py runserver 127.0.0.1:8001 \ - settings = mysite.settings
```

Cuando tenga que lidiar con múltiples entornos que requieran diferentes configuraciones, puede crear un archivo de configuración diferente para cada ambiente.

Recuerde que este servidor solo está destinado al desarrollo y no es adecuado para uso en producción. Para implementar Django en un entorno de producción, debería ejecutarlo como una aplicación WSGI utilizando un servidor web, como Apache, Gunicorn, o uWSGI, o como una aplicación ASGI usando un servidor como Uvicorn o Daphne. Tú puede encontrar más información sobre cómo implementar Django con diferentes servidores web en <https://docs.djangoproject.com/en/3.0/howto/deployment/wsgi/>.

Capítulo 14 , *Que se activarán* , explica cómo configurar un entorno de producción para su Proyectos de Django.

[7]

Creación de una aplicación de blog

Configuración del proyecto

Abramos el archivo settings.py y echemos un vistazo a la configuración del proyecto. Hay varias configuraciones que Django incluye en este archivo, pero estas son solo una parte de todas las configuraciones de Django disponibles. Puede ver todas las configuraciones y sus valores predeterminados en <https://docs.djangoproject.com/en/3.0/ref/settings/>.

Vale la pena mirar las siguientes configuraciones:

- **DEBUG** es un booleano que activa y desactiva el modo de depuración del proyecto. Si se establece en True , Django mostrará páginas de error detalladas cuando no se detecta la aplicación lanza una excepción. Cuando te mudas a una producción entorno, recuerde que debe configurarlo en False . Nunca implemente un sitio en producción con DEBUG activado porque expondrá datos relacionados con el proyecto.
- **ALLOWED_HOSTS** no se aplica mientras el modo de depuración está activado o cuando las pruebas se ejecutan. Una vez que mueva su sitio a producción y establezca DEBUG en False , tendrá que agregar su dominio / host a esta configuración para permitirlo para servir a su sitio Django.
- **INSTALLED_APPS** es una configuración que deberá editar para todos los proyectos. Esta La configuración le dice a Django qué aplicaciones están activas para este sitio. Por defecto, Django incluye las siguientes aplicaciones:
 - `django.contrib.admin` : un sitio de administración
 - `django.contrib.auth` : un marco de autenticación

- `django.contrib.contenttypes` : un marco para el manejo de tipos de contenido
 - `django.contrib.sessions` : un marco de sesión
 - `django.contrib.messages` : un marco de mensajería
 - `django.contrib.staticfiles` : un marco para administrar estática archivos
- **MIDDLEWARE** es una lista que contiene middleware para ejecutar.
 - **ROOT_URLCONF** indica el módulo de Python donde los patrones de URL raíz de su aplicación están definidos.
 - **DATABASES** es un diccionario que contiene la configuración de todas las bases de datos ser utilizado en el proyecto. Siempre debe haber una base de datos predeterminada. El valor por defecto La configuración utiliza una base de datos SQLite3.

[8]

Capítulo 1

- **LANGUAGE_CODE** define el código de idioma predeterminado para este sitio de Django.
- **USE_TZ** le dice a Django que active / desactive el soporte de zona horaria. Viene Django con soporte para datetime con reconocimiento de zona horaria. Esta configuración se establece en Verdadero cuando crea un nuevo proyecto utilizando el comando de gestión `startproject` .

No se preocupe si no comprende mucho sobre lo que está viendo aquí. Vas a aprender las diferentes configuraciones de Django en los siguientes capítulos.

Proyectos y aplicaciones

A lo largo de este libro, encontrará los términos **proyecto** y **aplicación** sobre y más. En Django, un proyecto se considera una instalación de Django con algunas configuraciones. Una aplicación es un grupo de modelos, vistas, plantillas y URL. Aplicaciones interactuar con el marco para proporcionar algunas funcionalidades específicas y puede ser reutilizado en varios proyectos. Puede pensar en un proyecto como su sitio web, que contiene varias aplicaciones, como un blog, wiki o foro, que también se pueden utilizar por otros proyectos.

El siguiente diagrama muestra la estructura de un proyecto de Django:

Figura 1.2: Estructura del proyecto / aplicación de Django

Creación de una aplicación de blog

Creando una aplicación

Ahora creemos su primera aplicación Django. Crearás una aplicación de blog desde cero. Desde el directorio raíz del proyecto, ejecute el siguiente comando:

blog de python manage.py startapp

Esto creará la estructura básica de la aplicación, que se ve así:

```
Blog/
  __init__.py
  admin.py
  apps.py
  migraciones /
    __init__.py
  modelos.py
  tests.py
  views.py
```

Estos archivos son los siguientes:

- **admin.py** : Aquí es donde registras los modelos para incluirlos en Django. sitio de administración: el uso de este sitio es opcional.
- **apps.py** : incluye la configuración principal de la aplicación de blog .
- **migraciones** : este directorio contendrá las migraciones de la base de datos de su solicitud. Las migraciones permiten a Django rastrear los cambios de su modelo y sincronice la base de datos en consecuencia.
- **models.py** : esto incluye los modelos de datos de su aplicación; todo Django las aplicaciones deben tener un archivo `models.py` , pero este archivo puede dejarse vacío.
- **tests.py** : aquí es donde puede agregar pruebas para su aplicación.
- **views.py** : Aquí va la lógica de su aplicación; cada vista recibe un Solicitud HTTP, la procesa y devuelve una respuesta.

Diseñar el esquema de datos del blog

Comenzará a diseñar el esquema de datos de su blog definiendo los modelos de datos para tu blog. Un modelo es una clase de Python que subclasifica `django.db.models.Model` en el que cada atributo representa un campo de base de datos. Django creará una tabla para cada modelo definido en el archivo `models.py` . Cuando crea un modelo, Django proporcionarle una API práctica para consultar objetos en la base de datos fácilmente.

Primero, debe definir un modelo de publicación . Agregue las siguientes líneas a models.py archivo de la aplicación del blog :

```

desde modelos de importación django.db
desde django.utils importar zona horaria
desde django.contrib.auth.models import User

class Post (modelos.Modelo):
    STATUS_CHOICES = (
        ('borrador', 'Borrador'),
        ('publicado', 'Publicado'),
    )
    title = modelos.CharField (max_length = 250)
    slug = modelos.SlugField (max_length = 250,
                             unique_for_date = 'publicar')
    autor = modelos.ForeignKey (Usuario,
                                on_delete = modelos.CASCADE,
                                related_name = 'blog_posts')

    cuerpo = modelos.TextField ()
    publicar = modelos.DateTimeField (predeterminado = timezone.now)
    creado = modelos.DateTimeField (auto_now_add = True)
    actualizado = modelos.DateTimeField (auto_now = True)
    status = modelos.CharField (max_length = 10,
                                opciones = STATUS_CHOICES,
                                predeterminado = 'borrador')

class Meta:
    ordering = ('-publish',)

def __str__ (yo):
    volver self.title

```

Este es su modelo de datos para publicaciones de blog. Echemos un vistazo a los campos que acaba de definir para este modelo:

- **título** : este es el campo para el título de la publicación. Este campo es CharField , que se traduce en una columna VARCHAR en la base de datos SQL.
- **slug** : este es un campo destinado a ser utilizado en URL. Una babosa es una etiqueta corta que contiene solo letras, números, guiones bajos o guiones. Usará el campo slug para crear URL hermosas y compatibles con SEO para sus publicaciones de blog. Ha agregado el parámetro unique_for_date a este campo para que puede crear URL para publicaciones usando su fecha de publicación y slug . Django lo hará evitar que varias publicaciones tengan el mismo slug para una fecha determinada.

[11]

Creación de una aplicación de blog

- **autor** : este campo define una relación de varios a uno, lo que significa que cada la publicación es escrita por un usuario, y un usuario puede escribir cualquier cantidad de publicaciones. Para esto campo, Django creará una clave externa en la base de datos utilizando el clave del modelo relacionado. En este caso, confía en el modelo de usuario de el sistema de autenticación Django. El parámetro on_delete especifica el comportamiento a adoptar cuando se elimina el objeto referenciado. Esto no es específico a Django; es un estándar SQL. Usando CASCADE , especificas que cuando

el usuario referenciado se elimina, la base de datos también eliminará todos los blogs relacionados puestos. Puedes echar un vistazo a todas las opciones posibles en https://docs.djangoproject.com/en/3.0/ref/models/fields/#django.db.models.ForeignKey.on_delete.

Usted especifica el nombre de la relación inversa, de usuario para Publica , con el related_name atributo. Esto te permitirá acceder fácilmente a objetos relacionados. Aprenderá más sobre esto más adelante.

- cuerpo : Este es el cuerpo de la publicación. Este campo es un campo de texto que se traduce en una columna de TEXTO en la base de datos SQL.
- publicar : esta fecha y hora indica cuándo se publicó la publicación. Tú use el método de zona horaria ahora de Django como valor predeterminado. Esto devuelve el fecha y hora actual en un formato compatible con la zona horaria. Puedes pensar en ello como un versión compatible con la zona horaria del método estándar datetime.now de Python .
- creado : esta fecha y hora indica cuándo se creó la publicación. Desde que tu están usando auto_now_add aquí, la fecha se guardará automáticamente cuando creando un objeto.
- actualizado : esta fecha y hora indica la última vez que se actualizó la publicación. Ya que estás usando auto_now aquí, la fecha se actualizará automáticamente cuando salvar un objeto.
- estado : este campo muestra el estado de una publicación. Usas un parámetro de opciones , por lo que el valor de este campo solo se puede establecer en una de las opciones dadas.

Django viene con diferentes tipos de campos que puede usar para definir sus modelos.

Puede encontrar todos los tipos de campos en <https://docs.djangoproject.com/en/3.0/ref/models/fields/>.

La clase Meta dentro del modelo contiene metadatos. Le dices a Django que ordene los resultados por el campo de publicación en orden descendente de forma predeterminada cuando consulta la base de datos. El orden descendente se especifica mediante el prefijo negativo. Al hacer esto, las publicaciones publicado recientemente aparecerá primero.

El método `__str__()` es la representación legible por humanos predeterminada del objeto. Django lo usará en muchos lugares, como el sitio de administración.

[12]

Si viene de usar Python 2.x, tenga en cuenta que en Python 3, todos las cadenas se consideran nativamente Unicode; por lo tanto, solo usamos el El método `__str__()` y el método `__unicode__()` están obsoletos.

Activando la aplicación

Para que Django realice un seguimiento de su aplicación y pueda crear una base de datos tablas para sus modelos, tienes que activarlo. Para hacer esto, edite el archivo `settings.py` y agregue `blog.apps.BlogConfig` a la configuración `INSTALLED_APPS` . Debe tener un aspecto como este:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'blog.apps.BlogConfig',  
]
```

La clase `BlogConfig` es la configuración de su aplicación. Ahora Django lo sabe

su aplicación está activa para este proyecto y podrá cargar sus modelos.

Crear y aplicar migraciones

Ahora que tiene un modelo de datos para las publicaciones de su blog, necesitará una base de datos mesa para ello. Django viene con un sistema de migración que rastrea los cambios realizados en modelos y les permite propagarse a la base de datos. Como se mencionó, la migración el comando aplica migraciones para todas las aplicaciones enumeradas en `INSTALLED_APPS` ; eso sincroniza la base de datos con los modelos actuales y las migraciones existentes.

Primero, deberá crear una migración inicial para su modelo de publicación . En la raíz directorio de su proyecto, ejecute el siguiente comando:

blog de makemigrations de python manage.py

Debería obtener el siguiente resultado:

Migraciones para 'blog':

blog / migrations / 0001_initial.py
- Crear publicación modelo

[13]

Creación de una aplicación de blog

Django acaba de crear el archivo `0001_initial.py` dentro del directorio de migraciones de la aplicación de blog . Puede abrir ese archivo para ver cómo aparece una migración. Una migración específica dependencias de otras migraciones y operaciones para realizar en la base de datos para sincronizarlo con los cambios del modelo.

Echemos un vistazo al código SQL que Django ejecutará en la base de datos para crear la mesa para su modelo. El comando `sqlmigrate` toma los nombres de la migración y devuelve su SQL sin ejecutarlo. Ejecute el siguiente comando para inspeccionar el Salida SQL de su primera migración:

Python manage.py sqlmigrate blog 0001

La salida debería verse como sigue:

EMPEZAR;

-

- Crear publicación modelo

-

**CREATE TABLE "blog_post" ("id" entero NOT NULL CLAVE PRINCIPAL
AUTOINCREMENT, "título" varchar (250) NOT NULL, "slug" varchar (250) NOT
NULL, "cuerpo" texto NOT NULL, "publicación" fecha y hora NOT NULL, "creado"
datetime NOT NULL, datetime "actualizado" NOT NULL, "status" varchar (10)
NOT NULL, "author_id" entero NOT NULL REFERENCES "auth_user" ("id")
DEFERRABLE INICIALMENTE APLAZADO);**

CREAR ÍNDICE "blog_post_slug_b95473f2" ON "blog_post" ("slug");

CREAR ÍNDICE "blog_post_author_id_dd7a8485" ON "blog_post" ("author_id");

COMETER;

La salida exacta depende de la base de datos que esté utilizando. La salida anterior se genera para SQLite. Como puede ver en la salida, Django genera la tabla nombres combinando el nombre de la aplicación y el nombre en minúsculas del modelo (`blog_post`), pero también puede especificar un nombre de base de datos personalizado para su modelo en la clase Meta del modelo usando el atributo `db_table` .

Django crea una clave primaria automáticamente para cada modelo, pero también puede anule esto especificando `primary_key = True` en uno de los campos de su modelo.

La clave primaria predeterminada es una columna de identificación , que consta de un número entero que es incrementa automáticamente. Esta columna corresponde al campo `id` que es

agregado automáticamente a sus modelos.
Sincronicemos su base de datos con el nuevo modelo. Ejecute el siguiente comando para aplicar migraciones existentes:

```
python manage.py migrar
```

[14]

Obtendrá una salida que termina con la siguiente línea:

```
Aplicando blog.0001_initial ... OK
```

Acaba de aplicar migraciones para las aplicaciones enumeradas en `INSTALLED_APPS`, incluidas su aplicación de blog. Después de aplicar las migraciones, la base de datos refleja la estado actual de sus modelos.

Si edita el archivo `models.py` para agregar, eliminar o cambiar los campos de modelos, o si agrega nuevos modelos, tendrá que crear una nueva migración utilizando el comando `makemigrations`. La migración permitirá a Django realizar un seguimiento del modelo cambios. Luego, tendrá que aplicarlo con el comando `migrate` para mantener el base de datos sincronizada con sus modelos.

Creación de un sitio de administración para modelos

Ahora que ha definido el modelo de publicación, creará una administración simple sitio para administrar las publicaciones de su blog. Django viene con una administración incorporada interfaz que es muy útil para editar contenido. El sitio de Django se construye dinámicamente leyendo los metadatos de su modelo y proporcionando una interfaz lista para producción para editar contenido. Puede usarlo de inmediato, configurando cómo desea su modelos que se mostrarán en él.

La aplicación `django.contrib.admin` ya está incluida en `INSTALLED_APPS` ajuste, por lo que no es necesario agregarlo.

Creando un superusuario

Primero, deberá crear un usuario para administrar el sitio de administración. Ejecutar el siguiente comando:

```
python manage.py crea superusuario
```

Verá la siguiente salida; ingrese su nombre de usuario, correo electrónico y contraseña, de la siguiente manera:

Nombre de usuario (déjelo en blanco para usar 'admin'): admin

Dirección de correo electrónico: admin@admin.com

Contraseña: *****

Contraseña de nuevo): *****

Superusuario creado correctamente.

[15]

Creación de una aplicación de blog

El sitio de administración de Django

Ahora inicie el servidor de desarrollo con `python manage.py runserver` y abra `http://127.0.0.1:8000/admin/` en su navegador. Deberías consultar la página de inicio de sesión de administración, como se muestra en la siguiente captura de pantalla:

Figura 1.3: La pantalla de inicio de sesión del sitio de administración de Django

Inicie sesión con las credenciales del usuario que creó en el paso anterior. Vas a consultar la página de índice del sitio de administración, como se muestra en la siguiente captura de pantalla:

Figura 1.4: La página de índice del sitio de administración de Django

Los modelos de Grupo y Usuario que puede ver en la captura de pantalla anterior son parte del marco de autenticación de Django ubicado en `django.contrib.auth`. Si tu haga clic en **Usuarios**, verá el usuario que creó anteriormente.

Agregar modelos al sitio de administración

Agreguemos sus modelos de blog al sitio de administración. Edite el archivo `admin.py` del aplicación de blog y haz que se vea así:

```
desde django.contrib import admin
desde .models import Post
```

```
admin.site.register (Publicar)
```

Ahora vuelva a cargar el sitio de administración en su navegador. Deberías ver tu publicación modelo en el sitio, de la siguiente manera:

Figura 1.5: El modelo de publicación de la aplicación de blog incluida en la página de índice del sitio de administración de Django

Eso fue fácil, ¿verdad? Cuando registra un modelo en el sitio de administración de Django, obtienes una interfaz fácil de usar generada mediante la introspección de tus modelos que permite para enumerar, editar, crear y eliminar objetos de una manera sencilla.

Creación de una aplicación de blog

Haga clic en el enlace **Agregar** junto a **Publicaciones** para agregar una nueva publicación. Notarás la forma que Django ha generado dinámicamente para su modelo, como se muestra a continuación captura de pantalla:

Figura 1.6: El formulario de edición del sitio de administración de Django para el modelo Post

Django usa diferentes widgets de formulario para cada tipo de campo. Incluso campos complejos, como `DateTimeField`, se muestran con una interfaz sencilla, como un Selector de fechas de JavaScript.

Complete el formulario y haga clic en el botón **GUARDAR**. Debería ser redirigido a la publicación. página de lista con un mensaje de éxito y la publicación que acaba de crear, como se muestra en la siguiente captura de pantalla:

[18]

Figura 1.7: Vista de lista del sitio de administración de Django para el modelo Post con un mensaje agregado exitosamente

Personalizando la forma en que son los modelos desplegado

Ahora, veremos cómo personalizar el sitio de administración. Edite el `admin.py` archivo de su aplicación de blog y cámbielo, de la siguiente manera:

```
desde django.contrib import admin
desde .models import Post

@ admin.register (Publicar)
clase PostAdmin (admin.ModelAdmin):
    list_display = ('título', 'slug', 'autor', 'publicar', 'estado')
```

Le está diciendo al sitio de administración de Django que su modelo está registrado en el sitio utilizando una clase personalizada que hereda de `ModelAdmin`. En esta clase, tu puede incluir información sobre cómo mostrar el modelo en el sitio y cómo interactuar con él.

El atributo `list_display` le permite establecer los campos de su modelo que desea mostrar en la página de lista de objetos de administración. El `@ admin.register ()` decorador realiza la misma función que el `admin.site.register ()` función que reemplazó, registrando la clase `ModelAdmin` que decora.

[19]

Creación de una aplicación de blog

Personalicemos el modelo de administración con algunas opciones más, usando el siguiente código:

```
@ admin.register (Publicar)
class PostAdmin (admin.ModelAdmin):
    list_display = ('titulo', 'slug', 'autor', 'publicar', 'estado')
    list_filter = ('estado', 'creado', 'publicar', 'autor')
    search_fields = ('titulo', 'cuerpo')
    campos_prepoplados = {'slug': ('titulo',)}
    raw_id_fields = ('autor',)
    date_hierarchy = 'publicar'
    ordering = ('estado', 'publicar')
```

Regrese a su navegador y vuelva a cargar la página de la lista de publicaciones. Ahora, se verá así:

Figura 1.8: Vista de lista personalizada del sitio de administración de Django para el modelo Post

Puede ver que los campos que se muestran en la página de la lista de publicaciones son los que especificó en el atributo `list_display`. La página de lista ahora incluye una barra lateral derecha que le permite filtrar los resultados por los campos incluidos en el atributo `list_filter`.

Ha aparecido una barra de búsqueda en la página. Esto se debe a que ha definido una lista de campos de búsqueda utilizando el atributo `search_fields`. Justo debajo de la búsqueda barra, hay enlaces de navegación para navegar a través de una jerarquía de fechas; esto tiene definido por el atributo `date_hierarchy`. También puede ver que las publicaciones están ordenados por columnas **STATUS** y **PUBLISH** de forma predeterminada. Tú has especificado los criterios de ordenación predeterminados mediante el atributo de ordenación.

[20]

A continuación, haga clic en el enlace **AÑADIR POST**. También notará algunos cambios aquí. Como escribe el título de una nueva publicación, el campo slug se rellena automáticamente. Tienes le dijo a Django que rellenara previamente el campo slug con la entrada del campo de título usando el atributo `prepopulated_fields`.

Además, el campo de autor ahora se muestra con un widget de búsqueda que puede escalar mucho mejor que una entrada de selección desplegable cuando tienes miles de usuarios. Esto es logrado con el atributo `raw_id_fields` y se ve así:

Figura 1.9: El widget para seleccionar objetos relacionados para el campo de autor del modelo Post

Con unas pocas líneas de código, ha personalizado la forma en que se muestra su modelo. en el sitio de administración. Hay muchas formas de personalizar y ampliar la Sitio de administración de Django; aprenderá más sobre esto más adelante en este libro.

Trabajar con QuerySets y administradores

Ahora que tiene un sitio de administración completamente funcional para administrar los contenido, es hora de aprender a recuperar información de la base de datos e interactuar con eso. Django viene con una poderosa API de abstracción de base de datos que le permite crear, recuperar, actualizar y eliminar objetos fácilmente. El **mapeador relacional de objetos de Django** (**ORM**) es compatible con MySQL, PostgreSQL, SQLite, Oracle y MariaDB.

Recuerda que puedes definir la base de datos de tu proyecto en la configuración BASES DE DATOS del archivo `settings.py` de su proyecto. Django puede trabajar con múltiples bases de datos en un tiempo, y puede programar enrutadores de base de datos para crear esquemas de enrutamiento personalizados.

Una vez que haya creado sus modelos de datos, Django le ofrece una API gratuita para interactuar con ellos. Puede encontrar la referencia del modelo de datos de la documentación oficial en <https://docs.djangoproject.com/en/3.0/ref/models/>.

El ORM de Django se basa en QuerySets. Un QuerySet es una colección de bases de datos consultas para recuperar objetos de su base de datos. Puede aplicar filtros a QuerySets para limitar los resultados de la consulta en función de los parámetros dados.

Creando objetos

Abra la terminal y ejecute el siguiente comando para abrir el shell de Python:

shell de python manage.py

```
>>> desde django.contrib.auth.models import User
>>> de blog.models importar Publicación
>>> usuario = User.objects.get (nombre de usuario = 'admin')
>>> publicación = Publicación (título = 'Otra publicación',
...                             slug = 'otra-publicación',
...                             body = 'Post body.',
...                             autor = usuario)
>>> post.save ()
```

Analicemos qué hace este código. Primero, recupera el objeto de usuario con el nombre de usuario admin :

```
usuario = User.objects.get (username = 'admin')
```

El método `get ()` le permite recuperar un solo objeto de la base de datos. Nota que este método espera un resultado que coincida con la consulta. Si no se devuelven resultados por la base de datos, este método generará una excepción `DoesNotExist`, y si el La base de datos devuelve más de un resultado, generará un `MultipleObjectsReturned` excepción. Ambas excepciones son atributos de la clase de modelo que la consulta está siendo realizado en.

Luego, crea una instancia de `Post` con un título, slug y cuerpo personalizados, y configura el usuario que recuperó anteriormente como autor de la publicación:

```
publicación = Publicación (título = 'Otra publicación', slug = 'otra publicación', cuerpo = 'Publicación
cuerpo.', autor = usuario)
```

Este objeto está en la memoria y no se conserva en la base de datos.

Finalmente, guarde el objeto `Post` en la base de datos usando el método `save ()` :

```
post.save ()
```

La acción anterior realiza una instrucción `INSERT SQL` entre bastidores. Ha visto cómo crear un objeto en la memoria primero y luego persistirlo en el base de datos, pero también puede crear el objeto y conservarlo en la base de datos en una operación única usando el método `create ()`, como sigue:

```
Post.objects.create (title = 'Una publicación más',
                    slug = 'una-publicación-más',
                    body = 'Post body.',
                    autor = usuario)
```

[22]

Actualizar objetos

Ahora, cambie el título de la publicación a algo diferente y guarde el objeto nuevamente:

```
>>> post.title = 'Nuevo título'
>>> post.save ()
```

Esta vez, el método `save ()` realiza una instrucción `UPDATE SQL`.

Los cambios que realice en el objeto no se conservan en el base de datos hasta que llame al método `save ()`.

Recuperando objetos

Ya sabe cómo recuperar un solo objeto de la base de datos usando `get()` método. Accedí a este método utilizando `Post.objects.get()`. Cada modelo de Django tiene al menos un administrador y el administrador predeterminado se llama `objects`. Obtiene un Objeto `QuerySet` usando su administrador de modelos. Para recuperar todos los objetos de una tabla, simplemente usa el método `all()` en el administrador de objetos predeterminado, así:

```
>>> all_posts = Post.objects.all()
```

Así es como se crea un `QuerySet` que devuelve todos los objetos de la base de datos. Tenga en cuenta que este `QuerySet` aún no se ha ejecutado. Los `QuerySets` de Django son *perezosos*, lo que significa sólo se evalúan cuando se les obliga a hacerlo. Este comportamiento hace que `QuerySets` muy eficiente. Si no configura el `QuerySet` en una variable, sino que lo escribe directamente en el shell de Python, la declaración SQL del `QuerySet` se ejecuta porque fuerza para generar resultados:

```
>>> all_posts
```

Usando el método `filter()`

Para filtrar un `QuerySet`, puede utilizar el método `filter()` del administrador. Por ejemplo, puede recuperar todas las publicaciones publicadas en el año 2020 utilizando el siguiente `QuerySet`:

```
>>> Post.objects.filter(publish__year = 2020)
```

También puede filtrar por varios campos. Por ejemplo, puede recuperar todas las publicaciones publicado en 2020 por el autor con el nombre de usuario `admin`:

```
>>> Post.objects.filter(publish__year = 2020, author__username = 'admin')
```

[23]

Creación de una aplicación de blog

Esto equivale a construir el mismo `QuerySet` encadenando múltiples filtros:

```
>>> Post.objects.filter(publish__year = 2020) \
>>>     .filter(author__username = 'admin')
```

Las consultas con métodos de búsqueda de campo se crean utilizando dos subraya, por ejemplo, `publish__year`, pero el mismo La notación también se utiliza para acceder a campos de modelos relacionados, como `autor__nombreusuario`.

Usando `exclude()`

Puede excluir ciertos resultados de su `QuerySet` utilizando el método `exclude()` del gerente. Por ejemplo, puede recuperar todas las publicaciones publicadas en 2020 cuyo los títulos no comienzan con `Por qué`:

```
>>> Post.objects.filter(publish__year = 2020) \
>>>     .exclude(title__startswith = 'Por qué')
```

Usando `order_by()`

Puede ordenar los resultados por diferentes campos usando el método `order_by()` del gerente. Por ejemplo, puede recuperar todos los objetos ordenados por su título, de la siguiente manera:

```
>>> Post.objects.order_by('titulo')
```

El orden ascendente está implícito. Puede indicar el orden descendente con un signo negativo prefijo, así:

```
>>> Post.objects.order_by('- titulo')
```

Eliminar objetos

Para eliminar un objeto, puede hacerlo desde la instancia del objeto usando el método `delete ()` :

```
>>> publicación = Post.objects.get (id = 1)
>>> post.delete ()
```

Tenga en cuenta que eliminar objetos también eliminará cualquier dependiente relaciones para objetos `ForeignKey` definidos con `on_delete` establecido a la cascada.

[24]

Cuando se evalúan los QuerySets

La creación de un `QuerySet` no implica ninguna actividad de base de datos hasta que se evalúa. Los `QuerySets` suelen devolver otro `QuerySet` no evaluado. Puede concatenar como tantos filtros como desee para un `QuerySet`, y no llegará a la base de datos hasta que se evalúa `QuerySet`. Cuando se evalúa un `QuerySet`, se traduce en un SQL consulta a la base de datos.

Los `QuerySets` solo se evalúan en los siguientes casos:

- La primera vez que los repites
- Cuando los corta, por ejemplo, `Post.objects.all () [: 3]`
- Cuando los conserva en vinagre o en caché
- Cuando llamas a `repr ()` o `len ()` en ellos
- Cuando llama explícitamente a `list ()` en ellos
- Cuando los prueba en una declaración, como `bool ()` , `o` , `y` , o `si`

Creando administradores de modelos

Como mencioné anteriormente, los objetos son el administrador predeterminado de cada modelo que recupera todos los objetos de la base de datos. Sin embargo, también puede definir administradores personalizados para tus modelos. Creará un administrador personalizado para recuperar todas las publicaciones con el estado publicado .

Hay dos formas de agregar o personalizar administradores para sus modelos: puede agregar métodos de administrador adicionales a un administrador existente, o crear un nuevo administrador modificando el `QuerySet` inicial que devuelve el administrador. El primer método proporciona usted con una API `QuerySet` como `Post.objects.my_manager ()` , y el último le proporciona `Post.my_manager.all ()` . El gerente le permitirá recuperar publicaciones usando `Post.published.all ()` .

Edite el archivo `models.py` de su aplicación de blog para agregar el administrador personalizado:

```
class PublishedManager (modelos.Manager):
    def get_queryset (yo):
        return super (PublishedManager,
                        self).get_queryset () \
                        .filter (estado = 'publicado')

class Post (modelos.Modelo):
    # ...
    objects = modelos.Manager () # El administrador predeterminado.
    Published = PublishedManager () # Nuestro administrador personalizado.
```

[25]

Creación de una aplicación de blog

El primer administrador declarado en un modelo se convierte en el administrador predeterminado. Puedes usar el meta atributo `default_manager_name` para especificar un administrador predeterminado diferente. Si no se define ningún administrador en el modelo, Django crea automáticamente los objetos administrador predeterminado para ello. Si declara gerentes para su modelo pero desea para mantener el administrador de objetos también, debe agregarlo explícitamente a su modelo. En el código anterior, agrega el administrador de objetos predeterminado y el publicado administrador personalizado para el modelo `Post` .

El método `get_queryset()` de un administrador devuelve el `QuerySet` que será ejecutado. Anula este método para incluir su filtro personalizado en la `QuerySet`.

Ahora ha definido su administrador personalizado y lo ha agregado al modelo de publicación ; tú puede utilizarlo para realizar consultas. Probémoslo.

Inicie el servidor de desarrollo nuevamente con el siguiente comando:

shell de python `manage.py`

Ahora, puede importar el modelo de publicación y recuperar todas las publicaciones publicadas cuyo título comienza con `Who` , ejecutando el siguiente `QuerySet`:

```
>>> de blog.models importar Publicación
>>> Post.published.filter (title__startswith = 'Quién')
```

Para obtener resultados para este `QuerySet`, asegúrese de establecer el campo publicado en Verdadero en el objeto `Post` cuyo título comienza con `Who` .

Lista de construcción y vistas de detalle

Ahora que tiene conocimiento sobre cómo usar el ORM, está listo para construir las vistas de la aplicación de blog . Una vista de Django es solo una función de Python que recibe una solicitud web y devuelve una respuesta web. Toda la lógica para devolver el La respuesta deseada entra en la vista.

Primero, creará las vistas de su aplicación, luego definirá un patrón de URL para cada vista y, finalmente, creará plantillas HTML para representar los datos generados por las vistas. Cada vista renderizará una plantilla, pasándole variables y devuelve una respuesta HTTP con la salida renderizada.

Crear vistas de lista y detalle

Comencemos por crear una vista para mostrar la lista de publicaciones. Edite el archivo `views.py` de tu aplicación de blog y haz que se vea así:

```

desde django.shortcuts import render, get_object_or_404
desde .models import Post

```

```

def post_list(solicitud):
    posts = Publicar.publicados.todos ()
    return render (solicitud,
                    'blog / post / list.html',
                    {'publicaciones': publicaciones})

```

Acaba de crear su primera vista de Django. La vista `post_list` toma la solicitud objeto como único parámetro. Este parámetro es obligatorio para todas las vistas. En esta vista, recuperas todas las publicaciones con el estado publicado usando el administrador publicado que creó anteriormente.

Finalmente, usa el atajo `render ()` proporcionado por Django para renderizar la lista de publicaciones con la plantilla dada. Esta función toma el objeto de solicitud, la ruta de la plantilla, y las variables de contexto para representar la plantilla dada. Devuelve una `HttpResponse` objeto con el texto renderizado (normalmente código HTML). El atajo `render ()` tiene en cuenta el contexto de la solicitud, por lo que cualquier variable establecida por el contexto de la plantilla procesadores es accesible por la plantilla dada. Los procesadores de contexto de plantilla son simplemente llamadas que establecen variables en el contexto. Aprenderás a usarlos en el *Capítulo 3, Ampliación de la aplicación de blog*.

Creemos una segunda vista para mostrar una sola publicación. Agregue la siguiente función a el archivo `views.py`:

```

def post_detail (solicitud, año, mes, día, publicación):
    post = get_object_or_404 (Publicar, slug = publicar,
                              status = 'publicado',
                              Publish__year = año,
                              publish__month = mes,
                              Publish__day = día)

    return render (solicitud,
                    'blog / post / detail.html',
                    {'publicación': publicación})

```

Esta es la vista detallada de la publicación. Esta vista toma el año, mes, día y publicación. argumentos para recuperar una publicación publicada con el slug y la fecha dados. Tenga en cuenta que cuando creó el modelo de publicación, agregó el parámetro `unique_for_date` al campo de las babosas. Esto asegura que solo habrá una publicación con un slug para un dada la fecha y, por lo tanto, puede recuperar publicaciones individuales utilizando la fecha y el slug. En el vista detallada, usa el acceso directo `get_object_or_404 ()` para recuperar el enviar. Esta función recupera el objeto que coincide con los parámetros dados o un Excepción HTTP 404 (no encontrado) si no se encuentra ningún objeto. Finalmente, usa el `render ()` atajo para representar la publicación recuperada usando una plantilla.

[27]

Creación de una aplicación de blog

Agregar patrones de URL para sus vistas

Los patrones de URL le permiten asignar URL a vistas. Un patrón de URL se compone de un patrón de cadena, una vista y, opcionalmente, un nombre que le permite nombrar la URL en todo el proyecto. Django recorre cada patrón de URL y se detiene en el primero que coincide con la URL solicitada. Luego, Django importa la vista de la coincidencia Patrón de URL y lo ejecuta, pasando una instancia de la clase `HttpRequest` y la palabra clave o los argumentos posicionales.

Cree un archivo `urls.py` en el directorio de la aplicación de blog y agregue lo siguiente líneas a él:

```

desde la ruta de importación django.urls
de . importar vistas

```



```

app_name = 'blog'

urlpatterns = [
    # vistas de publicaciones
    ruta ('', views.post_list, name = 'post_list'),
    ruta ('<int: año> / <int: mes> / <int: día> / <slug: post> /',
        views.post_detail,
        name = 'post_detail'),
]

```

En el código anterior, define un espacio de nombres de aplicación con el nombre_aplicación variable. Esto le permite organizar las URL por aplicación y usar el nombre cuando refiriéndose a ellos. Usted define dos patrones diferentes usando la función path () . los El primer patrón de URL no acepta argumentos y se asigna a la vista post_list . El segundo patrón toma los siguientes cuatro argumentos y se asigna al post_vista detallada :

- año : requiere un número entero
- mes : requiere un número entero
- día : requiere un número entero
- publicación : puede estar compuesta de palabras y guiones

Utiliza corchetes angulares para capturar los valores de la URL. Cualquier valor especificado en el patrón de URL como <parámetro> se captura como una cadena. Usas convertidores de ruta, como <int: year> , para hacer coincidir y devolver específicamente un entero y <slug: post> para que coincida específicamente con una babosa. Puede ver todos los convertidores de ruta proporcionados por Django en <https://docs.djangoproject.com/en/3.0/topics/http/urls/#path-convertidores> .

[28]

Si usar path () y convertidores no es suficiente para usted, puede usar re_path () en lugar de definir patrones de URL complejos con expresiones regulares de Python. Usted puede Obtenga más información sobre cómo definir patrones de URL con expresiones regulares en https://docs.djangoproject.com/en/3.0/ref/urls/#django.urls.re_path . Si no lo has hecho trabajado con expresiones regulares antes, es posible que desee echar un vistazo a la *CÓMO de expresión regular* ubicado en <https://docs.python.org/3/howto/regex.html> primero.

Crear un archivo urls.py para cada aplicación es la mejor manera de Haga que sus aplicaciones sean reutilizables por otros proyectos.

A continuación, debe incluir los patrones de URL de la aplicación de blog en la Patrones de URL del proyecto.

Edite el archivo urls.py ubicado en el directorio mysite de su proyecto y hágalo tener el siguiente aspecto:

```

de la ruta de importación django.urls, incluir
desde django.contrib import admin

urlpatterns = [
    ruta ('admin /', admin.site.urls),
    ruta ('blog /', include ('blog.urls', namespace = 'blog')) ,
]

```

El nuevo patrón de URL definido con incluir se refiere a los patrones de URL definidos en

la aplicación de blog para que se incluyan en el blog / ruta. Usted incluye estos patrones en el blog del espacio de nombres. Los espacios de nombres deben ser únicos en todo su proyecto. Más tarde, podrá consultar las URL de su blog fácilmente utilizando el espacio de nombres seguido de dos puntos y el nombre de la URL, por ejemplo, blog: post_list y blog: post_detail. Puede obtener más información sobre los espacios de nombres de URL en <https://docs.djangoproject.com/en/3.0/topics/http/urls/#url-namespaces>.

URL canónicas para modelos

Una URL canónica es la URL preferida para un recurso. Puede tener diferentes páginas en su sitio donde muestra publicaciones, pero hay una única URL que usa como URL principal de una publicación de blog. La convención en Django es agregar un `get_absolute_url()` al modelo que devuelve la URL canónica del objeto.

[29]

Creación de una aplicación de blog

Puede utilizar la URL `post_detail` que ha definido en la sección anterior para crear la URL canónica para los objetos `Post`. Para este método, utilizará el método `reverse()`, que le permite crear URL por su nombre y contraseña parámetros opcionales. Puede obtener más información sobre las funciones de la utilidad URL en <https://docs.djangoproject.com/en/3.0/ref/urlresolvers/>.

Edite el archivo `models.py` de la aplicación de blog y agregue el siguiente código:

```
desde django.urls import reverse

clase Post (modelos.Modelo):
    # ...
    def get_absolute_url (yo):
        return reverse ('blog: post_detail',
                        args = [self.publish.year,
                               self.publish.month,
                               self.publish.day, self.slug])
```

Utilizará el método `get_absolute_url()` en sus plantillas para enlazar a puestos específicos.

Creando plantillas para sus vistas

Ha creado vistas y patrones de URL para la aplicación de blog. Patrones de URL asignar URL a vistas, y las vistas deciden qué datos se devuelven al usuario. Las plantillas definen cómo se muestran los datos; normalmente están escritos en HTML en combinación con el lenguaje de plantilla Django. Puedes encontrar más información sobre el lenguaje de la plantilla de Django en <https://docs.djangoproject.com/es/3.0/ref/templates/language/>.

Agreguemos plantillas a su aplicación para mostrar publicaciones de una manera fácil de usar.

Cree los siguientes directorios y archivos dentro del directorio de su aplicación de blog :

```
plantillas/
  Blog/
    base.html
    enviar/
      list.html
      detail.html
```

La estructura anterior será la estructura de archivos de sus plantillas. La `base.html` El archivo incluirá la estructura HTML principal del sitio web y dividirá el contenido en el área de contenido principal y una barra lateral. Los archivos `list.html` y `detail.html` heredarán del archivo `base.html` para representar la lista de publicaciones del blog y las vistas detalladas,

Capítulo 1

Django tiene un poderoso lenguaje de plantillas que le permite especificar cómo se encuentran los datos desplegado. Está basado en *etiquetas de plantilla*, *variables de la plantilla*, y *los filtros de plantilla*:

- Las etiquetas de plantilla controlan la representación de la plantilla y se ven como `{% tag %}`
- Las variables de plantilla se reemplazan con valores cuando se renderiza la plantilla y parece `{{variable}}`
- Los filtros de plantilla le permiten modificar las variables para que se muestren y se vean como `{{ variable | filtro }}`.

Puede ver todas las etiquetas y filtros de plantilla integrados en <https://docs.djangoproject.com/es/3.0/ref/templates/builtins/>.

Edite el archivo `base.html` y agregue el siguiente código:

```
{% load static %}
<!DOCTYPE html>
<html>
<cabeza>
  <title> {% block title %} {% endblock %} </title>
  <link href = "{% static 'css / blog.css'" rel = "hoja de estilo">
</head>
<cuerpo>
  <div id = "contenido">
    {% block content %}
    {% endblock %}
  </div>
  <div id = "barra lateral">
    <h2> Mi blog </h2>
    <p> Este es mi blog. </p>
  </div>
</body>
</html>
```

`{% load static %}` le dice a Django que cargue las etiquetas de plantilla estáticas que se proporcionan por la aplicación `django.contrib.staticfiles`, que está contenida en el

Configuración `INSTALLED_APPS`. Después de cargarlos, puede usar el `{% static %}` etiqueta de plantilla a lo largo de esta plantilla. Con esta etiqueta de plantilla, puede incluir el archivos estáticos, como el archivo `blog.css`, que encontrará en el código de este ejemplo en el directorio `static /` de la aplicación de blog. Copie el directorio estático / del código que acompaña a este capítulo en la misma ubicación que su proyecto para aplicar los estilos CSS a las plantillas. Puedes encontrar el contenido del directorio en <https://github.com/PacktPublishing/Django-3-by-Example/tree/master/Capitulo01/misito/blog/static>.

Puede ver que hay dos etiquetas `{% block%}` . Estos le dicen a Django que quieres para definir un bloque en esa área. Las plantillas que heredan de esta plantilla pueden completar los bloques con contenido. Ha definido un bloque llamado título y un bloque llamado contenido .

Vamos a editar el poste / list.html archivo y hacer que se vea como la siguiente:

```
{% extiende "blog / base.html"%}

{% block title%} Mi blog {% endblock%}

{% block content%}
<h1> Mi blog </h1>
{% para publicación en publicaciones%}
<h2>
    <a href="{{ post.get_absolute_url }}">
        {{ título de la entrada }}
    </a>
</h2>
<p class = "fecha">
    Publicado {{post.publish}} por {{post.author}}
</p>
{{post.body | truncatewords: 30 | saltos de línea}}
{% endfor%}
{% endblock%}
```

Con la etiqueta de plantilla `{% extensions%}` , le dices a Django que herede del blog / plantilla base.html . Luego, llenas los bloques de título y contenido de la base. plantilla con contenido. Recorre las publicaciones y muestra su título, fecha, autor y cuerpo, incluido un enlace en el título a la URL canónica de la publicación.

En el cuerpo de la publicación, aplica dos filtros de plantilla: `truncatewords` trunca el valor al número de palabras especificadas, y los saltos de línea convierten la salida en Saltos de línea HTML. Puede concatenar tantos filtros de plantilla como desee; cada uno se aplicará a la salida generada por el anterior.

Abra el shell y ejecute el comando `python manage.py runserver` para comenzar el servidor de desarrollo. Abra `http://127.0.0.1:8000/blog/` en su navegador; verá todo funcionando. Tenga en cuenta que debe tener algunas publicaciones con el Estado publicado para mostrarlos aquí. Debería ver algo como esto:

Figura 1.10: La página para la vista de lista de publicaciones

A continuación, edite el archivo `post / detail.html` :

```
{% extiende "blog / base.html"%}

{% block title%} {{post.title}} {% endblock%}

{% block content%}
<h1> {{post.title}} </h1>
<p class = "fecha">
    Publicado {{post.publish}} por {{post.author}}
</p>
{{post.body | linebreak}}
{% endblock%}
```

A continuación, puede volver a su navegador y hacer clic en uno de los títulos de las publicaciones para tomar una mira la vista detallada de la publicación. Debería ver algo como esto:

Figura 1.11: La página para la vista detallada de la publicación

[33]

Creación de una aplicación de blog

Eche un vistazo a la URL; debería ser `/blog/2020/1/1/who-was-django-reinhardt/`. Ha diseñado URL compatibles con SEO para las publicaciones de su blog.

Agregar paginación

Cuando comience a agregar contenido a su blog, puede llegar fácilmente al punto en el que decenas o cientos de publicaciones se almacenan en su base de datos. En lugar de mostrar todos los publicaciones en una sola página, es posible que desee dividir la lista de publicaciones en varias páginas. Esto se puede lograr mediante la paginación. Puede definir la cantidad de publicaciones que desea que se muestre por página y recuperar las publicaciones que corresponden a la página solicitada por el usuario. Django tiene una clase de paginación incorporada que le permite gestione datos paginados fácilmente.

Edite el archivo `views.py` de la aplicación de blog para importar el paginador de Django clases y modifique la vista `post_list`, de la siguiente manera:

```
desde django.core.paginator import Paginator, EmptyPage, \
    PageNotAnInteger

def post_list(solicitud):
    object_list = Publicar.publicado.todos()
    paginator = Paginator(object_list, 3) # 3 publicaciones en cada página
    page = request.GET.get('página')
    tratar:
        posts = paginator.page(página)
```

```

excepto PageNotAnInteger:
    # Si la página no es un número entero, entregue la primera página
    publicaciones = paginator.page(1)
excepto EmptyPage:
    # Si la página está fuera del rango, entregue la última página de resultados
    publicaciones = paginator.page(paginator.num_pages)
return render (solicitud,
               'blog / post / list.html',
               { 'página': página,
                 'publicaciones': publicaciones})

```

Así es como funciona la paginación:

1. Crea una instancia de la clase Paginator con la cantidad de objetos que desea para mostrar en cada página.
2. Obtiene el parámetro page GET , que indica el número de página actual.
3. Obtiene los objetos para la página deseada llamando al método page () de Paginator .

[34]

4. Si el parámetro de página no es un número entero, recupera la primera página de resultados. Si este parámetro es un número mayor que la última página de resultados, recupera la última página.
5. Pasa el número de página y los objetos recuperados a la plantilla.

Ahora tienes que crear una plantilla para mostrar el paginador para que pueda ser incluido en cualquier plantilla que utilice paginación. En la carpeta templates / del blog aplicación, cree un nuevo archivo y asígnele el nombre pagination.html . Agregue lo siguiente Código HTML para el archivo:

```

<div class = "pagination">
  <span class = "step-links">
    {% if page.has_previous%}
      <a href="?page={{ page.previous_page_number }}"> Anterior </a>
    {% terminara si %}
    <span class = "actual">
      Página {{page.number}} de {{page.paginator.num_pages}}.
    </span>
    {% if page.has_next%}
      <a href="?page={{ page.next_page_number }}"> Siguiente </a>
    {% terminara si %}
  </span>
</div>

```

La plantilla de paginación espera un objeto Page para representar el anterior y siguientes enlaces y para mostrar la página actual y las páginas totales de resultados. Volvamos a la plantilla blog / post / list.html e incluya la plantilla pagination.html en la parte inferior del bloque {% content%} , de la siguiente manera:

```

{% block content%}
...
{% incluye "pagination.html" con page = posts%}
{% endblock%}

```

Dado que el objeto de página que está pasando a la plantilla se llama publicaciones , incluye la plantilla de paginación en la plantilla de lista de publicaciones, pasando los parámetros para renderizar correctamente. Puede seguir este método para reutilizar su plantilla de paginación en el vistas paginadas de diferentes modelos.

Creación de una aplicación de blog

Ahora abra `http://127.0.0.1:8000/blog/` en su navegador. Deberías ver la paginación en la parte inferior de la lista de publicaciones y debería poder navegar a través de páginas:

Figura 1.12: La página de la lista de publicaciones que incluye la paginación

Usar vistas basadas en clases

Las vistas basadas en clases son una forma alternativa de implementar vistas como objetos de Python en lugar de funciones. Dado que una vista es invocable, toma una solicitud web y devuelve una respuesta web, también puede definir sus vistas como métodos de clase. Django proporciona clases de vista base para esto. Todos ellos heredan de la clase `View`, que maneja Despacho del método HTTP y otras funcionalidades comunes.

Las vistas basadas en clases ofrecen ventajas sobre las vistas basadas en funciones para algunos casos de uso. Tienen las siguientes características:

- Organizar el código relacionado con los métodos HTTP, como GET, POST o PUT, en métodos separados, en lugar de utilizar la ramificación condicional
- Usar herencia múltiple para crear clases de vista reutilizables (también conocidas como *mixins*)

Puede echar un vistazo a una introducción a las vistas basadas en clases en <https://docs.djangoproject.com/en/3.0/topics/class-based-views/intro/>.

Cambiará su vista de lista de publicaciones a una vista basada en clases para usar el genérico `ListView` ofrecido por Django. Esta vista base le permite listar objetos de cualquier tipo.

Edite el archivo `views.py` de su aplicación de blog y agregue el siguiente código:

```
desde django.views.generic import ListView

class PostListView (ListView):
    queryset = Publicar.publicado.todos ()
    context_object_name = 'publicaciones'
    paginate_by = 3
    template_name = 'blog / post / list.html'
```

Esta vista basada en clases es análoga a la vista anterior `post_list`. En el código anterior, le está diciendo a `ListView` que haga lo siguiente:

- Utilice un `QuerySet` específico en lugar de recuperar todos los objetos. En lugar de definir un conjunto de consultas de atributos, que podría haber determinado `modelo = Mensaje` y Django habría construido el `Post.objects.all ()` `QuerySet` genérico para usted.
- Utilice las publicaciones de la variable de contexto para los resultados de la consulta. La variable predeterminada es `object_list` si no especifica ningún `context_object_name`.
- Pagar el resultado, mostrando tres objetos por página.
- Utilice una plantilla personalizada para representar la página. Si no establece una plantilla predeterminada, `ListView` utilizará `blog / post_list.html`.

Ahora abra el archivo `urls.py` de su aplicación de blog, comente la publicación anterior_ enumerar el patrón de URL y agregar un nuevo patrón de URL utilizando la clase `PostListView`, como sigue:

```
urlpatterns = [
    # vistas de publicaciones
    # ruta ('', views.post_list, name = 'post_list'),
    ruta ('', vistas.PostListView.as_view (), nombre = 'post_list'),
    ruta ('<int: año> / <int: mes> / <int: día> / <slug: post> /',
        views.post_detail,
        name = 'post_detail'),
]
```

Creación de una aplicación de blog

Para que la paginación siga funcionando, debe utilizar el objeto de página correcto que pasado a la plantilla. La vista genérica `ListView` de Django pasa la página seleccionada en una variable llamada `page_obj`, por lo que debe editar su plantilla `post / list.html` en consecuencia para incluir el paginador usando la variable correcta, como sigue:

```
{% incluye "pagination.html" con page = page_obj %}
```


Abra <http://127.0.0.1:8000/blog/> en su navegador y verifique que todo funciona de la misma manera que con la vista anterior de lista de publicaciones . Este es un ejemplo simple de una vista basada en clases que usa una clase genérica proporcionada por Django. Aprenderás más sobre las vistas basadas en clases en el *Capítulo 10* , *Creación de una plataforma de aprendizaje electrónico* , y capítulos sucesivos.

Resumen

En este capítulo, aprendió los conceptos básicos del marco web de Django creando una sencilla aplicación de blog. Diseñaste los modelos de datos y aplicaste las migraciones a su proyecto. También creó las vistas, las plantillas y las URL para su blog, incluida la paginación de objetos.

En el siguiente capítulo, descubrirá cómo mejorar su aplicación de blog con un sistema de comentarios y funcionalidad de etiquetado, y cómo permitir que sus usuarios compartir publicaciones por correo electrónico.

2

Mejorando su blog con Características avanzadas

En el capítulo anterior, creó una aplicación de blog básica. A continuación, girará su aplicación en un blog completamente funcional con las funcionalidades avanzadas

que ofrecen muchos blogs hoy en día. Implementarás las siguientes características en tu blog:

- **Compartir publicaciones por correo electrónico** : cuando a los lectores les gusta un artículo, es posible que quieran compartirlo con alguien más. Implementarás la funcionalidad para compartir publicaciones por correo electrónico.
- **Agregar comentarios a una publicación** : muchas personas quieren permitir que su audiencia para comentar publicaciones y crear debates. Dejarás que tus lectores agreguen comentarios a las publicaciones de su blog.
- **Etiquetado de publicaciones** : las etiquetas le permiten categorizar el contenido de forma no jerárquica. manera, usando palabras clave simples. Implementarás un sistema de etiquetado, que es una característica muy popular para los blogs.
- **Recomendar publicaciones similares** : una vez que tenga un método de clasificación en su lugar, como un sistema de etiquetado, puede utilizarlo para proporcionar contenido recomendaciones a sus lectores. Construirá un sistema que recomienda otras publicaciones que comparten etiquetas con una determinada publicación de blog.

Estas funcionalidades convertirán su aplicación en un blog con todas las funciones.

[39]

Mejora tu blog con funciones avanzadas

En este capítulo, cubriremos los siguientes temas:

- Envío de correos electrónicos con Django
- Crear formularios y manejarlos en vistas.
- Crear formularios a partir de modelos.
- Integración de aplicaciones de terceros
- Creación de QuerySets complejos

Compartir publicaciones por correo electrónico

Primero, permitamos que los usuarios compartan publicaciones enviándolas por correo electrónico. Tómase un minuto para Piense en cómo podría usar *vistas* , *URL* y *plantillas* para crear esta funcionalidad. utilizando lo que aprendió en el capítulo anterior. Para permitir que sus usuarios compartir publicaciones por correo electrónico, deberá hacer lo siguiente:

- Cree un formulario para que los usuarios completen su nombre, su correo electrónico, el destinatario del correo electrónico, y comentarios opcionales
- Cree una vista en el archivo `views.py` que maneje los datos publicados y envíe el correo electrónico
- Agregue un patrón de URL para la nueva vista en el archivo `urls.py` del blog. solicitud
- Cree una plantilla para mostrar el formulario

Creando formularios con Django

Comencemos por crear el formulario para compartir publicaciones. Django tiene un marco de formularios incorporado que le permite crear formularios de una manera sencilla. El marco de formularios lo hace simple para definir los campos de su formulario, especificar cómo deben mostrarse y Indicar cómo deben validar los datos de entrada. El framework de formularios de Django ofrece una forma flexible de representar formularios y manejar datos.

Django viene con dos clases base para construir formularios:

- Formulario : le permite crear formularios estándar

- **ModelForm** : le permite crear formularios vinculados a instancias de modelo

Primero, cree un archivo `forms.py` dentro del directorio de su aplicación de blog y haga se ve así:

desde formularios de importación de django

[40]

```
class EmailPostForm (formularios.Form):
    nombre = formularios.CharField (longitud_máxima = 25)
    correo electrónico = formularios.EmailField ()
    to = forms.EmailField ()
    comentarios = formularios.CharField (obligatorio = Falso,
                                         widget = formularios.Textarea)
```

Esta es su primera forma de Django. Eche un vistazo al código. Has creado un formulario heredando la clase Form base . Usas diferentes tipos de campos para que Django valide campos en consecuencia.

Los formularios pueden residir en cualquier lugar de su proyecto Django. La Convención es colocarlos dentro de un archivo `forms.py` para cada aplicación.

El campo de nombre es `CharField` . Este tipo de campo se representa como `<input type = "texto">` elemento HTML. Cada tipo de campo tiene un widget predeterminado que determina cómo se representa el campo en HTML. El widget predeterminado se puede anular con el atributo de `widget` . En el campo de comentarios , usa un widget de `Textarea` para mostrarlo como un elemento HTML `<textarea>` en lugar del elemento `<input>` predeterminado .

La validación de campo también depende del tipo de campo. Por ejemplo, el correo electrónico y para Los campos son campos de `EmailField` . Ambos campos requieren una dirección de correo electrónico válida; el campo de lo contrario, la validación generará una excepción de `form.ValidationError` y el formulario no validará. También se tienen en cuenta otros parámetros para la validación del formulario: define una longitud máxima de 25 caracteres para el campo de nombre y hace que campo de comentarios opcional con `required = False` . Todo esto también se tiene en cuenta para validación de campo. Los tipos de campo utilizados en este formulario son solo una parte de Django campos de formulario. Para obtener una lista de todos los campos de formulario disponibles, puede visitar <https://docs.djangoproject.com/en/3.0/ref/forms/fields/>.

Manejo de formularios en vistas

Necesita crear una nueva vista que maneje el formulario y envíe un correo electrónico cuando esté enviado con éxito. Edite el archivo `views.py` de su aplicación de blog y agregue el siguiente código:

```
desde .forms importar EmailPostForm

def post_share (solicitud, post_id):
    # Recuperar publicación por id
    publicación = get_object_or_404 (Publicación, id = post_id, estado = 'publicado')
```

[41]

Mejora tu blog con funciones avanzadas

```
if request.method == 'POST':
    # Se envió el formulario
    formulario = EmailPostForm (request.POST)
    si form.is_valid ():
        # Los campos del formulario pasaron la validación
        cd = form.cleaned_data
        # ... enviar correo electrónico

más:
    formulario = EmailPostForm ()
    return render (request, 'blog / post / share.html', {'post': post,
                                                         'formulario': formulario})
```

Esta vista funciona de la siguiente manera:

- Usted define la vista `post_share` que toma el objeto de solicitud y el `post_id` variable como parámetros.
- Utiliza el acceso directo `get_object_or_404()` para recuperar la publicación por ID y asegúrese de que la publicación recuperada tenga un estado publicado .
- Utiliza la misma vista tanto para mostrar el formulario inicial como para procesar los datos enviados. Puedes diferenciar si el formulario fue enviado o no se basa en el método de solicitud y envíe el formulario mediante POST . Tú suponga que si recibe una solicitud GET , se debe mostrar un formulario vacío, y si recibe una solicitud POST , el formulario se envía y debe ser procesada. Por lo tanto, usa `request.method == 'POST'` para distinguir entre los dos escenarios.

El siguiente es el proceso para mostrar y manejar el formulario:

1. Cuando la vista se carga inicialmente con una solicitud GET , crea un formulario nuevo instancia que se utilizará para mostrar el formulario vacío en la plantilla:

```
formulario = EmailPostForm ()
```
2. El usuario completa el formulario y lo envía vía POST . Luego, crea un formulario instancia utilizando los datos enviados que están contenidos en `request.POST` :

```
if request.method == 'POST':
    # Se envió el formulario
    formulario = EmailPostForm (request.POST)
```
3. Después de esto, valida los datos enviados usando el `formulario.is_valid()` método. Este método valida los datos introducidos en el formulario y devuelve Verdadero si todos los campos contienen datos válidos. Si algún campo contiene datos no válidos, luego `is_valid()` devuelve False . Puede ver una lista de errores de validación por accediendo a `form.errors` .

[42]

4. Si el formulario no es válido, vuelva a representar el formulario en la plantilla con el datos enviados. Mostrará errores de validación en la plantilla.
5. Si el formulario es válido, recupera los datos validados accediendo al formulario.

clean_data . Este atributo es un diccionario de campos de formulario y sus valores.

Si los datos de su formulario no se validan, clean_data contendrá solo los campos válidos.

Ahora, exploremos cómo enviar correos electrónicos usando Django para armar todo.

Enviar correos electrónicos con Django

Enviar correos electrónicos con Django es bastante sencillo. Primero, necesitas tener un servidor de **Protocolo simple de transferencia de correo (SMTP)** local , o debe definir la configuración de un servidor SMTP externo agregando las siguientes configuraciones al settings.py archivo de su proyecto:

- EMAIL_HOST : el host del servidor SMTP; el predeterminado es localhost
- EMAIL_PORT : el puerto SMTP; el valor predeterminado es 25
- EMAIL_HOST_USER : el nombre de usuario del servidor SMTP
- EMAIL_HOST_PASSWORD : la contraseña del servidor SMTP
- EMAIL_USE_TLS : si se debe utilizar una **seguridad de la capa de transporte (TLS)** segura conexión
- EMAIL_USE_SSL : si se debe utilizar una conexión segura TLS implícita

Si no puede usar un servidor SMTP, puede decirle a Django que escriba correos electrónicos en la consola agregando la siguiente configuración al archivo settings.py :

```
EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'
```

Al usar esta configuración, Django enviará todos los correos electrónicos al shell. Esto es muy útil para probar su aplicación sin un servidor SMTP.

Si desea enviar correos electrónicos pero no tiene un servidor SMTP local, puede probablemente use el servidor SMTP de su proveedor de servicios de correo electrónico. La siguiente muestra La configuración es válida para enviar correos electrónicos a través de servidores de Gmail utilizando una cuenta de Google:

```
EMAIL_HOST = 'smtp.gmail.com'
EMAIL_HOST_USER = 'your_account@gmail.com'
EMAIL_HOST_PASSWORD = 'su_contraseña'
```

[43]

Mejora tu blog con funciones avanzadas

```
EMAIL_PORT = 587
EMAIL_USE_TLS = Verdadero
```

Ejecute el comando de shell python manage.py para abrir el shell de Python y enviar un correo electrónico, de la siguiente manera:

```
>>> de django.core.mail importar send_mail

>>> send_mail ('Correo de Django', 'Este correo electrónico fue enviado con Django.', 'your_
account@gmail.com ', [' your_account@gmail.com '], fail_silently = False)
```

La función send_mail () toma el asunto, el mensaje, el remitente y la lista de destinatarios como argumentos requeridos. Al establecer el argumento opcional fail_silently = False , le está diciendo que genere una excepción si el correo electrónico no se pudo enviar correctamente. Si el la salida que ve es 1 , entonces su correo electrónico se envió correctamente.

Si envía correos electrónicos mediante Gmail con la configuración anterior, tiene que habilitar el acceso para aplicaciones menos seguras en <https://myaccount.google.com/lesssecureapps>, como sigue:

Figura 2.1: La pantalla de acceso a la aplicación menos segura de Google

En algunos casos, es posible que también deba deshabilitar el captcha de Gmail en <https://cuentas.google.com/displayunlockcaptcha> para enviar correos electrónicos con Django.

Edite la vista `post_share` en el archivo `views.py` de la aplicación de blog, de la siguiente manera:

```
desde django.core.mail importar send_mail

def post_share (solicitud, post_id):
    # Recuperar publicación por id
    publicación = get_object_or_404 (Publicación, id = post_id, estado = 'publicado')
    enviado = falso

    if request.method == 'POST':
        # Se envió el formulario
        formulario = EmailPostForm (request.POST)
        si form.is_valid ():
```

[44]

Capítulo 2

```
# Los campos del formulario pasaron la validación
cd = form.cleaned_data
post_url = request.build_absolute_uri (
    post.get_absolute_url ()
subject = f "{cd ['nombre']} te recomienda leer" \
    f "{post.title}"
message = f "Leer {post.title} en {post_url} \n \n"
    f "Comentarios de {cd ['nombre']} \: {cd ['comentarios']}"
send_mail (asunto, mensaje, 'admin@myblog.com',
    [cd ['a']])
enviado = Verdadero

más:
    formulario = EmailPostForm ()
return render (request, 'blog / post / share.html', {'post': post,
                                                    'formulario': formulario,
                                                    'enviado': enviado})
```

Reemplace `admin@myblog.com` con su cuenta de correo electrónico real si está utilizando un SMTP servidor en lugar de la consola `EmailBackend`.

En el código anterior, declara una variable `enviado` y la establece en `True` cuando la publicación fue expedido. Utilizará esa variable más adelante en la plantilla para mostrar un mensaje de éxito cuando el formulario se envía correctamente.

Como tiene que incluir un enlace a la publicación en el correo electrónico, recupera el valor absoluto ruta de la publicación usando su método `get_absolute_url ()`. Usas este camino como un entrada para `request.build_absolute_uri ()` para construir una URL completa, incluida la Esquema HTTP y nombre de host. Construyes el asunto y el cuerpo del mensaje del correo electrónico utilizando los datos limpios del formulario validado y, finalmente, envíe el correo electrónico al dirección de correo electrónico contenida en el campo `para` del formulario.

Ahora que su vista está completa, recuerde agregarle un nuevo patrón de URL. Abre el `urls.py` de su aplicación de blog y agregue el patrón de URL `post_share`, de la siguiente manera:

```
urlpatterns = [
    # ...
```

```

    ruta ('<int: post_id> / share /',
         views.post_share, name = 'post_share'),
]

```

Representación de formularios en plantillas

Después de crear el formulario, programar la vista y agregar el patrón de URL, solo falta la plantilla para esta vista. Crea un nuevo archivo en el blog / templates / blog / post / directorio y asígnele el nombre share.html . Agregue el siguiente código:

```
{% extiende "blog / base.html"%}
```

[45]

Mejora tu blog con funciones avanzadas

```

{% block title%} Comparte una publicación {% endblock%}

{% block content%}
    {% si se envía%}
        <h1> Correo electrónico enviado correctamente </h1>
        <p>
            "{{post.title}}" se envió correctamente a {{form.cleaned_
data.to}}.
        </p>
    {% else%}
        <h1> Comparte "{{post.title}}" por correo electrónico </h1>
        <método de formulario = "publicar">
            {{form.as_p}}
            {% csrf_token%}
            <input type = "submit" value = "Enviar correo electrónico">
        </form>
    {% terminara si %}
{% endblock%}

```

Esta es la plantilla para mostrar el formulario o un mensaje de éxito cuando se envía. Como tu notará, crea el elemento de formulario HTML, lo que indica que debe enviarse por el método POST :

```
<método de formulario = "publicar">
```

Luego, incluye la instancia de formulario real. Le dice a Django que renderice sus campos en Elementos de párrafo HTML <p> con el método as_p . También puede renderizar el formulario como una lista desordenada con as_ul o como una tabla HTML con as_table . Si tu quieres para representar cada campo, puede iterar a través de los campos, {{form.as_p}} como en el siguiente ejemplo:

```

{% para el campo en el formulario%}
    <div>
        {{field.errors}}
        {{field.label_tag}} {{field}}
    </div>
{% endfor%}

```

La etiqueta de plantilla {% csrf_token%} introduce un campo oculto con un token autogenerado para evitar ataques de **falsificación de solicitudes entre sitios (CSRF)** . Estas Los ataques consisten en un sitio web o programa malicioso que realiza una acción no deseada. para un usuario en su sitio. Puede encontrar más información sobre esto en <https://owasp.org/www-comunidad/ataques/csrf> .

La etiqueta anterior genera un campo oculto que se ve así:

```
<tipo de entrada = nombre 'oculto' = 'csrfmiddlewaretoken' valor = '26JjKo2lcEtYkGo
V9z4XmjJEHLXN5LDR' />
```

De forma predeterminada, Django busca el token CSRF en todas las solicitudes POST. Recuerde incluir la etiqueta `csrf_token` en todos los formularios que estén enviados a través de POST.

Edite la plantilla `blog / post / detail.html` y agregue el siguiente enlace al compartir la URL de la publicación después de la variable `{{post.body | linebreaks}}` :

```
<p>
  <a href="{% url 'blog:post_share' post.id %}">
    Compartir esta publicacion
  </a>
</p>
```

Recuerde que está creando la URL de forma dinámica con la plantilla `{% url %}` etiqueta proporcionada por Django. Estás usando el espacio de nombres llamado `blog` y la URL llamado `post_share`, y está pasando el ID de la publicación como un parámetro para construir el URL absoluta.

Ahora, inicie el servidor de desarrollo con `python manage.py runserver` y abra `http://127.0.0.1:8000/blog/` en su navegador. Haga clic en cualquier título de la publicación para ver su página de detalles. Debajo del cuerpo de la publicación, debería ver el enlace que que acaba de agregar, como se muestra en la siguiente captura de pantalla:

Figura 2.2: La página de detalles de la publicación, incluido un enlace para compartir la publicación

Haga clic en **Compartir esta publicación** y debería ver la página, incluido el formulario para compartir esta publicación por correo electrónico, de la siguiente manera:

Figura 2.3: La página para compartir una publicación por correo electrónico

Los estilos CSS para el formulario se incluyen en el código de ejemplo en `static / css / blog.css` expediente. Cuando hace clic en el botón **ENVIAR CORREO ELECTRÓNICO**, se envía el formulario y validado. Si todos los campos contienen datos válidos, recibirá un mensaje de éxito, como se indica a continuación:

Figura 2.4: Un mensaje de éxito para una publicación compartida por correo electrónico

Si ingresa datos no válidos, el formulario se vuelve a procesar, incluidos todos los errores de validación:

Figura 2.5: El formulario de publicación compartida que muestra errores de datos no válidos

Tenga en cuenta que algunos navegadores modernos evitarán que envíe un formulario con campos vacíos o erróneos. Esto se debe a la validación del formulario realizada por el navegador, según los tipos de campo y las restricciones por campo. En este caso, el formulario no será enviado y el navegador mostrará un mensaje de error para los campos que son incorrecto.

Su formulario para compartir publicaciones por correo electrónico ya está completo. Ahora creemos un comentario sistema para tu blog.

[49]

Mejora tu blog con funciones avanzadas

Creando un sistema de comentarios

Construirá un sistema de comentarios en el que los usuarios podrán comentar las publicaciones. Para crear el sistema de comentarios, debe hacer lo siguiente:

1. Crea un modelo para guardar comentarios
2. Cree un formulario para enviar comentarios y validar los datos de entrada.
3. Agregue una vista que procese el formulario y guarde un nuevo comentario en el base de datos
4. Edite la plantilla de detalles de la publicación para mostrar la lista de comentarios y el formulario para agregar un nuevo comentario

Construyendo un modelo

Primero, construyamos un modelo para almacenar comentarios. Abra el archivo `models.py` de su blog aplicación y agregue el siguiente código:

```
comentario de clase (modelos.Modelo):
    post = modelos.ForeignKey (Publicar,
                                on_delete = modelos.CASCADE,
                                related_name = 'comentarios')

    nombre = modelos.CharField (longitud_máxima = 80)
    correo electrónico = modelos.EmailField ()
    cuerpo = modelos.TextField ()
    creado = modelos.DateTimeField (auto_now_add = True)
    actualizado = modelos.DateTimeField (auto_now = True)
    active = modelos.BooleanField (predeterminado = verdadero)

    clase Meta:
        ordering = ('creado',)
```

```
def __str__(yo):
    return f'Comentario de {self.name} en {self.post} '
```

Este es su modelo de comentarios . Contiene una ForeignKey para asociar un comentario con un publicación única. Esta relación de varios a uno se define en el modelo de comentario porque cada comentario se hará en una publicación y cada publicación puede tener varios comentarios.

El atributo related_name le permite nombrar el atributo que usa para la relación entre el objeto relacionado y este. Después de definir esto, puede recuperar la publicación de un objeto de comentario usando comment.post y recuperar todo comentarios de una publicación utilizando post.comments.all() . Si no define el related_ atributo de nombre , Django usará el nombre del modelo en minúsculas, seguido de _ set (es decir, comment_set) para nombrar la relación del objeto relacionado con el objeto del modelo, donde se ha definido esta relación.

[50]

Puede obtener más información sobre las relaciones de varios a uno en https://docs.djangoproject.com/en/3.0/topics/db/examples/many_to_one/.

Ha incluido un campo booleano activo que utilizará para desactivar manualmente comentarios inapropiados. Utiliza el campo creado para ordenar los comentarios en un orden cronológico por defecto.

El nuevo modelo de comentario que acaba de crear aún no está sincronizado con el base de datos. Ejecute el siguiente comando para generar una nueva migración que refleje la creación del nuevo modelo:

blog de makemigrations de python manage.py

Debería ver el siguiente resultado:

Migraciones para 'blog':

```
blog / migrations / 0002_comment.py
- Crear comentario modelo
```

Django ha generado un archivo 0002_comment.py dentro del directorio migrations / de la aplicación de blog . Ahora, necesita crear el esquema de base de datos relacionado y aplicar los cambios a la base de datos. Ejecute el siguiente comando para aplicar existente migraciones:

python manage.py migrar

Otendrá una salida que incluye la siguiente línea:

Aplicando blog.0002_comment ... OK

Se aplicó la migración que acaba de crear; ahora una tabla blog_comment existe en la base de datos.

A continuación, puede agregar su nuevo modelo al sitio de administración para administrar comentarios a través de una interfaz sencilla. Abra el archivo admin.py del blog , importe el modelo de comentario y agregue la siguiente clase ModelAdmin :

```
de .models importar Publicar, Comentar

@ admin.register (comentario)
class CommentAdmin (admin.ModelAdmin):
    list_display = ('nombre', 'correo electrónico', 'publicación', 'creado', 'activo')
    list_filter = ('activo', 'creado', 'actualizado')
    search_fields = ('nombre', 'correo electrónico', 'cuerpo')
```

Mejora tu blog con funciones avanzadas

Inicie el servidor de desarrollo con el comando `python manage.py runserver` y abra `http://127.0.0.1:8000/admin/` en su navegador. Deberías ver el nuevo modelo incluido en la sección **BLOG**, como se muestra en la siguiente captura de pantalla:

Figura 2.6: Modelos de aplicaciones de blogs en la página de índice de administración de Django

El modelo ahora está registrado en el sitio de administración y puede administrar Comentarios instancias usando una interfaz simple.

Creando formas a partir de modelos

Aún necesita crear un formulario para permitir que sus usuarios comenten en las publicaciones del blog. Recuerda que Django tiene dos clases base para construir formularios: `Form` y `ModelForm`. Usaste el primero anteriormente para permitir a sus usuarios compartir publicaciones por correo electrónico. En el caso presente, necesitará usar `ModelForm` porque tiene que construir un formulario dinámicamente de su modelo de comentario. Edite el archivo `forms.py` de su aplicación de blog y agregue las siguientes líneas:

```
de .models importar comentario

clase CommentForm (formularios.ModelForm):
    clase Meta:
        modelo = Comentario
        campos = ('nombre', 'correo electrónico', 'cuerpo')
```

Para crear un formulario a partir de un modelo, solo necesita indicar qué modelo usar para construir el formulario en la clase `Meta` del formulario. Django realiza una introspección del modelo y crea el forma dinámicamente para usted.

Cada tipo de campo de modelo tiene un tipo de campo de formulario predeterminado correspondiente. La forma que que defina los campos de su modelo se tiene en cuenta para la validación del formulario. Por defecto, Django crea un campo de formulario para cada campo contenido en el modelo. Sin embargo, puedes dígale explícitamente al marco qué campos desea incluir en su formulario usando una lista de campos, o defina qué campos desea excluir utilizando una lista de exclusión de campos. Para su formulario `CommentForm`, solo usará el nombre, el correo electrónico y el cuerpo campos, porque esos son los únicos campos que sus usuarios podrán completar.

Manejo de ModelForms en vistas

Utilizará la vista de detalles de la publicación para crear una instancia del formulario y procesarlo, en orden para que sea sencillo. Edite el archivo `views.py`, agregue importaciones para el modelo de comentario y la `CommentForm` forma, y modificar el `post_detail` fin de hacer que parezca que la siguiendo:

```

de .models importar Publicar, Comentar
desde .forms importar EmailPostForm, CommentForm

def post_detail (solicitud, año, mes, día, publicación):
    post = get_object_or_404 (Publicar, slug = publicar,
                                status = 'publicado',
                                Publish__year = año,
                                publish__month = mes,
                                Publish__day = día)

    # Lista de comentarios activos para esta publicación
    comentarios = post.comments.filter (activo = Verdadero)

    new_comment = Ninguno

    if request.method == 'POST':
        # Se publicó un comentario
        comment_form = CommentForm (datos = solicitud.POST)
        si comment_form.is_valid ():
            # Crear objeto de comentario pero no guardarlo en la base de datos todavía
            new_comment = comment_form.save (commit = False)
            # Asignar la publicación actual al comentario
            new_comment.post = publicación
            # Guarda el comentario en la base de datos
            new_comment.save ()

    más:
        comment_form = CommentForm ()

    return render (solicitud,
                    'blog / post / detail.html',
                    {'publicación': publicación,
                    'comentarios': comentarios,
                    'nuevo_comment': nuevo_comment,
                    'formulario_comentario': formulario_comentario })

```

Revisemos lo que ha agregado a su vista. Utilizó el `post_detail` vista para mostrar la publicación y sus comentarios. Agregó un `QuerySet` para recuperar todos los activos comentarios para esta publicación, de la siguiente manera:

```
comentarios = post.comments.filter (activo = Verdadero)
```

[53]

Construye este `QuerySet`, comenzando desde el objeto de publicación. En lugar de construir un `QuerySet` para el modelo de comentarios directamente, aprovecha el objeto de publicación para recuperar el objetos de comentario relacionados. Utiliza el administrador para los objetos relacionados que definió como comentarios usando el atributo `related_name` de la relación en el comentario modelo. Utiliza la misma vista para permitir que sus usuarios agreguen un nuevo comentario. Tu inicializas la variable `new_comment` configurándola en `None`. Utilizará esta variable cuando un se crea un nuevo comentario.

Construye una instancia de formulario con `comment_form = CommentForm ()` si la vista es llamado por una solicitud `GET`. Si la solicitud se realiza a través de `POST`, crea una instancia del formulario usando los datos enviados y validarlos usando el método `is_valid ()`. Si la forma no es válido, representa la plantilla con los errores de validación. Si el formulario es válido,

tomar las siguientes acciones:

1. Creas un nuevo objeto Comment llamando al método save () del formulario y asígnalo a la variable new_comment , de la siguiente manera:

```
new_comment = comment_form.save (commit = False)
```

El método save () crea una instancia del modelo al que está vinculado el formulario y lo guarda en la base de datos. Si lo llama usando commit = False , crea la instancia del modelo, pero no la guarde todavía en la base de datos. Esto viene muy bien cuando desee modificar el objeto antes de finalmente guardarlo, que es lo que harás a continuación.

El método save () está disponible para ModelForm pero no para las instancias de formulario, ya que no están vinculadas a ningún modelo.

2. Asignas la publicación actual al comentario que acabas de crear:

```
new_comment.post = publicación
```

Al hacer esto, especificas que el nuevo comentario pertenece a esta publicación.

3. Finalmente, guarde el nuevo comentario en la base de datos llamando a su save () método:

```
new_comment.save ()
```

Su vista ahora está lista para mostrar y procesar nuevos comentarios.

Agregar comentarios a la plantilla de detalles de la publicación

Ha creado la funcionalidad para administrar los comentarios de una publicación. Ahora necesitas para adaptar su plantilla post / detail.html para hacer lo siguiente:

- Muestra el número total de comentarios de una publicación.

[54]

- Mostrar la lista de comentarios
- Mostrar un formulario para que los usuarios agreguen un nuevo comentario.

Primero, agregará el total de comentarios. Abra la plantilla post / detail.html y agregue el siguiente código al bloque de contenido :

```
{% with comments.count as total_comments%}
<h2>
  {{total_comments}} comentario {{total_comments | pluralize}}
</h2>
{% terminar con %}
```

Está utilizando el ORM de Django en la plantilla, ejecutando los comentarios de QuerySet . contar () . Tenga en cuenta que el lenguaje de la plantilla de Django no usa paréntesis para llamar métodos. La etiqueta {% with%} le permite asignar un valor a una nueva variable que estará disponible para su uso hasta la etiqueta {% endwith%} .

La etiqueta de plantilla {% with%} es útil para evitar presionar la base de datos o acceder a métodos costosos varias veces.

Utiliza el filtro de plantilla de pluralizar para mostrar un sufijo plural para la palabra "comentario", según el valor de total_comments . Los filtros de plantilla toman el valor de la variable a la que se aplican como entrada y devuelven un valor calculado. Nosotros analizaremos los filtros de plantilla en el *Capítulo 3 , Ampliación de la aplicación de blog* .

El filtro de plantilla de pluralizar devuelve una cadena con la letra "s" si el valor es diferente de 1. El texto anterior se representará como *0 comentarios*, *1 comentario* o *N comentarios*. Django incluye muchas etiquetas de plantilla y filtros que pueden ayudarlo para mostrar la información de la forma que desee.

Ahora, incluyamos la lista de comentarios. Agregue las siguientes líneas a la publicación / detail.html plantilla debajo del código anterior:

```
{% por comentario en comentarios%}
<div class = "comentario">
  <p class = "info">
    Comentario {{forloop.counter}} de {{comment.name}}
    {{comment.created}}
  </p>
  {{comentario.cuerpo | saltos de línea}}
</div>
{% vacio%}
<p> Aún no hay comentarios. </p>
{% endfor%}
```

[55]

Mejora tu blog con funciones avanzadas

Utiliza la etiqueta de plantilla {% for%} para recorrer los comentarios. Muestra un mensaje predeterminado si la lista de comentarios está vacía, informando a sus usuarios que no hay comentarios sobre esta publicación todavía. Enumeras los comentarios con {{forloop.counter}} variable, que contiene el contador de bucle en cada iteración. Luego, muestra el nombre del usuario que publicó el comentario, la fecha y el cuerpo del comentario.

Finalmente, necesita renderizar el formulario o mostrar un mensaje de éxito en su lugar cuando esté enviado con éxito. Agregue las siguientes líneas justo debajo del código anterior:

```
{% if new_comment%}
  <h2> Su comentario ha sido agregado. </h2>
{% else%}
  <h2> Agregar un comentario nuevo </h2>
  <método de formulario = "publicar">
    {{comment_form.as_p}}
    {% csrf_token%}
    <p> <input type = "submit" value = "Agregar comentario"> </p>
  </form>
{% terminara si %}
```

El código es bastante sencillo: si el objeto new_comment existe, muestra un mensaje de éxito porque el comentario se creó correctamente. De lo contrario, tu renderizar el formulario con un párrafo, <p>, elemento para cada campo e incluir el CSRF token necesario para las solicitudes POST.

Abra <http://127.0.0.1:8000/blog/> en su navegador y haga clic en el título de una publicación para echa un vistazo a su página de detalles. Verá algo como la siguiente captura de pantalla:

Capítulo 2

Agregue un par de comentarios usando el formulario. Deberían aparecer debajo de tu publicación en orden cronológico, como sigue:

Figura 2.8: La lista de comentarios en la página de detalles de la publicación

Abra <http://127.0.0.1:8000/admin/blog/comment/> en su navegador. Tú verá la página de administración con la lista de comentarios que creó. Haga clic en el nombre de uno de ellos para editarlo, desmarque la casilla **Activo** y haga clic en el Botón **Guardar** . Será redirigido a la lista de comentarios nuevamente, y el **ACTIVO** La columna mostrará un icono inactivo para el comentario. Debería verse como el primero comentar en la siguiente captura de pantalla:

Figura 2.9: Comentarios activos / inactivos en el sitio de administración de Django

Si regresa a la vista de detalles de la publicación, notará que el comentario inactivo no es mostrado más; tampoco se cuenta para el número total de comentarios. Gracias al campo activo , puede desactivar los comentarios inapropiados y evitar mostrar ellos en sus publicaciones.

Añadiendo la funcionalidad de etiquetado

Después de implementar su sistema de comentarios, debe crear una forma de etiquetar su puestos. Hará esto integrando una aplicación de etiquetado Django de terceros en tu proyecto. django-taggit es una aplicación reutilizable que ofrece principalmente una

Modelo de etiquetas y administrador para agregar etiquetas fácilmente a cualquier modelo. Puedes echar un vistazo a su código fuente en <https://github.com/jazzband/django-taggit>.

Primero, necesita instalar django-taggit a través de pip ejecutando lo siguiente mando:

```
pip install django_taggit == 1.2.0
```

Luego, abra el archivo settings.py del proyecto mysite y agregue taggit a su

Configuración INSTALLED_APPS , de la siguiente manera:

```
INSTALLED_APPS = [  
    # ...  
    'blog.apps.BlogConfig',  
    'taggit',  
]
```

Abra el archivo models.py de su aplicación de blog y agregue el TaggableManager administrador proporcionado por django-taggit al modelo Post usando el siguiente código:

```
de taggit.managers import TaggableManager  
  
clase Post (modelos.Modelo):  
    # ...  
    tags = TaggableManager ()
```

El administrador de etiquetas le permitirá agregar, recuperar y eliminar etiquetas de la publicación objetos.

Ejecute el siguiente comando para crear una migración para los cambios de su modelo:

```
blog de makemigrations de python manage.py
```

Debería obtener el siguiente resultado:

Migraciones para 'blog':

```
blog / migrations / 0003_post_tags.py  
- Agregar etiquetas de campo para publicar
```

[58]

Ahora, ejecute el siguiente comando para crear las tablas de base de datos necesarias para django-modelos taggit y para sincronizar los cambios de su modelo:

```
python manage.py migrar
```

Verá un resultado que indica que se han aplicado las migraciones, de la siguiente manera:

```
Aplicando taggit.0001_initial ... OK
```

```
Aplicando taggit.0002_auto_20150616_2121 ... OK
```

Aplicando taggit.0003_taggeditem_add_unique_index ... OK

Aplicando blog.0003_post_tags ... OK

Su base de datos ahora está lista para usar los modelos django-taggit .

Exploremos cómo usar el administrador de etiquetas . Abre la terminal con python manage.py shell comando e ingrese el siguiente código. Primero, recuperará una de tus publicaciones (la que tiene el ID 1):

```
>>> de blog.models importar Publicación
```

```
>>> publicación = Post.objects.get (id = 1)
```

Luego, agregue algunas etiquetas y recupere sus etiquetas para verificar si fueron Agregado exitosamente:

```
>>> post.tags.add ('música', 'jazz', 'django')
```

```
>>> post.tags.all ()
```

```
<QuerySet [<Etiqueta: jazz>, <Etiqueta: música>, <Etiqueta: django>]>
```

Finalmente, elimine una etiqueta y revise la lista de etiquetas nuevamente:

```
>>> post.tags.remove ('django')
```

```
>>> post.tags.all ()
```

```
<QuerySet [<Etiqueta: jazz>, <Etiqueta: música>]>
```

Eso fue fácil, ¿verdad? Ejecute el comando python manage.py runserver para iniciar el servidor de desarrollo de nuevo y abra <http://127.0.0.1:8000/admin/taggit/tag/> en su navegador.

[59]

Mejora tu blog con funciones avanzadas

Verá la página de administración con la lista de objetos de etiqueta del taggit solicitud:

Figura 2.10: Vista de la lista de cambios de etiquetas en el sitio de administración de Django

Vaya a <http://127.0.0.1:8000/admin/blog/post/> y haga clic en una publicación para editarlo. Verá que las publicaciones ahora incluyen un nuevo campo **Etiquetas** , como se muestra a continuación, donde puede editar etiquetas fácilmente:

Figura 2.11: El campo de etiquetas relacionadas de un objeto de publicación

Ahora, debe editar las publicaciones de su blog para mostrar etiquetas. Abra el blog / publicación / lista.html y agregue el siguiente código HTML debajo del título de la publicación:

```
<p class = "tags"> Etiquetas: {{post.tags.all | join: ", "}} </p>
```

El filtro de plantilla de combinación funciona igual que el método join () de cadena de Python para concatenar elementos con la cadena dada. Abra <http://127.0.0.1:8000/blog/> en su navegador. Debería poder ver la lista de etiquetas debajo del título de cada publicación:

Figura 2.12: El elemento de la lista Publicar, incluidas las etiquetas relacionadas

[60]

Capítulo 2

A continuación, editará la vista post_list para permitir que los usuarios enumeren todas las publicaciones etiquetadas con un etiqueta específica. Abra el archivo views.py de su aplicación de blog , importe el modelo de etiqueta form django-taggit , y cambie la vista post_list para filtrar opcionalmente las publicaciones por una etiqueta, como sigue:

de taggit.models etiqueta de importación

```
def post_list (solicitud, tag_slug = Ninguno ):
    object_list = Publicar.publicado.todos ()
    tag = Ninguno

    si tag_slug:
        tag = get_object_or_404 (Etiqueta, slug = tag_slug)
        object_list = object_list.filter (tags__in = [etiqueta])

    paginador = Paginator (object_list, 3) # 3 publicaciones en cada página
    # ...
```

La vista post_list ahora funciona de la siguiente manera:

1. Toma un parámetro tag_slug opcional que tiene un valor predeterminado Ninguno . Este parámetro se pasará en la URL.
2. Dentro de la vista, crea el QuerySet inicial y recupera todos los publicaciones, y si hay una etiqueta determinada, obtienes el objeto Etiqueta con la etiqueta dada slug usando el atajo get_object_or_404 () .
3. Luego, filtra la lista de publicaciones por aquellas que contienen la etiqueta dada. Dado que esta es una relación de muchos a muchos, debe filtrar las publicaciones por etiquetas contenidas en una lista determinada, que, en su caso, contiene solo una elemento. Utiliza la búsqueda de campo __in . Se producen relaciones de muchos a muchos cuando varios objetos de un modelo están asociados con varios objetos de otro modelo. En su aplicación, una publicación puede tener varias etiquetas y una La etiqueta puede estar relacionada con varias publicaciones. Aprenderá a crear muchos relaciones con muchos en el *Capítulo 5 , Compartir contenido en su sitio web* . Tú puede descubrir más sobre las relaciones de varios a varios en https://docs.djangoproject.com/en/3.0/topics/db/examples/many_to_many/.

Recuerde que los QuerySets son perezosos. Los QuerySets para recuperar publicaciones solo serán evaluado cuando recorre la lista de publicaciones al representar la plantilla.

Finalmente, modifique la función render () en la parte inferior de la vista para pasar la etiqueta

variable a la plantilla. La vista debería verse así:

```
def post_list(solicitud, tag_slug = Ninguno):
    object_list = Publicar.publicado.todos()
    tag = Ninguno
```

[61]

Página 87

Mejora tu blog con funciones avanzadas

```
si tag_slug:
    tag = get_object_or_404(Etiqueta, slug = tag_slug)
    object_list = object_list.filter(tags__in = [etiqueta])

paginator = Paginator(object_list, 3) # 3 publicaciones en cada página
page = request.GET.get('página')
tratar:
    posts = paginator.page(página)
excepto PageNotAnInteger:
    # Si la página no es un número entero, entregue la primera página
    publicaciones = paginator.page(1)
excepto EmptyPage:
    # Si la página está fuera del rango, entregue la última página de resultados
    publicaciones = paginator.page(paginator.num_pages)
return render(solicitud, 'blog / post / list.html', {'página': página,
                                                    'publicaciones': publicaciones,
                                                    'etiqueta': etiqueta })
```

Abra el archivo urls.py de su aplicación de blog , comente el archivo basado en clases.

Patrón de URL de PostListView y descomente la vista de post_list , así:

```
ruta ("", views.post_list, name = 'post_list'),
# ruta ("", vistas.PostListView.as_view (), nombre = 'post_list'),
```

Agregue el siguiente patrón de URL adicional para enumerar las publicaciones por etiqueta:

```
ruta ('etiqueta / <slug: tag_slug> /',
      views.post_list, name = 'post_list_by_tag'),
```

Como puede ver, ambos patrones apuntan a la misma vista, pero los está nombrando diferentemente. El primer patrón llamará a la vista post_list sin ningún opcional parámetros, mientras que el segundo patrón llamará a la vista con el tag_slug parámetro. Utiliza un convertidor de ruta slug para hacer coincidir el parámetro en minúsculas cadena con letras o números ASCII, más el guión y los caracteres de subrayado.

Dado que está utilizando la vista post_list , edite la plantilla blog / post / list.html y modificar la paginación para usar el objeto posts :

```
{% incluye "pagination.html" con page = posts %}
```

Agregue las siguientes líneas encima del ciclo {% for%} :

```
{% if tag%}
<h2> Publicaciones etiquetadas con "{{tag.name}}" </h2>
{% terminara si %}
```

[62]

Si un usuario accede al blog, verá la lista de todas las publicaciones. Si filtran por publicaciones etiquetados con una etiqueta específica, verán la etiqueta por la que están filtrando.

Ahora, cambie la forma en que se muestran las etiquetas, de la siguiente manera:

```
<p class = "etiquetas">
  Etiquetas:
  {% para etiqueta en post.tags.all%}
    <a href="{% url 'blog:post_list_by_tag' tag.slug %}">
      {{tag.name}}
    </a>
    {% si no es forloop.last%, {% endif%}
  {% endfor%}
</p>
```

En el código anterior, recorre todas las etiquetas de una publicación que muestra un enlace personalizado a la URL para filtrar publicaciones por esa etiqueta. La URL se crea con {% url "blog: post_list_by_tag "tag.slug%" }, utilizando el nombre de la URL y la etiqueta slug como su parámetro. Separa las etiquetas con comas.

Abra <http://127.0.0.1:8000/blog/> en su navegador y haga clic en cualquier enlace de etiqueta. Verá la lista de publicaciones filtradas por esa etiqueta, así:

Figura 2.13: Una publicación filtrada por la etiqueta "jazz"

Recuperar publicaciones por similitud

Ahora que ha implementado el etiquetado para las publicaciones de su blog, puede hacer muchas cosas interesantes con etiquetas. Las etiquetas le permiten clasificar las publicaciones de forma no jerárquica. Las publicaciones sobre temas similares tendrán varias etiquetas en común. Tu construirás

una funcionalidad para mostrar publicaciones similares por la cantidad de etiquetas que comparten. De este modo, cuando un usuario lee una publicación, puedes sugerirle que lea otras publicaciones relacionadas.

Para recuperar publicaciones similares para una publicación específica, debe realizar la siguientes pasos:

1. Recupera todas las etiquetas de la publicación actual.
2. Obtén todas las publicaciones que están etiquetadas con cualquiera de esas etiquetas.
3. Excluya la publicación actual de esa lista para evitar recomendar la misma publicación.
4. Ordena los resultados por el número de etiquetas compartidas con la publicación actual.
5. En el caso de dos o más publicaciones con el mismo número de etiquetas, recomiende la publicación más reciente
6. Limite la consulta a la cantidad de publicaciones que desea recomendar.

Estos pasos se traducen en un QuerySet complejo que incluirá en su vista `post_detail`. nueva línea el archivo `views.py` de su aplicación de blog y agregue el siguiente importación en la parte superior:

```
from django.db.models import Count
```

Esta es la función de agregación `Count` del ORM de Django. Esta función permitirá que realice recuentos agregados de etiquetas. `django.db.models` incluye lo siguiente funciones de agregación:

- Promedio : el valor medio
- Max : el valor máximo
- Min : el valor mínimo
- Recuento : el número total de objetos

Puede obtener información sobre la agregación en <https://docs.djangoproject.com/en/3.0/temas/db/aggregation/>.

Agregue las siguientes líneas dentro de la vista `post_detail` antes de la función `render()`, con el mismo nivel de sangría:

```
# Lista de publicaciones similares
post_tags_ids = post.tags.values_list('id', flat=True)
similar_posts = Post.published.filter(tags__in=post_tags_ids) \
```

[64]

```
.exclude(id=post.id)
similares_posts = similar_posts.annotate(same_tags=Count('etiquetas')) \
    .order_by('-same_tags', '-publicar')[:4]
```

El código anterior es el siguiente:

1. Recupera una lista de Python de ID para las etiquetas de la publicación actual. Los valores_
list() QuerySet devuelve tuplas con los valores de los campos dados. Tú
pass flat=True to it para obtener valores únicos como [1, 2, 3, ...] en su lugar
de una tupla como [(1,), (2,), (3,) ...].
2. Obtienes todas las publicaciones que contienen cualquiera de estas etiquetas, excepto la publicación actual.
sí mismo.
3. Utiliza la función de agregación de recuento para generar un campo calculado: mismo_
etiquetas: contiene la cantidad de etiquetas compartidas con todas las etiquetas consultadas.
4. Ordena el resultado por el número de etiquetas compartidas (orden descendente) y
por publicar para mostrar las publicaciones recientes primero para las publicaciones con el mismo número
de etiquetas compartidas. Corta el resultado para recuperar solo las primeras cuatro publicaciones.

Agregue el objeto `similar_posts` al diccionario de contexto para la función `render()`, como sigue:

```

return render (solicitud,
               'blog / post / detail.html',
               {'publicación': publicación,
               'comentarios': comentarios,
               'nuevo_comment': nuevo_comment,
               'formulario_comentario': formulario_comentario,
               'similares_posts': similar_posts })

```

Ahora, edite la plantilla `blog / post / detail.html` y agregue el siguiente código antes la lista de comentarios de la publicación:

```

<h2> Publicaciones similares </h2>
{% para publicación en similar_posts%}
    <p>
        <a href="{{ post.get_absolute_url }}"> {{post.title}} </a>
    </p>
{% vacío%}
    Aún no hay publicaciones similares.
{% endfor%}

```

[65]

Mejora tu blog con funciones avanzadas

La página de detalles de la publicación debería verse así:

Figura 2.14: La página de detalles de la publicación, incluida una lista de publicaciones similares

Ahora puede recomendar con éxito publicaciones similares a sus usuarios. `django-taggit` también incluye un administrador de objetos similares () que puede usar para recuperar objetos por etiquetas compartidas. Puede echar un vistazo a todos los administradores de `django-taggit` en <https://django-taggit.readthedocs.io/en/latest/api.html>.

También puede agregar la lista de etiquetas a la plantilla de detalles de su publicación de la misma manera que que hiciste en la plantilla `blog / post / list.html`.

Resumen

En este capítulo, aprendió a trabajar con formularios de Django y modelos de formularios. Creó un sistema para compartir el contenido de su sitio por correo electrónico y creó un comentario sistema para tu blog. Agregó etiquetado a las publicaciones de su blog, integrando un reutilizable aplicación y construyó `QuerySets` complejos para recuperar objetos por similitud.

En el próximo capítulo, aprenderá a crear etiquetas y filtros de plantilla personalizados. También creará un mapa del sitio personalizado y un feed para las publicaciones de su blog, e implementará la función de búsqueda de texto completo para sus publicaciones.

3

Extendiendo su Aplicación de blog

El capítulo anterior pasó por los conceptos básicos de las formas y la creación de un sistema de comentarios. También aprendió a enviar correos electrónicos con Django, y implementó un sistema de etiquetado mediante la integración de una aplicación de terceros con su proyecto. En este capítulo, ampliará la aplicación de su blog con otras características populares utilizadas en plataformas de blogs. También aprenderá sobre otros componentes y funcionalidades con Django.

El capítulo cubrirá los siguientes puntos:

- **Creación de etiquetas y filtros de plantilla personalizados** : aprenderá a crear su propias etiquetas de plantilla y filtros de plantilla para explotar las capacidades de Django plantillas.
- **Agregar un mapa del sitio y un feed de publicaciones** : aprenderá a usar los mapas del sitio framework y framework de sindicación que vienen con Django.
- **Implementación de la búsqueda de texto completo con PostgreSQL** : la búsqueda es un método muy popular función para blogs. Aprenderá a implementar una búsqueda avanzada. motor para su aplicación de blog.

Ampliación de la aplicación de su blog

Crear etiquetas y filtros de plantilla personalizados

Django ofrece una variedad de etiquetas de plantilla integradas, como `{% if %}` o `{% block %}`. Usó diferentes etiquetas de plantilla en el *Capítulo 1*, *Creación de una aplicación de blog* y *Capítulo 2*, *Mejora de su blog con funciones avanzadas*. Puede encontrar una referencia completa de etiquetas de plantilla integradas y filtros en <https://docs.djangoproject.com/en/3.0/ref/templates/builtins/>.

Django también le permite crear sus propias etiquetas de plantilla para realizar acciones personalizadas. Las etiquetas de plantilla personalizadas son muy útiles cuando necesita agregar una funcionalidad a sus plantillas que no están cubiertas por el conjunto básico de etiquetas de plantilla de Django. Esto podría ser una etiqueta para realizar un QuerySet o cualquier procesamiento del lado del servidor que desee reutilizar a través de plantillas. Por ejemplo, puede crear una etiqueta de plantilla para mostrar la lista de últimas publicaciones publicadas en tu blog. Puede incluir esta lista en la barra lateral del blog para varias páginas, independientemente de la vista.

Etiquetas de plantilla personalizadas

Django proporciona las siguientes funciones de ayuda que le permiten crear sus propias etiquetas de plantilla de una manera sencilla:

- `simple_tag` : procesa los datos y devuelve una cadena
- `include_tag` : procesa los datos y devuelve una plantilla renderizada

Las etiquetas de plantilla deben vivir dentro de las aplicaciones de Django.

Dentro del directorio de su aplicación de blog, cree un nuevo directorio, asígnele un nombre `templatetags` y agregue un archivo `__init__.py` vacío. Cree otro archivo en la misma carpeta y asígnele el nombre `blog_tags.py`. La estructura de archivos de la aplicación de blog debería verse así:

```
Blog/
  __init__.py
  modelos.py
  ...
  templatetags /
    __init__.py
    blog_tags.py
```

La forma en que le asigna un nombre al archivo es importante. Usará el nombre de este módulo para cargar etiquetas en plantillas.

Comencemos creando una etiqueta simple para recuperar el total de publicaciones publicadas en el blog. Edite el archivo `blog_tags.py` que acaba de crear y agregue el siguiente código:

```
desde la plantilla de importación de django
desde ..models import Post
```

```
registrarse = template.Library()
```

```
@register.simple_tag
def total_posts():
    return Post.published.count()
```

Ha creado una etiqueta de plantilla simple que devuelve el número de publicaciones publicadas hasta aquí. Cada módulo que contiene etiquetas de plantilla necesita definir una variable llamada `registrarse` para ser una biblioteca de etiquetas válida. Esta variable es una instancia de `template.Library`, y se utiliza para registrar sus propias etiquetas y filtros de plantilla.

En el código anterior, define una etiqueta llamada `total_posts` con una función de Python y use el decorador `@register.simple_tag` para registrar la función como una etiqueta simple. Django usará el nombre de la función como nombre de etiqueta. Si quieres registrarlo usando un nombre diferente, puede hacerlo especificando un atributo de nombre, como `@register.simple_tag(nombre='my_tag')`.

Después de agregar un nuevo módulo de etiquetas de plantilla, deberá reiniciar el servidor de desarrollo de Django para usar las nuevas etiquetas y filtros en plantillas.

Antes de usar etiquetas de plantilla personalizadas, debe hacerlas disponibles para la plantilla. utilizando la etiqueta `{% load %}`. Como se mencionó anteriormente, debe usar el nombre del Módulo de Python que contiene sus etiquetas de plantilla y filtros.

Abra la plantilla `blog/templates/base.html` y agregue `{% load blog_tags %}` en la parte superior para cargar el módulo de etiquetas de plantilla. Luego, usa la etiqueta que creaste para mostrar el total de publicaciones. Simplemente agregue `{% total_posts %}` a su plantilla. los la plantilla debería verse así:

```
{% load blog_tags %}
{% load static %}
<!DOCTYPE html>
<html>
<cabeza>
  <title> {% block title %} {% endblock %} </title>
  <link href = "{% static 'css/blog.css' %}" rel = "hoja de estilo">
</head>
< cuerpo>
```

[69]

Ampliación de la aplicación de su blog

```
<div id = "contenido">
  {% block content %}
  {% endblock %}
</div>
<div id = "barra lateral">
  <h2> Mi blog </h2>
  <p> Este es mi blog. He escrito {% total_posts %} publicaciones hasta ahora. </p>
</div>
</body>
</html>
```

Deberá reiniciar el servidor para realizar un seguimiento de los nuevos archivos agregados al proyecto. Detenga el servidor de desarrollo con `Ctrl + C` y ejecútelo nuevamente usando lo siguiente mando:

```
python manage.py runserver
```

Abra `http://127.0.0.1:8000/blog/` en su navegador. Deberías ver el total número de publicaciones en la barra lateral del sitio, de la siguiente manera:

Figura 3.1: El total de publicaciones publicadas incluidas en la barra lateral

El poder de las etiquetas de plantilla personalizadas es que puede procesar cualquier dato y agregarlo a cualquier plantilla independientemente de la vista ejecutada. Puede realizar QuerySets o procesar cualquier datos para mostrar resultados en sus plantillas.

Ahora, creará otra etiqueta para mostrar las últimas publicaciones en la barra lateral de su Blog. Esta vez, utilizará una etiqueta de inclusión. Con una etiqueta de inclusión, puede representar una plantilla con variables de contexto devueltas por su etiqueta de plantilla.

Edite el archivo `blog_tags.py` y agregue el siguiente código:

```
@ register.inclusion_tag ('blog / post / latest_posts.html')
def show_latest_posts (recuento = 5):
    latest_posts = Post.published.order_by ('- publicar') [: count]
    return {'latest_posts': latest_posts}
```

En el código anterior, registra la etiqueta de la plantilla usando `@ register.inclusion_` etiqueta y especifique la plantilla que se renderizará con los valores devueltos usando `blog / post / latest_posts.html`. Su etiqueta de plantilla aceptará un recuento opcional parámetro que por defecto es 5. Este parámetro le permite especificar el número de publicaciones que desea mostrar. Utiliza esta variable para limitar los resultados de la consulta `Publicar`. `Published.order_by ('- publish') [: count]`.

[70]

Capítulo 3

Tenga en cuenta que la función devuelve un diccionario de variables en lugar de un valor simple. Las etiquetas de inclusión deben devolver un diccionario de valores, que se utiliza como contexto para renderizar la plantilla especificada. La etiqueta de plantilla que acaba de crear le permite especificar la cantidad opcional de publicaciones que se mostrarán como `{% show_latest_posts 3%}`.

Ahora, cree un nuevo archivo de plantilla en `blog / post /` y asígnele el nombre `latest_posts`.

html. Agregue el siguiente código:

```
<ul>
    {% para publicación en latest_posts%}
        <li>
            <a href="{% post.get_absolute_url %}"> {{post.title}} </a>
        </li>
    {% endfor%}
</ul>
```

En el código anterior, muestra una lista desordenada de publicaciones usando el último_

Publica la variable devuelta por la etiqueta de su plantilla. Ahora, edite el `blog / base.html` template y agregue la nueva etiqueta de plantilla para mostrar las últimas tres publicaciones. La barra lateral el código debería tener el siguiente aspecto:

```
<div id = "barra lateral">
    <h2> Mi blog </h2>
    <p> Este es mi blog. He escrito {% total_posts%} publicaciones hasta ahora. </p>
    <h3> Últimas publicaciones </h3>
    {% show_latest_posts 3%}
</div>
```

Se llama a la etiqueta de la plantilla, pasando el número de publicaciones a mostrar y la plantilla se representa en su lugar con el contexto dado.

Luego, regrese a su navegador y actualice la página. La barra lateral debería verse ahora Me gusta esto:

Ampliación de la aplicación de su blog

Finalmente, creará una etiqueta de plantilla simple que devuelva un valor. Almacenará el dan como resultado una variable que se puede reutilizar, en lugar de generarla directamente. Vas a crea una etiqueta para mostrar las publicaciones más comentadas.

Edite el archivo `blog_tags.py` y agregue la siguiente etiqueta de importación y plantilla:

```
from django.db.models import Count

@register.simple_tag
def get_most_commented_posts(recuento = 5):
    return Post.published.annotate (
        total_comments = Count ('comentarios')
    ).order_by ('- total_comments') [: count]
```

En la etiqueta de plantilla anterior, crea un `QuerySet` usando la función `annotate ()` para sumar el número total de comentarios de cada publicación. Usas el recuento función de agregación para almacenar el número de comentarios en el campo calculado `total_comments` para cada objeto de publicación . Usted ordena el `QuerySet` por el campo calculado en orden descendiente. También proporciona una variable de recuento opcional para limitar el total número de objetos devueltos.

Además de `Count` , Django ofrece las funciones de agregación `Avg` , `Max` , `Min` y `Sum` . Puede leer más sobre las funciones de agregación en <https://docs.djangoproject.com/en/3.0/topics/db/aggregation/>.

A continuación, edite la plantilla `blog / base.html` y agregue el siguiente código al elemento `<div>` de la barra lateral :

```
<h3> Publicaciones más comentadas </h3>
{% get_most_commented_posts as most_commented_posts %}
<ul>
    {% para publicación en most_commented_posts %}
        <li>
            <a href="{ post.get_absolute_url }"> {{ post.title }} </a>
        </li>
    {% endfor %}
</ul>
```

En el código anterior, almacena el resultado en una variable personalizada usando `as` argumento seguido del nombre de la variable. Para su etiqueta de plantilla, usa `{% get_most_commented_posts as most_commented_posts %}` para almacenar el resultado de la etiqueta de plantilla en una nueva variable llamada `most_commented_posts` . Entonces, muestra las publicaciones devueltas usando una lista desordenada.

Ahora abra su navegador y actualice la página para ver el resultado final. Debería verse como el siguiente:

Figura 3.3: Vista de lista de publicaciones, incluida la barra lateral completa con las publicaciones más recientes y comentadas

Ahora tiene una idea clara sobre cómo crear etiquetas de plantilla personalizadas. Puedes leer más sobre ellos en <https://docs.djangoproject.com/en/3.0/howto/custom-etiquetas-de-plantilla/>.

Filtros de plantilla personalizados

Django tiene una variedad de filtros de plantilla incorporados que le permiten alterar variables en plantillas. Estas son funciones de Python que toman uno o dos parámetros, el valor de la variable a la que se aplica el filtro y un argumento opcional. Ellos devuelven un valor que puede ser mostrado o tratado por otro filtro. Un filtro se parece a

`{{variable | mi_filtro }}`. Los filtros con un argumento se parecen a `{{variable | my_filtro: "foo" }}`. Por ejemplo, puede utilizar el filtro `capfirst` para capitalizar el primer carácter del valor, como `{{value | capfirst }}`. Si el valor es "django", el la salida será "Django". Puede aplicar tantos filtros como desee a una variable, por ejemplo, `{{variable | filter1 | filter2}}`, y cada uno de ellos se aplicará a la salida generada por el filtro anterior.

Puede encontrar la lista de filtros de plantilla integrados de Django en <https://docs.djangoproject.com/en/3.0/ref/templates/builtins/#built-in-filter-referencia>.

Crearé un filtro personalizado que le permitirá utilizar la sintaxis de rebajas en su blog. publicaciones y luego convierta el contenido de la publicación a HTML en las plantillas. Reducción es una sintaxis de formato de texto sin formato que es muy simple de usar y está destinada a ser convertido a HTML. Puede escribir publicaciones usando una sintaxis simple de rebajas y obtener el contenido se convierte automáticamente en código HTML. Aprender la sintaxis de Markdown es mucho más fácil que aprender HTML. Al usar Markdown, puede obtener otras colaboradores experimentados para escribir fácilmente publicaciones para su blog. Puede aprender los conceptos básicos del formato de rebajas en <https://daringfireball.net/projects/markdown/basics>.

Primero, instale el módulo de reducción de Python a través de pip usando el siguiente comando:

```
pip install markdown == 3.2.1
```

Luego, edite el archivo `blog_tags.py` e incluya el siguiente código:

```
de django.utils.safestring import mark_safe
descuento de importación

@ register.filter (nombre = 'markdown')
def markdown_format (texto):
    return mark_safe (markdown.markdown (texto))
```

Los filtros de plantilla se registran de la misma forma que las etiquetas de plantilla. Para evitar un nombre choque entre el nombre de su función y el módulo de rebajas, usted nombra su función `markdown_format` y nombra la rebaja del filtro para usar en plantillas, como `{% variable | markdown %}`. Django escapa del código HTML generado por filtros; los caracteres de las entidades HTML se reemplazan por sus caracteres codificados en HTML. por ejemplo, `<p>` se convierte en `<p>`; `p & gt;` (símbolo *menor que*, carácter *p*, *mayor que* símbolo). Utiliza la función `mark_safe` proporcionada por Django para marcar el resultado como HTML seguro para ser renderizado en la plantilla. Por defecto, Django no confiará en ningún Código HTML y lo escapará antes de colocarlo en la salida. Las únicas excepciones son variables que están marcadas como seguras de escapar. Este comportamiento evita que Django generar HTML potencialmente peligroso y le permite crear excepciones para devolviendo HTML seguro.

Ahora, cargue su módulo de etiquetas de plantilla en la lista de publicaciones y las plantillas de detalles. Añade el la siguiente línea en la parte superior del `blog / post / list.html` y `blog / post / detail.html` plantillas después de la etiqueta `{% extensions %}`:

```
{% load blog_tags %}
```

En la plantilla `post / detail.html`, busque la siguiente línea:

```
{{ post.body | linebreak }}
```

[74]

Reemplácelo por el siguiente:

```
{{ post.body | markdown }}
```

Luego, en la plantilla `post / list.html`, busque la siguiente línea:

```
{{ post.body | truncatewords: 30 | saltos de línea }}
```

Reemplácelo por el siguiente:

```
{{ post.body | markdown | truncatewords_html: 30 }}
```

El `truncatewords_html` filtro trunca una cadena después de un cierto número de palabras, evitando las etiquetas HTML no cerradas.

Ahora abra `http://127.0.0.1:8000/admin/blog/post/add/` en su navegador y

agregue una publicación con el siguiente cuerpo:

Esta es una publicación formateada con markdown

* Esto se enfatiza * y ** esto se enfatiza más **.

Aquí hay una lista:

- * Uno
- * Dos
- * Tres

Y un [enlace al sitio web de Django] (<https://www.djangoproject.com/>)

Abra su navegador y observe cómo se representa la publicación. Deberías ver el siguiente salida:

Figura 3.4: La publicación con contenido de rebajas representado como HTML

[75]

Ampliación de la aplicación de su blog

Como puede ver en la captura de pantalla anterior, los filtros de plantilla personalizados son muy útiles para personalizar el formato. Puede encontrar más información sobre filtros personalizados en <https://docs.djangoproject.com/en/3.0/howto/custom-template-tags/#writing-custom-template-filters>.

Agregar un mapa del sitio a su sitio

Django viene con un marco de mapa del sitio, que le permite generar mapas del sitio para su sitio de forma dinámica. Un mapa del sitio es un archivo XML que le dice a los motores de búsqueda las páginas de su sitio web, su relevancia y la frecuencia con la que se actualizan. Usando un mapa del sitio hará que su sitio sea más visible en las clasificaciones de los motores de búsqueda: los mapas del sitio ayudan a los rastreadores a indexar el contenido de su sitio web.

El marco del mapa del sitio de Django depende de `django.contrib.sites`, que permite que asocie objetos a sitios web particulares que se ejecutan con su proyecto. Esto resulta útil cuando desea ejecutar varios sitios con un solo Django proyecto. Para instalar el marco del mapa del sitio, deberá activar ambos sitios y las aplicaciones de mapas del sitio de su proyecto.

Edite el archivo `settings.py` de su proyecto y agregue `django.contrib.sites` y `django.contrib.sitemaps` a la configuración `INSTALLED_APPS`. Además, defina un nuevo configuración para el ID del sitio, de la siguiente manera:

```
SITE_ID = 1

# Definición de la aplicación
INSTALLED_APPS = [
    # ...
    'django.contrib.sites',
    'django.contrib.sitemaps',

```

]

Ahora ejecute el siguiente comando para crear las tablas de la aplicación del sitio Django en la base de datos:

```
python manage.py migrar
```

Debería ver un resultado que contiene las siguientes líneas:

```
Aplicando sitios.0001_initial ... OK
```

```
Aplicando sitios.0002_alter_domain_unique ... OK
```

La aplicación de sitios ahora está sincronizada con la base de datos.

A continuación, cree un nuevo archivo dentro del directorio de su aplicación de blog y asígnele el nombre `sitemaps.py`. Abra el archivo y agregue el siguiente código:

```
from django.contrib.sitemaps import Sitemap
```

[76]

```
from django.contrib.sitemaps import Sitemap
```

```
class PostSitemap(Sitemap):
```

```
    changefreq = 'semanal'
```

```
    priority = 0.9
```

```
    def items(self):
```

```
        return Post.published.all()
```

```
    def lastmod(self, obj):
```

```
        return obj.updated
```

Usted crea un mapa del sitio personalizado heredando la clase de mapa del sitio de los mapas del sitio. módulo. Los atributos `changefreq` y `priority` indican la frecuencia de cambio de las páginas de sus publicaciones y su relevancia en su sitio web (el valor máximo es 1).

El método `items()` devuelve el `QuerySet` de objetos para incluir en este mapa del sitio. Por predeterminado, Django llama al método `get_absolute_url()` en cada objeto para recuperar su URL. Recuerde que creó este método en el *Capítulo 1*, *Creación de un blog Aplicación*, para recuperar la URL canónica de las publicaciones. Si desea especificar la URL para cada objeto, puede agregar un método de ubicación a su clase de mapa del sitio.

El método `lastmod` recibe cada objeto devuelto por `items()` y devuelve el último hora en que se modificó el objeto.

Tanto los atributos `changefreq` como los de `priority` pueden ser métodos o atributos. Puede echar un vistazo a la referencia completa del mapa del sitio en el Django oficial. documentación ubicada en <https://docs.djangoproject.com/en/3.0/ref/contrib/sitemaps/>.

Finalmente, solo necesita agregar la URL de su mapa de sitio. Edite el archivo `urls.py` principal de su proyecto y agregue el mapa del sitio, de la siguiente manera:

```
from django.conf.urls import include, url
```

```
from django.contrib import admin
```

```
from django.contrib.sitemaps.views import Sitemap
```

```
from blog.sitemaps import PostSitemap
```

```
sitemaps = {
    'publicaciones': PostSitemap,
}
```

```
urlpatterns = [
```

```
    url('admin/', admin.site.urls),
```

```
    url('blog/', include('blog.urls', namespace='blog')),
```

```
    url('sitemap.xml', Sitemap.as_view({'sitemaps': sitemaps}),
```

```
        name='django.contrib.sitemaps.views.sitemap')
```

```
]
```


Ampliación de la aplicación de su blog

En el código anterior, incluye las importaciones necesarias y define un diccionario de mapas del sitio. Usted define un patrón de URL que coincide con sitemap.xml y utiliza el mapa del sitio. ver. El diccionario de mapas del sitio se pasa a la vista del mapa del sitio .

Ahora ejecute el servidor de desarrollo y abra `http://127.0.0.1:8000/sitemap.xml` en su navegador. Verá la siguiente salida XML:

```
<? xml version = "1.0" encoding = "utf-8"?>
<urlset xmlns = "http://www.sitemaps.org/schemas/sitemap/0.9">
  <url>
    <loc> http://example.com/blog/2020/1/2/markdown-post/ </loc>
    <lastmod> 2020-01-02 </lastmod>
    <changefreq> semanalmente </changefreq>
    <priority> 0.9 </priority>
  </url>
  <url>
    <loc>
      http://example.com/blog/2020/1/1/who-was-django-reinhardt/
    </loc>
    <lastmod> 2020-01-02 </lastmod>
    <changefreq> semanalmente </changefreq>
    <priority> 0.9 </priority>
  </url>
</urlset>
```

La URL de cada publicación se ha creado llamando a su método `get_absolute_url()` .

El atributo `lastmod` corresponde al campo de fecha de actualización posterior , como especificó en su mapa del sitio, y los atributos `changefreq` y `prioridad` también se toman del

Clase `PostSitemap` .

Puede ver que el dominio utilizado para crear las URL es `example.com` . Este dominio proviene de un objeto `Site` almacenado en la base de datos. Este objeto predeterminado fue creado cuando sincronizaste el marco del sitio con tu base de datos.

Abra `http://127.0.0.1:8000/admin/sites/site/` en su navegador. Deberías ver algo como esto:

Figura 3.5: Vista de la lista de administración de Django para el modelo de sitio del marco del sitio

La captura de pantalla anterior contiene la vista de administración de visualización de lista para el sitio marco de referencia. Aquí, puede configurar el dominio o el host que utilizará el marco del sitio. y las aplicaciones que dependen de él. Para generar URL que existan en su entorno local, cambie el nombre de dominio a localhost: 8000 , como se muestra en el siguiente captura de pantalla y guárdela:

Figura 3.6: Vista de edición de la administración de Django para el modelo de sitio del marco del sitio

Las URL que se muestran en su feed ahora se crearán con este nombre de host. en un entorno de producción, tendrá que utilizar su propio nombre de dominio para el marco del sitio.

Ampliación de la aplicación de su blog

Crear feeds para las publicaciones de tu blog

Django tiene un marco de alimentación de sindicación incorporado que puede usar para dinámicamente generar feeds RSS o Atom de manera similar a la creación de mapas de sitio utilizando el marco del sitio. Una fuente web es un formato de datos (generalmente XML) que proporciona a los usuarios con el contenido actualizado más recientemente. Los usuarios podrán suscribirse a su alimentar utilizando un agregador de feeds (software que se utiliza para leer feeds y obtener contenido nuevo notificaciones).

Cree un nuevo archivo en el directorio de aplicaciones de su blog y asígnele el nombre feeds.py . Añadir las siguientes líneas a él:

```

de django.contrib.syndication.views import Feed
de django.template.defaultfilters import truncatewords
de django.urls import reverse_lazy
desde .models import Post

```

```

clase LatestPostsFeed (Feed):
    title = 'Mi blog'
    link = reverse_lazy ('blog: post_list')
    description = 'Nuevas publicaciones de mi blog.'

    elementos def (uno mismo):
        return Post.published.all () [: 5]

    def item_title (self, item):
        devolver item.title

    def item_description (self, item):
        devolver palabras truncadas (item.body, 30)

```

Primero, subclases la clase Feed del marco de distribución. El título , enlace , y los atributos de descripción corresponden a <title> , <link> y <description> Elementos RSS, respectivamente.

Utiliza reverse_lazy () para generar la URL para el atributo de enlace . El reverse () El método le permite crear URL por su nombre y pasar parámetros opcionales. Usó reverse () en el *Capítulo 1 , Creación de una aplicación de blog* . El reverse_lazy () La función de utilidad es una versión evaluada de forma perezosa de reverse () . Te permite usar una URL reversion antes de que se cargue la configuración de URL del proyecto.

El método items () recupera los objetos que se incluirán en el feed. Usted está recuperando solo las últimas cinco publicaciones publicadas para este feed. El item_title () y Los métodos item_description () recibirán cada objeto devuelto por items () y devuelve el título y la descripción de cada artículo. Usas las palabras truncadas integradas filtro de plantilla para construir la descripción de la publicación del blog con las primeras 30 palabras.

[80]

Ahora edite el archivo blog / urls.py , importe el LatestPostsFeed que acaba de crear, y cree una instancia del feed en un nuevo patrón de URL:

```

desde .feeds import LatestPostsFeed

urlpatterns = [
    # ...
    ruta ('feed /', LatestPostsFeed (), name = 'post_feed'),
]

```

Vaya a <http://127.0.0.1:8000/blog/feed/> en su navegador. Deberías ahora vea la fuente RSS, incluidas las últimas cinco publicaciones del blog:

```

<? xml version = "1.0" encoding = "utf-8"?>
<rss xmlns: atom = "http://www.w3.org/2005/Atom" version = "2.0">
  <canal>
    <title> Mi blog </title>
    <link> http: // localhost: 8000 / blog / </link>
    <description> Nuevas publicaciones de mi blog. </description>
    <atom: link href = "http: // localhost: 8000 / blog / feed /" rel = "self" />
    <language> en-us </language>
    <lastBuildDate> Vie, 2 de enero de 2020 09:56:40 +0000 </lastBuildDate>
  <item>
    <title> ¿Quién era Django Reinhardt? </title>
    <enlace> http: // localhost: 8000 / blog / 2020/1/2 / who-was-django-
reinhardt / </link>
    <description> ¿Quién era Django Reinhardt? </description>

```

```

    <guid> http://localhost:8000/blog/2020/1/2/who-was-django-
    reinhardt / </guid>
  </item>
  ...
</canal>
</rss>

```

Si abre la misma URL en un cliente RSS, podrá ver su feed con un interfaz amigable.

El último paso es agregar un enlace de suscripción al feed en la barra lateral del blog. Abre el `blog/base.html` y agregue la siguiente línea debajo del número total de publicaciones dentro de la barra lateral `div` :

```

<p>
  <a href="{% url 'blog:post_feed' %}"> Suscribete a mi fuente RSS </a>
</p>

```

[81]

Ampliación de la aplicación de su blog

Ahora abra `http://127.0.0.1:8000/blog/` en su navegador y eche un vistazo al barra lateral. El nuevo enlace debería llevarlo al feed de su blog:

Figura 3.7: El enlace de suscripción a la fuente RSS agregado a la barra lateral

Puede leer más sobre el marco de alimentación de distribución de Django en <https://docs.djangoproject.com/en/3.0/ref/contrib/syndication/>.

Agregar búsqueda de texto completo a su blog

A continuación, agregará capacidades de búsqueda a su blog. Buscando datos en la base de datos con la entrada del usuario es una tarea común para las aplicaciones web. El ORM de Django te permite para realizar operaciones de coincidencia simples utilizando, por ejemplo, el filtro `contains` (o su versión que no distingue entre mayúsculas y minúsculas, `icontains`). Puede utilizar la siguiente consulta para buscar publicaciones que contienen la palabra `marco` en su cuerpo:

```

from blog.models import Publicación
Post.objects.filter(body__contains='framework')

```

Sin embargo, si desea realizar búsquedas complejas, recupere los resultados similitud, o ponderando los términos según la frecuencia con la que aparecen en el texto o por la importancia de los diferentes campos (por ejemplo, la relevancia del término que aparece en el título frente al cuerpo), deberá utilizar un motor de búsqueda de texto completo. Cuando considera grandes bloques de texto, construyendo consultas con operaciones en una cadena de los personajes no es suficiente. La búsqueda de texto completo examina las palabras reales frente a las contenido ya que intenta coincidir con los criterios de búsqueda.

Django proporciona una poderosa funcionalidad de búsqueda construida sobre PostgreSQL funciones de búsqueda de texto completo. El módulo `django.contrib.postgres` proporciona funcionalidades ofrecidas por PostgreSQL que no son compartidas por otras bases de datos que admite Django. Puede obtener información sobre la búsqueda de texto completo de PostgreSQL en <https://www.postgresql.org/docs/12/static/textsearch.html>.

Aunque Django es un marco web independiente de la base de datos, proporciona un módulo que admite parte del rico conjunto de funciones ofrecidas por PostgreSQL, que no es ofrecido por otras bases de datos que Django apoyos.

Instalación de PostgreSQL

Actualmente está utilizando SQLite para su proyecto de blog . Esto es suficiente para fines de desarrollo. Sin embargo, para un entorno de producción, necesitará una base de datos más potente, como PostgreSQL, MariaDB, MySQL u Oracle. Tú cambiará su base de datos a PostgreSQL para beneficiarse de sus funciones de búsqueda de texto completo.

Si está utilizando Linux, instale PostgreSQL con el siguiente comando:

```
sudo apt-get install postgresql postgresql-contrib
```

Si está utilizando macOS o Windows, descargue PostgreSQL desde <https://www.postgresql.org/download/> e instálelo.

También necesita instalar el adaptador psycopg2 PostgreSQL para Python. Ejecutar el siguiente comando en el shell para instalarlo:

```
pip instalar psycopg2-binary == 2.8.4
```

Creemos un usuario para su base de datos PostgreSQL. Abra el caparazón y ejecute el siguientes comandos:

```
su postgres
```

```
blog createuser -dP
```

Se le pedirá una contraseña para el nuevo usuario. Ingrese la contraseña deseada y luego cree la base de datos del blog y dé la propiedad al usuario del blog que acaba de creado con el siguiente comando:

```
createdb -E utf8 -U blog blog
```

Luego, edite el archivo settings.py de su proyecto y modifique la configuración de BASES DE DATOS para que tenga el siguiente aspecto:

```
BASES DE DATOS = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'blog',
        'USER': 'blog',
        'PASSWORD': '*****',
    }
}
```

Ampliación de la aplicación de su blog

```
}
```

Reemplace los datos anteriores con el nombre de la base de datos y las credenciales del usuario. tu creaste. La nueva base de datos está vacía. Ejecute el siguiente comando para aplicar todas las migraciones de bases de datos:

```
python manage.py migrar
```

Finalmente, cree un superusuario con el siguiente comando:

```
python manage.py crea superusuario
```

Ahora puede ejecutar el servidor de desarrollo y acceder al sitio de administración en `http://127.0.0.1:8000/admin/` con el nuevo superusuario.

Desde que cambió la base de datos, no hay publicaciones almacenadas en ella. Complete su nuevo base de datos con un par de publicaciones de blog de muestra para que pueda realizar búsquedas en la base de datos.

Búsquedas de búsqueda simples

Edite el archivo `settings.py` de su proyecto y agregue `django.contrib.postgres` al

Configuración `INSTALLED_APPS`, de la siguiente manera:

```
INSTALLED_APPS = [
    # ...
    'django.contrib.postgres',
]
```

Ahora puede buscar en un solo campo utilizando la búsqueda de búsqueda `QuerySet`, Me gusta esto:

```
from blog.models import Publicación
Post.objects.filter(body__search = 'django')
```

Esta consulta usa PostgreSQL para crear un vector de búsqueda para el campo del cuerpo y una búsqueda consulta del término `django`. Los resultados se obtienen haciendo coincidir la consulta con el vector.

Buscando en múltiples campos

Es posible que desee buscar en varios campos. En este caso, deberá definir un objeto `SearchVector`. Construyamos un vector que le permita buscar contra el

Campos de título y cuerpo del modelo de publicación :

[84]

```
from django.contrib.postgres.search import SearchVector
from blog.models import Publicación
```

```
Post.objects.annotate (
    search = SearchVector ('título', 'cuerpo'),
) .filter (búsqueda = 'django')
```

Al usar `anotar` y definir `SearchVector` con ambos campos, proporciona

una funcionalidad para hacer coincidir la consulta con el título y el cuerpo de las publicaciones.

La búsqueda de texto completo es un proceso intensivo. Si estas buscando más de unos cientos de filas, debe definir un funcional índice que coincide con el vector de búsqueda que está utilizando. Django proporciona un campo `SearchVectorField` para sus modelos. Usted puede leer más sobre esto [en https://docs.djangoproject.com/es/3.0/ref/contrib/postgres/search/#performance](https://docs.djangoproject.com/es/3.0/ref/contrib/postgres/search/#performance).

Construyendo una vista de búsqueda

Ahora, creará una vista personalizada para permitir que sus usuarios busquen publicaciones. Primero tú necesitará un formulario de búsqueda. Edite el archivo `forms.py` de la aplicación de blog y agregue el siguiente forma:

```
class SearchForm (formularios.Form):
    consulta = formularios.CharField ()
```

Utilizará el campo de consulta para permitir que los usuarios introduzcan términos de búsqueda. Edita `views.py` archivo de la aplicación de blog y agregue el siguiente código:

```
desde django.contrib.postgres.search importar SearchVector
desde .forms importar EmailPostForm, CommentForm, SearchForm
```

```
def post_search (solicitud):
    formulario = SearchForm ()
    consulta = Ninguno
    resultados = []
    si 'consulta' en request.GET:
        formulario = SearchForm (request.GET)
        si form.is_valid ():
            query = form.cleaned_data ['consulta']
            resultados = Publicar.publicado.annotate (
                search = SearchVector ('título', 'cuerpo'),
            ).filter (búsqueda = consulta)
    return render (solicitud,
```

[85]

Ampliación de la aplicación de su blog

```
'blog / post / search.html',
{'formulario': formulario,
 'consulta': consulta,
 'resultados': resultados})
```

En la vista anterior, primero crea una instancia del formulario `SearchForm`. Para comprobar si se envía el formulario, busca el parámetro de consulta en la solicitud. diccionario. Envíe el formulario utilizando el método `GET` en lugar de `POST`, de modo que el La URL resultante incluye el parámetro de consulta y es fácil de compartir. Cuando la forma es enviado, lo crea con los datos `GET` enviados y verifica que el formulario los datos son válidos. Si el formulario es válido, busca publicaciones publicadas con un `SearchVector` instancia construida con los campos de título y cuerpo.

La vista de búsqueda está lista ahora. Necesita crear una plantilla para mostrar el formulario y los resultados cuando el usuario realiza una búsqueda. Crea un nuevo archivo dentro del `blog / publicación /` directorio de plantillas, asígnele el nombre `search.html` y agregue el siguiente código:

```
{% extiende "blog / base.html"%}
{% load blog_tags%

{% block title%} Buscar {% endblock%}

{% block content%}
    {% si consulta%
```

```

<h1> Publicaciones que contienen "{{query}}" </h1>
<h3>
    {% con results.count as total_results%}
        Se encontró {{total_results}} resultado {{total_results | pluralize}}
    {% terminar con %}
</h3>
{% para publicación en resultados%}
    <h4> <a href="{{ post.get_absolute_url }}"> {{post.title}} </a> </
h4>
    {{post.body | markdown | truncatewords_html: 5}}
{% vacío%}
    <p> No hay resultados para su consulta. </p>
{% endfor%}
<p> <a href="{% url "blog:post_search" %}"> Buscar de nuevo </a> </p>
{% else%}
    <h1> Buscar publicaciones </h1>
    <método de formulario = "obtener">
        {{form.as_p}}
        <input type = "submit" value = "Buscar">
    </form>
    {% terminara si %}
{% endblock%}

```

[86]

Capítulo 3

Al igual que en la vista de búsqueda, puede distinguir si el formulario se ha enviado por la presencia del parámetro de consulta . Antes de enviar la consulta, muestra el formulario y un botón de envío. Después de enviar la publicación, muestra la consulta realizado, el número total de resultados y la lista de publicaciones devueltas.

Finalmente, edite el archivo `urls.py` de su aplicación de blog y agregue la siguiente URL patrón:

```

ruta ('buscar /', vistas.post_search, nombre = 'post_search'),

```

A continuación, abra `http://127.0.0.1:8000/blog/search/` en su navegador. Deberías ver el siguiente formulario de búsqueda:

Figura 3.8: El formulario con el campo de consulta para buscar publicaciones

Ingresa una consulta y haga clic en el botón **BUSCAR** . Verás los resultados de la búsqueda consulta, de la siguiente manera:

¡Felicidades! Ha creado un motor de búsqueda básico para su blog.

[87]

Ampliación de la aplicación de su blog

Derivar y clasificar los resultados

La derivación es el proceso de reducir las palabras a su raíz, base o raíz.

Los motores de búsqueda utilizan la derivación para reducir las palabras indexadas a su raíz y para ser capaz de hacer coincidir palabras con inflexiones o derivadas. Por ejemplo, "música" y "músico" un motor de búsqueda puede considerar palabras similares.

Django proporciona una clase `SearchQuery` para traducir términos en un objeto de consulta de búsqueda.

De forma predeterminada, los términos se pasan a través de algoritmos de derivación, lo que le ayuda para obtener mejores coincidencias. También desea ordenar los resultados por relevancia. PostgreSQL proporciona una función de clasificación que ordena los resultados en función de la frecuencia con la que aparecen los términos de la consulta aparecen y qué tan cerca están.

Edite el archivo `views.py` de su aplicación de blog y agregue las siguientes importaciones:

```
de django.contrib.postgres.search importar SearchVector, SearchQuery,
SearchRank
```

Luego, eche un vistazo a las siguientes líneas:

```
resultados = Publicar.publicado.annotate (
    search = SearchVector ('título', 'cuerpo'),
) .filter (búsqueda = consulta)
```

Reemplácelos por los siguientes:

```
search_vector = SearchVector ('título', 'cuerpo')
search_query = SearchQuery (consulta)
resultados = Publicar.publicado.annotate (
    búsqueda = vector_búsqueda,
    rango = SearchRank (vector_búsqueda, consulta_búsqueda)
) .filter (búsqueda = consulta_búsqueda ) .orden_por ('- rango')
```

En el código anterior, crea un objeto `SearchQuery`, filtra los resultados por él y utilice `SearchRank` para ordenar los resultados por relevancia.

Puede abrir `http://127.0.0.1:8000/blog/search/` en su navegador y probar diferentes búsquedas para probar la derivación y la clasificación. El siguiente es un ejemplo de clasificación según el número de apariciones de la palabra `django` en el título y el cuerpo de las publicaciones:

[88]

Figura 3.10: Resultados de la búsqueda del término "django"

Consultas de ponderación

Puede impulsar vectores específicos para que se les atribuya más peso cuando ordenar los resultados por relevancia. Por ejemplo, puede usar esto para dar más relevancia a publicaciones que coinciden por título en lugar de por contenido.

Edite las líneas anteriores del archivo `views.py` de su aplicación de blog y conviértalas se parece a esto:

```
search_vector = SearchVector ('título', peso = 'A') + \
    SearchVector ('cuerpo', peso = 'B')
search_query = SearchQuery (consulta)
resultados = Publicar.publicado.annotate (
    rango = SearchRank (vector_búsqueda, consulta_búsqueda)
).filter ( rango_gte = 0.3 ) .order_by ('- rango')
```

En el código anterior, aplica diferentes pesos a los vectores de búsqueda contruidos usando los campos de título y cuerpo . Los pesos predeterminados son D , C , B y A , y se refieren a los números 0.1 , 0.2 , 0.4 y 1.0 , respectivamente. Aplica un peso de 1.0 a la

vector de búsqueda de título y un peso de 0.4 para el vector de cuerpo . Los partidos por el título prevalecerán sobre coincidencias de contenido corporal. Filtra los resultados para mostrar solo los que tienen un rango superior a 0,3 .

Búsqueda con similitud de trigramas

Otro enfoque de búsqueda es la similitud de trigramas. Un trigramas es un grupo de tres caracteres consecutivos. Puede medir la similitud de dos cadenas contando la cantidad de trigramas que comparten. Este enfoque resulta muy eficaz

para medir la similitud de palabras en muchos idiomas.

Para usar trigrams en PostgreSQL, necesitará instalar pg_trgm extensión primero. Ejecute el siguiente comando desde el shell para conectarse a su base de datos:

```
blog psql
```

Luego, ejecute el siguiente comando para instalar la extensión pg_trgm :

```
CREAR EXTENSIÓN pg_trgm;
```

Editamos su vista y modifiquémosla para buscar trigramas. Edite el archivo views.py de su aplicación de blog y agregue la siguiente importación:

```
de django.contrib.postgres.search import TrigramSimilarity
```

Luego, reemplace la consulta de búsqueda de publicaciones con las siguientes líneas:

```
resultados = Publicar.publicado.annotate (
    similarity = TrigramSimilarity ('titulo', consulta),
) .filter ( similarity__gt = 0.1 ) .order_by ( '-similarity' )
```

Abra <http://127.0.0.1:8000/blog/search/> en su navegador y pruebe diferentes busca trigramas. El siguiente ejemplo muestra un error tipográfico hipotético en el término django , mostrando resultados de búsqueda para yango :

Figura 3.11: Resultados de búsqueda del término "yango"

Ahora tiene un potente motor de búsqueda integrado en su proyecto. Puedes encontrar más información sobre la búsqueda de texto completo en <https://docs.djangoproject.com/en/3.0/ref/contrib/postgres/search/>.

[90]

Otros motores de búsqueda de texto completo

Es posible que desee utilizar un motor de búsqueda de texto completo que no sea PostgreSQL. Si tu desea utilizar Solr o Elasticsearch, puede integrarlos en su proyecto Django usando Haystack. Haystack es una aplicación de Django que funciona como una abstracción capa para varios motores de búsqueda. Ofrece una API de búsqueda simple que es muy similar a Django QuerySets. Puede encontrar más información sobre Haystack en <https://django-haystack.readthedocs.io/en/master/>.

Resumen

En este capítulo, aprendió a crear filtros y etiquetas de plantilla de Django personalizados para proporcionar plantillas con una funcionalidad personalizada. También creaste un mapa del sitio para motores de búsqueda para rastrear su sitio y una fuente RSS para que los usuarios se suscriban a su Blog. Luego creó un motor de búsqueda para su blog utilizando el motor de búsqueda de texto completo de PostgreSQL.

En el próximo capítulo, aprenderá a crear un sitio web social utilizando la Marco de autenticación de Django, cree perfiles de usuario personalizados y desarrolle redes sociales

4

Construyendo un sitio web social

En el capítulo anterior, aprendió a crear mapas del sitio y feeds, y construyó un motor de búsqueda para su aplicación de blog. En este capítulo, descubrirás cómo desarrollar una aplicación social, lo que significa que los usuarios pueden unirse a una plataforma en línea e interactuar entre sí compartiendo contenido. Durante los próximos capítulos, nos centraremos en la creación de una plataforma para compartir imágenes. Los usuarios podrán para marcar cualquier imagen en Internet y compartirla con otros. Ellos también serán capaz de ver la actividad en la plataforma de los usuarios que siguen y les gusta / a diferencia del imágenes compartidas por ellos.

En este capítulo, comenzaremos creando una funcionalidad para que los usuarios inicien sesión, cierren la sesión, edite su contraseña y restablezca su contraseña. Aprenderá a crear un personalizado perfil para sus usuarios y agregará autenticación social a su sitio.

Este capítulo cubrirá los siguientes temas:

- Usando el marco de autenticación de Django
- Creación de vistas de registro de usuarios
- Ampliación del modelo de usuario con un modelo de perfil personalizado

- Agregar autenticación social con Python Social Auth

Comencemos por crear su nuevo proyecto.

[93]

Construyendo un sitio web social

Creación de un proyecto de sitio web social

Vas a crear una aplicación social que permitirá a los usuarios compartir imágenes que encuentran en Internet. Deberá crear los siguientes elementos para este proyecto:

- Un sistema de autenticación para que los usuarios se registren, inicien sesión, editen su perfil, y cambiar o restablecer su contraseña
- Un sistema de seguimiento para permitir que los usuarios se sigan unos a otros en el sitio web
- Una funcionalidad para mostrar imágenes compartidas e implementar un marcador para que los usuarios compartan imágenes desde cualquier sitio web
- Un flujo de actividad que permite a los usuarios ver el contenido subido por el personas que siguen

Este capítulo abordará el primer punto de la lista.

Comenzando su proyecto de sitio web social

Abra la terminal y use los siguientes comandos para crear un entorno virtual para su proyecto y actívelo:

```
mkdir env
python3 -m venv env / bookmarks
env de origen / marcadores / bin / activar
```

El indicador de shell mostrará su entorno virtual activo, de la siguiente manera:

```
(marcadores) portátil: ~ zenx $
```

Instale Django en su entorno virtual con el siguiente comando:

```
pip instalar Django == 3.0.*
```

Ejecute el siguiente comando para crear un nuevo proyecto:

```
marcadores de inicio de proyecto de django-admin
```

Se ha creado la estructura del proyecto inicial. Utilice los siguientes comandos para obtener en el directorio de su proyecto y cree una nueva aplicación llamada cuenta :

```
marcadores de cd /
cuenta django-admin startapp
```

Recuerde que debe agregar la nueva aplicación a su proyecto agregando el nombre de la aplicación a la configuración `INSTALLED_APPS` en el archivo `settings.py` . Colocararlo en la lista `INSTALLED_APPS` antes que cualquiera de las otras aplicaciones instaladas:

[94]

```
INSTALLED_APPS = [
    'account.apps.AccountConfig',
    # ...
]
```

Definirá las plantillas de autenticación de Django más adelante. Al colocar su aplicación primero en la configuración `INSTALLED_APPS`, se asegura de que su autenticación. Las plantillas se utilizarán de forma predeterminada en lugar de cualquier otra plantilla de autenticación contenida en otras aplicaciones. Django busca plantillas por orden de aplicación apariencia en la configuración `INSTALLED_APPS`.

Ejecute el siguiente comando para sincronizar la base de datos con los modelos predeterminados. aplicaciones incluidas en la configuración `INSTALLED_APPS`:

```
python manage.py migrar
```

Verá que se aplican todas las migraciones iniciales de la base de datos de Django. A continuación, lo hará construya un sistema de autenticación en su proyecto usando la autenticación Django marco de referencia.

Usando la autenticación de Django marco de referencia

Django viene con un marco de autenticación integrado que puede manejar usuarios autenticación, sesiones, permisos y grupos de usuarios. El sistema de autenticación incluye vistas para acciones comunes del usuario, como iniciar sesión, cerrar sesión, cambio de contraseña, y restablecimiento de contraseña.

El marco de autenticación se encuentra en `django.contrib.auth` y es utilizado por otros paquetes contrib de Django. Recuerda que ya usaste el marco de autenticación en el *Capítulo 1*, *Creación de una aplicación de blog*, para crear una superusuario para que su aplicación de blog acceda al sitio de administración.

Cuando crea un nuevo proyecto Django usando el comando `startproject`, el El marco de autenticación está incluido en la configuración predeterminada de su proyecto. Consiste en la aplicación `django.contrib.auth` y las dos siguientes clases de middleware que se encuentran en la configuración de `MIDDLEWARE` de su proyecto:

- `AuthenticationMiddleware`: asocia a los usuarios con solicitudes mediante sesiones
- `SessionMiddleware`: maneja la sesión actual entre solicitudes

o fase de respuesta. Utilizará clases de middleware en varias ocasiones durante este libro, y usted aprenderá cómo crear middleware de costumbre en *el capítulo 14, Going Vivir*.

El marco de autenticación también incluye los siguientes modelos:

- Usuario : un modelo de usuario con campos básicos; los principales campos de este modelo son nombre de usuario , contraseña , correo electrónico , nombre apellido , apellidos , y is_active
- Grupo : un modelo de grupo para categorizar usuarios
- Permiso : banderas para que los usuarios o grupos realicen determinadas acciones.

El marco también incluye vistas y formularios de autenticación predeterminados, que usará más tarde.

Crear una vista de inicio de sesión

Comenzaremos esta sección usando el marco de autenticación de Django para permitir usuarios para iniciar sesión en su sitio web. Su vista debe realizar las siguientes acciones para iniciar sesión como usuario:

- Obtenga el nombre de usuario y la contraseña que publica el usuario mediante un formulario de inicio de sesión
- Autenticar al usuario con los datos almacenados en la base de datos.
- Verifique si el usuario está activo
- Inicie la sesión del usuario en el sitio web e inicie una sesión autenticada

Primero, creará un formulario de inicio de sesión. Cree un nuevo archivo forms.py en su cuenta directorio de la aplicación y agregue las siguientes líneas:

```
desde formularios de importación de django

clase LoginForm (formularios.Formulario):
    nombre de usuario = formularios.CharField ()
    contraseña = formularios.CharField (widget = formularios.PasswordInput)
```

Este formulario se utilizará para autenticar a los usuarios en la base de datos. Tenga en cuenta que usa el widget PasswordInput para representar el elemento HTML de contraseña . Esto incluirá type = "contraseña" en el HTML para que el navegador lo trate como una entrada de contraseña.

Edite el archivo views.py de la aplicación de su cuenta y agregue el siguiente código:

```
desde django.http import HttpResponseRedirect
desde django.shortcuts import render
desde django.contrib.auth import autenticar, iniciar sesión
desde .forms importar LoginForm
```

[96]

```
def user_login (solicitud):
    if request.method == 'POST':
        formulario = LoginForm (request.POST)
        si form.is_valid ():
            cd = form.cleaned_data
            usuario = autenticar (solicitud,
                                username = cd ['nombre de usuario'],
                                contraseña = cd ['contraseña'])

            si el usuario no es Ninguno:
                si user.is_active:
                    inicio de sesión (solicitud, usuario)
                    return HttpResponseRedirect ('Autenticado' \
                                                'exitosamente')

            más:
                return HttpResponseRedirect ('Cuenta deshabilitada')

            más:
                return HttpResponseRedirect ('Inicio de sesión no válido')
```

```

más: formulario = LoginForm ()
return render (solicitud, 'cuenta / login.html', {'formulario': formulario})

```

Esto es lo que hace la vista de inicio de sesión básica: cuando se llama a la vista `user_login` con un

GET request, crea una instancia de un nuevo formulario de inicio de sesión con `form = LoginForm ()` para mostrar en la plantilla. Cuando el usuario envía el formulario a través de POST, realiza el siguientes acciones:

- Cree una instancia del formulario con los datos enviados con `form = LoginForm (request.POST)`.
- Verifique si el formulario es válido con `form.is_valid ()`. Si no es válido, muestra los errores de formulario en su plantilla (por ejemplo, si el usuario no complete uno de los campos).
- Si los datos enviados son válidos, autentica al usuario en la base de datos utilizando el método `authenticate ()`. Este método toma el objeto de solicitud, el nombre de usuario y los parámetros de contraseña y devuelve el objeto `Usuario` si el usuario se ha autenticado correctamente, o `Ninguno` de lo contrario. Si el usuario no ha sido autenticado, devuelve una `HttpResponse` sin procesar, mostrando el Mensaje de **inicio de sesión no válido**.
- Si el usuario se autenticó correctamente, compruebe si el usuario es activo accediendo al atributo `is_active`. Este es un atributo de Django modelo de usuario. Si el usuario no está activo, devuelve una `HttpResponse` que muestra el mensaje **Cuenta deshabilitada**.
- Si el usuario está activo, lo registra en el sitio web. Establece al usuario en la sesión llamando al método `login ()` y devuelva el **Authenticated** mensaje con **éxito**.

[97]

Construyendo un sitio web social

Tenga en cuenta la diferencia entre autenticar e iniciar sesión: `authenticate ()` comprueba las credenciales del usuario y devuelve un usuario objeto si son correctos; `login ()` establece al usuario en el actual sesión.

Ahora deberá crear un patrón de URL para esta vista. Cree un nuevo archivo `urls.py` en el directorio de la aplicación de su cuenta y agregue el siguiente código:

```

desde la ruta de importación django.urls
de . importar vistas

urlpatterns = [
    # vistas de publicaciones
    ruta ('login /', views.user_login, name = 'login'),
]

```

Edite el archivo `urls.py` principal ubicado en el directorio del proyecto de marcadores, importe incluir y agregar los patrones de URL de la aplicación de la cuenta, de la siguiente manera:

```

de la ruta de importación django.urls, incluir
desde django.contrib import admin

urlpatterns = [
    ruta ('admin /', admin.site.urls),
    ruta ('cuenta /', incluir ('cuenta.urls')),
]

```

Ahora se puede acceder a la vista de inicio de sesión mediante una URL. Es hora de crear una plantilla para esta vista. Como no tiene ninguna plantilla para este proyecto, puede empezar por creando una plantilla base que se puede ampliar con la plantilla de inicio de sesión. Crea el siguientes archivos y directorios dentro del directorio de la aplicación de la cuenta :


```
plantillas/  
  cuenta/  
    login.html  
  base.html
```

Edite la plantilla base.html y agréguele el siguiente código:

```
{% load static%}  
<!DOCTYPE html>  
<html>  
<cabeza>  
  <title> {% block title%} {% endblock%} </title>  
  <link href = "{% static" css / base.css "%}" rel = "stylesheet">
```

[98]

```
</head>  
<cuerpo>  
  <div id = "header">  
    <span class = "logo"> Marcadores </span>  
  </div>  
  <div id = "contenido">  
    {% block content%}  
    {% endblock%}  
  </div>  
</body>  
</html>
```

Esta será la plantilla base para el sitio web. Como hiciste en tu proyecto anterior, incluir los estilos CSS en la plantilla principal. Puede encontrar estos archivos estáticos en el código que viene con este capítulo. Copie el directorio estático / de la cuenta aplicación desde el código fuente del capítulo a la misma ubicación en su proyecto, por lo que puede utilizar los archivos estáticos. Puede encontrar el contenido del directorio en <https://github.com/PacktPublishing/Django-3-by-Example/tree/master/Chapter04/marcadores/cuenta/estática>.

La plantilla base define un bloque de título y un bloque de contenido que se puede llenar con contenido por las plantillas que se extienden a partir de él.

Completemos la plantilla para su formulario de inicio de sesión. Abra la cuenta / login.html plantilla y agregue el siguiente código:

```
{% extiende "base.html"%}  
  
{% block title%} Iniciar sesión {% endblock%}  
  
{% block content%}  
  <h1> Iniciar sesión </h1>  
  <p> Por favor, utilice el siguiente formulario para iniciar sesión: </p>  
  <método de formulario = "publicar">  
    {{form.as_p}}  
    {% csrf_token%}  
  <p> <input type = "submit" value = "Iniciar sesión"> </p>  
  </form>  
{% endblock%}
```

Esta plantilla incluye el formulario que se instancia en la vista. Desde tu forma se enviará mediante POST, incluirá la etiqueta de plantilla {% csrf_token%} para protección de **falsificación de solicitud entre sitios (CSRF)**. Aprendió sobre la protección CSRF en *Capítulo 2, Mejora de su blog con funciones avanzadas*.

Construyendo un sitio web social

Aún no hay usuarios en su base de datos. Primero deberá crear un superusuario para poder acceder al sitio de administración para gestionar otros usuarios. Abierto la línea de comando y ejecute `python manage.py createduperuser` . Complete el nombre de usuario, correo electrónico y contraseña deseados. Luego, ejecute el servidor de desarrollo usando el comando `python manage.py runserver` y abra `http://127.0.0.1:8000/admin/` en su navegador. Acceda al sitio de administración utilizando las credenciales del usuario que acaba de crear. Verá el sitio de administración de Django, incluido el Modelos de usuario y grupo del marco de autenticación de Django.

Se verá de la siguiente manera:

Figura 4.1: La página de índice del sitio de administración de Django, incluidos usuarios y grupos

Cree un nuevo usuario utilizando el sitio de administración y abra `http://127.0.0.1:8000/cuenta/` / iniciar sesión / en su navegador. Debería ver la plantilla renderizada, incluida el formulario de inicio de sesión:

Figura 4.2: La página de inicio de sesión del usuario

[100]

Ahora, envíe el formulario, dejando uno de los campos vacío. En este caso, verá que el formulario no es válido y muestra errores, de la siguiente manera:

Figura 4.3: El formulario de inicio de sesión con errores de campo

Tenga en cuenta que algunos navegadores modernos evitarán que envíe el formulario con campos vacíos o erróneos. Esto se debe a la validación del formulario realizada por el navegador basado en tipos de campo y restricciones por campo. En este caso, el formulario no será enviado y el navegador mostrará un mensaje de error para los campos que son incorrectos.

Si ingresa un usuario inexistente o una contraseña incorrecta, obtendrá un **inicio de sesión no válido** mensaje.

Si ingresa credenciales válidas, recibirá un mensaje **Autenticado exitosamente**, Me gusta esto:

Figura 4.4: Respuesta de texto sin formato de autenticación exitosa

Usar vistas de autenticación de Django

Django incluye varias formas y vistas en el marco de autenticación que puede usar de inmediato. La vista de inicio de sesión que ha creado es un buen ejercicio para comprender el proceso de autenticación de usuarios en Django. Sin embargo, puede utilizar las vistas de autenticación de Django predeterminadas en la mayoría de los casos.

[101]

Construyendo un sitio web social

Django proporciona las siguientes vistas basadas en clases para lidiar con la autenticación. Todos ellos se encuentran en `django.contrib.auth.views`:

- `LoginView`: maneja un formulario de inicio de sesión e inicia sesión en un usuario
- `LogoutView`: cierra la sesión de un usuario

Django proporciona las siguientes vistas para manejar los cambios de contraseña:

- `PasswordChangeView`: maneja un formulario para cambiar la contraseña del usuario
- `PasswordChangeDoneView`: la vista correcta a la que se redirige al usuario después de un cambio de contraseña exitoso

Django también incluye las siguientes vistas para permitir a los usuarios restablecer su contraseña:

- PasswordResetView : permite a los usuarios restablecer su contraseña. Genera un enlace de uso único con un token y lo envía a la cuenta de correo electrónico de un usuario.
- PasswordResetDoneView : les dice a los usuarios que un correo electrónico, incluido un enlace a restablecer su contraseña — se les ha enviado.
- PasswordResetConfirmView : permite a los usuarios establecer una nueva contraseña.
- PasswordResetCompleteView : la vista correcta a la que se redirige al usuario a después de restablecer con éxito su contraseña.

Las vistas enumeradas en las listas anteriores pueden ahorrarle mucho tiempo al crear una sitio web con cuentas de usuario. Las vistas utilizan valores predeterminados que puede anular, como la ubicación de la plantilla que se va a representar o el formulario que utilizará la vista.

Puede obtener más información sobre las vistas de autenticación integradas en

<https://docs.djangoproject.com/en/3.0/topics/auth/default/#all-vistas-de-autenticación>.

Vistas de inicio y cierre de sesión

Edite el archivo `urls.py` de la aplicación de su cuenta, así:

```

desde la ruta de importación django.urls
desde django.contrib.auth importar vistas como auth_views
de . importar vistas

urlpatterns = [
    # vista de inicio de sesión anterior
    # ruta ('login /', views.user_login, name = 'login'),
    ruta ('login /', auth_views.LoginView.as_view (), name = 'login') ,
    ruta ('cerrar sesión /', auth_views.LogoutView.as_view (), nombre = 'cerrar sesión') ,
]
```

[102]

En el código anterior, comenta el patrón de URL para la vista `user_login` que creó anteriormente para usar la vista `LoginView` de la autenticación de Django marco de referencia. También agrega un patrón de URL para la vista `LogoutView`.

Cree un nuevo directorio dentro del directorio de plantillas de su aplicación de cuenta y asígnele el nombre de registro . Esta es la ruta predeterminada donde la autenticación de Django Las vistas esperan que sus plantillas de autenticación sean.

El módulo `django.contrib.admin` incluye algunas de las plantillas de autenticación que se utilizan para el sitio de administración. Ha colocado la solicitud de cuenta en la parte superior de la configuración `INSTALLED_APPS` para que Django use sus plantillas por defecto en lugar de las plantillas de autenticación definidas en otras aplicaciones.

Cree un nuevo archivo dentro del directorio de plantillas / registro , asígnele el nombre `login`.

html y agregue el siguiente código:

```

{% extiende "base.html"%}

{% block title%} Iniciar sesión {% endblock%}

{% block content%}
    <h1> Iniciar sesión </h1>
    {% if form.errors%}
    <p>
        Tu nombre de usuario y contraseña no coinciden.
        Inténtalo de nuevo.
    </p>
    {% else%}
    <p> Por favor, utilice el siguiente formulario para iniciar sesión: </p>

```

```

{% terminara si %}
<div class = "formulario de inicio de sesión">
    <form action = "{% url 'login'%}" method = "post">
        {{form.as_p}}
        {% csrf_token%}
        <input type = "hidden" name = "next" value = "{{next}}"/>
        <p> <input type = "submit" value = "Iniciar sesión"> </p>
    </form>
</div>
{% endblock%}

```

Esta plantilla de inicio de sesión es bastante similar a la que creó antes. Django usa el AuthenticationForm formulario ubicado en django.contrib.auth.forms por defecto. Este formulario intenta autenticar al usuario y genera un error de validación si el inicio de sesión fue fracasado. En este caso, puede buscar errores usando {% if form.errors%} en la plantilla para comprobar si las credenciales proporcionadas son incorrectas.

[103]

Construyendo un sitio web social

Tenga en cuenta que ha agregado un elemento HTML <input> oculto para enviar el valor de una variable llamada siguiente . Esta variable la establece primero la vista de inicio de sesión cuando pasa un siguiente parámetro en la solicitud (por ejemplo, http://127.0.0.1:8000/account/iniciar sesión/? siguiente = / cuenta /).

El siguiente parámetro tiene que ser una URL. Si se proporciona este parámetro, el inicio de sesión de Django view redirigirá al usuario a la URL dada después de un inicio de sesión exitoso.

Ahora, cree una plantilla log_out.html dentro de la plantilla de registro directorio y que se vea así:

```

{% extiende "base.html"%}

{% block title%} Se desconectó {% endblock%}

{% block content%}
    <h1> Desconectado </h1>
    <p>
        Ha sido desconectado exitosamente.
        Puede <a href="{% url "login" %}"> iniciar sesión de nuevo </a>.
    </p>
{% endblock%}

```

Esta es la plantilla que Django mostrará después de que el usuario cierre la sesión.

Después de agregar los patrones de URL y las plantillas para las vistas de inicio y cierre de sesión, su El sitio web ahora está listo para que los usuarios inicien sesión utilizando las vistas de autenticación de Django.

Ahora, creará una nueva vista para mostrar un panel cuando los usuarios inicien sesión en su cuenta. Abra el archivo views.py de la aplicación de su cuenta y agregue lo siguiente codificarlo:

```

de django.contrib.auth.decorators importar login_required

@Necesario iniciar sesión
def dashboard (solicitud):
    return render (solicitud,
                    'account / dashboard.html',
                    {'section': 'dashboard'})

```

Decora su vista con el decorador login_required de la autenticación marco de referencia. El decorador login_required comprueba si el usuario actual es autenticado. Si el usuario está autenticado, ejecuta la vista decorada; Si el usuario no está autenticado, dirige al usuario a la URL de inicio de sesión con el original URL solicitada como parámetro GET con el nombre siguiente .

Capítulo 4

Al hacer esto, la vista de inicio de sesión redirige a los usuarios a la URL que estaban intentando acceder después de iniciar sesión correctamente. Recuerde que agregó una entrada oculta en el formulario de su plantilla de inicio de sesión para este propósito.

También puede definir una variable de sección . Utilizará esta variable para rastrear el la sección del sitio que el usuario está navegando. Varias vistas pueden corresponder a la misma sección. Esta es una forma sencilla de definir la sección a la que corresponde cada vista.

A continuación, deberá crear una plantilla para la vista del panel. Crea un archivo nuevo dentro del directorio templates / account / y asígnele el nombre dashboard.html . Hazlo se parece a esto:

```
{% extiende "base.html"%}

{% block title%} Panel de control {% endblock%}

{% block content%}
<h1> Panel </h1>
<p> Bienvenido a su panel. </p>
{% endblock%}
```

Luego, agregue el siguiente patrón de URL para esta vista en el archivo urls.py de la cuenta solicitud:

```
urlpatterns = [
    # ...
    ruta ('', vistas.dashboard, nombre = 'panel') ,
]
```

Edite el archivo settings.py de su proyecto y agregue el siguiente código:

```
LOGIN_REDIRECT_URL = 'tablero'
LOGIN_URL = 'iniciar sesión'
LOGOUT_URL = 'cerrar sesión'
```

Los ajustes definidos en el código anterior son los siguientes:

- LOGIN_REDIRECT_URL : le dice a Django a qué URL redirigir al usuario después un inicio de sesión exitoso si no hay el siguiente parámetro presente en la solicitud
- LOGIN_URL : la URL para redirigir al usuario para que inicie sesión (por ejemplo, vistas usando el decorador login_required)
- LOGOUT_URL : la URL para redirigir al usuario para que cierre la sesión

Estás usando los nombres de los patrones de URL que definiste previamente usando el atributo de nombre de la función ruta () . URL codificadas en lugar de nombres de URL también se puede utilizar para estos ajustes.

Resumamos lo que ha hecho hasta ahora:

- Ha agregado el inicio y cierre de sesión de autenticación de Django integrado vistas a su proyecto
- Ha creado plantillas personalizadas para ambas vistas y ha definido una simple vista de panel para redirigir a los usuarios después de iniciar sesión
- Finalmente, ha configurado su configuración para que Django use estas URL por defecto

Ahora, agregará enlaces de inicio y cierre de sesión a su plantilla base para poner todo juntos. Para hacer esto, debe determinar si el usuario actual es iniciado sesión o no para mostrar el enlace correspondiente a cada caso. La corriente El usuario se establece en el objeto HttpRequest mediante el middleware de autenticación. Usted puede acceder a él con request.user . Encontrará un objeto Usuario en la solicitud incluso si el usuario no está autenticado. Un usuario no autenticado se establece en la solicitud como un instancia de AnonymousUser . La mejor forma de comprobar si el usuario actual es autenticado es accediendo al atributo de solo lectura is_authenticated .

Edite su plantilla base.html y modifique el elemento <div> con un ID de encabezado , Me gusta esto:

```
<div id = "header">
  <span class = "logo"> Marcadores </span>
  {% if request.user.is_authenticated%}
    <ul class = "menu">
      <li {% if section == "dashboard"%} class = "selected" {% endif%}>
        <a href="{% url "dashboard" %}"> Mi panel </a>
      </li>
      <li {% if section == "images"%} class = "selected" {% endif%}>
        <a href="#"> Imágenes </a>
      </li>
      <li {% if section == "people"%} class = "selected" {% endif%}>
        <a href="#"> Personas </a>
      </li>
    </ul>
  {% terminara si %}

  <span class = "usuario">
    {% if request.user.is_authenticated%}
      Hola, {{request.user.first_name}}:
      <a href="{% url "logout" %}"> Cerrar sesión </a>
    {% else%}
      <a href="{% url "login" %}"> Iniciar sesión </a>
    {% terminara si %}
  </span>
</div>
```

[106]

Como puede ver en el código anterior, solo muestra el menú del sitio para usuarios autenticados. También revisa la sección actual para agregar una clase seleccionada atributo al elemento correspondiente para resaltar la sección actual en el menú usando CSS. Muestra el nombre del usuario y un enlace para cerrar sesión si el usuario está autenticado, o un enlace para iniciar sesión en caso contrario.

Ahora, abra <http://127.0.0.1:8000/account/login/> en su navegador. Tú debería ver la página de inicio de sesión. Ingrese un nombre de usuario y contraseña válidos y haga clic en el Botón **de** inicio de **sesión** . Debería ver el siguiente resultado:

Figura 4.5: La página del tablero

Puede ver que la sección **Mi panel** está resaltada con CSS porque tiene una clase seleccionada. Dado que el usuario está autenticado, el nombre del usuario es que se muestra en el lado derecho del encabezado. Haga clic en el enlace **Cerrar sesión**. Debieras consulte la siguiente página:

Figura 4.6: La página de cierre de sesión

En la página de la captura de pantalla anterior, puede ver que el usuario está desconectado, y, por tanto, el menú del sitio web ya no se muestra. Ahora el enlace en el lado derecho del encabezado muestra **Iniciar sesión**.

Si ve la página de cierre de sesión del sitio de administración de Django en lugar de su propia página desconectada, consulte `INSTALLED_APPS` configuración de su proyecto y asegúrese de que `django.contrib.admin` viene después de la aplicación de la cuenta. Ambas plantillas son ubicado en la misma ruta relativa, y el cargador de plantillas de Django usará el primero que encuentre.

[107]

Construyendo un sitio web social

Cambiar las vistas de contraseña

También necesita que sus usuarios puedan cambiar su contraseña después de iniciar sesión en su sitio. Integrará las vistas de autenticación de Django para un cambio de contraseña.

Abra el archivo `urls.py` de la aplicación de la cuenta y agregue los siguientes patrones de URL lo:

```
# cambiar URL de contraseña
ruta ('cambio_contraseña /',
     auth_views.PasswordChangeView.as_view (),
     nombre = 'cambio_contraseña'),
ruta ('cambio_contraseña / hecho /',
     auth_views.PasswordChangeDoneView.as_view (),
     nombre = 'cambio_contraseña_done'),
```

La vista `PasswordChangeView` manejará el formulario para cambiar la contraseña y la vista `PasswordChangeDoneView` mostrará un mensaje de éxito después de que el usuario ha cambiado correctamente su contraseña. Creemos una plantilla para cada vista.

Agregue un nuevo archivo dentro del directorio `templates / registration /` de su cuenta aplicación y asigne el nombre `password_change_form.html`. Agregue el siguiente código:

```
{% extiende "base.html"%}

{% block title%} Cambia tu contraseña {% endblock%}

{% block content%}
```



```

<h1> Cambie su contraseña </h1>
<p> Utilice el formulario a continuación para cambiar su contraseña. </p>
<método de formulario = "publicar">
  {{form.as_p}}
  <p> <input type = "submit" value = "Change"> </p>
  {% csrf_token%}
</form>
{% endblock%}

```

La plantilla password_change_form.html incluye el formulario para cambiar la contraseña.

Ahora cree otro archivo en el mismo directorio y asígnele el nombre password_change_done.

html . Agregue el siguiente código:

```

{% extiende "base.html"%}

{% block title%} Se cambió la contraseña {% endblock%}

{% block content%}
  <h1> Contraseña cambiada </h1>
  <p> Su contraseña se ha cambiado correctamente. </p>
{% endblock%}

```

[108]

La plantilla password_change_done.html solo contiene el mensaje de éxito para se mostrará cuando el usuario haya cambiado correctamente su contraseña.

Abra http://127.0.0.1:8000/account/password_change/ en su navegador.

Si no ha iniciado sesión, el navegador le redirigirá a la página de inicio de sesión. Después de ti se autentican correctamente, verá la siguiente página de cambio de contraseña:

Figura 4.7: El formulario de cambio de contraseña

Complete el formulario con su contraseña actual y su nueva contraseña, y haga clic en el botón **CAMBIAR** . Verá la siguiente página de éxito:

Construyendo un sitio web social

Cierre sesión e inicie sesión nuevamente con su nueva contraseña para verificar que todo funciona como se esperaba.

Restablecer vistas de contraseña

Agregue los siguientes patrones de URL para restaurar la contraseña al archivo `urls.py` del solicitud de cuenta :

```
# restablecer URL de contraseña
ruta ('contraseña_restablecer /',
      auth_views.PasswordResetView.as_view (),
      nombre = 'contraseña_reset'),
ruta ('contraseña_restablecer / hecho /',
      auth_views.PasswordResetDoneView.as_view (),
      nombre = 'contraseña_reset_done'),
ruta ('reset / <uidb64> / <token> /',
      auth_views.PasswordResetConfirmView.as_view (),
      nombre = 'contraseña_reset_confirm'),
ruta ('restablecer / listo /',
      auth_views.PasswordResetCompleteView.as_view (),
      name = 'password_reset_complete'),
```

Agregue un nuevo archivo en el directorio `templates / registration /` de su cuenta aplicación y asígnele el nombre `password_reset_form.html` . Agregue el siguiente código:

```
{% extiende "base.html"%}

{% block title%} Restablece tu contraseña {% endblock%}

{% block content%}
<h1> ¿Olvidó su contraseña? </h1>
<p> Ingrese su dirección de correo electrónico para obtener una nueva contraseña. </p>
<método de formulario = "publicar">
  {{form.as_p}}
  <p> <input type = "submit" value = "Enviar correo electrónico"> </p>
  {% csrf_token%}
</form>
{% endblock%}
```

Ahora cree otro archivo en el mismo directorio y asígnele el nombre `password_reset_email.html` . Agregue el siguiente código:

```
Alguien solicitó el restablecimiento de la contraseña del correo electrónico {{email}}. Siga el
enlace debajo:
{{protocolo}}:// {{dominio}} {{ url "password_reset_confirm"
uidb64 = uid token = token%}}
Su nombre de usuario, en caso de que lo haya olvidado: {{user.get_username}}
```

La plantilla `password_reset_email.html` se utilizará para procesar el correo electrónico enviado a usuarios para restablecer su contraseña. Incluye un token de restablecimiento que genera la vista.

Cree otro archivo en el mismo directorio y asígnele el nombre `password_reset_done.html`. Agregue el siguiente código:

```
{% extiende "base.html"%}

{% block title%} Restablece tu contraseña {% endblock%}

{% block content%}
    <h1> Restablece tu contraseña </h1>
    <p> Le enviamos instrucciones por correo electrónico para configurar su contraseña. </p>
    <p> Si no recibe un correo electrónico, asegúrese de haber ingresado
    la dirección con la que se registró. </p>
{% endblock%}
```

Cree otra plantilla en el mismo directorio y asígnele el nombre `password_reset_confirm.html`. Agregue el siguiente código:

```
{% extiende "base.html"%}

{% block title%} Restablece tu contraseña {% endblock%}

{% block content%}
    <h1> Restablece tu contraseña </h1>
    {% if validlink%}
        <p> Ingrese su nueva contraseña dos veces: </p>
        <método de formulario = "publicar">
            {{form.as_p}}
            {% csrf_token%}
            <p> <input type = "submit" value = "Cambiar mi contraseña" /> </p>
        </form>
    {% else%}
        <p> El enlace para restablecer la contraseña no era válido, posiblemente porque tiene
        ya se ha utilizado. Solicite un nuevo restablecimiento de contraseña. </p>
    {% terminara si %}
{% endblock%}
```

En esta plantilla, verifica si el enlace para restablecer la contraseña es válido comprobando la variable `validlink`. La vista `PasswordResetConfirmView` verifica la validez del token proporcionado en la URL y pasa la variable `validlink` a la plantilla. Si el enlace es válido, muestra el formulario de restablecimiento de contraseña de usuario. Los usuarios solo pueden establecer una nueva contraseña si tienen un enlace válido para restablecer la contraseña.

Cree otra plantilla y asígnele el nombre `password_reset_complete.html`. Introducir el siguiente código en él:

```
{% extiende "base.html"%}

{% block title%} Restablecimiento de contraseña {% endblock%}
```

```
{% block content%}
<h1> Contraseña establecida </h1>
<p> Se ha establecido su contraseña. Usted puede
<a href="{% url 'login' %}"> inicie sesión ahora </a> </p>
{% endblock%}
```

Finalmente, edite la plantilla registration / login.html de la aplicación de la cuenta , y agregue el siguiente código después del elemento <form> :

```
<p> <a href="{% url 'password_reset' %}"> ¿Olvidó su contraseña? </a> </p>
```

Ahora, abra http://127.0.0.1:8000/account/login/ en su navegador y haga clic en ¿ **Olvidó su contraseña?** enlace. Debería ver la siguiente página:

Figura 4.9: El formulario de restauración de contraseña

En este punto, debe agregar un **Protocolo simple de transferencia de correo (SMTP)** configuración al archivo settings.py de su proyecto para que Django pueda enviar correos electrónicos. Aprendió cómo agregar configuraciones de correo electrónico a su proyecto en el *Capítulo 2 , Mejorando su blog con funciones avanzadas* . Sin embargo, durante el desarrollo, puede configurar Django para escribir correos electrónicos en la salida estándar en lugar de enviar a través de un servidor SMTP. Django proporciona un backend de correo electrónico para escribir correos electrónicos a la consola. Edite el archivo settings.py de su proyecto y agregue la siguiente línea:

[112]

```
EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'
```

La configuración EMAIL_BACKEND indica la clase que se usará para enviar correos electrónicos.

Regrese a su navegador, ingrese la dirección de correo electrónico de un usuario existente y haga clic en el botón **ENVIAR E-MAIL** . Debería ver la siguiente página:

Figura 4.10: Página de envío de correo electrónico para restablecer la contraseña

Eche un vistazo a la consola donde está ejecutando el servidor de desarrollo. Tú verá el correo electrónico generado, de la siguiente manera:

Tipo de contenido: texto / sin formato; charset = "utf-8"

Versión MIME: 1.0

Codificación de transferencia de contenido: 7 bits
Asunto: restablecimiento de contraseña en 127.0.0.1:8000
De: webmaster @ localhost
Para: usuario@dominio.com
Fecha: vie, 3 de enero de 2020 14:35:08 -0000
ID de mensaje: <20150924143508.62996.55653@zenx.local>

Alguien solicitó restablecer la contraseña del correo electrónico usuario@dominio.com. Siga el enlace debajo:

<http://127.0.0.1:8000/account/reset/MQ/45f-9c3f30caafd523055fcc/>

Su nombre de usuario, en caso de que lo haya olvidado: zenx

El correo electrónico se procesa con la plantilla password_reset_email.html que creado anteriormente. La URL para restablecer la contraseña incluye un token que se generó dinámicamente por Django.

[113]

Construyendo un sitio web social

Copie la URL y ábrala en su navegador. Debería ver la siguiente página:

Figura 4.11: El formulario de restablecimiento de contraseña

La página para establecer una nueva contraseña usa password_reset_confirm.html modelo. Ingrese una nueva contraseña y haga clic en **CAMBIAR MI CONTRASEÑA** botón. Django crea una nueva contraseña hash y la guarda en la base de datos. Tú verá la siguiente página de éxito:

Figura 4.12: La página de restablecimiento de contraseña exitoso

Ahora puede volver a iniciar sesión en su cuenta con su nueva contraseña.

Cada token para establecer una nueva contraseña se puede usar solo una vez. Si abre el enlace que recibió de nuevo, recibirá un mensaje que indica que el token no es válido.

Ahora ha integrado las vistas del marco de autenticación de Django en tu proyecto. Estas vistas son adecuadas para la mayoría de los casos. Sin embargo, puede crear sus propias opiniones si necesita un comportamiento diferente.

[114]

Django también proporciona los patrones de URL de autenticación que acaba de crear. Tú puedes comentar los patrones de URL de autenticación que agregó a `urls.py` archivo de la aplicación de la cuenta e incluir `django.contrib.auth.urls` en su lugar, como sigue:

```
de la ruta de importación django.urls, incluir
# ...

urlpatterns = [
    # ...
    ruta('', incluir ('django.contrib.auth.urls')),
]
```

Puede ver los patrones de URL de autenticación incluidos en <https://github.com/django/django/blob/master/django/contrib/auth/urls.py>.

Registro de usuarios y perfiles de usuarios

Los usuarios existentes ahora pueden iniciar sesión, cerrar sesión, cambiar su contraseña y restablecer su contraseña. Ahora, deberá crear una vista para permitir que los visitantes creen un usuario cuenta.

Registro de usuario

Creemos una vista simple para permitir el registro de usuarios en su sitio web. Inicialmente, tienes que crear un formulario para permitir que el usuario ingrese un nombre de usuario, su nombre real y un contraseña.

Edite el archivo `forms.py` ubicado dentro del directorio de la aplicación de la cuenta y agregue el siguiente código:

```
desde django.contrib.auth.models import User

clase UserRegistrationForm (formularios.ModelForm):
    contraseña = formularios.CharField (etiqueta = 'Contraseña',
                                         widget = formularios.PasswordInput)
    contraseña2 = formularios.CharField (etiqueta = 'Repetir contraseña',
                                         widget = formularios.PasswordInput)

    clase Meta:
        modelo = Usuario
        campos = ('nombre de usuario', 'primer_nombre', 'correo electrónico')

    def clean_password2 (yo):
```

[115]

Construyendo un sitio web social

```
cd = self.cleaned_data
if cd ['contraseña'] != cd ['contraseña2']:
    subir formularios.ValidationError ('Las contraseñas no coinciden')
return cd ['contraseña2']
```

Ha creado un formulario modelo para el modelo de usuario. En su formulario, solo incluye los nombre de usuario , first_name , y correo electrónico campos del modelo. Estos campos serán validado en base a sus campos de modelo correspondientes. Por ejemplo, si el usuario elige un nombre de usuario que ya existe, obtendrán un error de validación porque el nombre de usuario es un campo definido con único = Verdadero .

Ha agregado dos campos adicionales, contraseña y contraseña2, para que los usuarios establezcan su contraseña y confirmela. Ha definido un método clean_password2 () para comparar la segunda contraseña con la primera y no permitir que el formulario se valide si el las contraseñas no coinciden. Esta verificación se realiza cuando valida el formulario llamando su método is_valid () . Puede proporcionar un método clean_ <fieldname> () a cualquier de los campos de su formulario para limpiar el valor o generar errores de validación del formulario para un campo específico. Los formularios también incluyen un método clean () general para validar el formulario, que es útil para validar campos que dependen unos de otros. En este caso, usted use la validación clean_password2 () específica del campo en lugar de anular la método clean () del formulario. Esto evita anular otras verificaciones específicas de campo que la ModelForm obtiene de las restricciones establecidas en el modelo (por ejemplo, la validación que el nombre de usuario es único).

Django también proporciona un formulario UserCreationForm que puede usar, que reside en django.contrib.auth.forms y es muy similar al que ha creado.

Edite el archivo views.py de la aplicación de la cuenta y agregue el siguiente código:

```
desde .forms importar LoginForm, UserRegistrationForm

def registrar (solicitud):
    if request.method == 'POST':
        user_form = UserRegistrationForm (solicitud.POST)
        si user_form.is_valid ():
            # Cree un nuevo objeto de usuario pero evite guardarlo todavía
            new_user = user_form.save (commit = False)
            # Establecer la contraseña elegida
            new_user.set_password (
                user_form.cleaned_data ['contraseña'])
            # Guardar el objeto Usuario
            new_user.save ()
            return render (solicitud,
                           'cuenta / register_done.html',
                           {'new_user': new_user})
```

[116]

```
más:
    user_form = UserRegistrationForm ()
return render (solicitud,
```

```
'cuenta/registro.html'  
{user_form: user_form})
```

La vista para crear cuentas de usuario es bastante simple. Por razones de seguridad, en lugar de guardando la contraseña sin procesar ingresada por el usuario, usa el método `set_password()` del modelo de usuario que maneja el hash.

Ahora, edite el archivo `urls.py` de la aplicación de su cuenta y agregue lo siguiente Patrón de URL:

```
ruta('registro/', vistas.registro, nombre='registro'),
```

Finalmente, cree una nueva plantilla en el directorio de la cuenta / plantilla, asígnele el nombre `register.html` y haga que se vea de la siguiente manera:

```
{% extiende "base.html"%}  
  
{% block title%} Cree una cuenta {% endblock%}  
  
{% block content%}  
  <h1> Cree una cuenta </h1>  
  <p> Por favor, regístrese mediante el siguiente formulario: </p>  
  <método de formulario = "publicar">  
    {{user_form.as_p}}  
    {% csrf_token%}  
    <p> <input type = "submit" value = "Crear mi cuenta"> </p>  
  </form>  
{% endblock%}
```

Agregue un archivo de plantilla en el mismo directorio y asígnele el nombre `register_done.html`. Agregue el siguiente código:

```
{% extiende "base.html"%}  
  
{% block title%} Bienvenido {% endblock%}  
  
{% block content%}  
  <h1> ¡Bienvenido, {{new_user.first_name}}! </h1>  
  <p> Su cuenta se ha creado correctamente. Ahora puedes <a  
href = "{% url "login %}"> iniciar sesión </a>. </p>  
{% endblock%}
```

[117]

Figura 4.13: El formulario de creación de cuenta

Complete los detalles para un nuevo usuario y haga clic en el botón **CREAR MI CUENTA** . Si todos los campos son válidos, se creará el usuario y obtendrá el siguiente éxito mensaje:

[118]

Figura 4.14: La cuenta se creó correctamente en la página

Haga clic en el enlace de **inicio de sesión** e ingrese su nombre de usuario y contraseña para verificar que puede acceder a su cuenta.

También puede agregar un enlace al registro en su plantilla de inicio de sesión. Edite el `registration / login.html` plantilla y busque la siguiente línea:

```
<p> Por favor, utilice el siguiente formulario para iniciar sesión: </p>
```

Reemplácelo con lo siguiente:

```
<p> Por favor, utilice el siguiente formulario para iniciar sesión. Si no tienes un  
cuenta <a href="{% url 'register' %}"> regístrese aquí </a> </p>
```

Ha hecho que la página de registro sea accesible desde la página de inicio de sesión.

Ampliando el modelo de usuario

Cuando tenga que lidiar con cuentas de usuario, encontrará que el modelo de usuario de el marco de autenticación de Django es adecuado para casos comunes. sin embargo, el El modelo de usuario viene con campos muy básicos. Es posible que desee ampliarlo para incluir datos adicionales. La mejor forma de hacerlo es creando un modelo de perfil que contenga todos los campos adicionales y una relación uno a uno con el modelo de usuario de Django . Una relación uno a uno es similar a un campo `ForeignKey` con el parámetro `unique = Verdadero` . El reverso de la relación es un uno a uno implícito relación con el modelo relacionado en lugar de un administrador para múltiples elementos.

De cada lado de la relación, recupera un único objeto relacionado.

Edite el archivo `models.py` de la aplicación de su cuenta y agregue el siguiente código:

```
desde modelos de importación django.db
desde la configuración de importación django.conf

Perfil de clase (modelos.Modelo):
    usuario = modelos.OneToOneField (settings.AUTH_USER_MODEL,
                                    on_delete = modelos.CASCADE)
```

[119]

Construyendo un sitio web social

```
date_of_birth = modelos.DateField (en blanco = Verdadero, nulo = Verdadero)
photo = modelos.ImageField (upload_to = 'usuarios /% Y /% m /% d /',
                             en blanco = Verdadero)
```

```
def __str__ (yo):
    return f'Perfil del usuario {self.user.username}'
```

Para mantener su código genérico, use `get_user_model()` método para recuperar el modelo de usuario y el `AUTH_USER_MODEL` ajuste para referirse a él al definir la relación de un modelo con el modelo de usuario, en lugar de hacer referencia al modelo de usuario de autenticación directamente. Puede leer más información sobre esto en https://docs.djangoproject.com/en/3.0/topics/auth/customizing/#django.contrib.auth.get_user_model.

El campo de usuario uno a uno le permite asociar perfiles con usuarios. Tu usas `CASCADE` para el parámetro `on_delete` para que su perfil relacionado también se elimine cuando se elimina un usuario. El campo de la foto es un campo `ImageField`. Necesitaras instale la biblioteca Pillow para manejar imágenes. Instale Pillow ejecutando lo siguiente comando en su shell:

pip install Pillow == 7.0.0

Para permitir que Django sirva archivos multimedia cargados por usuarios con el desarrollo servidor, agregue la siguiente configuración al archivo `settings.py` de su proyecto:

```
MEDIA_URL = '/ media /'
MEDIA_ROOT = os.path.join (BASE_DIR, 'media /')
```

`MEDIA_URL` es la URL base utilizada para servir los archivos multimedia cargados por los usuarios, y `MEDIA_ROOT` es la ruta local donde residen. Construyes el camino de forma dinámica en relación con la ruta de su proyecto para hacer que su código sea más genérico.

Ahora, edite el archivo `urls.py` principal del proyecto de marcadores y modifique el código, como sigue:

```
desde django.contrib import admin
de la ruta de importación django.urls, incluir
desde la configuración de importación django.conf
desde django.conf.urls.static import static

urlpatterns = [
    ruta ('admin /', admin.site.urls),
    ruta ('cuenta /', incluir ('cuenta.urls')),
]
```

[120]

si la configuración.DEBUG:

```
urlpatterns += static(settings.MEDIA_URL,
                      document_root = settings.MEDIA_ROOT)
```

De esta forma, el servidor de desarrollo de Django se encargará de dar servicio a los medios archivos durante el desarrollo (es decir, cuando la configuración DEBUG se establece en True).

La función auxiliar static () es adecuada para el desarrollo, pero no para uso de producción. Django es muy ineficiente para servir estática archivos. Nunca sirva sus archivos estáticos con Django en una producción ambiente. Aprenderá a servir archivos estáticos en un entorno de producción en el *Capítulo 14 , Puesta en marcha* .

Abra el shell y ejecute el siguiente comando para crear la migración de la base de datos para el nuevo modelo:

```
python manage.py makemigrations
```

Obtendrá el siguiente resultado:

Migraciones para 'cuenta':

```
account / migrations / 0001_initial.py
- Crear perfil de modelo
```

A continuación, sincronice la base de datos con el siguiente comando:

```
python manage.py migrar
```

Verá una salida que incluye la siguiente línea:

Aplicando cuenta.0001_initial ... OK

Edite el archivo admin.py de la aplicación de la cuenta y registre el modelo de perfil en el sitio de administración, así:

```
desde django.contrib import admin
desde .models importar perfil

@ admin.register (Perfil)
class ProfileAdmin (admin.ModelAdmin):
    list_display = ['usuario', 'fecha_de_nacimiento', 'foto']
```



```

        si user_form.is_valid () y profile_form.is_valid (files = request.FILES):
            user_form.save ()
            profile_form.save ()
    más:
        user_form = UserEditForm (instancia = solicitud.usuario)
        profile_form = ProfileEditForm (
                                instancia = request.user.profile)

    return render (solicitud,
                  'cuenta / edit.html',
                  {'user_form': user_form,
                  'profile_form': profile_form})

```

Utiliza el decorador `login_required` porque los usuarios deben estar autenticados para editar su perfil. En este caso, está utilizando dos formularios modelo: `UserEditForm` para almacenar los datos del modelo de usuario incorporado y `ProfileEditForm` para almacenar el datos de perfil adicionales en el modelo de perfil personalizado . Para validar el enviado data, ejecuta el método `is_valid ()` de ambas formas. Si ambas formas contienen válidas datos, guarda ambos formularios, llamando al método `save ()` para actualizar el correspondiente objetos en la base de datos.

Agregue el siguiente patrón de URL al archivo `urls.py` de la aplicación de la cuenta :

```

ruta ('editar /', vistas.editar, nombre = 'editar'),

```

Finalmente, cree una plantilla para esta vista en `templates / account /` y asígnele el nombre `edit.html` . Agregue el siguiente código:

```

{% extiende "base.html"%}

{% block title%} Edite su cuenta {% endblock%}

```

[123]

Construyendo un sitio web social

```

{% block content%}
    <h1> Edite su cuenta </h1>
    <p> Puede editar su cuenta mediante el siguiente formulario: </p>
    <form method = "post" enctype = "multipart / form-data">
        {{user_form.as_p}}
        {{profile_form.as_p}}
        {% csrf_token%}
        <p> <input type = "submit" value = "Guardar cambios"> </p>
    </form>
{% endblock%}

```

En el código anterior, incluye `enctype = "multipart / form-data"` en su formulario para habilitar la carga de archivos. Utiliza un formulario HTML para enviar tanto el `user_form` como el formularios `profile_form` .

Registre un nuevo usuario desde la URL `http://127.0.0.1:8000/account/register/` y abra `http://127.0.0.1:8000/account/edit/` . Deberías ver el página siguiente:

Figura 4.16: El formulario de edición de perfil

[124]

También puede editar la página del panel e incluir enlaces al perfil de edición y cambiar las páginas de contraseña. Abra la plantilla `account / dashboard.html` y busque la siguiente línea:

```
<p> Bienvenido a su panel. </p>
```

Reemplace la línea anterior por la siguiente:

```
<p> Bienvenido a su panel de control. Puede <a href="{% url 'edit' %}"> editar  
su perfil </a> o <a href="{% url 'password_change' %}"> cambie su  
contraseña </a>. </p>
```

Los usuarios ahora pueden acceder al formulario para editar su perfil desde su panel de control. Abierto `http://127.0.0.1:8000/account/` en su navegador y pruebe el nuevo enlace para editar un perfil de usuario:

Figura 4.17: Contenido de la página del panel, incluidos enlaces para editar un perfil y cambiar una contraseña

Usando un modelo de usuario personalizado

Django también ofrece una forma de sustituir todo el modelo de usuario con su propio modelo personalizado. Su clase de usuario debe heredar de la clase `AbstractUser` de Django, que proporciona la implementación completa del usuario predeterminado como modelo abstracto. Usted puede leer más sobre este método en <https://docs.djangoproject.com/en/3.0/topics/auth/customizing/#sustituyendo-un-modelo-de-usuario-personalizado>.

El uso de un modelo de usuario personalizado le dará más flexibilidad, pero también puede resultar en una integración más difícil con aplicaciones conectables que interactúan con Django modelo de usuario de autenticación.

Usando el marco de mensajes

Al permitir que los usuarios interactúen con su plataforma, hay muchos casos en los que es posible que desee informarles sobre el resultado de sus acciones. Django tiene un marco de mensajes incorporado que le permite mostrar notificaciones únicas a sus usuarios.

Construyendo un sitio web social

El marco de mensajes se encuentra en `django.contrib.messages` y está incluido en la lista `INSTALLED_APPS` predeterminada del archivo `settings.py` cuando crea un nuevo proyectos que utilizan `python manage.py startproject`. Notarás que tu configuración El archivo contiene un middleware llamado `django.contrib.messages.middleware.MessageMiddleware` en la configuración de `MIDDLEWARE`.

El marco de mensajes proporciona una forma sencilla de agregar mensajes a los usuarios. Los mensajes se almacenan en una cookie de forma predeterminada (recurriendo al almacenamiento de la sesión) y se muestran en la siguiente solicitud del usuario. Puedes usar los mensajes framework en sus vistas importando el módulo de mensajes y agregando nuevos mensajes con accesos directos sencillos, de la siguiente manera:

```
desde django.contrib importar mensajes
messages.error (solicitud, 'Algo salió mal')
```

Puede crear mensajes nuevos usando el método `add_message()` o cualquiera de los siguientes métodos de acceso directo:

- `exit()` : mensajes de éxito que se mostrarán después de que se haya realizado una acción. exitoso
- `info()` : mensajes informativos
- `warning()` : algo aún no ha fallado, pero puede fallar de forma inminente
- `error()` : una acción no tuvo éxito o algo falló
- `debug()` : mensajes de depuración que se eliminarán o ignorarán en una producción ambiente

Agreguemos mensajes a su plataforma. Dado que se aplica el marco de mensajes globalmente al proyecto, puede mostrar mensajes para el usuario en su plantilla base. Abra la plantilla `base.html` de la aplicación de la cuenta y agregue lo siguiente código entre el elemento `<div>` con el ID del encabezado y el elemento `<div>` con el ID de contenido :

```
{% if messages %}
<ul class = "mensajes">
    {% para mensaje en mensajes %}
    <li class = "{{message.tags}}">
        {{mensaje | seguro}}
        <a href="#" class="close"> x </a>
    </li>
    {% endfor %}
</ul>
{% terminara si %}
```

El marco de mensajes incluye el procesador de contexto `django.contrib`.

`messages.context_processors.messages`, que agrega una variable de mensajes al contexto de la solicitud. Puede encontrarlo en la lista `context_processors` del

PLANTILLAS de configuración de su proyecto. Puede utilizar la variable de mensajes en su plantilla para mostrar todos los mensajes existentes al usuario.

Un procesador de contexto es una función de Python que toma la solicitud objeto como argumento y devuelve un diccionario que se agrega a el contexto de la solicitud. Aprenderás a crear tu propio contexto. procesadores en el *Capítulo 7, Creación de una tienda en línea*.

Modifiquemos su vista de edición para usar el marco de mensajes. Edita `views.py` archivo de la aplicación de la cuenta, importar mensajes y hacer que la vista de edición se vea como sigue:

desde `django.contrib` importar mensajes

```
@Necesario iniciar sesión
def editar (solicitud):
    if request.method == 'POST':
        # ...
        si user_form.is_valid () y profile_form.is_valid ():
            user_form.save ()
            profile_form.save ()
            messages.success (solicitud, 'Perfil actualizado' \
                             'exitosamente')

        más:
            messages.error (solicitud, 'Error al actualizar su perfil')

    más:
        user_form = UserEditForm (instancia = solicitud.usuario)
        # ...
```

Agrega un mensaje de éxito cuando el usuario actualiza correctamente su perfil. Si alguna de los formularios contienen datos no válidos, agregue un mensaje de error en su lugar.

[127]

Construyendo un sitio web social

Abra `http://127.0.0.1:8000/account/edit/` en su navegador y edite su perfil. Cuando el perfil se actualice correctamente, debería ver lo siguiente mensaje:

Figura 4.18: Mensaje de edición exitosa del perfil

Cuando los datos no son válidos, por ejemplo, si hay una fecha formateada incorrectamente para

el campo **de fecha de nacimiento** , debería ver el siguiente mensaje:

Figura 4.19: Mensaje de error al actualizar el perfil

Puede obtener más información sobre el marco de mensajes en [https:// docs.django project.com/en/3.0/ref/contrib/messages/](https://docs.djangoproject.com/en/3.0/ref/contrib/messages/) .

Construyendo una autenticación personalizada backend

Django te permite autenticarte con diferentes fuentes. La `AUTENTICACIÓN_`

La configuración de `BACKENDS` incluye la lista de backends de autenticación para su proyecto. Por predeterminado, esta configuración se establece de la siguiente manera:

```
['django.contrib.auth.backends.ModelBackend']
```

El `ModelBackend` predeterminado autentica a los usuarios contra la base de datos utilizando el usuario modelo de `django.contrib.auth` . Esto se adaptará a la mayoría de sus proyectos. Sin embargo, puede crear backends personalizados para autenticar a su usuario frente a otras fuentes, como un **directorio de Protocolo ligero de acceso a directorios (LDAP)** o cualquier otro sistema.

[128]

Puede leer más información sobre cómo personalizar la autenticación en [https:// docs.django project.com/en/3.0/topics/auth/customizing/#other-fuentes de autenticación](https://docs.djangoproject.com/en/3.0/topics/auth/customizing/#other-fuentes-de-autenticación) .

Siempre que use la función `authenticate()` de `django.contrib.auth` , Django intenta autenticar al usuario contra cada uno de los backends definidos en `AUTHENTICATION_`

`BACKENDS` uno por uno, hasta que uno de ellos autentique exitosamente al usuario. Solo si todos de los backends no se autentican si el usuario no se autenticará en su sitio.

Django proporciona una forma sencilla de definir sus propios backends de autenticación. Un backend de autenticación es una clase que proporciona los dos métodos siguientes:

- `authenticate()` : toma el objeto de solicitud y las credenciales de usuario como parámetros. Tiene que devolver un objeto de usuario que coincida con esas credenciales, si las credenciales son válidas, o `None` en caso contrario. El parámetro de solicitud es un objeto `HttpRequest` , o `None` si no se proporciona para `authenticate()` .
- `get_user()` : toma un parámetro de ID de usuario y debe devolver un objeto de usuario .

Crear un backend de autenticación personalizado es tan simple como escribir una clase de Python que implementa ambos métodos. Creemos un backend de autenticación para permitir a los usuarios autenticarse en su sitio utilizando su dirección de correo electrónico en lugar de su nombre de usuario.

Cree un nuevo archivo dentro del directorio de aplicaciones de su cuenta y asígnele un nombre `autenticación.py` . Agregue el siguiente código:

```
desde django.contrib.auth.models import User

class EmailAuthBackend (objeto):
    """ """
```

```

Autentíquese con una dirección de correo electrónico.
"""
def autenticar (auto, solicitud, nombre de usuario = Ninguno, contraseña = Ninguno):
    tratar:
        usuario = User.objects.get (correo electrónico = nombre de usuario)
        if user.check_password (contraseña):
            usuario de retorno
            regresar Ninguno
        excepto User.DoesNotExist:
            regresar Ninguno

def get_user (self, user_id):
    tratar:
        return User.objects.get (pk = user_id)
    excepto User.DoesNotExist:
        regresar Ninguno

```

[129]

Construyendo un sitio web social

El código anterior es un backend de autenticación simple. El autenticar () el método recibe un objeto de solicitud y el nombre de usuario y la contraseña son opcionales parámetros. Puede usar diferentes parámetros, pero usa nombre de usuario y contraseña para que su backend funcione con las vistas del marco de autenticación inmediatamente. El código anterior funciona de la siguiente manera:

- authenticate () : intenta recuperar un usuario con la dirección de correo electrónico proporcionada y verifique la contraseña usando el método check_password () incorporado de el modelo de usuario. Este método maneja el hash de contraseña para comparar la contraseña dada con la contraseña almacenada en la base de datos.
- get_user () : obtiene un usuario a través del ID proporcionado en el user_id parámetro. Django usa el backend que autenticó al usuario para recuperar el objeto Usuario durante la duración de la sesión del usuario.

Edite el archivo settings.py de su proyecto y agregue la siguiente configuración:

```

AUTHENTICATION_BACKENDS = [
    'django.contrib.auth.backends.ModelBackend',
    'account.authentication.EmailAuthBackend',
]

```

En la configuración anterior, mantiene el ModelBackend predeterminado que se utiliza para autenticarse con el nombre de usuario y la contraseña e incluir su propio correo electrónico backend de autenticación.

Ahora abra <http://127.0.0.1:8000/account/login/> en su navegador. Recuerda que Django intentará autenticar al usuario contra cada uno de los backends, así que ahora debería poder iniciar sesión sin problemas con su nombre de usuario o cuenta de correo electrónico. Las credenciales de usuario se comprobarán mediante el backend de autenticación ModelBackend , y si no se devuelve ningún usuario, las credenciales se verificarán usando su personalizado Backend de EmailAuthBackend .

El orden de los backends enumerados en AUTHENTICATION_BACKENDS el entorno importa. Si las mismas credenciales son válidas para múltiples backends, Django se detendrá en el primer backend que autentica al usuario.

Agregar autenticación social a su sitio

Es posible que también desee agregar autenticación social a su sitio mediante servicios como Facebook, Twitter o Google. Python Social Auth es un módulo de Python que simplifica el proceso de agregar autenticación social a su sitio web. Con este módulo,

puede permitir que sus usuarios inicien sesión en su sitio web utilizando sus cuentas de otros servicios.

[130]

Capítulo 4

La autenticación social es una característica ampliamente utilizada que hace que el proceso de autenticación más fácil para los usuarios. Puede encontrar el código de este módulo en <https://github.com/Python-social-auth>.

Este módulo viene con backends de autenticación para diferentes marcos de Python, incluido Django. Para instalar el paquete Django a través de pip, abre la consola y ejecuta el siguiente comando:

```
pip install social-auth-app-django == 3.1.0
```

Luego agregue social_django a la configuración INSTALLED_APPS en el archivo settings.py de su proyecto:

```
INSTALLED_APPS = [  
    # ...  
    'social_django',  
]
```

Esta es la aplicación predeterminada para agregar Python Social Auth a proyectos de Django. Ahora ejecute el siguiente comando para sincronizar los modelos de autenticación social de Python con su base de datos:

```
python manage.py migrar
```

Debería ver que las migraciones para la aplicación predeterminada se aplican de la siguiente manera:

```
Aplicando social_django.0001_initial ... OK
```

```
Aplicando social_django.0002_add_related_name ... OK
```

```
...
```

```
Aplicando social_django.0008_partial_timestamp ... OK
```

Python Social Auth incluye backends para múltiples servicios. Puedes ver una lista de todos los backends en <https://python-social-auth.readthedocs.io/en/latest/backends/index.html#compatibles-backends>.

Incluyamos backends de autenticación para Facebook, Twitter y Google.

Deberá agregar patrones de URL de inicio de sesión social a su proyecto. Abre las URL principales .

py del proyecto de marcadores e incluya los patrones de URL social_django como sigue:

```
urlpatterns = [  
    ruta('admin/', admin.site.urls),  
    ruta('cuenta/', incluir('cuenta.urls')),  
    ruta('social-auth/',  
        include('social_django.urls', namespace='social')),  
]
```

[131]

Varios servicios sociales no permitirán redirigir a los usuarios a 127.0.0.1 o localhost después de una autenticación exitosa; esperan un nombre de dominio. Para poder hacer trabajo de autenticación social, necesitará un dominio. Para solucionar este problema en Linux o macOS, edite su archivo / etc / hosts y agregue la siguiente línea:

```
127.0.0.1 mysite.com
```

Esto le dirá a su computadora que apunte el nombre de host de mysite.com al suyo máquina. Si está utilizando Windows, su archivo de hosts se encuentra en C: \ Windows \ System32 \ Drivers \ etc \ hosts .

Para verificar que su asociación de nombre de host funcionó, inicie el servidor de desarrollo con python manage.py runserver y abra http://mysite.com:8000/ cuenta / iniciar sesión / en su navegador. Verá el siguiente error:

Figura 4.20: El mensaje de encabezado de host no válido

Django controla los hosts que pueden servir su aplicación usando ALLOWED_

Configuración de HOSTS . Esta es una medida de seguridad para evitar ataques de encabezado de host HTTP. Django solo permitirá que los hosts incluidos en esta lista sirvan la aplicación. Tú puede obtener más información sobre la configuración ALLOWED_HOSTS en <https://docs.djangoproject.com/es/3.0/ref/settings/#permitidos-hosts>.

Edite el archivo settings.py de su proyecto y edite la configuración ALLOWED_HOSTS como sigue:

```
ALLOWED_HOSTS = [ 'misitio.com', 'localhost', '127.0.0.1' ]
```

Además del host mysite.com , incluye explícitamente localhost y 127.0.0.1 .

Esto es para poder acceder al sitio a través de localhost , que es el predeterminado

Comportamiento de Django cuando DEBUG es True y ALLOWED_HOSTS está vacío. Ahora deberías poder abrir http://mysite.com:8000/account/login/ en su navegador.

Ejecutando el servidor de desarrollo a través HTTPS

Algunos de los métodos de autenticación social que va a utilizar requieren un HTTPS conexión. El **protocolo Transport Layer Security (TLS)** es el estándar para servir sitios web a través de una conexión segura. El predecesor de TLS es **Secure Sockets Capa (SSL)**.

[132]

Aunque SSL ahora está en desuso, en varias bibliotecas y documentación en línea encontrará referencias a los términos TLS y SSL. El desarrollo de Django el servidor no puede servir su sitio a través de HTTPS, ya que no es su intención utilizar. Para probar la funcionalidad de autenticación social que ofrece su sitio a través de HTTPS, vas a usar la extensión RunServerPlus del paquete Django Extensiones. Django Extensions es una colección de extensiones personalizadas de terceros para Django. Tenga en cuenta que este nunca es el método que debe utilizar para publicar su sitio. en un entorno real; este es un servidor de desarrollo.

Utilice el siguiente comando para instalar las extensiones de Django:

```
pip install django-extensions == 2.2.5
```

Ahora necesita instalar Werkzeug, que contiene una capa de depuración requerida por la extensión RunServerPlus. Utilice el siguiente comando para instalarlo:

```
pip install werkzeug == 0.16.0
```

Finalmente, use el siguiente comando para instalar pyOpenSSL, que se requiere para usar la funcionalidad SSL / TLS de RunServerPlus:

```
pip install pyOpenSSL == 19.0.0
```

Edite el archivo settings.py de su proyecto y agregue extensiones de Django al

Configuración INSTALLED_APPS, de la siguiente manera:

```
INSTALLED_APPS = [  
    # ...  
    'django_extensions',  
]
```

Utilice el comando de administración runserver_plus proporcionado por Django Extensions para ejecutar el servidor de desarrollo, de la siguiente manera:

```
python manage.py runserver_plus --cert-file cert.crt
```

Proporciona un nombre de archivo al comando runserver_plus para SSL / TLS certificado. Las extensiones de Django generarán una clave y un certificado automáticamente.

Abra <https://mysite.com:8000/account/login/> en su navegador. Ahora tu estás accediendo a su sitio a través de HTTPS. Su navegador puede mostrar una seguridad advertencia porque está utilizando un certificado autogenerado. Si este es el caso, acceda la información avanzada que muestra su navegador y acepta el autofirmado certificado para que su navegador confíe en el certificado.

[133]

Construyendo un sitio web social

Verá que la URL comienza con https:// y un icono de seguridad que indica que la conexión es segura.

4.21 La URL con el icono de conexión segura

Ahora puede publicar su sitio a través de HTTPS durante el desarrollo para probar autenticación social con Facebook, Twitter y Google.

Autenticación mediante Facebook

Para usar la autenticación de Facebook para iniciar sesión en su sitio, agregue la siguiente línea al AUTHENTICATION_BACKENDS en el archivo settings.py de su proyecto:

```
'social_core.backends.facebook.FacebookOAuth2',
```

Necesitará una cuenta de desarrollador de Facebook y deberá crear una nueva Aplicación de Facebook. Abierto <https://developers.facebook.com/apps/>, en su navegador. Después de crear una cuenta de desarrollador de Facebook, verá un sitio con la siguiente encabezado:

Figura 4.22: Menú del portal de desarrolladores de Facebook

En el elemento de menú **Mis aplicaciones** , haga clic en el botón **Crear aplicación** . Verás el siguiente formulario para crear una nueva ID de aplicación:

[134]

Figura 4.23: El formulario de ID de aplicación de creación de Facebook

Introduzca Marcadores como **Nombre para mostrar** , agregue una dirección de correo electrónico de contacto y haga clic en **Crear ID de aplicación** . Verá un panel para su nueva aplicación que muestra diferentes funciones que puede configurar para ello. Busque el siguiente cuadro de **inicio de sesión de Facebook** y haga clic en **Configurar** :

Figura 4.24: El bloque de producto de inicio de sesión de Facebook

Construyendo un sitio web social

Se le pedirá que elija la plataforma, de la siguiente manera:

Figura 4.25: Selección de plataforma para iniciar sesión en Facebook

Seleccione la plataforma **web** . Verá el siguiente formulario:

Figura 4.26: Configuración de la plataforma web para iniciar sesión en Facebook

Ingrese `http://mysite.com:8000/` como la **URL de su sitio** y haga clic en el botón **Guardar** . Puede omitir el resto del proceso de inicio rápido. En el menú de la izquierda, haga clic en **Configuración** y luego en **Básico** . Verá algo similar a lo siguiente:

Figura 4.27: Detalles de la aplicación para la aplicación de Facebook

Copie el **ID de la aplicación** y las claves **secretas de la aplicación** y agréguelas al archivo settings.py de su proyecto, de la siguiente manera:

```
SOCIAL_AUTH_FACEBOOK_KEY = 'XXX' # ID de la aplicación de Facebook
SOCIAL_AUTH_FACEBOOK_SECRET = 'XXX' # Secreto de la aplicación de Facebook
```

Opcionalmente, puede definir una configuración SOCIAL_AUTH_FACEBOOK_SCOPE con el extra permisos que desea solicitar a los usuarios de Facebook:

```
SOCIAL_AUTH_FACEBOOK_SCOPE = ['correo electrónico']
```

Ahora, regrese a Facebook y haga clic en **Configuración**. Verá un formulario con múltiples configuración para su aplicación. Agregue mysite.com en **App Domains**, de la siguiente manera:

Figura 4.28: Dominios permitidos para la aplicación de Facebook

Haga clic en **Guardar cambios**. Luego, en el menú de la izquierda en **Productos**, haga clic en **Inicio de sesión de Facebook** y luego **Configuración**, como se muestra aquí:

Figura 4.29: El menú de inicio de sesión de Facebook

Asegúrese de que solo estén activos los siguientes ajustes:

- **Inicio de sesión de OAuth del cliente**
- **Inicio de sesión en Web OAuth**
- **Hacer cumplir HTTPS**
- **Inicio de sesión de OAuth del navegador integrado**

Figura 4.30: Configuración de OAuth del cliente para el inicio de sesión de Facebook

Abra la plantilla registration / login.html de su aplicación de cuenta y agregue el siguiente código en la parte inferior del bloque de contenido :

```
<div class = "social">
  <ul>
    <li class = "facebook">
      <a href="{% url 'social:begin' 'facebook' %}"> Iniciar sesión con
      Facebook </a>
    </li>
  </ul>
</div>
```

Abra <https://mysite.com:8000/account/login/> en su navegador. Ahora el La página de inicio de sesión se verá de la siguiente manera:

[138]

Figura 4.31: La página de inicio de sesión que incluye el botón para la autenticación de Facebook

Haga clic en el botón **Iniciar sesión con Facebook** . Serás redirigido a Facebook,

y verá un cuadro de diálogo modal que le pedirá permiso para permitir que los *marcadores* aplicación acceda a su perfil público de Facebook:

Figura 4.32: El cuadro de diálogo modal para otorgar permisos a la aplicación

[139]

Construyendo un sitio web social

Haga clic en el botón **Continuar como** . Se iniciará sesión y se le redirigirá a la página del panel de control de su sitio. Recuerde que ha configurado esta URL en el LOGIN_ Configuración de REDIRECT_URL . Como puede ver, agregar autenticación social a su sitio es muy claro.

Autenticación mediante Twitter

Para la autenticación social usando Twitter, agregue la siguiente línea al AUTHENTICATION_BACKENDS en el archivo settings.py de su proyecto:

```
'social_core.backends.twitter.TwitterOAuth',
```

Deberá crear una nueva aplicación en su cuenta de Twitter. Abrir [https:// developer.twitter.com/en/apps/create](https://developer.twitter.com/en/apps/create) en su navegador. Te harán preguntas varias preguntas para crear una cuenta de desarrollador de Twitter si aún no lo ha hecho. Una vez que tenga una cuenta de desarrollador, al crear una nueva aplicación, verá el siguiente formulario:

Figura 4.33: Configuración de la aplicación de Twitter

[140]

Ingrese los detalles de su aplicación, incluida la siguiente configuración:

- **Sitio web** : `https://mysite.com:8000/`
- **URL de devolución de llamada** : `https://mysite.com:8000/social-auth/complete/gorjeo/`

Asegúrese de activar **Habilitar inicio de sesión con Twitter** . Luego, haga clic en **Crear** . Verá los detalles de la aplicación. Haga clic en la pestaña **Llaves y fichas** . Debieras consulte la siguiente información:

Figura 4.34: Claves API de la aplicación Twitter

Copie la **clave API** y la **clave secreta API** en la siguiente configuración en `settings.py` archivo de su proyecto:

```
SOCIAL_AUTH_TWITTER_KEY = 'XXX' # Clave de API de Twitter
SOCIAL_AUTH_TWITTER_SECRET = 'XXX' # Secreto de la API de Twitter
```

Ahora edite la plantilla `registration / login.html` y agregue el siguiente código a el elemento `<ul>` :

```
<li class = "twitter">
  <a href="{% url "social:begin" "twitter" %}"> Iniciar sesión con Twitter </a>
</li>
```

[141]

Construyendo un sitio web social

Abra <https://mysite.com:8000/account/login/> en su navegador y haga clic en el enlace **Iniciar sesión con Twitter**. Serás redirigido a Twitter y te preguntará autoriza la aplicación de la siguiente manera:

Figura 4.35: El cuadro de diálogo modal para otorgar permisos a la aplicación

Haga clic en el botón **Autorizar aplicación**. Se iniciará sesión y se le redirigirá a la página del panel de control de su sitio.

Autenticación mediante Google

Google ofrece autenticación social mediante OAuth2. Puede leer acerca de Google Implementación de OAuth2 en <https://developers.google.com/identity/protocolos/OAuth2>.

Para implementar la autenticación con Google, agregue la siguiente línea al AUTHENTICATION_BACKENDS en el archivo settings.py de su proyecto:

```
'social_core.backends.google.GoogleOAuth2',
```

Primero, deberá crear una clave API en su Consola de desarrollador de Google. Abierto <https://console.developers.google.com/apis/credentials> en su navegador.

Haga clic en **Seleccionar un proyecto** y luego en **Nuevo proyecto** cree un nuevo proyecto, de la siguiente manera:

[142]

Figura 4.36: El formulario de creación de proyectos de Google

Una vez creado el proyecto, en **Credenciales**, haga clic en **CREAR CREDENCIALES** y elija el **ID de cliente de OAuth**, de la siguiente manera:

Figura 4.37: Creación de credenciales de API de Google API

Google le pedirá que configure la pantalla de consentimiento primero:

Figura 4.38: La alerta para configurar la pantalla de consentimiento de OAuth

[143]

Complete el formulario con la siguiente información:

- **Nombre de la aplicación** : introduzca marcadores
- **Dominios autorizados** : ingrese mysite.com

La pantalla debería verse así:

Figura 4.40: Configuración de la pantalla de consentimiento de Google OAuth

[144]

Haga clic en el botón Guardar. Se configurará la pantalla de consentimiento para su aplicación y verá los detalles de la pantalla de consentimiento de su solicitud, de la siguiente manera:

Figura 4.41: Detalles de la pantalla de consentimiento de Google OAuth

En el menú de la barra lateral izquierda, haga clic en **Credenciales** y vuelva a hacer clic en **CREAR**.

CREDENCIALES y luego en el **ID de cliente de OAuth**.

Como siguiente paso, ingrese la siguiente información:

- **Tipo de aplicación** : seleccione la **aplicación web**
- **Nombre** : introduzca marcadores
- **URI de redireccionamiento autorizado** : agregue `https://mysite.com:8000/social-auth/complete / google-oauth2 /`

[145]

Construyendo un sitio web social

El formulario debería verse así:

Figura 4.42: El formulario de creación de la aplicación de Google

Haga clic en el botón **Crear** . Obtendrá la **identificación del cliente** y las claves **secretas del cliente** . Agréguelos a su archivo `settings.py` , así:

```
SOCIAL_AUTH_GOOGLE_OAUTH2_KEY = 'XXX' # Clave de consumidor de Google  
SOCIAL_AUTH_GOOGLE_OAUTH2_SECRET = 'XXX' # Secreto del consumidor de Google
```

En el menú de la izquierda de Google Developers Console, debajo de **API y servicios** sección, haga clic en el elemento **Biblioteca** . Verá una lista que contiene todas las API de Google. Haga clic en la **API de Google+** y luego haga clic en el botón **HABILITAR** en la siguiente página:

[146]

Figura 4.43: El bloque de API de Google+

Edite la plantilla `registration / login.html` y agregue el siguiente código al `<ul>` elemento:

```
<li class = "google">  
  <a href="{% url "social:begin" "google-oauth2" %}"> Iniciar sesión con  
  Google </a>  
</li>
```

Abra `https://mysite.com:8000/account/login/` en su navegador. El inicio de sesión
La página ahora debería verse como sigue:

Figura 4.44: La página de inicio de sesión que incluye botones para la autenticación de Twitter y Google

Haga clic en el botón **Iniciar sesión con Google** . Se iniciará sesión y se le redirigirá a la página del panel de control de su sitio web.

Ahora ha agregado autenticación social a su proyecto. Puede implementar fácilmente autenticación social con otros servicios en línea populares que utilizan Python Social Auth.

Resumen

En este capítulo, aprendió cómo crear un sistema de autenticación en su sitio.

Implementó todas las vistas necesarias para que los usuarios se registren, inicien sesión, cierren sesión, edite su contraseña y restablezca su contraseña. Construyó un modelo para usuario personalizado perfiles y ha creado un backend de autenticación personalizado para permitir que los usuarios inicien sesión en su sitio utilizando su dirección de correo electrónico. También agregó autenticación social a su sitio para que los usuarios puedan usar su cuenta existente de Facebook, Twitter o Google para iniciar sesión.

En el siguiente capítulo, aprenderá a crear un sistema de marcadores de imágenes, generar miniaturas de imágenes y crear vistas AJAX.

en tu sitio web

En el capítulo anterior, creó el registro de usuario y la autenticación en su sitio web. Aprendió a crear un modelo de perfil personalizado para sus usuarios y para usted. agregó autenticación social a su sitio con las principales redes sociales.

En este capítulo, aprenderá a crear un marcador de JavaScript para compartir contenido de otros sitios en su sitio web, e implementará funciones AJAX en su proyecto usando jQuery y Django.

Este capítulo cubrirá los siguientes puntos:

- Crear relaciones de varios a varios
- Personalización del comportamiento de los formularios.
- Usando jQuery con Django
- Construyendo un bookmarklet jQuery
- Generación de miniaturas de imágenes usando miniaturas fáciles
- Implementar vistas AJAX e integrarlas con jQuery
- Creación de decoradores personalizados para vistas.
- Construyendo paginación AJAX

[149]

Compartir contenido en su sitio web

Creación de un sitio web de marcadores de imágenes

En este capítulo, aprenderá cómo permitir a los usuarios marcar y compartir imágenes que encuentran en otros sitios web y en su sitio. Para esto, necesitará hacer el siguientes tareas:

1. Defina un modelo para almacenar imágenes y su información
2. Cree un formulario y una vista para manejar la carga de imágenes.
3. Cree un sistema para que los usuarios puedan publicar imágenes que encuentren en dispositivos externos. sitios web

Primero, cree una nueva aplicación dentro de su directorio de proyecto de marcadores con el siguiente comando:

Imágenes de startapp de django-admin

Agregue la nueva aplicación a la configuración `INSTALLED_APPS` en el archivo `settings.py`, como sigue:

```
INSTALLED_APPS = [  
    # ...  
    'images.apps.ImagesConfig',  
]
```

Ha activado la aplicación de imágenes en el proyecto.

Construyendo el modelo de imagen

Edite el archivo `models.py` de la aplicación de imágenes y agregue el siguiente código:

```
desde modelos de importación django.db
```

desde la configuración de importación django.conf

Imagen de clase (modelos.Modelo):

```
usuario = modelos.ForeignKey (settings.AUTH_USER_MODEL,
                               related_name = 'imagenes_created',
                               on_delete = modelos.CASCADE)

title = modelos.CharField (longitud_máxima = 200)
slug = modelos.SlugField (max_length = 200,
                          en blanco = Verdadero)

url = modelos.URLField ()
image = modelos.ImageField (upload_to = 'imágenes /% Y /% m /% d /')
descripción = modelos.TextField (en blanco = verdadero)
creado = modelos.DateField (auto_now_add = True,
                             db_index = Verdadero)
```

[150]

```
def __str__ (yo):
    volver self.title
```

Este es el modelo que utilizará para almacenar imágenes recuperadas de diferentes sitios.

Echemos un vistazo a los campos de este modelo:

- usuario : esto indica el objeto Usuario que marcó esta imagen. Esto es un campo de clave externa porque especifica una relación de uno a varios: un usuario puede publicar varias imágenes, pero cada imagen es publicada por un solo usuario. Tú use CASCADE para el parámetro on_delete para que las imágenes relacionadas también sean eliminado cuando se elimina un usuario.
- título : título de la imagen.
- slug : una etiqueta corta que contiene solo letras, números, guiones bajos o guiones que se utilizarán para crear hermosas URL compatibles con SEO.
- url : la URL original de esta imagen.
- imagen : el archivo de imagen.
- descripción : una descripción opcional de la imagen.
- creado : la fecha y hora que indican cuándo se creó el objeto en la base de datos. Dado que utiliza auto_now_add , esta fecha y hora se se establece cuando se crea el objeto. Se utiliza db_index = True para que Django crea un índice en la base de datos para este campo.

Los índices de la base de datos mejoran el rendimiento de las consultas. Considere establecer db_index = True para los campos que consulta con frecuencia utilizando filter (), exclude () o order_by (). Campos ForeignKey o Los campos con unique = True implican la creación de un índice. Usted puede también use Meta.index_together o Meta.indexes para crear índices para varios campos. Puede obtener más información sobre la base de datos índices [en https://docs.djangoproject.com/en/3.0/ref/models/opciones/#django.db.models.Options.indexes](https://docs.djangoproject.com/en/3.0/ref/models/opciones/#django.db.models.Options.indexes).

Anulará el método save () del modelo de imagen para generar automáticamente el campo slug basado en el valor del campo de título . Importar la función slugify () y agregue un método save () al modelo de imagen , de la siguiente manera:

desde django.utils.text import slugify

Imagen de clase (modelos.Modelo):

```
# ...
def save (self, * args, ** kwargs):
    si no es self.slug:
        self.slug = slugify (self.title)
    super (). guardar (* argumentos, ** kwargs)
```

Compartir contenido en su sitio web

En el código anterior, usa la función `slugify()` proporcionada por Django para genera automáticamente la imagen slug para el título dado cuando no se proporciona ninguna slug. Luego, guarda el objeto. Al generar slugs automáticamente, los usuarios no tienen para ingresar manualmente un slug para cada imagen.

Crear relaciones de varios a varios

A continuación, agregará otro campo al modelo de imagen para almacenar los usuarios a los que les gusta un imagen. Necesitará una relación de varios a varios en este caso porque un usuario puede como varias imágenes y cada imagen puede gustar a varios usuarios.

Agregue el siguiente campo al modelo de imagen :

```
users_like = models.ManyToManyField(settings.AUTH_USER_MODEL,
                                   related_name='images_liked',
                                   en blanco = Verdadero)
```

Cuando define un `ManyToManyField`, Django crea una tabla de unión intermedia utilizando las claves primarias de ambos modelos. El `ManyToManyField` se puede definir de cualquiera de los dos modelos relacionados.

Al igual que con `ForeignKey` campos, la `related_name` atributos de `ManyToManyField` le permite nombrar la relación del objeto relacionado con este. los

Los campos `ManyToManyField` proporcionan un administrador de muchos a muchos que le permite recuperar objetos relacionados, como `image.users_like.all()`, u obtenerlos de un objeto de usuario, como `user.images_liked.all()`.

Puede obtener más información sobre las relaciones de varios a varios en [https:// docs. django project. com/ en/ 3. 0/ topics/ db/ examples/ many_ to_ many/](https://docs.djangoproject.com/en/3.0/topics/db/examples/many_to_many/).

Abra la línea de comando y ejecute el siguiente comando para crear una inicial migración:

imágenes de makemigrations de python manage.py

Debería ver el siguiente resultado:

Migraciones de 'imágenes':

```
images / migrations / 0001_initial.py
- Crear imagen de modelo
```

Ahora ejecute el siguiente comando para aplicar su migración:

python manage.py migrar imágenes

Obtendrá una salida que incluye la siguiente línea:

```
Aplicando imágenes.0001_initial ... OK
```

El modelo de imagen ahora está sincronizado con la base de datos.

Registrar el modelo de imagen en el sitio de administración

Edite el archivo `admin.py` de la aplicación de imágenes y registre el modelo de imagen en el sitio de administración, de la siguiente manera:

```
desde django.contrib import admin
desde .models importar imagen

@ admin.register (Imagen)
clase ImageAdmin (admin.ModelAdmin):
    list_display = ['título', 'slug', 'imagen', 'creado']
    list_filter = ['creado']
```

Inicie el servidor de desarrollo con el siguiente comando:

```
python manage.py runserver_plus --cert-file cert.crt
```

Abra `https://127.0.0.1:8000/admin/` en su navegador y verá el Modelo de imagen en el sitio de administración, así:

Figura 5.1: El bloque IMÁGENES en la página de índice del sitio de administración de Django

Publicar contenido de otros sitios web

Permitirá que los usuarios marquen imágenes de sitios web externos. El usuario proporcione la URL de la imagen, un título y una descripción opcional. Su aplicación descargará la imagen y creará un nuevo objeto `Image` en la base de datos.

Comencemos por crear un formulario para enviar nuevas imágenes. Crea un nuevo `forms.py` archivo dentro del directorio de la aplicación Imágenes y agregue el siguiente código:

```
desde formularios de importación de django
```

[153]

Compartir contenido en su sitio web

```
desde .models importar imagen
```

```
clase ImageCreateForm (formularios.ModelForm):
    clase Meta:
        modelo = Imagen
        campos = ('título', 'url', 'descripción')
        widgets = {
            'url': formularios.HiddenInput,
```

Como notará en el código anterior, este formulario es un formulario `ModelForm` construido a partir de el modelo de imagen , incluidos solo los campos de título , URL y descripción . Usuarios

no ingresará la URL de la imagen directamente en el formulario. En cambio, les proporcionará con una herramienta de JavaScript para elegir una imagen de un sitio externo, y su formulario recibe su URL como parámetro. Anula el widget predeterminado del campo de URL para utilice un widget `HiddenInput`. Este widget se representa como un elemento de entrada HTML con un atributo `type = "hidden"`. Usas este widget porque no quieres esto campo para que sea visible para los usuarios.

Campos de formulario de limpieza

Para verificar que la URL de la imagen proporcionada es válida, comprobará que la El nombre de archivo termina con una extensión `.jpg` o `.jpeg` para permitir solo archivos JPEG. Como viste en el capítulo anterior, Django le permite definir métodos de formulario para limpiar específicos campos usando la convención `clean_<nombrcampo>()`. Este método se ejecuta para cada campo, si está presente, cuando llama a `is_valid()` en una instancia de formulario. En el método `limpio`, puede modificar el valor del campo o generar errores de validación para este campo específico cuando sea necesario. Agregue el siguiente método a `ImageCreateForm`:

```
def clean_url (yo):
    url = self.cleaned_data ['url']
    valid_extensions = ['jpg', 'jpeg']
    extensión = url.rsplit ('.', 1) [1].lower ()
    si la extensión no está en valid_extensions:
        subir formularios.ValidationError ('La URL dada no lo hace' \
                                          'coincidir con extensiones de imagen válidas').

    devolver URL
```

En el código anterior, define un método `clean_url()` para limpiar el campo de la URL. Los el código funciona de la siguiente manera:

1. Obtiene el valor del campo URL al acceder al diccionario `clean_data` de la instancia de formulario.

[154]

2. Divide la URL para obtener la extensión del archivo y comprueba si es una de las extensiones válidas. Si la extensión no es válida, genera `ValidationError` y la instancia del formulario no se validará. Aquí, estás realizando una muy validación simple. Puede utilizar métodos más avanzados para comprobar si la URL proporcionada proporciona un archivo de imagen válido.

Además de validar la URL dada, también necesita descargar el archivo de imagen y guárdalo. Podría, por ejemplo, usar la vista que maneja el formulario para descargar el archivo de imagen. En su lugar, adoptemos un enfoque más general anulando `save()` método de su formulario modelo para realizar esta tarea cada vez que se guarda el formulario.

Anulando el método `save()` de un `ModelForm`

Como sabe, `ModelForm` proporciona un método `save()` para guardar el modelo actual instancia a la base de datos y devolver el objeto. Este método recibe un booleano

parámetro de confirmación, que le permite especificar si el objeto debe persistir a la base de datos. Si la confirmación es falsa, el método `save()` devolverá una instancia de modelo pero no lo guardará en la base de datos. Anulará el método `save()` de su formulario para recuperar la imagen dada y guardarla.

Agregue las siguientes importaciones en la parte superior del archivo `forms.py`:

```
desde la solicitud de importación urllib
desde django.core.files.base import ContentFile
desde django.utils.text import slugify
```

Luego, agregue el siguiente método `save()` al formulario `ImageCreateForm`:

```

def save (self, force_insert = False,
          force_update = False,
          cometer = Verdadero):
    imagen = super (). guardar (confirmar = falso)
    image_url = self.cleaned_data ['url']
    nombre = slugify (image.title)
    extensión = image_url.rsplit ('.', 1) [1] .lower ()
    image_name = f' {nombre}. {extensión}'

    # descargar imagen de la URL dada
    respuesta = request.urlopen (image_url)
    image.image.save (image_name,
                      ContentFile (response.read ()),
                      guardar = falso)

    si comete:
        image.save ()
    devolver imagen

```

[155]

Compartir contenido en su sitio web

Anula el save () método, manteniendo los parámetros requeridos por ModelForm .

El código anterior se puede explicar de la siguiente manera:

1. Creas una nueva instancia de imagen llamando al método save () del formulario.
con commit = False .
2. Obtiene la URL del diccionario clean_data del formulario.
3. El nombre de la imagen se genera combinando el slug del título de la imagen con el
extensión de archivo original.
4. Utiliza el módulo urllib de Python para descargar la imagen y luego llama al
método save () del campo de imagen, pasándole un objeto ContentFile que es
instanciado con el contenido del archivo descargado. De esta forma, guarda el archivo
al directorio de medios de su proyecto. Se pasa el Falso Save = parámetro
para evitar guardar el objeto en la base de datos todavía.
5. Para mantener el mismo comportamiento que el método save () que anula,
guarda el formulario en la base de datos solo cuando el parámetro de confirmación es Verdadero .

Para utilizar el urllib para recuperar imágenes de URL servidas a través de HTTPS,
necesita instalar el paquete Certifi Python. Certifi es una colección de root
certificados para validar la confiabilidad de los certificados SSL / TLS.

Instale certifi con el siguiente comando:

instalar pip - actualizar certifi

Necesitará una vista para manejar el formulario. Edite el archivo views.py de las imágenes
aplicación y agregue el siguiente código:

```

de django.shortcuts import render, redireccionar
de django.contrib.auth.decorators import login_required
desde django.contrib import mensajes
desde .forms import ImageCreateForm

@Necesario iniciar sesión
def image_create (solicitud):
    if request.method == 'POST':
        # formulario enviado
        formulario = ImageCreateForm (datos = solicitud.POST)
        si form.is_valid ():
            # los datos del formulario son válidos
            cd = form.cleaned_data
            new_item = form.save (commit = False)

```

```
# asignar usuario actual al artículo
new_item.user = request.user
```

[156]

Capítulo 5

```
new_item.save ()
messages.success (solicitud, 'Imagen agregada correctamente')

# redirigir a la vista de detalles del nuevo elemento creado
volver a redireccionar (new_item.get_absolute_url ())

más:

# crear formulario con datos proporcionados por el bookmarklet a través de GET
formulario = ImageCreateForm (datos = solicitud.GET)

return render (solicitud,
                'images / image / create.html',
                {'sección': 'imágenes',
                 'formulario': formulario})
```

En el código anterior, usa el decorador `login_required` para `image_create` view para evitar el acceso de usuarios no autenticados. Así es como funciona esta vista:

1. Espera datos iniciales a través de GET para crear una instancia del formulario. Esta Los datos consistirán en los atributos de URL y título de una imagen de un sitio web y se proporcionará a través de GET mediante la herramienta JavaScript que crear más tarde. Por ahora, simplemente asume que estos datos estarán allí inicialmente.
2. Si se envía el formulario, compruebe si es válido. Si los datos del formulario son válidos, crea una nueva instancia de Imagen , pero evita que el objeto se guarde en la base de datos aún pasando `commit = False` al método `save ()` del formulario .
3. Asigne el usuario actual al nuevo objeto de imagen . Así es como puedes saber quién subió cada imagen.
4. Guarde el objeto de imagen en la base de datos.
5. Finalmente, crea un mensaje de éxito utilizando el marco de mensajería de Django. y redirigir al usuario a la URL canónica de la nueva imagen. Aun no lo has hecho implementó el método `get_absolute_url ()` del modelo `Image` ; Vas a haz eso más tarde.

Cree un nuevo archivo `urls.py` dentro de la aplicación de imágenes y agregue el siguiente código lo:

```
desde la ruta de importación django.urls
de . importar vistas

app_name = 'imágenes'

urlpatterns = [
    ruta ('crear /', vistas.imagen_create, nombre = 'crear'),
]
```

[157]

Compartir contenido en su sitio web

Edite el archivo `urls.py` principal del proyecto de marcadores para incluir los patrones del aplicación de imágenes , de la siguiente manera:

```
urlpatterns = [
    ruta ('admin /', admin.site.urls),
    ruta ('cuenta /', incluir ('cuenta.urls')),
    ruta ('social-auth /',
        include ('social_django.urls', namespace = 'social')),
    ruta ('imágenes /', incluir ('imágenes.urls', espacio de nombres = 'imágenes')),
]
```

Finalmente, necesita crear una plantilla para renderizar el formulario. Crea lo siguiente estructura de directorio dentro del directorio de la aplicación de imágenes :

```
plantillas/
  imágenes /
    imagen/
      create.html
```

Edite la nueva plantilla `create.html` y agregue el siguiente código:

```
{% extiende "base.html"%}

{% block title%} Agregar una imagen a marcadores {% endblock%}

{% block content%}
    <h1> Agregar una imagen a marcadores </h1>
    <img src = "{{request.GET.url}}" class = "image-preview">
    <método de formulario = "publicar">
        {{form.as_p}}
        {% csrf_token%}
        <input type = "submit" value = "¡Marcarlo!">
    </form>
{% endblock%}
```

Ejecute el servidor de desarrollo con `runserver_plus` y abra

`https://127.0.0.1:8000/images/create/?title=...&url=...` en su navegador, incluidos los parámetros GET de título y URL , que proporcionan un URL de imagen JPEG en este último. Por ejemplo, puede utilizar la siguiente URL:

`https://127.0.0.1:8000/images/create/?title=%20Django%20and%20Duke&url=https://upload.wikimedia.org/wikipedia/commons/8/85/Django_Reinhardt_and_Duke_Ellington_%28Gottlieb%29.jpg` .

Verá el formulario con una vista previa de la imagen, como la siguiente:

Figura 5.2: La página de creación de una nueva imagen de marcador

Agregue una descripción y haga clic en **BOOKMARK IT!** botón. Un nuevo objeto de imagen se guardará en su base de datos. Sin embargo, obtendrá un error que indica que el modelo de imagen no tiene el método `get_absolute_url()`, como sigue:

Figura 5.3: Un error que muestra que el objeto Imagen no tiene atributo `get_absolute_url`

No se preocupe por este error por ahora; vas a agregar un `get_absolute_url` método al modelo de imagen más tarde.

Abra <https://127.0.0.1:8000/admin/images/image/> en su navegador y verifique que el nuevo objeto de imagen se ha guardado, así:

Figura 5.4: La página de lista de imágenes del sitio de administración que muestra el objeto Imagen creado

[159]

Compartir contenido en su sitio web

Construyendo un bookmarklet con jQuery

Un marcador es un marcador almacenado en un navegador web que contiene JavaScript. código para ampliar la funcionalidad del navegador. Al hacer clic en el marcador, el El código JavaScript se ejecuta en el sitio web que se muestra en el navegador. Esto es muy útil para crear herramientas que interactúen con otros sitios web.

Algunos servicios en línea, como Pinterest, implementan sus propios marcadores para permitir los usuarios comparten contenido de otros sitios en su plataforma. Creemos un bookmarklet de manera similar para su sitio web, usando jQuery para construir su bookmarklet. jQuery es una biblioteca de JavaScript popular que le permite desarrollar clientes funcionalidad lateral más rápida. Puede leer más sobre jQuery en su sitio web oficial: <https://jquery.com/>.

Así es como sus usuarios agregarán un marcador a su navegador y lo usarán:

1. El usuario arrastra un enlace desde su sitio a los marcadores de su navegador. El enlace contiene código JavaScript en su atributo `href`. Este código se almacenará en el marcador.
2. El usuario navega a cualquier sitio web y hace clic en el marcador. El JavaScript se ejecuta el código del marcador.

Dado que el código JavaScript se almacenará como marcador, no podrá actualízelo más tarde. Este es un inconveniente importante que puede resolver implementando

Cree una nueva plantilla en imágenes / plantillas / y asígnele el nombre bookmarket_launcher.js . Este será el script de inicio. Agregue el siguiente código JavaScript a este archivo:

[160]

Agreguemos el lanzador de marcadores a las páginas del tablero para que sus usuarios puedan copiarlo en sus marcadores. Edite la plantilla `account / dashboard.html` del aplicación de cuenta y haga que se vea como lo siguiente:

El panel ahora muestra el número total de imágenes marcadas por el usuario. Utilice la etiqueta de plantilla `{% with %}` para establecer una variable con el número total de

imágenes marcadas por el usuario actual. Incluye un enlace con un atributo href que contiene el script de inicio de bookmarklet. Incluirás este código JavaScript desde la plantilla bookmarklet_launcher.js .

[161]

Compartir contenido en su sitio web

Abra <https://127.0.0.1:8000/account/> en su navegador. Deberías ver el página siguiente:

Figura 5.5: La página del panel, incluido el total de imágenes marcadas y el botón del marcador.

Ahora cree los siguientes directorios y archivos dentro de la aplicación de imágenes directorio:

```
estático/  
  js /  
    bookmarklet.js
```

Encontrará un directorio `estático / css /` en el directorio de la aplicación de imágenes en el código que acompaña a este capítulo. Copie el directorio `css /` en el directorio `estático /` de su código. Puede encontrar el contenido del directorio en <https://github.com/PacktPublishing/Django-3-by-Example/tree/master/Chapter05/bookmarks/images/static>.

El archivo `css / bookmarklet.css` proporciona los estilos para su bookmarklet JavaScript.

Edite el archivo `estático bookmarklet.js` y agréguele el siguiente código JavaScript:

```
(función(){  
  var jquery_version = '3.4.1';  
  var site_url = 'https://127.0.0.1:8000/';  
  var static_url = site_url + 'static /';  
  var min_width = 100;  
  var min_height = 100;  
  
  función bookmarklet (msg) {  
    // Aquí va nuestro código de marcador  
  };  
  
  // Comprueba si jQuery está cargado
```

[162]

```

if (typeof window.jQuery != 'undefined') {
    bookmarklet ();
} más {
    // Busca conflictos
    var conflicto = tipo de ventana. $! = 'definido';
    // Cree el script y apunte a la API de Google
    var script = document.createElement ('script');
    script.src = '//ajax.googleapis.com/ajax/libs/jquery/' +
        jquery_version + '/jquery.min.js';
    // Agrega el script al 'encabezado' para procesar
    document.head.appendChild (script);
    // Crea una forma de esperar hasta que se cargue el script
    var intentos = 15;
    (función){
        // Compruebe de nuevo si jQuery no está definido
        if (typeof window.jQuery == 'undefined') {
            if (- intentos > 0) {
                // Se llama a sí mismo en unos pocos milisegundos
                window.setTimeout (argumentos.callee, 250)
            } más {
                // Demasiados intentos de carga, enviar error
                alert ('Se produjo un error al cargar jQuery')
            }
        }
    } más {
        bookmarklet ();
    }
}
}) 0;
}
}) 0

```

Este es el script principal del cargador de jQuery. Se encarga de usar jQuery si ya lo ha hecho cargado en el sitio web actual. Si jQuery no está cargado, el script carga jQuery de la **red de entrega de contenido (CDN)** de Google , que aloja el popular código JavaScript marcos. Cuando se carga jQuery, ejecuta la función bookmarklet () que contendrá su código de marcador. Además, establezca algunas variables en la parte superior del archivo:

- `jquery_version` : la versión de jQuery para cargar
- `site_url` y `static_url` : la URL base de su sitio web y la base URL para archivos estáticos
- `min_width` y `min_height` : el ancho y alto mínimo en píxeles para las imágenes que su bookmarklet intentará encontrar en el sitio

Compartir contenido en su sitio web

Ahora implementemos la función bookmarklet . Edite la función bookmarklet () para haz que se vea así:

```

función bookmarklet (msg) {
    // cargar CSS

```

Por razones de seguridad, su navegador le impedirá ejecutar el bookmarklet a través de HTTP en un sitio servido a través de HTTPS. Deberá poder cargar el bookmarklet en cualquier sitio, incluidos los sitios protegidos mediante HTTPS. Para ejecutar tu

servidor de desarrollo con un certificado SSL / TLS generado automáticamente, utilizará RunServerPlus de Django Extensions, que instaló en el capítulo anterior.

Ejecute el servidor de desarrollo RunServerPlus con el siguiente comando:

```
python manage.py runserver_plus --cert-file cert.crt
```

Abra `https://127.0.0.1:8000/account/` en su navegador. Inicie sesión con un usuario y luego arrastre el botón **BOOKMARK IT** a la barra de herramientas de marcadores de su navegador, de la siguiente manera:

Figura 5.6: Agregar el botón BOOKMARK IT a la barra de herramientas de marcadores

[165]

Compartir contenido en su sitio web

Abra un sitio web de su elección en su navegador y haga clic en su marcador. Verá que aparece un nuevo cuadro blanco en el sitio web, que muestra todos los archivos JPEG imágenes encontradas con dimensiones superiores a 100×100 píxeles. Debería verse como el siguiente ejemplo:

Figura 5.7: El bookmarklet cargado en un sitio web externo

El contenedor HTML incluye las imágenes que se pueden marcar. Quiere que el usuario para hacer clic en la imagen deseada y marcarla. Edite `js / bookmarklet.js` archivo estático y agregue el siguiente código en la parte inferior de la función `bookmarklet ()` :

```
// cuando se selecciona una imagen, abre la URL con ella
jQuery('#bookmarklet .images a').click(function (e) {
    selected_image = jQuery(esto).children('img').attr('src');
    // ocultar bookmarklet
    jQuery('#bookmarklet').hide();
    // abre una nueva ventana para enviar la imagen
```

```

window.open (site_url + 'images / create /? url ='
+ encodeURIComponent (imagen_seleccionada)
+ '& título ='
+ encodeURIComponent (jQuery ('título'). texto ()),
'_blanco');
});

```

[166]

El código anterior funciona de la siguiente manera:

1. Adjunta un evento click () al elemento de enlace de cada imagen.
2. Cuando un usuario hace clic en una imagen, establece una nueva variable llamada selected_ imagen que contiene la URL de la imagen seleccionada.
3. Oculta el marcador y abre una nueva ventana del navegador con la URL de marcar una nueva imagen en su sitio. Pasas el contenido del <title> elemento del sitio web y la URL de la imagen seleccionada como parámetros GET .

Abra una nueva URL con su navegador y haga clic en su marcador nuevamente para mostrar el cuadro de selección de imagen. Si hace clic en una imagen, será redirigido a la imagen. crear página, pasando el título del sitio web y la URL de la imagen seleccionada como GET parámetros:

Figura 5.8: Agregar un nuevo marcador de imagen

¡Felicidades! Este es su primer bookmarklet de JavaScript y está completamente integrado en su proyecto Django.

[167]

Compartir contenido en su sitio web

Crear una vista de detalle para imágenes

Creemos ahora una vista de detalle simple para mostrar una imagen que se ha guardado en Tú sitio. Abra el archivo `views.py` de la aplicación de imágenes y agregue lo siguiente codificarlo:

```
desde django.shortcuts import get_object_or_404
desde .models importar imagen

def image_detail (solicitud, id, slug):
    image = get_object_or_404 (Imagen, id = id, slug = slug)
    return render (solicitud,
                  'images / image / detail.html',
                  {'sección': 'imágenes',
                  'imagen': imagen})
```

Esta es una vista simple para mostrar una imagen. Edite el archivo `urls.py` de las imágenes aplicación y agregue el siguiente patrón de URL:

```
ruta ('detalle / <int: id> / <slug: slug> /',
      views.image_detail, name = 'detalle'),
```

Edite el archivo `models.py` de la aplicación de imágenes y agregue `get_absolute_url ()` método al modelo de imagen , de la siguiente manera:

```
desde django.urls import reverse

Imagen de clase (modelos.Modelo):
# ...
def get_absolute_url (yo):
    return reverse ('imágenes: detalle', args = [self.id, self.slug])
```

Recuerde que el patrón común para proporcionar URL canónicas para objetos es para definir un método `get_absolute_url ()` en el modelo.

aplicación de imágenes y asígnele el nombre detail.html . Agregue el siguiente código:

```
{% extiende "base.html"%}

{% block title%} {{image.title}} {% endblock%}

{% block content%}
<h1> {{image.title}} </h1>
<img src = "{{image.image.url}}" class = "image-detail">
{% con total_likes = image.users_like.count%}
<div class = "image-info">
<div>
<span class = "count">
{{total_likes}} como {{total_likes | pluralize}}
</span>
</div>
{{image.description | linebreak}}
</div>
<div class = "imagen-me gusta">
{% para el usuario en image.users_like.all%}
<div>
<img src = "{{user.profile.photo.url}}">
<p> {{user.first_name}} </p>
</div>
{% vacío%}
A nadie le gusta esta imagen todavía.
{% endfor%}
</div>
{% terminar con %}
{% endblock%}
```

Esta es la plantilla para mostrar la vista detallada de una imagen marcada. Tú haces uso de la etiqueta {% with%} para almacenar el resultado del QuerySet, contando todos los Me gusta del usuario en una nueva variable llamada total_likes . Al hacerlo, evita evaluar el mismo QuerySet dos veces. También incluye la descripción de la imagen e itera sobre la imagen. users_like.all para mostrar todos los usuarios a los que les gusta esta imagen.

Compartir contenido en su sitio web

Siempre que necesite repetir una consulta en su plantilla, utilice el
{% with%} etiqueta de plantilla para evitar consultas adicionales en la base de datos.

A continuación, marque una nueva imagen con el bookmarklet. Serás redirigido a la página de detalles de la imagen después de publicar la imagen. La página incluirá un mensaje de éxito, como sigue:

Figura 5.9: La página de detalles de la imagen para el marcador de imagen

Crear miniaturas de imágenes usando miniaturas fáciles

Está mostrando la imagen original en la página de detalles, pero las dimensiones de las diferentes imágenes pueden variar considerablemente. Además, los archivos originales de algunas imágenes pueden ser enorme y cargarlos puede llevar demasiado tiempo. La mejor forma de visualización optimizada imágenes de manera uniforme es generar miniaturas. Usemos una aplicación Django llamados miniaturas fáciles para este propósito.

[170]

Abra la terminal e instale easy-thumbnails usando el siguiente comando:

```
pip install easy-thumbnails == 2.7
```

Edite el archivo settings.py del proyecto de marcadores y agregue easy_thumbnails al Configuración INSTALLED_APPS, de la siguiente manera:

```
INSTALLED_APPS = [  
    # ...  
    'easy_thumbnails',  
]
```

Luego, ejecute el siguiente comando para sincronizar la aplicación con su base de datos:

```
python manage.py migrar
```

Debería ver un resultado que incluye las siguientes líneas:

```
Aplicando easy_thumbnails.0001_initial ... OK
```

```
Aplicando easy_thumbnails.0002_thumbnaildimensions ... OK
```

La aplicación easy-thumbnails le ofrece diferentes formas de definir la imagen miniaturas. La aplicación proporciona una etiqueta de plantilla {% thumbnail%} para generar miniaturas en plantillas y un ImageField personalizado si desea definir miniaturas en tus modelos. Usemos el enfoque de etiqueta de plantilla. Edite las imágenes / imagen / detail.html plantilla y considere la siguiente línea:

```
<img src = "{{image.image.url}}" class = "image-detail">
```

Las siguientes líneas deben reemplazar a la anterior:

```
{% load thumbnail%}  
<a href="{{ image.image.url }}">  
    <img src = "{% imagen en miniatura.imagen 300x0%}" class = "imagen-detalle">  
</a>
```

Defina una miniatura con un ancho fijo de 300 píxeles y una altura flexible para mantenga la relación de aspecto utilizando el valor 0 . La primera vez que un usuario carga esta página, se crea una imagen en miniatura. La miniatura se almacena en el mismo directorio del archivo original. La ubicación se define mediante la configuración MEDIA_ROOT y upload_to atributo de la imagen de campo de la imagen del modelo.

La miniatura generada se sirve en las siguientes solicitudes. Iniciar el desarrollo servidor y acceda a la página de detalles de la imagen para una imagen existente. La miniatura será generado y mostrado en el sitio. Si comprueba la URL de la imagen generada, verá que el nombre del archivo original va seguido de detalles adicionales de la configuración utilizada para crear la miniatura; por ejemplo, filename.jpg.300x0_q85.jpg. 85 es el valor para la calidad JPEG predeterminada que utiliza la biblioteca para generar la miniatura.

[171]

Compartir contenido en su sitio web

Puede utilizar un valor de calidad diferente utilizando el parámetro de calidad . Para establecer el más alto Calidad JPEG, puede usar el valor 100 , como esta { % imagen en miniatura.imagen
Calidad 300x0 = 100% } .

La aplicación easy-thumbnails ofrece varias opciones para personalizar su miniaturas, incluidos algoritmos de recorte y diferentes efectos que se pueden aplicar. Si tiene alguna dificultad para generar miniaturas, puede agregar THUMBNAIL_DEBUG = Fiel al archivo settings.py para obtener información de depuración. Usted puede lea la documentación completa de easy-thumbnails en <https://easy-thumbnails.readthedocs.io/>.

Agregar acciones AJAX con jQuery

Ahora agreguemos acciones AJAX a su aplicación. AJAX proviene de Asynchronous JavaScript y XML, que abarcan un grupo de técnicas para hacer asincrónicas Solicitudes HTTP. Consiste en enviar y recuperar datos del servidor de forma asincrónica, sin recargar toda la página. A pesar del nombre, XML no es necesario. Puede enviar o recuperar datos en otros formatos, como JSON, HTML o Texto sin formato.

Va a agregar un enlace a la página de detalles de la imagen para permitir que los usuarios hagan clic en él para como una imagen. Realizará esta acción con una llamada AJAX para evitar recargar el toda la pagina.

Primero, cree una vista para que a los usuarios les gusten o no las imágenes. Edite el archivo views.py del aplicación de imágenes y agregue el siguiente código:

```
desde django.http import JsonResponse
desde django.views.decorators.http import require_POST

@Necesario iniciar sesión
@require_POST
def image_like (solicitud):
    image_id = request.POST.get ('id')
    action = request.POST.get ('acción')
    si image_id y action:
        tratar:
            image = Image.objects.get (id = image_id)
            si acción == 'me gusta':
                image.users_like.add (request.user)
            más:
                image.users_like.remove (request.user)
            return JsonResponse ({'estado': 'ok'})
    excepto:
```

[172]

```

        pasar
    return JsonResponse ({'estado': 'error'})

```

Utiliza dos decoradores para su vista. El decorador `login_required` evita los usuarios que no han iniciado sesión para acceder a esta vista. El decorador `require_POST` devuelve un objeto `HttpResponseNotAllowed` (código de estado 405) si la solicitud HTTP no se realiza mediante POST . De esta forma, solo permite solicitudes POST para esta vista.

Django también proporciona un decorador `require_GET` para permitir solo solicitudes GET y un decorador `require_http_methods` al que puede pasar una lista de métodos permitidos como argumento.

En esta vista, usa dos parámetros POST :

- `image_id` : el ID del objeto de imagen en el que el usuario está realizando la acción
- `acción` : la acción que el usuario desea realizar, que usted asume ser una cadena con el valor similar o diferente

Utiliza el administrador proporcionado por Django para el campo `users_like` many-to-many del modelo de imagen para agregar o eliminar objetos de la relación utilizando el `agregar ()` o `eliminar ()` métodos. Llamar a `add ()` , es decir, pasar un objeto que ya está presente en el conjunto de objetos relacionados, no lo duplica. Llamar a `remove ()` y pasar un objeto que no está en el conjunto de objetos relacionados no hace nada. Otro método útil de los administradores de muchos a muchos es `claro ()` , que elimina todos los objetos de los conjunto de objetos.

Finalmente, usa la clase `JsonResponse` proporcionada por Django, que devuelve un Respuesta HTTP con un tipo de contenido `application / json` , convirtiendo el objeto dado en una salida JSON.

Edite el archivo `urls.py` de la aplicación de imágenes y agregue el siguiente patrón de URL:

```

ruta ('like /', views.image_like, name = 'like'),

```

Cargando jQuery

Deberá agregar la funcionalidad AJAX a su plantilla de detalles de imagen. En orden para usar jQuery en sus plantillas, lo incluirá en la plantilla `base.html` de su proyecto primero. Edite la plantilla `base.html` de la aplicación de la cuenta y incluya el siguiente código antes de la etiqueta HTML `</body>` de cierre :

```

<script src = "https://ajax.googleapis.com/ajax/libs/jquery/3.4.1/
jquery.min.js "> </script>
<script>
    $(document).ready(function () {

```

```
{% block domready%}
{% endblock%}
});
</script>
```

Carga el marco jQuery desde la CDN de Google. También puedes descargar jQuery de <https://jquery.com/> y agréguelo al directorio estático de su aplicación en lugar.

Agrega una etiqueta `<script>` para incluir código JavaScript. `$(document).ready()` es una función jQuery que toma un controlador que se ejecuta cuando el **objeto de documento** La jerarquía del **modelo (DOM)** se ha construido completamente. El DOM es creado por el navegador cuando se carga una página web, y se construye como un árbol de objetos. Por incluyendo su código dentro de esta función, se asegurará de que todos los elementos HTML con los que va a interactuar se cargan en el DOM. Tu código solo será ejecutado una vez que el DOM está listo.

Dentro de la función del controlador listo para documentos, incluye un bloque de plantilla de Django llamado `domready`, en el que las plantillas que amplían la plantilla base podrán incluir JavaScript específico. No se confunda con el código JavaScript y Django etiquetas de plantilla. El lenguaje de la plantilla de Django se representa en el lado del servidor, generando el documento HTML final y JavaScript se ejecuta en el lado del cliente. En algunos casos, es útil generar código JavaScript dinámicamente usando Django, para poder usar el resultados de QuerySets o cálculos del lado del servidor para definir variables en JavaScript.

Los ejemplos de este capítulo incluyen código JavaScript en plantillas de Django. Los La forma preferida de incluir código JavaScript es cargando archivos `.js`, que se sirven como archivos estáticos, especialmente cuando son scripts grandes.

Falsificación de solicitudes entre sitios en solicitudes AJAX

Aprendió sobre la **falsificación de solicitudes entre sitios (CSRF)** en el *Capítulo 2, Mejora de su blog con funciones avanzadas*. Con la protección CSRF activa, Django busca un token CSRF en todas las solicitudes POST. Cuando envía formularios, puede utilizar el `{% csrf_token %}` etiqueta de plantilla para enviar el token junto con el formulario. Sin embargo, es un poco incómodo. para las solicitudes AJAX para pasar el token CSRF como datos POST con cada solicitud POST. Por lo tanto, Django le permite configurar un encabezado `X-CSRFToken` personalizado en su AJAX solicitudes con el valor del token CSRF. Esto le permite configurar jQuery o cualquier otra biblioteca de JavaScript para configurar automáticamente el encabezado `X-CSRFToken` en cada solicitud.

Para incluir el token en todas las solicitudes, debe seguir los siguientes pasos:

1. Recupere el token CSRF de la cookie `csrftoken`, que se establece si CSRF la protección está activa
2. Envíe el token en la solicitud AJAX utilizando el encabezado `X-CSRFToken`

[174]

Puede encontrar más información sobre la protección CSRF y AJAX en <https://docs.djangoproject.com/en/3.0/ref/csrf/#ajax>.

Edite el último código que incluyó en su plantilla `base.html` y haga que se vea como el seguimiento:

```
<script src = "https://ajax.googleapis.com/ajax/libs/jquery/3.4.1/
jquery.min.js "> </script>
<script src = "https://cdn.jsdelivr.net/npm/js-cookie@2.2.1/src/
js.cookie.min.js "> </script>
<script>
    var csrftoken = Cookies.get('csrftoken');
    function csrfSafeMethod(método) {
        // estos métodos HTTP no requieren protección CSRF
```

```

    }
    return (/^(GET|HEAD|OPTIONS|TRACE)$/.test(method));
}
$.ajaxSetup({
  beforeSend: function (xhr, settings) {
    if (! csrfSafeMethod (settings.type) &&! this.crossDomain) {
      xhr.setRequestHeader ("X-CSRFToken", csrftoken);
    }
  }
});
$(document).ready(function () {
  {% block domready%}
  {% endblock%}
});
</script>

```

El código anterior es el siguiente:

1. Carga el complemento JS Cookie desde una CDN pública para que pueda interactuar con las cookies. JS Cookie es una API de JavaScript liviana para manejar galletas. Puede obtener más información al respecto en <https://github.com/js-cookie/js-cookie>.
2. Lea el valor de la csrftoken galleta con Cookies.get () .
3. Defina la función csrfSafeMethod () para comprobar si un HTTP El método es seguro. Los métodos seguros no requieren protección CSRF; estos son GET , HEAD , OPTIONS y TRACE .
4. Configura las solicitudes jQuery AJAX usando \$.ajaxSetup () . Antes de cada AJAX se realiza la solicitud, verifica si el método de solicitud es seguro y que la solicitud actual no es de dominio cruzado. Si la solicitud no es segura, establezca el Cabecera X-CSRFToken con el valor obtenido de la cookie. Esta configuración se aplicará a todas las solicitudes AJAX realizadas con jQuery.

[175]

Compartir contenido en su sitio web

El token CSRF se incluirá en todas las solicitudes AJAX que utilizan HTTP inseguro métodos, como POST o PUT .

Realización de solicitudes AJAX con jQuery

Edite la plantilla images / image / detail.html de la aplicación de imágenes y considere la siguiente línea:

```
{% con total_likes = image.users_like.count%}
```

Reemplace la línea anterior por la siguiente:

```
{% con total_likes = image.users_like.count users_like = image.users_
like.all %}
```

Asegúrese de que la etiqueta de la plantilla esté dividida en varias líneas.

Reemplace la línea que define el bucle for :

```
{% para el usuario en image.users_like.all%}
```

con el siguiente:

```
{% para el usuario en users_like %}
```

Luego, modifique el elemento <div> con la clase image-info , de la siguiente manera:

```
<div class = "image-info">
  <div>
    <span class = "count">
      <span class = "total"> {{total_likes}} </span>
      como {{total_likes | pluralize}}
    </span>
  </div>
</div>

```

```

<a href = "#" data-id = "{{image.id}}" data-action = "{% if
request.user en users_like%} un {% endif%} like "
class = "botón me gusta">
    {% si request.user no está en users_like%}
        Me gusta
    {% else%}
        diferente a
    {% terminara si %}
</a>
</div>
{{image.description | linebreak}}
</div>

```

Primero, agrega otra variable a la etiqueta de plantilla {% with%} para almacenar el resultados de la consulta image.users_like.all y evite ejecutarla dos veces. Tu usas la variable para el bucle for que itera sobre los usuarios a los que les gusta esta imagen.

[176]

Capítulo 5

Muestra el número total de usuarios a los que les gusta la imagen e incluye un enlace a Me gusta / a diferencia de la imagen. Verifica si el usuario está en el conjunto de objetos relacionados de users_les gusta mostrar me gusta o no , según la relación actual entre usuario y esta imagen. Agrega los siguientes atributos al elemento <a> HTML:

- data-id : el ID de la imagen mostrada.
- data-action : la acción que se ejecutará cuando el usuario haga clic en el enlace. Esto puede ser me gusta o no .

Cualquier atributo en cualquier elemento HTML cuyo nombre de atributo comience con datos- es un atributo de datos. Los atributos de datos se utilizan para almacenar datos personalizados para su aplicación.

Enviaré el valor de ambos atributos en la solicitud AJAX a image_like ver. Cuando un usuario hace clic en el enlace Me gusta / No me gusta , realizará lo siguiente acciones en el lado del cliente:

1. Llame a la vista AJAX, pasándole el ID de la imagen y los parámetros de acción.
2. Si la solicitud AJAX es exitosa, actualice el atributo de acción de datos del <a> HTML elemento con la acción opuesta (como / diferencia), y modificar su texto de visualización en consecuencia
3. Actualiza la cantidad total de Me gusta que se muestra.

Agregue el bloque domready en la parte inferior de la plantilla images / image / detail.html con el siguiente código JavaScript:

```

{% block domready%}
    $( 'a.like' ). click (function (e) {
        e.preventDefault ();
        $.post ( '{% url "imagenes: como"%}',
            {
                id: $ (esto) .data ('id'),
                acción: $ (esto) .data ('acción')
            },
            función (datos) {
                if (datos ['estado'] == 'ok')
                {
                    var acción_anterior = $ ( 'a.like' ). data ('acción');

                    // alternar acción-datos
                    $ ( 'a.like' ). data ('action', previous_action == 'like'?
                        'diferente': 'me gusta');
                }
            }
        );
    });

```


Compartir contenido en su sitio web

```

        // alternar el texto del enlace
        $('a.like').text(previous_action == 'like'? 'A diferencia de':
        'Me gusta');

        // actualizar el total de Me gusta
        var Me gusta_anterior = parseInt($('span.count .total').
texto());
        $('span.count .total').text(previous_action == 'like'?
        Me gusta_anterior + 1: Me gusta_anterior - 1);
    }
}
);
});
{% endblock%}

```

El código anterior funciona de la siguiente manera:

1. Utiliza el selector de jQuery \$('a.like') para encontrar todos los elementos <a> del Documento HTML con la clase similar .
2. Defina una función de controlador para el evento de clic . Esta función será ejecutado cada vez que el usuario hace clic en el enlace Me gusta / No me gusta .
3. Dentro de la función del controlador, usa e.preventDefault () para evitar la comportamiento predeterminado del elemento <a> . Esto evitará que el enlace tome usted en cualquier lugar.
4. Utiliza \$.post () para realizar una solicitud POST asincrónica al servidor. jQuery también proporciona un método \$.get () para realizar solicitudes GET y un método level \$.ajax () .
5. Utiliza la etiqueta de plantilla {% url%} de Django para crear la URL para AJAX. solicitud.
6. Construye el diccionario de parámetros POST para enviar la solicitud. los Los parámetros son el ID y los parámetros de acción que espera su vista de Django. Recuperar estos valores de la <a> del elemento de datos ID y los datos de acción atributos.
7. Defina una función de devolución de llamada que se ejecuta cuando la respuesta HTTP es recibido; toma un atributo de datos que contiene el contenido de la respuesta.
8. Accede al atributo de estado de los datos recibidos y comprueba si es igual a ok . Si los datos devueltos son los esperados, cambia la acción de datos atributo del enlace y su texto. Esto permite al usuario deshacer su acción.
9. Aumenta o disminuye el recuento total de Me gusta en uno, según la acción. realizado.

Abra la página de detalles de la imagen en su navegador para ver una imagen que haya subido. Debería poder ver el siguiente recuento inicial de Me gusta y el botón **ME GUSTA** , como sigue:

Figura 5.10: El recuento de Me gusta y el botón ME GUSTA en la plantilla de detalles de la imagen

Haga clic en el botón **ME GUSTA** . Notará que el recuento total de Me gusta aumenta en uno. y el texto del botón cambia a **UNLIKE** , de la siguiente manera:

Figura 5.11: El recuento de Me gusta y el botón después de hacer clic en el botón ME GUSTA

Si hace clic en el botón **NO ME GUSTA** , se realiza la acción y luego el botón el texto vuelve a cambiar a **LIKE** y el recuento total cambia en consecuencia.

Al programar JavaScript, especialmente al realizar solicitudes AJAX, Se recomienda que utilice una herramienta para depurar solicitudes de JavaScript y HTTP. La mayoría de los navegadores modernos incluyen herramientas de desarrollo para depurar JavaScript. Usualmente tu puede hacer clic con el botón derecho en cualquier lugar del sitio web y hacer clic en **Inspeccionar elemento** para acceder al herramientas de desarrollo web.

Creando decoradores personalizados para tu puntos de vista

Restringamos sus vistas AJAX para permitir solo solicitudes generadas a través de AJAX. los El objeto de solicitud de Django proporciona un método `is_ajax()` que comprueba si el la solicitud se realiza con `XMLHttpRequest` , lo que significa que es un AJAX solicitud. Este valor se establece en el encabezado `HTTP_X_REQUESTED_WITH` , que es incluido en las solicitudes AJAX por la mayoría de las bibliotecas de JavaScript.

A continuación, creará un decorador para verificar el encabezado `HTTP_X_REQUESTED_WITH` en sus opiniones. Un decorador es una función que toma otra función y extiende el comportamiento de este último sin modificarlo explícitamente. Si el concepto de decoradores es ajeno a usted, es posible que desee echar un vistazo a <https://www.python.org/dev/peps/pep-0318/> antes de continuar leyendo.

[179]

Compartir contenido en su sitio web

Dado que su decorador será genérico y podría aplicarse a cualquier vista, cree un paquete Python común en su proyecto. Crea el siguiente directorio y archivos dentro del directorio del proyecto de marcadores :

```
común/
  __init__.py
  decorators.py
```

Edite el archivo `decorators.py` y agregue el siguiente código:

```
desde django.http import HttpResponseRedirect
```

```
def ajax_required (f):
    def wrap (solicitud, * args, ** kwargs):
        si no request.is_ajax ():
            return HttpResponseRedirect ()
        return f (solicitud, * args, ** kwargs)
    envolver .__ doc __ = f .__ doc __
    envolver .__ nombre __ = f .__ nombre __
    envoltura de devolución
```

El código anterior es su decorador personalizado `ajax_required` . Define una envoltura función que devuelve un objeto `HttpResponseBadRequest` (código HTTP 400) si el la solicitud no es AJAX. De lo contrario, devuelve la función decorada.

Ahora puede editar el archivo `views.py` de la aplicación de imágenes y agregar este decorador a su vista `image_like` AJAX, de la siguiente manera:

```
de common.decorators importar ajax_required
```

```
@ajax_required
@Necesario iniciar sesión
@require_POST
def image_like (solicitud):
    # ...
```

Si intenta acceder a `https://127.0.0.1:8000/images/like/` directamente con su navegador, obtendrá una respuesta HTTP 400 .

Cree decoradores personalizados para sus puntos de vista si encuentra que está repitiendo las mismas comprobaciones en varias vistas.

[180]

Agregando paginación AJAX a su lista puntos de vista

A continuación, debe enumerar todas las imágenes marcadas en su sitio web. Usará AJAX paginación para construir una funcionalidad de desplazamiento infinito. El desplazamiento infinito se logra mediante cargando los siguientes resultados automáticamente cuando el usuario se desplaza hasta la parte inferior de la página.

Implementemos una vista de lista de imágenes que manejará ambas solicitudes de navegador estándar y solicitudes AJAX, incluida la paginación. Cuando el usuario carga inicialmente la imagen página de lista, mostrará la primera página de imágenes. Cuando se desplazan hasta el final de la página, cargará la siguiente página de elementos a través de AJAX y la agregará a la parte inferior de la página principal.

La misma vista manejará la paginación estándar y AJAX. Edita `views.py` archivo de la aplicación de imágenes y agregue el siguiente código:

```
desde django.http importar HttpResponseRedirect
desde django.core.paginator importar Paginator, EmptyPage, \
    PageNotAnInteger

@Necesario iniciar sesión
def image_list (solicitud):
    imágenes = Image.objects.all ()
    paginator = Paginator (imágenes, 8)
    page = request.GET.get ('página')
```

```

tratar:
    images = paginator.page (página)
excepto PageNotAnInteger:
    # Si la página no es un número entero, entregue la primera página
    imágenes = paginator.page (1)
excepto EmptyPage:
    si request.is_ajax ():
        # Si la solicitud es AJAX y la página está fuera de rango
        # devuelve una página vacía
        return HttpResponse ("")
    # Si la página está fuera del rango, entregue la última página de resultados
    imágenes = paginator.page (paginator.num_pages)
si request.is_ajax ():
    return render (solicitud,
                    'images / image / list_ajax.html',
                    {'sección': 'imágenes', 'imágenes': imágenes})

return render (solicitud,
                'images / image / list.html',
                {'sección': 'imágenes', 'imágenes': imágenes})

```

[181]

Compartir contenido en su sitio web

En esta vista, crea un QuerySet para devolver todas las imágenes de la base de datos. Entonces tú construya un objeto Paginator para paginar los resultados, recuperando ocho imágenes por página. Obtiene una excepción EmptyPage si la página solicitada está fuera de rango. Si este es el caso y la solicitud se realiza a través de AJAX, devuelve un HttpResponse vacío que ayudarlo a detener la paginación AJAX en el lado del cliente. Prestas los resultados a dos plantillas diferentes:

- Para solicitudes AJAX, renderiza la plantilla list_ajax.html . Esta plantilla solo contendrá las imágenes de la página solicitada.
- Para solicitudes estándar, renderiza la plantilla list.html . Esta plantilla extenderá la plantilla base.html para mostrar toda la página y incluya la plantilla list_ajax.html para incluir la lista de imágenes.

Edite el archivo urls.py de la aplicación de imágenes y agregue el siguiente patrón de URL:

```
ruta ("", views.image_list, name = 'lista'),
```

Finalmente, debe crear las plantillas mencionadas aquí. Dentro de las imágenes / imagen / directorio de plantillas, cree una nueva plantilla y asígnele el nombre list_ajax.html . Añade el siguiente código:

```

{% load thumbnail%}

{% para imagen en imágenes%}
<div class = "imagen">
    <a href="{{ image.get_absolute_url }}">
        {% thumbnail image.image 300x300 crop = "inteligente" como im%}
        <a href="{{ image.get_absolute_url }}">
            <img src = "{{ im.url }}">
        </a>
    </a>
    <div class = "info">
        <a href="{{ image.get_absolute_url }}" class="title">
            {{ titulo de la imagen }}
        </a>
    </div>
</div>
{% endfor%}

```

La plantilla anterior muestra la lista de imágenes. Lo usará para devolver resultados para solicitudes AJAX. En este código, itera sobre imágenes y generas un cuadrado miniatura de cada imagen. Normalizas el tamaño de las miniaturas a 300x300

píxeles. También usa la opción de recorte inteligente . Esta opción indica que la imagen debe recortarse gradualmente hasta el tamaño solicitado mediante la eliminación de cortes de los bordes con la menor entropía.

[182]

Cree otra plantilla en el mismo directorio y asígnele el nombre list.html . Añade el siguiente código:

```
{% extiende "base.html"%}

{% block title%} Imágenes agregadas a marcadores {% endblock%}

{% block content%}
    <h1> Imágenes agregadas a marcadores </h1>
    <div id = "image-list">
        {% incluyen "images / image / list_ajax.html"%}
    </div>
{% endblock%}
```

La plantilla de lista amplía la plantilla base.html . Para evitar la repetición de código, incluir la plantilla list_ajax.html para mostrar imágenes. La lista.html

La plantilla contendrá el código JavaScript para cargar páginas adicionales al desplazarse al final de la página.

Agregue el siguiente código a la plantilla list.html :

```
{% block domready%}
    var page = 1;
    var página_vacia = falso;
    var block_request = falso;

    $(ventana).scroll(function () {
        var margin = $(documento).height () - $(ventana).height () - 200;
        if ($(ventana).scrollTop ()> margin && empty_page == false &&
            block_request == falso) {
            block_request = verdadero;
            página + = 1;
            $.get ('? page =' + página, función (datos) {
                si (datos == "") {
                    página_vacia = verdadero;
                }
                else {
                    block_request = falso;
                    $('# lista-de-imágenes'). append (datos);
                }
            });
        }
    });
{% endblock%}
```

[183]

Compartir contenido en su sitio web

El código anterior proporciona la funcionalidad de desplazamiento infinito. Incluye el Código JavaScript en el bloque domready que definió en la plantilla base.html . El código es el siguiente:

1. Defina las siguientes variables:
 - `página` : almacena el número de página actual.
 - `página_vacia` : le permite saber si el usuario está en la última página y recupera una página vacía. Tan pronto como obtenga una página vacía, dejará de enviar solicitudes AJAX adicionales porque asumirá que no hay más resultados.
 - `block_request` : le impide enviar solicitudes adicionales mientras hay una solicitud AJAX en curso.
2. Utiliza `$(ventana).scroll()` para capturar el evento de desplazamiento y también para definir una función de controlador para ello.
3. Calcula la variable de margen para obtener la diferencia entre el total la altura del documento y la altura de la ventana, porque esa es la altura del contenido restante para que el usuario se desplace. Restas un valor de 200 del resultado para que cargue la página siguiente cuando el usuario esté a menos de 200 píxeles al final de la página.
4. Solo envía una solicitud AJAX si no se realiza ninguna otra solicitud AJAX (`block_request` tiene que ser falso) y el usuario no llegó a la última página de resultados (`página_vacia` también es falsa).
5. Establece `block_request` en true para evitar una situación en la que el evento de desplazamiento desencadena solicitudes AJAX adicionales y aumenta el contador de páginas en uno, para recuperar la página siguiente.
6. Realiza una solicitud AJAX GET usando `$.get()` y recibe el HTML respuesta en una variable llamada `datos` . Los siguientes son los dos escenarios:
 - **La respuesta no tiene contenido** : llegó al final de los resultados, y no hay más páginas para cargar. Establece `empty_page` en verdadero para evitar solicitudes AJAX adicionales.
 - **La respuesta contiene datos** : agrega los datos al HTML elemento con el ID de lista de imágenes . El contenido de la página se expande verticalmente, agregando resultados cuando el usuario se acerca a la parte inferior de la página.

[184]

Abra <https://127.0.0.1:8000/images/> en su navegador. Verás la lista de imágenes que ha marcado hasta ahora. Debería verse similar a esto:

Figura 5.12: La página de lista de imágenes con paginación AJAX

Desplácese hasta el final de la página para cargar páginas adicionales. Asegúrese de tener marcó más de ocho imágenes con el bookmarklet, porque esa es la número de imágenes que está mostrando por página. Recuerda que puedes usar Firebug o una herramienta similar para rastrear las solicitudes AJAX y depurar su código JavaScript.

Finalmente, edite la plantilla base.html de la aplicación de la cuenta y agregue la URL para el elemento de imágenes del menú principal, de la siguiente manera:

```
<li {% if section == "images"%} class = "selected" {% endif%}>
  <a href= "{% url "images:list" %}"> Imágenes </a>
</li>
```

Ahora puede acceder a la lista de imágenes desde el menú principal.

[185]

Compartir contenido en su sitio web

Resumen

En este capítulo, creó modelos con relaciones de muchos a muchos y aprendió cómo personalizar el comportamiento de los formularios. Usaste jQuery con Django para compilar un marcador de JavaScript para compartir imágenes de otros sitios web en su sitio. Esta El capítulo también ha cubierto la creación de miniaturas de imágenes utilizando las miniaturas fáciles biblioteca. Finalmente, implementó vistas AJAX con jQuery y agregó AJAX paginación a la vista de lista de imágenes.

En el siguiente capítulo, aprenderá a crear un sistema de seguimiento y una actividad corriente. Trabajará con relaciones genéricas, señales y desnormalización. Tú también aprenderá a usar Redis con Django.

6

Seguimiento de las acciones del usuari

En el capítulo anterior, implementó vistas AJAX en su proyecto usando jQuery y creó un bookmarklet JavaScript para compartir contenido de otros sitios web en tu plataforma.

En este capítulo, aprenderá cómo crear un sistema de seguimiento y crear una actividad de usuario. corriente. También descubrirá cómo funcionan las señales de Django e integrará las funciones rápidas de Redis. Almacenamiento de E / S en su proyecto para almacenar vistas de elementos.

Este capítulo cubrirá los siguientes puntos:

- Construyendo un sistema de seguimiento
- Crear relaciones de muchos a muchos con un modelo intermedio.
- Crear una aplicación de flujo de actividades
- Agregar relaciones genéricas a modelos
- Optimización de QuerySets para objetos relacionados
- Uso de señales para desnormalizar recuentos
- Almacenamiento de vistas de elementos en Redis

Construyendo un sistema de seguimiento

or el sistema de seguimiento de acciones del usuario. El usuario puede seguir a otros usuarios, puede seguirse y rastrear lo que otros usuarios comparten en la plataforma. La relación entre usuarios es una relación de muchos a muchos: un usuario puede seguir a varios usuarios y ellos, a su vez, pueden ser seguidos por varios usuarios.

[187]

Seguimiento de las acciones del usuario

Creando relaciones de muchos a muchos con un modelo intermedio

En capítulos anteriores, creé relaciones de muchos a muchos agregando el `ManyToManyField` a uno de los modelos relacionados y dejando que Django cree la tabla de base de datos para la relación. Esto es adecuado para la mayoría de los casos, pero a veces es posible que deba crear un modelo intermedio para la relación. Creando un modelo intermedio es necesario cuando desea almacenar información adicional para la relación, por ejemplo, la fecha en que se creó la relación, o un campo que describe la naturaleza de la relación.

Creemos un modelo intermedio para construir relaciones entre usuarios. Existen dos razones para utilizar un modelo intermedio:

- Está utilizando el modelo de usuario proporcionado por Django y desea evitar alterarlo
- Quiere almacenar la hora en que se creó la relación

Edite el archivo `models.py` de la aplicación de su cuenta y agregue el siguiente código:

```
contacto = models.Modelo():
    user_from = models.ForeignKey('auth.User',
                                  related_name = 'rel_from_set',
                                  on_delete = models.CASCADE)

    user_to = models.ForeignKey('auth.User',
                                related_name = 'rel_to_set',
                                on_delete = models.CASCADE)

    creado = models.DateTimeField(auto_now_add = True,
                                  db_index = Verdadero)

    class Meta:
        ordering = ('-creado',)

    def __str__(yo):
        return f'{self.user_from} sigue a {self.user_to}'
```

El código anterior muestra el modelo de contacto que usará para el usuario relaciones. Contiene los siguientes campos:

- `user_from` : una `ForeignKey` para el usuario que crea la relación
- `user_to` : una `ForeignKey` para el usuario que se está siguiendo
- `creado` : un campo `DateTimeField` con `auto_now_add = True` para almacenar la hora cuando se creó la relación

[188]

Se crea automáticamente un índice de base de datos en los campos ForeignKey . Se utiliza db_index = True para crear un índice de base de datos para el campo creado . Esto mejorará rendimiento de la consulta al ordenar QuerySets por este campo.

Usando el ORM, puede crear una relación para un usuario, user1 , siguiendo a otro usuario, usuario2 , así:

```
user1 = User.objects.get(id = 1)
user2 = User.objects.get(id = 2)
Contact.objects.create(user_from = user1, user_to = user2)
```

Los administradores relacionados, rel_from_set y rel_to_set , devolverán un QuerySet para el modelo de contacto . Para acceder al lado final de la relación desde el Usuario modelo, sería deseable que el usuario contenga ManyToManyField , como sigue:

```
siguiente = modelos.ManyToManyField('self',
                                   a través de = Contacto,
                                   related_name = 'seguidores',
                                   simétrico = falso)
```

En el ejemplo anterior, le dice a Django que use su modelo intermediario personalizado para la relación agregando through = Contact al ManyToManyField . Esta es una relación de varios a varios del modelo de usuario a sí mismo; te refieres a 'yo' en el campo ManyToManyField para crear una relación con el mismo modelo.

Cuando necesite campos adicionales en una relación de varios a varios, Cree un modelo personalizado con ForeignKey para cada lado del relación. Agregue un ManyToManyField en uno de los relacionados modelos e indique a Django que su modelo intermediario debe usarse incluyéndolo en el parámetro through.

Si el modelo de usuario era parte de su aplicación, podría agregar el campo anterior al modelo. Sin embargo, no puede modificar la clase de usuario directamente porque pertenece a la aplicación django.contrib.auth . Adoptemos un enfoque ligeramente diferente al agregando este campo dinámicamente al modelo de usuario .

Edite el archivo models.py de la aplicación de la cuenta y agregue las siguientes líneas:

```
desde django.contrib.auth import get_user_model

# Agregar el siguiente campo al usuario de forma dinámica
user_model = get_user_model()
user_model.add_to_class('siguiente',
                       modelos.ManyToManyField('self',
                                               a través de = Contacto,
                                               related_name = 'seguidores',
                                               simétrico = falso))
```

[189]

En el código anterior, recupera el modelo de usuario mediante la función genérica get_user_model() , proporcionado por Django. Se utiliza el add_to_class() método de los modelos Django para parchear el modelo de usuario . Tenga en cuenta que usar

aprovechando la forma recomendada de agregar campos a los modelos. Sin embargo, manteniendo todas las ventajas del modelo de usuario integrado de Django .

También simplifica la forma en que recupera objetos relacionados usando Django ORM con `user.followers.all ()` y `user.following.all ()` . Usas el Modelo de contacto intermediario y evitar consultas complejas que involucren uniones de base de datos adicionales, como hubiera sido el caso si hubiera definido el relación en su modelo de perfil personalizado . La tabla para estos muchos a muchos La relación se creará utilizando el modelo de contacto . Por lo tanto, `ManyToManyField` , agregado dinámicamente, no implicará ningún cambio en la base de datos para el modelo de usuario de Django .

Tenga en cuenta que, en la mayoría de los casos, es preferible agregar campos al Perfil modelo que creó antes, en lugar de parchear el modelo de usuario . Idealmente, no debe alterar el modelo de usuario de Django existente . Django te permite usar modelos de usuario personalizados. Si desea utilizar su modelo de usuario personalizado, eche un vistazo a la documentación en <https://docs.djangoproject.com/en/3.0/topics/auth/customizing/#especificando-un-modelo-de-usuario-personalizado>.

Tenga en cuenta que la relación incluye `simétrico = Falso` . Cuando defines un `ManyToManyField` en el modelo que crea una relación consigo mismo, Django fuerza la relación sea simétrica. En este caso, está configurando `simétrico = Falso` para definir una relación no simétrica (si te sigo, no significa que tú sígueme automáticamente).

Cuando usa un modelo intermedio para muchos a muchos relaciones, algunos de los métodos del administrador relacionados están deshabilitados, como `agregar ()`, `crear ()` o `eliminar ()`. Necesitas crear o en su lugar, elimine las instancias del modelo intermedio.

Ejecute el siguiente comando para generar las migraciones iniciales para la cuenta solicitud:

cuenta de makemigrations de python manage.py

Obtendrá el siguiente resultado:

Migraciones para 'cuenta':

account / migrations / 0002_contact.py
- Crear modelo Contacto

[190]

Ahora, ejecute el siguiente comando para sincronizar la aplicación con la base de datos:

python manage.py migrar cuenta

Debería ver un resultado que incluye la siguiente línea:

Aplicando cuenta.0002_contact ... OK

El modelo de contacto ahora está sincronizado con la base de datos y puede crear relaciones entre usuarios. Sin embargo, su sitio no ofrece una forma de navegar usuarios o ver el perfil de un usuario en particular. Construyamos vistas de lista y detalle para el Modelo de usuario .

Creación de vistas de lista y detalle para perfiles de usuario

Abra el archivo `views.py` de la aplicación de la cuenta y agregue el siguiente código:

```
desde django.shortcuts import get_object_or_404
desde django.contrib.auth.models import User
```

```

@Necesario iniciar sesión
def user_list (solicitud):
    usuarios = User.objects.filter (is_active = True)
    return render (solicitud,
                    'cuenta / usuario / list.html',
                    {'sección': 'personas',
                    'usuarios': usuarios})

@Necesario iniciar sesión
def user_detail (solicitud, nombre de usuario):
    user = get_object_or_404 (Usuario,
                               nombre de usuario = nombre de usuario,
                               is_active = True)

    return render (solicitud,
                    'cuenta / usuario / detalle.html',
                    {'sección': 'personas',
                    'usuario': usuario})

```

Se trata de vistas sencillas de lista y detalle para objetos de usuario . La vista `user_list` obtiene todos los usuarios activos. El modelo de usuario de Django contiene un indicador `is_active` para designar si la cuenta de usuario se considera activa. Filtra la consulta por `is_active = True` para devolver solo los usuarios activos. Esta vista devuelve todos los resultados, pero usted puede mejorarlo agregando paginación de la misma manera que lo hizo para la imagen_ vista de lista .

[191]

Seguimiento de las acciones del usuario

La vista `user_detail` usa el atajo `get_object_or_404 ()` para recuperar el usuario activo con el nombre de usuario dado. La vista devuelve una respuesta HTTP 404 si no se encuentra ningún usuario activo con el nombre de usuario dado.

Edite el archivo `urls.py` de la aplicación de la cuenta y agregue un patrón de URL para cada ver, de la siguiente manera:

```

urlpatterns = [
    # ...
    ruta ('usuarios /', vistas.lista_usuario, nombre = 'lista_usuario'),
    ruta ('users / <username> /', views.user_detail, name = 'user_detail'),
]

```

Utilizará el patrón de URL `user_detail` para generar la URL canónica para los usuarios. Ya ha definido un método `get_absolute_url ()` en un modelo para devolver el URL canónica para cada objeto. Otra forma de especificar la URL de un modelo es mediante agregando la configuración `ABSOLUTE_URL_OVERRIDES` a su proyecto.

Edite el archivo `settings.py` de su proyecto y agregue el siguiente código:

```

de django.urls importar reverse_lazy

ABSOLUTE_URL_OVERRIDES = {
    'auth.user': lambda u: reverse_lazy ('user_detail',
                                         args = [u.username])
}

```

Django agrega un método `get_absolute_url ()` dinámicamente a cualquier modelo que aparezca en la configuración `ABSOLUTE_URL_OVERRIDES` . Este método devuelve el correspondiente URL para el modelo dado especificado en la configuración. Que devuelva el `user_detail` URL para el usuario dado. Ahora, puede usar `get_absolute_url ()` en una instancia de usuario para recuperar su URL correspondiente.

Abra el shell de Python con el comando de shell `python manage.py` y ejecute el siguiente código para probarlo:

```

>>> desde django.contrib.auth.models importar User

```

```
>>> usuario = User.objects.latest('id')
>>> str(usuario.get_absolute_url())
'/ cuenta / usuarios / ellington /'
```

[192]

La URL devuelta es la esperada.

Deberá crear plantillas para las vistas que acaba de crear. Agregue lo siguiente directorio y archivos a las plantillas / cuenta / directorio de la aplicación de la cuenta :

```
/usuario/
    detail.html
    list.html
```

Edite la plantilla account / user / list.html y agréguele el siguiente código:

```
{% extiende "base.html"%}
{% load thumbnail%

{% block title%} Personas {% endblock%}

{% block content%}
    <h1> Personas </h1>
    <div id = "lista de personas">
        {% para usuario en usuarios%}
            <div class = "usuario">
                <a href="{{ user.get_absolute_url }}">
                    <img src = "{% thumbnail user.profile.photo 180x180%}">
                </a>
                <div class = "info">
                    <a href="{{ user.get_absolute_url }}" class="title">
                        {{user.get_full_name}}
                    </a>
                </div>
            </div>
        {% endfor%}
    </div>
{% endblock%}
```

La plantilla anterior le permite enumerar todos los usuarios activos en el sitio. Tú iterar sobre los usuarios dados y utilizar la etiqueta de plantilla {% thumbnail%} de easy-thumbnails para generar miniaturas de imágenes de perfil.

Abra la plantilla base.html de su proyecto e incluya la URL user_list en el atributo href del siguiente elemento de menú:

```
<li {% if section == "people"%} class = "selected" {% endif%}>
    <a href= "{% url "user_list" %}"> Personas </a>
</li>
```

[193]

Seguimiento de las acciones del usuario

Inicie el servidor de desarrollo con el comando `python manage.py runserver` y abra `http://127.0.0.1:8000/account/users/` en su navegador. Deberías ver una lista de usuarios como la siguiente:

Figura 6.1: La página de la lista de usuarios con miniaturas de imágenes de perfil

Recuerde que si tiene alguna dificultad para generar miniaturas, puede agregar `THUMBNAIL_DEBUG = True` a su archivo `settings.py` para obtener depuración de información en el caparazón.

Edite la plantilla `account / user / detail.html` de la aplicación de la cuenta y agregue el siguiente código:

```
{% extiende "base.html"%}
{% load thumbnail%

{% block title%} {{user.get_full_name}} {% endblock%}

{% block content%}
    <h1> {{user.get_full_name}} </h1>
```

```

<div class = "profile-info">
  <img src = "{% thumbnail user.profile.photo 180x180%}" class = "usuario-
detalle ">
</div>
{% con total_followers = user.followers.count%}
  <span class = "count">
    <span class = "total"> {{total_followers}} </span>
    seguidor {{total_followers | pluralize}}
  </span>
  <a href = "#" data-id = "{{user.id}}" data-action = "{% si solicitud.
usuario en user.followers.all%} un {% endif%} follow "class = " follow
botón ">
    {% si request.user no está en user.followers.all%}
      Seguir
    {% else%}
      Dejar de seguir
    {% terminara si %}
  </a>
  <div id = "image-list" class = "image-container">
    {% incluye "images / image / list_ajax.html" con images = user.
images_created.all%}
  </div>
  {% terminar con %}
{% endblock%}

```

Asegúrese de que ninguna etiqueta de plantilla se divida en varias líneas; Django no es compatible etiquetas de varias líneas.

En la plantilla de detalles , muestra el perfil de usuario y usa la {% miniatura%} etiqueta de plantilla para mostrar la imagen de perfil. Muestra el número total de seguidores y un enlace para seguir o dejar de seguir al usuario. Realiza una solicitud AJAX para seguir / dejar de seguir a un usuario en particular. Agrega atributos data-id y data-action al <a> Elemento HTML, incluido el ID de usuario y la acción inicial a realizar cuando el enlace se hace clic en el elemento: seguir o dejar de seguir , lo que depende del usuario que solicita el sea seguidor de este otro usuario o no, según sea el caso. Muestra el imágenes marcadas por el usuario, incluidas images / image / list_ajax.html modelo.

Seguimiento de las acciones del usuario

Abra su navegador nuevamente y haga clic en un usuario que haya marcado algunas imágenes. Verá los detalles del perfil, de la siguiente manera:

Construyendo una vista AJAX para seguir a los usuarios

Creemos una vista simple para seguir / dejar de seguir a un usuario que usa AJAX. Edite las vistas.

py de la aplicación de la cuenta y agregue el siguiente código:

```
desde django.http import JsonResponse
desde django.views.decorators.http import require_POST
de common.decorators importar ajax_required
desde .models importar Contacto
```

```
@ajax_required
@require_POST
@Necesario iniciar sesión
def user_follow (solicitud):
    user_id = request.POST.get ('id')
    action = request.POST.get ('acción')
    si user_id y action:
        tratar:
            usuario = User.objects.get (id = user_id)
            if action == 'seguir':
                Contact.objects.get_or_create (
```

[196]

```
        user_from = request.user,
        user_to = usuario)
    más:
        Contact.objects.filter (user_from = request.user,
                                usuario_a = usuario) .delete ()
    return JsonResponse ({'estado': 'ok'})
excepto User.DoesNotExist:
    return JsonResponse ({'estado': 'error'})
return JsonResponse ({'estado': 'error'})
```

La vista `user_follow` es bastante similar a la vista `image_like` que creaste antes de. Dado que está utilizando un modelo intermediario personalizado para el usuario de muchos a muchas relaciones, los métodos predeterminados `add ()` y `remove ()` de la administrador de `ManyToManyField` no están disponibles. Utiliza el intermediario `Contacto` modelo para crear o eliminar relaciones de usuario.

Edite el archivo `urls.py` de la aplicación de la cuenta y agregue el siguiente patrón de URL lo:

```
ruta ('users / follow /', views.user_follow, name = 'user_follow'),
```

Asegúrese de colocar el patrón anterior antes del patrón de URL `user_detail`.

De lo contrario, cualquier solicitud a `/ users / follow /` coincidirá con la expresión regular de el patrón `user_detail` y esa vista se ejecutará en su lugar. Recuerda que en cada solicitud HTTP, Django verifica la URL solicitada con cada patrón en orden de aparición y se detiene en el primer partido.

Edite la plantilla `user / detail.html` de la aplicación de la cuenta y agregue el siguiente código:

```
{% block domready%}
$ ('a.follow'). click (function (e) {
    e.preventDefault ();
```



```
$.post('{% url "user_follow"%}',
{
    id: $(esto).data('id'),
    acción: $(esto).data('acción')
},
función(datos) {
    if(datos['estado'] == 'ok') {
        var acción_anterior = $(a.follow).data('acción');

        // alternar acción-datos
        $(a.follow).data('acción',
            previous_action == 'seguir'? 'dejar de seguir': 'seguir');
        // alternar el texto del enlace
```

[197]

Seguimiento de las acciones del usuario

```
$(a.follow).text (
    previous_action == 'seguir'? 'Dejar de seguir': 'Seguir');

// actualizar el total de seguidores
var seguidores_anteriores = parseInt (
    $('span.count .total').text ());
$( 'span.count .total').text (previous_action == 'follow'?
    seguidores_anteriores + 1: seguidores_anteriores - 1);
}
}
);
});
{% endblock%}
```

El código anterior es el código JavaScript para realizar la solicitud AJAX a seguir o dejar de seguir a un usuario en particular y también para alternar el enlace seguir / dejar de seguir. Tu usas jQuery para realizar la solicitud AJAX y establecer tanto el atributo de acción de datos como el texto del elemento HTML <a> basado en su valor anterior. Cuando el AJAX se realiza la acción, también actualiza el recuento total de seguidores que se muestra en la página.

Abra la página de detalles del usuario de un usuario existente y haga clic en el enlace **SEGUIR** para probar la funcionalidad que acaba de crear. Verás que se incrementa el recuento de seguidores:

Figura 6.3: Los seguidores cuentan y el botón de seguir / dejar de seguir

Crear un flujo de actividad genérico solicitud

Muchos sitios web sociales muestran un flujo de actividad a sus usuarios para que puedan rastrear lo que hacen otros usuarios en la plataforma. Un flujo de actividad es una lista de actividades recientes. realizado por un usuario o un grupo de usuarios. Por ejemplo, el News Feed de Facebook es un Últimas acciones. Las acciones de muestra pueden ser *la imagen Y* marcada por el *usuario X* o el *usuario X ahora siguiente usuario Y*.

Va a crear una aplicación de flujo de actividades para que todos los usuarios puedan ver la interacciones recientes de los usuarios que siguen. Para hacerlo, necesitará un modelo para guardar las acciones realizadas por los usuarios en el sitio web y una forma sencilla de agregar acciones a la alimentación.

Cree una nueva aplicación denominada acciones dentro de su proyecto con lo siguiente mando:

acciones de python manage.py startapp

Agregue la nueva aplicación a INSTALLED_APPS en el archivo settings.py de su proyecto para activar la aplicación en su proyecto:

```
INSTALLED_APPS = [
    # ...
    'actions.apps.ActionsConfig',
]
```

Edite el archivo models.py de la aplicación de acciones y agréguele el siguiente código:

```
desde modelos de importación django.db

class Action (modelos.Modelo):
    usuario = modelos.ForeignKey ('auth.User',
                                related_name = 'acciones',
                                db_index = Verdadero,
                                on_delete = modelos.CASCADE)

    verbo = modelos.CharField (longitud_máxima = 255)
    creado = modelos.DateTimeField (auto_now_add = True,
                                   db_index = Verdadero)

    clase Meta:
        ordering = ('-creado',)
```

El código anterior muestra el modelo de acción que se utilizará para almacenar las actividades de los usuarios. Los campos de este modelo son los siguientes:

- usuario : el usuario que realizó la acción; esta es una ForeignKey para el Django Modelo de usuario .
- verbo : el verbo que describe la acción que ha realizado el usuario.
- creado : la fecha y la hora en que se creó esta acción. Se utiliza auto_now_add = True para establecer automáticamente esto en la fecha y hora actual cuando el objeto se guarda por primera vez en la base de datos.

Con este modelo básico, solo puede almacenar acciones, como que el *usuario X hizo algo* . Tú necesita un campo ForeignKey adicional para guardar acciones que involucran un objeto de destino , como *usuario X marcado como favorito imagen Y* o *usuario X está ahora siguiendo usuario Y* . Como ya sabe, una ForeignKey normal puede apuntar a un solo modelo. En cambio, necesitará una forma de que el objeto de destino de la acción sea una instancia de un modelo existente. Esto es lo que el framework de tipos de contenido de Django le ayudará a hacer.

Usando el marco de contenttypes

Django incluye un marco de tipos de contenido ubicado en `django.contrib`.

tipos de contenido . Esta aplicación puede rastrear todos los modelos instalados en su proyecto y proporciona una interfaz genérica para interactuar con sus modelos.

La aplicación `django.contrib.contenttypes` está incluida en `INSTALLED_APPS` configuración predeterminada cuando crea un nuevo proyecto con el comando `startproject` . Lo utilizan otros paquetes `contrib` , como el marco de autenticación y el aplicación de administración.

La aplicación `contenttypes` contiene un modelo `ContentType` . Ejemplos de esto modelo representan los modelos reales de su aplicación y nuevas instancias de

`ContentType` se crea automáticamente cuando se instalan nuevos modelos en su proyecto. El modelo `ContentType` tiene los siguientes campos:

- `app_label` : indica el nombre de la aplicación que el modelo pertenece a. Esto se toma automáticamente del atributo `app_label` del modelo `Meta` opciones. Por ejemplo, su modelo de imagen pertenece a las imágenes solicitud.
- `modelo` : el nombre de la clase de modelo.
- `nombre` : indica el nombre legible por humanos del modelo. Esto es tomado automáticamente del atributo `verbose_name` del modelo `Meta` opciones.

Echemos un vistazo a cómo puede interactuar con los objetos `ContentType` . Abre la cáscara usando el comando de shell `python manage.py` . Puede obtener `ContentType` objeto correspondiente a un modelo específico mediante la realización de una consulta con el `app_label` y atributos del modelo , como sigue:

```
>>> de django.contrib.contenttypes.models importar ContentType
>>> image_type = ContentType.objects.get (app_label = 'imágenes',
modelo = 'imagen')
>>> tipo_imagen
<ContentType: imágenes | imagen>
```

También puede recuperar la clase de modelo de un objeto `ContentType` llamando a su método `model_class ()` :

```
>>> image_type.model_class ()
<class 'images.models.Image'>
```

[200]

También es común obtener el objeto `ContentType` para una clase de modelo en particular, como sigue:

```
>>> de images.models importar imagen
>>> ContentType.objects.get_for_model (Imagen)
<ContentType: imágenes | imagen>
```

Estos son solo algunos ejemplos del uso de tipos de contenido . Django ofrece más formas de trabajar con ellos. Puede encontrar la documentación oficial sobre los tipos de contenido marco en <https://docs.djangoproject.com/en/3.0/ref/contrib/contenttypes/> .

Agregar relaciones genéricas a sus modelos

En las relaciones genéricas, los objetos ContentType juegan el papel de apuntar al modelo utilizado para la relación. Necesitará tres campos para configurar una relación genérica en un modelo:

- Un campo ForeignKey para ContentType : esto le indicará el modelo para el relación
- Un campo para almacenar la clave principal del objeto relacionado: normalmente será un PositiveIntegerField para que coincida con los campos de clave primaria automáticos de Django
- Un campo para definir y gestionar la relación genérica utilizando los dos anteriores. Los campos: ContentType ofrece un marco GenericForeignKey de campo para este propósito

Edite el archivo models.py de la aplicación de acciones y haga que se vea así:

```
desde modelos de importación django.db
desde django.contrib.contenttypes.models import ContentType
de django.contrib.contenttypes.fields import GenericForeignKey

class Action (modelos.Modelo):
    usuario = modelos.ForeignKey ('auth.User',
                                  related_name = 'acciones',
                                  db_index = Verdadero,
                                  on_delete = modelos.CASCADE)

    verbo = modelos.CharField (longitud_máxima = 255)
    target_ct = modelos.ForeignKey (ContentType,
                                    en blanco = Verdadero,
                                    nulo = Verdadero,
                                    related_name = 'target_obj',
                                    on_delete = modelos.CASCADE)

    target_id = modelos.PositiveIntegerField (null = True,
                                              en blanco = Verdadero,

[ 201 ]
```

Seguimiento de las acciones del usuario

```
                                db_index = Verdadero)

    target = GenericForeignKey ('target_ct', 'target_id')
    creado = modelos.DateTimeField (auto_now_add = True,
                                    db_index = Verdadero)

    clase Meta:
        ordering = ('-creado',)
```

Ha agregado los siguientes campos al modelo de acción :

- target_ct : un campo ForeignKey que apunta al modelo ContentType
- target_id : Un PositiveIntegerField para almacenar la clave principal del objeto relacionado
- target : un campo GenericForeignKey para el objeto relacionado basado en el combinación de los dos campos anteriores

Django no crea ningún campo en la base de datos para los campos GenericForeignKey .

Los únicos campos que se asignan a los campos de la base de datos son target_ct y target_id .

Ambos campos tienen atributos en blanco = Verdadero y nulo = Verdadero , por lo que un objeto de destino es no es necesario al guardar objetos de acción .

Puede hacer que sus aplicaciones sean más flexibles utilizando relaciones en lugar de claves externas.

Ejecute el siguiente comando para crear migraciones iniciales para esta aplicación:

python manage.py acciones de makemigrations

Debería ver el siguiente resultado:

Migraciones por 'acciones':

actions / migrations / 0001_initial.py

- Crear modelo de acción

Luego, ejecute el siguiente comando para sincronizar la aplicación con la base de datos:

python manage.py migrar

La salida del comando debe indicar que las nuevas migraciones se han aplicado, de la siguiente manera:

Aplicando acciones.0001_initial ... OK

[202]

Agreguemos el modelo de acción al sitio de administración. Edite el archivo `admin.py` del de acciones y agregue el siguiente código:

```
desde django.contrib import admin
desde .models Import Action

@ admin.register (Acción)
clase ActionAdmin (admin.ModelAdmin):
    list_display = ('usuario', 'verbo', 'destino', 'creado')
    list_filter = ('creado',)
    search_fields = ('verbo',)
```

Acaba de registrar el modelo `Action` en el sitio de administración. Ejecuta la pitón comando `manage.py runserver` para iniciar el servidor de desarrollo y abrir `http://127.0.0.1:8000/admin/actions/action/add/` en su navegador. Debería ver la página para crear un nuevo objeto de acción , de la siguiente manera:

Figura 6.4: La página de acción para agregar en el sitio de administración de Django

Como notará en la captura de pantalla anterior, solo `target_ct` y `target_id` Se muestran los campos que se asignan a los campos reales de la base de datos. `The GenericForeignKey` El campo no aparece en el formulario. El campo `target_ct` le permite seleccionar cualquiera de los modelos registrados de su proyecto Django. Puede restringir los tipos de contenido a elegir entre un conjunto limitado de modelos utilizando el atributo `limit_choices_to` en el campo `target_ct` ; el atributo `limit_choices_to` le permite restringir el contenido de los campos `ForeignKey` a un conjunto específico de valores.

Seguimiento de las acciones del usuario

Cree un nuevo archivo dentro del directorio de la aplicación de acciones y asígnele el nombre `utils.py` . Debe definir una función de acceso directo que le permitirá crear una nueva acción objetos de una manera sencilla. Edite el nuevo archivo `utils.py` y agréguele el siguiente código:

```
desde django.contrib.contenttypes.models import ContentType
desde .models Import Action

def create_action (usuario, verbo, objetivo = Ninguno):
    acción = acción (usuario = usuario, verbo = verbo, objetivo = objetivo)
    action.save ()
```

La función `create_action ()` le permite crear acciones que opcionalmente incluyen un objeto de destino . Puede utilizar esta función en cualquier parte de su código como un atajo para agregue nuevas acciones al flujo de actividades.

Evitar acciones duplicadas en la actividad corriente

A veces, sus usuarios pueden hacer clic varias veces en el botón **ME GUSTA** o **NO ME GUSTA** o realizar la misma acción varias veces en un corto periodo de tiempo. Esto fácilmente conducir a almacenar y mostrar acciones duplicadas. Para evitar esto, mejoremos el función `create_action ()` para omitir acciones duplicadas obvias.

Edite el archivo `utils.py` de la aplicación de acciones de la siguiente manera:

```
importar fecha y hora
desde django.utils importar zona horaria
desde django.contrib.contenttypes.models import ContentType
desde .models Import Action

def create_action (usuario, verbo, objetivo = Ninguno):
    # compruebe si se ha realizado alguna acción similar en el último minuto
    ahora = timezone.now ()
    last_minute = ahora - datetime.timedelta (segundos = 60)
    similares_actions = Action.objects.filter (user_id = user.id,
                                                verbo = verbo,
                                                created__gte = last_minute)

    si objetivo:
        target_ct = ContentType.objects.get_for_model (objetivo)
        similares_acciones = similar_actions.filter (
                                                    target_ct = target_ct,
                                                    target_id = target.id)

    si no acciones similares:
        # no se encontraron acciones existentes
```

```

        acción = acción (usuario = usuario, verbo = verbo, objetivo = objetivo)
        action.save ()
        volver verdadero
falso retorno

```

Ha cambiado la función `create_action ()` para evitar guardar acciones duplicadas y devolver booleano para indicarle si la acción se guardó. Así es como evitas duplicados:

- Primero, obtiene la hora actual usando el método `timezone.now ()` proporcionado por Django. Este método hace lo mismo que `datetime.datetime.now ()` pero devuelve un objeto que reconoce la zona horaria. Django proporciona una configuración llamada `USE_TZ` para habilitar o deshabilitar el soporte de zona horaria. El archivo `settings.py` predeterminado creado el uso del comando `startproject` incluye `USE_TZ = True`.
- Utiliza la variable `last_minute` para almacenar la fecha y hora desde un minuto y recuperar cualquier acción idéntica realizada por el usuario desde entonces.
- Crea un objeto Acción si no existe una acción idéntica en la última minuto. Devuelve `True` si se creó un objeto `Action`, o `False` en caso contrario.

Agregar acciones de usuario al flujo de actividades

Es hora de agregar algunas acciones a sus vistas para crear el flujo de actividades para sus usuarios. Almacenará una acción para cada una de las siguientes interacciones:

- Un usuario marca una imagen como favorita
- A un usuario le gusta una imagen
- Un usuario crea una cuenta
- Un usuario comienza a seguir a otro usuario

Edite el archivo `views.py` de la aplicación de imágenes y agregue la siguiente importación:

```

desde actions.utils import create_action

```

En la vista `image_create`, agregue `create_action ()` después de guardar la imagen, así:

```

new_item.save ()
create_action (request.user, 'imagen marcada', new_item)

```

En la vista `image_like`, agregue `create_action ()` después de agregar el usuario a `users_` como relación, como sigue:

```

image.users_like.add (request.user)
create_action (request.user, 'me gusta', imagen)

```

Seguimiento de las acciones del usuario

Ahora, edite el archivo `views.py` de la aplicación de la cuenta y agregue lo siguiente importar:

```

desde actions.utils import create_action

```

En la vista de registro, agregue `create_action ()` después de crear el objeto Perfil, como sigue:

```

Profile.objects.create (usuario = nuevo_usuario)
create_action (new_user, 'ha creado una cuenta')

```

En la vista `user_follow` , agregue `create_action ()` :

```
Contact.objects.get_or_create (user_from = request.user,
                                user_to = usuario)

create_action (request.user, 'está siguiendo', usuario)
```

Como puede ver en el código anterior, gracias a su modelo `Action` y su ayudante función, es muy fácil guardar nuevas acciones en el flujo de actividades.

Visualización del flujo de actividades

Finalmente, necesita una forma de mostrar el flujo de actividades de cada usuario. Incluirás el flujo de actividad en el panel del usuario. Edite el archivo `views.py` de la cuenta `solicitud`. Importe el modelo de acción y modifique la vista del panel de la siguiente manera:

```
desde actions.models import Action

@Necesario iniciar sesión
def dashboard (solicitud):
    # Mostrar todas las acciones por defecto
    acciones = Action.objects.exclude (usuario = solicitud.usuario)
    next_ids = request.user.following.values_list ('id',
                                                    plano = Verdadero)

    si siguientes_ids:
        # Si el usuario está siguiendo a otros, recupere solo sus acciones
        acciones = actions.filter (user_id__in = siguientes_ids)
    acciones = acciones [: 10]

    return render (solicitud,
                    'account / dashboard.html',
                    {'section': 'dashboard',
                     'acciones': acciones })
```

[206]

En la vista anterior, recupera todas las acciones de la base de datos, excluyendo las realizadas por el usuario actual. De forma predeterminada, recupera las últimas acciones realizadas por todos los usuarios de la plataforma. Si el usuario sigue a otros usuarios, restringe el consulta para recuperar solo las acciones realizadas por los usuarios que siguen. Finalmente, limita el resultado a las primeras 10 acciones devueltas. No usa `order_by ()` en `QuerySet` porque confía en el orden predeterminado que proporcionó en el Meta opciones del modelo `Action` . Las acciones recientes serán lo primero desde que configuró `ordering = ('-created',)` en el modelo `Action` .

Optimización de QuerySets que involucran relacionados objetos

Cada vez que recupera un objeto `Action` , normalmente accederá a su `User` relacionado objeto y el objeto de perfil relacionado del usuario . El ORM de Django ofrece una forma sencilla para recuperar objetos relacionados al mismo tiempo, evitando así consultas adicionales a la base de datos.

Usando `select_related ()`

Django ofrece un método `QuerySet` llamado `select_related ()` que le permite recuperar objetos relacionados para relaciones de uno a varios. Esto se traduce en un solo, `QuerySet` más complejo, pero evita consultas adicionales al acceder al objetos relacionados. El método `select_related` es para `ForeignKey` y `OneToOne`

campos. Funciona realizando un SQL JOIN e incluyendo los campos de los relacionados objeto en la instrucción SELECT .

Para aprovechar select_related () , edite la siguiente línea de la anterior código:

```
acciones = acciones [: 10]
```

Además, agregue select_related a los campos que usará, así:

```
acciones = acciones .select_related ('usuario', 'perfil_usuario') [: 10]
```

Utilice user__profile para unirse a la tabla de perfiles en una sola consulta SQL. Si llamas

select_related () sin pasarle ningún argumento, recuperará objetos de todas las relaciones ForeignKey . Limite siempre select_related () a las relaciones al que se accederá posteriormente.

Usar select_related () con cuidado puede mejorar enormemente la ejecución hora.

[207]

Seguimiento de las acciones del usuario

Usando prefetch_related ()

select_related () le ayudará a mejorar el rendimiento para recuperar objetos relacionados en relaciones de uno a varios. Sin embargo, select_related () no funciona para muchos relaciones a varios o varios a uno (campos ManyToMany o ForeignKey inverso). Django ofrece un método QuerySet diferente llamado prefetch_related que funciona para relaciones de muchos a muchos y de muchos a uno, además de las relaciones apoyado por select_related () . El método prefetch_related () realiza una búsqueda separada para cada relación y une los resultados usando Python. Esta El método también admite la captación previa de GenericRelation y GenericForeignKey .

Edite el archivo views.py de la aplicación de la cuenta y complete su consulta agregando prefetch_related () para el campo GenericForeignKey de destino , como sigue:

```
actions = actions.select_related ('usuario', 'perfil_usuario') \
    .prefetch_related ('objetivo') [: 10]
```

Esta consulta ahora está optimizada para recuperar las acciones del usuario, incluidos los objetos relacionados.

Creando plantillas para acciones

Creemos ahora la plantilla para mostrar un objeto de acción en particular . Crear un nuevo directorio dentro del directorio de la aplicación de acciones y asígnele el nombre templates . Añadir la siguiente estructura de archivos:

```
comportamiento/
  acción/
    detail.html
```

Edite el archivo de plantilla actions / action / detail.html y agregue las siguientes líneas lo:

```
{% load thumbnail%}

{% con usuario = action.user profile = action.user.profile%}
<div class = "action">
  <div class = "imágenes">
    {% if profile.photo%}
      {% thumbnail user.profile.photo "80x80" crop = "100%" as im%}
      <a href="{{ user.get_absolute_url }}">
        <img src = "{{ im.url }}" alt = "{{ user.get_full_name }}"
```

```

        class = "item-img">
    </a>
    {% terminara si %}

```

[208]

```

    {% if action.target%}
    {% con target = action.target%}
    {% if target.image%}
    {% thumbnail target.image "80x80" crop = "100%" as im%}
    <a href="{{ target.get_absolute_url }}">
    <img src = "{{ im.url }}" class = "item-img">
    </a>
    {% terminara si %}
    {% terminar con %}
    {% terminara si %}
</div>
<div class = "info">
    <p>
    <span class = "date"> {{action.created | timesince}} hace </span>
    <br />
    <a href="{{ user.get_absolute_url }}">
    {{user.first_name}}
    </a>
    {{ Verbo de acción }}
    {% if action.target%}
    {% con target = action.target%}
    <a href="{{ target.get_absolute_url }}"> {{target}} </a>
    {% terminar con %}
    {% terminara si %}
    </p>
</div>
</div>
{% terminar con %}

```

Esta es la plantilla que se utiliza para mostrar un objeto de acción . Primero, usa el {% con %} etiqueta de plantilla para recuperar el usuario que realiza la acción y el perfil relacionado objeto. Luego, muestra la imagen del objeto de destino si el objeto de acción tiene un objeto de destino relacionado . Finalmente, muestra el enlace al usuario que realizó la acción, el verbo y el objeto de destino , si lo hay.

Edite la plantilla account / dashboard.html de la aplicación de la cuenta y agregue el siguiente código al final del bloque de contenido :

```

<h2> Qué está pasando </h2>
<div id = "lista de acciones">
    {% de acción en acciones%}
    {% incluyen "actions / action / detail.html"%}
    {% endfor%}
</div>

```

[209]

Seguimiento de las acciones del usuario

Abra `http://127.0.0.1:8000/account/` en su navegador. Inicie sesión como existente usuario y realizar varias acciones para que se almacenen en la base de datos. Entonces, inicie sesión utilizando otro usuario, siga al usuario anterior y observe la acción generada transmitir en la página del panel. Debería tener el siguiente aspecto:

Figura 6.5: El flujo de actividad del usuario actual

Acaba de crear un flujo de actividad completo para sus usuarios y puede agregar fácilmente nuevas acciones del usuario. También puede agregar una funcionalidad de desplazamiento infinito a la actividad transmitir implementando el mismo paginador AJAX que usó para la imagen_ vista de lista .

Usando señales para desnormalizar recuentos

Hay algunos casos en los que es posible que desee desnormalizar sus datos.

La desnormalización hace que los datos sean redundantes de tal manera que optimiza la lectura actuación. Por ejemplo, podría estar copiando datos relacionados a un objeto para evitar costosas consultas de lectura a la base de datos al recuperar los datos relacionados. Tienes tener cuidado con la desnormalización y empezar a usarlo solo cuando realmente lo necesite eso. El mayor problema que encontrará con la desnormalización es que es difícil de mantener sus datos desnormalizados actualizados.

[210]

Capítulo 6

Echemos un vistazo a un ejemplo de cómo mejorar sus consultas desnormalizando cuenta. Desnormalizará los datos de su modelo de imagen y usará señales de Django para mantener los datos actualizados.

Trabajando con señales

Django viene con un despachador de señales que permite que las funciones del receptor sean notificadas cuando ocurren ciertas acciones. Las señales son muy útiles cuando necesita su código para hacer algo cada vez que sucede algo más. Las señales le permiten desacoplar la lógica: puede capturar una determinada acción, independientemente de la aplicación o el código que activó esa acción e implementar la lógica que se ejecuta cada vez que ocurre esa acción. Por ejemplo, puede crear una función de receptor de señal que se ejecute cada vez se guarda un objeto de usuario. También puede crear sus propias señales para que otros puedan obtener notificado cuando ocurre un evento.

Django proporciona varias señales para los modelos ubicados en `django.db.models.signals`. Algunas de estas señales son las siguientes:

- `pre_save` y `post_save` se envían antes o después de llamar al método `save()` de un modelo
- `pre_delete` y `post_delete` se envían antes o después de llamar a `delete()` método de un modelo o `QuerySet`
- `m2m_changed` se envía cuando se cambia `ManyToManyField` en un modelo

Estos son solo un subconjunto de las señales proporcionadas por Django. Puede encontrar una lista de todos las señales integradas en <https://docs.djangoproject.com/en/3.0/ref/signals/>.

Supongamos que desea recuperar imágenes por popularidad. Puedes usar Django funciones de agregación para recuperar imágenes ordenadas por el número de usuarios a los que les gusta ellos. Recuerde que utilizó las funciones de agregación de Django en el *Capítulo 3, Ampliación Su aplicación de blog*. El siguiente código recuperará imágenes según su número de me gusta:

```
from django.db.models import Count
from images.models import Image

images_by_popularity = Image.objects.annotate(
    total_likes=Count('users_like')).order_by('-total_likes')
```

Sin embargo, ordenar imágenes contando el total de Me gusta es más caro en términos de rendimiento que ordenarlos por un campo que almacena recuentos totales. Tú puede agregar un campo al modelo de imagen para desnormalizar el número total de me gusta a impulsar el rendimiento en consultas que involucren este campo. El problema es cómo mantener esto campo actualizado.

[211]

Seguimiento de las acciones del usuario

Edite el archivo `models.py` de la aplicación de imágenes y agregue el siguiente `total_likes` al modelo de imagen:

```
class Image(models.Model):
    # ...
    total_likes = models.PositiveIntegerField(db_index=True,
                                           predetermined=0)
```

El campo `total_likes` le permitirá almacenar el recuento total de usuarios a los que les gusta cada imagen. La desnormalización de los recuentos es útil cuando desea filtrar u ordenar `QuerySets` por ellos.

Hay varias formas de mejorar el rendimiento que debe tener en cuenta antes de desnormalizar campos. Considere la base de datos índices, optimización de consultas y almacenamiento en caché antes de comenzar a desnormalizar sus datos.

Ejecute el siguiente comando para crear las migraciones para agregar el nuevo campo al tabla de base de datos:

```
python manage.py makemigrations
```

Debería ver el siguiente resultado:

Migraciones de 'imágenes':

```
images / migrations / 0002_image_total_likes.py
- Agregar campo total_likes a la imagen
```

Luego, ejecute el siguiente comando para aplicar la migración:

```
python manage.py migrar imágenes
```

La salida debe incluir la siguiente línea:

```
Aplicando imágenes.0002_image_total_likes ... OK
```

Necesita adjuntar una función de receptor a la señal `m2m_changed`. Crea un archivo nuevo dentro del directorio de la aplicación de imágenes y asígnele el nombre `Signal.py`. Agregue lo siguiente codificarlo:

```
de django.db.models.signals import m2m_changed
desde el receptor de importación django.dispatch
desde .models importar imagen

@receiver (m2m_changed, sender = Image.users_like.through)
def users_like_changed (remitente, instancia, ** kwargs):
```

[212]

```
instance.total_likes = instance.users_like.count ()
instancia.save ()
```

Primero, registra la función `users_like_changed` como una función receptora usando la receptor () decorador. Lo adjunta a la señal `m2m_changed`. Entonces, te conectas la función a `Image.users_like.through` para que la función solo se llame si el

La señal `m2m_changed` ha sido lanzada por este remitente. Hay un método alternativo para registrar una función de receptor; Consiste en utilizar el método `connect ()` del Objeto de señal.

Las señales de Django son sincrónicas y bloqueantes. No confundas las señales con tareas asincrónicas. Sin embargo, puede combinar ambos para lanzar tareas asincrónicas cuando su código sea notificado por un señal. Aprenderás a crear tareas asincrónicas con Celery en el *Capítulo 7, Creación de una tienda online*.

Tienes que conectar la función de tu receptor a una señal para que se llame cada hora en que se envía la señal. El método recomendado para registrar sus señales es importándolos en el método `ready ()` de la clase de configuración de su aplicación. Django proporciona un registro de aplicaciones que le permite configurar e introspectar sus aplicaciones.

Clases de configuración de aplicaciones

Django te permite especificar clases de configuración para tus aplicaciones. Cuando usted crea una aplicación usando el comando `startapp`, Django agrega un archivo `apps.py` al directorio de la aplicación, incluida una configuración básica de la aplicación que hereda de la clase `AppConfig`.

La clase de configuración de la aplicación le permite almacenar metadatos y la configuración para la aplicación, y proporciona introspección para la aplicación. Puede encontrar más información sobre las configuraciones de la aplicación en <https://docs.djangoproject.com/en/3.0/ref/applications/>.

Para registrar las funciones de su receptor de señal, cuando utilice el receptor () decorador, solo necesita importar el módulo de señales de su aplicación dentro el método `ready ()` de la clase de configuración de la aplicación. Este método se llama

tan pronto como el registro de la aplicación esté completo. Cualquier otra inicialización para su aplicación también debe incluirse en este método.

Edite el archivo apps.py de la aplicación de imágenes y haga que se vea así:

```
desde django.apps importar AppConfig

clase ImagesConfig (AppConfig):
```

[213]

Seguimiento de las acciones del usuario

```
nombre = 'imágenes'

def ready (yo mismo):
    # Importar manejadores de señales
    importar imágenes, señales
```

Importa las señales para esta aplicación en el método ready () para que sean importado cuando se carga la aplicación de imágenes .

Ejecute el servidor de desarrollo con el siguiente comando:

```
python manage.py runserver
```

Abra su navegador para ver una página de detalles de la imagen y haga clic en el botón **ME GUSTA** . Regrese al sitio de administración, navegue hasta la URL de edición de la imagen, como

<http://127.0.0.1:8000/admin/images/image/1/change/> , y echa un vistazo en el atributo total_likes . Debería ver que el atributo total_likes es

actualizado con el número total de usuarios a los que les gusta la imagen, de la siguiente manera:

Figura 6.6: La página de edición de imágenes en el sitio de administración, incluida la desnormalización para el total de Me gusta

Ahora, puede utilizar el atributo total_likes para ordenar las imágenes por popularidad. o mostrar el valor en cualquier lugar, evitando el uso de consultas complejas para calcularlo. Considere la siguiente consulta para ordenar las imágenes según su recuento de Me gusta:

```
de django.db.models import Count

images_by_popularity = Image.objects.annotate (
    likes = Count ('users_like')). order_by ('- likes')
```

La consulta anterior ahora se puede escribir de la siguiente manera:

```
images_by_popularity = Image.objects.order_by ('- total_likes')
```

[214]

Esto da como resultado una consulta SQL menos costosa. Este es solo un ejemplo de cómo usar Señales de Django.

Utilice las señales con precaución, ya que dificultan conocer el flujo de control. En muchos casos, puede evitar el uso de señales si saber qué destinatarios deben ser notificados.

Deberá establecer recuentos iniciales para el resto de los objetos de imagen para que coincidan estado actual de la base de datos. Abra el shell con el shell python manage.py comando y ejecute el siguiente código:

```
desde images.models importar imagen
para la imagen en Image.objects.all ():
    image.total_likes = image.users_like.count ()
    image.save ()
```

El recuento de Me gusta para cada imagen ahora está actualizado.

Usar Redis para almacenar vistas de elementos

Redis es una base de datos de clave / valor avanzada que le permite guardar diferentes tipos de datos. También tiene operaciones de E / S extremadamente rápidas. Redis almacena todo en la memoria, pero los datos se pueden conservar volcando el conjunto de datos en el disco de vez en cuando, o agregando cada comando a un registro. Redis es muy versátil en comparación con otras claves / almacenes de valor: proporciona un conjunto de comandos potentes y admite diversos datos estructuras, como cadenas, hashes, listas, conjuntos, conjuntos ordenados e incluso mapas de bits o HyperLogLogs.

Aunque SQL se adapta mejor al almacenamiento de datos persistentes definidos por esquema, Redis ofrece numerosas ventajas cuando se trata de datos que cambian rápidamente, almacenamiento volátil, o cuando se necesita un caché rápido. Echemos un vistazo a cómo se puede usar Redis para construir una nueva funcionalidad en su proyecto.

Instalación de Redis

Si está utilizando Linux o macOS, descargue la última versión de Redis desde <https://redis.io/download>. Descomprima el archivo tar.gz , ingrese al directorio redis y compile Redis usa el comando make , de la siguiente manera:

```
cd redis-5.0.8
hacer
```

[215]

Seguimiento de las acciones del usuario

Redis ahora está instalado en su máquina. Si está utilizando Windows, el preferido El método para instalar Redis es habilitar el **Subsistema de Windows para Linux (WSL)** e instalarlo en el sistema Linux. Puede leer las instrucciones sobre cómo habilitar WSL

e instalando Redis en <https://redislabs.com/blog/redis-on-windows-10/>.

Después de instalar Redis, use el siguiente comando de shell para iniciar el servidor Redis:

```
src / redis-servidor
```

Debería ver una salida que termina con las siguientes líneas:

```
# Servidor inicializado
```

```
* Listo para aceptar conexiones
```

De forma predeterminada, Redis se ejecuta en el puerto 6379 . Puede especificar un puerto personalizado usando `--port` bandera, por ejemplo, `redis-server --port 6655` .

Mantenga el servidor Redis en funcionamiento y abra otro shell. Inicie el cliente de Redis con el siguiente comando:

```
src / redis-cli
```

Verá el indicador de shell del cliente de Redis, como este:

```
127.0.0.1:6379>
```

El cliente de Redis le permite ejecutar comandos de Redis directamente desde el shell. Vamos prueba algunos comandos. Ingrese el comando `SET` en el shell de Redis para almacenar un valor en un llave:

```
127.0.0.1:6379> SET nombre "Peter"
```

```
Okay
```

El comando anterior crea una clave de nombre con el valor de cadena "Peter" en el Base de datos de Redis. La salida OK indica que la clave se ha guardado correctamente.

A continuación, recupere el valor mediante el comando `GET` , de la siguiente manera:

```
127.0.0.1:6379> OBTENER nombre
```

```
"Peter"
```

También puede verificar si existe una clave usando el comando `EXISTS` . Este comando devuelve 1 si existe la clave dada, y 0 en caso contrario:

```
127.0.0.1:6379> EXISTE nombre
```

```
(entero) 1
```

[216]

Capítulo 6

Puede establecer el tiempo de expiración de una clave mediante el comando `EXPIRE` , que permite configurar el tiempo de vida en segundos. Otra opción es usar el comando `EXPIREAT` , que espera una marca de tiempo Unix. La expiración de la clave es útil para usar Redis como caché o para almacenar datos volátiles:

```
127.0.0.1:6379> OBTENER nombre
```

```
"Peter"
```

```
127.0.0.1:6379> EXPIRE nombre 2
```

```
(entero) 1
```

Espere dos segundos e intente obtener la misma clave nuevamente:

```
127.0.0.1:6379> OBTENER nombre
```

```
(nulo)
```

La respuesta (nula) es una respuesta nula y significa que no se ha encontrado ninguna clave. Tú también puede eliminar cualquier clave usando el comando `DEL` , de la siguiente manera:

```
127.0.0.1:6379> SET total 1
```


Okay

```
127.0.0.1:6379> DEL total
```

(entero) 1

```
127.0.0.1:6379> OBTENER total
```

(nulo)

Estos son solo comandos básicos para operaciones clave. Puedes echar un vistazo a todos los Redis comandos en <https://redis.io/commands> y todos los tipos de datos de Redis en <https://redis.io/topics/data-types>.

Usando Redis con Python

Necesitará enlaces de Python para Redis. Instale redis-py a través de pip usando el siguiente comando:

```
pip install redis == 3.4.1
```

Puede encontrar la documentación de redis-py en <https://redis-py.readthedocs.io/>.

El paquete redis-py interactúa con Redis, proporcionando una interfaz Python que sigue la sintaxis de comandos de Redis. Abra el shell de Python y ejecute el siguiente código:

```
>>> importar redis
```

```
>>> r = redis.Redis(host = 'localhost', puerto = 6379, db = 0)
```

[217]

Seguimiento de las acciones del usuario

El código anterior crea una conexión con la base de datos de Redis. En Redis, bases de datos se identifican mediante un índice entero en lugar de un nombre de base de datos. Por defecto, un cliente es conectado a la base de datos 0. La cantidad de bases de datos de Redis disponibles se establece en 16, pero puede cambiar esto en el archivo de configuración redis.conf.

A continuación, configure una clave usando el shell de Python:

```
>>> r.set('foo', 'bar')
```

Cierto

El comando devuelve True, lo que indica que la clave se ha creado correctamente. Ahora puede recuperar la clave usando el comando get():

```
>>> r.get('foo')
```

b'bar'

Como observará en el código anterior, los métodos de Redis siguen a Redis sintaxis de comandos.

Integremos Redis en su proyecto. Edite el archivo settings.py de los marcadores proyecto y agregue la siguiente configuración:

```
REDIS_HOST = 'localhost'
REDIS_PORT = 6379
REDIS_DB = 0
```

Estas son las configuraciones para el servidor Redis y la base de datos que usará para tu proyecto.

Almacenamiento de vistas de elementos en Redis

Busquemos una manera de almacenar el número total de veces que se ha visto una imagen. Si tu implementar esto usando el Django ORM, implicará una consulta SQL UPDATE cada vez que se muestra una imagen. Si usa Redis en su lugar, solo necesita incrementar un contador almacenado en la memoria, lo que resulta en un rendimiento mucho mejor y menos

Edite el archivo views.py de la aplicación de imágenes y agregue el siguiente código después de las declaraciones de importación existentes :

```
importar redis
desde la configuración de importación django.conf

# conectarse a redis
r = redis.Redis (host = settings.REDIS_HOST,
                 port = settings.REDIS_PORT,
                 db = settings.REDIS_DB)
```

Con el código anterior, establece la conexión Redis para poder usarlo en sus opiniones. Edite el archivo views.py de la aplicación de imágenes y modifique el vista image_detail , como esta:

```
def image_detail (solicitud, id, slug):
    image = get_object_or_404 (Imagen, id = id, slug = slug)
    # Incrementar el total de visualizaciones de imágenes en 1
    total_views = r.incr (f'image: {image.id}: views ')
    return render (solicitud,
                  'images / image / detail.html',
                  {'sección': 'imágenes',
                  'imagen': imagen ,
                  'total_views': total_views })
```

En esta vista, usa el comando incr que incrementa el valor de una clave dada por

1 . Si la clave no existe, el comando incr la crea. El método incr () devuelve el valor final de la clave después de realizar la operación. Almacena el valor en el total_views y pasarla al contexto de la plantilla. Construyes la clave de Redis usando una notación, como object-type: id: field (por ejemplo, image: 33: id).

La convención para nombrar claves de Redis es usar un signo de dos puntos como un separador para crear claves con espacios de nombres. Al hacerlo, la clave Los nombres son especialmente detallados y las claves relacionadas comparten parte del mismo esquema en sus nombres.

Edite la plantilla images / image / detail.html de la aplicación de imágenes y agregue el siguiente código después del elemento existente :

```
<span class = "count">
    {{total_views}} vista {{total_views | pluralize}}
</span>
```

Seguimiento de las acciones del usuario

Ahora, abra una página de detalles de la imagen en su navegador y vuelva a cargarla varias veces. Verá que cada vez que se procesa la vista, el total de vistas mostradas es incrementado en 1. Observe el siguiente ejemplo:

Figura 6.7: La página de detalles de la imagen, incluido el recuento de me gusta y vistas

¡Excelente! Ha integrado Redis con éxito en su proyecto para almacenar recuentos de artículos.

Almacenamiento de una clasificación en Redis

Construyamos algo más complejo con Redis. Creará una clasificación de los imágenes más vistas en su plataforma. Para construir este ranking, usará Redis conjuntos ordenados. Un conjunto ordenado es una colección no repetitiva de cadenas en la que cada miembro está asociado con una puntuación. Los elementos se ordenan por su puntuación.

Edite el archivo `views.py` de la aplicación de imágenes y haga la vista `image_detail` mira como sigue:

```
def image_detail(solicitud, id, slug):
    image = get_object_or_404(Image, id=id, slug=slug)
    # Incrementar el total de visualizaciones de imágenes en 1
    total_views = r.incr(f'image:{image.id}: views ')
    # Incrementar la clasificación de imágenes en 1
    r.zincrby('clasificación_imagen', 1, id.imagen)
    return render(solicitud,
                  'images / image / detail.html',
                  {'sección': 'imágenes',
                  'imagen': imagen,
                  'total_views': total_views})
```

[220]

Utilice el comando `zrange()` para almacenar vistas de imágenes en un conjunto ordenado con el `image`: clave de clasificación. Almacenará la identificación de la imagen y una puntuación relacionada de 1, que se agregará a la puntuación total de este elemento en el conjunto ordenado. Esto te permitirá para realizar un seguimiento de todas las vistas de imágenes a nivel mundial y tener un conjunto ordenado por el total número de vistas.

Ahora, cree una nueva vista para mostrar la clasificación de las imágenes más vistas. Añade el siguiente código al archivo `views.py` de la aplicación de imágenes:

```
@Necesario iniciar sesión
def image_ranking (solicitud):
    # obtener diccionario de clasificación de imágenes
    image_ranking = r.zrange ('image_ranking', 0, -1,
                             desc = Verdadero) [: 10]

    image_ranking_ids = [int (id) para id en image_ranking]
    # obtener las imágenes más vistas
    most_viewed = list (Image.objects.filter (
        id__in = image_ranking_ids))
    most_viewed.sort (key = lambda x: image_ranking_ids.index (x.id))
    return render (solicitud,
                   'images / image / ranking.html',
                   {'sección': 'imágenes',
                    'most_viewed': most_viewed})
```

La vista `image_ranking` funciona así:

1. Utilice el comando `zrange()` para obtener los elementos del conjunto ordenado. Esta El comando espera un rango personalizado de acuerdo con la puntuación más baja y más alta. Usando 0 como la puntuación más baja y -1 como la puntuación más alta, le está diciendo a Redis que devuelve todos los elementos del conjunto ordenado. También especifica `desc = True` para recuperar los elementos ordenados por puntuación descendente. Finalmente, corta los resultados usando `[: 10]` para obtener los primeros 10 elementos con la puntuación más alta.
2. Crea una lista de ID de imagen devueltos y la almacena en `image_ranking_ids` variable como una lista de enteros. Recupera los objetos de imagen para esos ID y forzar la ejecución de la consulta usando la función `list()`. Es importante forzar la ejecución de `QuerySet` porque usará el método de lista `sort()` en `it` (en este punto, necesita una lista de objetos en lugar de un `QuerySet`).
3. Ordena los objetos `Imagen` por su índice de apariencia en la clasificación de imágenes. Ahora puede usar la lista `most_viewed` en su plantilla para mostrar la 10 imágenes más vistas.

[221]

Seguimiento de las acciones del usuario

Crea una nueva plantilla `ranking.html` dentro del directorio `images / image / template` de la aplicación de imágenes y agregue el siguiente código:

```
{% extiende "base.html"%}

{% block title%} Clasificación de imágenes {% endblock%}

{% block content%}
<h1> Clasificación de imágenes </h1>
<ol>
    {% para la imagen en most_viewed%}
        <li>
            <a href="{{ image.get_absolute_url }}">
                {{ titulo de la imagen }}
            </a>
        </li>
    {% endfor%}
</ol>
</div>
```

```
</ol>
{% endblock%}
```

La plantilla es bastante sencilla. Se itera sobre los objetos de imagen contenidos en la lista `most_viewed` y mostrar sus nombres, incluido un enlace al detalle de la imagen página.

Finalmente, debe crear un patrón de URL para la nueva vista. Edite el archivo `urls.py` de la aplicación de imágenes y agregue el siguiente patrón:

```
ruta ('ranking /', views.image_ranking, name = 'ranking'),
```

Ejecute el servidor de desarrollo, acceda a su sitio en su navegador web y cargue la página de detalles de la imagen varias veces para diferentes imágenes. Entonces, acceda `http://127.0.0.1:8000/images/ranking/` desde su navegador. Deberías poder ver una clasificación de imágenes, de la siguiente manera:

Figura 6.8: La página de clasificación creada con datos recuperados de Redis

[222]

¡Excelente! Acabas de crear un ranking con Redis.

Próximos pasos con Redis

Redis no es un reemplazo para su base de datos SQL, pero ofrece una memoria rápida almacenamiento más adecuado para determinadas tareas. Agréguelo a su pila y úselo cuando realmente sientes que es necesario. Los siguientes son algunos escenarios en los que Redis podría sé útil:

- **Contadores** : como has visto, es muy fácil gestionar contadores con Redis. Puede usar `incr()` e `incrby()` para contar cosas.
- **Almacenamiento de elementos más recientes** : puede agregar elementos al inicio / final de una lista usando `lpush()` y `rpush()` . Quite y devuelva el primer / último elemento usando `lpop()` / `rpop()` . Puede recortar la longitud de la lista usando `ltrim()` para mantener su longitud.
- **Colas** : además de los comandos `push` y `pop` , Redis ofrece el bloqueo de comandos de cola.
- **Almacenamiento en caché** : el uso de `expire()` y `expireat()` le permite usar Redis como un caché. También puede encontrar backends de caché de Redis de terceros para Django.
- **Pub / sub** : Redis proporciona comandos para suscribirse / cancelar suscripción y envío de mensajes a canales.
- **Clasificaciones y tablas de clasificación** : los conjuntos ordenados de Redis con puntuaciones lo hacen muy fácil para crear tablas de clasificación.
- **Seguimiento en tiempo real** : la E / S rápida de Redis lo hace perfecto para escenarios en tiempo real.

Resumen

En este capítulo, ha creado un sistema de seguimiento utilizando relaciones de varios a varios con un modelo intermediario. También creó un flujo de actividades utilizando relaciones genéricas y optimizaste QuerySets para recuperar objetos relacionados. Este capítulo luego presentó a las señales de Django, y creó una función de receptor de señal para desnormalizar recuentos de objetos relacionados. Cubrimos las clases de configuración de la aplicación, que utilizó para cargar sus manejadores de señales. También aprendió a instalar y configurar Redis en su proyecto Django. Finalmente, usó Redis en su proyecto para almacenar vistas de elementos, y construyó una clasificación de imágenes con Redis.

En el próximo capítulo, aprenderá a crear una tienda en línea. Tu vas a crear un catálogo de productos y crear un carrito de compras mediante sesiones. También descubrirás cómo iniciar tareas asincrónicas usando Celery.

[223]

7

Creación de una tienda online

En el capítulo anterior, creó un sistema de seguimiento y creó un flujo de actividad del usuario. También aprendió cómo funcionan las señales de Django e integró Redis en su proyecto para contar vistas de imágenes.

En este capítulo, iniciará un nuevo proyecto de Django que consta de una Tienda online. Este capítulo y los dos capítulos siguientes le mostrarán cómo construir las funcionalidades esenciales de una plataforma de comercio electrónico. Tu tienda online permitirá clientes para buscar productos, agregarlos al carrito, aplicar códigos de descuento, revisar el proceso de pago, pague con tarjeta de crédito y obtenga una factura. También lo harás implementar un motor de recomendación para recomendar productos a sus clientes, y utilizará la internacionalización para ofrecer su sitio en varios idiomas.

En este capítulo, aprenderá a:

- Crear un catálogo de productos
- Construya un carrito de compras usando sesiones de Django
- Cree procesadores de contexto personalizados
- Gestionar pedidos de clientes
- Configure Apio en su proyecto con RabbitMQ como intermediario de mensajes
- Envíe notificaciones asincrónicas a los clientes que utilizan Celery
- Controle el apio con flor

[225]

Creación de una tienda online

Creación de un proyecto de tienda online

Comencemos con un nuevo proyecto de Django para construir una tienda en línea. Tus usuarios serán capaz de navegar a través de un catálogo de productos y agregar productos a un carrito de compras. Finalmente, podrán revisar el carrito y realizar un pedido. Este capítulo cubrirá las siguientes funcionalidades de una tienda online:

- Crear los modelos del catálogo de productos, agregarlos al sitio de administración, y construyendo las vistas básicas para mostrar el catálogo
- Construir un sistema de carrito de compras usando sesiones de Django para permitir a los usuarios mantener productos seleccionados mientras navegan por el sitio
- Crear el formulario y la funcionalidad para realizar pedidos en el sitio.
- Enviar una confirmación por correo electrónico asincrónico a los usuarios cuando colocan una orden

Abra un shell, cree un entorno virtual para el nuevo proyecto y actívelo con los siguientes comandos:

mkdir env

python3 -m venv env / myshop

fuente env / myshop / bin / activar

Instale Django en su entorno virtual con el siguiente comando:

pip instalar Django == 3.0. *

Inicie un nuevo proyecto llamado myshop con una aplicación llamada shop abriendo un shell y ejecutando los siguientes comandos:

django-admin startproject myshop

cd myshop /

tienda django-admin startapp

Edite el archivo settings.py de su proyecto y agregue la aplicación de la tienda al Configuración INSTALLED_APPS de la siguiente manera:

```
INSTALLED_APPS = [  
    # ...  
    'shop.apps.ShopConfig',  
]
```

Su aplicación ahora está activa para este proyecto. Definamos los modelos para el catalogo de producto.

Creación de modelos de catálogo de productos

El catálogo de tu tienda estará formado por productos que se organizan en diferentes categorías. Cada producto tendrá un nombre, una descripción opcional, una imagen, precio y disponibilidad.

Edite el archivo `models.py` de la aplicación de la tienda que acaba de crear y agregue el siguiente código:

```

desde modelos de importación django.db

class Categoría (modelos.Modelo):
    name = models.CharField (max_length = 200,
                             db_index = Verdadero)
    slug = models.SlugField (max_length = 200,
                             único = verdadero)

    class Meta:
        ordering = ('nombre',)
        verbose_name = 'categoría'
        verbose_name_plural = 'categorías'

    def __str__ (yo):
        return self.name

class Producto (modelos.Modelo):
    categoría = modelos.ForeignKey (Categoría,
                                    related_name = 'productos',
                                    on_delete = modelos.CASCADE)
    name = models.CharField (max_length = 200, db_index = True)
    slug = models.SlugField (max_length = 200, db_index = True)
    image = modelos.ImageField (upload_to = 'productos /% Y /% m /% d',
                                en blanco = Verdadero)
    descripción = modelos.TextField (en blanco = verdadero)
    precio = modelos.DecimalField (max_digits = 10, decimal_places = 2)
    disponible = modelos.BooleanField (predeterminado = Verdadero)
    creado = modelos.DateTimeField (auto_now_add = True)
    actualizado = modelos.DateTimeField (auto_now = True)

    class Meta:
        ordering = ('nombre',)
        index_together = (('id', 'slug'),)

    def __str__ (yo):
        return self.name

```

[227]

Creación de una tienda online

Estos son los modelos de Categoría y Producto . El modelo de Categoría consta de un nombre field y un campo slug único (único implica la creación de un índice). El producto Los campos del modelo son los siguientes:

- categoría : una ForeignKey para el modelo de categoría . Este es un uno a muchos relación: un producto pertenece a una categoría y una categoría contiene múltiples productos.
- nombre : el nombre del producto.
- slug : el slug de este producto para crear hermosas URL.
- imagen : una imagen de producto opcional.

- descripción : una descripción opcional del producto.
- precio : este campo utiliza el tipo decimal de Python . Tipo decimal para almacenar un número decimal de precisión. El número máximo de dígitos (incluido los lugares decimales) se establece utilizando el atributo max_digits y decimal lugares con el atributo decimal_places .
- disponible : un valor booleano que indica si el producto está disponible o no. Se utilizará para habilitar / deshabilitar el producto en el catálogo.
- creado : este campo almacena cuándo se creó el objeto.
- actualizado : este campo almacena cuándo se actualizó el objeto por última vez.

Para el campo de precio , usa DecimalField en lugar de FloatField para evitar cuestiones de redondeo.

Utilice siempre DecimalField para almacenar cantidades monetarias. FloatField usa el tipo de flotación de Python internamente, mientras que DecimalField usa el tipo decimal de Python. Usando el Tipo decimal, evitará problemas de redondeo flotante.

En la clase Meta del modelo Producto , usa la meta opción index_together para especificar un índice para los campos id y slug juntos. Define este índice porque planea consultar productos tanto por id como por slug . Ambos campos están indexados juntos para mejorar el rendimiento de las consultas que utilizan los dos campos.

Dado que va a tratar con imágenes en sus modelos, abra el shell e instale

Almohada con el siguiente comando:

```
pip install Pillow == 7.0.0
```

Ahora ejecute el siguiente comando para crear migraciones iniciales para su proyecto:

```
python manage.py makemigrations
```

[228]

Verá el siguiente resultado:

Migraciones para 'tienda':

```
shop / migrations / 0001_initial.py
```

```
- Crear categoría de modelo
```

```
- Crear producto modelo
```

Ejecute el siguiente comando para sincronizar la base de datos:

```
python manage.py migrar
```

Verá un resultado que incluye la siguiente línea:

```
Aplicando shop.0001_initial ... OK
```

La base de datos ahora está sincronizada con sus modelos.

Registro de modelos de catálogo en el sitio de administración

Agreguemos sus modelos al sitio de administración para que pueda administrar fácilmente categorías y productos. Edite el archivo admin.py de la aplicación de la tienda y agregue el siguiente código:

```
desde django.contrib import admin
de .models importar Categoría, Producto
```

```
@admin.register(Categoría)
class CategoryAdmin(admin.ModelAdmin):
    list_display = ['nombre', 'slug']
    campos_prepoplados = {'slug': ('nombre',)}

@admin.register(Producto)
class ProductAdmin(admin.ModelAdmin):
    list_display = ['nombre', 'slug', 'precio',
                  'disponible', 'creado', 'actualizado']
    list_filter = ['disponible', 'creado', 'actualizado']
    list_editable = ['precio', 'disponible']
    campos_prepoplados = {'slug': ('nombre',)}
```

Recuerde que usa el atributo `prepopulated_fields` para especificar campos donde el valor se establece automáticamente utilizando el valor de otros campos. Como has visto antes, esto es conveniente para generar babosas.

[229]

Creación de una tienda online

Se utiliza el `list_editable` atributo en el `ProductAdmin` clase para configurar los campos que se puede editar desde la página de visualización de lista del sitio de administración. Esto permitira que edite varias filas a la vez. Cualquier campo en `list_editable` también debe incluirse en el atributo `list_display` , ya que solo se pueden editar los campos mostrados.

Ahora cree un superusuario para su sitio usando el siguiente comando:

python manage.py crea superusuario

Inicie el servidor de desarrollo con el comando `python manage.py runserver` .

Abra `http://127.0.0.1:8000/admin/shop/product/add/` en su navegador y inicie sesión con el usuario que acaba de crear. Agregue una nueva categoría y producto usando la interfaz de administración. La página de la lista de cambios de producto de la administración. la página se verá así:

Figura 7.1: La página de la lista de cambios de producto

Creación de vistas de catálogo

Para mostrar el catálogo de productos, debe crear una vista para enumerar todos los productos o filtrar productos por una categoría determinada. Edite el archivo `views.py` del tienda de aplicaciones y agregue el siguiente código:

```
desde django.shortcuts import render, get_object_or_404
```

```
def product_list(solicitud, category_slug = None):
    categoría = Ninguno
    categorías = Categoría.objetos.todos ()
    productos = Product.objects.filter (disponible = Verdadero)
    si category_slug:
        categoría = get_object_or_404 (Categoría, slug = category_slug)
        productos = productos.filtro (categoría = categoría)
    return render (solicitud,
                   'shop / product / list.html',
                   {'categoría': categoría,
                   'categorías': categorías,
                   'productos': productos})
```

En el código anterior, filtra el QuerySet con `available = True` para recuperar solo Productos disponibles. Utiliza un parámetro `category_slug` opcional para opcionalmente filtrar productos por una categoría determinada.

También necesita una vista para recuperar y mostrar un solo producto. Agregue lo siguiente ver el archivo `views.py` :

```
def product_detail(solicitud, id, slug):
    product = get_object_or_404 (Producto,
                                id = id,
                                babosa = babosa,
                                disponible = Verdadero)

    return render (solicitud,
                   'shop / product / detail.html',
                   {'producto': producto})
```

La vista `product_detail` espera los parámetros `id` y `slug` para recuperar la instancia del `Producto` . Puede obtener esta instancia solo a través del ID, ya que es un atributo único. Sin embargo, incluye el `slug` en la URL para compilar URL optimizadas para SEO para productos.

Después de crear la lista de productos y las vistas detalladas, debe definir patrones de URL para ellos. Cree un nuevo archivo dentro del directorio de la aplicación de la tienda y asígnele un nombre URL. `py` . Agregue el siguiente código:

```
desde la ruta de importación django.urls
de . importar vistas

app_name = 'tienda'

urlpatterns = [
    ruta ('', views.product_list, name = 'product_list'),
```

Creación de una tienda online

```
    ruta ('<slug: category_slug> /', views.product_list,
        name = 'product_list_by_category'),
    ruta ('<int: id> / <slug: slug> /', views.product_detail,
        name = 'product_detail'),
]
```

Estos son los patrones de URL para su catálogo de productos. Has definido dos diferentes patrones de URL para la vista `product_list` : un patrón llamado `product_list` , que llama a la vista `product_list` sin ningún parámetro, y un patrón llamado `product_list_by_category` , que proporciona un parámetro `category_slug` al vista para filtrar productos según una categoría determinada. Agregaste un patrón para la vista `product_detail` , que pasa los parámetros `id` y `slug` a la vista para recuperar un producto específico.

Editar la `urls.py` archivo de la MyShop proyecto para que se vea como esto:

```
desde django.contrib import admin
de la ruta de importación django.urls , incluir

urlpatterns = [
    ruta ('admin /', admin.site.urls),
    ruta ('', include ('shop.urls', namespace = 'shop')),
]
```

En los principales patrones de URL del proyecto, incluye las URL de la aplicación de la tienda . bajo un espacio de nombres personalizado llamado `tienda` .

A continuación, edite el archivo `models.py` de la aplicación de la tienda , importe la función `reverse()` , y agregue un método `get_absolute_url()` a los modelos de Categoría y Producto como sigue:

```
desde django.urls import reverse
# ...
clase Categoría (modelos.Modelo):
    # ...
    def get_absolute_url (yo):
        return reverse ('tienda: lista_productos_por_categoria',
                       args = [self.slug])

clase Producto (modelos.Modelo):
    # ...
    def get_absolute_url (yo):
        return reverse ('tienda: product_detail',
                       args = [self.id, self.slug])
```

[232]

Como ya sabe, `get_absolute_url()` es la convención para recuperar el URL de un objeto determinado. Aquí, usa los patrones de URL que acaba de definir en el archivo `urls.py`.

Crear plantillas de catálogo

Ahora necesita crear plantillas para la lista de productos y las vistas detalladas. Crear el siguiente directorio y estructura de archivos dentro del directorio de la aplicación de la tienda :

```
plantillas/  
  tienda/  
    base.html  
  producto/  
    list.html  
    detail.html
```

Debe definir una plantilla base y luego extenderla en la lista de productos y detalles plantillas. Edite la plantilla shop / base.html y agregue el siguiente código:

```
{% load static%}  
<!-- DOCTYPE html -->  
<html>  
  <head>  
    <meta charset = "utf-8" />  
    <title> {% block title%} Mi tienda {% endblock%} </title>  
    <link href = "{% static "css / base.css "%}" rel = "stylesheet">  
  </head>  
  <body>  
    <div id = "header">  
      <a href = "/" class = "logo"> Mi tienda </a>  
    </div>  
    <div id = "subheader">  
      <div class = "carrito">  
        Tu carrito esta vacío.  
      </div>  
    </div>  
    <div id = "contenido">  
      {% block content%}  
      {% endblock%}  
    </div>  
  </body>  
</html>
```

[233]

Creación de una tienda online

Esta es la plantilla base que utilizará para su tienda. Para incluir el Estilos e imágenes CSS que utilizan las plantillas, debe copiar el archivo estático. archivos que acompañan a este capítulo, que se encuentran en el directorio static / del aplicación de la tienda . Cópielos en la misma ubicación en su proyecto. Puedes encontrar el contenido del directorio en <https://github.com/PacktPublishing/Django-3-by-Example/tree/master/Chapter07/myshop/shop/static> .

Edite la plantilla shop / product / list.html y agréguele el siguiente código:

```
{% extends "shop / base.html"%}  
{% load static%}  
  
{% block title%}  
  {% if category%} {{category.name}} {% else%} Productos {% endif%}  
{% endblock%}  
  
{% block content%}  
  <div id = "barra lateral">  
    <h3> Categorías </h3>  
    <ul>  
      <li {% si no es categoría%} class = "selected" {% endif%}>  
        <a href = "{% url "shop:product_list" %}"> Todo </a>  
      </li>  
      {% para c en categorías%}
```

```

        <li {% if category.slug == c.slug%} class = "selected"
        {% endif%}>
            <a href="{{ c.get_absolute_url }}"> {{c.name}} </a>
        </li>
    {% endfor%}
</ul>
</div>
<div id = "main" class = "lista-de-productos">
    <h1> {% if category%} {{category.name}} {% else%} Productos
    {% endif%} </h1>
    {% para producto en productos%}
        <div class = "elemento">
            <a href="{{ product.get_absolute_url }}">
                <img src = "{% if product.image%} {{product.image.url}} {%
else%} {% static "img / no_image.png"%} {% endif%} ">
            </a>
            <a href="{{ product.get_absolute_url }}"> {{product.name}} </
a>
            <br>
            $ {{product.price}}
        </div>
    
```

[234]

```

    {% endfor%}
</div>
{% endblock%}

```

Asegúrese de que ninguna etiqueta de plantilla se divida en varias líneas.

Esta es la plantilla de lista de productos. Extiende la plantilla shop / base.html y utiliza la variable de contexto de categorías para mostrar todas las categorías en una barra lateral, y productos para mostrar los productos de la página actual. Se utiliza la misma plantilla para ambos enumeran todos los productos disponibles y enumeran los productos filtrados por categoría. Ya que el campo de imagen del modelo del producto puede estar en blanco, debe proporcionar un valor predeterminado imagen para los productos que no tienen imagen. La imagen se encuentra en su estática. directorio de archivos con la ruta relativa img / no_image.png .

Dado que está utilizando ImageField para almacenar imágenes de productos, necesita el desarrollo servidor para servir archivos de imagen cargados.

Edite el archivo settings.py de myshop y agregue la siguiente configuración:

```

MEDIA_URL = '/ media /'
MEDIA_ROOT = os.path.join (BASE_DIR, 'media /')

```

MEDIA_URL es la URL base que ofrece archivos multimedia cargados por los usuarios. MEDIA_ROOT es la ruta local donde residen estos archivos, que se construye anteponiendo dinámicamente la variable BASE_DIR .

Para que Django sirva los archivos multimedia cargados usando el servidor de desarrollo, edite el urls.py principal de myshop y agregue el siguiente código:

```

desde la configuración de importación django.conf
desde django.conf.urls.static import static

urlpatterns = [
    # ...
]
si la configuración.DEBUG:
    urlpatterns += static (settings.MEDIA_URL,
                           document_root = settings.MEDIA_ROOT)

```

Recuerde que solo sirve archivos estáticos de esta manera durante el desarrollo. En un entorno de producción, nunca debería servir archivos estáticos con Django; el El servidor de desarrollo Django no sirve archivos estáticos de manera eficiente. *Capítulo*

14 , *Going Live* , le enseñará cómo entregar archivos estáticos en un entorno de producción.

[235]

Página 261

Creación de una tienda online

Agregue un par de productos a su tienda usando el sitio de administración y abra <http://127.0.0.1:8000/> en su navegador. Verá la página de la lista de productos, que se verá similar a esto:

Figura 7.2: La página de la lista de productos

Si crea un producto utilizando el sitio de administración y no carga ninguna imagen para En su lugar, se mostrará la imagen predeterminada `no_image.png` :

Figura 7.3: La lista de productos que muestra una imagen predeterminada para los productos que no tienen imagen

Edite la plantilla `shop / product / detail.html` y agréguele el siguiente código:

```
{% extiende "shop / base.html"%}
{% load static%

{% block title%
    {{ nombre del producto }}
{% endblock%}

{% block content%
    <div class = "product-detail">
```

[236]

```
<img src = "{% if product.image%} {{product.image.url}} {% else%}
{% static 'img / no_image.png"%} {% endif%} ">
<h1> {{product.name}} </h1>
<h2>
  <a href="{ {{ product.category.get_absolute_url }}}">
    {{ categoria de producto }}
  </a>
</h2>
<p class = "price"> $ {{product.price}} </p>
{{product.description | linebreaks}}
</div>
{% endblock%}
```

En el código anterior, llamas al método `get_absolute_url()` en el objeto de categoría para mostrar los productos disponibles que pertenecen a la misma categoría.

Ahora abra `http://127.0.0.1:8000/` en su navegador y haga clic en cualquier producto para consulte la página de detalles del producto. Se verá de la siguiente manera:

Figura 7.4: La página de detalles del producto

Ahora ha creado un catálogo de productos básico.

Construyendo un carrito de compras

Después de crear el catálogo de productos, el siguiente paso es crear un carrito de compras para que los usuarios puedan elegir los productos que desean comprar. Un carrito de compras permite que los usuarios seleccionen productos y establezcan la cantidad que desean pedir, y luego almacenen esta información temporalmente mientras navegan por el sitio, hasta que finalmente colocan un orden. El carrito debe persistir en la sesión para que se mantengan los artículos del carrito durante la visita de un usuario.

[237]

Creación de una tienda online

Utilizará el marco de sesión de Django para conservar el carrito. El carrito se mantendrá en la sesión hasta que finalice o el usuario se retire del carrito. También necesitará Cree modelos de Django adicionales para el carrito y sus artículos.

Usando sesiones de Django

Django proporciona un marco de sesión que admite sesiones anónimas y de usuario.

El marco de la sesión le permite almacenar datos arbitrarios para cada visitante. Sesión

Los datos se almacenan en el lado del servidor y las cookies contienen el ID de sesión a menos que utilice

el motor de sesión basado en cookies. El middleware de sesión gestiona el envío y

recepción de cookies. El motor de sesión predeterminado almacena los datos de la sesión en la base de datos, pero puede elegir otros motores de sesión.

Para usar sesiones, debe asegurarse de que la configuración de MIDDLEWARE de su proyecto

contiene 'django.contrib.sessions.middleware.SessionMiddleware' . Esta

middleware gestiona sesiones. Se agrega de forma predeterminada a la configuración de MIDDLEWARE

cundo crea un nuevo proyecto usando el comando startproject .

El middleware de sesión hace que la sesión actual esté disponible en el objeto de solicitud .

Puede acceder a la sesión actual usando request.session , tratándola como un Python

diccionario para almacenar y recuperar datos de la sesión. El diccionario de sesión acepta cualquier

Objeto de Python de forma predeterminada que se puede serializar en JSON. Puede establecer una variable en la sesión como esta:

```
request.session ['foo'] = 'bar'
```

Recupere una clave de sesión de la siguiente manera:

```
request.session.get ('foo')
```

Elimine una clave que almacenó previamente en la sesión de la siguiente manera:

```
del request.session ['foo']
```

Cuando los usuarios inician sesión en el sitio, su sesión anónima se pierde y se crea una nueva sesión para usuarios autenticados. Si almacena artículos en una sesión anónima que debe mantener después de que el usuario inicie sesión , deberá copiar los datos de la sesión anterior en la nueva sesión. Puede hacer esto recuperando los datos de la sesión antes de iniciar sesión el usuario que usa la función login () de la autenticación de Django sistema y almacenarlo en la sesión posterior.

[238]

Configuración de la sesión

Hay varias configuraciones que puede usar para configurar sesiones para su proyecto. los

el más importante es SESSION_ENGINE . Esta configuración le permite establecer el lugar donde

las sesiones se almacenan. De forma predeterminada, Django almacena sesiones en la base de datos usando el

Modelo de sesión de la aplicación django.contrib.sessions .

Django ofrece las siguientes opciones para almacenar datos de sesión:

- **Sesiones de base de datos** : los datos de la sesión se almacenan en la base de datos. Este es el predeterminado motor de sesión.
- **Sesiones basadas en archivos** : los datos de la sesión se almacenan en el sistema de archivos.
- **Sesiones en caché** : los datos de la sesión se almacenan en un backend de caché. Puede especificar caché de backends usando la configuración CACHES . Almacenar datos de la sesión en una caché El sistema proporciona el mejor rendimiento.
- **Sesiones de base de datos en caché** : los datos de la sesión se almacenan en un caché de escritura simultánea y base de datos. Las lecturas solo usan la base de datos si los datos aún no están en el cache.
- **Sesiones basadas en cookies**: los datos de la sesión se almacenan en las cookies que se envían

al navegador.

Para un mejor rendimiento, utilice un motor de sesión basado en caché. Django admite Memcached listo para usar y puede encontrar terceros backends de caché para Redis y otros sistemas de caché.

Puede personalizar sesiones con configuraciones específicas. Éstos son algunos de los importantes configuraciones relacionadas con la sesión:

- `SESSION_COOKIE_AGE` : La duración de las cookies de sesión en segundos. El valor predeterminado es 1209600 (dos semanas).
- `SESSION_COOKIE_DOMAIN` : El dominio utilizado para las cookies de sesión. Establecer esto en `midominio.com` para habilitar las cookies entre dominios o utilizar `Ninguno` para un estándar cookie de dominio.
- `SESSION_COOKIE_SECURE` : un booleano que indica que la cookie solo debe enviarse si la conexión es HTTPS.
- `SESSION_EXPIRE_AT_BROWSER_CLOSE` : un booleano que indica que la sesión tiene que caducar cuando se cierra el navegador.
- `SESSION_SAVE_EVERY_REQUEST` : un booleano que, si es verdadero, guardará la sesión a la base de datos en cada solicitud. La caducidad de la sesión también se actualiza cada tiempo se ahorra.

[239]

Creación de una tienda online

Puede ver todos los ajustes de la sesión y sus valores predeterminados en <https://docs.djangoproject.com/en/3.0/ref/settings/#sessions>.

Caducidad de la sesión

Puede optar por utilizar sesiones de duración del navegador o sesiones persistentes mediante el ajuste `SESSION_EXPIRE_AT_BROWSER_CLOSE`. Esto está configurado como Falso de forma predeterminada, forzando la duración de la sesión al valor almacenado en la configuración `SESSION_COOKIE_AGE`. Si establece `SESSION_EXPIRE_AT_BROWSER_CLOSE` en True, la sesión caducará cuando el usuario cierra el navegador y la configuración `SESSION_COOKIE_AGE` no tener algún efecto.

Puede utilizar el método `set_expiry()` de `request.session` para sobrescribir el duración de la sesión actual.

Almacenar carritos de compras en sesiones

Necesita crear una estructura simple que se pueda serializar en JSON para almacenar el carrito elementos en una sesión. El carrito debe incluir los siguientes datos para cada artículo contenido en eso:

- El ID de una instancia de producto
- La cantidad seleccionada para el producto
- El precio unitario del producto

Dado que los precios de los productos pueden variar, adoptemos el enfoque de almacenar el precio del producto junto con el producto en sí cuando se agrega al carrito. Al hacerlo, utiliza el precio actual del producto cuando los usuarios lo agregan a su carrito, sin importar si el precio del producto se cambia posteriormente. Esto significa que el precio que tiene el artículo cuando el cliente lo agrega al carrito se mantiene para ese cliente en la sesión hasta el pago se completa o la sesión termina.

A continuación, debe crear funciones para crear carritos de compras y asociarlos con sesiones. Esto tiene que funcionar de la siguiente manera:

- Cuando se necesita un carrito, comprueba si se ha configurado una clave de sesión personalizada. Si no se configura ningún carrito en la sesión, usted crea un carrito nuevo y lo guarda en el carrito clave de sesión.
- Para sucesivas solicitudes, realiza la misma verificación y obtiene los artículos del carrito desde la clave de sesión del carrito. Recupera los artículos del carrito de la sesión y sus objetos de Producto relacionados de la base de datos.

[240]

Edite el archivo settings.py de su proyecto y agregue la siguiente configuración:

```
CART_SESSION_ID = 'carrito'
```

Esta es la clave que vas a utilizar para almacenar el carrito en la sesión de usuario. Ya que Las sesiones de Django se administran por visitante, puede usar la misma clave de sesión de carrito para todas las sesiones.

Creemos una aplicación para administrar carritos de compras. Abra la terminal y cree una nueva aplicación, ejecutando el siguiente comando desde el directorio del proyecto:

carrito python manage.py startapp

Luego, edite el archivo settings.py de su proyecto y agregue la nueva aplicación al Configuración INSTALLED_APPS , de la siguiente manera:

```
INSTALLED_APPS = [
    # ...
    'shop.apps.ShopConfig',
    'cart.apps.CartConfig',
]
```

Crear un nuevo archivo dentro de la cesta directorio de la aplicación y el nombre que cart.py . Añade el siguiente código:

```
desde decimal import Decimal
desde la configuración de importación django.conf
desde shop.models import Product
```

clase Carrito (objeto):

```
def __init__(self, request):
    """
    Inicialice el carrito.
    """
    self.session = request.session
    carrito = self.session.get(settings.CART_SESSION_ID)
    si no carro:
        # guardar un carrito vacío en la sesión
        carrito = self.session[settings.CART_SESSION_ID] = {}
    self.cart = carrito
```

Esta es la clase Carrito que le permitirá administrar el carrito de compras. Necesitas el carrito se inicializará con un objeto de solicitud . Almacena la sesión actual usando self.session = request.session para que sea accesible a los otros métodos del Clase de carrito .

[241]

Creación de una tienda online

Primero, intenta obtener el carrito de la sesión actual utilizando `self.session`.

obtener (`settings.CART_SESSION_ID`). Si no hay carrito presente en la sesión, crea un carrito vacío configurando un diccionario vacío en la sesión.

Construirá su diccionario de carrito con ID de producto como claves y para cada producto clave, un diccionario será un valor que incluye cantidad y precio. Al hacer esto, puede garantizar que un producto no se agregará más de una vez al carrito. De esta manera, también puede simplificar la recuperación de artículos del carrito.

Creemos un método para agregar productos al carrito o actualizar su cantidad. Añade el siguientes métodos `add()` y `save()` a la clase `Cart`:

```
class Carrito (objeto):
    # ...
    def add (self, product, amount = 1, override_quantity = False):
        """
        Agregue un producto al carrito o actualice su cantidad.
        """
        product_id = str (product.id)
        si product_id no está en self.cart:
            self.cart [product_id] = {'cantidad': 0,
                                      'precio': str (producto.precio)}

        si override_quantity:
            self.cart [product_id] ['cantidad'] = cantidad
        más:
            self.cart [product_id] ['cantidad'] += cantidad
        self.save ()

    def guardar (auto):
        # marcar la sesión como "modificada" para asegurarse de que se guarde
        self.session.modified = Verdadero
```

El método `add()` toma los siguientes parámetros como entrada:

- `producto`: la instancia de producto para agregar o actualizar en el carrito.
- `cantidad`: un entero opcional con la cantidad de producto. Este valor predeterminado es 1.
- `override_quantity`: este es un booleano que indica si la cantidad debe anularse con la cantidad dada (`Verdadero`), o si el nuevo La cantidad debe agregarse a la cantidad existente (`Falso`).

Utiliza el ID del producto como clave en el diccionario de contenido del carrito. Usted convierte el ID de producto en una cadena porque Django usa JSON para serializar datos de sesión, y JSON solo permite nombres de clave de cadena. El ID de producto es la clave y el valor que persiste es un diccionario con cifras de cantidad y precio del producto. El precio del producto se convierte de decimal a una cadena para serializar eso. Finalmente, llama al método `save()` para guardar el carrito en la sesión.

[242]

El método `save()` marca la sesión como modificada usando `session.modified =` Es cierto. Esto le dice a Django que la sesión ha cambiado y debe guardarse.

También necesita un método para retirar productos del carrito. Agregue lo siguiente método a la clase Cart :

```
clase Carrito (objeto):
# ...
def remove (self, product):
    """
    Retire un producto del carrito.
    """
    product_id = str (product.id)
    si product_id en self.cart:
        del self.cart [product_id]
    self.save ()
```

El método remove () elimina un producto dado del diccionario del carrito y llama el método save () para actualizar el carrito en la sesión.

Tendrá que recorrer los artículos contenidos en el carrito y acceder a los Instancias de productos . Para hacerlo, puede definir un método __iter__ () en su clase. Agregue el siguiente método a la clase Cart :

```
clase Carrito (objeto):
# ...
def __iter__ (yo):
    """
    Repita los artículos del carrito y obtenga los productos.
    de la base de datos.
    """
    product_ids = self.cart.keys ()
    # obtener los objetos del producto y agregarlos al carrito
    productos = Product.objects.filter (id__in = product_ids)

    carrito = self.cart.copy ()
    para producto en productos:
        carrito [str (product.id)] ['producto'] = producto

    para artículo en cart.values ():
        artículo ['precio'] = Decimal (artículo ['precio'])
        artículo ['precio_total'] = artículo ['precio'] * artículo ['cantidad']
        artículo de rendimiento
```

[243]

Creación de una tienda online

En el método __iter__ () , recupera las instancias de Producto que están presentes en el carrito para incluirlos en los artículos del carrito. Copias el carrito actual en el carrito variable y agréguele las instancias de Producto . Finalmente, itera sobre los elementos del carrito, convertir el precio de cada artículo en decimal y agregar un atributo total_price a cada artículo. Este método __iter__ () le permitirá iterar fácilmente sobre los elementos en el carrito en vistas y plantillas.

También necesita una forma de devolver el número total de artículos en el carrito. Cuando el len () La función se ejecuta en un objeto, Python llama a su método __len__ () para recuperar su longitud. A continuación, va a definir un método __len__ () personalizado para devolver el total número de artículos almacenados en el carrito.

Agregue el siguiente método __len__ () a la clase Cart :

```
clase Carrito (objeto):
# ...
def __len__ (yo):
    """
```

```
Cuenta todos los artículos del carrito.
```

```
""" "
```

```
devolver suma (artículo ['cantidad'] para el artículo en self.cart.values ()
```

Devuelve la suma de las cantidades de todos los artículos del carrito.

Agregue el siguiente método para calcular el costo total de los artículos en el carrito:

```
clase Carrito (objeto):
```

```
# ...
```

```
def get_total_price (auto):
```

```
    devuelve suma (decimal (artículo ['precio']) * artículo ['cantidad'] por artículo
en self.cart.values ())
```

Finalmente, agregue un método para borrar la sesión del carrito:

```
clase Carrito (objeto):
```

```
# ...
```

```
def clear (auto):
```

```
    # eliminar carrito de la sesión
```

```
    del self.session [settings.CART_SESSION_ID]
```

```
    self.save ()
```

Su clase Carrito ahora está lista para administrar carritos de compras.

[244]

Crear vistas de carrito de compras

Ahora que tiene una clase de carrito para administrar el carrito, debe crear las vistas para agregar, actualizar o eliminar elementos. Necesita crear las siguientes vistas:

- Una vista para agregar o actualizar artículos en el carrito que pueden manejar nuevos y actuales cantidades
- Una vista para eliminar artículos del carrito
- Una vista para mostrar los artículos del carrito y los totales

Agregar artículos al carrito

Para agregar artículos al carrito, necesita un formulario que le permita al usuario seleccionar una cantidad. Cree un archivo forms.py dentro del directorio de la aplicación del carrito y agregue el siguiente código:

```
desde formularios de importación de django
```

```
PRODUCT_QUANTITY_CHOICES = [(i, str (i)) para i en el rango (1, 21)]
```

```
clase CartAddProductForm (formularios.Form):
```

```
    cantidad = formularios.TypedChoiceField (
```

```
        opciones = PRODUCT_QUANTITY_CHOICES,
        coaccionar = int)
```

```
    override = forms.BooleanField (obligatorio = Falso,
```

```
        inicial = Falso,
```

```
        widget = formularios.HiddenInput)
```

Utilizará este formulario para agregar productos al carrito. Su CartAddProductForm clase contiene los siguientes dos campos:

- cantidad : Esto permite al usuario seleccionar una cantidad entre uno y 20. Usted use un campo TypedChoiceField con coerce = int para convertir la entrada en

- un entero.
- anular : esto le permite indicar si se debe agregar la cantidad a cualquier cantidad existente en el carrito para este producto (Falso), o si el La cantidad existente debe anularse con la cantidad dada (Verdadero). Tú use un widget HiddenInput para este campo, ya que no desea mostrarlo al usuario.

[245]

Creación de una tienda online

Creemos una vista para agregar artículos al carrito. Edite el archivo views.py del carrito aplicación y agregue el siguiente código:

```

desde django.shortcuts import render, redirect, get_object_or_404
desde django.views.decorators.http import require_POST
desde shop.models import Product
desde el carrito de importación .cart
desde .forms import CartAddProductForm

@require_POST
def cart_add (solicitud, product_id):
    cart = Carrito (solicitud)
    product = get_object_or_404 (Producto, id = product_id)
    formulario = CartAddProductForm (request.POST)
    si form.is_valid ():
        cd = form.cleaned_data
        cart.add (producto = producto,
                  cantidad = cd ['cantidad'],
                  override_quantity = cd ['override'])
    return redirect ('carrito: cart_detail')

```

Esta es la vista para agregar productos al carrito o actualizar cantidades para productos. Utiliza el decorador require_POST para permitir solo solicitudes POST . los view recibe el ID de producto como parámetro. Recupera la instancia del producto con el ID proporcionado y valide CartAddProductForm . Si el formulario es válido, agregar o actualizar el producto en el carrito. La vista redirige a la URL cart_detail , que mostrará el contenido del carrito. Vas a crear el cart_detail ver en breve.

También necesita una vista para eliminar artículos del carrito. Agregue el siguiente código al views.py archivo de la aplicación del carrito :

```

@require_POST
def cart_remove (solicitud, product_id):
    cart = Carrito (solicitud)
    product = get_object_or_404 (Producto, id = product_id)
    cart.remove (producto)
    return redirect ('carrito: cart_detail')

```

La vista cart_remove recibe el ID del producto como parámetro. Usas el require_POST decorator para permitir solo solicitudes POST . Recupera el Producto instancia con el ID proporcionado y retire el producto del carrito. Entonces, redirige el usuario a la URL cart_detail .

[246]

Finalmente, necesita una vista para mostrar el carrito y sus artículos. Agrega la siguiente vista al archivo `views.py` de la aplicación del carrito :

```
def cart_detail(solicitud):
    cart = Carrito(solicitud)
    return render(solicitud, 'carrito / detalle.html', {'carrito': carrito})
```

La vista `cart_detail` obtiene el carrito actual para mostrarlo.

Ha creado vistas para agregar artículos al carrito, actualizar cantidades, eliminar artículos del carro y mostrar el contenido del carro. Agreguemos patrones de URL para estas vistas. Crear un nuevo archivo dentro de la cesta directorio de la aplicación y el nombre `urls.py`. Añadir las siguientes URL a él:

```
desde la ruta de importación django.urls
de . importar vistas

app_name = 'carrito'

urlpatterns = [
    ruta ('', views.cart_detail, name = 'cart_detail'),
    ruta ('agregar / <int: product_id> /', views.cart_add, name = 'cart_add'),
    ruta ('eliminar / <int: product_id> /', views.cart_remove,
                                                name = 'cart_remove'),
]
```

Edite el archivo `urls.py` principal del proyecto `myshop` y agregue el siguiente patrón de URL para incluir las URL del carrito:

```
urlpatterns = [
    ruta ('admin /', admin.site.urls),
    ruta ('carrito /', include ('cart.urls', namespace = 'carrito')),
    ruta ('', include ('shop.urls', namespace = 'shop')),
]
```

Asegúrese de incluir este patrón de URL antes del patrón `shop.urls`, ya que es más restrictivo que el último.

Construyendo una plantilla para mostrar el carrito

Las vistas `cart_add` y `cart_remove` no representan ninguna plantilla, pero necesita para crear una plantilla para la vista `cart_detail` para mostrar los artículos del carrito y los totales.

Cree la siguiente estructura de archivos dentro del directorio de la aplicación del carrito :

```
plantillas/
  carro/
    detail.html
```


Edite la plantilla cart / detail.html y agregue el siguiente código:

```
{% extiende "shop / base.html"%}
{% load static%}

{% block title%}
    Tu carrito de la compra
{% endblock%}

{% block content%}
<h1> Su carrito de compras </h1>
<table class = "carrito">
    <thead>
        <tr>
            <th> Imagen </th>
            <th> Producto </th>
            <th> Cantidad </th>
            <th> Eliminar </th>
            <th> Precio unitario </th>
            <th> Precio </th>
        </tr>
    </thead>
    <tbody>
        {% para el artículo en el carrito%}
            {% con producto = artículo.producto%}
                <tr>
                    <td>
                        <a href="{{ product.get_absolute_url }}">
                            <img src = "{% if product.image%} {{product.image.url
}}

                            {% else%} {% static "img / no_image.png"%} {% endif%} ">

                        </a>
                    </td>
                    <td> {{product.name}} </td>
                    <td> {{item.quantity}} </td>
                    <td>
                        <form action = "{% url" carrito: cart_remove "product.id%"
método = "publicar">
                            <input type = "submit" value = "Eliminar">
                            {% csrf_token%}
                        </form>
                    </td>
                    <td class = "num"> $ {{item.price}} </td>
                    <td class = "num"> $ {{item.total_price}} </td>
                </tr>
            {% terminar con %}
        {% endfor%}
```

[248]

```
<tr class = "total">
    <td> Total </td>
    <td colspan = "4"> </td>
    <td class = "num"> $ {{cart.get_total_price}} </td>
</tr>
</tbody>
</table>
<p class = "text-right">
    Botón <a href = "{% url" shop: product_list "%" class = "
    light "> Continuar comprando </a>
    <a href="#" class="button"> Pagar </a>
</p>
{% endblock%}
```

Asegúrese de que ninguna etiqueta de plantilla se divida en varias líneas.

Esta es la plantilla que se utiliza para mostrar el contenido del carrito. Contiene una mesa con los artículos almacenados en el carrito actual. Permite que los usuarios cambien la cantidad de los productos seleccionados mediante un formulario que se publica en la vista `cart_add`. También permite a los usuarios eliminar artículos del carrito proporcionando un botón **Eliminar** para cada uno de ellos. Finalmente, usa un formulario HTML con un atributo de acción que apunta al `cart_remove` URL que incluye el ID del producto.

Agregar productos al carrito

Ahora debe agregar un botón **Agregar al carrito** a la página de detalles del producto. Editar el archivo `views.py` de la aplicación de la tienda y agregue `CartAddProductForm` al vista `product_detail`, de la siguiente manera:

```
from cart.forms import CartAddProductForm

def product_detail(solicitud, id, slug):
    product = get_object_or_404(Producto, id = id,
                                babosa = babosa,
                                disponible = Verdadero)

    cart_product_form = CartAddProductForm()
    return render(solicitud,
                  'shop / product / detail.html',
                  {'product': product,
                  'cart_product_form': cart_product_form })
```

Edite la plantilla `shop / product / detail.html` de la aplicación de la tienda y agregue el siguiente formulario al precio del producto de la siguiente manera:

```
<p class = "price"> $ {{product.price}} </p>
<form action = "{% url 'cart_add' product.id%" method = "post">
```

[249]

Creación de una tienda online

```
{{cart_product_form}}
{% csrf_token%}
<input type = "submit" value = "Agregar al carrito">
</form>
{{product.description | linebreaks}}
```

Asegúrese de que el servidor de desarrollo se esté ejecutando con el comando `python manage.py runserver`. Ahora abra `http://127.0.0.1:8000/` en su navegador y navegue a la página de detalles de un producto. Contendrá un formulario para elegir una cantidad antes agregando el producto al carrito. La página se verá así:

Figura 7.5: La página de detalles del producto, incluido el formulario para agregar al carrito

Elija una cantidad y haga clic en el botón **Agregar al carrito**. El formulario se envía a la vista `cart_add` a través de `POST`. La vista agrega el producto al carrito en la sesión, incluyendo su precio actual y la cantidad seleccionada. Luego, redirige al usuario a la página de detalles del carrito, que se verá como la siguiente captura de pantalla:

Figura 7.6: La página de detalles del carrito

[250]

Actualización de cantidades de productos en el carrito

Cuando los usuarios ven el carrito, es posible que deseen cambiar las cantidades de producto antes de colocar una orden. Permitirá a los usuarios cambiar cantidades desde la página de detalles del carrito.

Edite el archivo `views.py` de la aplicación del carrito y cambie la vista `cart_detail` a esto:

```
def cart_detail (solicitud):
    cart = Carrito (solicitud)
    para el artículo en el carrito:
        item ['update_quantity_form'] = CartAddProductForm (inicial = {
            'cantidad': artículo ['cantidad'],
            'anular': Verdadero))
    return render (solicitud, 'carrito / detalle.html', {'carrito': carrito})
```

Crea una instancia de `CartAddProductForm` para cada artículo en el carrito para permitir cambiar las cantidades de producto. Inicializa el formulario con la cantidad de artículo actual y establezca el campo de anulación en `Verdadero` para que cuando envíe el formulario al carrito_ agregar vista, la cantidad actual se reemplaza por la nueva.

Ahora edite la plantilla `cart / detail.html` de la aplicación de carrito y busque el siguiente línea:

```
<td> {{item.quantity}} </td>
```

Reemplace la línea anterior con el siguiente código:

```
<td>
    <form action = "{% url 'carrito: cart_add 'product.id%'" method = "post">
        {{item.update_quantity_form.quantity}}
        {{item.update_quantity_form.override}}
        <input type = "submit" value = "Actualizar">
        {% csrf_token%}
    </form>
</td>
```

Asegúrese de que el servidor de desarrollo se esté ejecutando con el comando `python manage.py runserver`. Abra `http://127.0.0.1:8000/cart/` en su navegador.

Creación de una tienda online

Verá un formulario para editar la cantidad de cada artículo del carrito, de la siguiente manera:

Figura 7.7: La página de detalles del carrito, incluido el formulario para actualizar las cantidades de productos

Cambie la cantidad de un artículo y haga clic en el botón **Actualizar** para probar la nueva funcionalidad. También puede eliminar un artículo del carrito haciendo clic en el **Quitar** botón.

Creando un procesador de contexto para el actual carro

Es posible que haya notado que el mensaje **Su carrito está vacío** se muestra en el encabezado del sitio, incluso cuando el carrito contiene artículos. Deberías mostrar el total número de artículos en el carrito y el costo total en su lugar. Dado que esto tiene que ser mostrado en todas las páginas, debe crear un procesador de contexto para incluir el carrito actual en el contexto de la solicitud, independientemente de la vista que procese la solicitud.

Procesadores de contexto

Un procesador de contexto es una función de Python que toma el objeto de solicitud como un argumento y devuelve un diccionario que se agrega al contexto de la solicitud. Contexto Los procesadores son útiles cuando necesita hacer algo disponible a nivel mundial a todas las plantillas.

De forma predeterminada, cuando crea un nuevo proyecto con el comando `startproject`, su proyecto contiene los siguientes procesadores de contexto de plantilla en el Opción `context_processors` dentro de la configuración `TEMPLATES`:

- `django.template.context_processors.debug` : esto establece el booleano `debug` y `sql_queries` en el contexto, que representan la lista de Consultas SQL ejecutadas en la solicitud
- `django.template.context_processors.request` : esto establece la solicitud variable en el contexto
- `django.contrib.auth.context_processors.auth` : esto establece el usuario variable en la solicitud
- `django.contrib.messages.context_processors.messages` : esto establece una variable de mensajes en el contexto que contiene todos los mensajes que tienen generado utilizando el marco de mensajes

Django también habilita `django.template.context_processors.csrf` para evitar

Ataques de **falsificación de solicitudes entre sitios** (**CSRF**). Este procesador de contexto no está presente en la configuración, pero siempre está habilitada y no se puede desactivar por razones de seguridad.

Puede ver la lista de todos los procesadores de contexto integrados en <https://docs.djangoproject.com/en/3.0/ref/templates/api/#built-in-template-processors>.

Configurar el carrito en el contexto de la solicitud

Creemos un procesador de contexto para configurar el carrito actual en el contexto de la solicitud. Con él podrás acceder al carrito en cualquier plantilla.

Cree un nuevo archivo dentro del directorio de la aplicación del carrito y asígnele el nombre `context_`

`procesadores.py` . Los procesadores de contexto pueden residir en cualquier lugar de su código, pero creando ellos aquí mantendrán su código bien organizado. Agregue el siguiente código al archivo:

```

desde el carrito de importación .cart

def cart (solicitud):
    return {'carrito': Carrito (solicitud)}
```

En su procesador de contexto, crea una instancia del carrito usando el objeto de solicitud y hace está disponible para las plantillas como una variable denominada `carrito` .

Edite el archivo `settings.py` de su proyecto y agregue `cart.context_processors.cart` a la opción `context_processors` dentro de la configuración `TEMPLATES` , de la siguiente manera:

```

PLANTILLAS = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': verdadero,
        'OPCIONES': {
```

[253]

Creación de una tienda online

```

        'context_processors': [
            # ...
            'cart.context_processors.cart',
        ],
    },
],
```

El procesador de contexto del carrito se ejecutará cada vez que se represente una plantilla usando `RequestContext` de Django . La variable del carrito se establecerá en el contexto de tus plantillas. Puede leer más sobre `RequestContext` en <https://>

docs.djangoproject.com/en/3.0/ref/templates/api/#django.template.RequestContext.

Los procesadores de contexto se ejecutan en todas las solicitudes que utilizan RequestContext. Es posible que desee crear una plantilla personalizada etiqueta en lugar de un procesador de contexto si su funcionalidad no es necesario en todas las plantillas, especialmente si se trata de consultas a la base de datos.

A continuación, edite la plantilla shop / base.html de la aplicación de la tienda y busque la siguientes líneas:

```
<div class = "carrito">
    Tu carrito esta vacío.
</div>
```

Reemplace las líneas anteriores con el siguiente código:

```
<div class = "carrito">
    {% con total_items = carrito | longitud%}
    {% si total_items> 0%}
        Tu carrito:
        <a href="{% url "cart:cart_detail" %}">
            {{total_items}} elemento {{total_items | pluralize}},
            $ {{cart.get_total_price}}
        </a>
    {% else%}
        Tu carrito esta vacío.
    {% terminara si %}
    {% terminar con %}
</div>
```

Recargue su servidor usando el comando `python manage.py runserver` . Abierto `http://127.0.0.1:8000/` en su navegador y agregue algunos productos al carrito.

[254]

En el encabezado del sitio web, ahora puede ver el número total de artículos en el carrito y el costo total, como sigue:

Figura 7.8: El encabezado del sitio que muestra los artículos actuales en el carrito

Registro de pedidos de clientes

Cuando se retira un carrito de compras, debe guardar un pedido en la base de datos. Los pedidos contendrán información sobre los clientes y los productos que están comprando.

Cree una nueva aplicación para administrar los pedidos de los clientes utilizando lo siguiente mando:

pedidos de python manage.py startapp

Edite el archivo `settings.py` de su proyecto y agregue la nueva aplicación al Configuración `INSTALLED_APPS` , de la siguiente manera:

```
INSTALLED_APPS = [
    # ...
    'orders.apps.OrdersConfig',
```

```
]

```

Has activado la aplicación de pedidos .

Creando modelos de pedidos

Necesitará un modelo para almacenar los detalles del pedido y un segundo modelo para almacenar artículos comprados, incluido su precio y cantidad. Edite el archivo `models.py` de los pedidos aplicación y agregue el siguiente código:

```
desde modelos de importación django.db
desde shop.models import Product

class Orden (modelos.Modelo):
    first_name = modelos.CharField (max_length = 50)
    last_name = modelos.CharField (max_length = 50)
    correo electrónico = modelos.EmailField ()
    address = modelos.CharField (max_length = 250)
    postal_code = modelos.CharField (max_length = 20)
```

[255]

Creación de una tienda online

```
city = modelos.CharField (max_length = 100)
creado = modelos.DateTimeField (auto_now_add = True)
actualizado = modelos.DateTimeField (auto_now = True)
pagado = modelos.BooleanField (predeterminado = Falso)

class Meta:
    ordering = ('-creado',)

def __str__ (yo):
    return f'Order {self.id}'

def get_total_cost (self):
    devolver suma (item.get_cost () para el artículo en self.items.all ())
```

```
class OrderItem (modelos.Modelo):
    order = modelos.ForeignKey (Pedido,
                                related_name = 'elementos',
                                on_delete = modelos.CASCADE)
    producto = modelos.ForeignKey (Producto,
                                   related_name = 'order_items',
                                   on_delete = modelos.CASCADE)
    precio = modelos.DecimalField (max_digits = 10, decimal_places = 2)
    cantidad = modelos.PositiveIntegerField (predeterminado = 1)

    def __str__ (yo):
        return str (self.id)

    def get_cost (auto):
        return self.price * self.quantity
```

La Orden de modelo contiene varios campos para almacenar la información del cliente y una paga Campo booleano, cuyo valor predeterminado es Falso . Más adelante, utilizará este campo para diferenciar entre pedidos pagados y no pagados. También define un `get_total_cost ()` método para obtener el costo total de los artículos comprados en este pedido.

El modelo `OrderItem` le permite almacenar el producto, la cantidad y el precio pagado Cada artículo. Incluya `get_cost ()` para devolver el costo del artículo.

Ejecute el siguiente comando para crear migraciones iniciales para la aplicación de pedidos :

```
python manage.py makemigrations
```

Verá el siguiente resultado:

Migraciones por 'pedidos':

```
pedidos / migraciones / 0001_initial.py
```

```
- Crear orden modelo
```

```
- Crear modelo OrderItem
```

Ejecute el siguiente comando para aplicar la nueva migración:

```
python manage.py migrar
```

Sus modelos de pedidos ahora están sincronizados con la base de datos.

Incluyendo modelos de pedidos en la administración sitio

Agreguemos los modelos de pedidos al sitio de administración. Edite el archivo admin.py del

Solicitud de pedidos para que se vea así:

```
desde django.contrib import admin
de .models import Order, OrderItem

class OrderItemInline (admin.TabularInline):
    model = OrderItem
    raw_id_fields = ['producto']

@ admin.register (Orden)
class OrderAdmin (admin.ModelAdmin):
    list_display = ['id', 'first_name', 'last_name', 'email',
                  'dirección', 'código_postal', 'ciudad', 'pagado',
                  'creado', 'actualizado']
    list_filter = ['pagado', 'creado', 'actualizado']
    inlines = [OrderItemInline]
```

Utiliza una clase `ModelInline` para que el modelo `OrderItem` lo incluya como un elemento en *línea* en la clase `OrderAdmin`. Un inline le permite incluir un modelo en la misma edición página como modelo relacionado.

Creación de una tienda online

Ejecute el servidor de desarrollo con el comando `python manage.py runserver` , y luego abra `http://127.0.0.1:8000/admin/orders/order/add/` en su navegador. Verá la siguiente página:

Figura 7.9: El formulario Agregar pedido, incluido `OrderItemInline`

Creación de pedidos de clientes

Utilizará los modelos de pedido que creó para conservar los elementos contenidos en el carrito de compras cuando el usuario finalmente realiza un pedido. Se creará un nuevo pedido siguiendo estos pasos:

- Presentar a un usuario un formulario de pedido para completar sus datos
- Cree una nueva instancia de Pedido con los datos ingresados y cree un Instancia de `OrderItem` para cada artículo del carrito
- Borrar todo el contenido del carrito y redirigir al usuario a una página de éxito

[258]

Capítulo 7

Primero, necesita un formulario para ingresar los detalles del pedido. Crea un nuevo archivo dentro de los pedidos directorio de la aplicación y asígnele el nombre `forms.py` . Agregue el siguiente código:

```

desde formularios de importación de django
de .models import Order

clase OrderCreateForm (formularios.ModelForm):
    clase Meta:
```

```

        modelo = Orden
        fields = ['first_name', 'last_name', 'email', 'address',
                  'Código postal de la ciudad']

```

Este es el formulario que utilizará para crear nuevos objetos Order . Ahora tu Necesito una vista para manejar el formulario y crear un nuevo pedido. Edite el archivo views.py del solicitud de pedidos y agregue el siguiente código:

```

    desde django.shortcuts import render
    desde .models import OrderItem
    desde .forms import OrderCreateForm
    desde cart.cart import Cart

    def order_create (solicitud):
        cart = Carrito (solicitud)
        if request.method == 'POST':
            formulario = OrderCreateForm (request.POST)
            si form.is_valid ():
                order = form.save ()
                para el artículo en el carrito:
                    OrderItem.objects.create (order = order,
                                                producto = artículo ['producto'],
                                                precio = artículo ['precio'],
                                                cantidad = artículo ['cantidad'])

            # limpiar el carrito
            cart.clear ()
            return render (solicitud,
                           'orders / order / created.html',
                           {'orden orden'})

        más:
            formulario = OrderCreateForm ()
        return render (solicitud,
                       'orders / order / create.html',
                       {'carrito': carrito, 'formulario': formulario})

```

[259]

Creación de una tienda online

En la vista order_create , obtiene el carrito actual de la sesión con

cart = Carrito (solicitud) . Dependiendo del método de solicitud, realiza las siguientes tareas:

- **Solicitud GET** : crea una instancia del formulario OrderCreateForm y renderiza el orders / order / create.html plantilla.
- **Solicitud POST** : Valida los datos enviados en la solicitud. Si los datos son válidos, crea un nuevo pedido en la base de datos usando order = form.save () . Tú iterar sobre los elementos del carrito y crear un artículo de pedido para cada uno de ellos. Finalmente, borras el contenido del carrito y renderizas la plantilla pedidos / pedido / created.html .

Crear un nuevo archivo dentro de la órdenes de directorio de la aplicación y el nombre urls.py . Agregue el siguiente código:

```

    desde la ruta de importación django.urls
    de . importar vistas

    app_name = 'pedidos'

    urlpatterns = [
        ruta ('crear /', vistas.order_create, nombre = 'order_create'),
    ]

```

Este es el patrón de URL para la vista `order_create`. Editar la `urls.py` archivo de `MyShop` e incluye el siguiente patrón. Recuerda colocarlo antes del patrón `shop.urls`:

```
ruta ('pedidos /', incluir ('pedidos.urls', espacio de nombres = 'pedidos')),
```

Edite la plantilla `cart / detail.html` de la aplicación del carrito y localice esta línea:

```
<a href="#" class="button"> Pagar </a>
```

Agregue la URL `order_create` de la siguiente manera:

```
<a href= "{% url "orders:order_create" %}" class="button">
  Revisa
</a>
```

Los usuarios ahora pueden navegar desde la página de detalles del carrito hasta el formulario de pedido.

Aún necesita definir plantillas para realizar pedidos. Crea el siguiente archivo estructura dentro del directorio de la aplicación de pedidos:

```
plantillas/
  pedidos/
    orden/
      create.html
      created.html
```

[260]

Edite la plantilla `orders / order / create.html` e incluya el siguiente código:

```
{% extiende "shop / base.html"%}

{% block title%}
  Revisa
{% endblock%}

{% block content%}
  <h1> Pagar </h1>

  <div class = "información de pedido">
    <h3> Su pedido </h3>
    <ul>
      {% para el artículo en el carrito%}
        <li>
          {{item.quantity}} x {{item.product.name}}
          <span> $ {{item.total_price}} </span>
        </li>
      {% endfor%}
    </ul>
    <p> Total: $ {{cart.get_total_price}} </p>
  </div>

  <form method = "post" class = "order-form">
    {{form.as_p}}
    <p> <input type = "submit" value = "Realizar pedido"> </p>
    {% csrf_token%}
  </form>
{% endblock%}
```

Esta plantilla muestra los artículos del carrito, incluidos los totales y el formulario para realizar un pedido.

Edite la plantilla `orders / order / created.html` y agregue el siguiente código:

```
{% extiende "shop / base.html"%}

{% block title%}
  Gracias
```

```
{% endblock%}
{% block content%}
    <h1> Gracias </h1>
    <p> Su pedido se completó con éxito. tu numero de orden es
    <strong> {{order.id}} </strong>. </p>
{% endblock%}
```

[261]

Creación de una tienda online

Esta es la plantilla que renderiza cuando el pedido se crea correctamente.

Inicie el servidor de desarrollo web para cargar nuevos archivos. Abra <http://127.0.0.1:8000/> en su navegador, agregue un par de productos al carrito y continúe con el pago página. Verá una página como la siguiente:

Figura 7.10: La página de creación de pedidos, incluido el formulario de pago del gráfico y los detalles del pedido

Complete el formulario con datos válidos y haga clic en el botón **Realizar pedido** . La orden ser creado y verá una página de éxito como esta:

Figura 7.11: Plantilla de pedido creado que muestra el número de pedido

[262]

Ahora abra el sitio de administración en `http://127.0.0.1:8000/admin/orders/` orden / . Verá que el pedido se ha creado correctamente.

Lanzar tareas asincrónicas con Apio

Todo lo que ejecuta en una vista afecta los tiempos de respuesta. En muchas situaciones es posible que desee devolver una respuesta al usuario lo más rápido posible y dejar que el servidor ejecute algún proceso de forma asincrónica. Esto es especialmente relevante para el tiempo Consumir procesos o procesos sujetos a fallas, que pueden necesitar un reintento. política. Por ejemplo, una plataforma para compartir videos permite a los usuarios subir videos pero requiere mucho tiempo para transcodificar los videos cargados. El sitio puede devolver una respuesta a los usuarios para informarles que la transcodificación comenzará pronto y comenzará a transcodificar el video de forma asincrónica. Otro ejemplo es el envío de correos electrónicos a los usuarios. Si su sitio envía notificaciones por correo electrónico desde una vista, el **Protocolo simple de transferencia de correo (SMTP)** la conexión puede fallar o ralentizar la respuesta. Lanzar tareas asincrónicas es fundamental para evitar bloquear la ejecución del código.

El apio es una cola de tareas distribuida que puede procesar grandes cantidades de mensajes. Utilizando Apio, no solo puedes crear tareas asincrónicas fácilmente y dejar que se ejecuten por los trabajadores lo antes posible, pero también puede programarlos para que se ejecuten en un hora.

Puede encontrar la documentación de apio en <http://docs.celeryproject.org/en/latest/index.html>.

Instalación de apio

Instalemos Celery e integremoslo en su proyecto. Instale Apio vía pip usando el siguiente comando:

```
pip instalar apio == 4.4.2
```

El apio requiere un corredor de mensajes para manejar las solicitudes de un fuente. Un intermediario de mensajes se utiliza para traducir mensajes a mensajes formales. protocolo y administrar colas de mensajes para múltiples receptores, proporcionando almacenamiento y entrega de mensajes garantizada. Utiliza un corredor de mensajes para enviar mensajes a los trabajadores de Apio, que procesan las tareas a medida que las reciben.

Instalación de RabbitMQ

Hay varias opciones para un agente de mensajes para Apio, incluidas las tiendas de clave / valor como Redis, o un sistema de mensajes real como RabbitMQ. Configuremos Apio

con RabbitMQ, ya que es el trabajador de mensajes recomendado para Celery. RabbitMQ es liviano, admite múltiples protocolos de mensajería y se puede usar cuando se requieren escalabilidad y alta disponibilidad.

Si está usando Linux, puede instalar RabbitMQ desde el shell usando lo siguiente mando:

apt-get install rabbitmq

Si necesita instalar RabbitMQ en macOS o Windows, puede encontrarlo de forma independiente versiones en <https://www.rabbitmq.com/download.html>. En este sitio, también puede encontrar guías de instalación detalladas para diferentes distribuciones de Linux, otras sistemas y contenedores.

Después de instalarlo, inicie RabbitMQ usando el siguiente comando desde el shell:

rabbitmq-servidor

Verá un resultado que termina con la siguiente línea:

Agente inicial ... completado con 10 complementos.

RabbitMQ se está ejecutando y está listo para recibir mensajes.

Agregando apio a su proyecto

Debe proporcionar una configuración para la instancia de Celery. Crea un nuevo archivo a continuación al archivo settings.py de myshop y asígnele el nombre celery.py. Este archivo contendrá el Configuración de apio para su proyecto. Agregue el siguiente código:

```
importar sistema operativo
de apio importado Apio

# establece el módulo de configuración predeterminado de Django para el programa 'apio'.
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'myshop.settings')

aplicación = Apio('myshop')

app.config_from_object('django.conf: settings', namespace = 'CELERY')
app.autodiscover_tasks()
```

En este código, haces lo siguiente:

- Establece la variable DJANGO_SETTINGS_MODULE para la línea de comandos de Apio programa.

[264]

- Creas una instancia de la aplicación con app = Celery('myshop').
- Carga cualquier configuración personalizada de la configuración de su proyecto utilizando el método config_from_object(). El atributo de espacio de nombres especifica el prefijo que tendrá la configuración relacionada con el apio en su archivo settings.py. Al configurar el espacio de nombres APIO, todas las configuraciones de Apio deben incluir el CELERY_ prefijo en su nombre (por ejemplo, CELERY_BROKER_URL).
- Finalmente, le dice a Celery que descubra automáticamente tareas asincrónicas para su aplicaciones. Apio buscará un archivo tasks.py en cada aplicación directorio de aplicaciones agregadas a INSTALLED_APPS para cargar Tareas asincrónicas definidas en él.

Necesita importar el módulo de apio en el archivo __init__.py de su proyecto para asegúrese de que esté cargado cuando se inicie Django. Edite el archivo myshop / __init__.py y agregue el siguiente código:

```
# apio de importación
desde la aplicación de importación .celery como celery_app
```

Ahora puede comenzar a programar tareas asincrónicas para sus aplicaciones.

La configuración `CELERY_ALWAYS_EAGER` le permite ejecutar tareas localmente de forma síncrona, en lugar de enviarlos a la cola. Esto es útil para ejecutar pruebas unitarias o ejecutar la aplicación en su entorno local sin ejecutar Apio.

Agregar tareas asincrónicas a su solicitud

A continuación, creará una tarea asincrónica para enviar una notificación por correo electrónico a sus usuarios cuando realizan un pedido. La convención es incluir asincrónica tareas para su aplicación en un módulo de tareas dentro del directorio de su aplicación.

Cree un nuevo archivo dentro de la aplicación de pedidos y asígnele el nombre `tasks.py`. Este es el lugar donde Celery buscará tareas asincrónicas. Agregue el siguiente código:

```
de la tarea de importación de apio
desde django.core.mail importar send_mail
de .models import Order
```

```
@tarea
def order_created (order_id):
    """
```

[265]

Creación de una tienda online

```
Tarea para enviar una notificación por correo electrónico cuando se realiza un pedido.
creado con éxito.
"""
```

```
order = Order.objects.get (id = order_id)
asunto = f'Orden nr. {Solicitar ID}'
message = f'Estimado {order.first_name}, \n \n \'
            f'Ha realizado correctamente un pedido.' \
            f'El ID de su pedido es {order.id}\'.
```

```
mail_sent = send_mail (asunto,
                        mensaje,
                        'admin@myshop.com',
                        [order.email])
```

```
devolver mail_sent
```

La tarea `order_created` se define mediante el decorador de tareas . Como puedes ver, una tarea de Apio es solo una función de Python decorada con `@task` . Tu función de tarea recibe un parámetro `order_id` . Siempre se recomienda pasar solo las ID a la tarea funciones y objetos de búsqueda cuando se ejecuta la tarea. Se utiliza el `send_mail ()` función proporcionada por Django para enviar una notificación por correo electrónico al usuario que colocó el orden.

Aprendió cómo configurar Django para usar su servidor SMTP en el *Capítulo 2* , *Mejorando su blog con funciones avanzadas* . Si no desea configurar el correo electrónico, puede decirle a Django que escriba correos electrónicos en la consola agregando la siguiente configuración a el archivo `settings.py` :

```
EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'
```

Utilice tareas asincrónicas no solo para procesos que requieren mucho tiempo, sino también para otros procesos que no tardan tanto en ser ejecutados pero que están sujetos a fallas de conexión o requieren una política de reintento.

Ahora tienes que agregar la tarea a tu vista `order_create` . Edite el archivo `views.py` de la aplicación de pedidos , importe la tarea y llame a `order_created` asynchronous

tarea después de limpiar el carro, de la siguiente manera:

```
desde .tasks import order_created

def order_create (solicitud):
    # ...
```

[266]

```
if request.method == 'POST':
    # ...
    si form.is_valid ():
        # ...
        cart.clear ()
        # lanzar tarea asincrónica
        order_created.delay (order.id)
    # ...
```

Llamas al método `delay ()` de la tarea para ejecutarla de forma asincrónica. La tarea se agregará a la cola y será ejecutado por un trabajador lo antes posible.

Abra otro shell e inicie el trabajador de apio desde el directorio de su proyecto, usando el siguiente comando:

apio -Un trabajador de myshop -l info

El trabajador de Apio ahora está en ejecución y listo para procesar tareas. Asegúrese de que el El servidor de desarrollo Django también se está ejecutando.

Abra `http://127.0.0.1:8000/` en su navegador, agregue algunos productos a su carrito de compras y completar un pedido. En el caparazón, comenzaste el trabajador de apio y verá una salida similar a esta:

```
[2020-01-04 17: 43: 11,462: INFO / MainProcess] Tarea recibida: orders.tasks.
order_created [e990ddae-2e30-4e36-b0e4-78bbd4f2738e]
```

...

```
[2020-01-04 17: 43: 11,685: INFO / ForkPoolWorker-4] Pedidos de tareas.tareas.
order_created [e990ddae-2e30-4e36-b0e4-78bbd4f2738e] logró
0.02019841300789267s: 1
```

La tarea se ha ejecutado y se ha enviado una notificación por correo electrónico para su pedido o que se muestra en la salida del trabajador de Apio si está utilizando el servidor de correo electrónico de la consola.

Monitoreo de apio

Es posible que desee supervisar las tareas asincrónicas que se ejecutan. La flor es una herramienta basada en web para el seguimiento de Apio. Puedes instalar Flower usando este comando:

pip instalar flor == 0.9.3

Una vez instalado, puede iniciar Flower ejecutando el siguiente comando desde el directorio de su proyecto:

apio -Una flor de myshop

[267]

Creación de una tienda online

Abra `http://localhost:5555/dashboard` en su navegador. Podrás ver los trabajadores activos de Apio y las estadísticas de tareas asincrónicas:

Figura 7.12: El tablero de Flower

Puede encontrar la documentación de Flower en <https://flower.readthedocs.io/>.

Resumen

En este capítulo, creó una aplicación de tienda básica. Hiciste un catálogo de productos y construyó un carrito de compras usando sesiones. Implementaste un contexto personalizado procesador para que el carrito esté disponible para sus plantillas y creó un formulario para Colocando órdenes. También aprendió a iniciar tareas asincrónicas con Celery.

En el siguiente capítulo, descubrirá cómo integrar una pasarela de pago en su tienda, agregue acciones personalizadas al sitio de administración, exporte datos en CSV formato y generar archivos PDF de forma dinámica.

8

Gestionar pagos y pedidos

En el capítulo anterior, creó una tienda en línea básica con un catálogo de productos y un carrito de compras. También aprendió a iniciar tareas asincrónicas con Celery.

En este capítulo, aprenderá cómo integrar una pasarela de pago en su sitio. para permitir a los usuarios pagar con tarjeta de crédito. También ampliará el sitio de administración con diferentes características.

En este capítulo, podrá:

- Integre una pasarela de pago en su proyecto
- Exportar pedidos a archivos CSV
- Cree vistas personalizadas para el sitio de administración
- Genere facturas en PDF de forma dinámica

Integrando una pasarela de pago

Una pasarela de pago le permite procesar pagos en línea. Usando un pago pasarela, puede administrar los pedidos de los clientes y delegar el procesamiento de pagos a un tercero confiable y seguro. No tendrá que preocuparse por procesar el crédito tarjetas en su propio sistema.

Hay varios proveedores de pasarelas de pago para elegir. Tu vas para integrar Braintree, que es utilizado por servicios en línea populares como Uber y Airbnb.

[269]

Gestión de pagos y pedidos

Braintree proporciona una API que le permite procesar pagos en línea con múltiples métodos de pago, como tarjeta de crédito, PayPal, Google Pay y Apple Pay. Usted puede aprender más sobre Braintree en <https://www.braintreepayments.com/>.

Braintree ofrece diferentes opciones de integración. El más simple es el *Drop-in* integración, que contiene un formulario de pago preformateado. Sin embargo, para para personalizar el comportamiento y la experiencia de su pago, utilizará la integración avanzada de *Hosted Fields*. Puedes aprender más sobre esta integración en <https://developers.braintreepayments.com/guides/hosted-fields/descripción-general/javascript/v3>.

Ciertos campos de pago en la página de pago, como el número de tarjeta de crédito, CVV número, o fecha de vencimiento, debe estar alojado de forma segura. La integración de Hosted Fields aloja los campos de pago en el dominio de la pasarela de pago y genera un iframe para presentar los campos a los usuarios. Esto le brinda la posibilidad de personalizar el aspecto y sentir la forma de pago, mientras se asegura de que cumple con el **pago**

Requisitos de la **industria de tarjetas (PCI)**. Dado que puede personalizar la apariencia de la campos de formulario, los usuarios no notarán el iframe.

Crear una cuenta de caja de arena de Braintree

Necesita una cuenta de Braintree para integrar la pasarela de pago en su sitio.

Creemos una cuenta de sandbox para probar la API de Braintree. Abierto <https://www.braintreepayments.com/sandbox>

en su navegador. Verás un formulario como el siguiente:

Figura 8.1: El formulario de registro de Braintree sandbox

[270]

Complete los detalles para crear una nueva cuenta de sandbox. Recibirás un correo electrónico de Braintree con un enlace. Siga el enlace y complete la configuración de su cuenta.

Una vez que haya terminado, inicie sesión en <https://sandbox.braintreegateway.com/login>.

Su ID de comerciante y sus claves públicas / privadas se mostrarán así:

Figura 8.2: El ID de comerciante de la zona de pruebas de Braintree y las claves públicas / privadas

Necesitará esta información para autenticar solicitudes a la API de Braintree. Siempre mantenga su clave privada en secreto.

Instalación del módulo Braintree Python

Braintree proporciona un módulo de Python que simplifica el manejo de su API. Usted está va a integrar la pasarela de pago en su proyecto utilizando el braintree módulo.

Instale el módulo braintree desde el shell usando el siguiente comando:

```
pip instalar braintree == 3.59.0
```

Agregue la siguiente configuración al archivo settings.py de su proyecto:

```
# Configuración de Braintree
BRAINTREE_MERCHANT_ID = 'XXX' # ID de comerciante
BRAINTREE_PUBLIC_KEY = 'XXX' # Clave pública
BRAINTREE_PRIVATE_KEY = 'XXX' # Clave privada

importar braintree

BRAINTREE_CONF = braintree.Configuration (
    braintree.Environment.Sandbox,
    BRAINTREE_MERCHANT_ID,
    BRAINTREE_PUBLIC_KEY,
    BRAINTREE_PRIVATE_KEY
)
```

[271]

Gestión de pagos y pedidos

Reemplace BRAINTREE_MERCHANT_ID , BRAINTREE_PUBLIC_KEY y BRAINTREE_PRIVATE_KEY con los de su cuenta.

Utiliza Environment.Sandbox para integrar el salvadera. Una vez que entre en vivo y cree una cuenta real, Necesito cambiar esto a Environment.Production. Braintree le proporcionará una nueva identificación de comerciante y claves privadas / públicas para el entorno de producción. En el *Capítulo 14 , Puesta en marcha* , aprenderá a configurar los ajustes para varios entornos.

Integremos la pasarela de pago en el proceso de pago.

Integrando la pasarela de pago

El proceso de pago funcionará de la siguiente manera:

1. Agregue artículos al carrito de compras
2. Consulte el carrito de compras
3. Ingrese los detalles de la tarjeta de crédito y pague

Vas a crear una nueva aplicación para gestionar pagos. Crear un nuevo aplicación en su proyecto usando el siguiente comando:

pago python manage.py startapp

Edite el archivo settings.py de su proyecto y agregue la nueva aplicación al Configuración INSTALLED_APPS , de la siguiente manera:

```
INSTALLED_APPS = [
    # ...
    'payment.apps.PaymentConfig' ,
]
```

La aplicación de pago ahora está activa.

Después de que los clientes realicen un pedido, debe redirigirlos al proceso de pago. Editar el archivo views.py de la aplicación de pedidos e incluyen las siguientes importaciones:

```
desde django.urls import reverse
de django.shortcuts import render , redireccionar
```

En el mismo archivo, busque las siguientes líneas de la vista order_create :

```
# lanzar tarea asíncrona
order_created.delay (order.id)
```

[272]

```
return render (solicitud,
               'orders / order / created.html',
               locales ())
```

Reemplácelos con lo siguiente:

```
# lanzar tarea asincrónica
order_created.delay (order.id)
# establecer el orden en la sesión
request.session ['order_id'] = order.id
# redireccionamiento para pago
return redirect (reverse ('pago: proceso'))
```

Con este código, después de crear correctamente un pedido, establece el ID del pedido en el sesión actual usando la clave de sesión `order_id`. Luego, dirige al usuario a la pago: URL de proceso, que vas a implementar más adelante. Recuerda que tu necesita ejecutar Celery para que la tarea `order_created` se ponga en cola y se ejecute.

Cada vez que se crea un pedido en Braintree, se genera un identificador de transacción único. generado. Va a añadir un nuevo campo a la Orden de modelo de órdenes de aplicaciones para almacenar el ID de la transacción. Esto le permitirá vincular cada pedido con sus Transacción Braintree.

Edita el archivo `models.py` de la aplicación de pedidos y agregue el siguiente campo al Modelo de pedido :

```
class Orden (modelos.Modelo):
    # ...
    braintree_id = models.CharField (max_length = 150, blank = True)
```

Sincronicemos este campo con la base de datos. Utilice el siguiente comando para generar migraciones:

```
python manage.py makemigrations
```

Verá el siguiente resultado:

Migraciones por 'pedidos':

```
orders / migrations / 0002_order_braintree_id.py
- Agregar campo braintree_id al pedido
```

Aplica la migración a la base de datos con el siguiente comando:

```
python manage.py migrar
```

Verá un resultado que termina con la siguiente línea:

```
Aplicando pedidos.0002_order_braintree_id ... OK
```

Los cambios del modelo ahora están sincronizados con la base de datos. Ahora puede almacenar el ID de transacción de Braintree para cada pedido. Integremos la pasarela de pago.

Integración de Braintree mediante campos alojados

La integración de Hosted Fields le permite crear su propio formulario de pago utilizando estilos y diseños personalizados. Un iframe se agrega dinámicamente a la página usando el **Kit de desarrollo de software** Braintree JavaScript (**SDK**). El iframe incluye el Formulario de pago de Hosted Fields. Cuando el cliente envía el formulario, los campos alojados recopila los detalles de la tarjeta de forma segura e intenta tokenizarlos. Si tokenización tiene éxito, puede enviar el token nonce generado a su vista para hacer un transacción utilizando el módulo Braintree de Python . Un nonce simbólico es un seguro, uno-referencia de uso del tiempo a la información de pago. Te permite enviar pagos sensibles información a Braintree sin tocar los datos brutos.

Creemos una vista para procesar pagos. Todo el proceso de pago funcionará como sigue:

1. En la vista, se genera un token de cliente utilizando el módulo Braintree Python. Este token se utiliza en el siguiente paso para crear una instancia de Braintree JavaScript cliente; no es la moneda de pago.
2. La vista muestra la plantilla de pago. La plantilla carga el Braintree SDK de JavaScript utilizando el token del cliente y genera el iframe con el campos del formulario de pago alojado.
3. Los usuarios ingresan los detalles de su tarjeta de crédito y envían el formulario. Un token de pago nonce se genera con el cliente JavaScript de Braintree. Tu envías el token a su vista con una solicitud POST .
4. La vista de pago recibe el token nonce y usted lo usa para generar una transacción que usa el módulo Braintree Python.

Comencemos con la vista de pago y envío. Edite el archivo views.py del pago aplicación y agregue el siguiente código:

```
import braintree
from django.shortcuts import render, redirect, get_object_or_404
from django.conf import settings
from orders.models import Order

# instanciar la pasarela de pago de Braintree
gateway = braintree.BraintreeGateway(settings.BRAINTREE_CONF)

def proceso_pago(solicitud):
    order_id = request.session.get('order_id')
```

[274]

```
order = get_object_or_404(Order, id = order_id)
costo_total = order.get_total_cost()

if request.method == 'POST':
    # recuperar nonce
    nonce = request.POST.get('método_pago_noce', Ninguno)
    # crear y enviar transacción
    resultado = gateway.transaction.sale({
        'monto': f'{costo_total:.2f}',
        'payment_method_nonce': nonce,
        'opciones': {
            'submit_for_settlement': verdadero
        }
    })
```

```

    ))
    si result.is_success:
        # marcar el pedido como pagado
        order.paid = True
        # almacenar el ID de transacción único
        order.braintree_id = result.transaction.id
        order.save ()
        return redirect ('pago: hecho')
    más:
        return redirect ('pago: cancelado')
    más:
        # generar token
        client_token = gateway.client_token.generate ()
        return render (solicitud,
                        'pago / proceso.html',
                        {'orden orden',
                        'client_token': client_token})

```

En el código anterior, importa el módulo braintree y crea una instancia de la puerta de enlace Braintree usando BraintreeGateway () , con la configuración definida en la configuración BRAINTREE_CONF del proyecto.

La vista payment_process gestiona el proceso de pago. En esta vista, toma las siguientes acciones:

1. Obtienes el pedido actual de la clave de sesión order_id , que se almacenó previamente en la sesión por la vista order_create .
2. Recupera el objeto Order para el ID proporcionado o genera una excepción Http404 si no se encuentra.

[275]

Gestión de pagos y pedidos

3. Cuando la vista se carga con una solicitud POST , recupera el pago_method_nonce para generar una nueva transacción usando gateway.transaction.venta () . Le pasa los siguientes parámetros:
 - monto : El monto total a cobrar al cliente. Esto es una cuerda con la cantidad total formateada con dos decimales.
 - payment_method_nonce : el token nonce generado por Braintree para el pago. Se generará en la plantilla utilizando Braintree SDK de JavaScript.
 - opciones : envía la opción submit_for_settlement con True para que la transacción se envíe automáticamente para su liquidación.
4. Si la transacción se procesa correctamente, marca el pedido como pagado por establecer su atributo pagado en Verdadero y almacenar el ID de transacción único devuelto por la puerta de enlace en el atributo braintree_id . Redirige al usuario a la pago: URL realizada si el pago es exitoso; de lo contrario, redirige ellos al pago: cancelado .
5. Si la vista se cargó con una solicitud GET , genere un token de cliente con gateway.client_token.generate () que usará en la plantilla para crear una instancia del cliente Braintree JavaScript.

Creemos vistas básicas para redirigir a los usuarios cuando su pago se haya realizado correctamente. o cuando se haya cancelado por cualquier motivo. Agregue el siguiente código a views.py archivo de la solicitud de pago :

```

def payment_done (solicitud):
    return render (solicitud, 'pago / hecho.html')

```

```
def payment_canceled(solicitud):
    return render(solicitud, 'pago / cancelado.html')
```

Crear un nuevo archivo en el interior del pago directorio de la aplicación y el nombre urls.py .
Agregue el siguiente código:

```
desde la ruta de importación django.urls
de . importar vistas

app_name = 'pago'

urlpatterns = [
    ruta ('proceso /', vistas.proceso_pago, nombre = 'proceso'),
    ruta ('hecho /', vistas.pago_done, nombre = 'hecho'),
    ruta ('cancelado /', vistas.pago_cancelado, nombre = 'cancelado'),
]
```

[276]

Estas son las URL para el flujo de trabajo de pago. Has incluido lo siguiente
Patrones de URL:

- proceso : la vista que procesa el pago
- hecho : la vista para redirigir al usuario si el pago es exitoso
- cancelado : la vista para redirigir al usuario si el pago no es exitoso

Edite el archivo urls.py principal del proyecto myshop e incluya los patrones de URL para la aplicación de pago , de la siguiente manera:

```
urlpatterns = [
    # ...
    ruta ('pago /', incluir ('pago.urls', espacio de nombres = 'pago')),
    ruta ('', include ('shop.urls', namespace = 'shop')),
]
```

Recuerde colocar la nueva ruta antes del patrón shop.urls para evitar un
coincidencia de patrón no intencionado con un patrón definido en shop.urls . Recuerda eso
Django recorre cada patrón de URL en orden y se detiene en el primero que
coincide con la URL solicitada.

Cree la siguiente estructura de archivos dentro del directorio de la aplicación de pago :

```
plantillas/
  pago/
    process.html
    done.html
    cancelado.html
```

Edite la plantilla payment / process.html y agregue el siguiente código:

```
{% extiende "shop / base.html"%}

{% block title%} Paga con tarjeta de crédito {% endblock%}

{% block content%}
<h1> Pague con tarjeta de crédito </h1>
<formulario id = "pago" método = "publicación">

<label for = "card-number"> Número de tarjeta </label>
<div id = "card-number" class = "field"> </div>

<label for = "cvv"> CVV </label>
<div id = "cvv" class = "campo"> </div>
```


<label for = "expiration-date"> Fecha de vencimiento </label>

[277]

Gestión de pagos y pedidos

```
<div id = "fecha de expiración" class = "campo"> </div>

<input type = "hidden" id = "nonce" name = "payment_method_nonce"
valor = "">
  {% csrf_token%}
  <input type = "submit" value = "Pagar">
</form>
<! - incluye el SDK del cliente Braintree JS ->
<script src = "https://js.braintreegateway.com/web/3.44.2/js/client.
min.js "> </script>
<script src = "https://js.braintreegateway.com/web/3.44.2/js/hosted-
fields.min.js "> </script>
<script>
  var form = document.querySelector ('# pago');
  var submit = document.querySelector ('input [type = "submit"]');

  braintree.client.create ({
    autorización: '{{client_token}}'
  }, función (clientErr, clientInstance) {
    if (clientErr) {
      console.error (clientErr);
      regreso;
    }

    braintree.hostedFields.create ({
      cliente: clientInstance,
      estilos: {
        'input': {'font-size': '13px'},
        'input.invalid': {'color': 'red'},
        'input.valid': {'color': 'green'}
      },
      campos: {
        número: {selector: '# card-number'},
        cvv: {selector: '#cvv'},
        expirationDate: {selector: '# expiration-date'}
      }
    }, función (hospedadoFieldsErr, hospedadoFieldsInstance) {
      if (hospedadoFieldsErr) {
        console.error (hospedadoFieldsErr);
        regreso;
      }

      submit.removeAttribute ('desactivado');
```

[278]

```

form.addEventListener('enviar', función (evento) {
    event.preventDefault ();

    hostsFieldsInstance.tokenize (función (tokenizeErr,
payload) {
        if (tokenizeErr) {
            console.error (tokenizeErr);
            regreso;
        }
        // establece nonce para enviar al servidor
        document.getElementById ('nonce'). value = payload.nonce;
        // enviar formulario
        document.getElementById ('pago'). submit ();
    });
}, falso);
});
</script>
{% endblock%}

```

Esta es la plantilla que muestra el formulario de pago y procesa el pago.

Define contenedores <div> en lugar de elementos <input> para la entrada de la tarjeta de crédito campos: el número de la tarjeta de crédito, el número CVV y la fecha de vencimiento. Así es como usted especifica los campos que el cliente de Braintree JavaScript representará en el iframe.

También incluye un elemento <input> llamado payment_method_nonce que usará para enviar el token nonce a su vista una vez que sea generado por Braintree Cliente JavaScript.

En su plantilla, carga el Braintree JavaScript SDK client.min.js y el Componente Hosted Fields hospedado-fields.min.js . Luego, ejecuta lo siguiente Código JavaScript:

1. Crea una instancia del cliente Braintree JavaScript con braintree.client.
método create () , usando el client_token generado por el pago_
vista de proceso .
2. Crea una instancia del componente Hosted Fields con braintree.
HostedFields.create () método.
3. Especifica estilos CSS personalizados para los campos de entrada .
4. Especifique los selectores de identificación para los campos número de tarjeta , cvv y
fecha de caducidad .

[279]

Gestión de pagos y pedidos

5. Utiliza form.addEventListener () para agregar un detector de eventos para el envío acción de la forma; esta es una función que espera la acción de envío y se ejecuta cuando ocurre. Cuando se envía el formulario, los campos son tokenizado con el SDK de Braintree y el token nonce se establece en el pago_
campo method_nonce . Luego, se envía el formulario para que su vista reciba el nonce para procesar el pago.

Edite la plantilla payment / done.html y agregue el siguiente código:

```
{% extiende "shop / base.html"%}
```

```
{% block title%} Pago exitoso {% endblock%}
```

```
{% block content%}
```

```
<h1> Su pago se realizó correctamente </h1>
```

```
<p> Su pago se ha procesado correctamente. </p>
```

```
{% endblock%}
```

Esta es la plantilla para la página a la que se redirige al usuario después de una exitosa pago.

Edite la plantilla de pago / cancelado.html y agregue el siguiente código:

```
{% extiende "shop / base.html"%}
```

```
{% block title%} Pago cancelado {% endblock%}
```

```
{% block content%}
```

```
<h1> Su pago no ha sido procesado </h1>
```

```
<p> Hubo un problema al procesar su pago. </p>
```

```
{% endblock%}
```

Esta es la plantilla para la página a la que se redirige al usuario cuando la transacción no tiene éxito. Probemos el proceso de pago.

Prueba de pagos

Abra un shell y ejecute RabbitMQ con el siguiente comando:

```
rabbitmq-servidor
```

Abra otro shell e inicie el trabajador de apio desde el directorio de su proyecto con el siguiente comando:

```
apio -Un trabajador de myshop -l info
```

Abra un shell más e inicie el servidor de desarrollo con este comando:

```
python manage.py runserver
```

Abra `http://127.0.0.1:8000/` en su navegador, agregue algunos productos al carrito de compras, y complete el formulario de pago. Al hacer clic en el botón **Realizar pedido**, el pedido se conservará en la base de datos, el ID del pedido se guardará en el actual sesión y será redirigido a la página del proceso de pago.

La página del proceso de pago recupera el pedido de la sesión y muestra el. Los campos alojados se forman en un `iframe`, de la siguiente manera:

Figura 8.3: El pago generado con la integración de Braintree Hosted Fields

Puede echar un vistazo al código fuente HTML para ver el HTML generado.

[281]

Gestión de pagos y pedidos

Braintree proporciona una lista de tarjetas de crédito exitosas y no exitosas para que pueda probar todos los escenarios posibles. Puede encontrar la lista de tarjetas de crédito para realizar pruebas en <https://developers.braintreepayments.com/guides/credit-cards/testing-go-live/python>. Vas a utilizar la tarjeta de prueba VISA 4111 1111 1111 1111, que devuelve una compra exitosa. Va a utilizar CVV 123 y cualquier fecha de vencimiento futura, como 12/28. Ingrese los detalles de la tarjeta de crédito de la siguiente manera:

Figura 8.4: El formulario de pago con los detalles de la tarjeta de crédito de prueba válida

Haga clic en el botón **Pagar**. Verá la siguiente página:

Figura 8.5: La página de pago exitoso

La transacción se ha procesado correctamente. Ahora puede iniciar sesión en su cuenta en <https://sandbox.braintreegateway.com/login>. En **Transacciones**,

poder ver la transacción:

[282]

Figura 8.6: La transacción almacenada en el panel de Braintree

A continuación, abra <http://127.0.0.1:8000/admin/orders/order/> en su navegador. El pedido ahora debería estar marcado como pagado y contener el Braintree relacionado ID de transacción:

Figura 8.7: Los campos de identificación Paid y Braintree del pedido que se ha procesado

¡Felicidades! Ha implementado una pasarela de pago para procesar tarjetas de crédito.

Tenga en cuenta que la vista `payment_process` no maneja los rechazos de transacciones. Braintree le proporciona los códigos de respuesta del procesador que devuelve la tarjeta de crédito procesador. Estos son especialmente útiles para saber por qué una transacción podría haber sido rechazado. Puede obtener un código de respuesta utilizando `result.transaction.processor_`

`response_code` y su texto de respuesta asociado usando `result.transaction.processor_response_text`. Puede encontrar la lista de autorización de pago respuestas en <https://developers.braintreepayments.com/reference/general/respuestas-del-procesador/respuestas-de-autorización>.

Va en directo

Una vez que haya probado su entorno, puede crear una cuenta real de Braintree en <https://www.braintreepayments.com>. Una vez que esté listo para pasar a la producción, recuerde cambiar las credenciales de su entorno en vivo en `settings.py` archivo de su proyecto y utilice `braintree.Environment.Production` para configurar tu entorno. Todos los pasos para comenzar a funcionar se resumen en <https://desarrolladores.braintreepayments.com/start/go-live/python>. Además de esto, puedes leer el *Capítulo 14*, *Puesta en marcha*, para aprender cómo configurar los ajustes del proyecto para múltiples Ambientes.

[283]

Gestión de pagos y pedidos

Exportación de pedidos a archivos CSV

A veces, es posible que desee exportar la información contenida en un modelo a un archivo para que pueda importarlo en otro sistema. Uno de los formatos más utilizados para exportar / importar datos son **valores separados por comas (CSV)**. Un archivo CSV es un texto sin formato archivo que consta de varios registros. Suele haber un registro por línea y algunos carácter delimitador, normalmente una coma literal, que separa los campos de registro. Usted está vamos a personalizar el sitio de administración para poder exportar pedidos a archivos CSV.

Agregar acciones personalizadas a la administración sitio

Django ofrece una amplia gama de opciones para personalizar el sitio de administración. Usted está va a modificar la vista de la lista de objetos para incluir una acción de administración personalizada. Tú puede implementar acciones de administración personalizadas para permitir que los usuarios del personal apliquen acciones a varios elementos a la vez en la vista de lista de cambios.

Una acción de administración funciona de la siguiente manera: un usuario selecciona objetos del página de lista de objetos de administración con casillas de verificación, luego seleccionan una acción para realizar en todos los elementos seleccionados y ejecute las acciones. La siguiente captura de pantalla muestra donde las acciones están ubicadas en el sitio de administración:

Figura 8.8: El menú desplegable para las acciones de administración de Django

Puede crear una acción personalizada escribiendo una función regular que reciba la siguientes parámetros:

- El `ModelAdmin` actual que se muestra
- El objeto de solicitud actual como una instancia de `HttpRequest`
- Un `QuerySet` para los objetos seleccionados por el usuario

[284]

Esta función se ejecutará cuando la acción se active desde el sitio de administración.

Vas a crear una acción de administración personalizada para descargar una lista de pedidos. como un archivo CSV. Edite el archivo `admin.py` de la aplicación de pedidos y agregue lo siguiente código antes de la clase `OrderAdmin` :

```

importar csv
importar fecha y hora
desde django.http import HttpResponseRedirect

def export_to_csv (modeladmin, request, queryset):
    opts = modeladmin.model._meta
    content_disposition = 'adjunto; filename = {opts.verbose_name}.
csv '

    respuesta = HttpResponseRedirect (content_type = 'texto / csv')
    respuesta ['Content-Disposition'] = content_disposition
    escritor = csv.writer (respuesta)

    fields = [campo por campo en opts.get_fields () si no \
field.many_to_many y no field.one_to_many]
    # Escribe una primera fila con información de encabezado
    Writer.writerow ([field.verbose_name for field in fields])
    # Escribir filas de datos
    para obj en queryset:
        data_row = []
        para campo en campos:
            valor = getattr (obj, field.name)
            if isinstance (valor, datetime.datetime):
                valor = valor.strftime ('% d /% m /% Y')
            data_row.append (valor)
        Writer.writerow (fila_de_datos)
    respuesta de retorno
export_to_csv.short_description = 'Exportar a CSV'

```

En este código, realiza las siguientes tareas:

1. Creas una instancia de HttpResponseRedirect , especificando el contenido text / csv type, para decirle al navegador que la respuesta debe tratarse como un archivo CSV. Tú también agregue un encabezado Content-Disposition para indicar que la respuesta HTTP contiene un archivo adjunto.
2. Se crea un archivo CSV escritor objeto que va a escribir a la respuesta del objeto.

[285]

Gestión de pagos y pedidos

3. Obtienes los campos del modelo dinámicamente usando el método get_fields () del modelo _meta options. Excluye varios a varios y uno a varios relaciones.
4. Escribe una fila de encabezado que incluya los nombres de los campos.
5. Se itera sobre el QuerySet dado y se escribe una fila para cada objeto devuelto. por el QuerySet. Te encargas de formatear los objetos de fecha y hora porque el El valor de salida para CSV debe ser una cadena.
6. Personaliza el nombre para mostrar de la acción en el menú desplegable de acciones. elemento del sitio de administración estableciendo un atributo short_description en la función.

Ha creado una acción de administración genérica que se puede agregar a cualquier Clase ModelAdmin .

Finalmente, agregue la nueva acción de administración export_to_csv a la clase OrderAdmin , como sigue:

```

clase OrderAdmin (admin.ModelAdmin):
    # ...
    acciones = [export_to_csv]

```

Inicie el servidor de desarrollo con el comando python manage.py runserver

y abra `http://127.0.0.1:8000/admin/orders/order/` en su navegador. los
La acción de administración resultante debería verse así:

Figura 8.9: Uso de la acción de administración personalizada Exportar a CSV

Seleccione algunos pedidos y elija la acción **Exportar a CSV** en el cuadro de selección, luego haga clic en el botón **Ir**. Su navegador descargará el archivo CSV generado llamado `order.csv`. Abra el archivo descargado con un editor de texto. Deberías ver contenido con el siguiente formato, incluida una fila de encabezado y una fila para cada objeto de pedido usted seleccionó:

[286]

ID, nombre, apellido, correo electrónico, dirección, código postal, ciudad, creado,
actualizado, pagado, identificación de braintree
3, Antonio, Melé, antonio.mele @ gmail.com, Bank Street, WS J11, Londres, 04/01 /
2020,04 / 01/2020, verdadero, 2bwkx5b6
...

Como puede ver, crear acciones de administración es bastante sencillo. Tú
puede obtener más información sobre cómo generar archivos CSV con Django en [https://docs.
djangoproject.com/en/3.0/howto/outputting-csv/](https://docs.djangoproject.com/en/3.0/howto/outputting-csv/).

Ampliando el sitio de administración con vistas personalizadas

A veces, es posible que desee personalizar el sitio de administración más allá de lo posible mediante la configuración de `ModelAdmin`, la creación de acciones de administración y plantillas de administración anuladas. Es posible que desee implementar más funcionalidades que no están disponibles en las vistas o plantillas de administración existentes. Si este es el caso, debe crear una vista de administración personalizada. Con una costumbre vista, puede crear cualquier funcionalidad que desee; solo tienes que asegurarte de que solo los usuarios del personal pueden acceder a su vista y que usted mantenga la apariencia de administración y sienta al hacer que su plantilla se extienda a una plantilla de administración.

Creemos una vista personalizada para mostrar información sobre un pedido. Edita `views.py` archivo de la aplicación de pedidos y agregue el siguiente código:

```
de django.contrib.admin.views.decorators import staff_member_required
desde django.shortcuts import get_object_or_404
de .models import Order

@staff_member_required
def admin_order_detail(solicitud, order_id):
    order = get_object_or_404(Order, id = order_id)
    return render (solicitud,
                    'admin / orders / order / detail.html',
                    {'orden': order})
```

El decorador `staff_member_required` verifica que tanto `is_active` como `is_`

Los campos de personal del usuario que solicita la página se establecen en Verdadero . En esta vista, obtienes el Objeto con el ID proporcionado y renderice una plantilla para mostrar el pedido.

[287]

Página 313

Gestión de pagos y pedidos

A continuación, edite el archivo `urls.py` de la aplicación de pedidos y agregue la siguiente URL patrón a él:

```
ruta ('admin / order / <int: order_id> /', views.admin_order_detail,
      name = 'admin_order_detail'),
```

Cree la siguiente estructura de archivos dentro del directorio `templates / de los pedidos solicitud:`

```
administración/
  pedidos/
    orden/
      detail.html
```

Edite la plantilla `detail.html` y agregue el siguiente contenido:

```
{% extiende "admin / base_site.html"%}

{% block title%}
  Pedido {{order.id}} {{block.super}}
{% endblock%}

{% block breadcrumbs%}
<div class = "breadcrumbs">
  <a href="{% url "admin:index" %}"> Inicio </a> & rsquo;
  <a href="{% url "admin:orders_order_changelist" %}"> Pedidos </a>
  & rsquo;
  <a href="{% url "admin:orders_order_change" order.id %}"> Pedido {{
order.id}} </a>
  & rsquo; Detalle
</div>
{% endblock%}

{% block content%}
<h1> Pedido {{order.id}} </h1>
<ul class = "objeto-herramientas">
  <li>
    <a href="#" onclick="window.print();"> Pedido de impresión </a>
  </li>
</ul>
<tabla>
  <tr>
    <th> Creado </th>
    <td> {{order.created}} </td>
  </tr>
</tr>
```

[288]

```
<th> Cliente </th>
<td> {{order.first_name}} {{order.last_name}} </td>
</tr>
<tr>
<th> Correo electrónico </th>
<td> <a href="mailto:{{ order.email }}"> {{order.email}} </a> </td>
</tr>
<tr>
<th> Dirección </th>
<td>
    {{order.address}},
    {{order.postal_code}} {{order.city}}
</td>
</tr>
<tr>
<th> Importe total </th>
<td> $ {{order.get_total_cost}} </td>
</tr>
<tr>
<th> Estado </th>
<td> {% if order.paid%} Paid {% else%} Pago pendiente {% endif%} </td>
</tr>
</table>

<div class = "módulo">
<h2> Artículos comprados </h2>
<table style = "ancho: 100%">
<thead>
<tr>
<th> Producto </th>
<th> Precio </th>
<th> Cantidad </th>
<th> Total </th>
</tr>
</thead>
<tbody>
{% para artículo en order.items.all%}
<tr class = "fila {% ciclo" 1 "" 2 "%}">
<td> {{item.product.name}} </td>
<td class = "num"> $ {{item.price}} </td>
<td class = "num"> {{item.quantity}} </td>
<td class = "num"> $ {{item.get_cost}} </td>
</tr>
```

Gestión de pagos y pedidos

```
{% endfor%}
<tr class = "total">
<td colspan = "3"> Total </td>
```

```

        <td class = "num"> $ {{order.get_total_cost}} </td>
    </tr>
</tbody>
</table>
</div>
{% endblock%
```

Asegúrese de que ninguna etiqueta de plantilla se divida en varias líneas.

Esta es la plantilla para mostrar los detalles de un pedido en el sitio de administración. Esta template extiende la plantilla admin / base_site.html de la administración de Django site, que contiene la estructura HTML principal y los estilos CSS. Usas los bloques definido en la plantilla principal para incluir su propio contenido. Muestra información sobre el pedido y los artículos comprados.

Cuando desee ampliar una plantilla de administración, debe conocer su estructura e identificar bloques existentes. Puede encontrar todas las plantillas de administración en <https://github.com/django/django/tree/3.0/django/contrib/admin/templates/admin>.

También puede anular una plantilla de administración si es necesario. Para hacerlo, copia una plantilla en su directorio templates / , manteniendo la misma ruta relativa y nombre del archivo. El sitio de administración de Django usará su plantilla personalizada en lugar de el predeterminado.

Finalmente, agreguemos un enlace a cada objeto Order en la página de visualización de lista del sitio de administración. Edite el archivo admin.py de la aplicación de pedidos y agregue el siguiente código, por encima de la clase OrderAdmin :

```

desde django.urls import reverse
de django.utils.safestring import mark_safe

def order_detail (obj):
    url = reverse ('pedidos: admin_order_detail', args = [obj.id])
    return mark_safe (f '<a href="{url}"> Ver </a>')
```

Esta es una función que toma un objeto Order como argumento y devuelve un HTML enlace para la URL admin_order_detail . Django escapa de la salida HTML de forma predeterminada. Debe usar la función mark_safe para evitar el escape automático.

Evite usar mark_safe en la entrada que proviene del usuario para evite **las secuencias de comandos entre sitios (XSS)** . XSS permite a los atacantes inyectar scripts del lado del cliente en el contenido web visto por otros usuarios.

[290]

Luego, edite la clase OrderAdmin para mostrar el enlace:

```

class OrderAdmin (admin.ModelAdmin):
    list_display = ['id',
                    'nombre de pila',
                    '# ...',
                    'actualizado',
                    'order_detail ]
```

Inicie el servidor de desarrollo con el comando python manage.py runserver y abra <http://127.0.0.1:8000/admin/orders/order/> en su navegador. Cada fila incluye un enlace **Ver** , como sigue:

Figura 8.10: El enlace Ver incluido en cada fila de pedido

Haga clic en el enlace **Ver** de cualquier pedido para cargar la página de detalles del pedido personalizado. Tú

debería ver una página como la siguiente:

Figura 8.11: La página de detalles de pedidos personalizados

[291]

Gestión de pagos y pedidos

Generar facturas en PDF de forma dinámica

Ahora que tiene un sistema de pago y pago completo, puede generar una factura en PDF para cada pedido. Hay varias bibliotecas de Python para generar PDF archivos. Una biblioteca popular para generar archivos PDF con código Python es ReportLab. Usted puede encontrar información sobre cómo generar archivos PDF con ReportLab en <https://docs.djangoproject.com/en/3.0/howto/outputting-pdf/>.

En la mayoría de los casos, deberá agregar estilos y formatos personalizados a su PDF archivos. Le resultará más conveniente renderizar una plantilla HTML y convertirla en un archivo PDF, manteniendo a Python alejado de la capa de presentación. Tu vas para seguir este enfoque y usar un módulo para generar archivos PDF con Django. Tú utilizará WeasyPrint, que es una biblioteca de Python que puede generar archivos PDF desde Plantillas HTML.

Instalación de WeasyPrint

Primero, instale las dependencias de WeasyPrint para su sistema operativo desde <https://weasyprint.readthedocs.io/en/latest/install.html>. Luego, instale WeasyPrint a través de pip usando el siguiente comando:

```
pip instalar WeasyPrint == 51
```

Creando una plantilla PDF

Necesita un documento HTML como entrada para WeasyPrint. Vas a crear un Plantilla HTML, renderícela usando Django y páselo a WeasyPrint para generar la Archivo PDF.

Cree un nuevo archivo de plantilla dentro del directorio templates / orders / order / del Solicitud de pedidos y asígnele el nombre pdf.html . Agregue el siguiente código:

```
<html>
<cuero>
```

```

<h1> Mi tienda </h1>
<p>
  Factura no. {{order.id}} </br>
  <span class = "secundaria">
    {{order.created | date: "M d, A"}}
  </span>
</p>

<h3> Factura a </h3>
<p>

```

[292]

Capítulo 8

```

  {{order.first_name}} {{order.last_name}} <br>
  {{order.email}} <br>
  {{order.address}} <br>
  {{order.postal_code}}, {{order.city}}
</p>

<h3> Artículos comprados </h3>
<tabla>
  <thead>
    <tr>
      <th> Producto </th>
      <th> Precio </th>
      <th> Cantidad </th>
      <th> Costo </th>
    </tr>
  </thead>
  <tbody>
    {% para articulo en order.items.all%}
      <tr class = "fila {% ciclo" 1 "" 2 "%}">
        <td> {{item.product.name}} </td>
        <td class = "num"> $ {{item.price}} </td>
        <td class = "num"> {{item.quantity}} </td>
        <td class = "num"> $ {{item.get_cost}} </td>
      </tr>
    {% endfor%}
    <tr class = "total">
      <td colspan = "3"> Total </td>
      <td class = "num"> $ {{order.get_total_cost}} </td>
    </tr>
  </tbody>
</table>

<span class = "{% if order.paid%} pagado {% else%} pendiente {% endif%}">
  {% if order.paid%} Paid {% else%} Pago pendiente {% endif%}
</span>
</body>
</html>

```

Esta es la plantilla para la factura en PDF. En esta plantilla, muestra todos los pedidos detalles y un elemento HTML <table> que incluye los productos. También incluye un mensaje para mostrar si se ha pagado el pedido.

[293]

Gestión de pagos y pedidos

Representación de archivos PDF

Va a crear una vista para generar facturas en PDF para pedidos existentes utilizando el sitio de administración. Edite el archivo `views.py` dentro del directorio de la aplicación de pedidos. y agregue el siguiente código:

```
desde la configuración de importación django.conf
desde django.http import HttpResponseRedirect
desde django.template.loader import render_to_string
importar weasyprint

@staff_member_required
def admin_order_pdf (solicitud, order_id):
    order = get_object_or_404 (Order, id = order_id)
    html = render_to_string ('pedidos / pedido / pdf.html',
                           {'orden': order})
    respuesta = HttpResponseRedirect (content_type = 'aplicación / pdf')
    respuesta ['Content-Disposition'] = f'filename = order_{order.id}.pdf'
    weasyprint.HTML (string = html).write_pdf (respuesta,
        hojas de estilo = [weasyprint.CSS (
            settings.STATIC_ROOT + 'css / pdf.css')])
    respuesta de retorno
```

Esta es la vista para generar una factura en PDF para un pedido. Usas el `staff_member_` se requiere decorador para asegurarse de que solo los usuarios del personal puedan acceder a esta vista.

Obtienes el objeto `Order` con la ID dada y usas `render_to_string ()` función proporcionada por Django para renderizar `orders / order / pdf.html`. El renderizado HTML se guarda en la variable `html`.

Luego, genera un nuevo objeto `HttpResponse` especificando la aplicación / pdf e incluye el encabezado `Content-Disposition` para especificar el nombre del archivo. Utiliza `WeasyPrint` para generar un archivo PDF a partir del código HTML renderizado y escriba el archivo en el objeto `HttpResponse`.

Utiliza el archivo estático `css / pdf.css` para agregar estilos CSS al archivo PDF generado. Luego, lo carga desde la ruta local utilizando la configuración `STATIC_ROOT`. Finalmente regresas la respuesta generada.

Si le faltan los estilos CSS, recuerde copiar los archivos estáticos ubicados en el `static /` directorio de la aplicación de la tienda a la misma ubicación de su proyecto.

Puede encontrar el contenido del directorio en <https://github.com/PacktPublishing/Django-3-por-ejemplo/árbol/master/Chapter08/myshop/shop/static>.

Dado que necesita utilizar la configuración `STATIC_ROOT`, debe agregarla a su proyecto. Esta es la ruta del proyecto donde residen los archivos estáticos. Edite el archivo `settings.py` del proyecto `myshop` y agregue la siguiente configuración:

```
STATIC_ROOT = os.path.join (BASE_DIR, 'static /')
```

[294]

Luego, ejecute el siguiente comando:

```
python manage.py collectstatic
```

Debería ver una salida que termina así:

133 archivos estáticos copiados a 'code / myshop / static'.

El comando collectstatic copia todos los archivos estáticos de sus aplicaciones en el directorio definido en la configuración STATIC_ROOT . Esto permite que cada aplicación proporcione sus propios archivos estáticos usando un directorio estático / que los contiene. También puede proporcionar fuentes de archivos estáticos adicionales en la configuración STATICFILES_DIRS . Todos los directorios especificado en la lista STATICFILES_DIRS también se copiará en STATIC_ROOT directorio cuando se ejecuta collectstatic . Siempre que ejecute collectstatic nuevamente, se le preguntará si desea anular los archivos estáticos existentes.

Edite el archivo urls.py dentro del directorio de la aplicación de pedidos y agregue lo siguiente Patrón de URL a él:

```
urlpatterns = [
    # ...
    ruta ('admin / order / <int: order_id> / pdf /',
          views.admin_order_pdf,
          nombre = 'admin_order_pdf'),
]
```

Ahora puede editar la página de visualización de la lista de administración para el modelo de pedido para agregue un enlace al archivo PDF para cada resultado. Edite el archivo admin.py dentro de los pedidos aplicación y agregue el siguiente código encima de la clase OrderAdmin :

```
def order_pdf (obj):
    url = reverse ('pedidos: admin_order_pdf', args = [obj.id])
    return mark_safe (f '<a href="{url}"> PDF </a>')
order_pdf.short_description = 'Factura'
```

Si especifica un atributo short_description para su invocable, Django lo usará para el nombre de la columna.

Agregue order_pdf al atributo list_display de la clase OrderAdmin , de la siguiente manera:

```
class OrderAdmin (admin.ModelAdmin):
    list_display = ['id',
                   # ...
                   detalle de la orden,
                   order_pdf ]
```

[295]

Gestión de pagos y pedidos

Asegúrese de que el servidor de desarrollo esté iniciado. Abra [http://127.0.0.1:8000/admin/pedidos / pedido /](http://127.0.0.1:8000/admin/pedidos/pedido/) en su navegador. Cada fila ahora debería incluir un enlace **PDF** , como este:

Figura 8.12: El enlace PDF incluido en cada fila de pedido

Haga clic en el enlace **PDF** para cualquier pedido. Debería ver un archivo PDF generado como el siguiente para pedidos que aún no se han pagado:

Figura 8.13: La factura en PDF de un pedido no pagado

[296]

Para pedidos pagados, verá el siguiente archivo PDF:

Figura 8.14: La factura en PDF de un pedido pagado

Envío de archivos PDF por correo electrónico

Cuando un pago sea exitoso, enviará un correo electrónico automático a su cliente. incluida la factura en PDF generada. Creará una tarea asincrónica para realizar esta acción.

Cree un nuevo archivo dentro del directorio de la aplicación de pago y asígnele el nombre `tasks.py` . Agregue el siguiente código:

```
desde io importar BytesIO
de la tarea de importación de apio
importar weasyprint
```

[297]

Gestión de pagos y pedidos

```
desde django.template.loader import render_to_string
desde django.core.mail importar EmailMessage
desde la configuración de importación django.conf
de orders.models import Order

@tarea
def payment_completed (order_id):
    """
    Tarea para enviar una notificación por correo electrónico cuando se realiza un pedido.
    creado con éxito.
    """
    order = Order.objects.get (id = order_id)

    # crear correo electrónico de factura
    subject = f'My Shop - EE Número de factura. {Solicitar ID}'
    message = 'Por favor, encuentre adjunta la factura de su
compra.'
    email = EmailMessage (asunto,
                                mensaje,
                                'admin@myshop.com',
                                [order.email])

    # generar PDF
    html = render_to_string ('pedidos / pedido / pdf.html', {'pedido': pedido})
    fuera = BytesIO ()
    hojas de estilo = [weasyprint.CSS (settings.STATIC_ROOT + 'css / pdf.css')]
    weasyprint.HTML (string = html) .write_pdf (fuera,
                                                hojas de estilo = hojas de estilo)

    # adjuntar archivo PDF
    email.attach (f'order_{order.id} .pdf',
                                out.getvalue (),
                                'Solicitud PDF')

    # enviar correo electrónico
    Enviar correo electrónico()
```

La tarea `payment_completed` se define mediante el decorador `@task` . En esta tarea, utiliza la clase `EmailMessage` proporcionada por Django para crear un objeto de correo electrónico . Luego, renderiza la plantilla en la variable `html` . Genera el archivo PDF desde el representa la plantilla y la envía a una instancia de `BytesIO` , que es un bytes en memoria buffer. Luego, adjunta el archivo PDF generado al objeto `EmailMessage` usando el método `attach ()` , incluido el contenido del búfer de salida . Finalmente, envías el correo electrónico.

Recuerde configurar su **Protocolo simple de transferencia de correo (SMTP)** en el archivo `settings.py` del proyecto para enviar correos electrónicos. Puede consultar el *Capítulo 2*, *Mejora de su blog con funciones avanzadas*, para ver un ejemplo funcional de un SMTP configuración. Si no desea configurar el correo electrónico, puede decirle a Django que escriba correos electrónicos a la consola agregando la siguiente configuración al archivo `settings.py`:

```
EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'
```

Vamos a añadir el `payment_completed` tarea a la vista. Edite el archivo `views.py` del aplicación de pago y modifíquela para que se vea así:

```
importar braintree
desde django.shortcuts import render, redirect, get_object_or_404
desde la configuración de importación django.conf
de orders.models import Order
de .tasks importar pago_completado

# instanciar la pasarela de pago de Braintree
gateway = braintree.BraintreeGateway (settings.BRAINTREE_CONF)

def proceso_pago (solicitud):
    order_id = request.session.get ('order_id')
    order = get_object_or_404 (Order, id = order_id)
    costo_total = order.get_total_cost ()

    if request.method == 'POST':
        # recuperar nonce
        nonce = request.POST.get ('método_pago_noce', Ninguno)
        # crear y enviar transacción
        resultado = gateway.transaction.sale ({
            'monto': f '{costo_total: .2f}',
            'payment_method_nonce': nonce,
            'opciones': {
                'submit_for_settlement': verdadero
            }
        })
        si resultado.is_success:
            # marcar el pedido como pagado
            order.paid = True
            # almacenar el ID de transacción único
            order.braintree_id = resultado.transaction.id
            order.save ()
            # lanzar tarea asincrónica
            payment_completed.delay (order.id)
            return redirect ('pago: hecho')
```

```
        más:
            return redirect ('pago: cancelado')

    más:
        # generar token
        client_token = gateway.client_token.generate ()
        return render (solicitud,
                        'pago / proceso.html',
                        {'orden orden',
                        'client_token': client_token})

def payment_done (solicitud):
    return render (solicitud, 'pago / hecho.html')

def payment_canceled (solicitud):
    return render (solicitud, 'pago / cancelado.html')
```

Se llama a la `payment_completed` tarea cuando el pago se ha realizado correctamente. Luego, llama al método `delay ()` de la tarea para ejecutarla de forma asincrónica. Los La tarea se agregará a la cola y será ejecutada por un trabajador de Apio tan pronto como sea posible.

Ahora puede completar un nuevo proceso de pago para recibir la factura en PDF en su correo electrónico.

Resumen

En este capítulo, integró la pasarela de pago de Braintree en su proyecto utilizando la integración de Hosted Fields. Creó una acción de administración personalizada para exportar pedidos a CSV. También personalizó el sitio de administración de Django usando vistas y plantillas personalizadas. Finalmente, aprendió a generar archivos PDF con WeasyPrint y cómo enviarlos por correo electrónico.

El próximo capítulo le dará una idea de la internacionalización y localización de proyectos Django. También creará un sistema de cupones utilizando Sesiones de Django y cree un motor de recomendación de productos con Redis.

Ampliando su tienda

9

En el capítulo anterior, aprendió cómo integrar una pasarela de pago en su tienda. También aprendió a generar archivos CSV y PDF.

En este capítulo, agregará un sistema de cupones a su tienda. También aprenderá cómo trabajo de internacionalización y localización, y creará una recomendación motor.

Este capítulo cubrirá los siguientes puntos:

- Creación de un sistema de cupones para aplicar descuentos.
- Añadiendo internacionalización a su proyecto
- Uso de Rosetta para administrar traducciones
- Traducir modelos usando django-parler
- Creación de un motor de recomendación de productos

Crear un sistema de cupones

Muchas tiendas en línea entregan cupones a los clientes que pueden canjearse por descuentos en sus compras. Un cupón en línea generalmente consiste en un código que es dado a los usuarios y es válido por un período de tiempo específico.

Vas a crear un sistema de cupones para tu tienda. Tus cupones serán válidos para los clientes en un período de tiempo determinado. Los cupones no tendrán limitaciones en términos de la cantidad de veces que se pueden canjear, y se aplicarán a el valor total del carrito de compras. Para esta funcionalidad, deberá crear un modelo para almacenar el código de cupón, un período de tiempo válido y el descuento para aplicar.

[301]

Ampliando su tienda

Cree una nueva aplicación dentro del proyecto myshop usando el siguiente comando:

cupones python manage.py startapp

Edite el archivo settings.py de myshop y agregue la aplicación a INSTALLED_APPS ajuste, como sigue:

```
INSTALLED_APPS = [  
    # ...  
    'cupons.apps.CouponsConfig',  
]
```

La nueva aplicación ahora está activa en su proyecto Django.

Construyendo el modelo de cupón

Comencemos por crear el modelo Cupón . Edite el archivo models.py de los cupones aplicación y agregue el siguiente código:

```
desde modelos de importación django.db  
de django.core.validators import MinValueValidator, \n                                     MaxValueValidator  
  
cupón de clase (modelos.Modelo):  
    code = models.CharField (max_length = 50,  
                             único = verdadero)  
    valid_from = models.DateTimeField ()  
    valid_to = models.DateTimeField ()
```

```

descuento = modelos.IntegerField (
    validadores = [MinValueValidator (0),
                   MaxValueValidator (100)])

active = modelos.BooleanField ()

def __str__ (yo):
    volver selfcode

```

Este es el modelo que vas a utilizar para almacenar cupones. El modelo de cupón contiene los siguientes campos:

- código : el código que los usuarios deben ingresar para aplicar el cupón a su compra.
- valid_from : el valor de fecha y hora que indica cuándo se convierte el cupón válido.

[302]

- valid_to : el valor de fecha y hora que indica cuándo se convierte el cupón inválido.
- descuento : la tasa de descuento que se aplicará (es un porcentaje, por lo que toma valores de 0 a 100). Utiliza validadores para este campo para limitar el mínimo y valores máximos aceptados.
- activo : un booleano que indica si el cupón está activo.

Ejecute el siguiente comando para generar la migración inicial para los cupones solicitud:

```
python manage.py makemigrations
```

La salida debe incluir las siguientes líneas:

Migraciones de 'cupones':

```

cupones / migraciones / 0001_initial.py:
- Crear modelo de cupón

```

Luego, ejecute el siguiente comando para aplicar migraciones:

```
python manage.py migrar
```

Debería ver un resultado que incluye la siguiente línea:

Aplicando cupones.0001_initial ... OK

Las migraciones ahora se aplican en la base de datos. Vamos a añadir el cupón modelo a la sitio de administración. Edite el archivo admin.py de la aplicación de cupones y agregue el siguiente código:

```

desde django.contrib import admin
de .models cupón de importación

@ admin.register (Cupón)
class CouponAdmin (admin.ModelAdmin):
    list_display = ['código', 'valid_from', 'valid_to',
                  'descuento', 'activo']
    list_filter = ['active', 'valid_from', 'valid_to']
    search_fields = ['código']

```

El modelo de cupón ahora está registrado en el sitio de administración. Asegúrese de que su el servidor local se ejecuta con el comando python manage.py runserver . Abierto <http://127.0.0.1:8000/admin/coupons/coupon/add/> en su navegador.

Ampliando su tienda

Debería ver el siguiente formulario:

Figura 9.1: El formulario Agregar cupón

Complete el formulario para crear un nuevo cupón que sea válido para la fecha actual y haga asegúrese de marcar la casilla de verificación **Activo** y haga clic en el botón **GUARDAR** .

Aplicar un cupón al carrito de compras

Puede almacenar nuevos cupones y realizar consultas para recuperar cupones existentes. Ahora necesita una forma para que los clientes apliquen cupones a sus compras. La funcionalidad Aplicar un cupón sería el siguiente:

1. El usuario agrega productos al carrito de compras.
2. El usuario puede ingresar un código de cupón en un formulario que se muestra en las compras. página de detalles del carrito.

3. Cuando el usuario ingresa un código de cupón y envía el formulario, busca un cupón existente con el código dado que es válido actualmente. Tienes para comprobar que el código del cupón coincide con el introducido por el usuario, que el atributo activo es Verdadero , y que la fecha y hora actual está entre el valor válido_ desde y valores válidos_to .
4. Si encuentra un cupón, guárdelo en la sesión del usuario y muestre el carrito, incluido el descuento que se le aplica y el importe total actualizado.
5. Cuando el usuario realiza un pedido, guarda el cupón en el pedido indicado.

Cree un nuevo archivo dentro del directorio de la aplicación de cupones y asígnele el nombre forms.py . Agregue el siguiente código:

```
desde formularios de importación de django
```

```
clase CouponApplyForm (formularios.Form):
    código = formularios.CharField ()
```

Este es el formulario que usará para que el usuario ingrese un código de cupón. Editar el archivo views.py dentro de la aplicación de cupones y agregue el siguiente código:

```
de django.shortcuts import render, redireccionar
desde django.utils importar zona horaria
desde django.views.decorators.http import require_POST
de .models cupón de importación
desde .forms import CouponApplyForm

@require_POST
def coupon_apply (solicitud):
    ahora = timezone.now ()
    form = CouponApplyForm (request.POST)
    si form.is_valid ():
        code = form.cleaned_data ['código']
        tratar:
            coupon = Coupon.objects.get (code__iexact = code,
                                         valid_from__lte = ahora,
                                         valid_to__gte = ahora,
                                         activo = Verdadero)

            request.session ['coupon_id'] = coupon.id
        excepto Coupon.DoesNotExist:
            request.session ['coupon_id'] = Ninguno
    return redirect ('carrito: cart_detail')
```

[305]

Ampliando su tienda

La vista coupon_apply valida el cupón y lo almacena en la sesión del usuario. Tú aplique el decorador require_POST a esta vista para restringirla a las solicitudes POST . En el vista, realiza las siguientes tareas:

1. Crea una instancia del formulario CouponApplyForm con los datos publicados y verifica que el formulario es válido.
2. Si el formulario es válido, obtiene el código ingresado por el usuario del formulario diccionario clean_data . Intenta recuperar el objeto Cupón con el

código dado. Utiliza la búsqueda de campo iexact para realizar una búsqueda sin distinción de mayúsculas y minúsculas con coincidencia exacta. El cupón debe estar actualmente activo (activo = verdadero) y vando para la fecha y hora actual. Utiliza la función timezone.now () de Django para obtener la fecha y hora actual que reconoce la zona horaria y la compara con el valor válido_ from y valid_to campos que realizan lte (menor o igual a) y gte (mayor o igual que) búsquedas de campo, respectivamente.

3. Almacena el ID del cupón en la sesión del usuario.
4. Redirige al usuario a la URL cart_detail para mostrar el carrito con el cupón aplicado.

Necesita un patrón de URL para la vista coupon_apply . Cree un nuevo archivo dentro del cupones de directorio de aplicaciones y el nombre de urls.py . Agregue el siguiente código:

```

desde la ruta de importación django.urls
de . importar vistas

app_name = 'cupones'

urlpatterns = [
    ruta ('aplicar /', vistas.coupon_apply, nombre = 'aplicar'),
]

```

Luego, edite el urls.py principal del proyecto myshop e incluya la URL de los cupones patrones, como sigue:

```

urlpatterns = [
    # ...
    ruta ('cupones /', include ('coupons.urls', namespace = 'cupones')),
    ruta ('', include ('shop.urls', namespace = 'shop')),
]

```

[306]

Recuerde colocar este patrón antes del patrón shop.urls .

Ahora, edite el archivo cart.py de la aplicación de carrito . Incluya la siguiente importación:

```

de coupons.models import Cupón

```

Agregue el siguiente código al final del método __init__ () de la clase Cart para inicializar el cupón de la sesión actual:

```

clase Carrito (objeto):
    def __init__ (self, request):
        # ...
        # cupón actual aplicado de la tienda
        self.coupon_id = self.session.get ('coupon_id')

```

En este código, intenta obtener la clave de sesión coupon_id de la sesión actual y almacenar su valor en el objeto Carrito . Agregue los siguientes métodos al objeto Carrito :

```

clase Carrito (objeto):
    # ...
    @propiedad
    def cupón (propio):
        si self.coupon_id:
            tratar:
                return Coupon.objects.get (id = self.coupon_id)
            excepto Coupon.DoesNotExist:

```



```

        pasar
    regresar Ninguno

def get_discount (auto):
    si self.coupon:
        retorno (self.coupon.discount / Decimal (100)) \
            * self.get_total_price ()
    return Decimal (0)

def get_total_price_after_discount (auto):
    return self.get_total_price () - self.get_discount ()

```

[307]

Ampliando su tienda

Estos métodos son los siguientes:

- `cupón ()` : define este método como una propiedad . Si el carrito contiene un atributo `coupon_id` , se devuelve el objeto Cupón con el ID proporcionado.
- `get_discount ()` : si el carrito contiene un cupón, recupera su descuento Tasa y devolución del importe a descontar del importe total del carrito.
- `get_total_price_after_discount ()` : devuelve el monto total del carrito después de deducir la cantidad devuelta por el método `get_discount ()` .

La clase Carrito ahora está preparada para manejar un cupón aplicado a la sesión actual. y aplicar el descuento correspondiente.

Incluamos el sistema de cupones en la vista detallada del carrito. Edite el archivo `views.py` del carrito de la aplicación y agregue la siguiente importación en la parte superior del archivo:

```

from coupons.forms import CouponApplyForm

```

Más abajo, edite la vista `cart_detail` y agregue el nuevo formulario, de la siguiente manera:

```

def cart_detail (solicitud):
    cart = Carrito (solicitud)
    para el artículo en el carrito:
        item ['update_quantity_form'] = CartAddProductForm (inicial = {
            'cantidad': artículo ['cantidad'],
            'anular': Verdadero))

    coupon_apply_form = CouponApplyForm ()

    return render (solicitud,
        'carrito / detalle.html',
        {'carrito': carrito ,
        'coupon_apply_form': coupon_apply_form })

```

Edite la plantilla `cart / detail.html` de la aplicación del carrito y localice el siguientes líneas:

```

<tr class = "total">
    <td> Total </td>
    <td colspan = "4"> </td>
    <td class = "num"> $ {{cart.get_total_price}} </td>
</tr>

```

Reemplácelos con lo siguiente:

```
{% if cart.coupon%}
<tr class = "subtotal">
  <td> Subtotal </td>
  <td colspan = "4"> </td>
  <td class = "num"> $ {{cart.get_total_price | floatformat: 2}} </td>
</tr>
<tr>
  <td>
    Cupón "{{cart.coupon.code}}"
    ({{cart.coupon.discount}}% de descuento)
  </td>
  <td colspan = "4"> </td>
  <td class = "num neg">
    - $ {{cart.get_discount | floatformat: 2}}
  </td>
</tr>
{% terminara si %}
<tr class = "total">
  <td> Total </td>
  <td colspan = "4"> </td>
  <td class = "num">
    $ {{carrito.get_total_price_after_discount | floatformat: 2 }}
  </td>
</tr>
```

Este es el código para mostrar un cupón opcional y su tasa de descuento. Si el carro contiene un cupón, muestra una primera fila, incluida la cantidad total del carrito como el subtotal. Luego, usa una segunda fila para mostrar el cupón actual aplicado a el carro. Finalmente, muestra el precio total, incluido cualquier descuento, llamando al `get_total_price_after_discount()` método del objeto de carrito .

En el mismo archivo, incluya el siguiente código después de la etiqueta HTML `</table>` :

```
<p> Aplicar un cupón: </p>
<form action = "{% url 'cupones: aplicar' %}" method = "post">
  {{coupon_apply_form}}
  <input type = "submit" value = "Apply">
  {% csrf_token%}
</form>
```

Ampliando su tienda

Esto mostrará el formulario para ingresar un código de cupón y aplicarlo al carrito actual.

Abra `http://127.0.0.1:8000/` en su navegador, agregue un producto al carrito y aplica el cupón que creaste ingresando su código en el formulario. Deberías ver eso el carrito muestra el cupón de descuento de la siguiente manera:

Figura 9.2: La página de detalles del carrito, incluidos los detalles del cupón y un formulario para aplicar un cupón

Agreguemos el cupón al siguiente paso del proceso de compra. Editar los pedidos / order / create.html plantilla de la aplicación de pedidos y localice el siguientes líneas:

```
<ul>
  {% para el artículo en el carrito%}
  <li>
    {{item.quantity}} x {{item.product.name}}
    <span> $ {{item.total_price}} </span>
  </li>
{% endfor%}
</ul>
```

Reemplácelos con el siguiente código:

```
<ul>
  {% para el artículo en el carrito%}
  <li>
    {{item.quantity}} x {{item.product.name}}
    <span> $ {{item.total_price | floatformat: 2 }} </span>
```

[310]

```
</li>
{% endfor%}
{% if cart.coupon%}
<li>
  "{{cart.coupon.code}}" ({{cart.coupon.discount}}% de descuento)
```



```
discount = models.IntegerField (predeterminado = 0,
                                validadores = [MinValueValidator (0),
                                                MaxValueValidator (100)])
```

Estos campos le permiten almacenar un cupón opcional para el pedido y el descuento. porcentaje aplicado con el cupón. El descuento se almacena en el Cupón relacionado objeto, pero lo incluye en el modelo de pedido para conservarlo si se modifica el cupón o eliminado. Establece on_delete en models.SET_NULL para que si el cupón recibe eliminado, el campo del cupón se establece en Nulo , pero el descuento se conserva.

Debe crear una migración para incluir los nuevos campos del modelo de pedido . Ejecutar el siguiente comando desde la línea de comando:

```
python manage.py makemigrations
```

Debería ver una salida como la siguiente:

Migraciones por 'pedidos':

```
orders / migrations / 0003_auto_20191213_1618.py:
```

- Agregar cupón de campo al pedido
- Agregar descuento de campo al pedido

[312]

Aplica la nueva migración con el siguiente comando:

```
python manage.py migrar pedidos
```

Debería ver una confirmación que indica que se ha aplicado la nueva migración. Los cambios en el campo del modelo de pedido ahora se sincronizan con la base de datos.

Vuelva al archivo models.py y cambie el método get_total_cost () del Modelo de pedido , como sigue:

```
class Orden (modelos.Modelo):
    # ...
    def get_total_cost (self):
        total_cost = sum (item.get_cost () para el artículo en self.items.all ())
        retorno costo_total - costo_total * \
            (auto.descuento / decimal (100))
```

El método get_total_cost () del modelo Order ahora tendrá en cuenta el descuento aplicado, si lo hay.

Edite el archivo views.py de la aplicación de pedidos y modifique la vista order_create para guardar el cupón relacionado y su descuento al crear un nuevo pedido. Encuentra el siguiente línea:

```
order = form.save ()
```

Reemplázelo con lo siguiente:

```
order = form.save ( commit = False )
si cart.coupon:
    order.coupon = cart.coupon
    order.discount = cart.coupon.discount
order.save ()
```

En el nuevo código, crea un objeto Order usando el método save () del Formulario OrderCreateForm . Evita guardarlo en la base de datos utilizando cometer = Falso . Si el carrito contiene un cupón, almacena el cupón relacionado y el descuento que se aplicó. Luego, guarda el objeto de pedido en la base de datos.

Asegúrese de que el servidor de desarrollo se esté ejecutando con el comando `python manage.py runserver`. Abra `http://127.0.0.1:8000/` en su navegador y complete una compra con el cupón que creó.

[313]

Página 339

Ampliando su tienda

Cuando termine una compra exitosa, puede ir a `http://127.0.0.1:8000/admin/orders/order/` y verifique que el objeto de la orden contenga el cupón y el descuento aplicado, de la siguiente manera:

Figura 9.4: El formulario de edición del pedido, incluido el cupón y el descuento aplicado

También puede modificar la plantilla de detalles de la orden de administración y el PDF de la orden factura para mostrar el cupón aplicado de la misma manera que lo hizo para el carrito.

A continuación, agregará la internacionalización a su proyecto.

Añadiendo internacionalización y localización

Django ofrece soporte completo de internacionalización y localización. Te permite traducir tu aplicación a varios idiomas y maneja configuraciones regionales específicas formato para fechas, horas, números y zonas horarias. Aclaremos la diferencia entre internacionalización y localización. **Internacionalización** (frecuentemente abreviado como **i18n**) es el proceso de adaptación del software para el uso potencial de diferentes idiomas y configuraciones regionales, de modo que no esté conectado a un idioma específico o localidad. **La localización** (abreviado como **l10n**) es el proceso de traducir realmente el software y adaptarlo a un lugar en particular. El propio Django se traduce en más de 50 idiomas utilizando su marco de internacionalización.

[314]

Internacionalización con Django

El marco de internacionalización le permite marcar fácilmente cadenas para traducción, tanto en código Python como en sus plantillas. Se basa en el gettext de GNU conjunto de herramientas para generar y administrar archivos de mensajes. Un **archivo de mensaje** es un archivo de texto sin formato que representa un idioma. Contiene una parte o todas las cadenas de traducción que se encuentran en su aplicación y sus respectivas traducciones para un solo idioma. Mensaje los archivos tienen la extensión .po . Una vez realizada la traducción, se compilan los archivos de mensajes. para ofrecer acceso rápido a cadenas traducidas. Los archivos de traducción compilados tienen la Extensión .mo .

Configuración de internacionalización y localización

Django proporciona varias configuraciones para la internacionalización. Los siguientes ajustes son los más relevantes:

- **USE_I18N** : un booleano que especifica si el sistema de traducción de Django es habilitado. Esto es True por defecto.
- **USE_L10N** : un booleano que indica si el formato localizado está habilitado. Cuando está activo, los formatos localizados se utilizan para representar fechas y números. Esto es falso por defecto.
- **USE_TZ** : un valor booleano que especifica si las fechas y horas tienen en cuenta la zona horaria. Cuando crea un proyecto con el comando startproject , esto se establece a Verdadero .
- **LANGUAGE_CODE** : el código de idioma predeterminado para el proyecto. Esto es en formato de ID de idioma estándar, por ejemplo, 'en-us' para inglés americano, o 'en-gb' para inglés británico. Esta configuración requiere USE_I18N ser establecido a Verdadero para que surta efecto. Puede encontrar una lista de ID de idiomas válidos en <http://www.i18nguy.com/unicode/language-identifiers.html>.
- **IDIOMAS** : una tupla que contiene los idiomas disponibles para el proyecto. Ellos vienen en dos tuplas de un **código de idioma** y un **nombre de idioma** . Puedes ver el lista de idiomas disponibles en django.conf.global_settings . Cuando usted elija en qué idiomas estará disponible su sitio, usted establece IDIOMAS a un subconjunto de esa lista.
- **LOCALE_PATHS** : una lista de directorios donde Django busca archivos de mensajes que contiene traducciones para el proyecto.
- **TIME_ZONE** : una cadena que representa la zona horaria del proyecto. Esto está configurado a 'UTC' cuando crea un nuevo proyecto usando el comando startproject . Puede configurarlo en cualquier otra zona horaria, como 'Europa / Madrid' .

[315]

Comandos de gestión de internacionalización

Django incluye los siguientes comandos de administración para administrar las traducciones:

- `makemessages` : se ejecuta sobre el árbol de origen para encontrar todas las cadenas marcadas para traducción y crea o actualiza los archivos de mensajes .po en la configuración regional directorio. Se crea un único archivo .po para cada idioma.
- `compilemessages` : Esto compila las existentes .po archivos de mensajes a .mo archivos que se utilizan para recuperar traducciones.

Necesitará el kit de herramientas gettext para poder crear, actualizar y compilar mensajes archivos. La mayoría de las distribuciones de Linux incluyen el kit de herramientas gettext. Si esta usando macOS, probablemente la forma más sencilla de instalarlo es a través de Homebrew, en <https://brew.sh/>, con el comando `brew install gettext` . Es posible que también necesite forzar vincularlo con el comando `brew link --force gettext` . Para ventanas, siga los pasos en <https://docs.djangoproject.com/en/3.0/topics/i18n/traducción/#gettext-on-windows>.

Cómo agregar traducciones a un proyecto de Django

Echemos un vistazo al proceso de internacionalización de su proyecto. Necesitarás hacer el seguimiento:

1. Marque cadenas para traducir en su código Python y sus plantillas
2. Ejecute el comando `makemessages` para crear o actualizar archivos de mensajes que incluir todas las cadenas de traducción de su código
3. Traduzca las cadenas contenidas en los archivos de mensajes y compílelas usando el comando de gestión `compilemessages`

Cómo determina Django el idioma actual

Django viene con un middleware que determina el idioma actual basado en los datos de la solicitud. Este es el middleware `LocaleMiddleware` que reside en `django.middleware.locale.LocaleMiddleware` realiza las siguientes tareas:

1. Si está utilizando `i18n_patterns` , es decir, está utilizando una URL traducida patrones, busca un prefijo de idioma en la URL solicitada para determinar el idioma actual.

[316]

2. Si no se encuentra ningún prefijo de idioma, busca un `LANGUAGE_SESSION_` existente CLAVE en la sesión del usuario actual.
3. Si el idioma no está configurado en la sesión, busca una cookie existente con el idioma actual. Se puede proporcionar un nombre personalizado para esta cookie en la configuración de `LANGUAGE_COOKIE_NAME` . De forma predeterminada, el nombre de esta cookie es `django_language` .
4. Si no se encuentra ninguna cookie, busca el encabezado `HTTP Accept-Language` de la solicitud.
5. Si el encabezado `Accept-Language` no especifica un idioma, Django usa el idioma definido en la configuración `LANGUAGE_CODE` .

Por defecto, Django utilizará el idioma definido en la configuración `LANGUAGE_CODE` a menos que esté utilizando `LocaleMiddleware` . El proceso descrito aquí solo se aplica al usar este middleware.

Preparando su proyecto para la internacionalización

Prepara tu proyecto para utilizar Django en los idiomas Vaso, Chaco y...

Versión en inglés y en español para tu tienda. Edite el archivo settings.py de su proyecto y agregue la siguiente configuración de IDIOMAS . Colóquelo junto a LANGUAGE_

Configuración de CÓDIGO :

```
IDIOMAS = (
    ('en', 'inglés'),
    ('es', 'español'),
)
```

La configuración IDIOMAS contiene dos tuplas que constan de un código de idioma y un nombre. Los códigos de idioma pueden ser específicos de la configuración regional, como en-us o en-gb , o genéricos, como en . Con esta configuración, especificas que tu aplicación solo estará disponible en ingles y español. Si no define una configuración de IDIOMAS personalizada , el sitio estará disponible en todos los idiomas a los que se traduce Django.

Haga que su configuración de LANGUAGE_CODE tenga el siguiente aspecto:

```
LANGUAGE_CODE = 'en'
```

Agregue 'django.middleware.locale.LocaleMiddleware' al MIDDLEWARE

ajuste. Asegúrese de que este middleware venga después de SessionMiddleware porque

LocaleMiddleware necesita usar datos de sesión. También debe colocarse antes

CommonMiddleware porque este último necesita un lenguaje activo para resolver el

URL solicitada. La configuración de MIDDLEWARE ahora debería verse como sigue:

```
MIDDLEWARE = [
```

[317]

Ampliando su tienda

```
'django.middleware.security.SecurityMiddleware',
'django.contrib.sessions.middleware.SessionMiddleware',
'django.middleware.locale.LocaleMiddleware',
'django.middleware.common.CommonMiddleware',
# ...
]
```

El orden de las clases de middleware es muy importante porque cada middleware puede depender de los datos establecidos por otro middleware ejecutado previamente. El middleware se aplica a las solicitudes en orden de aparición en MIDDLEWARE, y en orden inverso para las respuestas.

Cree la siguiente estructura de directorios dentro del directorio principal del proyecto, junto al archivo manage.py :

```
lugar/
es /
es /
```

El directorio de configuración regional es el lugar donde se ubicarán los archivos de mensajes de su aplicación. residir. Edite el archivo settings.py nuevamente y agregue la siguiente configuración:

```
LOCALE_PATHS = (
    os.path.join(BASE_DIR, 'locale /'),
)
```

La configuración LOCALE_PATHS especifica los directorios donde Django tiene que buscar archivos de traducción. Las rutas de configuración regional que aparecen primero tienen la mayor prioridad.

Cuando usa el comando makemessages desde el directorio de su proyecto, mensaje

Los archivos se generarán en la configuración regional / ruta que creó. Sin embargo, para aplicaciones

que contienen un directorio local / , los archivos de mensajes se generarán en ese directorio.

Traducir código Python

Para traducir literales en su código Python, puede marcar cadenas para traducir usando la función `gettext()` incluida en `django.utils.translation`. Esta función traduce el mensaje y devuelve una cadena. La convención es importar este `gettext` como un alias más corto llamado `_` (carácter de subrayado).

Puede encontrar toda la documentación sobre traducciones en <https://docs.djangoproject.com/en/3.0/topics/i18n/translation/>.

[318]

Traducciones estándar

El siguiente código muestra cómo marcar una cadena para su traducción:

```
de django.utils.translation importar gettext como _
output = _("Texto a traducir.")
```

Traducciones perezosas

Django incluye versiones **diferidas** para todas sus funciones de traducción, que tienen la sufijo `_lazy()`. Cuando se utilizan las funciones perezosas, las cadenas se traducen cuando el valor se accede, en lugar de cuando se llama a la función (es por eso que se traducen **perezosamente**). Las funciones de traducción diferida son útiles cuando las cadenas marcadas para la traducción están en rutas que se ejecutan cuando se cargan los módulos.

Usar `gettext_lazy()` en lugar de `gettext()` significa que las cadenas se traducen cuando se accede al valor. Django ofrece un perezoso versión para todas las funciones de traducción.

Traducciones que incluyen variables

Las cadenas marcadas para traducción pueden incluir marcadores de posición para incluir variables en las traducciones. El siguiente código es un ejemplo de una cadena de traducción con un marcador de posición:

```
de django.utils.translation importar gettext como _
mes = _("abril")
día = "14"
output = _("Hoy es% (mes) s% (día) s")% {'mes': mes,
                                         'día': día}
```

Mediante el uso de marcadores de posición, puede reordenar las variables de texto. Por ejemplo, un inglés traducción del ejemplo anterior podría ser *hoy es 14 de abril*, mientras que el español uno podría ser *hoy es 14 de abril*. Utilice siempre la interpolación de cadenas en lugar de posicional interpolación cuando tiene más de un parámetro para la cadena de traducción. Al hacerlo, podrá reordenar el texto del marcador de posición.

Formas plurales en traducciones

Para formas plurales, puede usar `ngettext()` y `ngettext_lazy()`. Estas funciones traducir formas singulares y plurales dependiendo de un argumento que indique la número de objetos. El siguiente ejemplo muestra cómo utilizarlos:

```
salida = ngettext('hay% (recuento) d producto',
                  'hay% (recuento) d productos',
                  count)% {'count': count}
```

[319]

Ampliando su tienda

Ahora que conoce los conceptos básicos sobre la traducción de literales en su código Python, es hora para aplicar traducciones a su proyecto.

Traduciendo tu propio código

Edite el archivo `settings.py` de su proyecto, importe la función `gettext_lazy()`, y cambie la configuración de `IDIOMAS` de la siguiente manera para traducir los nombres de los idiomas:

```
de django.utils.translation importar gettext_lazy como _

IDIOMAS = (
    ('en', _('Inglés')),
    ('es', _('español')),
)
```

Aquí, usa la función `gettext_lazy()` en lugar de `gettext()` para evitar una circular importar, traduciendo así los nombres de los idiomas cuando se accede a ellos.

Abra el shell y ejecute el siguiente comando desde el directorio de su proyecto:

django-admin makemessages --todos

Debería ver el siguiente resultado:

procesamiento de locale es

procesando locale en

Eche un vistazo al directorio `locale/`. Debería ver una estructura de archivo como la siguiente:

```
es /
  LC_MESSAGES /
    django.po

es /
  LC_MESSAGES /
    django.po
```

Se ha creado un archivo de mensaje `.po` para cada idioma. Abra `es/LC_MESSAGES/django.po` con un editor de texto. Al final del archivo, debería poder ver el siguiente:

```
#: myshop / settings.py: 118
msgid
msgstr ""

#: myshop / settings.py: 119
msgid
msgstr ""
```

[320]

Cada cadena de traducción está precedida por un comentario que muestra detalles sobre el archivo y la línea donde se encontró. Cada traducción incluye dos cadenas:

- `msgid` : la cadena de traducción tal como aparece en el código fuente.
- `msgstr` : la traducción del idioma, que está vacía de forma predeterminada. Aquí es donde debe ingresar la traducción real para la cadena dada.

Complete las traducciones de msgstr para la cadena de msgid dada , de la siguiente manera:

```
#: myshop / settings.py: 118
msgid
msgstr "Inglés"
```

```
#: myshop / settings.py: 119
msgid
msgstr "Español"
```

Guarde el archivo de mensaje modificado, abra el shell y ejecute el siguiente comando:

mensajes de compilación de django-admin

Si todo va bien, debería ver un resultado como el siguiente:

```
procesando el archivo django.po en myshop / locale / en / LC_MESSAGES
```

procesando el archivo django.po en myshop / locale / es / LC_MESSAGES

La salida le brinda información sobre los archivos de mensajes que se están compilando.

Eche un vistazo al directorio de configuración regional del proyecto myshop nuevamente. Deberías ver el siguientes archivos:

```
es / LC_MESSAGES /
      django.mo
      django.po
es / LC_MESSAGES /
      django.mo
      django.po
```

Puede ver que se ha generado un archivo de mensaje compilado .mo para cada idioma.

Ha traducido los nombres de los idiomas. Ahora, traduzcamos el modelo nombres de campo que se muestran en el sitio. Edite el archivo `models.py` de los pedidos aplicación y agregue nombres marcados para traducción para los campos del modelo de pedido como sigue:

de `django.utils.translation` importar `gettext_lazy` como `_`

[321]

Ampliando su tienda

[illegible]

Ha agregado nombres para los campos que se muestran cuando un usuario coloca un nuevo orden. Estos son nombre apellido , apellidos , correo electrónico , dirección , postal_code , y la ciudad . Recuerde que también puede usar el atributo verbose_name para nombrar los campos.

Cree la siguiente estructura de directorio dentro del directorio de la aplicación de pedidos :

```
lugar/  
  es /  
    es /
```

Al crear un directorio de configuración regional , las cadenas de traducción de esta aplicación se almacenarán en un archivo de mensajes en este directorio en lugar del archivo de mensajes principal. De esta manera, tu puede generar archivos de traducción separados para cada aplicación.

Abra el shell desde el directorio del proyecto y ejecute el siguiente comando:

```
django-admin makemessages --todos
```

Debería ver el siguiente resultado:

```
procesamiento de locale es  
procesando locale en
```

Abra el archivo locale / es / LC_MESSAGES / django.po de la aplicación de pedidos usando un editor de texto. Verá las cadenas de traducción para el modelo de pedido . Complete el siguientes traducciones de msgstr para las cadenas de msgid dadas :

```
#: orders / models.py: 11  
msgid "nombre"  
msgstr "nombre"  
  
#: orders / models.py: 12
```

[322]

```
msgid "apellido"  
msgstr "apellidos"  
  
#: orders / models.py: 13  
msgid  
msgstr "correo electrónico"  
  
#: orders / models.py: 13  
msgid "dirección"  
msgstr "dirección"  
  
#: orders / models.py: 14  
msgid "código postal"  
msgstr "código postal"  
  
#: orders / models.py: 15  
msgid "ciudad"  
msgstr "ciudad"
```

Una vez que haya terminado de agregar las traducciones, guarde el archivo.

Además de un editor de texto, puede utilizar Poedit para editar traducciones. Poedit es un software para editar traducciones que usan gettext. Está disponible para Linux, Windows, y macOS. Puede descargar Poedit desde <https://poedit.net/>.

Traduzcamos también las formas de su proyecto. El OrderCreateForm de los pedidos la aplicación no tiene que ser traducida, ya que es un ModelForm y usa la

Atributo verbose_name de los campos del modelo de pedido para las etiquetas de campo de formulario. Tú vamos a traducir los formularios de las solicitudes de cupones y carritos .

Edite el archivo forms.py dentro del directorio de la aplicación del carrito y agregue una etiqueta atributo al campo de cantidad de CartAddProductForm , y luego marque esto

campo para traducción, de la siguiente manera:

```
desde formularios de importación de django
de django.utils.translation importar gettext_lazy como _

PRODUCT_QUANTITY_CHOICES = [(i, str(i)) para i en el rango (1, 21)]

clase CartAddProductForm (formularios.Form):
    cantidad = formularios.TypedChoiceField (
        opciones = PRODUCT_QUANTITY_CHOICES,
        coerción = int,
        label = _ ('Cantidad') )
    override = forms.BooleanField (obligatorio = Falso,
        inicial = Falso,
        widget = formularios.HiddenInput)

[ 323 ]
```

Ampliando su tienda

Edite el archivo forms.py de la aplicación de cupones y traduzca el CouponApplyForm formulario, de la siguiente manera:

```
desde formularios de importación de django
de django.utils.translation importar gettext_lazy como _

clase CouponApplyForm (formularios.Form):
    código = formularios.CharField ( etiqueta = _ ('Cupón'))
```

Ha agregado una etiqueta al campo de código y lo ha marcado para su traducción.

Traducción de plantillas

Django ofrece las etiquetas de plantilla `{% trans %}` y `{% blocktrans %}` para traducir cadenas en plantillas. Para utilizar las etiquetas de la plantilla de traducción, debe agregar `{% load i18n %}` en la parte superior de su plantilla para cargarlos.

La etiqueta de plantilla `{% trans %}`

La etiqueta de plantilla `{% trans %}` le permite marcar un literal para traducir. Internamente, Django ejecuta `gettext ()` en el texto dado. Así es como se marca una cadena para traducción en una plantilla:

```
{% trans "Texto a traducir"%}
```

Puede usar `as` para almacenar el contenido traducido en una variable que puede usar a lo largo de su plantilla. El siguiente ejemplo almacena el texto traducido en una variable llamada `saludo` :

```
{% trans "¡Hola!" como saludo%}
<h1> {{saludo}} </h1>
```

La etiqueta `{% trans %}` es útil para cadenas de traducción simples, pero no puede manejar contenido para traducción que incluye variables.

La etiqueta de plantilla `{% blocktrans %}`

La etiqueta de plantilla `{% blocktrans %}` le permite marcar contenido que incluye literales y contenido variable mediante marcadores de posición. El siguiente ejemplo muestra cómo usar la etiqueta `{% blocktrans %}` , incluida una variable de nombre en el contenido para traducir:

```
{% blocktrans %} ¡Hola, {{name}}! {% endblocktrans %}
```

Puede usar `con` para incluir expresiones de plantilla, como acceder a atributos de objeto o aplicar filtros de plantilla a las variables. Siempre tienes que usar marcadores de posición para estas. No puede acceder a expresiones o atributos de objeto dentro del bloque `blocktrans` . El siguiente ejemplo muestra cómo utilizar `con` para incluir un atributo de objeto en que se aplica el filtro `capfirst` :

```
{% blocktrans con name = user.name | capfirst%}
    ¡Hola, {{name}}!
{% endblocktrans%}
```

Utilice la etiqueta `{% blocktrans%}` en lugar de `{% trans%}` cuando debe incluir contenido variable en su cadena de traducción.

Traducir las plantillas de la tienda

Edita la plantilla `shop / base.html` de la aplicación de la tienda . Asegúrate de cargar la etiqueta `i18n` en la parte superior de la plantilla y marque las cadenas para la traducción, de la siguiente manera:

```
{% load i18n%}
{% load static%}
<!DOCTYPE html>
<html>
< cabeza>
    <meta charset = "utf-8" />
    < título>
        {% block title%} {% trans "Mi tienda"%} {% endblock%}
    </ título>
    < link href = "{% static" css / base.css "%}" rel = "stylesheet">
</ head>
< cuerpo>
    < div id = "header">
        < a href="/" class="logo"> {% trans "Mi tienda"%} </ a>
    </ div>
    < div id = "subheader">
        < div class = "carrito">
            {% con total_items = carrito | longitud%}
            {% si total_items> 0%}
                {% trans "Your cart"%}:
                < a href="{% url "cart:cart_detail" %}">
                    {% blocktrans con total = cart.get_total_price count
items = total_items%}
                        {{items}} artículo, $ {{total}}
```

```

        {% plural%}
        {{items}} artículos, $ {{total}}
    {% endblocktrans%}
</a>
{% else%}
    {% trans "Su carrito está vacío". %}
    {% terminara si %}
    {% terminar con %}
</div>
</div>
<div id = "contenido">
    {% block content%}
    {% endblock%}
</div>
</body>
</html>

```

Asegúrese de que ninguna etiqueta de plantilla esté dividida en varias líneas.

Observe la etiqueta {% blocktrans%} para mostrar el resumen del carrito. El resumen del carrito anteriormente era el siguiente:

```

{{total_items}} elemento {{total_items | pluralize}},
$ {{cart.get_total_price}}

```

Lo cambió y ahora usa {% blocktrans con ...%} para configurar el marcador de posición total con el valor de cart.get_total_price (el método del objeto llamado aquí). También usa count , que le permite establecer una variable para contar objetos para que Django seleccione la forma plural correcta. Establece la variable de elementos para contar objetos con el valor de total_items . Esto le permite establecer una traducción para el formas singular y plural, que separa con la etiqueta {% plural%} dentro el bloque {% blocktrans%} . El código resultante es:

```

{% blocktrans con total = cart.get_total_price count items = total_items %}
    {{items}} artículo, $ {{total}}
{% plural%}
    {{items}} artículos, $ {{total}}
{% endblocktrans%}

```

A continuación, edite la plantilla shop / product / detail.html de la aplicación de la tienda y cargue las etiquetas i18n en la parte superior, pero después de la etiqueta {% extensions%} , que siempre tiene que ser la primera etiqueta de la plantilla:

```
{% load i18n%}
```

Luego, busque la siguiente línea:

```
<input type = "submit" value = "Agregar al carrito">
```

Reemplácelo con lo siguiente:

```
<input type = "submit" value = "{% trans "Agregar al carrito "%}" ">
```

Ahora, traduzca la plantilla de solicitud de pedidos . Edite los pedidos / pedido / crear.html de la aplicación de pedidos y marque el texto para traducir, de la siguiente manera:

```

{% extiende "shop / base.html"%}
{% load i18n%}

```



```

{% block title%}
    {% trans "Checkout"%}
{% endblock%}

{% block content%}
    <h1 > {% trans "Checkout"%} </h1>

    <div class = "información de pedido">
        <h3> {% trans "Su pedido"%} </h3>
        <ul>
            {% para el articulo en el carrito%}
            <li>
                {{item.quantity}} x {{item.product.name}}
                <span> $ {{item.total_price}} </span>
            </li>
            {% endfor%}
            {% if cart.coupon%}
            <li>
                {% blocktrans con código = cart.coupon.code discount = cart.
coupon.discount%}
                    "{{code}}" ({{discount}}% de descuento)
                {% endblocktrans%}
                <span class = "neg"> - $ {{cart.get_discount | floatformat: 2}} </
lazo>
                    </li>
                {% terminara si %}
            </ul>
            <p> {% trans "Total"%}: $ {{
cart.get_total_price_after_discount | floatformat: 2}} </p>
        </div>

        <form method = "post" class = "order-form">
            {{form.as_p}}
            <p> <input type = "submit" value = "{% trans " Realizar pedido "%} "> </p>

```

[327]

Ampliando su tienda

```

    {% csrf_token%}
</form>
{% endblock%}

```

Asegúrese de que ninguna etiqueta de plantilla esté dividida en varias líneas. Eche un vistazo al siguientes archivos en el código que acompaña a este capítulo para ver cómo las cadenas marcado para traducción:

- La aplicación de la tienda : Template shop / product / list.html
- La aplicación de pedidos : plantilla pedidos / pedido / created.html
- La aplicación del carrito : Template cart / detail.html

Puede encontrar el código fuente de este capítulo en <https://github.com/PacktPublishing/Django-3-by-Example/tree/master/Chapter09>.

Actualicemos los archivos de mensajes para incluir las nuevas cadenas de traducción. Abre la cáscara y ejecuta el siguiente comando:

django-admin makemessages --todos

Los archivos .po están dentro del directorio de configuración regional del proyecto myshop y verá que la aplicación de pedidos ahora contiene todas las cadenas que marcó para traducir.

Edite los archivos de traducción .po del proyecto y la aplicación de pedidos , e incluya Traducciones al español en el msgstr . También puede utilizar los archivos .po traducidos en el código fuente que acompaña a este capítulo.

Ejecute el siguiente comando para compilar los archivos de traducción:

mensajes de compilación de django-admin

Verá el siguiente resultado:

procesando el archivo django.po en myshop / locale / en / LC_MESSAGES

procesando el archivo django.po en myshop / locale / es / LC_MESSAGES

procesando el archivo django.po en myshop / orders / locale / en / LC_MESSAGES

procesando el archivo django.po en myshop / orders / locale / es / LC_MESSAGES

Se ha generado un archivo .mo que contiene traducciones compiladas para cada .po archivo de traducción.

Usando la interfaz de traducción de Rosetta

Rosetta es una aplicación de terceros que le permite editar traducciones usando la misma interfaz que el sitio de administración de Django. Rosetta hace que sea fácil editar archivos .po y actualiza los archivos de traducción compilados. Vamos a agregarlo a su proyecto.

[328]

Instale Rosetta a través de pip usando este comando:

pip instalar django-rosetta == 0.9.3

A continuación, añadir 'Rosetta' a la INSTALLED_APPS ajuste en su proyecto de settings.py archivo, de la siguiente manera:

```
INSTALLED_APPS = [  
    # ...  
    'rosetta',  
]
```

Debe agregar las URL de Rosetta a su configuración de URL principal. Edita el principal urls.py de su proyecto y agregue el siguiente patrón de URL:

```
urlpatterns = [  
    # ...  
    ruta('rosetta/', include('rosetta.urls')),  
    ruta('', include('shop.urls', namespace='shop')),  
]
```

Asegúrese de colocarlo antes del patrón shop.urls para evitar un indeseado coincidencia de patrones.

Abra `http://127.0.0.1:8000/admin/` e inicie sesión con un superusuario. Luego, navega a `http://127.0.0.1:8000/rosetta/` en su navegador. En el menú **Filtro**, haga clic en **TERCERO** para mostrar todos los archivos de mensajes disponibles, incluidos los que pertenecen a la solicitud de pedidos. Debería ver una lista de idiomas existentes, de la siguiente manera:

Ampliando su tienda

Haga clic en el enlace de **Myshop** en la sección en **español** para editar las traducciones al español. Debería ver una lista de cadenas de traducción, como se muestra a continuación:

Figura 9.6: Edición de traducciones al español con Rosetta

Puede ingresar las traducciones en la columna **ESPAÑOL**. La **OCURRENCIA (S)** muestra los archivos y la línea de código donde se encontró cada cadena de traducción. Las traducciones que incluyen marcadores de posición aparecerán de la siguiente manera:

Figura 9.7: Traducciones que incluyen marcadores de posición

Rosetta usa un color de fondo diferente para mostrar marcadores de posición. Cuando usted traducir contenido, asegúrese de mantener los marcadores de posición sin traducir. Por ejemplo, tomar la siguiente cadena:

% (elementos) s elementos, \$(total) s

Está traducido al español de la siguiente manera:

```
% (items) s productos, $% (total) s
```

Puede echar un vistazo al código fuente que viene con este capítulo para usar el mismas traducciones al español para su proyecto.

Cuando termine de editar traducciones, haga clic en el **Guardar y traducir bloque siguiente** botón para guardar las traducciones en el archivo .po. Rosetta compila el archivo de mensajes cuando guardar las traducciones, por lo que no es necesario que ejecute el comando `compilemessages`. Sin embargo, Rosetta requiere acceso de escritura a los directorios locales para escribir el archivos de mensajes. Asegúrese de que los directorios tengan permisos válidos.

Si desea que otros usuarios puedan editar traducciones, abra

`http://127.0.0.1:8000/admin/auth/group/add/` en su navegador y cree un nuevo grupo llamado traductores. Luego, acceda a `http://127.0.0.1:8000/admin/`

`auth/user/` para editar los usuarios a los que desea otorgar permisos para que puede editar traducciones. Al editar un usuario, en la sección **Permisos**, agregue el grupo de traductores a los **Grupos Elegidos** para cada usuario. Rosetta solo está disponible a superusuarios o usuarios que pertenecen al grupo de traductores.

Puede leer la documentación de Rosetta en <https://django-rosetta.readthedocs.io/>.

Cuando agrega nuevas traducciones a su entorno de producción, si sirve a Django con un servidor web real, tendrá que volver a cargar su servidor después de ejecutar el comando `compilemessages`, o después de guardar las traducciones con Rosetta, para que los cambios surtan efecto.

Traducciones difusas

Es posible que haya notado que hay una columna **FUZZY** en Rosetta. Esto no es una característica de Rosetta; es proporcionado por gettext. Si el indicador difuso está activo para una traducción, no se incluirá en los archivos de mensajes compilados. Esta bandera marca la traducción cadenas que deben ser revisadas por un traductor. Cuando los archivos .po se actualizan con nuevas cadenas de traducción, es posible que algunas cadenas de traducción ser marcado como difuso. Esto sucede cuando gettext encuentra algún msgid que ha sido ligeramente modificado. gettext lo empareja con lo que cree que era la traducción anterior y lo marca como borroso para su revisión. Luego, el traductor debe revisar las traducciones borrosas, elimine el indicador difuso y vuelva a compilar el archivo de traducción.

[331]

Ampliando su tienda

Patrones de URL para la internacionalización

Django ofrece capacidades de internacionalización para URL. Incluye dos principales características para URL internacionalizadas:

- **Prefijo de idioma en patrones de URL** : agregar un prefijo de idioma a las URL para publicar la versión de cada idioma en una URL base diferente
- **Patrones de URL traducidos** : traducción de patrones de URL para que cada URL

es diferente para cada idioma

Una razón para traducir URL es optimizar su sitio para los motores de búsqueda. Por agregando un prefijo de idioma a sus patrones, podrá indexar una URL para cada idioma en lugar de una única URL para todos ellos. Además, al traducir URL en cada idioma, proporcionará a los motores de búsqueda URL que se clasificarán mejor para cada idioma.

Agregar un prefijo de idioma a los patrones de URL

Django le permite agregar un prefijo de idioma a sus patrones de URL. Por ejemplo, la versión en inglés de su sitio se puede publicar bajo una ruta que comienza / en / , y la Versión en español bajo / es / . Para usar idiomas en patrones de URL, debe usar el LocaleMiddleware proporcionado por Django. El marco lo utilizará para identificar el idioma actual de la URL solicitada. Lo agregaste previamente al MIDDLEWARE de su proyecto, por lo que no es necesario que lo haga ahora.

Agreguemos un prefijo de idioma a sus patrones de URL. Edite el archivo urls.py principal del proyecto myshop y agregue i18n_patterns() , de la siguiente manera:

```
de django.conf.urls.i18n importar i18n_patterns

urlpatterns = i18n_patterns (
    ruta ('admin /', admin.site.urls),
    ruta ('carrito /', include ('cart.urls', namespace = 'carrito')),
    ruta ('pedidos /', incluir ('pedidos.urls', espacio de nombres = 'pedidos')),
    ruta ('pago /', incluir ('pago.urls', espacio de nombres = 'pago')),
    ruta ('cupones /', include ('coupons.urls', namespace = 'cupones')),
    ruta ('rosetta /', include ('rosetta.urls')),
    ruta ('', include ('shop.urls', namespace = 'shop')),
)
```

Puede combinar patrones de URL estándar no traducibles y patrones en

i18n_patterns para que algunos patrones incluyan un prefijo de idioma y otros no. Sin embargo, es mejor utilizar URL traducidas solo para evitar la posibilidad de que La URL traducida descuidadamente coincide con un patrón de URL no traducido.

[332]

Ejecute el servidor de desarrollo y abra http://127.0.0.1:8000/ en su navegador. Django realizará los pasos descritos anteriormente en *Cómo determina Django la sección de idioma actual* para determinar el idioma actual, y redirigirá a la URL solicitada, incluido el prefijo de idioma. Eche un vistazo a la URL en su navegador; ahora debería verse como http://127.0.0.1:8000/en/ . La corriente idioma es el establecido por el encabezado Accept-Language de su navegador si es Español o inglés; de lo contrario, es el LANGUAGE_CODE (inglés) predeterminado definido en su configuración.

Traducción de patrones de URL

Django admite cadenas traducidas en patrones de URL. Puedes usar un diferente traducción para cada idioma para un único patrón de URL. Puede marcar patrones de URL para la traducción de la misma manera que lo haría con literales, usando gettext_ función lazy() .

Edite el archivo urls.py principal del proyecto myshop y agregue cadenas de traducción a las expresiones regulares de los patrones de URL para el carrito , los pedidos , el pago y solicitudes de cupones , de la siguiente manera:

```
de django.utils.translation importar gettext_lazy como _

urlpatterns = i18n_patterns (
```

```

ruta ( _ (' admin / ' ) , admin.site.urls),
ruta ( _ (' carrito / ' ) , incluir ('cart.urls', espacio de nombres = 'carrito')),
ruta ( _ (' pedidos / ' ) , incluir ('pedidos.urls', espacio de nombres = 'pedidos')),
ruta ( _ (' pago / ' ) , incluir ('pago.urls', espacio de nombres = 'pago')),
ruta ( _ (' cupones / ' ) , incluir ('cupones.urls', espacio de nombres = 'cupones')),
ruta ('rosetta /', include ('rosetta.urls')),
ruta ('', include ('shop.urls', namespace = 'shop')),
)

```

Edite el archivo `urls.py` de la aplicación de pedidos y marque los patrones de URL para traducción, como sigue:

```

de django.utils.translation import gettext_lazy como _

urlpatterns = [
    ruta ( _ (' crear / ' ) , views.order_create, name = 'order_create'),
    # ...
]

```

Edite el archivo `urls.py` de la aplicación de pago y cambie el código al siguiendo:

```

de django.utils.translation import gettext_lazy como _

[ 333 ]

```

Ampliando su tienda

```

urlpatterns = [
    ruta ( _ (' proceso / ' ) , vistas.proceso_pago, nombre = 'proceso'),
    ruta ( _ (' hecho / ' ) , vistas.pago_done, nombre = 'hecho'),
    ruta ( _ (' cancelado / ' ) , vistas.pago_cancelado, nombre = 'cancelado'),
]

```

No es necesario traducir los patrones de URL de la aplicación de la tienda , ya que son construido con variables y no incluye ningún otro literal.

Abra el shell y ejecute el siguiente comando para actualizar los archivos de mensajes con el nuevo traducciones:

```
django-admin makemessages --todos
```

Asegúrese de que el servidor de desarrollo se esté ejecutando. Abra `http://127.0.0.1:8000/en/rosetta/` en su navegador y haga clic en el enlace de **Myshop** en la sección en **español** . Ahora verá los patrones de URL para la traducción. Puede hacer clic en Sin **traducir solo** para ver solo las cadenas que aún no se han traducido. Ahora puedes traducir las URL.

Permitir a los usuarios cambiar de idioma

Dado que está ofreciendo contenido que está disponible en varios idiomas, debe permita que sus usuarios cambien el idioma del sitio. Vas a agregar un selector de idioma a Tú sitio. El selector de idioma consistirá en una lista de idiomas disponibles mostrados utilizando enlaces.

Edite la plantilla `shop / base.html` de la aplicación de la tienda y localice lo siguiente líneas:

```

<div id = "header">
    <a href="/" class="logo"> {% trans "Mi tienda"%} </a>
</div>

```

Reemplácelos con el siguiente código:

```

<div id = "header">
    <a href="/" class="logo"> {% trans "Mi tienda"%} </a>
    {% get_current_language as LANGUAGE_CODE%}
    {% get_available_languages as LANGUAGES%}

```

```
{% get_language_info_list para LANGUAGES as languages%}
<div class = "idiomas">
  <p> {% trans "Language"%}: </p>
  <ul class = "idiomas">
    {% para idioma en idiomas%}
      <li>
```

[334]

```
      <a href = "/" {{language.code}} "/"
        {% if language.code == LANGUAGE_CODE%} class = "selected" {%
endif%}>
        {{language.name_local}}
      </a>
    </li>
  {% endfor%}
</ul>
</div>
</div>
```

Asegúrese de que ninguna etiqueta de plantilla se divida en varias líneas.

Así es como construyes tu selector de idioma:

1. Carga las etiquetas de internacionalización con {% load i18n%}
2. Utiliza la etiqueta {% get_current_language%} para recuperar la idioma
3. Obtienes los idiomas definidos en la configuración IDIOMAS usando el {% get_available_languages%} etiqueta de plantilla
4. Utiliza la etiqueta {% get_language_info_list%} para facilitar el acceso a los atributos del idioma
5. Crea una lista HTML para mostrar todos los idiomas disponibles y agrega un atributo de clase seleccionado al idioma activo actual

En el código del selector de idioma, usó las etiquetas de plantilla proporcionadas por i18n , según los idiomas disponibles en la configuración de su proyecto. Ahora abierto <http://127.0.0.1:8000/> en su navegador y eche un vistazo. Deberías ver el selector de idioma en la esquina superior derecha del sitio, de la siguiente manera:

Figura 9.8: La página de la lista de productos, incluido un selector de idioma en el encabezado del sitio

Ampliando su tienda

Los usuarios ahora pueden cambiar fácilmente a su idioma preferido haciendo clic en él.

Traducir modelos con django-parler

Django no proporciona una solución para traducir modelos de fábrica. Tienes para implementar su propia solución para administrar el contenido almacenado en diferentes idiomas, o utilice un módulo de terceros para la traducción de modelos. Hay varios terceros aplicaciones que le permiten traducir campos del modelo. Cada uno de ellos toma un diferente enfoque para almacenar y acceder a las traducciones. Una de estas aplicaciones es django-parler. Este módulo ofrece una forma muy eficaz de traducir modelos e integra sin problemas con el sitio de administración de Django.

django-parler genera una tabla de base de datos separada para cada modelo que contiene traducciones. Esta tabla incluye todos los campos traducidos y una clave externa para el objeto original al que pertenece la traducción. También contiene un campo de idioma, ya que cada fila almacena el contenido de un solo idioma.

Instalación de django-parler

Instale django-parler a través de pip usando el siguiente comando:

pip instalar django-parler == 2.0.1

Edite el archivo settings.py de su proyecto y agregue 'parler' a INSTALLED_APPS ajuste, como sigue:

```
INSTALLED_APPS = [  
    # ...  
    'parler',  
]
```

Además, agregue el siguiente código a su configuración:

```
PARLER_LANGUAGES = {  
    Ninguna: (  
        {'código': 'en'},  
        {'código': 'es'},  
    ),  
    'defecto': {  
        'fallback': 'en',  
        'hide_untranslated': Falso,  
    }  
}
```

Esta configuración define los idiomas disponibles, en y es , para django-parler . Tú especificar el idioma predeterminado en e indicar que django-parler no debe ocultarse contenido no traducido.

Traducir campos del modelo

Agreguemos traducciones para su catálogo de productos. django-parler proporciona una clase de modelo TranslatableModel y un contenedor TranslatedFields para traducir campos modelo. Edite el archivo models.py dentro del directorio de la aplicación de la tienda y agregue la siguiente importación:

```
de parler.models import TranslatableModel, TranslatedFields
```

Luego, modifique el modelo de Categoría para hacer que los campos de nombre y slug sean traducibles, como sigue:

```
class Categoría ( TranslatableModel ):
    traducciones = TranslatedFields (
        name = models.CharField (max_length = 200,
                                db_index = Verdadero),
        slug = models.SlugField (max_length = 200,
                                db_index = Verdadero,
                                único = verdadero)
    )
```

El modelo de categoría ahora hereda de TranslatableModel en lugar de modelos.

Model y tanto el nombre como los campos slug se incluyen en TranslatedFields envoltura.

Edite el modelo de producto para agregar traducciones para el nombre , slug y descripción campos, como sigue:

```
clase Producto ( TranslatableModel ):
    traducciones = TranslatedFields (
        name = models.CharField (max_length = 200, db_index = True),
        slug = models.SlugField (max_length = 200, db_index = True),
        descripción = modelos.TextField (en blanco = verdadero)
    )
    categoría = modelos.ForeignKey (Categoría,
                                related_name = 'productos')
    image = modelos.ImageField (upload_to = 'productos /% Y /% m /% d',
                                en blanco = Verdadero)
    precio = modelos.DecimalField (max_digits = 10, decimal_places = 2)
    disponible = modelos.BooleanField (predeterminado = Verdadero)
    creado = modelos.DateTimeField (auto_now_add = True)
    actualizado = modelos.DateTimeField (auto_now = True)
```

[337]

Ampliando su tienda

django-parler gestiona las traducciones generando otro modelo para cada modelo traducible. En el siguiente esquema, puede ver los campos del Producto model y cómo se verá el modelo ProductTranslation generado :

una `ProductTranslation` para cada objeto traducible en cada idioma. El código se basa en los a partir del producto de `ProductTranslation`. A `ProductTranslation` existirá objeto para cada idioma disponible de cada objeto Producto.

Dado que Django usa una tabla separada para las traducciones, hay algunas características de Django que no puedes usar. No es posible utilizar un orden predeterminado por un campo traducido. Puede filtrar por campos traducidos en consultas, pero no puede incluir un traducible campo en las opciones de meta de pedidos.

Edite el archivo `models.py` de la aplicación de la tienda y comente el pedido atributo de la categoría Meta clase:

```
class Categoría(TranslatableModel):
    # ...
    class Meta:
        # ordering = ('nombre',)
        verbose_name = 'categoría'
        verbose_name_plural = 'categorías'
```

También debe comentar los atributos `ordering` e `index_together` del

Meta clase de producto. La versión actual de `django-parler` no proporciona soporte para validar `index_together`. Comente la clase `Product Meta`, de la siguiente manera:

```
class Producto(TranslatableModel):
    # ...
```

[338]

```
# clase Meta:
# ordering = ('-name',)
# index_together = (('id', 'slug'),)
```

Puede leer más sobre la compatibilidad del módulo `django-parler` con Django en <https://django-parler.readthedocs.io/en/latest/compatibility.html>.

Integración de traducciones en el sitio de administración

`django-parler` se integra sin problemas con el sitio de administración de Django. Incluye una clase `TranslatableAdmin` que anula la clase `ModelAdmin` proporcionada por Django para gestionar las traducciones de modelos.

Edite el archivo `admin.py` de la aplicación de la tienda y agregue la siguiente importación:

```
de parler.admin importar TranslatableAdmin
```

Modifique las clases `CategoryAdmin` y `ProductAdmin` para heredar

`TranslatableAdmin` en lugar de `ModelAdmin`. `django-parler` no es compatible con atributo `prepopulated_fields`, pero admite el atributo `get_prepopulated_fields()` método que proporciona la misma funcionalidad. Vamos a cambiar esto en consecuencia. Edite el archivo `admin.py` para que tenga el siguiente aspecto:

```
desde django.contrib importar admin
de parler.admin importar TranslatableAdmin
de .models importar Categoría, Producto

@ admin.register (Categoría)
class CategoríaAdmin ( TranslatableAdmin ):
    list_display = ['nombre', 'slug']

    def get_prepopulated_fields(self, request, obj = None):
        return {'slug': ('nombre',)}

@ admin.register (Producto)
class ProductAdmin ( TranslatableAdmin ):
```

```
list_display = ['nombre', 'slug', 'precio',
               'disponible', 'creado', 'actualizado']
list_filter = ['disponible', 'creado', 'actualizado']
list_editable = ['precio', 'disponible']
```

```
def get_prepopulated_fields(self, request, obj = None):
    return {'slug': ('nombre',)}
```

Ha adaptado el sitio de administración para que funcione con los nuevos modelos traducidos. Ahora puede sincronizar la base de datos con los cambios de modelo que realizó.

[339]

Ampliando su tienda

Crear migraciones para traducciones de modelos

Abra el shell y ejecute el siguiente comando para crear una nueva migración para el traducciones modelo:

```
python manage.py makemigrations shop --nombre "traducciones"
```

Verá el siguiente resultado:

Migraciones para 'tienda':

```
shop / migrations / 0002_translations.py
- Cambiar las opciones de Meta en la categoría
- Cambiar las opciones de Meta en el producto
- Eliminar el nombre del campo de la categoría
- Eliminar babosa de campo de la categoría
- Modificar index_together para el producto (0 restricción (s))
- Eliminar la descripción del campo del producto
- Eliminar el nombre de campo del producto
- Eliminar la babosa de campo del producto
- Crear modelo ProductTranslation
- Crear modelo CategoryTranslation
```

Esta migración incluye automáticamente la traducción de categorías y

Modelos de ProductTranslation creados dinámicamente por django-parler . Es importante para tener en cuenta que esta migración elimina los campos existentes anteriores de sus modelos. Esta significa que perderá esos datos y tendrá que configurar sus categorías y productos nuevamente en el sitio de administración después de ejecutarlo.

Edite el archivo migrations / 0002_translations.py de la aplicación de la tienda y reemplace las dos apariciones de la siguiente línea:

```
bases = (parler.models.TranslatedFieldsModelMixin, modelos.Model),
```

con el siguiente:

```
bases = (parler.models.TranslatableModel, modelos.Model),
```

Esta es una solución para un problema menor que se encuentra en la versión de django-parler que está utilizando.

Este cambio es necesario para evitar que la migración falle al aplicarlo.

Este problema está relacionado con la creación de traducciones para campos existentes en el modelo y probablemente se solucionará en las versiones más recientes de django-parler .

Ejecute el siguiente comando para aplicar la migración:

```
python manage.py migrar tienda
```

[340]

Verá una salida que termina con la siguiente línea:

Aplicando shop.0002_translations ... OK

Sus modelos ahora están sincronizados con la base de datos.

Ejecute el servidor de desarrollo usando `python manage.py runserver` y abra

`http://127.0.0.1:8000/en/admin/shop/category/` en su navegador. Vas a ver que las categorías existentes perdieron su nombre y slug debido a la eliminación de esos campos y usando los modelos traducibles generados por `django-parler` en su lugar. Hacer clic en una categoría para editarla. Verá que la página **Cambiar categoría** incluye dos diferentes pestañas, una para las traducciones al inglés y otra al español:

Figura 9.10: El formulario de edición de categorías, incluidas las pestañas de idioma agregadas por `django-parler`

Asegúrese de ingresar un nombre y slug para todas las categorías existentes. Además, agregue una traducción al español para cada uno de ellos y haga clic en el botón **GUARDAR**. Asegurarse que guarde los cambios antes de cambiar de pestaña o los perderá.

Después de completar los datos de las categorías existentes, abra `http://127.0.0.1:8000/`

`es / admin / shop / product /` y edita cada uno de los productos, proporcionando un inglés y Nombre español, una babosa y una descripción.

Adaptación de vistas para traducciones

Tienes que adaptar las vistas de tu tienda para utilizar QuerySets de traducción. Ejecute lo siguiente comando para abrir el shell de Python:

shell de python manage.py

[341]

Echemos un vistazo a cómo puede recuperar y consultar campos de traducción. Para obtener el

objeto con campos traducibles traducidos a un idioma específico, puede usar el método de Django `activar()` función, de la siguiente manera:

```
>>> de shop.models import Product
>>> de django.utils.translation importar activar
>>> activar ('es')
>>> producto = Product.objects.first ()
>>> nombre.producto
'Té verde'
```

Otra forma de hacer esto es usando el administrador de language () proporcionado por django-parler, como sigue:

```
>>> producto = Product.objects.language ('en'). first ()
>>> nombre.producto
'Té verde'
```

Cuando accede a los campos traducidos, se resuelven en el idioma actual. Puede establecer un idioma actual diferente para que un objeto acceda a esa traducción, como sigue:

```
>>> product.set_current_language ('es')
>>> nombre.producto
'Té verde'
>>> product.get_current_language ()
'es'
```

Al realizar un QuerySet usando `filter()`, puede filtrar usando los objetos de traducción con la sintaxis `traducciones__`, como sigue:

```
>>> Product.objects.filter (Translations__name = 'Té verde')
<TranslatableQuerySet [<Producto: Té verde>]>
```

Adaptamos las vistas del catálogo de productos. Edite el archivo `views.py` de la aplicación de la tienda y, en el `product_list` vista, encontrar la siguiente línea:

```
categoria = get_object_or_404 (Categoría, slug = category_slug)
```

[342]

Reemplácelo con los siguientes:

```
language = request.LANGUAGE_CODE
categoria = get_object_or_404 (Categoría,
                             traducciones__language_code = idioma,
                             traducciones__slug = category_slug )
```

Luego, edite la vista `product_detail` y busque las siguientes líneas:

```
product = get_object_or_404 (Producto,
                             id = id,
                             babosa = babosa,
                             disponible = Verdadero)
```

Reemplácelos con el siguiente código:

```
language = request.LANGUAGE_CODE
product = get_object_or_404 (Producto,
```

```
id = id,  
traducciones__language_code = idioma,  
traducciones__slug = slug,  
disponible = Verdadero)
```

Las vistas `product_list` y `product_detail` ahora están adaptadas para recuperar objetos que utilizan campos traducidos. Ejecute el servidor de desarrollo y abra `http://127.0.0.1:8000/es/` en su navegador. Deberías ver el producto página de lista, que incluye todos los productos traducidos al español:

Figura 9.11: La versión en español de la página de lista de productos

[343]

Ampliando su tienda

Ahora, la URL de cada producto se crea utilizando el campo `slug` traducido a el idioma actual. Por ejemplo, la URL de un producto en español es `http://127.0.0.1:8000/es/2/te-rojo/`, mientras que en inglés, la URL es `http://127.0.0.1:8000/en/2/red-tea/`. Si navega a un detalle de producto página, verá la URL traducida y el contenido del idioma seleccionado, como se muestra en el siguiente ejemplo:

Figura 9.12: La versión en español de la página de detalles del producto

Si desea saber más sobre `django-parler`, puede encontrar el documentación en <https://django-parler.readthedocs.io/en/latest/>.

Ha aprendido a traducir código, plantillas, patrones de URL y modelos de Python campos. Para completar el proceso de internacionalización y localización, debe utilizar formato localizado para fechas, horas y números.

Localización de formato

Según la configuración regional del usuario, es posible que desee mostrar fechas, horas y números en diferentes formatos. El formateo localizado se puede activar cambiando USE_L10N en True en el archivo settings.py de su proyecto.

Cuando USE_L10N está habilitado, Django intentará usar un formato específico de la configuración regional siempre que genera un valor en una plantilla. Puedes ver que los números decimales en inglés la versión de su sitio se muestran con un separador de puntos para lugares decimales, mientras que en la versión en español, se muestran con una coma. Esto se debe a la formatos de configuración regional especificados para la configuración regional es por Django. Puedes echar un vistazo a Configuración de formato en español en <https://github.com/django/django/blob/master/django/conf/locale/es/formats.py>.

[344]

Normalmente, establecerá la configuración USE_L10N en True y dejará que Django aplique el formato localización para cada localidad. Sin embargo, puede haber situaciones en las que no desea utilizar valores localizados. Esto es especialmente relevante cuando se genera JavaScript, o JSON que debe proporcionar un formato legible por máquina.

Django ofrece una etiqueta de plantilla {% localize%} que le permite activar / desactivar localización de fragmentos de plantilla. Esto le da control sobre el formato localizado. Deberá cargar las etiquetas l10n para poder utilizar esta etiqueta de plantilla. El seguimiento es un ejemplo de cómo activar y desactivar la localización en una plantilla:

```
{% load l10n%}

{% localizar en%}
{{valor}}
{% endlocalize%}

{% localize off%}
{{valor}}
{% endlocalize%}
```

Django también ofrece los filtros de plantilla para localizar y deslocalizar para forzar o evitar la localización de un valor. Estos filtros se pueden aplicar de la siguiente manera:

```
{{valor | localizar}}
{{valor | deslocalizar}}
```

También puede crear archivos de formato personalizado para especificar el formato local. Puedes encontrar más información sobre la localización de formatos en <https://docs.djangoproject.com/es/3.0/topics/i18n/formatting/>.

Usando django-localflavor para validar el formulario campos

django-localflavor es un módulo de terceros que contiene una colección de utilidades, como como campos de formulario o campos modelo, que son específicos para cada país. Es muy útil para validación de regiones locales, números de teléfono locales, números de tarjetas de identidad, seguridad social números y así sucesivamente. El paquete está organizado en una serie de módulos que llevan el nombre Códigos de país ISO 3166.

Instale django-localflavor usando el siguiente comando:

```
pip install django-localflavor == 3.0.1
```

Ampliando su tienda

Edite el archivo settings.py de su proyecto y agregue localflavor a INSTALLED_APPS, de la siguiente manera:

```
INSTALLED_APPS = [
    # ...
    'sabor local',
]
```

Va a agregar el campo del código postal de los Estados Unidos para que un país válido se requiera el código postal para crear un nuevo pedido.

Edite el archivo forms.py de la aplicación de pedidos y haga que tenga el siguiente aspecto:

```
desde formularios de importación de django
de localflavor.us.forms importar USZipCodeField
de .models import Order

class OrderCreateForm (formularios.ModelForm):
    postal_code = USZipCodeField ()
    class Meta:
        modelo = Orden
        fields = ('first_name', 'last_name', 'email', 'address',
                  'Código postal de la ciudad')
```

Importar el USZipCodeField campo de la con nosotros paquete de localflavor y el uso que para el postal_code campo de la OrderCreateForm formulario.

Ejecute el servidor de desarrollo y abra <http://127.0.0.1:8000/en/orders/> crear / en su navegador. Complete todos los campos, ingrese un código postal de tres letras y luego envíe el formulario. Obtendrá el siguiente error de validación que se genera por USZipCodeField :

Ingrese un código postal en el formato XXXXX o XXXXX-XXXX.

Este es solo un breve ejemplo de cómo usar un campo personalizado de localflavor en su propio proyecto con fines de validación. Los componentes locales proporcionados por localflavor son muy útiles para adaptar su aplicación a países específicos. Puede leer la documentación de django-localflavor y ver todos los locales disponibles componentes para cada país en <https://django-localflavor.readthedocs.io/es/ultimo/>.

A continuación, va a crear un motor de recomendaciones en su tienda.

Construyendo un motor de recomendaciones

Un motor de recomendaciones es un sistema que predice la preferencia o calificación que un usuario le daría a un artículo. El sistema selecciona elementos relevantes para un usuario en función de su comportamiento y el conocimiento que tiene sobre ellos. Hoy en día, recomendación Los sistemas se utilizan en muchos servicios en línea. Ayudan a los usuarios seleccionando las cosas podrían estar interesados en la gran cantidad de datos disponibles que son irrelevantes a ellos. Ofrecer buenas recomendaciones mejora la participación de los usuarios. Comercio electrónico Los sitios también se benefician de ofrecer recomendaciones de productos relevantes al aumentar sus ingresos medios por usuario.

Va a crear un motor de recomendaciones simple pero poderoso que sugiere productos que normalmente se compran juntos. Sugerirás productos basado en ventas históricas, identificando así los productos que generalmente se compran juntos. Propondrá productos complementarios en dos escenarios diferentes:

- **Página de detalles del producto** : mostrará una lista de productos que normalmente comprado con el producto dado. Esto se mostrará como *usuarios que compraron esto también compraron X, Y, Z* . Necesita una estructura de datos que le permita almacenar el número de veces que se ha comprado cada producto junto con el producto que se muestra.
- **Página de detalles del carrito** : según los productos que los usuarios agreguen al carrito, para sugerir productos que se suelen comprar junto con estos. En esto En este caso, la puntuación que calcule para obtener productos relacionados debe agregarse.

Va a utilizar Redis para almacenar productos que se compran juntos. Recuerda que ya usó Redis en el *Capítulo 6* , *Seguimiento de las acciones del usuario* . Si no lo has hecho instalado Redis, puede encontrar las instrucciones de instalación en ese capítulo.

Recomendar productos basados en anteriores compras

Recomendará productos a los usuarios en función de lo que hayan agregado al carrito. Va a almacenar una clave en Redis para cada producto comprado en su sitio. los La clave de producto contendrá un conjunto ordenado de Redis con puntuaciones. Incrementarás la puntuación en 1 por cada producto comprado en conjunto cada vez que se completa una nueva compra. los El conjunto ordenado le permitirá dar puntajes a los productos que se compran juntos.

Recuerde instalar redis-py en su entorno usando el siguiente comando:

```
pip install redis == 3.4.1
```

[347]

Ampliando su tienda

Edite el archivo settings.py de su proyecto y agregue la siguiente configuración:

```
REDIS_HOST = 'localhost'
REDIS_PORT = 6379
REDIS_DB = 1
```

Éstas son las configuraciones necesarias para establecer una conexión con el servidor Redis. Crear un nuevo archivo dentro del directorio de la aplicación de la tienda y asígnele el nombre recomender.py . Añadir el siguiente código:

```
importar redis
desde la configuración de importación django.conf
desde .models Importar Producto
```

```
# conectarse a redis
r = redis.Redis (host = settings.REDIS_HOST,
                 port = settings.REDIS_PORT,
                 db = settings.REDIS_DB)
```

recomendación de clase (objeto):

```
def get_product_key (self, id):
    return f'product: {id}: purchase_with '

def productos_comprados (uno mismo, productos):
    product_ids = [p.id para p en productos]
    para product_id en product_ids:
        para with_id en product_ids:
            # obtener los otros productos comprados con cada producto
            si product_id != with_id:
                # puntaje de incremento para productos comprados juntos
                r.zincrby (self.get_product_key (product_id),
                          1,
                          with_id)
```

Esta es la clase de recomendación que le permitirá almacenar compras de productos y recuperar sugerencias de productos para un producto o productos determinados.

El método `get_product_key ()` recibe un ID de un objeto de producto y crea la clave de Redis para el conjunto ordenado donde se almacenan los productos relacionados, que parece `producto: [id]: comprado_con`.

El método `productos_bought ()` recibe una lista de objetos `Product` que han sido comprados juntos (es decir, pertenecen al mismo pedido).

[348]

En este método, realiza las siguientes tareas:

1. Obtiene los ID de producto para los objetos de `Producto` dados.
2. Se itera sobre los ID de producto. Para cada ID, vuelve a iterar sobre el ID de producto y omite el mismo producto para obtener los productos que se compran junto con cada producto.
3. Obtiene la clave de producto de Redis para cada producto comprado usando `get_product_id ()`. Para un producto con un ID de 33, este método devuelve el producto clave : 33: comprado_con. Esta es la clave para el conjunto ordenado que contiene los ID de producto de los productos que se compraron junto con este.
4. Aumenta la puntuación de cada ID de producto incluido en el conjunto ordenado por 1. La puntuación representa las veces que se ha comprado otro producto junto con el producto dado.

Ahora tiene un método para almacenar y calificar los productos que se compraron juntos.

A continuación, necesita un método para recuperar los productos que se compraron juntos durante una lista de productos dados. Agregue el siguiente método `sugerido_productos_para ()` al

Clase de recomendación :

```
def sugerir_productos_para (uno mismo, productos, max_results = 6):
    product_ids = [p.id para p en productos]
    if len (productos) == 1:
        # solo 1 producto
        sugerencias = r.zrange (
            self.get_product_key (product_ids [0]),
            0, -1, desc = True) [: max_results]
    más:
        # generar una clave temporal
```

```

flat_ids = " ".join([str(id) para id en product_ids])
tmp_key = f'tmp_{flat_ids}'
# productos múltiples, combina puntuaciones de todos los productos
# almacenar el conjunto ordenado resultante en una clave temporal
keys = [self.get_product_key(id) para id en product_ids]
r.zunionstore(tmp_key, claves)
# eliminar los ID de los productos para los que se recomienda
r.zrem(tmp_key, *product_ids)
# obtener los ID del producto por su puntuación, clasificación descendiente
sugerencias = r.zrange(tmp_key, 0, -1,
                        desc = True)[:max_results]

# quitar la llave temporal
r.delete(tmp_key)
sugeridos_productos_ids = [int(id) para id en sugerencias]

```

[349]

Ampliando su tienda

```

# obtener productos sugeridos y ordenarlos por orden de aparición
sugeridos_productos = lista(Producto.objetos.filtro(id__in = sugerido_
productos_id))
sugeridos_productos.sort(clave = lambda x: sugeridos_productos_ids.
índice(x.id))
devolver productos_sugeridos

```

El método `sugerido_productos_for()` recibe los siguientes parámetros:

- **productos** : esta es una lista de objetos `Producto` para los que se pueden obtener recomendaciones. Puede contener uno o más productos.
- **max_results** : este es un número entero que representa el número máximo de recomendaciones para volver.

En este método, realiza las siguientes acciones:

1. Obtiene los ID de producto para los objetos de `Producto` dados .
2. Si solo se proporciona un producto, recupera el ID de los productos que fueron comprado junto con el producto dado, pedido por el número total de veces que fueron comprados juntos. Para hacerlo, usa el comando `ZRANGE` de Redis . Limita el número de resultados al número especificado en `max_results` atributo (6 por defecto).
3. Si se proporciona más de un producto, genera una clave Redis temporal construida con los ID de los productos.
4. Combina y suma todas las puntuaciones de los elementos incluidos en el conjunto ordenado de cada uno de los productos dados. Esto se hace usando Redis `ZUNIONSTORE` comando. El comando `ZUNIONSTORE` realiza una unión de los conjuntos ordenados con las claves dadas y almacena la suma agregada de puntuaciones de los elementos en una nueva clave de Redis. Puede leer más sobre este comando en <https://redis.io/commands/ZUNIONSTORE>. Guarde las puntuaciones agregadas en el clave temporal.
5. Como está agregando puntajes, puede obtener los mismos productos que están recibiendo recomendaciones para. Los elimina del generado conjunto ordenado usando el comando `ZREM` .
6. Recupera los ID de los productos de la clave temporal, ordenados por su puntuación utilizando el comando `ZRANGE` . Limita el número de resultados a el número especificado en el atributo `max_results` . Luego, quita el clave temporal.
7. Por último, obtiene los objetos `Producto` con los ID dados y ordena el productos en el mismo orden que ellos.

Para fines prácticos, agreguemos también un método para borrar las recomendaciones. Añadir el siguiente método a la clase de recomendación :

```
def clear_purchases (auto):
    para id en Product.objects.values_list ('id', flat = True):
        r.delete (self.get_product_key (id))
```

Probemos su motor de recomendaciones. Asegúrese de incluir varios productos objetos en la base de datos e inicialice el servidor Redis usando el siguiente comando desde el shell en su directorio de Redis:

src / redis-servidor

Abra otro shell y ejecute el siguiente comando para abrir el shell de Python:

shell de python manage.py

Asegúrese de tener al menos cuatro productos diferentes en su base de datos. Recuperar cuatro productos diferentes por sus nombres:

```
>>> de shop.models import Product
>>> black_tea = Product.objects.get (traducciones__name = 'Té negro')
>>> red_tea = Product.objects.get (traducciones__name = 'Té rojo')
>>> green_tea = Product.objects.get (Translations__name = 'Té verde')
>>> tea_powder = Product.objects.get (Translations__name = 'Té en polvo')
```

Luego, agregue algunas compras de prueba al motor de recomendaciones:

```
>>> de shop.recommender importar recomendación
>>> r = Recomendador ()
>>> r.products_bought ((té_negro, té_rojo))
>>> r.products_bought ((té_negro, té_verde))
>>> r.products_bought ((té_rojo, té_negro, polvo_té))
>>> r.products_bought ((té_verde, polvo_té))
>>> r.products_bought ((té_negro, polvo_té))
>>> r.products_bought ((red_tea, green_tea))
```

Ha almacenado las siguientes puntuaciones:

```
té_negro: té_rojo (2), té_polvo (2), té_verde (1)
té_rojo: té_negro (2), té_polvo (1), té_verde (1)
té_verde: té_negro (1), té_polvo (1), té_rojo (1)
té_polvo: té_negro (2), té_rojo (1), té_verde (1)
```



```

        disponible = Verdadero)
    cart_product_form = CartAddProductForm ()

    r = Recomendador ()
    recomendados_productos = r.suggest_products_for ([producto], 4)

    return render (solicitud,
                    'shop / product / detail.html',
                    {'producto': producto,
                     'cart_product_form': cart_product_form,
                     'productos_recomendados': productos_recomendados })

```

Edite la plantilla `shop / product / detail.html` de la aplicación de la tienda y agregue el siguiente código después de `{{product.description | linebreaks}}` :

```

{% if recommended_products%}
<div class = "recomendaciones">
  <h3> {% trans "Las personas que compraron esto también compraron"% </h3>
  {% for p in recommended_products%}
    <div class = "elemento">
      <a href="{{ p.get_absolute_url }}">
        <img src = "{% if p.image%} {{p.image.url}} {% else%}
          {% static "img / no_image.png"%} {% endif%} ">
        </a>
        <p> <a href="{{ p.get_absolute_url }}"> {{p.name}} </a> </p>
      </div>
    {% endfor%}
  </div>
{% terminara si %}

```

[353]

Ampliando su tienda

Ejecute el servidor de desarrollo y abra `http://127.0.0.1:8000/en/` en su navegador. Haga clic en cualquier producto para ver sus detalles. Deberías ver lo recomendado. Los productos se muestran debajo del producto, como se muestra en la siguiente captura de pantalla:

También incluirá recomendaciones de productos en el carrito. Los
Las recomendaciones se basarán en los productos que el usuario haya añadido al carrito.

Edite views.py dentro de la aplicación del carrito, importe la clase de recomendación y edite
la vista cart_detail para que tenga el siguiente aspecto:

de shop.recommender importar recomendación

```
def cart_detail (solicitud):  
    cart = Carrito (solicitud)  
    para el artículo en el carrito:  
        item ['update_quantity_form'] = CartAddProductForm (inicial = {
```

[354]

```
'cantidad': artículo ['cantidad'],  
'anular': Verdadero))
```

```
coupon_apply_form = CouponApplyForm ()
```

```
r = Recomendador ()
```

```
cart_products = [artículo ['producto']] para el artículo en el carrito]
```

```
recomendados_productos = r.suggest_products_for (cart_products,
```

```
max_results = 4)
```

```
return render (solicitud,  
                'carrito / detalle.html',  
                {'carrito': carrito,  
                 'coupon_apply_form': coupon_apply_form,  
                 'productos_recomendados': productos_recomendados })
```

Edite la plantilla cart / detail.html de la aplicación del carrito y agregue lo siguiente
código justo después de la etiqueta HTML </table> :

```
{% if recommended_products%}  
<div class = "carrito de recomendaciones">  
    <h3> {% trans "Las personas que compraron esto también compraron"%} </h3>  
    {% for p in recommended_products%}  
        <div class = "elemento">  
            <a href="{{ p.get_absolute_url }}">  
                <img src = "{% if p.image%} {{p.image.url}} {% else%}  
                    {% static "img / no_image.png"%} {% endif%}" >  
            </a>  
            <p> <a href="{{ p.get_absolute_url }}"> {{p.name}} </a> </p>  
        </div>  
    {% endfor%}  
</div>  
{% terminara si %}
```

Abra <http://127.0.0.1:8000/en/> en su navegador y agregue un par de productos
a su carrito. Cuando navegue a <http://127.0.0.1:8000/en/cart/>, debe
consulte las recomendaciones de productos agregadas para los artículos del carrito, de la siguiente manera:

Ampliando su tienda

Figura 9.14: La página de detalles del carrito de compras, incluidos los productos recomendados

¡Felicidades! Ha creado un motor de recomendación completo usando Django y Redis.

Resumen

En este capítulo, creó un sistema de cupones utilizando sesiones. También aprendiste el conceptos básicos de internacionalización y localización para proyectos Django. Usted marcó cadenas de código y plantilla para la traducción, y descubrió cómo generar y compilar archivos de traducción. También instaló Rosetta en su proyecto para administrar traducciones a través de una interfaz de navegador. Tradujo patrones de URL y creó un selector de idioma para permitir a los usuarios cambiar el idioma del sitio. Luego, que utilizó Django-parler de traducir y modelos que utilizó Django-localflavor para validar campos de formulario localizados. Finalmente, creaste un motor de recomendaciones usando Redis para recomendar productos que generalmente se compran juntos.

En el próximo capítulo, comenzará un nuevo proyecto. Construirás un e-learning plataforma con Django usando vistas basadas en clases y creará una sistema de gestión de contenidos.

10

Construyendo un Plataforma de aprendizaje electrónico

En el capítulo anterior, agregó la internacionalización a su proyecto de tienda en línea. También creó un sistema de cupones utilizando sesiones y una recomendación de producto. motor usando Redis. En este capítulo, iniciará un nuevo proyecto de Django. Vas a crear una plataforma de aprendizaje electrónico con su propio **sistema de gestión de contenido (CMS)**. Las plataformas de aprendizaje en línea son un gran ejemplo de aplicaciones en las que necesita proporcionar herramientas para generar contenido teniendo en cuenta la flexibilidad. En este capítulo, aprenderá a construir la funcionalidad para que los instructores creen cursos y administren el contenido de los cursos de una manera versátil y eficiente.

En este capítulo, aprenderá a:

- Crear accesorios para tus modelos
- Usar herencia de modelos
- Cree campos de modelo personalizados
- Utilice vistas y mixins basados en clases
- Crear conjuntos de formularios
- Administrar grupos y permisos
- Crear un CMS

[357]

Construyendo una plataforma de aprendizaje electrónico

Configuración del proyecto de e-learning

Tu proyecto práctico final será una plataforma de e-learning. Primero, crea un virtual entorno para su nuevo proyecto y actívalo con los siguientes comandos:

```
mkdir env
python3 -m venv env / educa
```

```
source env / educa / bin / enable
```

Instale Django en su entorno virtual con el siguiente comando:

```
pip instalar Django == 3.0.*
```

Vas a administrar la carga de imágenes en tu proyecto, por lo que también necesitas instalar Almohada con el siguiente comando:

```
pip install Pillow == 7.0.0
```

Crea un nuevo proyecto usando el siguiente comando:

```
django-admin startproject educa
```

Ingresa al nuevo directorio educa y cree una nueva aplicación usando lo siguiente comandos:

```
cd educa
```

```
cursos de django-admin startapp
```

Edite el archivo settings.py del proyecto educa y agregue cursos al archivo INSTALLED_

Configuración de APPS , de la siguiente manera:

```
INSTALLED_APPS = [  
    'cursos.apps.CursosConfig',  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
]
```

La aplicación de cursos ahora está activa para el proyecto. Definamos los modelos para cursos y contenidos del curso.

[358]

Construyendo los modelos del curso

Su plataforma de e-learning ofrecerá cursos sobre diversos temas. Cada curso dividirse en un número configurable de módulos, y cada módulo contendrá un número configurable de contenidos. Los contenidos serán de varios tipos: texto, archivo, imagen o video. El siguiente ejemplo muestra cuál es la estructura de datos de su El catálogo de cursos se verá así:

```
Asunto 1  
  Curso 1  
    Módulo 1  
      Contenido 1 (imagen)  
      Contenido 2 (texto)  
    Módulo 2  
      Contenido 3 (texto)  
      Contenido 4 (archivo)  
      Contenido 5 (video)  
    ...
```

Construyamos los modelos del curso. Edite el archivo models.py de la aplicación de cursos y agregue el siguiente código:

```
desde modelos de importación django.db
```

```

desde django.contrib.auth.models import User

class Asunto (modelos.Modelo):
    title = models.CharField (longitud_máxima = 200)
    slug = models.SlugField (max_length = 200, unique = True)

    class Meta:
        ordering = ['titulo']

    def __str__ (yo):
        volver self.title

class Curso (modelos.Modelo):
    propietario = modelos.ForeignKey (Usuario,
                                      related_name = 'cursos_creados',
                                      on_delete = modelos.CASCADE)

    asunto = modelos.ForeignKey (Asunto,
                                related_name = 'cursos',
                                on_delete = modelos.CASCADE)

    title = models.CharField (longitud_máxima = 200)
    slug = models.SlugField (max_length = 200, unique = True)
    visión general = modelos.TextField ()

```

[359]

Construyendo una plataforma de aprendizaje electrónico

```

creado = modelos.DateTimeField (auto_now_add = True)

class Meta:
    ordering = ['-created']

def __str__ (yo):
    volver self.title

Módulo de clase (modelos.Modelo):
    curso = modelos.ForeignKey (Curso,
                                related_name = 'módulos',
                                on_delete = modelos.CASCADE)

    title = models.CharField (longitud_máxima = 200)
    descripción = modelos.TextField (en blanco = verdadero)

    def __str__ (yo):
        volver self.title

```

Estos son los modelos iniciales de Materia , Curso y Módulo . Los campos del modelo del curso son como sigue:

- propietario : el instructor que creó este curso.
- asignatura : la asignatura a la que pertenece este curso. Es un campo ForeignKey que apunta al modelo Subject .
- título : el título del curso.
- babosa : La babosa del campo. Esto se utilizará en las URL más adelante.
- descripción general : una columna TextField para almacenar una descripción general del curso.
- creado : la fecha y la hora en que se creó el curso. Será configurado automáticamente por Django al crear nuevos objetos debido a auto_now_agregar = Verdadero .

Cada curso se divide en varios módulos. Por tanto, el modelo Module contiene un campo ForeignKey que apunta al modelo del curso .

Abra el shell y ejecute el siguiente comando para crear la migración inicial para este solicitud:

```
python manage.py makemigrations
```

Verá el siguiente resultado:

Migraciones para 'cursos':

```
cursos / migraciones / 0001_initial.py:
```

[360]

- Crear curso modelo
- Crear módulo modelo
- Crear modelo Asunto
- Agregar tema de campo al curso

Luego, ejecute el siguiente comando para aplicar todas las migraciones a la base de datos:

```
python manage.py migrar
```

Debería ver un resultado que incluye todas las migraciones aplicadas, incluidas las de Django. La salida contendrá la siguiente línea:

```
Aplicando cursos.0001_initial ... OK
```

Los modelos de su aplicación de cursos se han sincronizado con la base de datos.

Registro de modelos en la administración sitio

Agreguemos los modelos del curso al sitio de administración. Edite el archivo admin.py dentro el directorio de la aplicación de cursos y agregue el siguiente código:

```
desde django.contrib import admin
desde .models importar Asunto, Curso, Módulo

@ admin.register (Asunto)
clase SubjectAdmin (admin.ModelAdmin):
    list_display = ['título', 'slug']
    campos_prepopulados = {'slug': ('título',)}

clase ModuleInline (admin.StackedInline):
    modelo = Módulo

@ admin.register (curso)
clase CourseAdmin (admin.ModelAdmin):
    list_display = ['título', 'asunto', 'creado']
    list_filter = ['creado', 'asunto']
    search_fields = ['título', 'descripción general']
    campos_prepopulados = {'slug': ('título',)}
    inlines = [ModuleInline]
```

Los modelos para la aplicación del curso ahora están registrados en el sitio de administración. Recuerde que usa el decorador `@ admin.register ()` para registrar modelos en el sitio de administración.

[361]

Usando accesorios para proporcionar datos iniciales para modelos

A veces, es posible que desee rellenar previamente su base de datos con datos codificados. Esto es útil para incluir automáticamente datos iniciales en la configuración del proyecto, en lugar de tener que agregarlos manualmente. Django viene con una forma sencilla de cargar y volcar datos de la base de datos en archivos que se denominan **accesorios** . Django admite accesorios en JSON, Formatos XML o YAML. Va a crear un accesorio para incluir varios Objetos sujetos para su proyecto.

Primero, cree un superusuario usando el siguiente comando:

```
python manage.py crea superusuario
```

Luego, ejecute el servidor de desarrollo con el siguiente comando:

```
python manage.py runserver
```

Abra `http://127.0.0.1:8000/admin/courses/subject/` en su navegador. Cree varios temas utilizando el sitio de administración. La página de visualización de la lista debe mirar como sigue:

Figura 10.1: Vista de la lista de cambios de asunto

Ejecute el siguiente comando desde el shell:

```
Python manage.py dumpdata cursos --indent = 2
```

Verá un resultado similar al siguiente:

```
[
{
  "modelo": "cursos.subject",
```

[362]

```
"pk": 1,
"campos": {
  "title": "Matemáticas",
```

```

        "slug": "matemáticas"
    }
},
{
    "modelo": "cursos.subject",
    "pk": 2,
    "campos": {
        "title": "Música",
        "slug": "música"
    }
},
{
    "modelo": "cursos.subject",
    "pk": 3,
    "campos": {
        "title": "Física",
        "slug": "física"
    }
},
{
    "modelo": "cursos.subject",
    "pk": 4,
    "campos": {
        "title": "Programación",
        "slug": "programación"
    }
}
}
]

```

El comando `dumpdata` vuelca los datos de la base de datos en el estándar salida, serializada en formato JSON por defecto. La estructura de datos resultante incluye información sobre el modelo y sus campos para que Django pueda para cargarlo en la base de datos.

[363]

Construyendo una plataforma de aprendizaje electrónico

Puede limitar la salida a los modelos de una aplicación proporcionando la aplicación nombres al comando, o especificando modelos únicos para la salida de datos usando el formato de modelo de aplicación . También puede especificar el formato usando la marca `--format` . Por por defecto, `dumpdata` envía los datos serializados a la salida estándar. Sin embargo, tu puede indicar un archivo de salida usando el indicador `--output` . La bandera `--indent` le permite para especificar la sangría. Para obtener más información sobre los parámetros de `dumpdata` , ejecute `python manage.py dumpdata --help` .

Guarde este volcado en un archivo de accesos en un nuevo directorio de accesos / en los cursos aplicación usando los siguientes comandos:

```

cursos / accesos de mkdir
python manage.py dumpdata cursos --indent = 2 --output = cursos / fixtures /
subject.json

```

Ejecute el servidor de desarrollo y utilice el sitio de administración para eliminar los temas. tu creaste. Luego, cargue el dispositivo en la base de datos usando el siguiente comando:

```
python manage.py loaddata subject.json
```

Todos los objetos Subject incluidos en el accesorio se cargan en la base de datos.

De forma predeterminada, Django busca archivos en el directorio fixtures / de cada aplicación, pero puede especificar la ruta completa al archivo de accesorios para el comando loaddata . Tú también puede usar la configuración FIXTURE_DIRS para indicarle a Django directorios adicionales en los que buscar para accesorios.

Los dispositivos no solo son útiles para configurar datos iniciales, sino también para proporcionar datos de muestra para su aplicación o datos necesarios para tus pruebas.

Puede leer acerca de cómo usar accesorios para realizar pruebas en <https://docs.djangoproject.com/en/3.0/topics/testing/tools/#fixture-loading>.

Si desea cargar accesorios en migraciones de modelos, eche un vistazo a Django's documentación sobre migraciones de datos. Puede encontrar la documentación para migrar datos en <https://docs.djangoproject.com/en/3.0/topics/migrations/#data-migraciones>.

[364]

Creando modelos para contenido diverso

Planea agregar diferentes tipos de contenido a los módulos del curso, como texto, imágenes, archivos y videos. Por lo tanto, necesita un modelo de datos versátil que permita para almacenar contenido diverso. En el *Capítulo 6 , Seguimiento de las acciones del usuario* , aprendió conveniencia de usar relaciones genéricas para crear claves externas que pueden apuntar al objetos de cualquier modelo. Vas a crear un modelo de contenido que represente la contenidos de los módulos, y definir una relación genérica para asociar cualquier tipo de contenido.

Edite el archivo models.py de la aplicación de cursos y agregue las siguientes importaciones:

```
desde django.contrib.contenttypes.models import ContentType
de django.contrib.contenttypes.fields import GenericForeignKey
```

Luego, agregue el siguiente código al final del archivo:

```
class Contenido (modelos.Modelo):
    módulo = modelos.ForeignKey (Módulo,
                                related_name = 'contenido',
                                on_delete = modelos.CASCADE)
    content_type = modelos.ForeignKey (ContentType,
                                      on_delete = modelos.CASCADE)
    object_id = modelos.PositiveIntegerField ()
    item = GenericForeignKey ('content_type', 'object_id')
```

Este es el modelo de contenido . Un módulo contiene varios contenidos, por lo que define un Campo ForeignKey que apunta al modelo del módulo . También configuras una relación genérica asociar objetos de diferentes modelos que representan diferentes tipos de contenido. Recuerde que necesita tres campos diferentes para configurar una relación genérica. En tus Modelo de contenido , estos son:

- content_type : un campo ForeignKey para el modelo ContentType
- object_id : Un PositiveIntegerField para almacenar la clave principal de los relacionados objeto
- item : un campo GenericForeignKey para el objeto relacionado que combina los dos

campos anteriores

Solo los campos `content_type` y `object_id` tienen una columna correspondiente en la tabla de base de datos de este modelo. El campo de elemento le permite recuperar o configurar el objeto directamente, y su funcionalidad se basa en los otros dos campos.

[365]

Construyendo una plataforma de aprendizaje electrónico

Vas a utilizar un modelo diferente para cada tipo de contenido. Tus modelos de contenido tendrán algunos campos comunes, pero diferirán en los datos reales que pueden almacenar.

Usando la herencia del modelo

Django admite la herencia de modelos. Funciona de forma similar a la clase estándar. herencia en Python. Django ofrece las siguientes tres opciones para usar el modelo herencia:

- **Modelos abstractos** : útiles cuando desea incluir información común en varios modelos.
- **Herencia del modelo de varias tablas** : se aplica cuando cada modelo de la jerarquía se considera un modelo completo en sí mismo.
- **Modelos proxy** : útil cuando necesita cambiar el comportamiento de un modelo, por ejemplo, al incluir métodos adicionales, cambiar el administrador predeterminado, o usando diferentes meta opciones.

Echemos un vistazo más de cerca a cada uno de ellos.

Modelos abstractos

Un modelo abstracto es una clase base en la que define los campos que desea incluir en todos los modelos secundarios. Django no crea tablas de base de datos para modelos abstractos. Se crea una tabla de base de datos para cada modelo hijo, incluidos los campos heredados de la clase abstracta y las definidas en el modelo hijo.

Para marcar un modelo como abstracto, debe incluir `abstract = True` en su clase Meta . Django reconocerá que es un modelo abstracto y no creará una tabla de base de datos para ello. Para crear modelos secundarios, solo necesita crear una subclase del modelo abstracto.

El siguiente ejemplo muestra un modelo de contenido abstracto y un modelo de texto secundario :

```
desde modelos de importación django.db

class BaseContent (modelos.Modelo):
    title = modelos.CharField (max_length = 100)
    creado = modelos.DateTimeField (auto_now_add = True)

    class Meta:
        abstracto = Verdadero

class Text (BaseContent):
    cuerpo = modelos.TextField ()
```

[366]

En este caso, Django crearía una tabla solo para el modelo `Text`, incluyendo el campos de título, creado y cuerpo.

Herencia del modelo de múltiples tablas

En la herencia de múltiples tablas, cada modelo corresponde a una tabla de base de datos. Django crea un campo `OneToOneField` para la relación entre el modelo hijo y su modelo padre. Para utilizar la herencia de varias tablas, debe crear una subclase de un modelo existente. Django creará una tabla de base de datos tanto para el modelo original como para el submodelo. El siguiente ejemplo muestra la herencia de varias tablas:

```
desde modelos de importación django.db

class BaseContent (modelos.Modelo):
    title = modelos.CharField (max_length = 100)
    creado = modelos.DateTimeField (auto_now_add = True)

class Text (BaseContent):
    cuerpo = modelos.TextField ()
```

Django incluiría un campo `OneToOneField` generado automáticamente en el texto modelar y crear una tabla de base de datos para cada modelo.

Modelos proxy

Un modelo proxy cambia el comportamiento de un modelo. Ambos modelos funcionan con tabla de base de datos del modelo original. Para crear un modelo de proxy, agregue `proxy = True` a la clase Meta del modelo. El siguiente ejemplo ilustra cómo crear un proxy. modelo:

```
desde modelos de importación django.db
desde django.utils importar zona horaria

class BaseContent (modelos.Modelo):
    title = modelos.CharField (max_length = 100)
    creado = modelos.DateTimeField (auto_now_add = True)

class OrderedContent (BaseContent):
    class Meta:
        proxy = Verdadero
        ordering = ['creado']

    def created_delta (yo):
        return timezone.now () - self.creado
```

[367]

Aquí, define un modelo `OrderedContent` que es un modelo proxy para el contenido modelo. Este modelo proporciona un orden predeterminado para `QuerySets` y un método `created_delta()`. Ambos modelos, `Content` y `OrderedContent`, operan en la misma tabla de base de datos, y los objetos son accesibles a través del ORM a través cualquier modelo.

Creando los modelos de contenido

El modelo de contenido de su aplicación de cursos contiene una relación genérica con asociarle diferentes tipos de contenido. Crearás un modelo diferente para cada tipo de contenido. Todos los modelos de contenido tendrán algunos campos en común y adicionales campos para almacenar datos personalizados. Vas a crear un modelo abstracto que proporcione los campos comunes para todos los modelos de contenido.

Edite el archivo `models.py` de la aplicación de cursos y agréguele el siguiente código:

```
class ItemBase (modelos.Modelo):
    propietario = modelos.ForeignKey (Usuario,
                                     related_name = '% (clase) s_related',
                                     on_delete = modelos.CASCADE)

    title = modelos.CharField (max_length = 250)
    creado = modelos.DateTimeField (auto_now_add = True)
    actualizado = modelos.DateTimeField (auto_now = True)

    class Meta:
        abstracto = Verdadero

    def __str__ (yo):
        volver self.title

class Text (ItemBase):
    contenido = modelos.TextField ()

Archivo de clase (ItemBase):
    file = modelos.FileField (upload_to = 'archivos')

Imagen de clase (ItemBase):
    file = modelos.FileField (upload_to = 'imágenes')

video de clase (ItemBase):
    url = modelos.URLField ()
```

En este código, define un modelo abstracto llamado `ItemBase`. Por lo tanto, estableces `abstract = True` en su clase `Meta`.

[368]

En este modelo, usted define el propietario, el título, los campos creados y actualizados. Estas Los campos comunes se utilizarán para todo tipo de contenido.

El campo de propietario le permite almacenar qué usuario creó el contenido. Dado que este campo se define en una clase abstracta, se necesita un diferente `related_name` para cada sub-modelo. Django le permite especificar un marcador de posición para el nombre de la clase del modelo en el atributo `related_name` como `%(clase)s`. Al hacerlo, `related_name` para cada niño El modelo se generará automáticamente. Dado que usa `%(class)s_related` como la `related_name`, será la relación inversa de los niños modelos `text_related`, `file_related`, `image_related` y `video_related`, respectivamente.

Ha definido cuatro modelos de contenido diferentes que heredan de `ItemBase` modelo abstracto. Estos son los siguientes:

- Texto : para almacenar contenido de texto
- Archivo : para almacenar archivos, como PDF

- Imagen : para almacenar archivos de imagen
- Video : para almacenar videos; utiliza un campo URLField para proporcionar una URL de video para incrustarlo

Cada modelo hijo contiene los campos definidos en la clase ItemBase además de su propios campos. Se creará una tabla de base de datos para el texto , archivo , imagen y video modelos, respectivamente. No habrá ninguna tabla de base de datos asociada con ItemBase modelo, ya que es un modelo abstracto.

Edite el modelo de contenido que creó anteriormente y modifique su campo content_type , como sigue:

```
content_type = modelos.ForeignKey (ContentType,
                                   on_delete = modelos.CASCADE ,
                                   limit_choices_to = {'model__in' :(
                                                         'texto',
                                                         'video',
                                                         'imagen',
                                                         'archivo')})
```

Agrega un argumento limit_choices_to para limitar los objetos ContentType que se pueden utilizado para la relación genérica. Utiliza la búsqueda de campo model__in para filtrar la consulta a los objetos ContentType con un atributo de modelo que es 'texto' , 'video' , 'imagen' , o 'archivo' .

Creemos una migración para incluir los nuevos modelos que ha agregado. Ejecutar el siguiente comando desde la línea de comando:

```
python manage.py makemigrations
```

[369]

Construyendo una plataforma de aprendizaje electrónico

Verá el siguiente resultado:

Migraciones para 'cursos':

```
cursos / migraciones / 0002_content_file_image_text_video.py
```

- Crear video modelo
- Crear texto modelo
- Crear imagen de modelo
- Crear archivo de modelo
- Crear contenido modelo

Luego, ejecute el siguiente comando para aplicar la nueva migración:

```
python manage.py migrar
```

La salida que ve debería terminar con la siguiente línea:

```
Aplicando cursos.0002_content_file_image_text_video ... OK
```

Ha creado modelos que son adecuados para agregar contenido diverso al curso. módulos. Sin embargo, todavía falta algo en sus modelos: el curso los módulos y los contenidos deben seguir un orden particular. Necesitas un campo que le permite ordenarlos fácilmente.

Crear campos de modelo personalizados

Django viene con una colección completa de campos de modelo que puede usar para construir tus modelos. Sin embargo, también puede crear sus propios campos de modelo para almacenar datos o alterar el comportamiento de los campos existentes.

Necesita un campo que le permita definir un orden para los objetos. Una forma facil especificar un orden para los objetos usando campos existentes de Django es agregando un

PositiveIntegerField a sus modelos. Usando números enteros, puede especificar fácilmente el orden de los objetos. Puede crear un campo de pedido personalizado que herede de

PositiveIntegerField y proporciona un comportamiento adicional.

Hay dos funcionalidades relevantes que integrará en su campo de pedido:

- **Asignar automáticamente un valor de pedido cuando no se proporciona un pedido específico :**

Al guardar un nuevo objeto sin un orden específico, su campo debe asignar automáticamente el número que viene después del último pedido existente objeto. Si hay dos objetos con orden 1 y 2 respectivamente, al guardar un tercer objeto, debe asignarle automáticamente el orden 3 si no se ha proporcionado el pedido.

[370]

- **Ordenar objetos con respecto a otros campos :** los módulos del curso se ordenarán con respecto al curso al que pertenecen y los contenidos del módulo con respecto al módulo al que pertenecen.

Cree un nuevo archivo fields.py dentro del directorio de la aplicación de cursos y agregue el siguiente código:

```
desde modelos de importación django.db
desde django.core.exceptions importar ObjectDoesNotExist

class OrderField (modelos.PositiveIntegerField):
    def __init__ (self, for_fields = None, * args, ** kwargs):
        self.for_fields = for_fields
        super () .__init__ (* argumentos, ** kwargs)

    def pre_save (self, model_instance, agregar):
        si getattr (model_instance, self.attname) es None:
            # sin valor actual
            tratar:
                qs = self.model.objects.all ()
                si self.for_fields:
                    # filtrar por objetos con los mismos valores de campo
                    # para los campos de "for_fields"
                    query = {field: getattr (model_instance, field) \
                        for field in self.for_fields}
                    qs = qs.filter (** consulta)
                # obtener el pedido del último artículo
                last_item = qs.latest (self.attname)
                value = last_item.order + 1
            except ObjectDoesNotExist:
                valor = 0
            setattr (modelo_instancia, self.attname, valor)
        valor de retorno
    más:
        return super (). pre_save (model_instance, agregar)
```

Este es su OrderField personalizado . Hereda de PositiveIntegerField campo proporcionado por Django. Su campo OrderField toma un for_fields opcional parámetro que le permite indicar los campos en los que se tiene que calcular el pedido con respecto a.

[371]

Construyendo una plataforma de aprendizaje electrónico

Su campo anula el método `pre_save()` del campo `PositiveIntegerField`, que se ejecuta antes de guardar el campo en la base de datos. En este método, usted realizar las siguientes acciones:

1. Verifica si ya existe un valor para este campo en la instancia del modelo. Utiliza `self.attname`, que es el nombre de atributo dado al campo en el modelo. Si el valor del atributo es diferente a `Ninguno`, calcula el orden que debería darlo de la siguiente manera:
 1. Construye un `QuerySet` para recuperar todos los objetos para el modelo del campo. Tú recupere la clase de modelo a la que pertenece el campo accediendo a `self.model`.
 2. Si hay algún nombre de campo en el atributo `for_fields` del campo, filtra el `QuerySet` por el valor actual de los campos del modelo en `for_fields`. Al hacerlo, calcula el orden con respecto a los campos dados.
 3. Recupera el objeto con el orden más alto con `last_item = qs.latest(self.attname)` de la base de datos. Si no se encuentra ningún objeto, asume que este objeto es el primero y le asigna el orden 0.
 4. Si se encuentra un objeto, agregue 1 al orden más alto encontrado.
 5. Asigne el orden calculado al valor del campo en el modelo. instancia usando `setattr()` y devolverlo.
2. Si la instancia del modelo tiene un valor para el campo actual, utilícelo en su lugar de calcularlo.

Cuando cree campos de modelo personalizados, conviértalos en genéricos. Evitar codificación de datos que dependen de un modelo o campo específico. Tu el campo debería funcionar en cualquier modelo.

Puede encontrar más información sobre cómo escribir campos de modelo personalizados en [https:// docs.django project.com/en/3.0/howto/custom-model-fields/](https://docs.djangoproject.com/en/3.0/howto/custom-model-fields/).

Agregar orden al módulo y al contenido objetos

Agreguemos el nuevo campo a sus modelos. Edite el archivo `models.py` de los cursos aplicación, e importe la clase `OrderField` y un campo al modelo del módulo, como sigue:

```
desde .fields importar OrderField
```

[372]

[illegible]

```

content_type = modelos.ForeignKey (ContentType,
                                   on_delete = modelos.CASCADE,
                                   limit_choices_to = {'model__in' : (
                                                           'texto',
                                                           'video',
                                                           'imagen',
                                                           'expediente')})

object_id = modelos.PositiveIntegerField ()
item = GenericForeignKey ('content_type', 'object_id')
order = OrderField (en blanco = True, for_fields = ['módulo'])

```

```

clase Meta:
    ordering = ['pedido']

```

Creemos una nueva migración de modelo que refleje los nuevos campos de orden. Abre la cáscara y ejecuta el siguiente comando:

cursos de makemigrations de python manage.py

Verá el siguiente resultado:

Está intentando agregar un 'orden' de campo no anulable al contenido sin un defecto; no podemos hacer eso (la base de datos necesita algo para poblar filas existentes).

Seleccione una solución:

1) Proporcione un valor predeterminado único ahora (se establecerá en todas las filas existentes con un valor nulo para esta columna)

2) Salga y déjeme agregar un valor predeterminado en models.py

Seleccione una opción:

[374]

Django le dice que debe proporcionar un valor predeterminado para el nuevo pedido campo para filas existentes en la base de datos. Si el campo tuviera nulo = Verdadero , aceptaría valores nulos y Django crearía la migración automáticamente en lugar de preguntar por un valor predeterminado. Puede especificar un valor predeterminado o cancelar la migración y agregue un atributo predeterminado al campo de pedido en el archivo models.py antes de crear la migración.

Ingrese 1 y presione *Enter* para proporcionar un valor predeterminado para los registros existentes. Ya verá la siguiente salida:

Ingrese el valor predeterminado ahora, como Python válido

Los módulos datetime y django.utils.timezone están disponibles, por lo que puede por ejemplo, timezone.now

Escriba 'salir' para salir de este mensaje

```
>>>
```

Ingrese 0 para que este sea el valor predeterminado para los registros existentes y presione *Enter* . Django también le pedirá un valor predeterminado para el modelo del módulo . Elige la primera opción e ingrese 0 como valor predeterminado nuevamente. Finalmente, verá una salida similar a la siguiente:

Migraciones para 'cursos':

cursos / migraciones / 0003_auto_20191214_1253.py

- Cambiar las opciones de Meta en el contenido
- Cambiar las opciones de Meta en el módulo
- Agregar orden de campo al contenido
- Agregar orden de campo al módulo

Luego, aplique las nuevas migraciones con el siguiente comando:

```
python manage.py migrar
```

El resultado del comando le informará que la migración se realizó correctamente. aplicado, de la siguiente manera:

```
Aplicando cursos.0003_auto_20191214_1253 ... OK
```

Problemos tu nuevo campo. Abra el shell con el siguiente comando:

```
shell de python manage.py
```

Cree un curso nuevo de la siguiente manera:

```
>>> desde django.contrib.auth.models import User
>>> de cursos.modelos importar Materia, Curso, Módulo
```

[375]

Construyendo una plataforma de aprendizaje electrónico

```
>>> usuario = User.objects.last ()
>>> asunto = Subject.objects.last ()
>>> c1 = Course.objects.create (asunto = asunto, propietario = usuario, título = 'Curso
1 ', slug = 'curso1 ')
```

Ha creado un curso en la base de datos. Ahora, agregará módulos al curso y vea cómo su orden se calcula automáticamente. Creas un módulo inicial y comprobar su orden:

```
>>> m1 = Module.objects.create (curso = c1, título = 'Módulo 1')
>>> m1.orden
0
```

OrderField establece su valor en 0 , ya que este es el primer objeto Module creado para el curso dado. Tú, crea un segundo módulo para el mismo curso:

```
>>> m2 = Module.objects.create (curso = c1, título = 'Módulo 2')
>>> m2.orden
1
```

OrderField calcula el siguiente valor de pedido, agregando 1 al pedido más alto para objetos. Creemos un tercer módulo, forzando un orden específico:

```
>>> m3 = Module.objects.create (course = c1, title = 'Module 3', order = 5)
>>> m3.orden
5
```

Si especifica un pedido personalizado, el campo OrderField no interfiere y el valor dado a la orden se utiliza.

Agreguemos un cuarto módulo:

```
>>> m4 = Module.objects.create (curso = c1, título = 'Módulo 4')
>>> m4.order
6
```

El orden de este módulo se ha establecido automáticamente. Su campo OrderField no garantiza que todos los valores de los pedidos sean consecutivos. Sin embargo, respeta los valores de pedido y siempre asigna el siguiente pedido en función del pedido existente más alto.

Creemos un segundo curso y agreguemos un módulo:

```
>>> c2 = Course.objects.create (asunto = asunto, título = 'Curso 2',
slug = 'curso2', propietario = usuario)
```



```
>>> m5 = Module.objects.create(curso = c2, título = 'Módulo 1')
```

[376]

```
>>> m5.order
```

```
0
```

Para calcular el orden del nuevo módulo, el campo solo toma en consideración módulos existentes que pertenecen al mismo curso. Dado que este es el primer módulo del segundo curso, el orden resultante es 0 . Esto se debe a que especificó para `fields = ['curso']` en el campo de orden del modelo del módulo .

¡Felicidades! Ha creado con éxito su primer campo de modelo personalizado.

Creando un CMS

Ahora que ha creado un modelo de datos versátil, va a crear el CMS.

El CMS permitirá a los instructores crear cursos y administrar sus contenidos. Tú necesita proporcionar la siguiente funcionalidad:

- Inicie sesión en el CMS
- Enumere los cursos creados por el instructor
- Crear, editar y eliminar cursos
- Agregar módulos a un curso y reordenarlos
- Agregue diferentes tipos de contenido a cada módulo y reordenarlos

Agregar un sistema de autenticación

Vas a utilizar el marco de autenticación de Django en tu plataforma. Ambos instructores y estudiantes serán instancias del modelo de usuario de Django , por lo que serán capaz de iniciar sesión en el sitio utilizando las vistas de autenticación de `django.contrib.auth` .

Editar el principal `urls.py` archivo de la Educa proyecto e incluir el inicio de sesión y cierre de sesión vistas del marco de autenticación de Django:

```
desde django.contrib import admin
desde la ruta de importación django.urls
desde django.contrib.auth importar vistas como auth_views

urlpatterns = [
    ruta ('cuentas / inicio de sesión /', auth_views.LoginView.as_view (),
        nombre = 'inicio de sesión'),
    ruta ('cuentas / cerrar sesión /', auth_views.LogoutView.as_view (),
        nombre = 'cerrar sesión'),
    ruta ('admin /', admin.site.urls),
]
```

[377]

Creando las plantillas de autenticación

Cree la siguiente estructura de archivos dentro del directorio de la aplicación de cursos :

```
plantillas/  
  base.html  
  registro/  
    login.html  
    log_out.html
```

Antes de crear las plantillas de autenticación, debe preparar la plantilla base para su proyecto. Edite el archivo de plantilla base.html y agregue el siguiente contenido:

```
{% load static%}  
<!DOCTYPE html>  
<html>  
<cabeza>  
  <meta charset = "utf-8" />  
  <title> {% block title%} Educa {% endblock%} </title>  
  <link href = "{% static" css / base.css" %}" rel = "stylesheet">  
</head>  
<cuerpo>  
  <div id = "header">  
    <a href="/" class="logo"> Educa </a>  
    <ul class = "menu">  
      {% if request.user.is_authenticated%}  
        <li> <a href="{% url "logout" %}"> Cerrar sesión </a> </li>  
      {% else%}  
        <li> <a href="{% url "login" %}"> Iniciar sesión </a> </li>  
      {% terminara si %}  
    </ul>  
  </div>  
  <div id = "contenido">  
    {% block content%}  
    {% endblock%}  
  </div>  
  
  <script src = "https://ajax.googleapis.com/ajax/libs/jquery/3.4.1/  
jquery.min.js ">  
</script>  
<script>  
  $(document).ready(function () {  
    {% block domready%}  
    {% endblock%}  
  });
```

[378]

```
</script>  
</body>  
</html>
```

Esta es la plantilla base que se ampliará con el resto de plantillas. En esta plantilla, define los siguientes bloques:

- título : el bloque para que otras plantillas agreguen un título personalizado para cada página.
- contenido : el bloque principal de contenido. Todas las plantillas que extienden la base

- La plantilla debe agregar contenido a este bloque.
- domready : se encuentra dentro de la función `$(document).ready()` de jQuery. Eso le permite ejecutar código cuando el **Modelo de objetos de documento (DOM)** tiene terminado de cargar.

Los estilos CSS utilizados en esta plantilla se encuentran en el directorio `static / del` aplicación de cursos en el código que acompaña a este capítulo. Copia el `static / directorio` en el mismo directorio de su proyecto para usarlos. Puedes encontrar el contenido del directorio en [https://github.com/PacktPublishing/Django-3-por-ejemplo-árbol / master / Chapter10 / educa / ourses / static](https://github.com/PacktPublishing/Django-3-por-ejemplo-árbol/master/Chapter10/educa/ourses/static).

Edite la plantilla `registration / login.html` y agréguele el siguiente código:

```
{% extiende "base.html"%}

{% block title%} Iniciar sesión {% endblock%}

{% block content%}
<h1> Iniciar sesión </h1>
<div class = "módulo">
    {% if form.errors%}
        <p> Su nombre de usuario y contraseña no coinciden. Vuelve a intentarlo. </p>
    {% else%}
        <p> Por favor, utilice el siguiente formulario para iniciar sesión: </p>
    {% terminara si %}
    <div class = "formulario de inicio de sesión">
        <form action = "{% url 'login'" method = "post">
            {{form.as_p}}
            {% csrf_token%}
            <input type = "hidden" name = "next" value = "{{next}}"/>
            <p> <input type = "submit" value = "Iniciar sesión"> </p>
        </form>
    </div>
</div>
{% endblock%}
```

[379]

Construyendo una plataforma de aprendizaje electrónico

Esta es una plantilla de inicio de sesión estándar para la vista de inicio de sesión de Django .

Edite la plantilla `registration / connected_out.html` y agréguele el siguiente código:

```
{% extiende "base.html"%}

{% block title%} Se desconectó {% endblock%}

{% block content%}
<h1> Desconectado </h1>
<div class = "módulo">
    <p> Se ha cerrado la sesión correctamente.
        Puede <a href="{% url 'login' %}"> iniciar sesión de nuevo </a>. </p>
    </div>
{% endblock%}
```

Esta es la plantilla que se mostrará al usuario después de cerrar la sesión. Ejecutar el servidor de desarrollo con el siguiente comando:

python manage.py runserver

Abra `http://127.0.0.1:8000/accounts/login/` en su navegador. Deberías ver la página de inicio de sesión:

Figura 10.2: La página de inicio de sesión de la cuenta

[380]

Crear vistas basadas en clases

Vas a crear vistas para crear, editar y eliminar cursos. Usarás class-vistas basadas en esto. Edite el archivo `views.py` de la aplicación de cursos y agregue el siguiente código:

```
desde django.views.generic.list import ListView
desde el curso de importación de .models

clase ManageCourseListView (ListView):
    modelo = Curso
    template_name = 'cursos / administrar / curso / list.html'

    def get_queryset (yo):
        qs = super (). get_queryset ()
        return qs.filter (propietario = self.request.user)
```

Esta es la vista `ManageCourseListView` . Hereda del `ListView` genérico de Django . Anula el `get_queryset ()` método de la vista para recuperar sólo los cursos creado por el usuario actual. Para evitar que los usuarios editen, actualicen o eliminen cursos que no crearon, también deberá anular `get_queryset ()` en las vistas de creación, actualización y eliminación. Cuando necesite proporcionar un comportamiento para varias vistas basadas en clases, se recomienda que utilice *mixins* .

Usando mixins para vistas basadas en clases

Los mixins son un tipo especial de herencia múltiple para una clase. Puedes usarlos para proporcionar una funcionalidad discreta común que, cuando se agrega a otras mezclas, permite usted para definir el comportamiento de una clase. Hay dos situaciones principales para usar mixins:

- Quiere proporcionar varias funciones opcionales para una clase
- Quieres usar una función en particular en varias clases

Django viene con varios mixins que brindan funcionalidad adicional a su clase. vistas basadas. Puede obtener más información sobre mixins en <https://docs.djangoproject.com/en/3.0/topics/class-based-views/mixins/>.

Vas a crear una clase mixin que incluya un comportamiento común y la usarás para las vistas del curso. Edite el archivo `views.py` de la aplicación de cursos y modifique de la siguiente manera:

```
de django.urls importar reverse_lazy
desde django.views.generic.list importar ListView
desde django.views.generic.edit importar CreateView, UpdateView, \
DeleteView
```

[381]

Construyendo una plataforma de aprendizaje electrónico

desde el curso de importación de .models

```
class OwnerMixin (objeto):
    def get_queryset (yo):
        qs = super (). get_queryset ()
        return qs.filter (propietario = self.request.user)

class OwnerEditMixin (objeto):
    def form_valid (self, form):
        form.instance.owner = self.request.user
        return super (). form_valid (formulario)

class OwnerCourseMixin (OwnerMixin):
    modelo = Curso
    campos = ['asunto', 'titulo', 'slug', 'descripción general']
    Success_url = reverse_lazy ('administrar lista de cursos')

class OwnerCourseEditMixin (OwnerCourseMixin, OwnerEditMixin):
    template_name = 'cursos / administrar / curso / formulario.html'

class ManageCourseListView (OwnerCourseMixin, ListView):
    template_name = 'cursos / administrar / curso / list.html'

class CourseCreateView (OwnerCourseEditMixin, CreateView):
    pasar

class CourseUpdateView (OwnerCourseEditMixin, UpdateView):
    pasar

class CourseDeleteView (OwnerCourseMixin, DeleteView):
    template_name = 'cursos / administrar / curso / eliminar.html'
```

En este código, crea los mixins OwnerMixin y OwnerEditMixin . Usarás estos mixins junto con ListView , CreateView , UpdateView y DeleteView vistas proporcionadas por Django. OwnerMixin implementa el método get_queryset () , que es utilizado por las vistas para obtener el QuerySet base. Tu mixin anulará esto método para filtrar objetos por el atributo propietario para recuperar objetos que pertenecen a la usuario actual (request.user).

OwnerEditMixin implementa el método form_valid () , que utilizan las vistas que usan el mixin ModelFormMixin de Django , es decir, vistas con formularios o modelo formularios como CreateView y UpdateView . form_valid () se ejecuta cuando el formulario enviado es válido.

[382]

El comportamiento predeterminado de este método es guardar la instancia (para formularios modelo) y redirigir al usuario a `success_url`. Anula este método para establezca el usuario actual en el atributo propietario del objeto que se está guardando. Al hacerlo, establece el propietario de un objeto automáticamente cuando se guarda.

Su clase `OwnerMixin` se puede utilizar para vistas que interactúan con cualquier modelo que contiene un atributo de propietario.

También define una clase `OwnerCourseMixin` que hereda `OwnerMixin` y proporciona los siguientes atributos para las vistas secundarias:

- `modelo` : el modelo utilizado para `QuerySets`; es utilizado por todas las vistas.
- `campos` : los campos del modelo para construir el formulario de modelo de `CreateView` y vistas `UpdateView`.
- `success_url` : utilizado por `CreateView`, `UpdateView` y `DeleteView` para Redirigir al usuario después de que el formulario se haya enviado correctamente o el objeto eliminado. Utiliza una URL con el nombre `manage_course_list`, que vamos a crear más tarde.

Usted define un mixin `OwnerCourseEditMixin` con el siguiente atributo:

- `template_name` : la plantilla que utilizará para `CreateView` y Vistas de `UpdateView`

Finalmente, crea las siguientes vistas que subclase `OwnerCourseMixin`:

- `ManageCourseListView` : enumera los cursos creados por el usuario. Hereda de `OwnerCourseMixin` y `ListView`. Define un `template_name` específico atributo de una plantilla para enumerar los cursos.
- `CourseCreateView` : utiliza un formulario modelo para crear un nuevo objeto de curso. Utiliza los campos definidos en `OwnerCourseMixin` para construir un modelo form y también subclases `CreateView`. Utiliza la plantilla definida en `OwnerCourseEditMixin`.
- `CourseUpdateView` : permite la edición de un objeto de curso existente. Utiliza los campos definidos en `OwnerCourseMixin` para construir un modelo form y también subclases `UpdateView`. Utiliza la plantilla definida en `OwnerCourseEditMixin`.
- `CourseDeleteView` : hereda de `OwnerCourseMixin` y el genérico `DeleteView`. Define un atributo de `template_name` específico para una plantilla para confirmar la eliminación del curso.

Trabajar con grupos y permisos

Ha creado las vistas básicas para gestionar los cursos. Actualmente, cualquier usuario puede acceder a estas vistas. Desea restringir estas vistas para que solo los instructores tengan la permiso para crear y gestionar cursos.

El marco de autenticación de Django incluye un sistema de permisos que le permite para asignar permisos a usuarios y grupos. Vas a crear un grupo para instructores y asignar permisos para crear, actualizar y eliminar cursos.

Ejecute el servidor de desarrollo con el comando `python manage.py runserver` y abra `http://127.0.0.1:8000/admin/auth/group/add/` en su navegador para crear un nuevo objeto de grupo . Agregue el nombre Instructores y elija todos los permisos de la aplicación de cursos , excepto los del modelo Asignatura , de la siguiente manera:

Figura 10.3: Permisos del grupo de instructores

Como puede ver, hay cuatro permisos diferentes para cada modelo: *puede ver* , *puede agregar* , *puede cambiar* y *puede eliminar* . Después de elegir los permisos para este grupo, haga clic en el botón **GUARDAR** .

Django crea permisos para modelos automáticamente, pero también puede crear permisos. Aprenderá a crear permisos personalizados en el *Capítulo 12, Creación una API* . Puede leer más sobre cómo agregar permisos personalizados en <https://docs.djangoproject.com/en/3.0/topics/auth/customizing/#custom-permissions> .

Abra `http://127.0.0.1:8000/admin/auth/user/add/` y cree un nuevo usuario. Edite el usuario y agréguelo al grupo de **instructores** , de la siguiente manera:

[384]

Figura 10.4: Selección de grupo de usuarios

Los usuarios heredan los permisos de los grupos a los que pertenecen, pero también puede agregar permisos individuales para un solo usuario que usa el sitio de administración. Usuarios que tener `is_superuser` establecido en `True` tiene todos los permisos automáticamente.

Restringir el acceso a las vistas basadas en clases

Va a restringir el acceso a las vistas para que solo los usuarios con el los permisos pueden agregar, cambiar o eliminar objetos del curso . Vas a usar el siguientes dos mixins proporcionados por `django.contrib.auth` para limitar el acceso a las vistas:

- `LoginRequiredMixin` : replica el decorador `login_required`

funcionalidad.

- `PermissionRequiredMixin` : otorga acceso a la vista a los usuarios con un permiso específico. Recuerde que los superusuarios tienen automáticamente todos los permisos.

Edite el archivo `views.py` de la aplicación de cursos y agregue la siguiente importación:

```
de django.contrib.auth.mixins importar LoginRequiredMixin, \
                                     PermissionRequiredMixin
```

Hacer que `OwnerCourseMixin` herede `LoginRequiredMixin` y

`PermissionRequiredMixin`, así:

```
clase OwnerCourseMixin (OwnerMixin ,
                        LoginRequiredMixin,
                        PermissionRequiredMixin ):
    modelo = Curso
    campos = ['asunto', 'titulo', 'slug', 'descripción general']
    Success_url = reverse_lazy ('administrar lista de cursos')
```

[385]

Construyendo una plataforma de aprendizaje electrónico

Luego, agregue un atributo `allow_required` a las vistas del curso, de la siguiente manera:

```
clase ManageCourseListView (OwnerCourseMixin, ListView):
    template_name = 'cursos / administrar / curso / list.html'
    allow_required = 'cursos.view_course'

clase CourseCreateView (OwnerCourseEditMixin, CreateView):
    allow_required = 'cursos.add_course'

clase CourseUpdateView (OwnerCourseEditMixin, UpdateView):
    permiso_required = 'cursos.cambiar_curso'

clase CourseDeleteView (OwnerCourseMixin, DeleteView):
    template_name = 'cursos / administrar / curso / eliminar.html'
    allow_required = 'cursos.delete_course'
```

`PermissionRequiredMixin` comprueba que el usuario que accede a la vista tiene la permiso especificado en el atributo `allow_required`. Tus puntos de vista son ahora solo es accesible para usuarios con los permisos adecuados.

Creemos URL para estas vistas. Crea un nuevo archivo dentro de la aplicación de cursos directorio y el nombre de `urls.py`. Agregue el siguiente código:

```
desde la ruta de importación django.urls
de . importar vistas

urlpatterns = [
    camino ('mío /',
            views.ManageCourseListView.as_view (),
            name = 'manage_course_list'),
    ruta ('crear /',
          views.CourseCreateView.as_view (),
          nombre = 'course_create'),
    ruta ('<pk> / edit /',
          views.CourseUpdateView.as_view (),
          name = 'course_edit'),
    ruta ('<pk> / delete /',
          views.CourseDeleteView.as_view (),
```



```

        name = 'course_delete'),
    ]

```

Estos son los patrones de URL para la lista, crear, editar y eliminar vistas del curso. Edite el principal `urls.py` archivo de la Educa proyecto e incluya los patrones de URL de los cursos aplicación, de la siguiente manera:

de la ruta de importación `django.urls` , **incluir**

[386]

```

urlpatterns = [
    ruta ('cuentas / inicio de sesión /', auth_views.LoginView.as_view (),
        nombre = 'inicio de sesión'),
    ruta ('cuentas / cerrar sesión /', auth_views.LogoutView.as_view (),
        nombre = 'cerrar sesión'),
    ruta ('admin /', admin.site.urls),
    ruta ('curso /', incluir ('cursos.urls')),
]

```

Necesita crear las plantillas para estas vistas. Cree los siguientes directorios y archivos dentro del directorio `templates /` de la aplicación de cursos :

```

cursos /
  gestionar/
    curso/
      list.html
      form.html
      delete.html

```

Edite la plantilla `course / manage / course / list.html` y agregue lo siguiente codificarlo:

```

{% extiende "base.html"%}

{% block title%} Mis cursos {% endblock%}

{% block content%}
<h1> Mis cursos </h1>

<div class = "módulo">
    {% para el curso en object_list%}
    <div class = "información del curso">
        <h3> {{course.title}} </h3>
        <p>
            <a href="{% url "course_edit" course.id %}"> Editar </a>
            <a href="{% url "course_delete" course.id %}"> Eliminar </a>
        </p>
    </div>
    {% vacío%}
    <p> Aún no ha creado ningún curso. </p>
    {% endfor%}
    <p>
        <a href="{% url "course_create" %}" class="button"> Crear nuevo
curso </a>
    </p>
    </div>
{% endblock%}

```

[387]

Construyendo una plataforma de aprendizaje electrónico

Esta es la plantilla para la vista `ManageCourseListView`. En esta plantilla, enumere los cursos creados por el usuario actual. Incluye enlaces para editar o eliminar cada curso y un enlace para crear nuevos cursos.

Ejecute el servidor de desarrollo con el comando `python manage.py runserver`. Abra `http://127.0.0.1:8000/accounts/login/?next=/course/mine/` en su navegador e inicie sesión con un usuario que pertenezca al grupo Instructores. Después de iniciar sesión en, será redirigido a la URL `http://127.0.0.1:8000/course/mine/` y debería ver la siguiente página:

Figura 10.5: La página de cursos del instructor sin cursos

Esta página mostrará todos los cursos creados por el usuario actual.

Creemos la plantilla que muestra el formulario para el curso de creación y actualización. puntos de vista. Edite la plantilla `cursos / manage / course / form.html` y escriba el siguiente código:

```
{% extiende "base.html"%}

{% block title%}
    {% if object%}
        Editar curso "{{object.title}}"
    {% else%}
        Crea un curso nuevo
    {% terminara si %}
{% endblock%}

{% block content%}
    <h1>
        {% if object%}
            Editar curso "{{object.title}}"
        {% else%}
```

[388]

```
        Crea un curso nuevo
    {% terminara si %}
```

```

</h1>
<div class = "módulo">
  <h2> Información del curso </h2>
  <método de formulario = "publicar">
    {{form.as_p}}
    {% csrf_token%}
    <p> <input type = "submit" value = "Guardar curso"> </p>
  </form>
</div>
{% endblock%}

```

La plantilla form.html se utiliza tanto para CourseCreateView como para

Vistas CourseUpdateView . En esta plantilla, verifica si una variable de objeto está en el contexto. Si el objeto existe en el contexto, sabe que está actualizando un curso existente, y lo usa en el título de la página. De lo contrario, está creando un nuevo Objeto del curso .

Abra <http://127.0.0.1:8000/course/mine/> en su navegador y haga clic en el Botón **CREAR CURSO NUEVO** . Verá la siguiente página:

Figura 10.6: El formulario para crear un nuevo curso

[389]

Complete el formulario y haga clic en el botón **GUARDAR CURSO** . El curso se guardará y será redirigido a la página de la lista de cursos. Debería verse como sigue:

Figura 10.7: La página de cursos del instructor con un curso

Luego, haga clic en el enlace **Editar** para el curso que acaba de crear. Verás el formulario de nuevo, pero esta vez está editando un objeto de curso existente en lugar de crear uno.

Finalmente, edite la plantilla `cursos / administrar / curso / eliminar.html` y agregue la siguiente código:

```
{% extiende "base.html"%}

{% block title%} Eliminar curso {% endblock%}

{% block content%}
<h1> Eliminar curso "{{object.title}}" </h1>

<div class = "módulo">
    <acción del formulario = "" método = "publicación">
        {% csrf_token%}
        <p> ¿Está seguro de que desea eliminar "{{object}}"? </p>
        <input type = "submit" value = "Confirmar">
    </form>
</div>
{% endblock%}
```

Esta es la plantilla para la vista `CourseDeleteView`. Esta vista hereda de

`DeleteView`, proporcionado por Django, que espera la confirmación del usuario para eliminar un objeto.

[390]

Abra la lista de cursos en el navegador y haga clic en el enlace **Eliminar** de su curso. Tú debería ver la siguiente página de confirmación:

Figura 10.8: La página de confirmación de eliminación del curso

Haga clic en el botón **CONFIRMAR**. El curso se eliminará y se le redirigirá a la página de la lista de cursos nuevamente.

Los profesores ahora pueden crear, editar y eliminar cursos. A continuación, debe proporcionarlos con un CMS para agregar módulos de curso y sus contenidos. Empezarás gestionando módulos del curso.

Gestionar los módulos del curso y sus contenido

Vas a construir un sistema para administrar los módulos del curso y sus contenidos. Tú necesitará crear formularios que se puedan usar para administrar múltiples módulos por curso y diferentes tipos de contenido para cada módulo. Ambos módulos y sus contenidos Tendrá que seguir un orden específico y debería poder reordenarlos usando el CMS.

Usar conjuntos de formularios para los módulos del curso

Django viene con una capa de abstracción para trabajar con múltiples formas en el mismo página. Estos grupos de formas se conocen como *formsets*. Los conjuntos de formularios gestionan múltiples instancias de un determinado Formulario o ModelForm. Todos los formularios se envían a la vez y el formset se encarga del número inicial de formularios para mostrar, limitando el máximo número de formularios que se pueden enviar y validando todos los formularios.

[391]

Página 417

Construyendo una plataforma de aprendizaje electrónico

Los conjuntos de formularios incluyen un método `is_valid()` para validar todos los formularios a la vez. Tú también puedes proporcionar datos iniciales para los formularios y especificar cuántos formularios vacíos adicionales monitor. Puede obtener más información sobre los conjuntos de formularios en <https://docs.djangoproject.com/es/3.0/topics/forms/formsets/> y sobre los conjuntos de formularios modelo en <https://docs.djangoproject.com/en/3.0/topics/forms/modelforms/#model-formsets>.

Dado que un curso se divide en un número variable de módulos, tiene sentido use conjuntos de formularios para administrarlos. Cree un archivo `forms.py` en la aplicación de cursos directorio y agregue el siguiente código:

```
desde formularios de importación de django
de django.forms.models importar inlineformset_factory
de .models import Curso, Módulo

ModuleFormSet = inlineformset_factory (Curso,
                                      Módulo,
                                      campos = ['título',
                                                'descripción'],
                                      extra = 2,
                                      can_delete = True)
```

Este es el conjunto de formularios `ModuleFormSet`. Lo construyes usando el `inlineformset_factory()` proporcionada por Django. Los conjuntos de formularios en línea son una pequeña abstracción además de conjuntos de formularios que simplifican el trabajo con objetos relacionados. Esta función permite crear un conjunto de formularios modelo dinámicamente para los objetos del módulo relacionados con un Objeto del curso.

Utiliza los siguientes parámetros para construir el conjunto de formularios:

- `campos` : los campos que se incluirán en cada formulario del conjunto de formularios.
- `extra` : le permite establecer el número de formularios extra vacíos que se mostrarán en el formset.
- `can_delete` : si establece esto en `True`, Django incluirá un campo booleano para cada formulario que se representará como una entrada de casilla de verificación. Te permite marcar los objetos que desea eliminar.

[392]

Edite el archivo `views.py` de la aplicación de cursos y agregue el siguiente código:

```

desde django.shortcuts import redirect, get_object_or_404
de django.views.generic.base import TemplateResponseMixin, View
desde .forms importar ModuleFormSet

clase CourseModuleUpdateView (TemplateResponseMixin, View):
    template_name = 'cursos / administrar / módulo / formset.html'
    curso = Ninguno

    def get_formset (self, data = None):
        return ModuleFormSet (instancia = self.course,
                               datos = datos)

    def dispatch (self, request, pk):
        self.course = get_object_or_404 (Curso,
                                           id = pk,
                                           propietario = solicitud.usuario)

        return super (). dispatch (solicitud, pk)

    def get (self, request, * args, ** kwargs):
        formset = self.get_formset ()
        return self.render_to_response ({'curso': self.course,
                                         'formset': formset})

    def post (self, request, * args, ** kwargs):
        formset = self.get_formset (datos = solicitud.POST)
        si formset.is_valid ():
            formset.save ()
            return redirect ('manage_course_list')
        return self.render_to_response ({'curso': self.course,
                                         'formset': formset})

```

La vista `CourseModuleUpdateView` maneja el conjunto de formularios para agregar, actualizar y eliminar módulos de un curso específico. Esta vista hereda de los siguientes mixins y vistas:

- `TemplateResponseMixin` : este mixin se encarga de renderizar las plantillas y devolviendo una respuesta HTTP. Requiere un atributo `template_name` que indica la plantilla que se va a renderizar y proporciona el `render_to_response()` para pasarle un contexto y renderizar la plantilla.
- Vista : la vista básica basada en clases proporcionada por Django.

En esta vista, implementa los siguientes métodos:

- `get_formset()` : define este método para evitar repetir el código para compilar el formset. Crea un objeto `ModuleFormSet` para el objeto `Course` dado con datos opcionales.
- `dispatch()` : este método lo proporciona la clase `View`. Se necesita un HTTP solicitud y sus parámetros e intenta delegar a un método en minúsculas que coincide con el método HTTP utilizado. Una solicitud GET se delega en `get()` y una solicitud POST a `post()`, respectivamente. En este método, usas la función de acceso directo `get_object_or_404()` para obtener el objeto `Course` para el dado el parámetro de identificación que pertenece al usuario actual. Incluye este código en el método `dispatch()` porque necesita recuperar el curso tanto para GET y solicitudes POST. Lo guarda en el atributo de curso de la vista para hacer es accesible a otros métodos.
- `get()` : ejecutado para solicitudes GET. Construyes un `ModuleFormSet` vacío formset y renderizarlo en la plantilla junto con el actual Objeto del curso utilizando el método `render_to_response()` proporcionado por `TemplateResponseMixin`.
- `post()` : ejecutado para solicitudes POST.

En este método, realiza las siguientes acciones:

1. Construye una instancia de `ModuleFormSet` utilizando los datos enviados.
2. Ejecuta el método `is_valid()` del formset para validar todos los sus formas.
3. Si el conjunto de formularios es válido, guárdelo llamando al método `save()`. A En este punto, cualquier cambio realizado, como agregar, actualizar o marcar módulos para su eliminación, se aplican a la base de datos. Entonces, redirige usuarios a la URL `manage_course_list`. Si el conjunto de formularios no es válido, renderiza la plantilla para mostrar cualquier error en su lugar.

[394]

Edite el archivo `urls.py` de la aplicación de cursos y agregue el siguiente patrón de URL lo:

```
ruta('<pk> / module /',
     views.CourseModuleUpdateView.as_view(),
     name = 'course_module_update'),
```

Cree un nuevo directorio dentro del directorio y nombre de cursos / administrar / plantilla el módulo. Cree una plantilla de cursos / manage / module / formset.html y agregue la siguiente código:

```
{% extiende "base.html"%}

{% block title%}
    Editar "{{course.title}}"
```

```

{% endblock%}

{% block content%}
<h1> Editar "{{course.title}}" </h1>
<div class = "módulo">
    <h2> Módulos del curso </h2>
    <método de formulario = "publicar">
        {{formset}}
        {{formset.management_form}}
        {% csrf_token%}
        <input type = "submit" value = "Guardar módulos">
    </form>
</div>
{% endblock%}

```

En esta plantilla, crea un elemento HTML <form> en el que incluye formset . También incluye el formulario de administración para el conjunto de formularios con la variable {{formset.management_form}} . El formulario de gestión incluye campos ocultos para controlar el número inicial, total, mínimo y máximo de formularios. Puedes ver que es muy fácil crear un conjunto de formularios.

Edite la plantilla course / manage / course / list.html y agregue el siguiente enlace para la URL course_module_update debajo de los enlaces de edición y eliminación del curso:

```

<a href="{% url 'course_edit' course.id %}"> Editar </a>
<a href="{% url 'course_delete' course.id %}"> Eliminar </a>
<a href="{% url 'course_module_update' course.id %}"> Editar módulos </a>

```

[395]

Ha incluido el enlace para editar los módulos del curso.

Abra <http://127.0.0.1:8000/course/mine/> en su navegador. Crea un curso y haga clic en el enlace **Editar módulos** correspondiente. Debería ver un conjunto de formularios, como sigue:

El conjunto de formularios incluye un formulario para cada objeto de módulo contenido en el curso. Después de estos, se muestran dos formularios adicionales vacíos porque establece `extra = 2` para `ModuleFormSet`. Cuando guarde el conjunto de formularios, Django incluirá otros dos campos adicionales para agregar nuevos módulos.

Agregar contenido a los módulos del curso

Ahora, necesita una forma de agregar contenido a los módulos del curso. Tienes cuatro diferentes tipos de contenido: texto, video, imagen y archivo. Podrías considerar crear cuatro diferentes vistas para crear contenido, con una para cada modelo. Sin embargo, vas para adoptar un enfoque más genérico y crear una vista que maneje la creación o actualización los objetos de cualquier modelo de contenido.

[396]

Edite el archivo `views.py` de la aplicación de cursos y agregue el siguiente código:

```

de django.forms.models import modelform_factory
desde django.apps importar aplicaciones
desde el módulo de importación .models, contenido

class ContentCreateUpdateView (TemplateResponseMixin, View):
    módulo = Ninguno
    modelo = Ninguno
    obj = Ninguno
    template_name = 'cursos / administrar / contenido / formulario.html'

    def get_model (self, model_name):
        si model_name en ['texto', 'video', 'imagen', 'archivo']:
            return apps.get_model (app_label = 'cursos',
                                   model_name = model_name)
        regresar Ninguno

    def get_form (self, model, * args, ** kwargs):
        Form = modelform_factory (model, exclude = ['propietario',
                                                    'orden',
                                                    'creado',
                                                    'actualizado'])
        formulario de retorno (* args, ** kwargs)

    def dispatch (self, request, module_id, model_name, id = None):
        self.module = get_object_or_404 (Módulo,
                                          id = module_id,
                                          course__owner = request.user)
        self.model = self.get_model (nombre_modelo)
        si yo d:
            self.obj = get_object_or_404 (self.model,
                                          id = id,
                                          propietario = solicitud.usuario)
        return super (). dispatch (solicitud, module_id, model_name, id)

```

Esta es la primera parte de `ContentCreateUpdateView`. Te permitirá crear y actualizar los contenidos de los diferentes modelos. Esta vista define los siguientes métodos:

- `get_model ()`: aquí, verifica que el nombre del modelo dado es uno de los cuatro modelos de contenido: texto, video, imagen o archivo. Luego, usas las aplicaciones de Django módulo para obtener la clase real para el nombre de modelo dado. Si el dado El nombre del modelo no es uno de los válidos, devuelve Ninguno.

Construyendo una plataforma de aprendizaje electrónico

- `get_form ()` : construyes un formulario dinámico usando `modelform_factory ()` función del marco del formulario. Dado que va a crear un formulario para los modelos de texto , vídeo , imagen y archivo , utiliza el parámetro de exclusión para especificar los campos comunes a excluir del formulario y dejar que todos los demás los atributos se incluirán automáticamente. Al hacerlo, no tienes que saber qué campos incluir según el modelo.
- `dispatch ()` : recibe los siguientes parámetros de URL y almacena el correspondiente módulo, modelo y objeto de contenido como atributos de clase:
 - `module_id` : el ID del módulo en el que el contenido es / será asociado con.
 - `model_name` : el nombre del modelo del contenido para crear / actualizar.
 - `id` : el ID del objeto que se está actualizando. No es nada para crear nuevos objetos.

Agregue los siguientes métodos `get ()` y `post ()` a `ContentCreateUpdateView` :

```
def get(self, request, module_id, model_name, id = None):
    formulario = self.get_form (self.model, instancia = self.obj)
    return self.render_to_response ({'formulario': formulario,
                                    'objeto': self.obj})

def post(self, request, module_id, model_name, id = None):
    formulario = self.get_form (self.model,
                                instancia = self.obj,
                                data = request.POST,
                                files = request.FILES)

    si form.is_valid ():
        obj = form.save (commit = False)
        obj.owner = request.user
        obj.save ()
        si no es id:
            # nuevo contenido
            Content.objects.create (módulo = self.module,
                                    item = obj)

        return redirect ('module_content_list', self.module.id)

    return self.render_to_response ({'formulario': formulario,
                                    'objeto': self.obj})
```

Estos métodos son los siguientes:

- `get ()` : se ejecuta cuando se recibe una solicitud GET . Tu construyes el modelo formulario para la instancia de texto , vídeo , imagen o archivo que se está actualizando. De lo contrario, no pasa ninguna instancia para crear un nuevo objeto, ya que `self.obj` es `None` si no se proporciona una identificación.

- `post()` : se ejecuta cuando se recibe una solicitud POST . Tu construyes el modelo formulario, pasándole los datos y archivos enviados. Luego, lo valida. Si el formulario es válido, crea un nuevo objeto y asigna `request.user` como su propietario antes de guardarlo en la base de datos. Comprueba el parámetro `id` . Si no hay ID siempre que sepa que el usuario está creando un nuevo objeto en lugar de actualizar un existente. Si se trata de un objeto nuevo, crea un objeto de contenido para el módulo y asocia el nuevo contenido con él.

Edita el archivo `urls.py` de la aplicación de cursos y agregue los siguientes patrones de URL lo:

```
ruta ('módulo / <int: module_id> / content / <model_name> / create /',
      views.ContentCreateUpdateView.as_view (),
      nombre = 'module_content_create'),

ruta ('module / <int: module_id> / content / <model_name> / <id> /',
      views.ContentCreateUpdateView.as_view (),
      name = 'module_content_update'),
```

Los nuevos patrones de URL son los siguientes:

- `module_content_create` : para crear nuevos objetos de texto, video, imagen o archivo y agréguelos a un módulo. Incluye el `module_id` y `nombre_del_modelo` parámetros. El primero permite vincular el nuevo objeto de contenido al dado módulo. Este último especifica el modelo de contenido para el que se creará el formulario.
- `module_content_update` : para actualizar un texto, video, imagen o archivo existente objeto. Incluye los `MODULE_ID` y `MODEL_NAME` parámetros y un ID parámetro para identificar el contenido que se está actualizando.

Cree un nuevo directorio dentro del directorio `cursos / administrar / plantilla` y asígnele un nombre contenido . Cree la plantilla `cursos / manage / content / form.html` y agregue el siguiente código:

```
{% extiende "base.html"%}

{% block title%}
  {% if object%}
    Editar contenido "{{object.title}}"
  {% else%}
    Agregar contenido nuevo
  {% terminara si %}
{% endblock%}

{% block content%}
  <h1>
```

[399]

Construyendo una plataforma de aprendizaje electrónico

```
{% if object%}
  Editar contenido "{{object.title}}"
{% else%}
  Agregar contenido nuevo
{% terminara si %}
</h1>
<div class = "módulo">
  <h2> Información del curso </h2>
  <form action = "" method = "post" enctype = "multipart / form-data">
    {{form.as_p}}
    {% csrf_token%}
```

```

        <p> <input type = "submit" value = "Guardar contenido"> </p>
    </form>
</div>
{% endblock%}

```

Esta es la plantilla para la vista `ContentCreateUpdateView` . En esta plantilla, compruebe si una variable de objeto está en el contexto. Si el objeto existe en el contexto, está actualizando un objeto existente. De lo contrario, está creando un nuevo objeto.

Incluya `enctype = "multipart / form-data"` en el elemento HTML `<form>` porque el formulario contiene una carga de archivo para los modelos de contenido de archivo e imagen .

Ejecute el servidor de desarrollo, abra `http://127.0.0.1:8000/course/mine/` , haga clic en **Edite módulos** para un curso existente y cree un módulo. Abra el shell de Python con el comando `python manage.py shell` y obtenga el ID del más reciente módulo creado, de la siguiente manera:

```

>>> del módulo de importación de cursos.
>>> Module.objects.latest('id').id
6

```

Ejecute el servidor de desarrollo y abra `http://127.0.0.1:8000/course/module/6/content / image / create /` en su navegador, reemplazando el ID del módulo con el que usted obtenido antes. Verá el formulario para crear un objeto Imagen , de la siguiente manera:

[400]

Figura 10.10: Formulario de adición de contenido de imagen del curso

No envíe el formulario todavía. Si intenta hacerlo, fallará porque no ha definido la URL `module_content_list` todavía. Lo vas a crear en un momento.

También necesita una vista para eliminar contenido. Edite el archivo `views.py` de los cursos aplicación y agregue el siguiente código:

```

class ContentDeleteView (Ver):

    def post (self, request, id):
        content = get_object_or_404 (Contenido,
                                     id = id,
                                     module__course__owner = request.user)

        module = content.module
        content.item.delete ()
        content.delete ()
        return redirect ('module_content_list', module.id)

```

[401]

Construyendo una plataforma de aprendizaje electrónico

La clase ContentDeleteView recupera el objeto Content con el ID proporcionado. Eso elimina el objeto de texto , vídeo , imagen o archivo relacionado . Finalmente, elimina el Contenido objeto y redirige al usuario a la URL module_content_list para enumerar el otro contenido del módulo.

Edite el archivo urls.py de la aplicación de cursos y agregue el siguiente patrón de URL lo:

```

ruta ('contenido / <int: id> / eliminar /',
     views.ContentDeleteView.as_view (),
     name = 'module_content_delete'),

```

Ahora los instructores pueden crear, actualizar y eliminar contenido fácilmente.

Gestión de módulos y sus contenidos

Ha creado vistas para crear, editar y eliminar módulos del curso y sus contenidos.

A continuación, necesita una vista para mostrar todos los módulos de un curso y enumerar el contenido de un módulo específico.

Edite el archivo views.py de la aplicación de cursos y agregue el siguiente código:

```

class ModuleContentListView (TemplateResponseMixin, View):
    template_name = 'cursos / administrar / module / content_list.html'

    def get (self, request, module_id):
        module = get_object_or_404 (Módulo,
                                    id = module_id,
                                    course__owner = request.user)

        return self.render_to_response ({'módulo': módulo})

```

Esta es la vista ModuleContentListView . Esta vista obtiene el objeto Módulo con el ID dado que pertenece al usuario actual y genera una plantilla con el módulo.

Edite el archivo urls.py de la aplicación de cursos y agregue el siguiente patrón de URL lo:

```

ruta ('módulo / <int: module_id> /',
     views.ModuleContentListView.as_view (),
     name = 'module_content_list'),

```

Cree una nueva plantilla dentro del directorio templates / ourses / manage / module /

y asígnele el nombre `content_list.html` . Agregue el siguiente código:

[402]

```
{% extiende "base.html"%}

{% block title%}
    Módulo {{module.order | add: 1}}: {{module.title}}
{% endblock%}

{% block content%}
{% with course = module.course%}
<h1> Curso "{{course.title}}" </h1>
<div class = "contenido">
    <h3> Módulos </h3>
    <ul id = "módulos">
        {% para m en course.modules.all%}
        <li data-id = "{{m.id}}" {% if m == módulo%}
            class = "selected" {% endif%}>
            <a href="{{% url "module_content_list" m.id %}}">
                <span>
                    Módulo <span class = "order"> {{m.order | add: 1}} </span>
                </span>
                <br>
                {{m.title}}
            </a>
        </li>
        {% vacío%}
        <li> Aún no hay módulos. </li>
        {% endfor%}
    </ul>
    <p> <a href="{{% url "course_module_update" course.id %}}">
        Editar módulos </a> </p>
</div>
<div class = "módulo">
    <h2> Módulo {{module.order | add: 1}}: {{module.title}} </h2>
    <h3> Contenido del módulo: </h3>

    <div id = "contenido del módulo">
        {% para el contenido en module.contents.all%}
        <div data-id = "{{content.id}}">
            {% con item = content.item%}
            <p> {{item}} </p>
            <a href="#"> Editar </a>
            <form action = "{{% url "module_content_delete" content.id
%}} "
                método = "publicar">
                <input type = "submit" value = "Delete">
                {% csrf_token%}
            </form>
            {% terminar con %}
```

[403]

```
        </div>
        {% vacío%}
        <p> Este módulo aún no tiene contenido. </p>
        {% endfor%}
    </div>
    <h3> Agregar contenido nuevo: </h3>
    <ul class = "tipos de contenido">
        <li> <a href = "{% url" module_content_create "module.id" text "
    {%} ">
        Texto </a> </li>
        <li> <a href = "{% url" module_content_create "module.id" image "
    {%} ">
        Imagen </a> </li>
        <li> <a href = "{% url" module_content_create "module.id" video "
    {%} ">
        Video </a> </li>
        <li> <a href = "{% url" module_content_create "module.id" file "
    {%} ">
        Archivo </a> </li>
    </ul>
</div>
{% terminar con %}
{% endblock%}
```

Asegúrese de que ninguna etiqueta de plantilla se divida en varias líneas.

Esta es la plantilla que muestra todos los módulos de un curso y el contenido del módulo seleccionado. Repetirás los módulos del curso para mostrarlos en una barra lateral.

Se itera sobre el contenido de un módulo y se accede a `content.item` para obtener el

Objeto de texto , video , imagen o archivo . También incluye enlaces para crear texto, video, imagen o contenido de archivo.

Desea saber qué tipo de objeto es cada uno de los objetos de elemento : texto , video ,

Imagen o Archivo . Necesita el nombre del modelo para crear la URL para editar el objeto. Además esto, puede mostrar cada elemento en la plantilla de manera diferente según el tipo de contenido que es. Puede obtener el nombre del modelo para un objeto de la clase Meta del modelo mediante acceder al atributo `_meta` del objeto . Sin embargo, Django no permite acceder variables o atributos que comienzan con un guión bajo en las plantillas para evitar la recuperación atributos privados o llamar a métodos privados. Puede resolver esto escribiendo una filtro de plantilla.

Cree la siguiente estructura de archivos dentro del directorio de la aplicación de cursos :

```
templatetags /
__init__.py
course.py
```

[404]

Edite el módulo `course.py` y agréguele el siguiente código:

desde la plantilla de importación de django

```
registrarse = template.Library()
```

```
@ register.filter
```

```
def nombre_modelo (obj):
```

```

    tratar:
        return obj._meta.model_name
    excepto AttributeError:
        regresar Ninguno

```

Este es el filtro de plantilla `model_name` . Puede aplicarlo en plantillas como `object | model_name` para obtener el nombre del modelo de un objeto.

Edite la plantilla `templates /ourses / manage / module / content_list.html` y agregue la siguiente línea debajo de la etiqueta de plantilla `{% extensions%` :

```
{% load course%
```

Esto cargará las etiquetas de la plantilla del curso . Luego, busque las siguientes líneas:

```

<p> {{item}} </p>
<a href="#"> Editar </a>

```

Reemplácelos por los siguientes:

```

<p> {{item}} ({{item | model_name}}) </p>
<a href = "% url " module_content_update "module.id item | model_name
item.id%" " >
    Editar
</a>

```

En el código anterior, muestra el nombre del modelo del artículo en la plantilla y también usa el nombre del modelo para construir el enlace para editar el objeto.

Edite la plantilla `courses / administrar / curso / list.html` y agregue un enlace a la

URL `module_content_list` , como esta:

```

<a href="{% url "course_module_update" course.id %}"> Editar módulos </a>
{% si course.modules.count> 0%}
    <a href="{% url "module_content_list" course.modules.first.id %}">
        Administrar contenidos </a>
{% terminara si %}

```

El nuevo enlace permite a los usuarios acceder a los contenidos del primer módulo del curso, si hay alguno.

[405]

Construyendo una plataforma de aprendizaje electrónico

Detenga el servidor de desarrollo y ejecútelo nuevamente usando el comando `python manage.`

`py runserver` . Al detener y ejecutar el servidor de desarrollo, se asegura que se cargue el archivo de etiquetas de la plantilla del curso .

Abra `http://127.0.0.1:8000/course/mine/` y haga clic en **Administrar contenido** enlace para un curso que contiene al menos un módulo. Verá una página como la siguiente:

Figura 10.11: La página para administrar los contenidos del módulo del curso

Cuando hace clic en un módulo en la barra lateral izquierda, su contenido se muestra en la zona. La plantilla también incluye enlaces para agregar nuevo texto, video, imagen o contenido de archivo para

el módulo que se muestra.

Agregue un par de tipos diferentes de contenido al módulo y observe el resultado.

El contenido del módulo aparecerá debajo **del contenido del módulo** :

Figura 10.12: Gestión de diferentes contenidos de módulo

[406]

Reordenación de módulos y su contenido

Debe proporcionar una forma sencilla de reordenar los módulos del curso y sus contenidos. Tú utilizará un widget de arrastrar y soltar de JavaScript para permitir a sus usuarios reordenar los módulos de un curso arrastrándolos. Cuando los usuarios terminen de arrastrar un módulo, iniciará una solicitud asíncrona (AJAX) para almacenar el nuevo pedido del módulo.

Usando mixins de django-braces

django-braces es un módulo de terceros que contiene una colección de mixins genéricos para Django. Estos mixins proporcionan características adicionales para vistas basadas en clases. Usted puede ver una lista de todos los mixins proporcionados por django-braces en <https://django-braces.readthedocs.io/>.

Utilizará las siguientes combinaciones de django-braces :

- **CsrfExemptMixin** : se utiliza para evitar comprobar la **falsificación de solicitudes entre sitios (CSRF)** en las solicitudes POST . Necesita esto para realizar AJAX POST solicitudes sin tener que generar un csrf_token .
- **JsonRequestResponseMixin** : analiza los datos de la solicitud como JSON y también serializa la respuesta como JSON y devuelve una respuesta HTTP con el tipo de contenido application / json .

Instale django-braces a través de pip usando el siguiente comando:

pip install django-braces == 1.14.0

Necesita una vista que reciba el nuevo orden de ID de módulo codificado en JSON. Edite el archivo views.py de la aplicación de cursos y agregue el siguiente código:

```
desde braces.views importar CsrfExemptMixin, JsonRequestResponseMixin

class ModuleOrderView (CsrfExemptMixin,
                        JsonRequestResponseMixin,
                        Ver):

    def post (auto, solicitud):
        para id, ordene self.request_json.items ():
            Module.objects.filter (id = id,
                                   course__owner = request.user) .update (order = order)
        return self.render_json_response ({'guardado': 'OK'})
```

Construyendo una plataforma de aprendizaje electrónico

Puede crear una vista similar para ordenar el contenido de un módulo. Agrega el siguiente código al archivo views.py :

```
class ContentOrderView (CsrfExemptMixin,
                        JsonRequestResponseMixin,
                        Ver):
    def post (auto, solicitud):
        para id, ordene self.request_json.items ():
            Content.objects.filter (id = id,
                                    module__course__owner = request.user) \
                                .update (orden = orden)
        return self.render_json_response ({'guardado': 'OK'})
```

Ahora, edite el archivo urls.py de la aplicación de cursos y agregue lo siguiente Patrones de URL a él:

```
ruta ('módulo / pedido /',
      views.ModuleOrderView.as_view (),
      name = 'module_order'),

ruta ('contenido / orden /',
      views.ContentOrderView.as_view (),
      name = 'content_order'),
```

Finalmente, debe implementar la funcionalidad de arrastrar y soltar en la plantilla. Utilizará la biblioteca jQuery UI para esto. La interfaz de usuario de jQuery está construida sobre jQuery y proporciona un conjunto de interacciones, efectos y widgets de interfaz. Usarás su elemento ordenable . Primero, debe cargar jQuery UI en la plantilla base. Abre el archivo base.html ubicado en el directorio templates / de la aplicación de cursos , y agregue jQuery UI debajo del script para cargar jQuery, de la siguiente manera:

```
<script src = "https://ajax.googleapis.com/ajax/libs/jquery/3.4.1/
jquery.min.js "> </script>
<script src = "https://ajax.googleapis.com/ajax/libs/jqueryui/1.12.1/
jquery-ui.min.js "> </script>
```

Carga la biblioteca jQuery UI justo debajo del marco jQuery. A continuación, edite el cursos / manage / module / content_list.html y agregue el siguiente código a él en la parte inferior de la plantilla:

```
{% block domready%}
$ ('# módulos'). sortable ({
    detener: función (evento, ui) {
        módulos_orden = {};
        $ ('# módulos'). hijos (). cada (función () {
            // actualiza el campo de la orden
```

```

    $(esto).find('.orden').texto($(esto).index() + 1);
    // asocia la identificación del módulo con su orden
    module_order[$(esto).data('id')] = $(esto).index();
  });
  $.ajax({
    tipo: 'POST',
    url: '{% url "module_order"%}',
    contentType: 'aplicación / json; charset = utf-8 ',
    dataType: 'json',
    datos: JSON.stringify(modules_order)
  });
}
});

$($('#module-contents').sortable({
  detener: función(evento, ui) {
    orden_de_contenidos = {};
    $($('#contenido-módulo').hijos().cada(función() {
      // asocia la identificación del módulo con su orden
      orden_contenido[$(esto).data('id')] = $(esto).index();
    }));

    $.ajax({
      tipo: 'POST',
      url: '{% url "content_order"%}',
      contentType: 'aplicación / json; charset = utf-8 ',
      dataType: 'json',
      datos: JSON.stringify(contenido_orden),
    });
  }
}));
{% endblock%}

```

Este código JavaScript está en el bloque {% block domready%} y, por lo tanto, incluirse en el evento \$(document).ready() de jQuery que definiste en el plantilla base.html. Esto garantiza que se ejecutará su código JavaScript una vez cargada la página.

Define un elemento ordenable para la lista de módulos en la barra lateral y uno diferente para la lista de contenidos del módulo. Ambos funcionan de manera similar.

En este código, realiza las siguientes tareas:

1. Define un elemento clasificable para el elemento HTML de los módulos. Recuerda que usa #modules, ya que jQuery usa notación CSS para los selectores.
2. Especifica una función para el evento de parada. Este evento se activa cada vez

el usuario termina de ordenar un elemento.

3. Crea un diccionario de `modules_order` vacío. Las claves de este diccionario serán los ID del módulo, y los valores serán el orden asignado para cada módulo.
4. Se itera sobre los elementos secundarios `#module`. Vuelve a calcular el muestra el orden de cada módulo y obtiene su atributo `data-id`, que contiene el ID del módulo. Agrega el ID como la clave de `modules_order` diccionario y el nuevo índice del módulo como valor.
5. Lanzas una solicitud POST AJAX a la URL `content_order`, incluida la datos JSON serializados de `modules_order` en la solicitud. El correspondiente `ModuleOrderView` se encarga de actualizar el orden de los módulos.

El elemento clasificable para ordenar los contenidos del módulo es bastante similar a este. Regresa a su navegador y vuelva a cargar la página. Ahora podrá hacer clic y arrastrar ambos módulos y su contenido para reordenarlos como en el siguiente ejemplo:

Figura 10.13: Reordenación de módulos con la función de arrastrar y soltar

¡Excelente! Ahora puede reordenar tanto los módulos del curso como los contenidos del módulo.

[410]

Resumen

En este capítulo, aprendió a usar accesorios para proporcionar datos iniciales para modelos. Por utilizando la herencia de modelos, creó un sistema versátil para administrar diferentes tipos de contenido para los módulos del curso. Implementó un campo de modelo personalizado para ordenar objetos. También descubrió cómo usar vistas y mixins basados en clases. Trabajaste con grupos y permisos para restringir el acceso a sus vistas. Finalmente, usaste conjuntos de formularios para administrar los módulos del curso, y creó una funcionalidad de arrastrar y soltar con jQuery UI para reordenar los módulos y su contenido.

En el próximo capítulo, creará un sistema de registro de estudiantes. También lo harás renderizar diferentes tipos de contenido, y aprenderá a trabajar con Django marco de caché.

11

Renderizado y Almacenamiento en caché de con1

En el capítulo anterior, usó la herencia del modelo y las relaciones genéricas para crear modelos de contenido de cursos flexibles. Implementó un campo de modelo personalizado y construyó un sistema de gestión de cursos utilizando vistas basadas en clases. Finalmente, creaste un Funcionalidad de arrastrar y soltar basada en AJAX para ordenar los módulos del curso y sus contenidos.

En este capítulo, desarrollará la funcionalidad para acceder a los contenidos del curso, crear un sistema de registro de estudiantes y administrar la inscripción de estudiantes en los cursos. También aprenderá cómo almacenar datos en caché usando el marco de caché de Django.

En este capítulo, podrá:

- Cree vistas públicas para mostrar información del curso
- Construir un sistema de registro de estudiantes
- Gestionar la inscripción de estudiantes en los cursos.
- Representar contenido diverso para los módulos del curso

- Instalar y configurar Memcached
- Caché de contenido usando el marco de caché de Django
- Monitorear Memcached usando el django-memcache-status

Comencemos por crear un catálogo de cursos para que los estudiantes exploren los cursos existentes y inscríbete en ellos.

[413]

Procesamiento y almacenamiento en caché de contenido

Visualización de cursos

Para su catálogo de cursos, debe crear las siguientes funcionalidades:

- Lista de todos los cursos disponibles, opcionalmente filtrados por tema
- Mostrar una descripción general de un solo curso

Edite el archivo `views.py` de la aplicación de cursos y agregue el siguiente código:

```

de django.db.models import Count
de .models import Asunto

clase CourseListView (TemplateResponseMixin, View):
    modelo = Curso
    template_name = 'cursos / curso / list.html'

    def get (self, request, subject = None):
        sujetos = Subject.objects.annotate (
            total_courses = Count ('cursos'))
        cursos = Course.objects.annotate (
            total_modules = Count ('módulos'))

        si el sujeto:
            subject = get_object_or_404 (Asunto, slug = asunto)
            cursos = cursos.filtro (asunto = asunto)
        return self.render_to_response ({'sujetos': sujetos,
                                         'sujeto': sujeto,
                                         'cursos': cursos})

```

Esta es la vista `CourseListView` . Hereda de `TemplateResponseMixin` y

`Ver` . En esta vista, realiza las siguientes tareas:

1. Recupera todos los sujetos, utilizando el método `annotate ()` del ORM con el Función de agregación `Count ()` para incluir el número total de cursos para cada tema
2. Recupera todos los cursos disponibles, incluido el número total de módulos. contenido en cada curso
3. Si se proporciona un parámetro de URL de slug de asunto, recupera el correspondiente objeto de la asignatura y limitar la consulta a los cursos que pertenecen al tema
4. Utiliza el método `render_to_response ()` proporcionado por `TemplateResponseMixin` para renderizar los objetos en una plantilla y devolver un Respuesta HTTP

[414]

Creemos una vista detallada para mostrar una descripción general de un solo curso. Agregue lo siguiente código al archivo `views.py` :

```

de django.views.generic.detail importar DetailView

clase CourseDetailView (DetailView):
    modelo = Curso
    template_name = 'cursos / curso / detalle.html'

```

Esta vista hereda del `DetailView` genérico proporcionado por Django. Tu especificas los atributos `model` y `template_name`. `DetailView` de Django espera un `primario key ( pk )` o parámetro `slug URL` para recuperar un solo objeto para el modelo dado. los `view` representa la plantilla especificada en `template_name`, incluido el objeto `Course` en el objeto de variable de contexto de plantilla.

Editar el principal `urls.py` archivo de la Educa proyecto y añadir el patrón siguiente URL lo:

```

de cursos.vistas importar CourseListView

urlpatterns = [
    # ...
    ruta ('', CourseListView.as_view (), name = 'course_list'),
]

```

Se agrega el `course_list` patrón de URL a la principal `urls.py` archivo de el proyecto porque desea mostrar la lista de cursos en la URL

`http://127.0.0.1:8000/`, y todas las demás URL para la aplicación de cursos tienen el prefijo `/curso /`.

Edite el archivo `urls.py` de la aplicación de cursos y agregue los siguientes patrones de URL:

```

ruta ('sujeto / <slug: sujeto> /',
     views.CourseListView.as_view (),
     name = 'course_list_subject'),

ruta ('<slug: slug> /',
     views.CourseDetailView.as_view (),
     name = 'course_detail'),

```

Defina los siguientes patrones de URL:

- `course_list_subject` : para mostrar todos los cursos de un tema
- `course_detail` : para mostrar una descripción general de un solo curso

Cree la siguiente estructura de archivos dentro de las plantillas / cursos / directorio del solicitud de cursos :

```
curso/
  list.html
  detail.html
```

Edite la plantilla de cursos / curso / list.html de la aplicación de cursos y escriba el siguiente código:

```
{% extiende "base.html"%}

{% block title%}
  {% si el sujeto%}
    Cursos de {{subject.title}}
  {% else%}
    Todos los cursos
  {% terminara si %}
{% endblock%}

{% block content%}
<h1>
  {% si el sujeto%}
    Cursos de {{subject.title}}
  {% else%}
    Todos los cursos
  {% terminara si %}
</h1>
<div class = "contenido">
  <h3> Sujetos </h3>
  <ul id = "módulos">
    <li {% if not subject%} class = "selected" {% endif%}>
      <a href="{% url "course_list" %}"> Todo </a>
    </li>
    {% para s en sujetos%}
      <li {% if subject == s%} class = "selected" {% endif%}>
        <a href="{% url "course_list_subject" s.slug %}">
          {{s.title}}
          <br> <span> {{s.total_courses}} cursos </span>
        </a>
      </li>
    {% endfor%}
  </ul>
```

[416]

```
</div>
<div class = "módulo">
  {% por curso en cursos%}
    {% with subject = course.subject%}
      <h3>
        <a href="{% url "course_detail" course.slug %}">
          {{ título del curso }}
        </a>
      </h3>
      <p>
        <a href="{% url "course_list_subject" subject.slug %}"> {{
asunto}} </a>.
        Módulos de {{course.total_modules}}.
        Instructor: {{course.owner.get_full_name}}
      </p>
    {% terminar con %}
  {% endfor%}
</div>
{% endblock%}
```


Asegúrese de que ninguna etiqueta de plantilla se divida en varias líneas.

Esta es la plantilla para enumerar los cursos disponibles. Creas una lista HTML para mostrar todos los objetos Subject y crear un enlace a la URL `course_list_subject` para cada uno de ellos. Agrega una clase HTML seleccionada para resaltar el tema actual si uno se selecciona el tema. Se itera sobre cada objeto del curso , mostrando el número total de módulos y el nombre del instructor.

Ejecute el servidor de desarrollo y abra `http://127.0.0.1:8000/` en su navegador. Debería ver una página similar a la siguiente:

Figura 11.1: La página de la lista de cursos

[417]

Procesamiento y almacenamiento en caché de contenido

La barra lateral izquierda contiene todas las materias, incluido el número total de cursos para cada de ellos. Puede hacer clic en cualquier tema para filtrar los cursos que se muestran.

Edite la plantilla `course / course / detail.html` y agréguele el siguiente código:

```
{% extiende "base.html"%}

{% block title%}
  {{object.title}}
{% endblock%}

{% block content%}
  {% with subject = object.subject%}
    <h1>
      {{object.title}}
    </h1>
    <div class = "módulo">
      <h2> Descripción general </h2>
      <p>
        <a href="{% url 'course_list_subject' subject.slug %}">
          {{subject.title}} </a>.
          Módulos {{object.modules.count}}.
          Instructor: {{object.owner.get_full_name}}
      </p>
      {{object.overview | linebreak}}
    </div>
  {% terminar con %}
{% endblock%}
```

En esta plantilla, muestra la descripción general y los detalles de un solo curso. Abierto `http://127.0.0.1:8000/` en su navegador y haga clic en uno de los cursos. Tú debería ver una página con la siguiente estructura:

Figura 11.2: La página de descripción general del curso

[418]

Ha creado un área pública para mostrar los cursos. A continuación, debe permitir a los usuarios para registrarse como estudiantes e inscribirse en cursos.

Agregar registro de estudiantes

Cree una nueva aplicación usando el siguiente comando:

estudiantes de python manage.py startapp

Edite el archivo settings.py del proyecto educa y agregue la nueva aplicación al Configuración INSTALLED_APPS , de la siguiente manera:

```
INSTALLED_APPS = [  
    # ...  
    'Students.apps.StudentsConfig',  
]
```

Crear una vista de registro de estudiantes

Edite el archivo views.py de la aplicación de los estudiantes y escriba el siguiente código:

```
de django.urls import reverse_lazy  
desde django.views.generic.edit import CreateView  
de django.contrib.auth.forms import UserCreationForm  
desde django.contrib.auth import autenticar, iniciar sesión  
  
class StudentRegistrationView (CreateView):  
    template_name = 'estudiantes / estudiante / registro.html'  
    form_class = UserCreationForm  
    Success_url = reverse_lazy ('lista de cursos_estudiantes')  
  
    def form_valid (self, form):  
        resultado = super (). form_valid (formulario)  
        cd = form.cleaned_data  
        usuario = autenticar (nombre de usuario = cd ['nombre de usuario'],  
                               contraseña = cd ['contraseña1'])  
        iniciar sesión (self.request, usuario)  
        devolver resultado
```

Esta es la vista que permite a los estudiantes registrarse en su sitio. Usas el genérico CreateView , que proporciona la funcionalidad para crear objetos de modelo. Esta vista requiere los siguientes atributos:

- template_name : la ruta de la plantilla para representar esta vista.

[419]

Procesamiento y almacenamiento en caché de contenido

- `form_class` : El formulario para crear objetos, que tiene que ser `ModelForm` .
Utiliza `UserCreationForm` de Django como formulario de registro para crear
Objetos de usuario .
- `success_url` : la URL a la que redirigir al usuario cuando el formulario se haya completado correctamente
presentada. Invierte la URL llamada `student_course_list` , que
que creará en la sección *Acceso al contenido del curso* para enumerar
cursos en los que están inscritos los estudiantes.

El método `form_valid()` se ejecuta cuando se han publicado datos de formulario válidos. Tiene
para devolver una respuesta HTTP. Anula este método para iniciar la sesión del usuario después de que
se ha registrado correctamente.

Crear un nuevo archivo dentro de la estudiantes directorio de la aplicación y el nombre `urls.py` .
Agregue el siguiente código:

```
desde la ruta de importación django.urls
de . importar vistas

urlpatterns = [
    ruta ('registrar /',
        views.StudentRegistrationView.as_view (),
        nombre = 'registro_estudiante'),
]
```

A continuación, edite el principal `urls.py` de la Educa proyecto e incluir las direcciones URL de la
aplicación para estudiantes agregando el siguiente patrón a la configuración de su URL:

```
urlpatterns = [
    # ...
    ruta ('estudiantes /', incluir ('estudiantes.urls')),
]
```

Cree la siguiente estructura de archivos dentro del directorio de la aplicación de los estudiantes :

```
plantillas/
  estudiantes /
    estudiante/
      registration.html
```

Edite la plantilla `Students / student / registration.html` y agregue lo siguiente
codificarlo:

```
{% extiende "base.html"%}

{% block title%}
  Regístrate
```

```

{% endblock%}

{% block content%}
<h1>
  Registrate
</h1>
<div class = "módulo">
  <p> Ingrese sus datos para crear una cuenta: </p>
  <método de formulario = "publicar">
    {{form.as_p}}
    {% csrf_token%}
    <p> <input type = "submit" value = "Crear mi cuenta"> </p>
  </form>
</div>
{% endblock%}

```

Ejecute el servidor de desarrollo y abra `http://127.0.0.1:8000/students/` registrarse / en su navegador. Debería ver un formulario de registro como este:

Figura 11.3: El formulario de registro de estudiantes

[421]

Procesamiento y almacenamiento en caché de contenido

Tenga en cuenta que la URL `student_course_list` especificada en el atributo `success_url` de la vista `StudentRegistrationView` aún no existe. Si envía el formulario, Django no encontrará la URL a la que lo redireccionará después de un registro exitoso. Como mencionado, creará esta URL en la sección *Acceso al contenido del curso* .

Inscribirse en cursos

Una vez que los usuarios crean una cuenta, deberían poder inscribirse en los cursos. En orden para almacenar inscripciones, debe crear una relación de varios a varios entre Modelos de curso y usuario .

Edite el archivo `models.py` de la aplicación de cursos y agregue el siguiente campo al Modelo de curso :

```

estudiantes = modelos.ManyToManyField (Usuario,
related_name = 'cursos_unidos',
en blanco = Verdadero)

```

Desde el shell, ejecute el siguiente comando para crear una migración para este cambio:

```
python manage.py makemigrations
```

Verá una salida similar a esta:

Migraciones para 'cursos':

```
cursos / migraciones / 0004_course_students.py
```

```
- Agregar estudiantes de campo al curso
```

Luego, ejecute el siguiente comando para aplicar las migraciones pendientes:

```
python manage.py migrar
```

Debería ver un resultado que termina con la siguiente línea:

Aplicando cursos.0004_course_students ... OK

Ahora puede asociar a los estudiantes con los cursos en los que están inscritos. Vamos crear la funcionalidad para que los estudiantes se inscriban en cursos.

Cree un nuevo archivo dentro del directorio de la aplicación de los estudiantes y asigne el nombre `forms.py` . Agregue el siguiente código:

```

desde formularios de importación de django
de cursos.modelos importar curso

clase CourseEnrollForm (formularios.Formulario):
    curso = formularios.ModelChoiceField (queryset = Course.objects.all (),
widget = formularios.HiddenInput)

```

[422]

Utilizará este formulario para que los estudiantes se inscriban en los cursos. El campo del curso es para el curso en el que se matriculará el usuario; por lo tanto, es un `ModelChoiceField` . Utiliza un widget `HiddenInput` porque no va a mostrar este campo a el usuario. Utilizará este formulario en la vista `CourseDetailView` para mostrar un botón para inscribirse.

Edite el archivo `views.py` de la aplicación de los estudiantes y agregue el siguiente código:

```

desde django.views.generic.edit importar FormView
de django.contrib.auth.mixins importar LoginRequiredMixin
desde .forms importar CourseEnrollForm

clase StudentEnrollCourseView (LoginRequiredMixin, FormView):
    curso = Ninguno
    form_class = CourseEnrollForm

    def form_valid (self, form):
        self.course = form.cleaned_data ['curso']
        self.course.students.add (self.request.user)
        return super (). form_valid (formulario)

    def get_success_url (yo):
        return reverse_lazy ('student_course_detail',
args = [self.course.id])

```

Esta es la vista `StudentEnrollCourseView` . Maneja a los estudiantes que se inscriben en cursos. La vista hereda del mixin `LoginRequiredMixin` de modo que solo los usuarios registrados pueden acceder a la vista. También hereda de la vista `FormView` de Django , ya que maneja el envío de un formulario. Utiliza el formulario `CourseEnrollForm` para

objetos cuando se llama a su método `get_success_url()` de curso para almacenar los datos en el curso.

El método `get_success_url()` devuelve la URL a la que se redirigirá al usuario a si el formulario se envió correctamente. Este método es equivalente al éxito_ atributo `url`. Luego, invierte la URL llamada `student_course_detail`.

Edite el archivo `urls.py` de la aplicación de los estudiantes y agregue el siguiente patrón de URL lo:

```
ruta ('curso de inscripción /',
     views.StudentEnrollCourseView.as_view (),
     name = 'estudiante_enroll_course'),
```

[423]

Procesamiento y almacenamiento en caché de contenido

Agreguemos el formulario del botón de inscripción a la página de descripción general del curso. Edite las vistas.py de la aplicación de cursos y modifique `CourseDetailView` para que parezca como sigue:

```
de Students.forms importar CourseEnrollForm

clase CourseDetailView (DetailView):
    modelo = Curso
    template_name = 'cursos / curso / detalle.html'

    def get_context_data (self, ** kwargs):
        context = super (). get_context_data (** kwargs)
        context ['enroll_form'] = CourseEnrollForm (
            initial = {'curso': self.object})

    devolver contexto
```

Utiliza el método `get_context_data()` para incluir el formulario de inscripción en el contexto para renderizar las plantillas. Inicializa el campo de curso oculto del formulario con el objeto del curso actual para que pueda enviarse directamente.

Edite la plantilla `course / course / detail.html` y localice la siguiente línea:

```
{{object.overview | linebreak}}
```

Reemplázelo con el siguiente código:

```
{{object.overview | linebreak}}
{% if request.user.is_authenticated%}
    <form action = "{% url "student_enroll_course" %}" method = "post">
        {{enroll_form}}
        {% csrf_token%}
        <input type = "submit" value = "Inscribase ahora">
    </form>
{% else%}
    <a href="{% url "student_registration" %}" class="button">
        Regístrese para inscribirse
    </a>
{% terminara si %}
```

Este es el botón para inscribirse en cursos. Si el usuario está autenticado, muestra el botón de inscripción, incluido el formulario oculto que apunta al alumno_

URL del curso de inscripción. Si el usuario no está autenticado, muestra un enlace para registrarse en la plataforma.

Asegúrese de que el servidor de desarrollo se esté ejecutando, abra `http://127.0.0.1:8000/` en su navegador y haga clic en un curso. Si ha iniciado sesión, debería ver una **INSCRIPCIÓN** Botón **AHORA** ubicado debajo de la descripción general del curso, de la siguiente manera:

Figura 11.4: La página de descripción general del curso, incluido el botón INSCRIBIRSE AHORA

Si no ha iniciado sesión, verá un botón **REGISTRARSE PARA INSCRIBIRSE** .

Acceder a los contenidos del curso

Necesita una vista para mostrar los cursos en los que están inscritos los estudiantes y una vista para acceder a los contenidos reales del curso. Edite el archivo `views.py` de los estudiantes aplicación y agregue el siguiente código:

```

desde django.views.generic.list import ListView
de cursos.modelos importar curso

clase StudentCourseListView (LoginRequiredMixin, ListView):
    modelo = Curso
    template_name = 'estudiantes / curso / list.html'

    def get_queryset (yo):
        qs = super (). get_queryset ()
        return qs.filter (estudiantes__in = [self.request.user])

```

Esta es la vista para ver los cursos en los que están inscritos los estudiantes. Hereda de `LoginRequiredMixin` para asegurarse de que solo los usuarios que hayan iniciado sesión puedan acceder a la vista. Eso también hereda del `ListView` genérico para mostrar una lista de objetos del curso . Tú anular el método `get_queryset ()` para recuperar solo los cursos que un estudiante está inscrito en; filtra el `QuerySet` por el campo `ManyToManyField` del estudiante para hacerlo.

Luego, agregue el siguiente código al archivo `views.py` de la aplicación de los estudiantes :

```

de django.views.generic.detail import DetailView

```

```

class StudentCourseDetailView (DetailView):
    modelo = Curso
    template_name = 'estudiantes / curso / detalle.html'

    def get_queryset (yo):
        qs = super (). get_queryset ()
        return qs.filter (estudiantes__in = [self.request.user])

    def get_context_data (self, ** kwargs):
        context = super (). get_context_data (** kwargs)
        # obtener el objeto del curso
        curso = self.get_object ()
        si 'module_id' en self.kwargs:
            # obtener el módulo actual
            contexto ['módulo'] = curso.modules.get (
                id = self.kwargs ['module_id'])
        más:
            # obtener el primer módulo
            contexto ['módulo'] = curso.modules.all () [0]
        devolver contexto

```

Esta es la vista StudentCourseDetailView . Anula el get_queryset () método para limitar el QuerySet base a los cursos en los que está matriculado el alumno. Tú también anula el método get_context_data () para establecer un módulo de curso en el contexto si se proporciona el parámetro de URL module_id . De lo contrario, configura el primer módulo del curso. De esta forma, los estudiantes podrán navegar por los módulos dentro de un curso.

Edite el archivo urls.py de la aplicación de los estudiantes y agregue la siguiente URL patrones a ella:

```

ruta ('cursos /',
     views.StudentCourseListView.as_view (),
     name = 'student_course_list'),

ruta ('curso / <pk> /',
     views.StudentCourseDetailView.as_view (),
     name = 'student_course_detail'),

ruta ('curso / <pk> / <module_id> /',
     views.StudentCourseDetailView.as_view (),
     nombre = 'módulo_detalle_del_curso_de_estudiante'),

```

[426]

Cree la siguiente estructura de archivos dentro del directorio templates / estudiantes / del solicitud de estudiantes :

```

curso/
  detail.html
  list.html

```

Edite la plantilla Students / course / list.html y agréguele el siguiente código:

```
{% extiende "base.html"%}
```



```

{% block title%} Mis cursos {% endblock%}

{% block content%}
<h1> Mis cursos </h1>

<div class = "módulo">
    {% para el curso en object_list%}
        <div class = "información del curso">
            <h3> {{course.title}} </h3>
            <p> <a href="{% url 'student_course_detail' course.id %}">
                Acceder a los contenidos </a> </p>
            </div>
        {% vacío%}
        <p>
            Aún no estás inscrito en ningún curso.
            <a href="{% url 'course_list' %}"> Explorar cursos </a>
            para inscribirse en un curso.
        </p>
    {% endfor%}
</div>
{% endblock%}

```

Esta plantilla muestra los cursos en los que está inscrito el alumno. Recuerda eso cuando un nuevo estudiante se registre con éxito en la plataforma, será redirigido a la URL de student_course_list . Redirigamos también a los alumnos a esta URL cuando inician sesión en la plataforma.

Edite el archivo settings.py del proyecto educa y agregue el siguiente código:

```

de django.urls import reverse_lazy
LOGIN_REDIRECT_URL = reverse_lazy ('lista de cursos_estudiantes')

```

[427]

Procesamiento y almacenamiento en caché de contenido

Esta es la configuración utilizada por el módulo de autenticación para redirigir al estudiante después de una exitosa iniciar sesión si no hay ningún parámetro siguiente en la solicitud. Después de un inicio de sesión exitoso, un estudiante será redirigido a la URL student_course_list para ver los cursos en el que están inscritos.

Edite la plantilla Students / course / detail.html y agréguele el siguiente código:

```

{% extiende "base.html"%}

{% block title%}
    {{object.title}}
{% endblock%}

{% block content%}
<h1>
    {{ Título de módulo }}
</h1>
<div class = "contenido">
    <h3> Módulos </h3>
    <ul id = "módulos">
        {% para m en object.modules.all%}
            <li data-id = "{{m.id}}" {% if m == módulo%}
class = "selected" {% endif%}>
                <a href = "{% url 'student_course_detail_module 'object.id
m.id%">
                    <span>
                        Módulo <span class = "order"> {{m.order | add: 1}} </span>

```

```

        </span>
        <br>
        {{m.title}}
    </a>
</li>
{% vacío%}
<li> Aún no hay módulos. </li>
{% endfor%}
</ul>
</div>
<div class = "módulo">
    {% para el contenido en module.contents.all%}
    {% con item = content.item%}
    <h2> {{item.title}} </h2>
    {{item.render}}
    {% terminar con %}
    {% endfor%}
</div>
{% endblock%}

```

[428]

Esta es la plantilla para que los estudiantes matriculados accedan a los contenidos de un curso. Primero, crea una lista HTML que incluye todos los módulos del curso y resalta el módulo. Luego, itera sobre el contenido del módulo actual y accede a cada contenido item para mostrarlo usando {{item.render}} . Vas a agregar el render () método a los modelos de contenido a continuación. Este método se encargará de renderizar el contenido correctamente.

Representar diferentes tipos de contenido

Debe proporcionar una forma de representar cada tipo de contenido. Edite el archivo models.py de la aplicación de cursos y agregue el siguiente método render () a ItemBase modelo:

```

desde django.template.loader import render_to_string

clase ItemBase (modelos.Modelo):
    # ...

    def render (auto):
        return render_to_string (
            f'courses / contenido / {self._meta.model_name} .html ',
            {'item': self})

```

Este método usa la función render_to_string () para renderizar una plantilla y devolviendo el contenido renderizado como una cadena. Cada tipo de contenido se procesa usando una plantilla que lleva el nombre del modelo de contenido. Utiliza self._meta.model_name para Genere el nombre de plantilla apropiado para cada modelo de contenido de forma dinámica. los El método render () proporciona una interfaz común para renderizar contenido diverso.

Cree la siguiente estructura de archivos dentro de las plantillas / cursos / directorio del solicitud de cursos :

```

contenido/
    text.html
    file.html
    image.html
    video.html

```

Edite la plantilla cursos / contenido / texto.html y escriba este código:

```

{{item.content | linebreak}}

```

Esta es la plantilla para renderizar contenido de texto. El filtro de plantilla de saltos de línea reemplaza

Procesamiento y almacenamiento en caché de contenido

Edite la plantilla cursos / contenido / archivo.html y agregue lo siguiente:

```
<p> <a href="{{ item.file.url }}" class="button"> Descargar archivo </a> </p>
```

Esta es la plantilla para renderizar archivos. Generas un enlace para descargar el archivo.

Edite la plantilla cursos / contenido / imagen.html y escriba:

```
<p> <img src = "{{item.file.url}}" alt = "{{item.title}}"> </p>
```

Esta es la plantilla para renderizar imágenes. Para archivos cargados con ImageField y FileField para que funcione, debe configurar su proyecto para servir archivos multimedia con el servidor de desarrollo.

Edite el archivo settings.py de su proyecto y agregue el siguiente código:

```
MEDIA_URL = '/ media /'
MEDIA_ROOT = os.path.join (BASE_DIR, 'media /')
```

Recuerde que MEDIA_URL es la URL base para publicar archivos multimedia subidos y MEDIA_ROOT es la ruta local donde se encuentran los archivos.

Edite el archivo urls.py principal de su proyecto y agregue las siguientes importaciones:

```
desde la configuración de importación django.conf
desde django.conf.urls.static import static
```

Luego, escribe las siguientes líneas al final del archivo:

```
si la configuración.DEBUG:
    urlpatterns += static (settings.MEDIA_URL,
                           document_root = settings.MEDIA_ROOT)
```

Su proyecto ahora está listo para cargar y publicar archivos multimedia. El desarrollo de Django servidor estará a cargo de servir los archivos multimedia durante el desarrollo (es decir, cuando la configuración DEBUG está establecida en True). Recuerde que el servidor de desarrollo no es adecuado para uso en producción. Aprenderá a configurar un entorno de producción. en el *Capítulo 14 , Puesta en marcha* .

También debe crear una plantilla para renderizar objetos de video . Que va a utilizar django-incrustar-video para incrustar contenido de video. django-embed-video es un tercero Aplicación Django que te permite incrustar videos en tus plantillas, desde fuentes como YouTube o Vimeo, simplemente proporcionando su URL pública.

Instale el paquete con el siguiente comando:

```
pip install django-embed-video == 1.3.2
```

Edite el archivo settings.py de su proyecto y agregue la aplicación a INSTALLED_ Configuración de APPS , de la siguiente manera:

```
INSTALLED_APPS = [
    # ...
    'embed_video',
]
```

Puede encontrar la documentación de la aplicación django-embed-video en <https://django-embed-video.readthedocs.io/en/latest/>.

Edite la plantilla cursos / contenido / video.html y escriba el siguiente código:

```
{% load embed_video_tags %}
{% video item.url "small"%}
```

Esta es la plantilla para renderizar videos.

Ahora ejecute el servidor de desarrollo y acceda a <http://127.0.0.1:8000/course/mio/> en su navegador. Acceder al sitio con un usuario perteneciente a los Instructores agrupar y agregar varios contenidos a un curso. Para incluir contenido de video, puede copiar cualquier URL de YouTube, como <https://www.youtube.com/watch?v=bgV39DlmZ2U> e inclúyalo en el campo de URL del formulario.

Después de agregar contenido al curso, abra <http://127.0.0.1:8000/> , haga clic en el curso y haga clic en el botón **INSCRIBIRSE AHORA** . Deberías estar inscrito en el curso y redirigido a la URL `student_course_detail` . El seguimiento La captura de pantalla muestra una página de contenido del curso de muestra:

Figura 11.5: Página de contenido de un curso

[431]

Procesamiento y almacenamiento en caché de contenido

¡Excelente! Ha creado una interfaz común para renderizar diferentes tipos de cursos contenido.

Usando el marco de caché

Las solicitudes HTTP a su aplicación web generalmente implican acceso a la base de datos, datos

procesamiento y renderizado de plantillas. Esto es mucho más caro en términos de procesamiento que servir un sitio web estático. La sobrecarga en algunas solicitudes puede ser significativo cuando su sitio comienza a recibir más y más tráfico. Aquí es donde el almacenamiento en caché se vuelve precioso. Al almacenar en caché consultas, resultados de cálculo o contenido renderizado en un Solicitud HTTP, evitará operaciones costosas en las siguientes solicitudes. Esta se traduce en tiempos de respuesta más cortos y menos procesamiento en el lado del servidor.

Django incluye un robusto sistema de caché que le permite almacenar datos en caché con diferentes niveles de granularidad. Puede almacenar en caché una sola consulta, el resultado de una vista específica, partes del contenido de la plantilla renderizada, o todo su sitio. Los elementos se almacenan en la caché sistema por un tiempo predeterminado. Puede especificar el tiempo de espera predeterminado para los datos en caché.

Así es como usualmente usará el marco de caché cuando su aplicación obtenga una solicitud HTTP:

1. Intenta encontrar los datos solicitados en la caché.
2. Si lo encuentra, devuelva los datos en caché
3. Si no lo encuentra, realice los siguientes pasos:
 - Realizar la consulta o tratamiento necesario para obtener los datos.
 - Guarde los datos generados en la caché
 - Devuelve los datos

Puede leer información detallada sobre el sistema de caché de Django en [https:// docs. django project. com/ en/ 3.0/ topics/ cache/](https://docs.djangoproject.com/en/3.0/topics/cache/).

Backends de caché disponibles

Django viene con varios backends de caché. Estos son los siguientes:

- `backends.memcached.MemcachedCache` o `backends.memcached`.
PyLibMCCache : un backend Memcached. Memcached es rápido y eficiente servidor de caché basado en memoria. El backend a usar depende de Memcached Enlaces de Python que elija.
- `backends.db.DatabaseCache` : usa la base de datos como un sistema de caché.

[432]

- `backends.filebased.FileBasedCache` : utiliza el sistema de almacenamiento de archivos. Esta serializa y almacena cada valor de caché como un archivo separado.
- `backends.locmem.LocMemCache` : un backend de caché de memoria local. Este el backend de caché predeterminado.
- `backends.dummy.DummyCache` : un backend de caché ficticio diseñado solo para desarrollo. Implementa la interfaz de caché sin almacenar en caché cualquier cosa. Esta caché es por proceso y segura para subprocessos.

Para un rendimiento óptimo, utilice un backend de caché basado en memoria como como el backend de Memcached.

Instalación de Memcached

Vas a utilizar el backend de Memcached. Memcached se ejecuta en la memoria y se le asigna una cantidad específica de RAM. Cuando la RAM asignada está llena, Memcached comienza a eliminar los datos más antiguos para almacenar nuevos datos.

Descarga Memcached desde <https://memcached.org/downloads> . Si esta usando Linux, puede instalar Memcached usando el siguiente comando:

```
./configure && make && make test && sudo make install
```

Si está utilizando macOS, puede instalar Memcached con el paquete Homebrew administrador usando el comando `brew install memcached`. Tú puedes descargar Homebrew de <https://brew.sh/>.

Después de instalar Memcached, abra un shell e inícielo con el siguiente comando:

```
memcached -l 127.0.0.1:11211
```

Memcached se ejecutará en el puerto 11211 de forma predeterminada. Sin embargo, puede especificar un host y puerto utilizando la opción `-l`. Puede encontrar más información sobre Memcached en <https://memcached.org>.

Después de instalar Memcached, debe instalar sus enlaces de Python. Puedes hacerlo con el siguiente comando:

```
pip install python-memcached == 1.59
```

[433]

Procesamiento y almacenamiento en caché de contenido

Configuración de caché

Django proporciona la siguiente configuración de caché:

- **CACHES** : un diccionario que contiene todos los cachés disponibles para el proyecto.
- **CACHE_MIDDLEWARE_ALIAS** : el alias de caché que se utilizará para el almacenamiento
- **CACHE_MIDDLEWARE_KEY_PREFIX** : el prefijo que se utilizará para las claves de caché
- Establezca un prefijo para evitar colisiones de claves si comparte la misma caché entre varios sitios
- **CACHE_MIDDLEWARE_SECONDS** : el número predeterminado de segundos para almacenar páginas en caché

El sistema de almacenamiento en caché del proyecto se puede configurar mediante la configuración de **CACHES**. Esta configuración le permite especificar la configuración para múltiples cachés. Cada caché incluido en el diccionario **CACHES** puede especificar los siguientes datos:

- **BACKEND** : el backend de caché que se utilizará.
- **KEY_FUNCTION** : una cadena que contiene una ruta de puntos a un invocable que toma un prefijo, versión y clave como argumentos y devuelve una clave de caché final.
- **KEY_PREFIX** : un prefijo de cadena para todas las claves de caché, para evitar colisiones.
- **UBICACIÓN** : la ubicación del caché. Dependiendo del backend de caché, esto podría ser un directorio, un host y un puerto, o un nombre para el backend en memoria.
- **OPCIONES** : cualquier parámetro adicional que se pasará al backend de la caché.
- **TIMEOUT** : el tiempo de espera predeterminado, en segundos, para almacenar las claves de caché. Es 300 segundos por defecto, que son cinco minutos. Si se establece en Ninguno, las claves de caché no caducarán.
- **VERSIÓN** : el número de versión predeterminado para las claves de caché. Útil para caché versionado.

Agregar Memcached a su proyecto

Configuremos la caché para su proyecto. Edite el archivo `settings.py` del educaproyecto y agregue el siguiente código:

```
CACHES = {  
    'defecto': {
```

```

        'BACKEND': 'django.core.cache.backends.memcached.
MemcachedCache',
        'UBICACIÓN': '127.0.0.1:11211',
    }
}

```

[434]

Estás usando el backend `MemcachedCache`. Usted especifica su ubicación usando el dirección: notación de puerto. Si tiene varias instancias de `Memcached`, puede usar una lista de `UBICACIÓN`.

Supervisión de Memcached

Para monitorear `Memcached`, usará un paquete de terceros llamado `django-Memcache-status`. Esta aplicación muestra estadísticas para sus instancias `Memcached` en el sitio de administración. Instálelo con el siguiente comando:

pip instalar django-memcache-status == 2.2

Edite el archivo `settings.py` y agregue `'memcache_status'` a `INSTALLED_APPS` ajuste:

```

INSTALLED_APPS = [
    # ...
    'memcache_status',
]

```

Edite el archivo `admin.py` de la aplicación de cursos y agregue las siguientes líneas:

```

# use el sitio de índice de administración de Memcache
admin.site.index_template = 'memcache_status / admin_index.html'

```

Asegúrese de que `Memcached` se esté ejecutando, inicie el servidor de desarrollo en otro shell ventana y abra `http://127.0.0.1:8000/admin/` en su navegador. Inicie sesión en sitio de administración usando un superusuario. Debería ver el siguiente bloque en el página de índice del sitio de administración:

Figura 11.6: El bloque de estado de Memcached

El bloque contiene un gráfico de barras que muestra la carga de la caché. El color verde representa caché libre, mientras que el rojo indica espacio utilizado. Si hace clic en el título del cuadro, se muestra estadísticas detalladas de su instancia de `Memcached`.

Ha configurado `Memcached` para su proyecto y puede monitorearlo. Empecemos caché de datos!

[435]

Procesamiento y almacenamiento en caché de contenido

Niveles de caché

Django proporciona los siguientes niveles de almacenamiento en caché, enumerados aquí por orden ascendente de granularidad:

- **API de caché de bajo nivel** : proporciona la mayor granularidad. Te permite almacenar en caché consultas o cálculos específicos.
- **Caché de plantilla** : le permite almacenar en caché fragmentos de plantilla.
- Caché por **vista** : proporciona almacenamiento en caché para vistas individuales.
- **Caché por sitio** : el **caché** de nivel más alto. Almacena en caché todo su sitio.

Piense en su estrategia de caché antes de implementar el almacenamiento en caché. Céntrese primero en consultas costosas o cálculos que no calculado por usuario.

Usando la API de caché de bajo nivel

La API de caché de bajo nivel le permite almacenar objetos en la caché con cualquier granularidad. Se encuentra en `django.core.cache` . Puedes importarlo así:

```
desde el caché de importación django.core.cache
```

Esto usa la caché predeterminada. Es equivalente a cachés ['por defecto'] . Accediendo un caché específico también es posible a través de su alias:

```
desde django.core.cache import caches
my_cache = caches ['alias']
```

Echemos un vistazo a cómo funciona la API de caché. Abra el shell con el comando `python manage.py shell` y ejecute el siguiente código:

```
>>> desde el caché de importación django.core.cache
>>> cache.set ('músico', 'Django Reinhardt', 20)
```

Accede al backend de caché predeterminado y usa `set` (clave, valor, tiempo de espera) para almacenar una clave llamada 'músico' con un valor que es la cadena 'Django Reinhardt' para 20 segundos. Si no especifica un tiempo de espera, Django usa el tiempo de espera predeterminado especificado para el backend de caché en la configuración de `CACHES` . Ahora, ejecute el siguiente código:

```
>>> cache.get ('músico')
'Django Reinhardt'
```

[436]

Recupera la clave del caché. Espere 20 segundos y ejecute el mismo código:

```
>>> cache.get ('músico')
```


Esta vez no se devuelve ningún valor. La clave de caché del 'músico' ha expirado y el comando `get()` en el método devuelve `None` porque la clave ya no está en la caché.

Evite siempre almacenar un valor Ninguno en una clave de caché porque no podrá distinguir entre el valor real y una falta de caché.

Guardemos en caché un `QuerySet` con el siguiente código:

```
>>> de cursos.modelos de importación Asunto
>>> sujetos = Subject.objects.all()
>>> cache.set('my_subjects', sujetos)
```

Realiza un `QuerySet` en el modelo `Subject` y almacena los objetos devueltos en el clave 'my_subjects'. Recuperemos los datos almacenados en caché:

```
>>> cache.get('mis_sujetos')
<QuerySet [ <Asunto: Matemáticas>, <Asunto: Música>, <Asunto: Física>,
<Asunto: Programación> ]>
```

Vas a almacenar en caché algunas consultas en tus vistas. Edite el archivo `views.py` del aplicación de cursos y agregue la siguiente importación:

```
from django.core.cache import cache
```

En el método `get()` de `CourseListView`, busque la siguiente línea:

```
sujetos = Subject.objects.annotate (
    total_courses = Count('cursos'))
```

Reemplácelo con los siguientes:

```
sujetos = cache.get('all_subjects')
if not sujetos:
    sujetos = Subject.objects.annotate (
        total_courses = Count('cursos'))
    cache.set('all_subjects', sujetos)
```

En este código, intenta obtener la clave `all_students` del caché usando el caché.

`obtener()`. Esto devuelve `None` si no se encuentra la clave dada. Si no se encuentra ninguna clave (no almacenada en caché aún o en caché pero agotado el tiempo), realiza la consulta para recuperar todos los objetos `Subject` y su número de cursos, y almacena en caché el resultado usando `cache.set()`.

[437]

Procesamiento y almacenamiento en caché de contenido

Ejecute el servidor de desarrollo y abra `http://127.0.0.1:8000/` en su navegador. Cuando se ejecuta la vista, la clave de caché no se encuentra y el `QuerySet` es ejecutado. Abra `http://127.0.0.1:8000/admin/` en su navegador y haga clic en el Sección `Memcached` para ampliar las estadísticas. Debería ver los datos de uso de la caché que es similar a la siguiente pantalla:

Figura 11.7: Estado de Memcached y detalles de uso

Eche un vistazo a los **artículos Curr**, que deberían ser **1**. Esto muestra que hay un elemento almacenado actualmente en la caché. **Get Hits** muestra cuántos comandos **get** fueron Success y **Get Misses** muestra las solicitudes de obtención de claves que faltan. El **índice de fallos** se calcula utilizando ambos.

A continuación, vuelva a <http://127.0.0.1:8000/> utilizando su navegador y vuelva a cargar el página varias veces. Si echas un vistazo a las estadísticas de caché ahora, verás varias más lecturas (**Get Hits** y **Cmd Get** aumentarán).

Almacenamiento en caché basado en datos dinámicos

A menudo, querrá almacenar en caché algo que se base en datos dinámicos. En estos casos, debe crear claves dinámicas que contengan toda la información necesaria para Identificar de forma única los datos almacenados en caché.

Edite el archivo `views.py` de la aplicación de cursos y modifique `CourseListView` vista para que se vea así:

[438]

```
class CourseListView (TemplateResponseMixin, View):
    modelo = Curso
    template_name = 'cursos / curso / list.html'

    def get (self, request, subject = None):
        sujetos = cache.get ('all_subjects')
        si no son sujetos:
            sujetos = Subject.objects.annotate (
                total_courses = Count ('cursos'))
            cache.set ('all_subjects', sujetos)
        all_courses = Course.objects.annotate (
            total_modules = Count ('módulos'))
        si el sujeto:
            subject = get_object_or_404 (Asunto, slug = asunto)
            key = f'subject_{subject.id}_courses '
            cursos = cache.get (clave)
            si no cursos:
                cursos = all_courses.filter (asunto = asunto)
                cache.set (clave, cursos)
        más:
            cursos = cache.get ('all_courses')
            si no cursos:
                cursos = todos_cursos
                cache.set ('todos_cursos', cursos)
        return self.render_to_response ({'sujetos': sujetos,
                                         'sujeto': sujeto,
                                         'cursos': cursos})
```

En este caso, también almacena en caché todos los cursos y los cursos filtrados por tema. Tu usas la clave de caché `all_courses` para almacenar todos los cursos si no se proporciona ningún tema. Si hay un sujeto, construyes la clave dinámicamente con `f'subject_{subject.id}_courses '`.

Es importante tener en cuenta que no puede utilizar un `QuerySet` en caché para crear otros `QuerySets`, ya que lo que almacenó en caché son en realidad los resultados del `QuerySet`. Entonces no puedes hacer el siguiendo:

```
cursos = cache.get('all_courses')
cursos.filtro (asunto = asunto)
```

En su lugar, debe crear el curso `QuerySet base.objects.annotate (total_módulos = Count ('módulos'))`, que no se ejecutará hasta que está forzado y utilícelo para restringir aún más el `QuerySet` con `all_courses`. `filtro (asunto = asunto)` en caso de que los datos no se encuentren en la caché.

[439]

Procesamiento y almacenamiento en caché de contenido

Almacenamiento en caché de fragmentos de plantilla

El almacenamiento en caché de fragmentos de plantilla es un enfoque de nivel superior. Necesitas cargar el caché etiquetas de plantilla en su plantilla usando `{% load cache%}`. Entonces, podrás utilice la etiqueta de plantilla `{% cache%}` para almacenar en caché fragmentos de plantilla específicos. Vas a Por lo general, use la etiqueta de plantilla de la siguiente manera:

```
{% cache 300 fragment_name%}
...
{% endcache%}
```

La etiqueta de plantilla `{% cache%}` tiene dos argumentos obligatorios: el tiempo de espera en segundos y un nombre para el fragmento. Si necesita almacenar en caché el contenido según la dinámica datos, puede hacerlo pasando argumentos adicionales a la plantilla `{% cache%}` etiqueta para identificar de forma única el fragmento.

Edite el `/students/course/detail.html` de la solicitud de los estudiantes . Añade el el siguiente código en la parte superior, justo después de la etiqueta `{% extensions%}` :

```
{% load cache%}
```

Luego, busque las siguientes líneas:

```
{% para el contenido en module.contents.all%}
  {% con item = content.item%}
    <h2> {{item.title}} </h2>
    {{item.render}}
  {% terminar con %}
{% endfor%}
```

Reemplácelos por los siguientes:

```
{% cache 600 module_contents module%}
  {% para el contenido en module.contents.all%}
    {% con item = content.item%}
      <h2> {{item.title}} </h2>
      {{item.render}}
    {% terminar con %}
  {% endfor%}
{% endcache%}
```

Guarde en caché este fragmento de plantilla con el nombre `module_contents` y pasando el objeto `Module` actual . Por lo tanto, identifica de forma única el fragmento. Esto es importante para evitar almacenar en caché el contenido de un módulo y ofrecer contenido incorrecto cuando se solicita un módulo diferente.

[440]

Si la configuración `USE_I18N` se establece en `True`, el middleware por sitio caché respetará el idioma activo. Si usa `{% cache %}` etiqueta de plantilla, debe utilizar una de las variables disponibles en plantillas para lograr el mismo resultado, como `{% cache 600 name request.LANGUAGE_CODE %}`.

Almacenamiento en caché de vistas

Puede almacenar en caché la salida de vistas individuales utilizando el decorador `cache_page` ubicado en `django.views.decorators.cache`. El decorador requiere un tiempo de espera argumento (en segundos).

Usémoslo en sus puntos de vista. Edite el archivo `urls.py` de la aplicación de los estudiantes y agregue la siguiente importación:

```
desde django.views.decorators.cache import cache_page
```

Luego, aplique el decorador `cache_page` al `student_course_detail` y

`Student_course_detail_module` patrones de URL, de la siguiente manera:

```
ruta ('curso / <pk> /',
     cache_page (60 * 15) ( vistas.StudentCourseDetailView.as_view () ),
     name = 'student_course_detail'),

ruta ('curso / <pk> / <module_id> /',
     cache_page (60 * 15) ( vistas.StudentCourseDetailView.as_view () ),
     nombre = 'módulo_detalle_del_curso_de_estudiante'),
```

Ahora, el resultado de `StudentCourseDetailView` se almacena en caché durante 15 minutos.

La caché por vista utiliza la URL para crear la clave de la caché. Múltiple Las URL que apuntan a la misma vista se almacenarán en caché por separado.

Usar la caché por sitio

Este es el caché de más alto nivel. Le permite almacenar en caché todo su sitio. A permitir la caché por sitio, editar el archivo `settings.py` de su proyecto y agregar las clases `UpdateCacheMiddleware` y `FetchFromCacheMiddleware` para el ajuste `MIDDLEWARE`, de la siguiente manera:

```
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
```

[441]

```

'django.middleware.cache.UpdateCacheMiddleware',
'django.middleware.common.CommonMiddleware',
'django.middleware.cache.FetchFromCacheMiddleware',
# ...
]

```

Recuerde que el middleware se ejecuta en el orden dado durante la solicitud fase, y en orden inverso durante la fase de respuesta. UpdateCacheMiddleware se coloca antes de CommonMiddleware porque se ejecuta durante el tiempo de respuesta, cuando El middleware se ejecuta en orden inverso. Se coloca FetchFromCacheMiddleware después de CommonMiddleware intencionalmente porque necesita acceder al conjunto de datos de solicitud por el último.

A continuación, agregue la siguiente configuración al archivo settings.py :

```

CACHE_MIDDLEWARE_ALIAS = 'predeterminado'
CACHE_MIDDLEWARE_SECONDS = 60 * 15 # 15 minutos
CACHE_MIDDLEWARE_KEY_PREFIX = 'educa'

```

En esta configuración, utiliza la caché predeterminada para su middleware de caché y establece la tiempo de espera de caché global a 15 minutos. También especifica un prefijo para todas las claves de caché para evite colisiones en caso de que use el mismo backend Memcached para múltiples proyectos. Su sitio ahora almacenará en caché y devolverá contenido en caché para todas las solicitudes GET .

Ha hecho esto para probar la funcionalidad de caché por sitio. Sin embargo, el por sitio el caché no es adecuado para usted, ya que las vistas de administración del curso deben mostrar datos actualizados para reflejar instantáneamente cualquier cambio. El mejor enfoque a seguir en su proyecto es almacenar en caché las plantillas o vistas que se utilizan para mostrar el contenido del curso a estudiantes.

Ha visto una descripción general de los métodos proporcionados por Django para almacenar datos en caché. Debe definir su estrategia de caché sabiamente y priorizar las más caras QuerySets o cálculos.

Resumen

En este capítulo, implementó las vistas públicas para el catálogo de cursos. Tú construyó un sistema para que los estudiantes se registren y se inscriban en cursos. También creaste el funcionalidad para representar diferentes tipos de contenido para los módulos del curso. Finalmente, aprendió a usar el marco de caché de Django e instaló y supervisó el backend de la caché Memcached.

En el siguiente capítulo, creará una API RESTful para su proyecto usando Django REST marco de referencia.

[442]

En el capítulo anterior, creó un sistema para el registro e inscripción de estudiantes en cursos. Creó vistas para mostrar los contenidos del curso y aprendió a usar Marco de caché de Django.

En este capítulo, creará una API RESTful para su plataforma de e-learning. Un La API le permite construir un núcleo común que se puede usar en múltiples plataformas. como sitios web, aplicaciones móviles, complementos, etc. Por ejemplo, puede crear una API para ser consumida por una aplicación móvil para su plataforma de e-learning. Si proporciona una API a terceros, estos podrán consumir información y operar con su aplicación mediante programación. Una API permite a los desarrolladores automatizar acciones en su plataforma e integrar su servicio con otras aplicaciones o servicios en línea. Construirá una API con todas las funciones para su plataforma de aprendizaje electrónico.

En este capítulo, podrá:

- Instalar el marco REST de Django
- Cree serializadores para sus modelos
- Construya una API RESTful
- Crear serializadores anidados
- Cree vistas de API personalizadas
- Manejar la autenticación de API
- Agregar permisos a las vistas de la API
- Crea un permiso personalizado
- Implementar conjuntos de vistas y enrutadores
- Utilice la biblioteca de solicitudes para consumir la API

[443]

Construyendo una API

Comencemos con la configuración de su API.

Construyendo una API RESTful

Al crear una API, hay varias formas de estructurar sus puntos finales y acciones, pero se recomienda seguir los principios REST. El **RESTO** arquitectura viene de **Transferencia de Estado Representacional**. Las API RESTful están basadas en recursos; tu los modelos representan recursos y métodos HTTP como GET, POST, PUT o DELETE se utilizan para recuperar, crear, actualizar o eliminar objetos. Los códigos de respuesta HTTP también son utilizado en este contexto. Se devuelven diferentes códigos de respuesta HTTP para indicar el resultado de la solicitud HTTP, por ejemplo, 2XX códigos de respuesta para el éxito, 4XX para errores, etc.

Los formatos más comunes para intercambiar datos en las API RESTful son JSON y XML. Construirá una API RESTful con serialización JSON para su proyecto. Tu API proporcionará la siguiente funcionalidad:

- Recuperar sujetos
- Recuperar cursos disponibles
- Recuperar los contenidos del curso
- Inscribirse en un curso

Puede crear una API desde cero con Django creando vistas personalizadas. Sin embargo, hay varios módulos de terceros que simplifican la creación de una API para su proyecto; el más popular entre ellos es el marco Django REST.

Instalación del marco REST de Django

El marco Django REST le permite crear fácilmente API RESTful para su proyecto.

Puede encontrar toda la información sobre el marco REST en <https://www.django-rest-framework.org/>.

Abra el shell e instale el marco con el siguiente comando:

pip instalar djangorestframework == 3.11.0

Editar la settings.py archivo de la Educa proyecto y añadir rest_framework a la

Configuración INSTALLED_APPS para activar la aplicación, de la siguiente manera:

```
INSTALLED_APPS = [  
    # ...  
    'rest_framework',  
]
```

[444]

Luego, agregue el siguiente código al archivo settings.py :

```
REST_FRAMEWORK = {  
    'DEFAULT_PERMISSION_CLASSES': [  
        'rest_framework.permissions.IsAuthenticatedOrReadOnly'  
    ]  
}
```

Puede proporcionar una configuración específica para su API utilizando REST_

Ajuste de MARCO . El marco REST ofrece una amplia gama de configuraciones para configurar comportamientos predeterminados. La configuración DEFAULT_PERMISSION_CLASSES especifica los permisos predeterminados para leer, crear, actualizar o eliminar objetos. Configura

DjangoModelPermissionsOrAnonReadOnly como la única clase de permiso predeterminada.

Esta clase se basa en el sistema de permisos de Django para permitir a los usuarios crear, actualizar, o eliminar objetos, al tiempo que proporciona acceso de solo lectura para usuarios anónimos. Vas a obtenga más información sobre los permisos más adelante en la sección *Agregar permisos a las vistas* .

Para obtener una lista completa de las configuraciones disponibles para el marco REST, puede visitar <https://www.django-rest-framework.org/api-guide/settings/>.

Definición de serializadores

Después de configurar el marco REST, debe especificar cómo serán sus datos serializado. Los datos de salida deben serializarse en un formato específico y los datos de entrada se deserializará para su procesamiento. El marco proporciona las siguientes clases para construir serializadores para objetos individuales:

- Serializador : proporciona serialización para instancias normales de clase Python
- ModelSerializer : proporciona serialización para instancias de modelo
- HyperlinkedModelSerializer : lo mismo que ModelSerializer , pero representa relaciones de objetos con enlaces en lugar de claves primarias

Construyamos su primer serializador. Crea la siguiente estructura de archivos dentro de los cursos directorio de la aplicación:

```
api /  
    __init__.py  
    serializers.py
```

Construirá toda la funcionalidad de la API dentro del directorio api para mantener todo bien organizado. Edite el archivo serializers.py y agregue el siguiente código:

```
from rest_framework import serializers  
from ..models import Asunto
```

[445]

Construyendo una API

```
class SubjectSerializer(serializers.ModelSerializer):
    class Meta:
        modelo = Asunto
        fields = ['id', 'title', 'slug']
```

Este es el serializador para el modelo Subject . Los serializadores se definen de forma similar fashion a las clases Form y ModelForm de Django . La clase Meta le permite especificar el modelo a serializar y los campos a incluir para la serialización. Todos los campos del modelo se incluirá si no establece un atributo de campos .

Probemos tu serializador. Abra la línea de comando e inicie el shell de Django con el siguiente comando:

shell de python manage.py

Ejecute el siguiente código:

```
>>> de cursos.modelos de importación Asunto
>>> de cursos.api.serializers importar SubjectSerializer
>>> sujeto = Subject.objects.latest ('id')
>>> serializer = SubjectSerializer (asunto)
>>> serializer.data
{'id': 4, 'title': 'Programación', 'slug': 'programación'}
```

En este ejemplo, obtiene un objeto Subject , crea una instancia de SubjectSerializer , y acceder a los datos serializados. Puede ver que los datos del modelo se traducen a Tipos de datos nativos de Python.

Comprensión de analizadores y representadores

Los datos serializados deben procesarse en un formato específico antes de devolverlos en una respuesta HTTP. Del mismo modo, cuando recibe una solicitud HTTP, debe analizar el datos entrantes y deserialícelos antes de poder operar con ellos. Marco REST incluye procesadores y analizadores para manejar eso.

Veamos cómo analizar los datos entrantes. Ejecute el siguiente código en el shell de Python:

```
>>> desde io importar BytesIO
>>> de rest_framework.parsers importar JSONParser
>>> datos = b '{"id": 4, "título": "Programación", "slug": "programación"}'
>>> JSONParser (). Parse (BytesIO (datos))
{'id': 4, 'title': 'Programación', 'slug': 'programación'}
```


Dada una entrada de cadena JSON, puede usar la clase `JSONParser` proporcionada por REST framework para convertirlo en un objeto Python.

El marco REST también incluye clases de `Renderer` que le permiten formatear API respuestas. El marco determina qué renderizador usar a través del contenido negociación mediante la inspección del encabezado de aceptación de la solicitud para determinar el esperado tipo de contenido de la respuesta. Opcionalmente, el renderizador está determinado por el formato sufixo de la URL. Por ejemplo, la URL `http://127.0.0.1:8000/api/data.json` podría ser un punto final que activa `JSONRenderer` para devolver un JSON respuesta.

Regrese al shell y ejecute el siguiente código para representar el objeto serializador del ejemplo de serializador anterior:

```
>>> de rest_framework.renderers import JSONRenderer
>>> JSONRenderer().render(serializer.data)
```

Verá el siguiente resultado:

```
b'{"id": 4, "title": "Programación", "slug": "programación"}'
```

Utiliza `JSONRenderer` para representar los datos serializados en JSON. Por predeterminado, el marco REST utiliza dos renderizadores diferentes: `JSONRenderer` y `BrowsableAPIRenderer`. Este último proporciona una interfaz web para navegar fácilmente por su API. Puede cambiar las clases de renderizador predeterminadas con `DEFAULT_RENDERER_CLASSES` de la configuración `REST_FRAMEWORK`.

Puede encontrar más información sobre procesadores y analizadores en <https://www.django-rest-framework.org/api-guide/renderers/> y <https://www.django-rest-framework.org/api-guide/parsers/>, respectivamente.

Lista de construcción y vistas de detalle

El marco REST viene con un conjunto de vistas genéricas y mixins que puede usar para construya sus vistas de API. Proporcionan la funcionalidad para recuperar, crear, actualizar o eliminar objetos de modelo. Puede ver todos los mixins genéricos y las vistas proporcionadas por REST marco en <https://www.django-rest-framework.org/api-guide/generic-vistas/>.

Creemos vistas de lista y detalle para recuperar objetos `Subject`. Crea un nuevo archivo dentro el directorio `curso / api /` y asígnele el nombre `views.py`. Agregue el siguiente código:

```
from rest_framework import generics
from ..models import Asunto
from ..serializers import SubjectSerializer
```

[447]

Construyendo una API

```
class SubjectListView (generics.ListAPIView):
    queryset = Subject.objects.all ()
    serializer_class = SubjectSerializer

class SubjectDetailView (generics.RetrieveAPIView):
    queryset = Subject.objects.all ()
    serializer_class = SubjectSerializer
```

En este código, se utiliza el genérico `ListAPIView` y `RetrieveAPIView` vistas del marco REST. Incluye un parámetro de URL `pk` para que la vista de detalles lo recupere el objeto para la clave primaria dada. Ambas vistas tienen los siguientes atributos:

- `queryset` : el `QuerySet` base que se utilizará para recuperar objetos

- `serializer_class` : la clase para serializar objetos

Agreguemos patrones de URL para sus vistas. Crea un nuevo archivo dentro de los cursos / api / directorio, asígnele el nombre `urls.py` y haga que tenga el siguiente aspecto:

```

desde la ruta de importación django.urls
de . importar vistas

app_name = 'cursos'

urlpatterns = [
    ruta ('sujetos /',
        views.SubjectListView.as_view (),
        name = 'subject_list'),

    ruta ('sujetos / <pk> /',
        views.SubjectDetailView.as_view (),
        name = 'subject_detail'),
]
```

Editar el principal `urls.py` archivo de la Educa proyecto e incluir los patrones de API, como sigue:

```

urlpatterns = [
    # ...
    ruta ('api /', include ('cursos.api.urls', espacio de nombres = 'api')),
]
```

Utiliza el espacio de nombres `api` para las URL de su API. Asegúrese de que su servidor esté funcionando con el comando `python manage.py runserver`. Abra el caparazón y recupere el URL `http://127.0.0.1:8000/api/subjects/` con `curl`, como sigue:

```
curl http://127.0.0.1:8000/api/subjects/
```

[448]

Obtendrá una respuesta similar a la siguiente:

```

[
  {
    "id": 1,
    "title": "Matemáticas",
    "slug": "matemáticas"
  },
  {
    "id": 2,
    "title": "Música",
    "slug": "música"
  },
  {
    "id": 3,
    "title": "Física",
    "slug": "física"
  },
  {
    "id": 4,
    "title": "Programación",
    "slug": "programación"
  }
]
```

]

Para obtener una respuesta JSON más legible y con sangría adecuada, puede usar curl con la utilidad json_pp , de la siguiente manera:

```
curl http://127.0.0.1:8000/api/subjects/ | json_pp
```

La respuesta HTTP contiene una lista de objetos Subject en formato JSON. Si tu El sistema operativo no viene con curl instalado, puedes descargarlo desde <https://curl.haxx.se/dlwin/>. En lugar de rizar , también puedes usar cualquier otra herramienta. para enviar solicitudes HTTP personalizadas, incluida una extensión de navegador como Postman, al cual puedes llegar <https://www.getpostman.com/>.

[449]

Construyendo una API

Abra `http://127.0.0.1:8000/api/subjects/` en su navegador. Verás REST API navegable de framework, de la siguiente manera:

Figura 12.1: La página de la lista de temas en la API navegable del marco REST

Esta interfaz HTML la proporciona el renderizador `BrowsableAPIRenderer` . Eso muestra los encabezados y el contenido de los resultados, y le permite realizar solicitudes. Tú También puede acceder a la vista de detalles de la API para un objeto Subject al incluir su ID en el URL.

Abra `http://127.0.0.1:8000/api/subjects/1/` en su navegador. Verás un Objeto de sujeto único representado en formato JSON.

Crear serializadores anidados

Va a crear un serializador para el modelo del curso . Edita la `api/serializers.py` de la aplicación de cursos y agregue el siguiente código:

```
desde ..models import Course

class CourseSerializer(serializers.ModelSerializer):
    class Meta:
        modelo = Curso
        fields = ['id', 'subject', 'title', 'slug', 'overview',
```

Echemos un vistazo a cómo se serializa un objeto de curso . Abre el caparazón, ejecuta Python manage.py shell y ejecute el siguiente código:

```
>>> de rest_framework.renderers importar JSONRenderer
>>> de cursos.modelos de importación Curso
>>> de cursos.api.serializers importar CourseSerializer
>>> curso = Course.objects.latest ('id')
>>> serializer = CourseSerializer (curso)
>>> JSONRenderer (). Render (serializer.data)
```

Obtendrá un objeto JSON con los campos que incluyó en CourseSerializer . Puede ver que los objetos relacionados del administrador de módulos se serializan como una lista de claves primarias, de la siguiente manera:

```
"módulos": [6, 7, 9, 10]
```

Desea incluir más información sobre cada módulo, por lo que debe serializar Modular objetos y anidarlos. Modifica el código anterior de los serializadores / api.py de la aplicación de cursos para que tenga el siguiente aspecto:

```
desde serializadores de importación rest_framework
from ..models import Course, Module

class ModuleSerializer (serializers.ModelSerializer):
    class Meta:
        modelo = Módulo
        campos = ['orden', 'título', 'descripción']

class CourseSerializer (serializers.ModelSerializer):
    módulos = ModuleSerializer (muchos = True, read_only = True)

    class Meta:
        modelo = Curso
        fields = ['id', 'subject', 'title', 'slug', 'overview',
            'creado', 'propietario', 'módulos']
```

Usted define ModuleSerializer para proporcionar serialización para el modelo de Módulo . Luego, agrega un atributo de módulos a CourseSerializer para anidar el ModuleSerializer serializador. Establece muchos = Verdadero para indicar que está serializando varios objetos. El parámetro read_only indica que este campo es de solo lectura y no debe ser incluido en cualquier entrada para crear o actualizar objetos.

Abra el shell y cree una instancia de CourseSerializer nuevamente. Renderizar el Atributo de datos del serializador con JSONRenderer . Esta vez, los módulos enumerados son siendo serializado con el serializador ModuleSerializer anidado , de la siguiente manera:

```
"módulos": [
    {
        "orden": 0,
        "title": "Introducción a la descripción general",
        "description": "Una breve descripción general sobre Web Framework".
    },
    {
        "orden": 1,
        "title": "Configurando Django",
        "description": "Cómo instalar Django".
    },
    ...
]
```

Puede leer más sobre serializadores en <https://www.django-rest-framework.org/api-guide/serializers/>.

Crear vistas de API personalizadas

El marco REST proporciona una clase APIView que crea la funcionalidad de API sobre La clase View de Django . La clase APIView difiere de View al usar el marco REST objetos de solicitud y respuesta personalizados , y manejo de excepciones de APIException para devolver las respuestas HTTP adecuadas. También tiene una autenticación integrada y sistema de autorización para gestionar el acceso a las vistas.

Va a crear una vista para que los usuarios se inscriban en los cursos. Edite la api / views.py archivo de la aplicación de cursos y agregue el siguiente código:

```
desde django.shortcuts import get_object_or_404
de rest_framework.views importar APIView
de rest_framework.response importar Respuesta
desde ..models import Course

clase CourseEnrollView (APIView):
    def post (self, request, pk, format = None):
        course = get_object_or_404 (Curso, pk = pk)
        course.students.add (request.user)
        respuesta de retorno ({'inscrito': True})
```

[452]

La vista CourseEnrollView maneja la inscripción de usuarios en los cursos. El precedente el código es el siguiente:

1. Crea una vista personalizada que subclasifica APIView .
2. Defina un método post () para las acciones POST . Ningún otro método HTTP ser permitido para esta vista.
3. Espera un parámetro de URL pk que contenga el ID de un curso. Recuperas el curso por el parámetro pk dado y generar una excepción 404 si no es encontró.

4. Agrega el usuario actual a la relación de varios a varios de los estudiantes
Objeto del curso y devuelva una respuesta satisfactoria.

Edite el archivo `api / urls.py` y agregue el siguiente patrón de URL para el

Vista de `CourseEnrollView` :

```
ruta ('cursos / <pk> / enroll /',
     views.CourseEnrollView.as_view (),
     name = 'course_enroll'),
```

Teóricamente, ahora podría realizar una solicitud POST para inscribir al usuario actual sobre un curso. Sin embargo, debe poder identificar al usuario y prevenir que los usuarios no autenticados accedan a esta vista. Veamos cómo la autenticación de API y los permisos funcionan.

Manejo de la autenticación

El marco REST proporciona clases de autenticación para identificar al usuario que realiza la solicitud. Si la autenticación es exitosa, el marco establece la autenticación

Objeto de usuario en `request.user` . Si no hay ningún usuario autenticado, una instancia de Django En su lugar, se establece `AnonymousUser` .

El marco REST proporciona los siguientes backends de autenticación:

- `BasicAuthentication` : esta es la autenticación básica HTTP. El usuario y El cliente envía la contraseña en el encabezado HTTP de autorización codificado con Base64. Puede obtener más información al respecto en https://en.wikipedia.org/wiki/autenticación_de_acceso_básico.
- `TokenAuthentication` : esta es la autenticación basada en token. Un modelo Token se utiliza para almacenar tokens de usuario. Los usuarios incluyen el token en la autorización Encabezado HTTP para autenticación.

[453]

Construyendo una API

- `SessionAuthentication` : utiliza el backend de sesión de Django para autenticación. Este backend es útil para realizar autenticación Solicitudes AJAX a la API desde la interfaz de su sitio web.
- `RemoteUserAuthentication` : esto le permite delegar la autenticación a su servidor web, que establece una variable de entorno `REMOTE_USER` .

Puede crear un backend de autenticación personalizado subclasificando el Clase `BaseAuthentication` proporcionada por el marco REST y anulando la `Authenticate ()` método.

Puede configurar la autenticación por vista o configurarla globalmente con `DEFAULT_ Configuración de AUTHENTICATION_CLASSES` .

La autenticación solo identifica al usuario que realiza la solicitud. Eso no permitirá ni denegará el acceso a las vistas. Tienes que usar permisos para restringir el acceso a las vistas.

Puede encontrar toda la información sobre autenticación en <https://www.django-rest-framework.org/api-guide/authentication/>.

Añadamos `BasicAuthentication` a la vista. Edite el archivo `api / views.py` de la aplicación de cursos y agregue un atributo `authentication_classes` a `CourseEnrollView` , de la siguiente manera:

```
de rest_framework.authentication import BasicAuthentication
```

```
class CourseEnrollView (APIView):
```

```
    Authentication_classes = (Autenticación básica,)
```

```
    # ...
```

Los usuarios serán identificados por las credenciales establecidas en el encabezado de Autorización del Solicitud HTTP.

Agregar permisos a las vistas

El marco REST incluye un sistema de permisos para restringir el acceso a las vistas. Algunos de los permisos integrados del marco REST son:

- AllowAny : acceso sin restricciones, independientemente de si un usuario está autenticado o no.
- IsAuthenticated : permite el acceso solo a usuarios autenticados.

[454]

- IsAuthenticatedOrReadOnly : acceso completo a usuarios autenticados. Los usuarios anónimos solo pueden ejecutar métodos de lectura como GET , HEAD u OPCIONES .
- DjangoModelPermissions : permisos vinculados a django.contrib.auth . los vista requiere un queryset atributo. Solo usuarios autenticados con modelo los permisos asignados tienen permiso.
- DjangoObjectPermissions : permisos de Django por objeto.

Si a los usuarios se les niega el permiso, generalmente obtendrán uno de los siguientes HTTP códigos de error:

- HTTP 401 : no autorizado
- HTTP 403 : permiso denegado

Puede leer más información sobre permisos en <https://www.django-rest-framework.org/api-guide/permissions/>.

Editar la API / views.py archivo del cursos de aplicación y añadir un permission_ clases atribuyen a CourseEnrollView , de la siguiente manera:

```
de rest_framework.authentication import BasicAuthentication
de rest_framework.permissions import IsAuthenticated
```

```
class CourseEnrollView (APIView):
```

```
    Authentication_classes = (Autenticación básica,)
```

```
    permisos_clases = (IsAuthenticated,)
```

```
    # ...
```

Incluye el permiso IsAuthenticated . Esto evitará que los usuarios anónimos de acceder a la vista. Ahora, puede realizar una solicitud POST a su nueva API método.

Asegúrese de que el servidor de desarrollo se esté ejecutando. Abra el shell y ejecute lo siguiente mando:

```
curl -i -X POST http://127.0.0.1:8000/api/courses/1/enroll/
```

Obtendrá la siguiente respuesta:

```
HTTP / 1.1 401 no autorizado
```

```
...
```

```
{"detail": "No se proporcionaron las credenciales de autenticación"}
```

[455]

Construyendo una API

Obtuvo un código HTTP 401 como se esperaba, ya que no está autenticado. Usemos autenticación básica con uno de sus usuarios. Ejecute el siguiente comando, reemplazando estudiante: contraseña con las credenciales de un usuario existente:

```
curl -i -X POST -u estudiante: contraseña http://127.0.0.1:8000/api/courses/1/inscribirse/
```

Obtendrá la siguiente respuesta:

```
HTTP / 1.1 200 OK
```

```
...
```

```
{"inscrito": verdadero}
```

Puede acceder al sitio de administración y verificar que el usuario ahora esté inscrito en el curso.

Crear conjuntos de vistas y enrutadores

Los ViewSets le permiten definir las interacciones de su API y dejar que el marco REST construya las URL dinámicamente con un objeto Router . Al usar conjuntos de vistas, puede evitar repetir la lógica para múltiples vistas. Los conjuntos de vistas incluyen acciones para lo siguiente operaciones estándar:

- Crear operación: crear ()
- Recuperar operación: lista () y recuperar ()
- Operación de actualización : actualización () y actualización_parcial ()
- Eliminar operación: destruir ()

Creemos un conjunto de vistas para el modelo del curso . Edite el archivo `api / views.py` y agregue el siguiente código:

```
desde rest_framework importar conjuntos de vistas
desde .serializers importar CourseSerializer

clase CourseViewSet (viewsets.ReadOnlyModelViewSet):
    queryset = Course.objects.all ()
    serializer_class = CourseSerializer
```

Tu subclase `ReadOnlyModelViewSet` , que proporciona la lista de acciones de solo lectura () y `retrieve ()` a ambos objetos de lista, o recupera un solo objeto.

Edite el archivo `api / urls.py` y cree un enrutador para su conjunto de vistas, de la siguiente manera:

```
de la ruta de importación django.urls, incluir
desde enrutadores de importación rest_framework
```

[456]


```
de . importar vistas

router = routers.DefaultRouter ()
router.register ('cursos', vistas.CourseViewSet)

urlpatterns = [
    # ...
    ruta ('', incluir (router.urls)),
]
```

Crea un objeto DefaultRouter y registra su conjunto de vistas con los cursos prefijo. El enrutador se encarga de generar URL automáticamente para su conjunto de vistas.

Abra `http://127.0.0.1:8000/api/` en su navegador. Verá que el enrutador enumera todos los conjuntos de vistas en su URL base, como se muestra en la siguiente captura de pantalla:

Figura 12.2: La página raíz de la API de la API navegable del marco REST

Puede acceder a `http://127.0.0.1:8000/api/courses/` para recuperar la lista de cursos.

Puede obtener más información sobre los conjuntos de vistas en <https://www.django-rest-framework.org/api-guide/viewsets/>. También puede encontrar más información sobre enrutadores en <https://www.django-rest-framework.org/api-guide/routers/>.

Agregar acciones adicionales a conjuntos de vistas

Puede agregar acciones adicionales a conjuntos de vistas. Cambiemos tu anterior

CourseEnrollView vista en una acción de conjunto de vistas personalizado. Edite el archivo `api / views.py` y modifique la clase CourseViewSet para que tenga el siguiente aspecto:

```
de rest_framework.decorators acción de importación
```

[457]

Construyendo una API

```
class CourseViewSet (viewsets.ReadOnlyModelViewSet):
    queryset = Course.objects.all ()
    serializer_class = CourseSerializer

    @acción (detalle = Verdadero,
            métodos = ['publicar'],
```

```

        authentication_classes = [BasicAuthentication],
        permis_classes = [IsAuthenticated])

def enroll (self, request, * args, ** kwargs):
    curso = self.get_object ()
    course.students.add (request.user)
    respuesta de retorno ({'inscrito': True})

```

En el código anterior, agrega un método enroll () personalizado que representa un acción adicional para este conjunto de vistas. El código anterior es el siguiente:

1. Utiliza el decorador de acciones del marco con el parámetro detalle = Verdadero para especificar que se trata de una acción a realizar en un solo objeto.
2. El decorador te permite agregar atributos personalizados para la acción. Tú especificar que solo el método post () está permitido para esta vista y establecer el clases de autenticación y permisos.
3. Utiliza self.get_object () para recuperar el objeto Course .
4. Agrega el usuario actual a la relación de varios a varios estudiantes y devuelve una respuesta de éxito personalizada.

Edite el archivo api / urls.py y elimine la siguiente URL, ya que no la necesita nunca más:

```

ruta ('cursos / <pk> / enroll /',
     views.CourseEnrollView.as_view (),
     name = 'course_enroll'),

```

Luego, edite el archivo api / views.py y elimine la clase CourseEnrollView .

La URL para inscribirse en los cursos ahora la genera automáticamente el enrutador. los

La URL sigue siendo la misma, ya que se construye dinámicamente usando su nombre de acción enroll .

Creando permisos personalizados

Quieres que los alumnos puedan acceder a los contenidos de los cursos en los que están matriculados

en. Solo los estudiantes matriculados en un curso deberían poder acceder a su contenido. El mejor

La forma de hacer esto es con una clase de permiso personalizada. Django proporciona un BasePermission clase que le permite definir los siguientes métodos:

[458]

- has_permission () : verificación de permisos a nivel de vista
- has_object_permission () : verificación de permisos a nivel de instancia

Estos métodos deben devolver True para otorgar acceso o False en caso contrario.

Cree un nuevo archivo dentro del directorio cursos / api / y asígnele el nombre permissions.py .

Agregue el siguiente código:

```

de rest_framework.permissions importar BasePermission

clase IsEnrolled (BasePermission):
    def has_object_permission (self, request, view, obj):
        return obj.students.filter (id = request.user.id) .exists ()

```

Subclase del BasePermission clase y anular el has_object_

permiso () . Comprueba que el usuario que realiza la solicitud está presente en el relación de los estudiantes con el objeto del curso . Vas a utilizar IsEnrolled permiso siguiente.

Serializar los contenidos del curso

Necesita serializar los contenidos del curso. El modelo de contenido incluye un extranjero genérico

clave que le permite asociar objetos de diferentes modelos de contenido. Sin embargo, agregaste un método `render()` común para todos los modelos de contenido del capítulo anterior. Usted puede utilizar este método para proporcionar contenido renderizado a su API.

Edite el archivo `api / serializers.py` de la aplicación de cursos y agregue lo siguiente codificarlo:

```
de ..modelos importar contenido

class ItemRelatedField(serializers.RelatedField):
    def to_representation(self, value):
        devolver valor.render()

class ContentSerializer(serializers.ModelSerializer):
    item = ItemRelatedField(read_only = True)

    class Meta:
        modelo = Contenido
        campos = ['pedido', 'artículo']
```

En este código, usted define un campo personalizado subclasificando el serializador `RelatedField` campo proporcionado por el marco REST y anulando el `to_representation()` método. Defina el serializador `ContentSerializer` para el modelo de contenido y utilice el campo personalizado para la clave externa genérica del artículo .

[459]

Construyendo una API

Necesita un serializador alternativo para el modelo del módulo que incluye su contenido, y también un serializador de curso extendido . Edite el archivo `api / serializers.py` y agregue el siguiente código:

```
class ModuleWithContentsSerializer(serializers.ModelSerializer):
    contenido = ContentSerializer(muchos = Verdadero)

    class Meta:
        modelo = Módulo
        campos = ['orden', 'título', 'descripción', 'contenido']

class CourseWithContentsSerializer(serializers.ModelSerializer):
    módulos = ModuleWithContentsSerializer(muchos = Verdadero)

    class Meta:
        modelo = Curso
        campos = ['id', 'asunto', 'título', 'slug',
                  'descripción general', 'creado', 'propietario', 'módulos']
```

Creemos una vista que imite el comportamiento de la acción `retrieve()` , pero que incluya los contenidos del curso. Edite el archivo `api / views.py` y agregue el siguiente método al Clase `CourseViewSet` :

```
desde .permissions importar IsEnrolled
desde .serializers importar CourseWithContentsSerializer

class CourseViewSet(viewsets.ReadOnlyModelViewSet):
    # ...
    @acción (detalle = Verdadero,
            métodos = ['obtener'],
            serializer_class = CourseWithContentsSerializer,
            authentication_classes = [BasicAuthentication],
            permis_classes = [IsAuthenticated, IsEnrolled])
    def contenido(self, request, * args, ** kwargs):
        return self.retrieve(solicitud, * args, ** kwargs)
```

La descripción de este método es la siguiente:

- Utiliza el decorador de acciones con el parámetro `detail = True` para especificar un acción que se realiza en un solo objeto.
- Especifica que solo se permite el método GET para esta acción.
- Utiliza la nueva clase de serializador `CourseWithContentsSerializer` que incluye contenidos del curso renderizados.

[460]

- Utiliza los permisos `IsAuthenticated` y `IsEnrolled` personalizados .
Al hacerlo, se asegura de que solo los usuarios inscritos en el curso puedan para acceder a su contenido.
- Utiliza la acción `retrieve()` existente para devolver el objeto `Course` .

Abra `http://127.0.0.1:8000/api/courses/1/contents/` en su navegador.
Si accede a la vista con las credenciales correctas, verá que cada módulo del curso incluye el HTML renderizado para los contenidos del curso, de la siguiente manera:

```
{
  "orden": 0,
  "title": "Introducción a Django",
  "description": "Breve introducción a Django Web Framework.",
  "contenido": [
    {
      "orden": 0,
      "item": "<p> Conoce a Django. Django es un
Marco web de Python
... </p> "
    },
    {
      "orden": 1,
      "elemento": "\n <iframe ancho = \" 480 \" alto = \" 360 \" "
src = \" http://www.youtube.com/embed/bgV39DlmZ2U?
wmode = opaco \" "
frameborder = \"0 \" allowfullscreen> </iframe> \n "
    }
  ]
}
```

Ha creado una API simple que permite que otros servicios accedan al curso. aplicación mediante programación. El marco REST también le permite manejar la creación y editar objetos con el conjunto de vistas `ModelViewSet`. Hemos cubierto los principales aspectos del marco Django REST, pero encontrará más información sobre sus características en su extensa documentación en <https://www.django-rest-framework.org/>.

Consumir la API RESTful

Ahora que ha implementado una API, puede consumirla en un programa manera de otras aplicaciones. Puede interactuar con la API usando JavaScript en la interfaz de su aplicación, de manera similar a las funcionalidades AJAX que construido en el *Capítulo 5* , *Compartir contenido en su sitio web* . También puedes consumir la API desde aplicaciones creadas con Python o cualquier otro lenguaje de programación.

[461]

Construyendo una API

Va a crear una aplicación Python simple que use la API RESTful para recuperar todos los cursos disponibles y luego inscribir a un estudiante en todos ellos. Aprenderás cómo autenticarse contra la API mediante la autenticación básica HTTP y realizar Solicitudes GET y POST .

Utilizará la biblioteca de solicitudes de Python para consumir la API. Solicitudes es el biblioteca HTTP más popular para Python. Abstrae la complejidad de lidiar con HTTP solicita y proporciona una interfaz muy simple para consumir servicios HTTP. Puede encontrar la documentación de la biblioteca de solicitudes en <https://solicitudes.readthedocs.io/en/master/>.

Abra el shell e instale la biblioteca de solicitudes con el siguiente comando:

solicitudes de instalación de pip == 2.23

Crear un nuevo directorio al lado de la Educa directorio del proyecto y el nombre api_ ejemplos . Cree un nuevo archivo dentro del directorio api_examples / y asígnele un nombre enroll_all.py . La estructura del archivo ahora debería verse así:

```
api_examples /
  enroll_all.py
educa /
...
```

Edite el archivo enroll_all.py y agréguele el siguiente código:

```
solicitudes de importación

base_url = 'http://127.0.0.1:8000/api/'

# recuperar todos los cursos
r = request.get(f'{base_url} cursos /')
cursos = r.json ()

available_courses = ','.join ([curso ['título'] para curso en cursos])
imprimir (f'Cursos disponibles: {available_courses} ')
```

En este código, realiza las siguientes acciones:

1. Importa la biblioteca de solicitudes y define la URL base para la API.
2. Utiliza request.get () para recuperar datos de la API enviando un GET solicitud a la URL http://127.0.0.1:8000/api/courses/ . Esta API El punto final es de acceso público, por lo que no requiere autenticación.

3. Utiliza el método json () del objeto de respuesta para decodificar los datos JSON devuelto por la API.

4. Imprime el atributo del título de cada curso.

Inicie el servidor de desarrollo desde el directorio del proyecto educa con lo siguiente mando:

```
python manage.py runserver
```

En otro shell, ejecute el siguiente comando desde el directorio api_examples / :

```
python enroll_all.py
```

Verá un resultado con una lista de todos los títulos de los cursos, como este:

Cursos disponibles: Introducción a Django, Python para principiantes, Álgebra lo esencial

Esta es su primera llamada automática a su API.

Edite el archivo enroll_all.py y cámbielo para que se vea así:

```
solicitudes de importación

nombre de usuario = ''
contraseña = ''

base_url = 'http://127.0.0.1:8000/api/'

# recuperar todos los cursos
r = request.get(f'{base_url} cursos /')
cursos = r.json ()

available_courses = ','.join ([curso ['título'] para curso en cursos])
imprimir (f'Cursos disponibles: {available_courses} ')

por curso en cursos:
    course_id = curso ['id']
    course_title = curso ['título']
    r = request.post (f '{base_url} cursos / {course_id} / enroll /',
                                auth = (nombre de usuario, contraseña))

    si r.status_code == 200:
        # solicitud exitosa
        imprimir (f 'inscrito con éxito en {course_title}')
```

Reemplace los valores de las variables de nombre de usuario y contraseña con las credenciales de un usuario existente.

[463]

Construyendo una API

Con el nuevo código, realiza las siguientes acciones:

1. Tú defines el nombre de usuario y la contraseña del alumno al que quieres matricular en cursos.
2. Repita los cursos disponibles recuperados de la API.
3. Almacena el atributo ID del curso en la variable `course_id` y el título atributo en la variable `course_title` .
4. Utiliza `request.post ()` para enviar una solicitud POST a la URL `http://127.0.0.1:8000/api/courses/[id]/enroll/` para cada curso. Esta URL corresponde a la vista de la API `CourseEnrollView` , que le permite inscribir a un usuario en un curso. Usted crea la URL para cada curso usando la variable `course_id` . La vista `CourseEnrollView` requiere autenticación. Utiliza el permiso `IsAuthenticated` y el Clase de autenticación `BasicAuthentication` . La biblioteca de solicitudes admite Autenticación básica HTTP lista para usar. Utiliza el parámetro `auth` para pasar una tupla con el nombre de usuario y la contraseña para autenticar al usuario usando Autenticación básica HTTP.

5. Si el código de estado de la respuesta es 200 OK , imprime un mensaje para indicar que el usuario se ha inscrito con éxito en el curso.

Puede utilizar diferentes tipos de autenticación con Solicitudes. Puedes encontrar más información sobre autenticación con Solicitudes en <https://requests.readthedocs.io/en/master/user/authentication/>.

Ejecute el siguiente comando desde el directorio `api_examples /` :

```
python enroll_all.py
```

Ahora verá un resultado como este:

Cursos disponibles: Introducción a Django, Python para principiantes, Álgebra lo esencial

Inscrito con éxito en Introducción a Django

Inscrito con éxito en Python para principiantes

Inscrito con éxito en los conceptos básicos de álgebra

¡Excelente! Ha inscrito correctamente al usuario en todos los cursos disponibles utilizando la API. Verá un mensaje de inscripción exitosa para cada curso en la plataforma. Como puede ver, es muy fácil consumir la API desde cualquier otra aplicación. Usted puede construir sin esfuerzo otras funcionalidades basadas en la API y deje que otros integren su API en sus aplicaciones.

[464]

Resumen

En este capítulo, aprendió a usar el marco REST de Django para construir un RESTful API para su proyecto. Creó serializadores y vistas para modelos, y creó vistas API personalizadas. También agregó autenticación a su API y restringió acceso a las vistas de la API mediante permisos. A continuación, descubrió cómo crear permisos e implementó conjuntos de vistas y enrutadores. Finalmente, usaste el Solicita a la biblioteca que consume la API desde un script externo de Python.

El siguiente capítulo le enseñará cómo construir un servidor de chat usando Canales Django. Implementará la comunicación asincrónica utilizando WebSockets y use Redis para configurar una capa de canal.

13

Construyendo un servidor de chat

En el capítulo anterior, creó una API RESTful para su proyecto. En este capítulo, creará un servidor de chat para estudiantes que utilicen los canales Django. Serán estudiantes poder acceder a una sala de chat diferente para cada curso en el que estén inscritos. A cree el servidor de chat, aprenderá cómo servir su proyecto Django a través de **Interfaz de puerta de enlace de servidor asincrónica (ASGI)**, y usted implementará comunicación asíncrona.

En este capítulo, podrá:

- Agrega canales a tu proyecto
- Construya un consumidor de WebSocket y un enrutamiento apropiado
- Implementar un cliente WebSocket
- Habilite una capa de canal con Redis
- Haga que su consumidor sea completamente asincrónico

Creando una aplicación de chat

Va a implementar un servidor de chat para proporcionar a los estudiantes una sala de chat para cada curso. Los estudiantes inscritos en un curso podrán acceder al curso sala de chat e intercambio de mensajes en tiempo real. Usarás canales para crear esta funcionalidad. Channels es una aplicación de Django que extiende Django para manejar protocolos que requieren conexiones de larga duración, como WebSockets, chatbots, o MQTT (un transporte ligero de mensajes de publicación / suscripción de uso común en proyectos de Internet de las cosas).

Construyendo un servidor de chat

Con los canales, puede implementar fácilmente funcionalidades asincrónicas o en tiempo real en su proyecto además de sus vistas sincrónicas HTTP estándar. Vas a Comience agregando una nueva aplicación a su proyecto. La nueva aplicación contendrá la lógica del servidor de chat.

Ejecute el siguiente comando desde el directorio educa del proyecto para crear el nuevo estructura de archivo de la aplicación:

chat de django-admin startapp

Edite el archivo settings.py del proyecto educa y active la aplicación de chat en su proyecto editando la configuración INSTALLED_APPS , de la siguiente manera:

```
INSTALLED_APPS = [
    # ...
    'charla',
]
```

La nueva aplicación de chat ahora está activa en su proyecto.

Implementar la vista de la sala de chat

Proporcionará a los estudiantes una sala de chat diferente para cada curso. Necesitas crear una vista para que los estudiantes se unan a la sala de chat de un curso determinado. Solo estudiantes quienes estén inscritos en un curso podrán acceder a la sala de chat del curso.

Edite el archivo views.py de la nueva aplicación de chat y agregue el siguiente código:

```
desde django.shortcuts import render, get_object_or_404
desde django.http import HttpResponseRedirect
de django.contrib.auth.decorators import login_required

@login_required
def course_chat_room (solicitud, course_id):
    tratar:
        # recuperar el curso con el ID dado al que se unió el usuario actual
        curso = request.user.courses_joined.get (id = course_id)
    except:
        # usuario no es alumno del curso o el curso no existe
        return HttpResponseRedirect ()
    return render (solicitud, 'chat / room.html', {'curso': curso})
```

Esta es la vista course_chat_room . En esta vista, usa @login_required decorador para evitar que cualquier usuario no autenticado acceda a la vista. los La vista recibe un parámetro course_id obligatorio que se utiliza para recuperar el curso. con la identificación dada .

[468]

Se accede a los cursos en los que está matriculado el usuario a través de la relación `cursos_` se unió y recupera el curso con la identificación dada de ese subconjunto de cursos. Si el curso con la identificación proporcionada no existe o el usuario no está inscrito en él, devuelve una respuesta `HttpResponseForbidden`, que se traduce en una respuesta HTTP con estado 403. Si el curso con la identificación dada existe y el usuario está inscrito en , renderiza la plantilla `chat / room.html`, pasando el objeto del curso al contexto de plantilla.

Necesita agregar un patrón de URL para esta vista. Crea un nuevo archivo dentro del chat directorio de la aplicación y el nombre `urls.py`. Agregue el siguiente código:

```
desde la ruta de importación django.urls
de . importar vistas

app_name = 'chat'

urlpatterns = [
    ruta ('habitación / <int: course_id> /', views.course_chat_room,
        name = 'course_chat_room'),
]
```

Este es el archivo de patrones de URL inicial para la aplicación de chat. Tú defines el curso_ patrón de URL de `chat_room`, incluido el parámetro `course_id` con el prefijo `int`, ya que solo espera un valor entero aquí.

Incluya los nuevos patrones de URL de la aplicación de chat en los principales patrones de URL de el proyecto. Editar el principal `urls.py` archivo de la Educa proyecto y agregue el siguiente línea a él:

```
urlpatterns = [
    # ...
    ruta ('chat /', include ('chat.urls', namespace = 'chat')),
]
```

Los patrones de URL para la aplicación de chat se agregan al proyecto en el `chat / ruta`.

Necesita crear una plantilla para la vista `course_chat_room`. Esta plantilla contener un área para visualizar los mensajes que se intercambian en el chat y un texto entrada con un botón de envío para enviar mensajes de texto al chat.

Cree la siguiente estructura de archivos dentro del directorio de la aplicación de chat :

```
plantillas/
  charla/
    room.html
```

Construyendo un servidor de chat

Edite la plantilla `chat / room.html` y agregue el siguiente código:

```
{% extiende "base.html"%}

{% block title%} Sala de chat para "{{course.title}}"{% endblock%}

{% block content%}
<div id = "chat">
</div>
<div id = "chat-input">
    <input id = "chat-message-input" type = "text">
    <input id = "chat-message-submit" type = "submit" value = "Enviar">
</div>
{% endblock%}
```

```
{% block domready%}  
{% endblock%}
```

Esta es la plantilla para la sala de chat del curso. En esta plantilla, extiendes la base.

html de su proyecto y llene su bloque de contenido . En la plantilla, define un elemento HTML <div> con el ID de chat que usará para mostrar el chat mensajes enviados por el usuario y por otros estudiantes. También define un segundo <div> elemento con una entrada de texto y un botón de envío que permitirá al usuario enviar mensajes. Incluya el bloque domready definido por la plantilla base.html , que vas a implementar más tarde usando JavaScript, para establecer una conexión con un WebSocket y enviar o recibir mensajes.

Ejecute el servidor de desarrollo y abra <http://127.0.0.1:8000/chat/room/1/> en su navegador, reemplazando 1 con la identificación de un curso existente en la base de datos. Acceda a la sala de chat con un usuario registrado que esté inscrito en el curso. Vas a ver la siguiente pantalla:

Figura 13.1: La página de la sala de chat del curso

Esta es la pantalla de la sala de chat del curso que los estudiantes usarán para discutir temas dentro de un curso.

[470]

Desactivación de la caché por sitio

En el *Capítulo 11* , *Procesamiento y almacenamiento en caché de contenido* , agregó una caché en todo el sitio a su Proyecto Django. Ahora, deberá seguir un enfoque más granular para el almacenamiento en caché para evitar que las páginas de la sala de chat se almacenen en caché. Desactivarás el por sitio caché para evitar el almacenamiento en caché en todo el sitio y solo use el almacenamiento en caché cuando sea necesario.

Edite el archivo settings.py y comente el UpdateCacheMiddleware y FetchFromCacheMiddleware clases de la configuración MIDDLEWARE , como sigue:

```
MIDDLEWARE = [  
    'django.middleware.security.SecurityMiddleware',  
    'django.contrib.sessions.middleware.SessionMiddleware',  
    # 'django.middleware.cache.UpdateCacheMiddleware',  
    'django.middleware.common.CommonMiddleware',  
    # 'django.middleware.cache.FetchFromCacheMiddleware',  
    # ...  
]
```

Ha desactivado la caché por sitio en su proyecto para evitar la nueva sala de chat vista desde la caché. A continuación, aprenderá cómo agregar canales a su Django proyecto para implementar un servidor de chat en tiempo real.

Django en tiempo real con canales

Está creando un servidor de chat para proporcionar a los estudiantes una sala de chat para cada curso. Los estudiantes inscritos en un curso podrán acceder a la sala de chat del curso y intercambiar mensajes. Esta funcionalidad requiere comunicación en tiempo real entre el servidor y el cliente. El cliente debería poder conectarse al chat y enviar o recibir datos en cualquier momento. Hay varias formas de implementar esta función. utilizando sondeo AJAX o sondeo largo en combinación con el almacenamiento de mensajes en

su base de datos o Redis. Sin embargo, no existe una forma eficaz de implementar un chat servidor utilizando una aplicación web síncrona estándar. Vas a construir un servidor de chat mediante comunicación asíncrona a través de ASGI.

Aplicaciones asincrónicas que utilizan ASGI

Django generalmente se implementa mediante la **Interfaz de puerta de enlace del servidor web (WSGI)**, que es la interfaz estándar para que las aplicaciones Python manejen solicitudes HTTP. Sin embargo, para trabajar con aplicaciones asincrónicas, necesita usar otra interfaz llamada ASGI, que también puede manejar solicitudes de WebSocket. ASGI es el Python emergente estándar para aplicaciones y servidores web asíncronos.

[471]

Construyendo un servidor de chat

Django 3 viene con soporte para ejecutar Python asíncrono a través de ASGI, pero todavía no es compatible con vistas asincrónicas o middleware. Sin embargo, como se mencionó, Channels extiende Django para manejar no solo HTTP, sino también protocolos que requieren conexiones de larga duración, como WebSockets y chatbots.

WebSockets proporciona comunicación full-duplex al establecer una comunicación persistente, abierta, Conexión bidireccional **del Protocolo de control de transmisión (TCP)** entre servidores y clientela. Vas a utilizar WebSockets para implementar tu servidor de chat.

Puede encontrar más información sobre cómo implementar Django con ASGI en <https://docs.djangoproject.com/en/3.0/howto/deployment/asgi/>.

El ciclo de solicitud / respuesta usando canales

Es importante comprender las diferencias en un ciclo de solicitud entre un estándar Ciclo de solicitud síncrona e implementación de Canales. El siguiente esquema muestra el ciclo de solicitud de una configuración síncrona de Django:

Figura 13.2: El ciclo de solicitud / respuesta de Django

Cuando el navegador envía una solicitud HTTP al servidor web, Django maneja la solicitud y pasa el objeto `HttpRequest` a la vista correspondiente. Los `view` procesa la solicitud y devuelve un objeto `HttpResponse` que se envía de vuelta al navegador como una respuesta HTTP. No hay ningún mecanismo para mantener una apertura conexión o enviar datos al navegador sin una solicitud HTTP asociada.

El siguiente esquema muestra el ciclo de solicitud de un proyecto de Django usando Canales con WebSockets:

[472]

Figura 13.3: El ciclo de solicitud / respuesta de los canales de Django

Channels reemplaza el ciclo de solicitud / respuesta de Django con mensajes que se envían a través de canales. Las solicitudes HTTP todavía se enrutan para ver funciones usando Django, pero se enrutan por canales. Esto permite el manejo de mensajes de WebSockets también, donde haya productores y consumidores que intercambien mensajes una capa de canal. Channels conserva la arquitectura sincrónica de Django, lo que permite puede elegir entre escribir código síncrono y código asíncrono, o un combinación de ambos.

Instalación de canales

Va a agregar canales a su proyecto y configurar el ASGI básico requerido enrutamiento de aplicaciones para que administre las solicitudes HTTP.

Instale Channels en su entorno virtual con el siguiente comando:

canales de instalación de pip == 2.4.0

Edite el archivo settings.py del proyecto educa y agregue canales al archivo INSTALLED_APPLICATIONS de la siguiente manera:

```
INSTALLED_APPS = [  
    # ...  
    'canales',  
]
```

La aplicación de canales ahora está activada en su proyecto.

Channels espera que defina una aplicación raíz única que se ejecutará para todas las solicitudes. Puede definir la aplicación raíz agregando ASGI_APPLICATION ajuste a su proyecto. Esto es similar a la configuración ROOT_URLCONF que apunta a la patrones de URL base de su proyecto. Puede colocar la aplicación raíz en cualquier lugar su proyecto, pero se recomienda ponerlo en un archivo de nivel de proyecto llamado enrutamiento.py .

Cree un nuevo archivo dentro del directorio del proyecto educa junto al archivo settings.py y asígnele el nombre routing.py .

Agregue el siguiente código:

```
desde los canales. enrutamiento de importación ProtocolTypeRouter  
  
application = ProtocolTypeRouter ({  
    # vacío por ahora  
})
```

Luego, agregue la siguiente línea al archivo settings.py de su proyecto:

```
ASGI_APPLICATION = 'educa.routing.application'
```

En el código anterior, define la aplicación ASGI principal que se ejecutará al servir su proyecto Django a través de ASGI. Usas el ProtocolTypeRouter clase proporcionada por Channels como el principal punto de entrada de su sistema de enrutamiento.

ProtocolTypeRouter toma un diccionario que mapea tipos de comunicación como http o websocket a aplicaciones ASGI. Crea una instancia de esta clase con un vacío diccionario que luego llenarás con una ruta para tu aplicación de chat WebSocket consumidor.

Cuando se agrega Canales a la configuración INSTALLED_APPS , toma el control sobre el comando runserver , reemplazando al servidor de desarrollo estándar de Django. Además manejo de enrutamiento de URL a vistas de Django para solicitudes síncronas, los canales El servidor de desarrollo también administra rutas a los consumidores de WebSocket.

[474]

Inicie el servidor de desarrollo con el siguiente comando:

```
python manage.py runserver
```

Verá un resultado similar al siguiente:

Observando cambios de archivos con StatReloader

Realización de comprobaciones del sistema ...

La verificación del sistema no identificó problemas (0 silenciados).

06 de febrero de 2020-23: 12: 33

Django versión 3.0, usando la configuración 'educa.settings'
Iniciando el servidor de desarrollo ASGI / Channels versión 2.4.0 en
`http://127.0.0.1:8000/`

Salga del servidor con `CONTROL-C`.

Compruebe que la salida contiene la línea Iniciando ASGI / Versión de canales

Servidor de desarrollo 2.4.0 . Esta línea confirma que está utilizando los canales
servidor de desarrollo, que es capaz de gestionar síncronos y asíncronos
solicitudes, en lugar del servidor de desarrollo estándar de Django. Solicitudes HTTP
continúan comportándose igual que antes, pero se enrutan a través de *canales* .

Ahora que Channels está instalado en su proyecto, puede crear el servidor de chat para
cursos. Para implementar el servidor de chat para su proyecto, necesitará
siga los siguientes pasos:

1. **Configure un consumidor** : los consumidores son fragmentos de código individuales que pueden manejar
WebSockets de una forma muy similar a las vistas HTTP tradicionales. Tu construirás
un consumidor para leer y escribir mensajes en un canal de comunicación.
2. **Configurar el enrutamiento** : los canales proporcionan clases de enrutamiento que le permiten
combine y apile a sus consumidores. Configuraré el enrutamiento de URL para
su consumidor de chat.
3. **Implementar un cliente WebSocket** : cuando el alumno accede a la sala de chat,
se conectará a WebSocket desde el navegador y enviará o recibirá
mensajes que utilizan JavaScript.
4. **Habilite una capa de canal** : las capas de canal le permiten hablar entre diferentes
instancias de una aplicación. Son una parte útil de hacer una distribución
aplicación en tiempo real. Configuraré una capa de canal usando Redis.

[475]

Construyendo un servidor de chat

Escribiendo un consumidor

Los consumidores son el equivalente a las vistas de Django para aplicaciones asincrónicas. Como
mencionado, manejan WebSockets de una manera muy similar a como las vistas tradicionales
manejan solicitudes HTTP. Los consumidores son aplicaciones ASGI que pueden manejar mensajes,
notificaciones y otras cosas. A diferencia de las vistas de Django, los consumidores están diseñados para
ejecutar la comunicación. Las URL se asignan a los consumidores mediante clases de enrutamiento
que le permiten combinar y apilar consumidores.

Implementemos un consumidor básico que pueda aceptar conexiones WebSocket
y se hace eco de todos los mensajes que recibe de WebSocket. Esta inicial
La funcionalidad permitirá al estudiante enviar mensajes al consumidor y recibir
devuelve los mensajes que envía.

Cree un nuevo archivo dentro del directorio de la aplicación de chat y asígnele el nombre `consumer.py` .
Agregue el siguiente código:

```
import json
from channels.generic.websocket import WebsocketConsumer

class ChatConsumer(WebsocketConsumer):
    def connect(self):
        # aceptar conexión
        self.accept()

    def disconnect(self, close_code):
        pasar
```

```
# recibir mensaje de WebSocket
def recibir (self, text_data):
    text_data_json = json.loads (text_data)
    message = text_data_json ['mensaje']
    # enviar mensaje a WebSocket
    self.send (text_data = json.dumps ({'mensaje': message}))
```

Este es el consumidor de ChatConsumer . Esta clase hereda de los Canales

Clase WebsocketConsumer para implementar un consumidor WebSocket básico. En esto consumidor, implementa los siguientes métodos:

- connect () : se llama cuando se recibe una nueva conexión. Aceptas cualquier conexión con self.accept () . También puede rechazar una conexión llamando a self.close () .
- desconectar () : se llama cuando se cierra el enchufe. Usas pass porque tu no es necesario implementar ninguna acción cuando un cliente cierra la conexión.

[476]

- recibir () : se llama cuando se reciben datos. Esperas que se reciba un mensaje de texto como text_data (esto también podría ser binary_data para datos binarios). Usted paga los datos de texto recibidos como JSON. Por lo tanto, usa json.loads () para cargar los datos JSON recibidos en un diccionario de Python. Accedes al mensaje key, que espera que esté presente en la estructura JSON recibida. A repita el mensaje, envía el mensaje de vuelta al WebSocket con self.send () , transformándolo en formato JSON nuevamente a través de json.dumps () .

La versión inicial de su consumidor ChatConsumer acepta cualquier WebSocket conexión y hace eco al cliente de WebSocket cada mensaje que recibe. Tenga en cuenta que el consumidor aún no transmite mensajes a otros clientes. Construirás esto funcionalidad implementando una capa de canal más tarde.

Enrutamiento

Debe definir una URL para enrutar las conexiones al consumidor de ChatConsumer que han implementado. Canales proporciona clases de enrutamiento que le permiten combinar y apilar consumidores para enviar en función de la conexión. Tu puedes pensar de ellos como el sistema de enrutamiento de URL de Django para aplicaciones asincrónicas.

Cree un nuevo archivo dentro del directorio de la aplicación de chat y asígnele el nombre routing.py . Agregue el siguiente código:

```
from django.urls import re_path
from . consumidores de importación

websocket_urlpatterns = [
    re_path (r'ws / chat / room / (? P <course_id> \ d +) / $ ', consumidores.
    ChatConsumer),
]
```

En este código, asigna un patrón de URL con la clase ChatConsumer que definió en el archivo chat / consumer.py . Usas re_path de Django para definir la ruta con expresiones regulares. La URL incluye un parámetro de número entero llamado course_id . Este parámetro estará disponible en el ámbito del consumidor y le permitirá para identificar la sala de chat del curso a la que se está conectando el usuario.

Es una buena práctica anteponer las URL de WebSocket con / ws / para diferenciarlas de las URL utilizadas para HTTP síncrono estándar peticiones. Esto también simplifica la configuración de producción cuando un HTTP el servidor enruta las solicitudes según la ruta.

Construyendo un servidor de chat

Edite el archivo `routing.py` global ubicado junto al archivo `settings.py` para que se vea Me gusta esto:

```

desde channels.auth import AuthMiddlewareStack
desde los canales de importación de enrutamiento ProtocolTypeRouter , URLRouter
importar chat.routing

application = ProtocolTypeRouter ({
    'websocket': AuthMiddlewareStack (
        URLRouter (
            chat.routing.websocket_urlpatterns
        )
    ),
})

```

En este código, usa `URLRouter` para mapear conexiones `websocket` a los patrones de URL definido en la lista `websocket_urlpatterns` del archivo de enrutamiento de la aplicación de chat . El enrutador `ProtocolTypeRouter` estándar asigna automáticamente las solicitudes HTTP a las vistas estándar de Django si no se proporciona un mapeo http específico . También usas `AuthMiddlewareStack` . La clase `AuthMiddlewareStack` proporcionada por Channels admite la autenticación estándar de Django, donde los detalles del usuario se almacenan en La sesión. Planea acceder a la instancia de usuario en el ámbito del consumidor para identificar al usuario que envía un mensaje.

Implementar el cliente WebSocket

Hasta ahora, ha creado la vista `course_chat_room` y su correspondiente plantilla para que los estudiantes accedan a la sala de chat del curso. Ha implementado un Consumidor de `WebSocket` para el servidor de chat y lo vinculó con el enrutamiento de URL. Ahora tu necesita construir un cliente `WebSocket` para establecer una conexión con `WebSocket` en la plantilla de la sala de chat del curso y poder enviar / recibir mensajes.

Va a implementar el cliente `WebSocket` con JavaScript para abrir y mantener una conexión en el navegador. Usarás jQuery para interactuar con Elementos del Modelo de objetos de documento (DOM), ya que ya lo cargó en la base plantilla del proyecto.

Edite la plantilla `chat / room.html` de la aplicación de chat y modifique `domready` bloque, de la siguiente manera:

```

{% block domready%}
    var url = 'ws: //' + window.location.host +
        ' / ws / chat / room / ' + '{{course.id}} / ';
    var chatSocket = new WebSocket (url);
{% endblock%}

```

Define una URL con el protocolo WebSocket, que se parece a `ws://` (o `wss://` para WebSockets seguros, como `https://`). Construye la URL usando el actual ubicación del navegador, que se obtiene de `window.location.host`. El resto de la URL se crea con la ruta del patrón de URL de la sala de chat que definió en el archivo `routing.py` de la aplicación de chat.

Escribe la URL completa en lugar de crearla a través de su nombre porque Channels lo hace no proporciona una forma de revertir las URL. Utiliza el ID del curso actual para generar el URL del curso actual y almacene la URL en una nueva variable denominada `url`.

Luego abre una conexión WebSocket a la URL almacenada usando `new WebSocket(url)`. Asigne el objeto cliente de WebSocket instanciado al nuevo variable `chatSocket`.

Ha creado un consumidor de WebSocket, ha incluido el enrutamiento para él y han implementado un cliente WebSocket básico. Probemos la versión inicial de tu chat.

Inicie el servidor de desarrollo con el siguiente comando:

```
python manage.py runserver
```

Abra la URL `http://127.0.0.1:8000/chat/room/1/` en su navegador, reemplazando

1 con la identificación de un curso existente en la base de datos. Echa un vistazo a la consola salida. Además de las solicitudes HTTP GET para la página y sus archivos estáticos, debe ver dos líneas que incluyen WebSocket HANDSHAKING y WebSocket CONNECT, como la siguiente salida:

```
HTTP GET / chat / room / 1/200 [0.02, 127.0.0.1:57141]
HTTP GET /static/css/base.css 200 [0.01, 127.0.0.1:57141]
WebSocket HANDSHAKING / ws / chat / room / 1 / [127.0.0.1:57144]
WebSocket CONNECT / ws / chat / room / 1 / [127.0.0.1:57144]
```

El servidor de desarrollo de canales escucha las conexiones de socket entrantes mediante un socket TCP estándar. El protocolo de enlace es el puente de HTTP a WebSockets. En el apretón de manos, los detalles de la conexión se negocian y cualquiera de las partes puede cerrar la conexión antes de la finalización. Recuerda que estás usando `self.accept()` para aceptar cualquier conexión en el método `connect()` de la clase `ChatConsumer` implementado en el archivo `consumer.py` de la aplicación de chat. La conexión es aceptado y, por lo tanto, verá el mensaje WebSocket CONNECT en la consola.

Si utiliza las herramientas de desarrollo del navegador para realizar un seguimiento de las conexiones de red, también puede consulte la información de la conexión WebSocket que se ha establecido.

[479]

Construyendo un servidor de chat

Debería verse como la siguiente captura de pantalla:

Ahora que puede conectarse a WebSocket, es hora de interactuar con él. Implementará los métodos para manejar eventos comunes, como recibir un mensaje y cerrando la conexión. Edite la plantilla chat / room.html de la aplicación de chat y modifique el bloque domready de la siguiente manera:

```
{% block domready%}
var url = 'ws://'+ window.location.host +
'/ws/chat/room/' + '{{course.id}} /';
var chatSocket = new WebSocket(url);

chatSocket.onmessage = function (e) {
var data = JSON.parse(e.data);
var message = data.message;

var $chat = $('#chat');
$chat.append('<div class = "mensaje">' + mensaje + '</div>');
$chat.scrollTop($chat[0].scrollHeight);
};

chatSocket.onclose = function (e) {
console.error('Conector de chat cerrado inesperadamente');
};
{% endblock%}
```

En este código, define los siguientes eventos para el cliente WebSocket:

- onmessage : se activa cuando se reciben datos a través de WebSocket. Tu analizas el mensaje, que espera en formato JSON, y acceda a su mensaje atributo. Luego agrega un nuevo elemento <div> con el mensaje al Elemento HTML con el ID de chat . Esto agregará nuevos mensajes al registro de chat, manteniendo todos los mensajes anteriores que se han agregado al registro. Tú Desplace el registro de chat <div> hasta la parte inferior para asegurarse de que se recibe el nuevo mensaje. visibilidad. Esto se logra desplazándose hasta la altura total desplazable del registro de chat, que se puede obtener accediendo a su atributo scrollHeight .
- onclose : se activa cuando se cierra la conexión con WebSocket. Tu no espera cerrar la conexión y, por lo tanto, escribe el error Chat socket cerrado inesperadamente en el registro de la consola si esto sucede.

[480]

Ha implementado la acción para mostrar el mensaje cuando se envía un mensaje nuevo. recibido. Necesita implementar la funcionalidad para enviar mensajes al socket también.

Edite la plantilla chat / room.html de la aplicación de chat y agregue lo siguiente Código JavaScript al final del bloque domready :

```
var $input = $('#chat-message-input');
var $enviar = $('#chat-mensaje-enviar');

$submit.click(function () {
var mensaje = $input.val();
if (mensaje) {
// enviar mensaje en formato JSON
chatSocket.send(JSON.stringify({'mensaje': mensaje}));

// borrar entrada
$input.val('');

// devolver el foco
$input.focus();
}
});
```

En este código, se define una función para el clic de eventos del botón de enviar, el cual

selecciona con el ID chat-mensaje-enviar . Cuando se hace clic en el botón, realizar las siguientes acciones:

1. Usted lee el mensaje ingresado por el usuario a partir del valor de la entrada de texto elemento con el ID chat-message-input
2. Comprueba si el mensaje tiene algún contenido con if (mensaje)
3. Si el usuario ha introducido un mensaje, forma contenido JSON como
{mensaje: 'cadena ingresada por el usuario'} usando JSON.
stringify ()
4. Envías el contenido JSON a través de WebSocket, llamando a send ()
método de cliente chatSocket
5. Borra el contenido de la entrada de texto estableciendo su valor en un espacio vacío.
cadena con \$ input.val ("")
6. Devuelve el foco a la entrada de texto con \$ input.focus () para que el usuario
puede escribir un mensaje nuevo inmediatamente

[481]

Construyendo un servidor de chat

El usuario ahora puede enviar mensajes usando la entrada de texto y haciendo clic en el botón de enviar.

Para mejorar la experiencia del usuario, se centrará en la entrada de texto como tan pronto como se cargue la página para que el usuario pueda escribir directamente en ella. Tu tambien capturarás Tecla de teclado presionada eventos para identificar la *tecla Enter / Retorno* y disparar el clic evento en el botón enviar. El usuario podrá hacer clic en el botón o presionar la *tecla Enter / Return* para enviar un mensaje.

Edite la plantilla chat / room.html de la aplicación de chat y agregue lo siguiente Código JavaScript al final del bloque domready :

```
$ input.focus ();
$ input.keyup (función (e) {
  si (e.que === 13) {
    // enviar con la tecla enter / return
    $ enviar.click ();
  }
});
```

En este código, le das el foco a la entrada de texto. También define una función para el evento keyup () de la entrada. Para cualquier tecla que presione el usuario, verifica si el código clave es 13 . Este es el código clave que corresponde a la *tecla Enter / Return* . Tú puede utilizar el recurso <https://keycode.info> para identificar el código clave de cualquier clave. Si se presiona la *tecla Enter / Return* , activa el evento de clic en el botón de enviar para enviar el mensaje al WebSocket.

El bloque completo domready de la plantilla chat / room.html debería verse ahora Me gusta esto:

```
{% block domready%}
var url = 'ws: //' + window.location.host +
  '/ ws / chat / room / ' + '{{course.id}} /';
var chatSocket = new WebSocket (url);

chatSocket.onmessage = function (e) {
  var data = JSON.parse (e.data);
  var message = data.message;

  var $ chat = $('# chat');
  $ chat.append ('<div class = "mensaje">' + mensaje + '</div>');
  $ chat.scrollTop ($ chat [0] .scrollHeight);
};
```

```

chatSocket.onclose = function (e) {
    console.error ('Conector de chat cerrado inesperadamente');
};

var $ input = $ ('# chat-message-input');

```

[482]

```

var $ enviar = $ ('# chat-mensaje-enviar');

$ submit.click (function () {
    var mensaje = $ input.val ();
    if (mensaje) {
        // enviar mensaje en formato JSON
        chatSocket.send (JSON.stringify ({'mensaje': mensaje}));

        // borrar entrada
        $ input.val ('');

        // devolver el foco
        $ input.focus ();
    }
});

$ input.focus ();
$ input.keyup (función (e) {
    si (e.que === 13) {
        // enviar con la tecla enter / return
        $ enviar.click ();
    }
});
{% endblock%}

```

Abra la URL <http://127.0.0.1:8000/chat/room/1/> en su navegador, reemplazando

1 con la identificación de un curso existente en la base de datos. Con un usuario que ha iniciado sesión inscrito en el curso, escriba un texto en el campo de entrada y haga clic en el botón enviar o presione la tecla *Enter* . Verá que su mensaje aparece en el registro de chat:

Figura 13.5: La página de la sala de chat, incluidos los mensajes enviados a través de WebSocket

¡Excelente! El mensaje se ha enviado a través de WebSocket y ChatConsumer el consumidor ha recibido el mensaje y lo ha devuelto a través de WebSocket. El cliente chatSocket ha recibido un evento de mensaje y la función onmessage ha sido despedido, agregando el mensaje al registro de chat.

[483]

Construyendo un servidor de chat

Ha implementado la funcionalidad con un consumidor de WebSocket y un Cliente WebSocket para establecer comunicación cliente / servidor y poder enviar o recibir eventos. Sin embargo, el servidor de chat no puede transmitir mensajes a otros clientela. Si abre una segunda pestaña del navegador e ingresa un mensaje, el mensaje no aparecen en la primera pestaña. Para construir la comunicación entre los consumidores, tiene que habilitar una capa de canal.

Habilitar una capa de canal

Las capas de canal le permiten comunicarse entre diferentes instancias de un solicitud. Una capa de canal es el mecanismo de transporte que permite múltiples instancias de consumidor para comunicarse entre sí y con otras partes de Django.

En su servidor de chat, planea tener varias instancias de ChatConsumer consumidor para la misma sala de chat del curso. Cada estudiante que se una a la sala de chat crear una instancia del cliente WebSocket en su navegador, y eso abrirá una conexión con una instancia del consumidor de WebSocket. Necesitas una capa de canal común para distribuir mensajes entre consumidores.

Canales y grupos

Las capas de canal proporcionan dos abstracciones para administrar las comunicaciones: canales y grupos:

- **Canal** : puede pensar en un canal como una bandeja de entrada donde los mensajes pueden enviado ao como una cola de tareas. Cada canal tiene un nombre. Se envían mensajes a un canal por cualquiera que conozca el nombre del canal y luego dado a consumidores que escuchan en ese canal.
- **Grupo** : se pueden agrupar varios canales en un grupo. Cada grupo tiene un nombre. Cualquiera puede agregar o eliminar un canal de un grupo quién sabe el nombre del grupo. Usando el nombre del grupo, también puede enviar un mensaje a todos los canales del grupo.

Trabjará con grupos de canales para implementar el servidor de chat. Creando un grupo de canales para cada sala de chat del curso, las instancias de ChatConsumer serán capaces de comunicarse entre sí.

Configurar una capa de canal con Redis

Redis es la opción preferida para una capa de canal, aunque Channels tiene soporte para otros tipos de capas de canal. Redis funciona como almacén de comunicaciones para capa de canal. Recuerde que ya usó Redis en el *Capítulo 6 , Seguimiento de usuarios Acciones* , y en el *Capítulo 9 , Ampliación de su tienda* .

[484]

Capítulo 13

Si aún no ha instalado Redis, puede encontrar las instrucciones de instalación en el *Capítulo 6 , Seguimiento de las acciones del usuario* .

Para utilizar Redis como capa de canal, debe instalar los canales-redis paquete. Instale canales-redis en su entorno virtual con lo siguiente

mando:

pip install channels-redis == 2.4.2

Edite el archivo settings.py del proyecto educa y agregue el siguiente código:

```
CHANNEL_LAYERS = {
    'defecto': {
        'BACKEND': 'channels_redis.core.RedisChannelLayer',
        'CONFIG': {
            'hosts': [('127.0.0.1', 6379)],
        },
    },
}
```

El ajuste CHANNEL_LAYERS define la configuración del canal capas disponibles para el proyecto. Defina una capa de canal predeterminada utilizando el RedisChannelLayer backend proporcionado por channels-redis y especifica el host 127.0.0.1 y el puerto 6379 en el que se ejecuta Redis.

Probemos la capa de canal. Inicialice el servidor Redis usando el siguiente comando desde el shell en su directorio de Redis:

src / redis-servidor

Abra el shell de Django con el siguiente comando:

shell de python manage.py

Para verificar que la capa del canal pueda comunicarse con Redis, escriba lo siguiente código para enviar un mensaje a un canal de prueba llamado test_channel y recibirlo de vuelta:

```
>>> importar canales.capas
>>> de asgiref.sync importar async_to_sync
>>> capa_canal = canales.capas.get_capa_canal 0
>>> async_to_sync (channel_layer.send) ('test_channel', {'mensaje':
'Hola'})
>>> async_to_sync (capa_canal.receive) ('canal_prueba')
```

Debería obtener el siguiente resultado:

```
{'mensaje': 'hola'}
```

[485]

Construyendo un servidor de chat

En el código anterior, envía un mensaje a un canal de prueba a través de la capa de canal, y luego lo recuperas de la capa del canal. La capa del canal se comunica con éxito con Redis.

Actualización del consumidor para transmitir mensajes

Va a editar el ChatConsumer cliente que utilice la capa de canal. Usarás un grupo de canales para cada sala de chat del curso. Por lo tanto, usará la identificación del curso para construir el nombre del grupo. Las instancias de ChatConsumer conocerán el nombre del grupo y podrán comunicarse entre sí.

Edite el archivo consumer.py de la aplicación de chat , importe async_to_sync () función y modificar el método connect () de la clase ChatConsumer , de la siguiente manera:

```
importar json
desde channels.generic.websocket importar WebsocketConsumer
de asgiref.sync importar async_to_sync

clase ChatConsumer (WebsocketConsumer):
```

```

def connect (auto):
    self.id = self.scope ['url_route'] ['kwargs'] ['course_id']
    self.room_group_name = 'chat_%s'% self.id
    # unirse al grupo de sala
    async_to_sync (self.channel_layer.group_add) (
        self.room_group_name,
        self.channel_name
    )
    # aceptar conexión
    self.accept ()

# ...

```

En este código, importa la función auxiliar `async_to_sync ()` para envolver llamadas a los métodos de capa de canal asincrónico. `ChatConsumer` es un sincrónico

`WebsocketConsumer` consumidor, pero necesita llamar a métodos asincrónicos de la capa del canal.

En el nuevo método `connect ()`, realiza las siguientes tareas:

1. Recupera el ID del curso del alcance para conocer el curso que sala de chat está asociada con. Accede a `self.scope ['url_route'] ['kwargs'] ['course_id']` para recuperar el parámetro `course_id` de la URL. Cada consumidor tiene un alcance con información sobre su conexión, argumentos pasados por la URL y el usuario autenticado, si lo hay.

[486]

2. Construye el nombre del grupo con la identificación del curso que el grupo corresponde a. Recuerde que tendrá un grupo de canales para cada curso de sala de chat. Almacena el nombre del grupo en `room_group_name` atributo del consumidor.
3. Te unes al grupo agregando el canal actual al grupo. Tú obtienes el nombre del canal del atributo `channel_name` del consumidor. Tú use el método `group_add` de la capa del canal para agregar el canal a la grupo. Usas el contenedor `async_to_sync ()` para usar la capa de canal método asincrónico.
4. Conserva la llamada `self.accept ()` para aceptar la conexión `WebSocket`.

Cuando el consumidor de `ChatConsumer` recibe una nueva conexión `WebSocket`, agrega el canal al grupo asociado al curso en su ámbito. El consumidor es ahora puede recibir cualquier mensaje enviado al grupo.

En el mismo archivo `consumer.py`, modifique el método `desconectar ()` del

Clase `ChatConsumer`, como sigue:

```

class ChatConsumer (WebsocketConsumer):
    # ...

    def desconectar (self, close_code):
        # salir del grupo de habitación
        async_to_sync (self.channel_layer.group_discard) (
            self.room_group_name,
            self.channel_name
        )

    # ...

```

Cuando se cierra la conexión, llama al método `group_discard ()` del canal capa para dejar el grupo. Usa el contenedor `async_to_sync ()` para usar el canal método asincrónico de capa.

En el mismo archivo `consumer.py`, modifique el método `receive ()` del `ChatConsumer` clase, como sigue:


```

class ChatConsumer (WebsocketConsumer):
    # ...

    # recibir mensaje de WebSocket
    def recibir (self, text_data):
        text_data_json = json.loads (text_data)
        message = text_data_json ['mensaje']

```

[487]

Página 513

Construyendo un servidor de chat

```

    # enviar mensaje al grupo de sala
    async_to_sync (self.channel_layer.group_send) (
        self.room_group_name,
        {
            'type': 'chat_message',
            'mensaje': mensaje,
        }
    )

```

Cuando recibe un mensaje de la conexión WebSocket, en lugar de enviar el mensaje al canal asociado, ahora envía el mensaje al grupo. Tú lo haces esto llamando al método `group_send ()` de la capa del canal. Usas el `async_to_sync ()` para usar el método asíncrono de capa de canal. Pasas el siguiente información en el evento enviado al grupo:

- **tipo** : el tipo de evento. Esta es una clave especial que corresponde al nombre de el método que debe invocarse en los consumidores que reciben el evento. Tú puede implementar un método en el consumidor con el mismo nombre que el mensaje `type` para que se ejecute cada vez que un mensaje con ese tipo específico sea recibido.
- **mensaje** : el mensaje real que está enviando.

En el mismo archivo `consumer.py`, agregue un nuevo método `chat_message ()` en el Clase `ChatConsumer`, como sigue:

```

class ChatConsumer (WebsocketConsumer):
    # ...

    # recibir mensaje del grupo de sala
    def chat_message (yo, evento):
        # Enviar mensaje a WebSocket
        self.send (text_data = json.dumps (evento))

```

Nombra este método `chat_message ()` para que coincida con la clave de tipo que se envía al grupo de canales cuando se recibe un mensaje de WebSocket. Cuando un mensaje con el tipo `chat_message` se envía al grupo, todos los consumidores suscritos al grupo recibirá el mensaje y ejecutará el método `chat_message ()`. En el `chat_message ()`, envía el mensaje de evento recibido a WebSocket.

El archivo completo `consumer.py` ahora debería verse así:

```

import json
desde channels.generic.websocket import WebsocketConsumer
de asgiref.sync importar async_to_sync

```

[488]

```

class ChatConsumer (WebsocketConsumer):
    def connect (auto):
        self.id = self.scope ['url_route'] ['kwargs'] ['course_id']
        self.room_group_name = 'chat_% s% self.id
        # unirse al grupo de sala
        async_to_sync (self.channel_layer.group_add) (
            self.room_group_name,
            self.channel_name
        )
        # aceptar conexión
        self.accept ()

    def desconectar (self, close_code):
        # salir del grupo de habitación
        async_to_sync (self.channel_layer.group_discard) (
            self.room_group_name,
            self.channel_name
        )

    # recibir mensaje de WebSocket
    def recibir (self, text_data):
        text_data_json = json.loads (text_data)
        message = text_data_json ['mensaje']

        # enviar mensaje al grupo de sala
        async_to_sync (self.channel_layer.group_send) (
            self.room_group_name,
            {
                'type': 'chat_message',
                'mensaje': mensaje,
            }
        )

    # recibir mensaje del grupo de sala
    def chat_message (yo, evento):
        # enviar mensaje a WebSocket
        self.send (text_data = json.dumps (evento))

```

Ha implementado una capa de canal en ChatConsumer , lo que permite a los consumidores difundir mensajes y comunicarse entre sí.

Ejecute el servidor de desarrollo con el siguiente comando:

python manage.py runserver

Luego, abra una segunda ventana del navegador y acceda a la misma URL. Enviar un mensaje desde cada ventana del navegador.

El resultado debería verse así:

Figura 13.6: La página de la sala de chat con mensajes enviados desde diferentes ventanas del navegador

Verá que el primer mensaje solo se muestra en la primera ventana del navegador. Cuando abre una segunda ventana del navegador, los mensajes enviados en cualquiera de las ventanas se muestran en ambos. Cuando abre una nueva ventana del navegador y acceder a la URL de la sala de chat, se establece una nueva conexión WebSocket entre el cliente JavaScript WebSocket en el navegador y el consumidor WebSocket en el servidor. Cada canal se agrega al grupo asociado con la identificación del curso aprobada a través de la URL al consumidor. Los mensajes se envían al grupo y los recibe todos los consumidores.

Agregar contexto a los mensajes

Ahora que los mensajes se pueden intercambiar entre todos los usuarios en una sala de chat, Probablemente quiera mostrar quién envió qué mensaje y cuándo se envió. Vamos agregue algo de contexto a los mensajes.

Edite el archivo `consumer.py` de la aplicación de chat e implemente lo siguiente cambios:

```
importar json
desde channels.generic.websocket import WebsocketConsumer
de asgiref.sync importar async_to_sync
```

[490]

desde `django.utils` importar `zona horaria`

```
clase ChatConsumer (WebsocketConsumer):
    def connect (auto):
        self.user = self.scope ['usuario']
        self.id = self.scope ['url_route'] ['kwargs'] ['course_id']
        self.room_group_name = 'chat_% s' % self.id
        # unirse al grupo de sala
        async_to_sync (self.channel_layer.group_add) (
            self.room_group_name,
            self.channel_name
        )
        # aceptar conexión
        self.accept ()

    def desconectar (self, close_code):
        # salir del grupo de habitación
```

```

        async_to_sync (self.channel_layer.group_discard) (
            self.room_group_name,
            self.channel_name
        )

# recibir mensaje de WebSocket
def recibir (self, text_data):
    text_data_json = json.loads (text_data)
    message = text_data_json ['mensaje']
    ahora = timezone.now ()

    # enviar mensaje al grupo de sala
    async_to_sync (self.channel_layer.group_send) (
        self.room_group_name,
        {
            'type': 'chat_message',
            'mensaje': mensaje,
            'usuario': self.user.username,
            'fecha y hora': ahora.isoformat (),
        }
    )

# recibir mensaje del grupo de sala
def chat_message (yo, evento):
    # enviar mensaje a WebSocket
    self.send (text_data = json.dumps (evento))

```

[491]

Construyendo un servidor de chat

Ahora importa el módulo de zona horaria proporcionado por Django. En el connect () método del consumidor, recupera el usuario actual del alcance con self.

alcance ['usuario'] y almacenarlos en un nuevo atributo de usuario del consumidor. Cuando el consumidor recibe un mensaje a través de WebSocket, obtiene la hora actual usando timezone.now () y pasa el usuario actual y la fecha y hora en formato ISO 8601 junto con el mensaje en el evento enviado al grupo de canales.

Edite la plantilla chat / room.html de la aplicación de chat y busque lo siguiente líneas:

```

var data = JSON.parse (e.data);
var message = data.message;

var $ chat = $ ('# chat');
$ chat.append ('<div class = "mensaje">' + mensaje + '</div>');

```

Reemplace esas líneas con el siguiente código:

```

var data = JSON.parse (e.data);
var message = data.message;

var dateOptions = {hora: 'numérico', minuto: 'numérico', hora12: verdadero};
var datetime = new Date (data ['datetime']). toLocaleString ('en',
dateOptions);

var isMe = data.user === '{request.user}';
var source = isMe? 'yo': 'otro';
var name = isMe? 'Yo': data.user;

var $ chat = $ ('# chat');
$ chat.append ('<div class = "mensaje ' + fuente + ' ">' +
    '<strong>' + nombre + '</strong>' +
    '<span class = "date">' + datetime + '</span> <br>' +
    mensaje +

```

```
'</div>');
```

En este código, implementas estos cambios:

1. Ahora convierte la fecha y hora recibida en el mensaje a una fecha de JavaScript. objeto y formatee con una configuración regional específica.
2. Recupera el usuario recibido en el mensaje y realiza una comparación con dos variables diferentes como ayudantes para identificar al usuario.

[492]

Capítulo 13

3. La fuente de la variable obtiene el valor `yo` si el usuario que envía el mensaje es el usuario actual, o cualquier otro. Obtienes el nombre de usuario usando El lenguaje de la plantilla de Django con `{{request.user}}` para comprobar si el mensaje se originó en el usuario actual u otro usuario. Entonces tú use el valor `fuentes` como una clase del elemento `<div>` principal para diferenciar mensajes enviados por el usuario actual a partir de mensajes enviados por otros. Diferente Los estilos CSS se aplican según el atributo de clase .
4. El nombre de la variable obtiene el valor `Me` si el usuario que envía el mensaje es el usuario actual o el nombre del usuario que envía el mensaje en caso contrario. Tú utilízelo para mostrar el nombre del usuario que envía el mensaje.
5. Utiliza el nombre de usuario y la fecha y hora en el mensaje que adjunta. al registro de chat.

Abra la URL `http://127.0.0.1:8000/chat/room/1/` en su navegador, reemplazando

1 con la identificación de un curso existente en la base de datos. Con un usuario que ha iniciado sesión inscrito en el curso, escriba un mensaje y envíelo.

Luego, abra una segunda ventana del navegador en modo incógnito para evitar el uso del misma sesión. Inicie sesión con un usuario diferente, también inscrito en el mismo curso, y envíe un mensaje.

Podrá intercambiar mensajes utilizando los dos usuarios diferentes y ver al usuario y tiempo, con una clara distinción entre los mensajes enviados por el usuario y los mensajes enviado por otros. La conversación entre dos usuarios debe ser similar a la siguiente:

Figura 13.7: La página de la sala de chat con mensajes de dos sesiones de usuario diferentes

[493]

Construyendo un servidor de chat

Modificar al consumidor para que sea plenamente asincrónico

El ChatConsumer que ha implementado hereda de la base

Clase WebsocketConsumer, que es síncrona. Los consumidores síncronos son conveniente para acceder a los modelos de Django y llamar a E / S síncronas regulares funciones. Sin embargo, los consumidores asincrónicos presentan un mayor rendimiento, ya que no requieren subprocesos adicionales cuando manejan solicitudes. Ya que estas usando las funciones de capa de canal asincrónico, puede reescribir fácilmente el ChatConsumer clase sea asincrónica.

Edite el archivo consumer.py de la aplicación de chat e implemente lo siguiente cambios:

```
importar json
de channels.generic.websocket import AsyncWebsocketConsumer
de asgiref.sync importar async_to_sync
desde django.utils importar zona horaria

clase ChatConsumer ( AsyncWebsocketConsumer ):
    async def connect (auto):
        self.user = self.scope ['usuario']
        self.id = self.scope ['url_route'] ['kwargs'] ['course_id']
        self.room_group_name = 'chat_% s'% self.id
        # unirse al grupo de sala
        aguardar self.channel_layer.group_add (
            self.room_group_name,
            self.channel_name
        )
        # aceptar conexión
        esperar self.accept ()

    async def desconectar (self, close_code):
        # salir del grupo de habitación
        aguardar self.channel_layer.group_discard (
            self.room_group_name,
            self.channel_name
        )

    # recibir mensaje de WebSocket
    async def recibir (self, text_data):
        text_data_json = json.loads (text_data)
        message = text_data_json ['mensaje']
        ahora = timezone.now ()
```

[494]

```

        # enviar mensaje al grupo de sala
        aguardar self.channel_layer.group_send (
            self.room_group_name,
            {
                'type': 'chat_message',
                'mensaje': mensaje,
                'usuario': self.user.username,
                'fecha y hora': ahora.isoformat ()
            }
        )

    # recibir mensaje del grupo de sala
    async def chat_message (self, event):
        # enviar mensaje a WebSocket
        espera self.send (text_data = json.dumps (evento))

```

Ha implementado los siguientes cambios:

- El consumidor de ChatConsumer ahora hereda del Clase AsyncWebsocketConsumer para implementar llamadas asincrónicas
- Ha cambiado la definición de todos los métodos de def a async def
- Utiliza await para llamar a funciones asincrónicas que realizan operaciones de E / S
- Ya no usa la función auxiliar async_to_sync () al llamar métodos en la capa del canal

Abra la URL <http://127.0.0.1:8000/chat/room/1> con dos navegadores diferentes Windows nuevamente y verifique que el servidor de chat aún funcione. El servidor de chat ahora totalmente asincrónico!

Integrando la aplicación de chat con vistas existentes

El servidor de chat ahora está completamente implementado y los estudiantes inscritos en un curso pueden para comunicarse entre sí. Agreguemos un enlace para que los estudiantes se unan a la sala de chat. para cada curso.

Edite la plantilla `Students / course / detail.html` de la aplicación para estudiantes . y agregue el siguiente código de elemento HTML `<h3>` en la parte inferior del `<div class = "contenido">` elemento:

```

<div class = "contenido">
    ...

```

[495]

Construyendo un servidor de chat

```

<h3>
    <a href="{% url 'chat:course_chat_room' object.id %}">
        Sala de chat del curso
    </a>
</h3>
</div>

```

Abra el navegador y acceda a cualquier curso en el que esté inscrito el alumno para ver el contenidos del curso. La barra lateral ahora contendrá un enlace a la **sala de chat del curso** que apunta a la vista de la sala de chat del curso. Si hace clic en él, entrará en la sala de chat.

Figura 13.8: La página de detalles del curso, incluido un enlace a la sala de chat del curso

Resumen

En este capítulo, aprendió cómo crear un servidor de chat usando Canales. Tú implementó un consumidor y cliente de WebSocket. También habilitaste la comunicación entre consumidores que utilizan una capa de canal con Redis y modifican el consumidor ser completamente asíncronico.

El siguiente capítulo le enseñará cómo crear un entorno de producción para su Proyecto Django usando NGINX, uWSGI y Daphne. También aprenderá a implementar un middleware personalizado y crear comandos de administración personalizados.

[496]

14

Va en directo

En el capítulo anterior, creaste un servidor de chat en tiempo real para estudiantes que usan Django Canales. Ahora que ha creado una plataforma de aprendizaje en línea completamente funcional, Necesita configurar un entorno de producción en un servidor en línea para que pueda se accede a través de Internet. Hasta ahora has estado trabajando en un desarrollo entorno, utilizando el servidor de desarrollo Django para ejecutar su sitio. En este capítulo, aprenderá cómo configurar un entorno de producción que pueda servir a su Proyecto Django de manera segura y eficiente.

Este capítulo cubrirá los siguientes temas:

- Configurar un entorno de producción
- Crear un middleware personalizado

- Implementar comandos de administración personalizados

Creando un ambiente de producción

Es hora de implementar su proyecto Django en un entorno de producción. Tu vas para seguir estos pasos para poner en marcha su proyecto:

- Configurar la configuración del proyecto para un entorno de producción
- Utilice una base de datos PostgreSQL
- Configure un servidor web con uWSGI y NGINX
- Sirve activos estáticos a través de NGINX
- Conexiones seguras mediante SSL
- Utilice Daphne para servir canales de Django

[497]

Va en directo

Administrar configuraciones para múltiples entornos

En proyectos del mundo real, tendrá que lidiar con múltiples entornos. Tú tendrá al menos un entorno local y de producción, pero podría tener otros entornos también, como entornos de prueba o de preproducción. Algún proyecto La configuración será común a todos los entornos, pero otras tendrán que ser anuladas por ambiente. Configuremos la configuración del proyecto para múltiples entornos, mientras manteniendo todo perfectamente organizado.

Cree un directorio `settings/` junto al archivo `settings.py` del proyecto `educa` . Cambie el nombre del archivo `settings.py` a `base.py` y muévelo a la nueva configuración / directorio. Cree los siguientes archivos adicionales dentro de la carpeta de configuración que el nuevo directorio tiene el siguiente aspecto:

```
configuración /
  __init__.py
  base.py
  local.py
  pro.py
```

Estos archivos son los siguientes:

- `base.py` : el archivo de configuración base que contiene configuraciones comunes (anteriormente `settings.py`)
- `local.py` : configuración personalizada para su entorno local
- `pro.py` : configuración personalizada para el entorno de producción

Edite el archivo `settings/ base.py` y reemplace la siguiente línea:

```
BASE_DIR = os.path.dirname (os.path.dirname (os.path.abspath (__file__)))
```

con el siguiente:

```
BASE_DIR = os.path.dirname (os.path.dirname (os.path.abspath (
    os.path.join (__archivo__, os.pardir) )))
```

Ha movido sus archivos de configuración a un directorio un nivel más bajo, por lo que necesita `BASE_` DIR para apuntar al directorio principal para que sea correcto. Lo logras apuntando a el directorio principal con `os.pardir` .

Edite el archivo `settings/ local.py` y agregue las siguientes líneas de código:

```
desde la importación .base *

DEBUG = Verdadero

BASES DE DATOS = {
```

```

'defecto': {
    'MOTOR': 'django.db.backends.sqlite3',
    'NOMBRE': os.path.join(BASE_DIR, 'db.sqlite3'),
}
}

```

Este es el archivo de configuración para su entorno local. Importa todos los ajustes definidos en el archivo `base.py` y solo define configuraciones específicas para este entorno. Tú copie la configuración `DEBUG` y `DATABASES` del archivo `base.py`, ya que estos ser configurado por entorno. Puede eliminar la configuración de `BASES DE DATOS` y `DEBUG` de el archivo de configuración `base.py`.

Edite el archivo `settings / pro.py` y haga que tenga el siguiente aspecto:

```

desde la importación .base *

DEBUG = Falso

ADMINISTRADORES = (
    ('Antonio M', 'email@midominio.com'),
)

ALLOWED_HOSTS = ['*']

BASES DE DATOS = {
    'defecto': {
    }
}

```

Éstos son los ajustes para el entorno de producción. Echemos un vistazo más de cerca a cada de ellos:

- **DEBUG** : establecer `DEBUG` en `False` debería ser obligatorio para cualquier producción ambiente. No hacerlo resultará en la información de rastreo y los datos de configuración confidenciales están expuestos a todos.
- **ADMINS** : cuando `DEBUG` es `False` y una vista genera una excepción, toda la información se enviará por correo electrónico a las personas que figuran en la configuración `ADMINS`. Asegurarse que reemplace la tupla de nombre / correo electrónico con su propia información.
- **ALLOWED_HOSTS** : Django solo permitirá que los hosts incluidos en esta lista servir la solicitud. Esta es una medida de seguridad. Incluye el asterisco símbolo, `*`, para hacer referencia a todos los nombres de host. Limitará los nombres de host que pueden ser utilizado para servir la aplicación más tarde.
- **BASES DE DATOS** : Simplemente mantenga esta configuración vacía. Vamos a cubrir el configuración de la base de datos para la producción posterior.

Va en directo

Cuando maneje varios entornos, cree un archivo de configuración base y un archivo de configuración para cada entorno. Archivos de configuración del entorno debe heredar la configuración común y anular el entorno ajustes específicos.

Ha colocado la configuración del proyecto en una ubicación diferente a la predeterminada archivo settings.py . No podrá ejecutar ningún comando con el archivo manage.py a menos que especifique el módulo de configuración que se utilizará. Deberá agregar un -marca de configuración cuando ejecuta comandos de administración desde el shell o establece un Variable de entorno DJANGO_SETTINGS_MODULE .

Abra el shell y ejecute el siguiente comando:

exportar DJANGO_SETTINGS_MODULE = educa.settings.pro

Esto establecerá la variable de entorno DJANGO_SETTINGS_MODULE para el actual sesión de shell. Si desea evitar ejecutar este comando para cada nuevo shell, agregue este comando a la configuración de su shell en el .bashrc o .bash_profile archivos. Si no configura esta variable, tendrá que ejecutar comandos de administración, incluido el indicador --settings , de la siguiente manera:

Python manage.py shell --settings = educa.settings.pro

Ha organizado correctamente la configuración para manejar varios entornos.

Usando PostgreSQL

A lo largo de este libro, ha utilizado principalmente la base de datos SQLite. SQLite es simple y rápido de configurar, pero para un entorno de producción, necesitará más poderosa base de datos, como PostgreSQL, MySQL u Oracle. Ya aprendiste cómo instalar PostgreSQL y configurar una base de datos PostgreSQL en el *Capítulo 3 , Ampliación Su aplicación de blog* . Si necesita instalar PostgreSQL, puede leer la *Instalación Sección PostgreSQL* del *Capítulo 3* .

Creemos un usuario de PostgreSQL. Abra el shell y ejecute los siguientes comandos para crear un usuario de base de datos:

su postgres

createuser -dP educa

Se le pedirá una contraseña y los permisos que desea otorgar. a este usuario. Ingrese la contraseña y los permisos deseados, y luego cree una nueva base de datos con el siguiente comando:

createdb -E utf8 -U educa educa

[500]

Luego, edite el archivo settings / pro.py y modifique la configuración DATABASES para que sea mira como sigue:

```
BASES DE DATOS = {
    'defecto': {
        'MOTOR': 'django.db.backends.postgresql',
        'NOMBRE': 'educa',
        'USUARIO': 'educa',
        'CONTRASEÑA': '*****',
    }
}
```

Reemplace los datos anteriores con el nombre de la base de datos y las credenciales del usuario. tu creaste. La nueva base de datos está vacía. Ejecute el siguiente comando para aplicar todo migraciones de base de datos:

```
python manage.py migrar
```

Finalmente, cree un superusuario con el siguiente comando:

```
python manage.py crea superusuario
```

Comprobando su proyecto

Django incluye la verificación de comandos de gestión para el control de su proyecto en cualquier hora. Este comando inspecciona las aplicaciones instaladas en su proyecto Django y emite cualquier error o advertencia. Si incluye la opción `--deploy`, adicionales Se activarán los controles solo relevantes para el uso de producción. Abre el caparazón y corre el siguiente comando para realizar una verificación:

```
Python manage.py comprobar - implementar
```

Verá una salida sin errores, pero con varias advertencias. Esto significa que el cheque fue exitoso, pero debe revisar las advertencias para ver si hay algo más que puede hacer para que su proyecto sea seguro para la producción. No vamos a profundizar mas en esto, pero tenga en cuenta que debe verificar su proyecto antes de la producción utilícelo para buscar cualquier problema relevante.

Sirviendo a Django a través de WSGI

La plataforma de implementación principal de Django es WSGI. **WSGI son las siglas de Web Server Gateway Interface** y es el estándar para servir aplicaciones Python en la web.

Cuando genera un nuevo proyecto usando el comando `startproject`, Django crea un archivo `wsgi.py` dentro del directorio de su proyecto. Este archivo contiene una aplicación WSGI invocable, que es un punto de acceso a su aplicación.

[501]

Va en directo

WSGI se usa tanto para ejecutar su proyecto con el servidor de desarrollo Django e implementar su aplicación con el servidor de su elección en una producción ambiente.

Puede obtener más información sobre WSGI en <https://wsgi.readthedocs.io/en/latest/>.

Instalación de uWSGI

A lo largo de este libro, ha estado utilizando el servidor de desarrollo Django para ejecutar proyectos en su entorno local. Sin embargo, necesita un servidor web real para implementar su aplicación en un entorno de producción.

uWSGI es un servidor de aplicaciones Python extremadamente rápido. Se comunica con tu Aplicación Python que utiliza la especificación WSGI. uWSGI traduce solicitudes web en un formato que su proyecto Django pueda procesar.

Instale uWSGI usando el siguiente comando:

```
pip install uwsgi == 2.0.18
```

Para construir uWSGI, necesitará un compilador de C, como gcc o clang . En un entorno Linux, puede instalar un compilador C con el comando `apt-get` instalar `build-essential` .

Si está utilizando macOS, puede instalar uWSGI con el paquete Homebrew administrador usando el comando `brew install uwsgi` .

Si desea instalar uWSGI en Windows, necesitará Cygwin: <https://www.cygwin.com>. Sin embargo, es conveniente utilizar uWSGI en entornos basados en UNIX.

Puede leer la documentación de uWSGI en <https://uwsgi-docs.readthedocs.io/es/ultimo/>.

Configuración de uWSGI

Puede ejecutar uWSGI desde la línea de comandos. Abra el shell y ejecute lo siguiente comando desde el directorio del proyecto educa :

```
sudo uwsgi --module = educa.wsgi: application \  
--env = DJANGO_SETTINGS_MODULE = educa.settings.pro \  
--master --pidfile = / tmp / project-master.pid \  
--http = 127.0.0.1: 8000 \  
--uid = 1000 \  
--virtualenv = / home / env / educa /
```

[502]

Reemplace la ruta en la opción virtualenv con su entorno virtual real directorio. Si no está utilizando un entorno virtual, puede omitir esta opción.

Es posible que deba anteponer sudo a este comando si no tiene los permisos. Es posible que también deba agregar la opción --plugin = python3 si el El módulo no se carga por defecto.

Con este comando, puede ejecutar uWSGI en su localhost con lo siguiente opciones:

- Utiliza educa.wsgi: aplicación WSGI invocable
- Carga la configuración para el entorno de producción
- Usted le dice a uWSGI utilizar la Educa entorno virtual

Si no está ejecutando el comando dentro del directorio del proyecto, incluya la opción --chdir = / ruta / a / educa / con la ruta a su proyecto.

Abra <http://127.0.0.1:8000/> en su navegador. Debería ver una pantalla como la siguiente:

Figura 14.1: La página de lista de cursos servida con uWSGI

Puede ver el HTML renderizado que corresponde a la vista de lista de cursos, pero no se están cargando imágenes o hojas de estilo CSS. La razón de esto es que no lo hiciste configurar uWSGI para servir archivos estáticos. Configuraré el servicio de archivos estáticos en el entorno de producción más adelante en este capítulo.

uWSGI le permite definir una configuración personalizada en un archivo .ini . Esto es más conveniente que pasar opciones a través de la línea de comando.

Cree la siguiente estructura de archivos dentro del directorio global educa / :

Va en directo

Edite el archivo config / uwsgi.ini y agregue el siguiente código:

```
[uwsgi]
# variables
nombreproyecto = educa
base = / inicio / proyectos / educa

# configuración
maestro = verdadero
virtualenv = / home / env / % (nombre del proyecto)
pythonpath = % (base)
chdir = % (base)
env = DJANGO_SETTINGS_MODULE = % (nombre del proyecto) .settings.pro
module = % (nombre del proyecto) .wsgi: aplicación
socket = /tmp/%(projectname).sock
chmod-socket = 666
```

En el archivo uwsgi.ini , define las siguientes variables:

- **projectname** : el nombre de su proyecto Django, que es educa .
- **base** : El camino absoluto hacia el proyecto educa . Reemplázelo con el absoluto camino a su proyecto.

Estas son variables personalizadas que usará en las opciones de uWSGI. Puedes definir cualquier otra variable que desee, siempre que los nombres sean diferentes a las opciones de uWSGI.

Configura las siguientes opciones:

- **maestro** : habilita el proceso maestro.
- **virtualenv** : la ruta a su entorno virtual. Reemplaza esta ruta con el camino apropiado.
- **pythonpath** : las rutas para agregar a su ruta de Python.
- **chdir** : la ruta al directorio de su proyecto, para que uWSGI cambie a ese directorio antes de cargar la aplicación.
- **env** : Variables de entorno. Incluye el DJANGO_SETTINGS_MODULE variable, que apunta a la configuración del entorno de producción.
- **módulo** : el módulo WSGI que se utilizará. Establece esto en la aplicación invocable contenido en el módulo wsgi de su proyecto.
- **socket** : El socket UNIX / TCP para vincular el servidor.
- **chmod-socket** : los permisos de archivo que se aplicarán al archivo de socket. En este caso, usa 666 para que NGINX pueda leer / escribir el socket.

La opción de socket está diseñada para comunicarse con algún enrutador de terceros, como NGINX, mientras que la opción http es para que uWSGI acepte HTTP entrante solicitudes y enrutarlas por sí mismo. Vas a ejecutar uWSGI usando un socket, ya que va a configurar NGINX como su servidor web y se comunicará con uWSGI a través del socket.

Puede encontrar la lista de opciones de uWSGI disponibles en <https://uwsgi-docs.readthedocs.io/en/latest/Options.html>.

Ahora, puede ejecutar uWSGI con su configuración personalizada usando este comando:

```
uwsgi -ini config / uwsgi.ini
```

No podrá acceder a su instancia de uWSGI desde su navegador ahora, ya que está corriendo a través de un enchufe. Completamos el entorno de producción.

Instalación de NGINX

Cuando está sirviendo un sitio web, debe ofrecer contenido dinámico, pero también deben servir archivos estáticos, como hojas de estilo CSS, archivos JavaScript e imágenes. Mientras uWSGI es capaz de servir archivos estáticos, agrega una sobrecarga innecesaria a HTTP solicitudes y, por lo tanto, se recomienda configurar un servidor web, como NGINX, en frente de eso.

NGINX es un servidor web enfocado en alta concurrencia, rendimiento y bajo uso de memoria. NGINX también actúa como un proxy inverso, recibiendo solicitudes HTTP y enrutarlos a diferentes backends. Como se mencionó, generalmente, utilizará una web servidor, como NGINX, frente a uWSGI para servir archivos estáticos de manera eficiente y rápidamente y enviará solicitudes dinámicas a los trabajadores de uWSGI. Mediante el uso NGINX, también puede aplicar reglas y beneficiarse de sus capacidades de proxy inverso.

Instale NGINX con el siguiente comando:

```
sudo apt-get install nginx
```

Si está usando macOS, puede instalar NGINX usando el comando `brew install nginx`.

Puede encontrar binarios de NGINX para Windows en <https://nginx.org/en/download.html>.

Abra un shell y ejecute NGINX con el siguiente comando:

```
sudo nginx
```

Va en directo

Abra la URL `http://127.0.0.1` en su navegador. Deberías ver lo siguiente pantalla:

Figura 14.2: La página predeterminada de NGINX

Si ve esta pantalla, NGINX se instaló correctamente. 80 es el puerto predeterminado Configuración de NGINX.

El entorno de producción

El siguiente diagrama muestra el ciclo de solicitud / respuesta de la producción entorno que está configurando:

Figura 14.3: El ciclo de solicitud / respuesta del entorno de producción

Lo siguiente ocurrirá cuando el navegador del cliente envíe una solicitud HTTP:

1. NGINX recibe la solicitud HTTP
2. NGINX delega la solicitud a uWSGI a través de un socket
3. uWSGI pasa la solicitud a Django para su procesamiento
4. Django devuelve una respuesta HTTP que se devuelve a NGINX, que a su vez, lo devuelve al navegador del cliente

Configurando NGINX

Cree un nuevo archivo dentro del directorio `config` / y asígnele el nombre `nginx.conf` . Añade el siguiente código:

[506]

```
# el componente ascendente al que nginx necesita conectarse
educa {
    servidor unix: ///tmp/educa.sock;
}

servidor {
    escucha 80;
    nombre_servidor www.educaproject.com educaproject.com;

    access_log off;
    error_log /home/projects/educa/logs/nginx_error.log;

    ubicación / {
        incluir / etc / nginx / uwsgi_params;
        uwsgi_pass educa;
    }
}
```

Esta es la configuración básica de NGINX. Configuras un upstream llamado `educa` , que apunta al socket creado por uWSGI. Usas el bloque de servidor y agregas la siguiente configuración:

- Le dice a NGINX que escuche en el puerto 80 .
- Se establece el nombre del servidor a la vez `www.educaproject.com` y `educaproject`.

- com . NGINX atenderá las solicitudes entrantes para ambos dominios.
- Se establece explícitamente `access_log` a fuera . Puede utilizar esta directiva para almacenar registros de acceso en un archivo.
- Utiliza la directiva `error_log` para establecer la ruta al archivo donde almacenar registros de errores. Reemplace esta ruta con la ruta donde le gustaría para almacenar registros de errores de NGINX. Analice este archivo de registro si tiene algún problema mientras usa NGINX.
- Incluye los parámetros de configuración predeterminados de uWSGI que vienen con NGINX. Estos se encuentran junto al archivo de configuración predeterminado para NGINX. Por lo general, puede encontrarlos en cualquiera de estas tres ubicaciones: `/usr/local/nginx/conf/` `usgi_params` , `/etc/nginx/usgi_params` o `/usr/local/etc/nginx/usgi_params` .
- Especifica que todo lo que se encuentra en `/path` debe enrutarse a `educa` enchufe (uWSGI).

Puede encontrar la documentación de NGINX en <https://nginx.org/en/docs/> .

El archivo de configuración predeterminado para NGINX se llama `nginx.conf` y normalmente reside en cualquiera de estos tres directorios: `/usr/local/nginx/conf` , `/etc/nginx` , o `/usr/local/etc/nginx` .

[507]

Va en directo

Localice su archivo de configuración `nginx.conf` y agregue la siguiente directiva `include` dentro del bloque `http` :

```
http {
    incluir /home/projects/educa/config/nginx.conf;
    # ...
}
```

Reemplace `/home/projects/educa/config/nginx.conf` con la ruta al archivo de configuración que creó para el proyecto `educa` . En este código, incluye el Archivo de configuración NGINX para su proyecto en la configuración NGINX predeterminada.

Abra un shell y ejecute uWSGI si aún no lo está ejecutando:

```
uwsgi --ini config / uwsgi.ini
```

Abra un segundo shell y vuelva a cargar NGINX con el siguiente comando:

```
sudo nginx -s recarga
```

Siempre que desee detener NGINX, puede hacerlo con gracia con lo siguiente mando:

```
sudo nginx -s quit
```

Si desea detener rápidamente NGINX, en lugar de salir, utilice la señal de parada . El abandono señal de espera a los procesos de trabajo para terminar de atender las solicitudes actuales, mientras que la parada la señal detiene NGINX abruptamente.

Dado que está utilizando un nombre de dominio de muestra, debe redirigirlo a su host local. Edite su archivo `/etc/hosts` y agregue la siguiente línea:

```
127.0.0.1 educaproject.com www.educaproject.com
```

Al hacerlo, está enrutando ambos nombres de host a su servidor local. En una producción servidor, no necesitará hacer esto, ya que tendrá una dirección IP fija y apuntará su nombre de host a su servidor en la configuración de DNS de su dominio.

Abra `http://educaproject.com/` en su navegador. Deberías poder ver su sitio, aún sin ningún activo estático cargado. Tu entorno de producción es casi listo.

Ahora puede restringir los hosts que pueden servir a su proyecto Django. Edite el configuración de producción del archivo `settings / pro.py` de su proyecto y cambie el `ALLOWED_`

Configuración HOSTS , de la siguiente manera:

```
ALLOWED_HOSTS = [ 'educaproject.com', 'www.educaproject.com' ]
```

Django ahora solo servirá su aplicación si se ejecuta en cualquiera de estos nombres de host. Puede leer más sobre la configuración ALLOWED_HOSTS en [https:// docs. django project. com/ en/ 3. 0/ ref/ settings/ # allowed- hosts.](https://docs.djangoproject.com/en/3.0/ref/settings/#allowed-hosts)

[508]

Sirviendo activos estáticos y multimedia

uWSGI es capaz de servir archivos estáticos sin problemas, pero no es tan rápido y efectivo como NGINX. Para obtener el mejor rendimiento, utilizará NGINX para entregar los archivos estáticos en su entorno de producción. Configuraré NGINX para servir ambos archivos estáticos de su aplicación (hojas de estilo CSS, archivos JavaScript e imágenes) y archivos multimedia subido por los instructores para los contenidos del curso.

Edite el archivo settings / base.py y agregue la siguiente línea justo debajo de STATIC_ Configuración de URL :

```
STATIC_ROOT = os.path.join (BASE_DIR, 'static /')
```

Cada aplicación en su proyecto Django puede contener archivos estáticos en un / directorio. Django proporciona un comando para recopilar archivos estáticos de todas las aplicaciones en un solo lugar. Esto simplifica la configuración para servir archivos estáticos en producción. El collectstatic comando recopila los archivos estáticos de todas las aplicaciones de la proyecto en la ruta definida en STATIC_ROOT .

Abra el shell y ejecute el siguiente comando:

```
python manage.py collectstatic
```

Verá esta salida:

165 archivos estáticos copiados en '/ educa / static'.

Archivos ubicados en el directorio static / de cada aplicación presente en el INSTALLED_APPS ajuste de haber sido copiado a lo global / Educa / estática / proyecto directorio.

Ahora, edite el archivo config / nginx.conf y cambie su código, así:

```
# el componente ascendente al que nginx necesita conectarse
educa {
    servidor unix: ///tmp/educa.sock;
}

servidor {
    escucha 80;
    nombre_servidor www.educaproject.com educaproject.com;

    access_log off;
    error_log /home/projects/educa/logs/nginx_error.log;

    ubicación / {
        incluir / etc / nginx / uwsgi_params;
```

[509]

Va en directo

```

        uwsgi_pass educa;
    }

    location /static / {
        alias /inicio / proyectos / educa / static /;
    }

    location /media / {
        alias /inicio / proyectos / educa / media /;
    }
}

```

Recuerde reemplazar la ruta /home/projects/educa/ con la ruta absoluta al directorio de su proyecto. Estas directivas le dicen a NGINX que sirva archivos estáticos ubicados en las rutas /static/ y /media/ directamente. Estos caminos son los siguientes:

- /static/ : corresponde a la ruta de la configuración STATIC_URL . El objetivo ruta corresponde al valor de la configuración STATIC_ROOT . Lo usas para servir los archivos estáticos de su aplicación.
- /media/ : corresponde a la ruta de la configuración MEDIA_URL y su destino ruta corresponde al valor de la configuración MEDIA_ROOT . Lo usas para servir los archivos multimedia cargados en los contenidos del curso.

El esquema del entorno de producción ahora se ve así:

Figura 14.4: El ciclo de solicitud / respuesta del entorno de producción, incluidos los archivos estáticos

Los archivos bajo las rutas /static/ y /media/ ahora son servidos directamente por NGINX, en lugar de ser reenviado a uWSGI. Las solicitudes a cualquier otra ruta todavía se pasan por NGINX a uWSGI a través del socket UNIX.

Vuelva a cargar la configuración de NGINX con el siguiente comando para realizar un seguimiento de la nueva caminos:

sudo nginx -s recarga

Abra <http://educaproject.com/> en su navegador. Deberías ver lo siguiente pantalla:

Figura 14.5: La página de lista de cursos servida con NGINX y uWSGI

Los recursos estáticos, como las imágenes y las hojas de estilo CSS, ahora se cargan correctamente. Las solicitudes HTTP para archivos estáticos ahora las atiende NGINX directamente, en lugar de siendo reenviado a uWSGI.

¡Excelente! Ha configurado correctamente NGINX para servir archivos estáticos.

Asegurar conexiones con SSL / TLS

El **protocolo Transport Layer Security (TLS)** es el estándar para servir sitios web a través de una conexión segura. El predecesor de TLS es **Secure Sockets Layer (SSL)**. Aunque SSL ahora está en desuso, en varias bibliotecas y documentación en línea encontrará referencias a los términos TLS y SSL. Se recomienda encarecidamente que sirva sus sitios web bajo HTTPS. Vas a configurar un SSL / TLS certificado en NGINX para servir su sitio de forma segura.

Crear un certificado SSL / TLS

Cree un nuevo directorio dentro del directorio del proyecto educa y asígnele el nombre ssl . Luego, generar un certificado SSL / TLS desde la línea de comando con lo siguiente mando:

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa: 2048 -keyout ssl /  
educa.key -out ssl / educa.crt
```

[511]

Va en directo

Está generando una clave privada y un certificado SSL / TLS de 2048 bits que es válido para un año. Se le pedirá que ingrese datos, de la siguiente manera:

```
Nombre del país (código de 2 letras) []:  
Nombre del estado o provincia (nombre completo) []:  
Nombre de la localidad (p. Ej., Ciudad) []:  
Nombre de la organización (p. Ej., Empresa) []:  
Nombre de la unidad organizativa (p. Ej., Sección) []:  
Nombre común (p. Ej., Nombre de host completo) []: educaproject.com  
Dirección de correo electrónico []: email@domain.com
```

Puede completar los datos solicitados con su propia información. El más importante campo es el nombre común . Debe especificar el nombre de dominio para el certificado. Utiliza educaproject.com . Esto generará, dentro del directorio ssl / , un archivo educa.key archivo de clave privada y un archivo educa.crt , que es el certificado real.

Configuración de NGINX para usar SSL / TLS

Editar la nginx.conf archivo de la Educa proyecto y editar el servidor de bloque para incluir SSL / TLS, como sigue:

```

servidor {
    escucha                80;
    escucha                443 ssl;
    ssl_certificate /home/projects/educa/ssl/educa.crt;
    ssl_certificate_key /home/projects/educa/ssl/educa.key;
    nombre del servidor    www.educaproject.com educaproject.com;

    # ...
}

```

Con el código anterior, su servidor ahora escucha HTTP a través del puerto 80 y HTTPS a través del puerto 443 . Indicas la ruta al certificado SSL / TLS con `ssl_certificate` y la clave del certificado con `ssl_certificate_key` .

Recargue NGINX con el siguiente comando:

```
sudo nginx -s recarga
```

NGINX cargará la nueva configuración. Abra `https://educaproject.com/` con su navegador. Debería ver un mensaje de advertencia similar al siguiente:

[512]

Figura 14.6: Una advertencia de certificado no válido

Esta pantalla puede variar según su navegador. Te alerta que tu sitio está no utilizar un certificado válido o de confianza; el navegador no puede verificar la identidad de su sitio. Esto se debe a que firmó su propio certificado en lugar de obtener uno de una **autoridad de certificación** (CA) de confianza . Cuando posee un dominio real, puede solicitar para que una CA de confianza emita un certificado SSL / TLS para que los navegadores puedan verificar su identidad. Si desea obtener un certificado de confianza para un dominio real, puede consultar al proyecto *Let's Encrypt* creado por la Fundación Linux. Es una CA sin fines de lucro que simplifica la obtención y renovación de certificados SSL / TLS confiables de forma gratuita. Tú puede encontrar más información en <https://letsencrypt.org> .

Haga clic en el enlace o botón que proporciona información adicional y elija visitar el sitio web, ignorando las advertencias. El navegador puede pedirle que agregue una excepción para este certificado o verifique que confía en él. Si está utilizando Chrome, puede No veo ninguna opción para proceder al sitio web. Si este es el caso, escriba `thisisunsafe` o `badidea` directamente en Chrome en la misma página de advertencia. Chrome se cargará el sitio web. Tenga en cuenta que hace esto con su propio certificado emitido; no confíes en nadie certificado desconocido o eludir las comprobaciones del certificado SSL / TLS del navegador para

dominios.

[513]

Va en directo

Cuando acceda al sitio, verá que el navegador muestra un icono de candado junto a la URL, de la siguiente manera:

Figura 14.7: La barra de direcciones del navegador, incluido un icono de candado de conexión segura

Si hace clic en el icono de candado, los detalles del certificado SSL / TLS se mostrarán de la siguiente manera:

Figura 14.8: Detalles del certificado TLS / SSL

En los detalles del certificado, puede ver que es un certificado autofirmado y puede ver su fecha de caducidad. Ahora está sirviendo su sitio de forma segura.

Configurando su proyecto Django para SSL / TLS

Django viene con configuraciones específicas para la compatibilidad con SSL / TLS. Edite la configuración / pro.py archivo de configuración y agregue las siguientes configuraciones:

```
SECURE_SSL_REDIRECT = Verdadero
CSRF_COOKIE_SECURE = Verdadero
```

Estos ajustes son los siguientes:

- **SECURE_SSL_REDIRECT** : si las solicitudes HTTP deben redirigirse a HTTPS
- **CSRF_COOKIE_SECURE** : debe configurarse para establecer una cookie segura para Protección de **falsificación de solicitud entre sitios (CSRF)**

[514]

Django ahora redirigirá las solicitudes HTTP a HTTPS y cookies para la protección CSRF ahora estará seguro.

Redirigir el tráfico HTTP a HTTPS

Estás redirigiendo solicitudes HTTP a HTTPS usando Django. Sin embargo, esto puede ser manejado de una manera más eficiente usando NGINX.

Editar la `nginx.conf` archivo de la Educa proyecto y cambiarlo de la siguiente manera:

```
# el componente ascendente al que nginx necesita conectarse
educa {
    servidor unix: ///tmp/educa.sock;
}

servidor {
    escucha 80;
    nombre_servidor www.educaproject.com educaproject.com;
    return 301 https://educaproject.com$request_uri;
}

servidor {
    escucha                443 ssl;
    ssl_certificate /home/projects/educa/ssl/educa.crt;
    ssl_certificate_key /home/projects/educa/ssl/educa.key;
    nombre_servidor www.educaproject.com educaproject.com;

    access_log off;
    error_log /home/projects/educa/logs/nginx_error.log;

    ubicación / {
        incluir / etc / nginx / uwsgi_params;
        uwsgi_pass educa;
    }

    location / static / {
        alias / inicio / proyectos / educa / static /;
    }

    location / media / {
        alias / inicio / proyectos / educa / media /;
    }
}
```

[515]

para que la plataforma solo esté disponible a través de SSL / TLS (puerto 443). En la parte superior de la bloque de servidor original , agrega un bloque de servidor adicional que solo escucha en puerto 80 y redirige todas las solicitudes HTTP a HTTPS. Para lograr esto, regresas un código de respuesta HTTP 301 (redireccionamiento permanente) que redirige a https: // versión de la URL solicitada.

Recargue NGINX con el siguiente comando:

```
sudo nginx -s recarga
```

Ahora está redirigiendo todo el tráfico HTTP a HTTPS utilizando NGINX.

Usar Daphne para canales Django

En el *Capítulo 13 , Creación de un servidor de chat* , usó Canales Django para crear un chat servidor utilizando WebSockets. uWSGI es adecuado para ejecutar Django o cualquier otro Aplicación WSGI, pero no admite la comunicación asincrónica mediante **Interfaz de puerta de enlace de servidor asíncrono (ASGI)** o WebSockets. Para correr Canales en producción, necesita un servidor web ASGI que sea capaz de administrar WebSockets.

Daphne es un servidor HTTP, HTTP2 y WebSocket para ASGI desarrollado para servir Canales. Puede ejecutar Daphne junto con uWSGI para servir tanto a ASGI como a WSGI aplicaciones de manera eficiente.

Daphne se instala automáticamente como una dependencia de los canales. Si tu fuiste a través de los pasos para instalar canales en el *Capítulo 13 , Creación de un servidor de chat* , Daphne ya está instalado en su entorno Python. También puede instalar Daphne con el siguiente comando:

```
pip instalar daphne == 2.4.1
```

Puede encontrar más información sobre Daphne en <https://github.com/django/daphne>.

Django 3 es compatible con WSGI y ASGI, pero todavía no es compatible con WebSockets. Por lo tanto, se va a editar el asgi.py archivo de la Educa proyecto para su uso Canales.

Edite el archivo educa / asgi.py de su proyecto y haga que se vea así:

```
importar sistema operativo
importar django
desde los canales. enrutamiento de importación get_default_application
```

[516]

```
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'educa.settings')
django.setup()
aplicación = get_default_application()
```

Está cargando la aplicación ASGI predeterminada utilizando Canales en lugar de la módulo ASGI estándar de Django. Puede encontrar más información sobre la implementación Daphne con servidores de protocolo en <https://channels.readthedocs.io/en/latest/deploying.html#run-protocol-servers>.

Abra un nuevo shell y configure la variable de entorno DJANGO_SETTINGS_MODULE con el entorno de producción utilizando el siguiente comando:

```
exportar DJANGO_SETTINGS_MODULE = educa.settings.pro
```

En el mismo shell, desde el directorio del proyecto educa , ejecute el siguiente comando:

```
daphne -u /tmp/daphne.sock educa.asgi: aplicación
```


Verá el siguiente resultado:

```
2020-02-11 00: 49: 44,223 INFO Servidor de inicio en unix: /tmp/daphne.sock
```

```
2020-02-11 00: 49: 44,223 INFO Soporte HTTP / 2 no habilitado (instale el  
http2 y tls Twisted extras)
```

```
2020-02-11 00: 49: 44,223 INFO Configuración de punto final unix: / tmp / daphne.  
calcetín
```

El resultado muestra que Daphne se está ejecutando correctamente en un socket UNIX.

Usar conexiones seguras para WebSockets

Ha configurado NGINX para utilizar conexiones seguras a través de SSL / TLS. Necesitas para cambiar las conexiones ws (WebSocket) para usar el protocolo wss (WebSocket Secure) ahora, de la misma manera que las conexiones HTTP ahora se sirven a través de HTTPS.

Edite la plantilla chat / room.html de la aplicación de chat y busque lo siguiente línea en el bloque domready :

```
var url = 'ws: //' + window.location.host +
```

Reemplaza esa línea con la siguiente:

```
var url = 'ws s: //' + window.location.host +
```

Ahora se conectará explícitamente a un WebSocket seguro.

[517]

Va en directo

Incluyendo Daphne en la configuración NGINX

En su configuración de producción, ejecutará Daphne en un socket UNIX y utilizará NGINX frente a él. NGINX pasará las solicitudes a Daphne según la solicitud camino. Expondrá Daphne a NGINX a través de una interfaz de socket UNIX, al igual que la configuración de uWSGI.

Editar la configuración / nginx.conf archivo de la Educa proyecto y hacer que parezca como sigue:

```
# los componentes ascendentes a los que nginx necesita conectarse
educa {
    servidor unix: /tmp/educa.sock;
}

río arriba daphne {
    servidor unix: /tmp/daphne.sock;
}

servidor {
    escucha 80;
    nombre_servidor www.educaproject.com educaproject.com;
    return 301 https: //educaproject.com$request_uri;
}

servidor {
    escucha                                443 ssl;
    ssl_certificate /home/projects/educa/ssl/educa.crt;
    ssl_certificate_key /home/projects/educa/ssl/educa.key;

    nombre_servidor www.educaproject.com educaproject.com;

    access_log off;
    error_log /home/projects/educa/logs/nginx_error.log;
```

```

ubicación / {
    incluir / etc / nginx / uwsgi_params;
    uwsgi_pass educa;
}

ubicación / ws / {
    proxy_http_version 1.1;
    proxy_set_header Actualización $ http_upgrade;
    proxy_set_header Conexión "actualización";
    proxy_redirect off;

    proxy_pass          http: // daphne;

```

[518]

```

}

location / static / {
    alias / inicio / proyectos / educa / static /;
}

location / media / {
    alias / inicio / proyectos / educa / media /;
}
}

```

En esta configuración, configura un nuevo upstream llamado daphne , que apunta a un socket creado por Daphne. En el bloque del servidor , configura la ubicación / ws / para enviar solicitudes a Daphne. Utiliza la directiva proxy_pass para pasar solicitudes a Daphne y tú incluís algunas directivas de proxy adicionales.

Con esta configuración, NGINX pasará cualquier solicitud de URL que comience con / ws / prefijo a Daphne y el resto a uWSGI, excepto para los archivos bajo / static / o / medios / rutas, que serán servidas directamente por NGINX.

La configuración de producción que incluye a Daphne ahora se ve así:

Figura 14.9: El ciclo de solicitud / respuesta del entorno de producción, incluido Daphne

NGINX se ejecuta frente a uWSGI y Daphne como un servidor proxy inverso. Caras NGINX la Web y pasa solicitudes al servidor de aplicaciones (uWSGI o Daphne) basado en su prefijo de ruta. Además de esto, NGINX también sirve archivos estáticos y redirige no solicitudes seguras a las seguras. Esta configuración reduce el tiempo de inactividad, consume menos servidor recursos y proporciona mayor rendimiento y seguridad.

Detenga e inicie uWSGI y Daphne, y luego vuelva a cargar NGINX con lo siguiente comando para realizar un seguimiento de la última configuración:

```
sudo nginx -s recarga
```

Va en directo

Utilice su navegador para crear un curso de muestra con un usuario instructor, inicie sesión con un usuario que está inscrito en el curso y abra <https://educaproject.com/chat/room/1/> con su navegador. Debería poder enviar y recibir mensajes como el siguiente ejemplo:

Figura 14.10: Mensajes de la sala de chat del curso servidos con NGINX y Daphne

Daphne está funcionando correctamente y NGINX le está pasando solicitudes. Todas las conexiones están protegidos mediante SSL / TLS.

¡Felicidades! Ha creado una pila personalizada lista para producción con NGINX, uWSGI y Daphne. Podría realizar una mayor optimización para obtener un rendimiento adicional y seguridad mejorada a través de ajustes de configuración en NGINX, uWSGI y Dafne. Sin embargo, ¡esta configuración de producción es un gran comienzo!

Crear un middleware personalizado

Ya conoce la configuración de MIDDLEWARE , que contiene el middleware para su proyecto. Puede considerarlo como un sistema de complementos de bajo nivel, que le permite implementar ganchos que se ejecutan en el proceso de solicitud / respuesta. Cada middleware es responsable de alguna acción específica que se ejecutará para todas las solicitudes HTTP o respuestas.

Evite agregar un procesamiento costoso al middleware, ya que son ejecutado en cada solicitud.

Cuando se recibe una solicitud HTTP, el middleware se ejecuta en orden de aparición en el ajuste MIDDLEWARE. Cuando Django genera una respuesta HTTP, la respuesta pasa por todo el middleware en orden inverso.

Un middleware se puede escribir como una función, de la siguiente manera:

```
def my_middleware (get_response):

    def middleware (solicitud):
        # Código ejecutado para cada solicitud antes
        # se llama a la vista (y al middleware posterior).

        respuesta = get_response (solicitud)

        # Código ejecutado para cada solicitud / respuesta después
        # la vista se llama.

        respuesta de retorno

    devolver middleware
```

Una fábrica de middleware es un invocable que toma un invocable `get_response` y devuelve un middleware. Un middleware es un invocable que toma una solicitud y devuelve una respuesta, como una vista. El GET RESPONSE exigible podría ser la próxima en el middleware cadena o la vista real en el caso del último middleware enumerado.

Si algún middleware devuelve una respuesta sin llamar a su `get_response` invocable, cortocircuita el proceso; no se ejecuta más middleware (tampoco la vista), y la respuesta vuelve a través de las mismas capas por las que pasó la solicitud.

El orden del middleware en la configuración de MIDDLEWARE es muy importante porque El middleware puede depender del conjunto de datos en la solicitud de otro middleware que tenga ejecutado previamente.

Al agregar un nuevo middleware a la configuración de MIDDLEWARE, haga asegúrese de colocarlo en la posición correcta. El middleware se ejecuta en orden de aparición en el escenario durante la fase de solicitud, y en orden inverso para las respuestas.

Puede encontrar más información sobre middleware en <https://docs.djangoproject.com/en/3.0/topics/http/middleware/>.

[521]

Va en directo

Crear un middleware de subdominio

Va a crear un middleware personalizado para permitir que los cursos sean accesibles a través de un subdominio personalizado. Cada URL de detalles del curso, que se parece a `https://educaproject.com/course/django/`, también será accesible a través del subdominio que hace uso del slug del curso, como `https://django.educaproject.com/`. Los usuarios podrán utilizar el subdominio como acceso directo para acceder a los detalles del curso. Cualquier solicitud a subdominios será redirigida a cada detalle del curso correspondiente. URL.

El middleware puede residir en cualquier lugar de su proyecto. Sin embargo, se recomienda para crear un archivo `middleware.py` en el directorio de su aplicación.

Cree un nuevo archivo dentro del directorio de la aplicación de cursos y asígnele el nombre `middleware`.

py . Agregue el siguiente código:

```
desde django.urls import reverse
desde django.shortcuts import get_object_or_404, redireccionar
desde el curso de importación de .models

def subdomain_course_middleware (get_response):
    """
    Subdominios para cursos
    """
    def middleware (solicitud):
        host_parts = request.get_host (). split ('.')
        if len (host_parts) > 2 y host_parts [0] != 'www':
            # obtener el curso para el subdominio dado
            course = get_object_or_404 (Curso, slug = host_parts [0])
            course_url = reverse ('curso_detalle',
                                args = [course.slug])

            # redirigir la solicitud actual a la vista course_detail
            url = '{:} // {} {}'.format (request.scheme,
                                       ' '.join (partes_host [1:]),
                                       course_url)

            volver a redireccionar (url)

        respuesta = get_response (solicitud)
        respuesta de retorno

    devolver middleware
```

[522]

Cuando se recibe una solicitud HTTP, realiza las siguientes tareas:

1. Obtiene el nombre de host que se está utilizando en la solicitud y lo divide en partes. Por ejemplo, si el usuario accede a mycourse.educaproject.com , genera la lista ['mycourse', 'educaproject', 'com'] .
2. Compruebe si el nombre de host incluye un subdominio comprobando si la división generó más de dos elementos. Si el nombre de host incluye un subdominio, y esto no es www , intentas obtener el curso con el slug proporcionado en el subdominio.
3. Si no se encuentra un curso, genera una excepción HTTP 404 . De lo contrario, tu Redirigir el navegador a la URL de detalles del curso.

Edite el archivo settings / base.py del proyecto y agregue 'cursos.middleware.

SubdomainCourseMiddleware ' en la parte inferior de la lista MIDDLEWARE , de la siguiente manera:

```
MIDDLEWARE = [
    # ...
    'cursos.middleware.subdomain_course_middleware',
]
```

El middleware ahora se ejecutará en cada solicitud.

Recuerde que los nombres de host permitidos para servir su proyecto Django se especifican en la configuración ALLOWED_HOSTS . Cambiemos esta configuración para que cualquier subdominio posible de educaproject.com está autorizado a servir su aplicación.

Edite el archivo settings / pro.py y modifique la configuración ALLOWED_HOSTS , de la siguiente manera:

```
ALLOWED_HOSTS = [ 'educaproject.com' ]
```

Un valor que comienza con un punto se utiliza como comodín de subdominio;
'educaproject.com' coincidirá con educaproject.com y cualquier subdominio para este dominio, por ejemplo course.educaproject.com y django.educaproject.com .

Sirviendo múltiples subdominios con NGINX

Necesita NGINX para poder servir su sitio con cualquier subdominio posible. Editar la config / nginx.conf archivo de la Educa proyecto y reemplace las dos ocurrencias de la siguiente línea:

```
nombre_servidor www.educaproject.com educaproject.com;
```

[523]

Va en directo

con el siguiente:

```
nombre_servidor * .educaproject.com educaproject.com;
```

Al usar el asterisco, esta regla se aplica a todos los subdominios de educaproject.com .
Para probar su middleware localmente, debe agregar los subdominios que desea probar en / etc / hosts . Para probar el middleware con un objeto Course con slug django , agregue la siguiente línea a su archivo / etc / hosts :

```
127.0.0.1 django.educaproject.com
```

Detenga e inicie uWSGI nuevamente y vuelva a cargar NGINX con el siguiente comando para realizar un seguimiento de la última configuración:

sudo nginx -s recarga

Luego, abra <https://django.educaproject.com/> en su navegador. El middleware encontrará el curso por subdominio y redirigirá su navegador a <https://educaproject.com/course/django/> .

Implementar la gestión personalizada comandos

Django permite que sus aplicaciones registren comandos de administración personalizados para la utilidad manage.py . Por ejemplo, usó los comandos de administración hacer mensajes y compilar mensajes en el *Capítulo 9 , Ampliación de su tienda* , para crear y compilar archivos de traducción.

Un comando de administración consiste en un módulo de Python que contiene una clase de comando que hereda de django.core.management.base.BaseCommand o uno de sus subclases. Puede crear comandos simples o hacer que tomen posiciones y argumentos opcionales como entrada.

Django busca comandos de administración en el directorio management / commands / para cada aplicación activa en la configuración INSTALLED_APPS . Cada módulo encontrado es registrado como un comando de gestión que lleva su nombre.

Puede obtener más información sobre los comandos de administración personalizados en <https://docs.djangoproject.com/en/3.0/howto/custom-management-commands/>.

Va a crear un comando de administración personalizado para recordarles a los estudiantes inscribirse al menos en un curso. El comando enviará un recordatorio por correo electrónico a los usuarios que han estado registrados por más tiempo que un período especificado que no están inscritos en cualquier curso todavía.

Cree la siguiente estructura de archivos dentro del directorio de la aplicación de los estudiantes :

```
administración/
  __init__.py
  comandos /
    __init__.py
    enroll_reminder.py
```

Edite el archivo enroll_reminder.py y agréguele el siguiente código:

```
importar fecha y hora
desde la configuración de importación django.conf
de django.core.management.base importar BaseCommand
desde django.core.mail importar send_mass_mail
desde django.contrib.auth.models import User
de django.db.models import Count
desde django.utils importar zona horaria

clase Command (BaseCommand):
    help = 'Envía un recordatorio por correo electrónico a los usuarios registrados más \
          de N días que aún no están inscritos en ningún curso '

    def add_arguments (self, parser):
        parser.add_argument ('- días', dest = 'días', tipo = int)

    def manejar (self, * args, ** opciones):
        correos electrónicos = []
        subject = 'Inscribase en un curso'
        date_joined = timezone.now (). today () - \
            datetime.timedelta (días = opciones ['días'])
        users = User.objects.annotate (course_count = Count ('cursos_
unido'))\
            .filtro (recuento_cursivo = 0,
                    date_joined__date__lte = date_joined)
        para usuario en usuarios:
            message = "" "Estimado {}:\
            Notamos que aún no se inscribió en ningún curso.\
            ¿Qué estás esperando? "" ". Formato (user.first_name)
            emails.append ((asunto,
                            mensaje,
                            settings.DEFAULT_FROM_EMAIL,
                            [user.email]))
        send_mass_mail (correos electrónicos)
        self.stdout.write ('Enviados {} recordatorios'.format (len (correos electrónicos)))
```

Va en directo

Este es su comando enroll_reminder . El código anterior es el siguiente:

- La clase Command hereda de BaseCommand .
- Incluye un atributo de ayuda . Este atributo proporciona una breve descripción de el comando que se imprime si ejecuta el comando python manage.py ayuda enroll_reminder .
- Utiliza el método add_arguments () para agregar el argumento con nombre --days . Este argumento se utiliza para especificar el número mínimo de días que un usuario tiene estar matriculado, sin haberse matriculado en ningún curso, para recibir la recordatorio.
- El comando handle () contiene el comando real. Tienes los días atributo analizado desde la línea de comando. Se utiliza la zona horaria de utilidad proporcionada por Django para recuperar la fecha actual de la zona horaria con timezone.now (). date (). (Puede establecer la zona horaria de su proyecto con el ajuste TIME_ZONE). Recupera los usuarios que se han registrado durante más de los días especificados y aún no están inscritos en ningún curso. Esto se logra anotando el QuerySet con el número total de cursos cada usuario está inscrito en. Usted genera el correo electrónico de recordatorio para cada usuario y añádalo a la lista de correos electrónicos . Finalmente, envías los correos electrónicos usando send_ función mass_mail () , que está optimizada para abrir una única conexión SMTP para enviar todos los correos electrónicos, en lugar de abrir una conexión por correo electrónico enviado.

Ha creado su primer comando de gestión. Abra el caparazón y ejecute su mando:

```
python manage.py enroll_reminder --days = 20
```

Si no tiene un servidor SMTP local en ejecución, puede consultar el *Capítulo 2* , *Mejora de su blog con funciones avanzadas* , donde configuré la configuración de SMTP para su primer proyecto de Django. Alternativamente, puede agregar la siguiente configuración a el archivo settings.py para hacer que los correos electrónicos de salida de Django a la salida estándar durante desarrollo:

```
EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'
```

Programemos su comando de administración para que el servidor lo ejecute todos los días a las 8 am Si está utilizando un sistema basado en UNIX como Linux o macOS, abra el shell y ejecute crontab -e para editar su crontab . Agregue la siguiente línea:

```
0 8 * * * python /path/to/educa/manage.py enroll_reminder --days = 20
--settings = educa.settings.pro
```

Si no está familiarizado con **cron** , puede encontrar una introducción a cron en <http://www.unixgeeks.org/security/newbie/unix/cron-1.html> .

[526]

Si está usando Windows, puede programar tareas usando el Programador de tareas. Tú puede encontrar más información al respecto en <https://docs.microsoft.com/en-us/windows/win32/taskschd/task-Scheduler-start-page> .

Otra opción para ejecutar acciones periódicamente es crear tareas y programar ellos con apio. Recuerde que usó Apio en el *Capítulo 7* , *Creación de un Shop* , para ejecutar tareas asincrónicas. En lugar de crear comandos de gestión y programarlos con cron, puede crear tareas asincrónicas y ejecutar ellos con el programador de ritmos de Apio. Puede obtener más información sobre la programación periódica tareas con Apio en <https://celery.readthedocs.io/en/latest/userguide/>

[tareas-periódicas.html](#).

Utilice los comandos de administración para los scripts independientes que desee programar con cron o el panel de control del programador de Windows.

Django también incluye una utilidad para llamar a comandos de administración usando Python. Usted puede ejecutar comandos de administración desde su código de la siguiente manera:

```
desde la gestión de importación django.core
management.call_command('enroll_reminder', días = 20)
```

¡Felicidades! Ahora puede crear comandos de administración personalizados para su aplicaciones y programarlas cuando sea necesario.

Resumen

En este capítulo, configuró un entorno de producción utilizando NGINX, uWSGI y Daphne. Aseguró su entorno a través de SSL / TLS. Tú también implementó un middleware personalizado y aprendió a crear comandos de gestión.

Ha llegado al final de este libro. ¡Felicidades! Has aprendido las habilidades requerido para construir aplicaciones web exitosas con Django. Este libro ha guiado a través del proceso de desarrollo de proyectos de la vida real e integración de Django con otras tecnologías. Ahora está listo para crear su propio proyecto Django, ya sea un simple prototipo o una aplicación web a gran escala.

¡Buena suerte con tu próxima aventura con Django!

[527]

Otros libros puede disfrutar

en estos otros libros de Packt:

LIBRO DE COCINA DE PANDAS 1.X - Segunda edición

Matt Harrison, Theodore Petrou

ISBN: 978-1-83921-310-6

- Exploración de datos maestros en pandas a través de docenas de problemas de práctica
- Agrupar, agregar, transformar, remodelar y filtrar datos
- Fusionar datos de diferentes fuentes a través de operaciones similares a pandas SQL
- Cree visualizaciones a través de ganchos de pandas para matplotlib y seaborn
- Utilice pandas, funcionalidad de series de tiempo para realizar análisis poderosos
- Cree flujos de trabajo para procesar macrodatos que no caben en la memoria

[529]

Aprendizaje profundo avanzado con tensorflow 2 y Keras - Segunda edición

Rowel Atienza

ISBN: 978-1-83882-165-4

- Utilice técnicas de maximización de información mutua para realizar actividades sin supervisión aprendizaje
- Utilice la segmentación para identificar la clase de píxeles de cada objeto en una imagen
- Identificar tanto el cuadro delimitador como la clase de objetos en una imagen usando object detección
- Aprenda los componentes básicos de las técnicas avanzadas: MLPs, CNN y RNN
- Comprender las redes neuronales profundas, incluidas ResNet y DenseNet
- Comprender y crear modelos autorregresivos: codificadores automáticos, VAE y GAN
- Descubra e implemente métodos de aprendizaje por refuerzo profundo

Otros libros que puede disfrutar

Deje una reseña - informe a otros lectores qué piensas

Comparta sus pensamientos sobre este libro con otros dejando una reseña en el sitio que se lo compraste. Si compró el libro en Amazon, déjenos un comentario honesto. reseña en la página de Amazon de este libro. Esto es vital para que otros lectores potenciales puedan ver y utilice su opinión imparcial para tomar decisiones de compra, podemos entender qué nuestros clientes piensan en nuestros productos y nuestros autores pueden ver sus comentarios sobre la título que han trabajado con Packt para crear. Solo tomará unos minutos de su tiempo, pero es valioso para otros clientes potenciales, nuestros autores y Packt. ¡Gracias!

Índice

Simbolos

`__str__()` método [12](#)
`__unicode__()` método [13](#)

UNA

modelo abstracto [366](#)
Últimas acciones
 mostrando [206](#), [2](#)
`add()` método, paré
 `override_quantity` [242](#)
 producto [242](#)
 cantidad [242](#)
sitio de administración
 creación, para modelos [15](#)
 acciones personalizadas, añadi [1](#)-286
 Sitio de administración de Djar
 extensible, con vistas nalizadas
 modelos, sumando [17](#)
 modelos, personaliza
 superusuario, creando [15](#)
plantillas de administración
 enlace de referencia [290](#)
agregación
 enlace de referencia [64](#)
funciones de agregación
 valor medio (Avg) [64](#)
 valor de recuento (recuento) [64](#)
 valor máximo (Max) [64](#)
 valor mínimo (Min) [64](#)
 enlace de referencia [72](#)
Acciones AJAX
 agregando, con jQuery [172](#)
Solicitud AJAX GET, escenari
 respuesta, con datos [184](#)
 respuesta, sin contenido [184](#)

Paginación AJAX
 añadiendo, a la vista de li 185
Solicitudes AJAX
 realizar, con jQuery [176](#) -179
 con falsificación de i e: is
 (CSRF) [174](#), [1](#)
Vista AJAX
 edificio, sigan a los usuarios [19](#)
Configuración ALLOWI s
 enlace de referencia
aplicación [9](#)
clases de conf i de aplicaciones
 aproximada [13](#) -215
 enlace de re [213](#)
aplicaciones asincronicas
 ASGI, usando [471](#)
consumidor asincr
 modificando [494](#)
Interfaz de puerta : de servidor asincrónico
 (ASGI) [2](#), [5](#)
tareas asincrónica
 añadiendo, a la aplicación [26](#)
 lanzamiento, con Apio [263](#)
backend de autenticación, métodos
 autenticar () [129](#)
 get_user () [129](#)

Si

clases base, Django
 Forma [40](#)
 Formulario modelo [40](#)
esquema de datos de blog
 aplicación, activ 13
 diseñar [10](#), [11](#)
 migraciones, ap do [13](#)
 migraciones, creando [13](#),

esquema de datos de blog, campos
 autor [12](#)
 cuerpo [12](#)
 creado [12](#)
 publicar [12](#)
 babosa [11](#)
 estado [12](#)

caché por sitio [436](#)
caché por vista [436](#)
caché de plantilla [436](#)
configuración de caché [434](#)
URL canónica
 para modelos [29](#)
CartAddProductForm clase, campos

título [11](#)
 actualizado [12](#)
 publicaciones de blog
 utilizado, para crear feeds [80](#) ,
 bookmarklet
 edificio, con jQuery [160](#) -16'
 Braintree
 integrar, con los campos Alojado [274](#) ·
 URL [270](#)
 Cuenta Braintree
 creando [283](#)
 enlace de referencia [283](#)
 Módulo Braintree
 instalación [271](#) ,
 Cuenta de sandbox Braintree
 creando [270](#) , 2'
 enlace de referencia [1](#)
 vistas de autenticación integradas
 enlace de referencia [102](#)
 señales incorporadas
 enlace de referencia [211](#)
 plantilla, etiquetas y filtros integrados
 enlace de referencia [31](#)

C

backends de caché [432](#) , ·
 marco de caché
 backends de caché [432](#) ,
 niveles de caché [436](#)
 configuración de caché [434](#)
 API de caché de bajo nivel, usando [18](#)
 Backend de Memcached, agregar proyecto [434](#) ,
 Backend de Memcached, instalando [433](#)
 fragmentos de plantilla, almacenamiento en caché [440](#)
 usando [432](#)
 vistas, almacenamiento en caché [441](#)
 niveles de caché
 alrededor de [436](#)
 API de caché de bajo nivel [436](#)
 anular [245](#)
 cantidad [245](#)
 detalle del carrito página [347](#)
 plantillas de catá
 la creación de ;
 Apio
 agregando, al proyecto [21](#)
 tareas asincrónicas, lanzar
 instalación [263](#)
 seguimiento [267](#) ,
 enlace de referencia
 autoridad de certificación (CA) [513](#)
 capa de canal
 consumidor, actualizar ; transmitir
 Mensajes [486](#) -49
 contexto, añadiendo a ; ajes ;
 habilitando [484](#)
 configuración, con Redis [484](#) ,
 capa de canal, abstracciones
 canal [484](#)
 grupo [484](#)
 Canales
 la instalación de ; 5
 usando, en tiempo de ejecución Django [471](#)
 utilizando, en el ciclo de solicitud / respuesta ; , 473
 aplicación de chat
 creando [467](#) , 41
 vista de sala de chat
 implementando [468](#)
 servidor de chat
 implementando [475](#)
 integrando, con vistas existentes [495](#)
 vistas basadas en clases
 el acceso, la restricción de ; 1
 características [36](#)
 enlace de referencia [37](#)
 utilizando ;
 valores separados por comas (CSV) [284](#)
 sistema de comentarios
 comentarios, agregando a los detalles de la publicación

[534]

plantilla [54](#) -5'
 creando [50](#)
 formas, creando a partir de modelos [52](#)
 Modelo, la construcción [50](#) -52
 ModelForms, manejo de vistas [53](#) , 54
 consumidor
 escribiendo [4](#)
 contenido
 publicación, desde otros sitios web [1](#)
 sistema de gestión de contenido (CMS)
 sistema de autenticación, agregando [377](#)
 plantillas de autenticación, la creación de ;
 vistas basadas en clases, creando [381](#)
 creando [377](#)
 grupos, trabajando con [383](#) ,
 mixins, utilizando para las vistas
 permisos, trabajando con [384](#) , 385
 contenido del directorio
 enlace de referencia [162](#)
 marco de tipos de contenido
 enlace de referencia [201](#)
 usando [200](#) ,
 procesador de ;
 OrderField clase, añadiendo al módulo [372](#)
 registrarse, en el sitio de administración [36](#)
 módulos del curso
 contenido, añadiendo ;
 el contenido, la gestión de ;
 contenido, reordenamiento
 formsets, utilizar
 la gestión de [391](#)
 reordenamiento
 cursos
 mostrando [414](#)
 inscripción [422](#)
 tarjetas de crédito ;
 enlace de referencia [282](#)
 cron
 : administrador de [526](#)
 enlace de referencia [526](#)
 Falsificación de solicitud entre sitios (CSRF)
 alrededor de [46](#)
 en solicitudes AJAX [174](#)
 enlace de referencia [46](#)
 Protección CSRF
 enlace de referencia [175](#)

utilizado, para el envío de mensajes de correo electrónico, ejecutando en Nginx, configurando [518](#) -519

Aplicación Django, usando para los servidores de desarrollo, ejecutando middleware, creando [520](#) , 521

Marco de autenticación de Django, configuraciones de middleware, la gestión de [498](#)

[536]

Nginx, configurar [506](#) -508

NGINX, instalando [505](#)

PostgreSQL, usando [500](#) , 501

entorno de producción, creando conexiones seguras, usando para WebSockets [517](#)

serviendo, a través de WSGI [50](#)

activos estáticos y medios de almacenamiento, sirviendo [506](#) -511

uWSGI, configurar [502](#) -503

uWSGI, instalando [502](#)

Proyecto Django, configuración

alrededor de [8](#)

ALLOWED_HOSTS [8](#)

BASES DE DATOS [8](#)

DEPURACIÓN [8](#)

INSTALLED_APPS [8](#)

LANGUAGE_CODE [9](#)

MIDDLEWARE [8](#)

enlace de referencia [8](#)

ROOT_URLCONF [8](#)

USE_TZ [9](#)

Marco Django REST

enlace de referencia [444](#)

Marco Django REST, características

enlace de referencia [461](#)

Marco Django REST, configuración

enlace de referencia [445](#)

Filtros de plantilla integrados de Django

enlace de referencia [73](#)

El sistema de caché de Django

enlace de referencia [432](#)

Sesiones de Django

caducidad [240](#)

ajustes [239](#)

utilizado, para el almacenamiento de cookies, usando [238](#)

Marco de alimentación de distribución de Django

enlace de referencia [82](#)

Django, etiquetado

enlace de referencia [58](#)

administradores django-taggit

enlace de referencia [66](#)

Lenguaje de plantilla Django

enlace de referencia [30](#)

Modelo de objetos de documento (DOM) [174](#)

acciones duplicadas

evitando, en el flujo de actividad [21](#)

datos dinámicos

almacenamiento, en [438](#) , 439

mi

miniaturas fáciles

enlace de referencia [172](#)

utilizado, para crear miniaturas de imágenes [171](#)

proyecto de e-learning

configurar [358](#)

correos electrónicos

el envío, con Django [43](#) -45

F

Cuenta de desarrollador de Facebook

enlace de referencia [134](#)

características, patrones de URL

prefijo de idioma [332](#)

traducido [332](#)

alimenta

creando, para publicaciones de Facebook [80](#) , 81

campos, modelo de acción

creando [199](#)

objetivo [202](#)

target_ct [202](#)

target_id [202](#)

usuario [199](#)

verbo [199](#)

campos, modelo ContentType

app_label [200](#)

modelo [200](#)

nombre [200](#)

campos, relaciones genéricas en modelo

Campo ForeignKey [201](#)

Campo de GenericForeignKey [201](#)

PositiveIntegerField [201](#)

accesorios [362](#)

uso de accesorios, para pruebas

enlace de referencia [364](#)

Tablero de flores

enlace de referencia [268](#)

sistema de seguimiento

Ver AJAX, la construcción sigan a los usuarios -198

edificio [187](#)

vistas detalladas, creando para el usuario

perfiles [191](#) -192

vistas de lista, la creación de perfiles de usuarios [191](#) -195

[537]

relaciones de muchos a muchos, con modelo intermediario [188](#)

Clave externa

 enlace de referencia [12](#)

localización de

 alrededor de ;

 enlace de ref [145](#)

campos de formu

 limpieza [154](#), 1

 enlace de refer

formas

 creando, con Django [40](#)

 manipulación, en vistas [45](#) -49

 la representación, en la:

conjuntos de formularios

 enlace de referencia [392](#)

búsqueda de texto completo

 agregando, al blog [82](#)

 enlace de referencia [90](#)

motor de búsqueda de texto completo [91](#)

Traducciones borrosas [331](#)

GRAMO

aplicación de flujo de actividad gen

 flujo de activid :ando [206](#)

 edificio [198](#), 15

 marco de tipos nido, utilizand

 acciones duplicadas o en activi

 corriente [204](#),

 las relaciones genér diendo al mo

 QuerySets, optimizando [207](#)

 plantillas, creación de acciones [208](#) -21

 acciones del usuari suman a l

 corriente [205](#),

mixins y vistas genéri

 enlace de referencia [447](#)

relaciones genéricas

 añadiendo, a modelo [20](#)

 get_form () método [398](#)

 get () método [398](#)

 get_model () método [397](#)

Captcha de Gmail, inhabilitando

 enlace de referencia [44](#)

Implementación de OAuth2 de Google

 enlace de referencia [142](#)

H

Alpaca

 enlace de referencia [91](#)

Cerveza casera

 enlace de descarga [433](#)

yo

sitio web de marcadores de imagen

 creando [150](#)

 modelo de imagen, edificio [15](#)

 modelo de imagen, registránd lministración

 sitio [153](#)

 relaciones de muchos a muchos, creando [152](#)

modelo de image

 edificio [150](#), 15

 registrarse, en e administración [153](#)

ranking de imágenes

 almacenamiento, en Re 222

imagenes

 vista de detalle, creando [16](#)

miniaturas de imágenes

 creando, con miniaturas fáciles [170](#), 1

internacionalización

 alrededor de [314](#)

 sumando [314](#)

1 comandos de gestión [316](#)

 proyecto, prep: [1](#)

 ajustes [315](#), 31

204on Django [315](#)

vistas de artículos

 almacenamiento, con Red 9

J

jQuery

 enlace de descarga [174](#)

 cargando [173](#)

 enlace de referencia [160](#)

 utilizado, para agregar acciones AJAX

 utilizado, para la construcción de boc -167

 utilizado, para la realización de AJAX : 79

Selectores de jQuery

 enlace de referencia [164](#)

Cookie JS

 enlace de referencia [175](#)

L

código de idioma [315](#)

creación, para traducciones de modelos

mixins

 enlace de referencia [381](#)

nombre del idioma [315](#)
 aplicaciones menos seguras, habilitación de acceso a [407](#) -41
 enlace de referencia [44](#)
 Protocolo ligero de acceso a directorios (LDAP) [128](#)
 vista de la lista
 Paginación AJAX, añadiendo [16](#)
 edificio [26](#)
 URL canónica modelos [29](#)
 creando [26](#) , 2
 crear, para pe de usua
 Patrones de URL, agregand
 localización
 alrededor de [314](#)
 sumando [314](#)
 ajustes [315](#) , 31
 API de caché ()
 usando [436](#)

METRO

gerentes
 trabajando con [21](#)
 relaciones de muchos a muchos
 creando [152](#)
 crear, con el model diario [188](#) -1
 enlace de referenci
 relaciones de muchos
 enlace de referencia [51](#)
 formato de rebajas
 enlace de referencia [74](#)
 Backend de Memcached
 añadiendo, al proyecto [434](#)
 enlace de descarga [433](#)
 instalación [433](#)
 seguimiento [435](#)
 archivo de mensaje [315](#)
 marco de mensajes
 enlace de re [128](#)
 utilizando [1](#)
 middleware
 alrededor de [96](#)
 enlace de referencia [521](#)
 comando de gestión de migrar [6](#)
 migraciones

usando, de django-braces [407](#) -41
 API modelo
 enlace de referencia [21](#)
 campos modelo
 traduciendo [337](#) ,
 conjuntos de formu
 enlace de referencia [392](#)
 herencia modelo
 modelo abstracto [366](#)
 herencia de modelo de varias tablas [367](#)
 modelo de proxy [367](#)
 usando [366](#)
 administradore odelos
 creando [25](#) , 2
 modelos
 traduciendo, con django-parler [336](#)
 varios campos
 buscando [84](#)
 múltiples subdominios
 servir, con NGINX [523](#) , 524
 herencia de modelo de múltip is [366](#)

norte

Binarios NGINX, para Windows
 enlace de referencia [505](#)
 Documentación de NGINX
 enlace de referencia [507](#)

O

mapeador relacional de objetos (ORM) [21](#)
 proyecto de tienda online
 catálogo de plantillas, la creación d 37
 creando [226](#)
 funcionalidades [226](#)
 catálogo de productos modelos, creando
 modelos del catálogo de productos, regis en
 sitio de administración [229](#)
 Vistas de catálogo del producto, la cons de [230](#) -232
 pedidos
 exportar, a archivos CSV [284](#)

[539]

PAGS

paginación
 sumando [34](#)
 trabajando [3](#)
 analizadores
 enlace de referencia [447](#)
 convertidores de ruta
 enlace de referencia [28](#)
 autorización de pago
 enlace de referencia [283](#)
 Industria de tarjetas de pago (PCI) [270](#)
 pasarela de pago
 Braintree, integrá n Hosted
 Los campos
 Módulo Braintree instalación [271](#) ,
 Cuenta de sandbox a ree,
 creando [270](#)
 la integración de [2](#)
 pagos

 enlace de referencia [82](#)
 Cartero
 enlace de referencia [449](#)
 post () método [399](#)
 publicaciones pc litud
 la recuperación [4](#) -66
 publicaciones, p reo electrónic
 correos electrónicos, enviando cor go [43](#) -45
 formas, creando con Django [4](#)
 formas, manejo en vistas [41](#) , 4
 formas, representación en las [5](#) -49
 compartir [40](#)
 prefetch_related ()
 usando [208](#)
 modelos de catál roductos
 la creación de
 registrarse, en e administración [229](#)
 vistas del catálo ductos
 la construcción -232
 detalle del produ na [347](#)

- pruebas de [28](#)
- Archivos PDF
 - renderizado [294](#)
 - el envío, por correo [297](#)-300
- Facturas en PDF
 - generando [292](#)
- PDF, salida con Django
 - enlace de referencia [292](#)
- Plantilla PDF
 - creando [292](#), 292
- permisos
 - enlace de referencia [455](#)
- caché por sitio
 - desactivando
 - usando [441](#), 441
- instalación de
 - enlace de referencia [3](#)
- paquete pip
 - usado, para instalar Django [3](#), 4
- Poedit
 - enlace de descarga [323](#)
- plantilla de detalle de publicación
 - comentarios, la adición de [57](#)
- PostgreSQL
 - enlace de descarga [3](#)
 - instalación [83](#), 83
- Búsqueda de texto
 - conjunto de PostgreSQL

- la recomendación del producto [34](#)
- proyecto [9](#)
- modelo de proxy [366](#)
- Pitón
 - Redis, usando [217](#), 217
- Python 3.8.0
 - enlace de descarga [4](#)
- Código Python
 - la traducción del código [4](#)
 - traducción perezosa [31](#)
 - traducción de formas plurales [31](#)
 - traducción estándar [319](#)
 - traduciendo [318](#)
 - traslación variable [319](#)
- Entorno Python
 - creando [3](#)
- Instalador de Python
 - enlace de descarga [2](#)
- Autenticación social de Python
 - enlace de referencia [131](#)

Q

- QuerySet
 - alrededor de [21](#)
 - evaluando [25](#)
 - excluir (), usando [24](#)

[540]

- método filter (), usando
- objetos, creando [21](#), 21
- objetos, borrando [24](#)
- objetos, recuperando [23](#)
- objetos, actualización [23](#)
- order_by (), usando [24](#)
- trabajando con [21](#)
- QuerySets
 - optimización [207](#)
 - prefetch_related (), usando [208](#)
 - select_related (), usando [207](#)

R

- RabbitMQ
 - instalación [264](#)
- RabbitMQ, instalando en macOS / Windows
 - enlace de referencia [264](#)
- clasificación de resultados [88](#)
- Django en tiempo real
 - con canales [471](#)
- motor de recomendación
 - edificio [347](#)
- Redis
 - alrededor de [223](#)
 - enlace de descarga [215](#)
 - clasificación de objetos, almacenamiento [220](#), 222
 - la instalación de [7](#)
 - vistas de elementos [218](#), 219
 - usado, para almacenar vistas [215](#)
 - usando, con Python [217](#), 217
- Comandos de Redis
 - enlace de referencia [217](#)
- Redis, instalando en Windows 10
 - enlace de referencia [216](#)
- documentación de redis-py

- renderizadores
 - enlace de referencia [447](#)
- Transferencia de estado representacional (REST) [444](#)
- RequestContext
 - enlace de referencia [254](#)
- ciclo de solicitud
 - Canales, usando [472](#), 472
- método de solicitud
 - OBTENER solicitud [260](#)
 - Solicitud POST [260](#)
- Solicita documentación de la biblioteca
 - enlace de referencia [46](#) 2
- ciclo de respuesta
 - Canales, usando [472](#), 472
- API REST
 - consumir [461](#)-461
- Marco REST
 - backends de autenticación [453](#)
- API RESTful
 - acciones, agregando a conjuntos [47](#), 458
 - autenticación, manejo [453](#), 454
 - edificio [444](#)
 - los contenidos del curso, serializando [461](#)
 - vistas de API personalizadas, creando
 - permisos personalizados, creando
 - Marco Django REST, instalación [444](#)
 - vistas de lista y de detalle, construyendo
 - serializadores [4](#), creando [4](#)
 - analizadores [4](#)
 - renderizadores
 - enrutadores, creando
 - serializadores, definiendo
 - conjuntos de vistas, creando
 - permisos de vistas, agregando
- Documentación de Rosetta
 - enlace de referencia [331](#)

[enlace de referencia](#) [217](#)
[almacenamiento en caché](#) [223](#)
[contando](#) [223](#)
[últimos artículos, almacenando](#) [223](#)
[pub / sub](#) [223](#)
[colas](#) [223](#)
[clasificaciones y tablas de clasificación](#) [223](#)
[seguimiento en tiempo real](#) [223](#)
expresiones regulares
[enlace de referencia](#) [29](#)

Interfaz de traducción de Rosetta
[usando](#) [328](#)
enrutadores
[enlace de referencia](#) [157](#)
enrutamiento

S

método save ()
anulando, de ModelForm [155](#) -15
Campo SearchVectorField

[541]

[enlace de referencia](#) [85](#)
vista de búsqueda
la construcción [85](#) -87
Capa de socket [ros \(SSL\)](#) [133](#) , 5:
select_related ()
usando [207](#)
serializadores
[enlace de referencia](#) [452](#)
datos de sesión, opciones
sesiones de base de datos en caché [239](#)
sesiones en caché [239](#)
sesiones basadas en cookies [239](#)
sesiones de base de datos [239](#)
sesiones basadas en archivos [239](#)
configuraciones, sesiones de Django
[enlace de referencia](#) [240](#)
carrito de compras
edificio [237](#)
procesador de contexto, creando [252](#)
Sesiones de Django, caducidad [240](#)
Sesiones de Django, configuración [239](#)
Sesiones de Django, [238](#)
artículos, añadiendo
productos, agregando
cantidades de productos a
almacenar, en sesiones de Django
plantilla, construcción [ra mostr](#) ntos y
total [247](#) , 2
vistas, creando [2](#)
señales
clases de configuración de la aplicación [2](#)
usado, para desnormalizar recuentos [210](#)
trabajar con [211](#) -21
Protocolo simple de transferencia de correo (SMTP) [43](#)
búsquedas de búsqueda simples [84](#)
marco del mapa de sitio
sumando [76](#)
[enlace de referencia](#) [77](#)
autenticación
la adición de
servidor de [, que se ejecuta a través de HTTPS](#)
[132](#) -:
Facebook, [3](#)
Google, utilizando [14](#)
[enlace de referencia](#)
Twitter, usando [140](#)
proyecto de sitio web
creando [94](#)
implementando [94](#) ,
Configuración de formulario en español
[enlace de referencia](#) [344](#)
Certificado SSL / TLS
conexiones, asociando con [511](#)
creando [511](#) , 5
Proyecto Django [urando](#) [514](#)
Tráfico HTTP, redirigiendo a HTTP!
utilizado, para la configuración de derivación [88](#)
registro de estudiantes
sumando 419
ver, creando 419-422
middleware de estado o
creando [522](#) , 5:

T

funcionalidad [quietado](#)
sumando [58](#)
filtros de plan [1](#)
fragmentos de plantilla
almacenamiento en caché [440](#)
plantillas
{% blocktrans%} etiqueta de plantilla
crear, para las acciones
crear, para las vistas de
formas, rindiendo [45](#) -
compras plantillas, traduciendo [32](#)
traduciendo [324](#)
{% trans%} etiqueta de plantilla [324](#)
etiquetas de plantilla [31](#)
variables de plantilla [31](#)
Seguridad de la capa de transporte (TLS)
similitud de trigramas
buscando con [90](#)
Cuenta de Twitter
[enlace de referencia](#) [140](#)
tipo de contenido
renderizado [429](#)

U

Espacios de nombres de URL
[enlace de referencia](#) [29](#)
Patrones de URL
para la internacionalización [332](#)

[542]

prefijo de idioma, agregando [3](#)
enlace de referenc
traduciendo [333](#) ,
Patrones de URL, co iones regulares
enlace de referencia [29](#)
Funciones de la utilidad de URL
enlace de referencia [30](#)
acciones del usuario
agregar, al flujo de actividad [205](#)
modelo de usuari
que se extiende
perfiles de usuari
alrededor de [115](#)
vista de detalle, la creac 195
vista de lista, la creaci ;
de registro de usuario [11](#)
usuarios
acceso, para cambiar de idioma [33](#).
Opciones de uWSGI
enlace de referencia [505](#)
Documentación de uWSGI
enlace de referencia [502](#)

V

puntos de vista
la adaptación, las traducciones [3](#)
almacenamiento en c [441](#)
formularios, manipul 42
las plantillas, la creac -33
conjuntos de vistas
enlace de referencia [457](#)
entorno virtual (venv)
enlace de referencia [3](#)

W

WeasyPrint
instalación [292](#)
enlace de referencia [292](#)
Interfaz de le enlace del servidor web (WSGI)
alrededo: 71
alrededo:
enlace de reterencia [502](#)
Cliente WebSocket
la implementación d 4
consultas de ponderac
Subsistema de Windows para Linux (WSL) [216](#)